

Introduction to Graph Theory

**A comprehensive guide combining mathematical rigor,
visualization, algorithmic implementations, and interview
practices**

Jiho Lee
University of California, San Diego

Math 154: Graph Theory

September 26, 2026

Contents

I	Foundations	1
1	Basic Classes of Graphs	1
1.1	Cycles	1
1.2	Paths	1
1.3	Degrees and Neighborhoods	1
1.4	Examples of Graphs and Their Vertex Degrees	2
1.4.1	Complete Graph K_n	2
1.4.2	Cycle Graph C_n	3
1.4.3	Path Graph P_n	3
1.4.4	Star Graph S_n	4
1.4.5	Wheel Graph W_n	4
1.4.6	Complete Bipartite Graph $K_{m,n}$	5
1.4.7	Regular Graph	6
1.4.8	Hypercube Graph Q_n	6
1.4.9	Petersen Graph	7
1.5	Mathematical Properties Related to Degrees	7
1.5.1	Handshaking Lemma	7
1.5.2	Degree Sequence	8
1.5.3	Average Degree	8
1.5.4	Maximum and Minimum Degree	8
1.5.5	Isolated and Pendant Vertices	9
1.6	Subgraphs	9
1.7	Walks and Paths in Graphs	10
1.7.1	More Examples of Walks, Paths, and Components	13
1.8	Chapter 1 Exercises	15
1.8.1	Easy Questions	15
1.8.2	Intermediate Questions	15
1.8.3	Hard Questions	16
1.8.4	Questions with Diagrams	16
2	Eulerian Graphs	18
2.1	Trails and Tours	18
2.2	Eulerian Digraphs and De Bruijn Sequences	26
2.2.1	Eulerian Digraphs	26
2.2.2	De Bruijn Sequences	26
2.3	De Bruijn Graphs	31
2.4	Chapter 2 Exercises	35
2.4.1	Easy Problems: Eulerian Graphs	35
2.4.2	Medium Problems: Eulerian Graphs	35
2.4.3	Hard Problems: Eulerian Graphs	35
2.4.4	Easy Problems: De Bruijn Sequences	35
2.4.5	Medium Problems: De Bruijn Sequences	36
2.4.6	Hard Problems: De Bruijn Sequences	36
2.4.7	Easy Problems: De Bruijn Graphs	36
2.4.8	Medium Problems: De Bruijn Graphs	36
2.4.9	Hard Problems: De Bruijn Graphs	36

3	Hamiltonian Graphs	37
3.1	Hamiltonian Walks, Paths, Cycles	37
3.2	Dirac's Theorem	40
3.3	Visual Illustration of Dirac's Theorem Proof	42
3.4	Alternative Form of Dirac's Theorem	43
3.5	Examples of Dirac's Theorem	44
3.6	Related Results and Extensions	48
	3.6.1 Ore's Theorem	48
	3.6.2 Chvátal's Theorem	49
3.7	Traveling Salesman Problem	51
	3.7.1 Approximation Algorithms	53
3.8	Chapter 3 Exercises	57
	3.8.1 Easy Problems	57
	3.8.2 Medium Problems	57
	3.8.3 Hard Problems	57
	3.8.4 Diagram Problems	58
4	Long Cycles	59
4.1	Long Cycles and Hamiltonian	59
4.2	Algorithm for Finding Long Cycles	62
4.3	Chapter 4 Exercises	62
	4.3.1 Easy Problems	62
	4.3.2 Medium Problems	64
	4.3.3 Hard Problems	64
	4.3.4 Diagram Problems	65
5	Trees and Forests	66
5.1	Trees, Forests, and Bridges	66
5.2	Characterization of Trees	68
5.3	Breadth-First Search and Depth-First Search Algorithms	70
	5.3.1 BFS Algorithm and BFS Tree	72
	5.3.2 DFS Algorithm and DFS Tree	75
	5.3.3 Comparison of BFS and DFS Traversals	77
5.4	Prim's Algorithm	79
5.5	Kruskal's Algorithm	84
	5.5.1 Key Similarities	87
5.6	Dijkstra's Algorithm	89
5.7	Floyd-Warshall Algorithm	93
5.8	Bellman-Ford Algorithm	96
	5.8.1 Algorithm Characteristics	99
5.9	Bipartiteness	101
5.10	Chapter 5 Exercises	105
	5.10.1 Visual Questions	105
6	Structure of Connected Graphs	108
6.1	Blocks, Cut Vertices, and Connectedness	108
6.2	Menger's Theorem	114
6.3	Structure of Connected Graphs Exercises	124
	6.3.1 Easy Questions	124
	6.3.2 Medium Questions	125
	6.3.3 Hard Questions	125
	6.3.4 Diagram Questions	126
	6.3.5 Additional Questions	126

7	Matchings and Factors	128
7.1	Independent Sets and Covers	131
7.2	Hall's Theorem and Various Matching Algorithms	139
7.3	Matchings and Factors Exercises	158
7.3.1	Easy Questions	158
7.3.2	Medium Questions	158
7.3.3	Hard Questions	158
7.3.4	Diagram Questions	159
8	Vertex and Edge-Coloring	160
8.1	Basic Definitions	160
8.2	König's Theorem on Edge-Coloring of Bipartite Graphs	160
8.2.1	Background and Motivation	160
8.2.2	Formal Statement	160
8.2.3	Implications and Significance	161
8.2.4	Equivalent Formulations	162
8.2.5	Historical Context	162
8.3	Vizing's Theorem	162
8.3.1	Formal Statement	162
8.3.2	Key Lemmas Supporting Vizing's Theorem	163
8.3.3	Implications and Applications	164
8.3.4	Historical Context	164
8.3.5	Examples of Vizing's and König's Theorem	164
8.4	Shannon's Edge Coloring Theorem	169
8.4.1	Explanation of Shannon's Bound	170
8.4.2	Proof of Shannon's Edge Coloring Theorem	170
8.5	Degenerate Graphs and Their Coloring Properties	177
8.6	Degeneracy Relationships for Specific Graph Classes	183
8.7	Extensions and Applications	184
8.8	Algorithms for d -Degenerate Graphs	185
8.9	Introduction to Scheduling Problems	187
8.9.1	Degenerate Graphs in Scheduling Problems	188
8.9.2	Auxiliary Algorithms for Scheduling Problems	194
8.10	Vertex and Edge-Coloring Exercises	217
8.10.1	Easy Questions	217
8.10.2	Medium Questions	217
8.10.3	Hard Questions	218
8.10.4	Diagram Questions	218
9	Planar Graphs	219
9.1	Definition and Basic Properties	219
9.2	Faces of Planar Graphs	219
9.3	Fundamental Theorems	220
9.4	Applications: Proving Non-Planarity	220
9.5	Introduction and Formal Definitions of Art Gallery Problems	222
9.6	Construction and Visualization	222
9.7	Triangulation	223
9.8	Three-Coloring	223
9.9	The Complete Guard Placement Algorithm	224
9.10	Orthogonal Polygons	224
9.11	Mobile Guards	224
9.12	Monotone Polygons	225
9.13	Fortress Problem	225
9.14	Prison Yard Problem	226

9.15	Open Problems	226
9.16	Planar Graphs and Art Gallery Problems	227
9.16.1	Easy Questions	227
9.16.2	Medium Questions	227
9.16.3	Hard Questions	228
9.16.4	Construction and Diagram Questions	228
9.16.5	Proof-Based Questions	229
9.16.6	Advanced Applications	229
10	Network Flows and the Max-Flow Min-Cut Theorem	230
10.1	Directed Graphs and Capacities	230
10.2	Flows and the Value of a Flow	230
10.3	Cuts and the Capacity of a Cut	231
10.4	Flow Value Equals Net Flow Across Any Cut	231
10.5	Residual Graphs and Augmenting Paths	231
10.6	The Max-Flow Min-Cut Theorem	232
10.7	The Ford–Fulkerson Algorithm	233
10.8	Applications	233
10.8.1	Menger’s Theorem	233
10.8.2	König’s Theorem Revisited	234
10.9	Worked Example	234
10.10	Exercises	235
10.10.1	Easy Questions	235
10.10.2	Medium Questions	235
10.10.3	Hard Questions	235
10.10.4	Diagram Questions	235
11	Advanced Topics in Graph Theory	236
11.1	Extremal Graph Theory	236
11.1.1	The Turán Number and the Turán Graph	236
11.1.2	Mantel’s Theorem: Three Proofs	236
11.1.3	Turán’s Theorem	237
11.1.4	Stability: The Erdős–Simonovits Theorem	238
11.1.5	The Erdős–Stone–Simonovits Theorem	238
11.1.6	The Bipartite Case: Kővári–Sós–Turán	239
11.1.7	The Zarankiewicz Problem	239
11.1.8	Graphs of High Girth: The Moore Bound	240
11.1.9	Supersaturation	240
11.1.10	Exercises	240
11.2	Ramsey Theory	241
11.2.1	The Classical Ramsey Numbers	241
11.2.2	Probabilistic Lower Bounds	243
11.2.3	Ramsey Numbers for Graphs	243
11.2.4	Hypergraph Ramsey Numbers	244
11.2.5	Schur’s Theorem and van der Waerden	244
11.2.6	Erdős–Szekeres: Happy Ending	244
11.2.7	Canonical Ramsey and Induced Ramsey	245
11.2.8	Exercises	245
11.3	The Probabilistic Method	246
11.3.1	First Moment: Linearity of Expectation	246
11.3.2	The Alteration (Deletion) Method	246
11.3.3	Second Moment and Paley–Zygmund	247
11.3.4	The Lovász Local Lemma	247
11.3.5	Concentration: Azuma and Talagrand	248

11.3.6	Dependent Random Choice	248
11.3.7	The Entropy Method	249
11.3.8	Exercises	249
11.4	Spectral Graph Theory	250
11.4.1	Matrices of a Graph	250
11.4.2	Walk Counting and Basic Identities	250
11.4.3	The Min-Max Principle and Rayleigh Quotient	251
11.4.4	Perron–Frobenius	251
11.4.5	The Matrix-Tree Theorem	251
11.4.6	Algebraic Connectivity and Cheeger’s Inequality	251
11.4.7	Expander Graphs	252
11.4.8	Random Walks and Mixing	253
11.4.9	Hoffman’s Bound and Chromatic Number	253
11.4.10	Strongly Regular Graphs	253
11.4.11	Exercises	254
11.5	Random Graphs	255
11.5.1	Random Graph Models	255
11.5.2	Subgraph Thresholds	255
11.5.3	Degree Distribution	256
11.5.4	Threshold for Connectivity	256
11.5.5	The Giant Component Phase Transition	256
11.5.6	Chromatic Number of $G(n, p)$	257
11.5.7	Hamiltonicity	257
11.5.8	Random Regular Graphs	257
11.5.9	Preferential Attachment and Scale-Free Models	257
11.5.10	Small Subgraph Central Limit Theorem	257
11.5.11	Exercises	258
11.6	Szemerédi’s Regularity Lemma	258
11.6.1	ε -Regular Pairs	259
11.6.2	The Regularity Lemma	259
11.6.3	The Counting Lemma	260
11.6.4	The Removal Lemma	260
11.6.5	From Triangle Removal to Roth	261
11.6.6	The Blow-up Lemma	261
11.6.7	Graphons and the Limit Theory	261
11.6.8	Property Testing	261
11.6.9	Exercises	262
11.7	Graph Minors and Treewidth	262
11.7.1	Minors and Topological Minors	263
11.7.2	Tree Decompositions and Treewidth	263
11.7.3	Duality: Brambles and Cops–Robbers	264
11.7.4	Courcelle’s Theorem and Fixed-Parameter Tractability	264
11.7.5	Excluded Minors and Sparsity	264
11.7.6	The Graph Minor Structure Theorem	265
11.7.7	The Graph Minor Theorem	265
11.7.8	The Disjoint Paths Problem	265
11.7.9	Exercises	266
11.8	Comprehensive Chapter Exercises	267

12 Algebraic Graph Theory: Auxiliary Topics	268
12.1 The Matchings Polynomial and the Heilmann–Lieb Theorem	269
12.1.1 Definition and Basic Recurrences	269
12.1.2 The Path-Tree Construction	270
12.1.3 The Heilmann–Lieb Theorem	270
12.1.4 Special Cases and Classical Polynomials	271
12.1.5 Exercises	271
12.2 Equitable Partitions and Quotient Interlacing	272
12.2.1 Definition and Basic Theory	272
12.2.2 Cauchy Interlacing for Quotients	272
12.2.3 Walk-Regularity and Distance Partitions	273
12.2.4 Exercises	273
12.3 Distance-Regular Graphs and Association Schemes	274
12.3.1 Distance-Regular Graphs	274
12.3.2 Association Schemes	274
12.3.3 The Hamming and Johnson Schemes	275
12.3.4 Exercises	275
12.4 Cayley Graphs and Vertex-Transitive Graphs	276
12.4.1 Definitions and First Examples	276
12.4.2 Spectrum from Characters	276
12.4.3 Non-Abelian Cayley Graphs	277
12.4.4 Vertex-Transitive Graphs and Cayley-ness	277
12.4.5 Exercises	277
12.5 The Tutte Polynomial and Graphic Matroids	278
12.5.1 Deletion, Contraction, and the Tutte Polynomial	278
12.5.2 Specializations: Chromatic, Flow, Reliability	279
12.5.3 Whitney’s Broken-Circuit Theorem	279
12.5.4 Graphic Matroids and Cycle/Bond Duality	279
12.5.5 Connections: Stanley’s Chromatic Symmetric Function	280
12.5.6 Exercises	280
12.6 Heaps of Pieces and Walk Generating Functions	281
12.6.1 Commutation Monoids and Heaps	281
12.6.2 The Cartier–Foata Master Theorem	281
12.6.3 Walks on Graphs as Heaps of Cycles	281
12.6.4 Flajolet’s Continued-Fraction Theorem	282
12.6.5 Exercises	282
12.7 The Ihara Zeta Function of a Graph	283
12.7.1 Non-Backtracking Walks and Primes	283
12.7.2 The Determinantal Formula	283
12.7.3 Riemann Hypothesis for Graphs	284
12.7.4 Functional Equation and Special Values	284
12.7.5 Exercises	285
12.8 Combinatorial Hopf Algebras of Graphs	285
12.8.1 Hopf Algebras: A Five-Line Reminder	285
12.8.2 The Graph Hopf Algebra of Schmitt	286
12.8.3 Characters and the Aguiar–Bergeron–Sottile Theorem	286
12.8.4 Application: Stanley’s Chromatic Symmetric Function	286
12.8.5 Other Combinatorial Hopf Algebras	287
12.8.6 Exercises	287
12.9 Hodge Theory of Matroids and Log-Concavity	288
12.9.1 Log-Concavity Conjectures	288
12.9.2 The Chow Ring of a Matroid	288
12.9.3 From Hodge–Riemann to Log-Concavity	289
12.9.4 Spectral Independence: A Modern Cousin	289

12.9.5 Exercises	289
12.10 The Permanent, Matching Counting, and #P-Hardness	290
12.10.1 The Permanent and Bipartite Matchings	290
12.10.2 Ryser's Inclusion–Exclusion Formula	291
12.10.3 Valiant's Theorem: #P-Completeness	291
12.10.4 The Permanent–Determinant Gap	291
12.10.5 Approximate Counting: JSV's FPRAS	291
12.10.6 Graph-Theoretic Synthesis	292
12.10.7 Exercises	292
12.11 Weisfeiler–Leman, Color Refinement, and Graph Isomorphism	293
12.11.1 Color Refinement (1-WL)	293
12.11.2 Higher-Dimensional Weisfeiler–Leman	293
12.11.3 Babai's Quasi-Polynomial Algorithm	293
12.11.4 WL and Graph Neural Networks	294
12.11.5 Exercises	294
12.12 Cluster Algebras from Quivers	294
12.12.1 Quivers, Skew-Symmetric Matrices, and Mutation	295
12.12.2 Cluster Algebras	295
12.12.3 Finite-Type Classification	295
12.12.4 Triangulations of Surfaces	295
12.12.5 Exercises	296
12.13 Chip-Firing, Sandpile Groups, and Riemann–Roch for Graphs	296
12.13.1 Divisors and Chip Firing	296
12.13.2 The Sandpile Group	297
12.13.3 The Baker–Norine Riemann–Roch Theorem	297
12.13.4 Graph-Theoretic Connections	297
12.13.5 Exercises	297
12.14 Random Walks via Representation Theory	298
12.14.1 The Upper Bound Lemma	298
12.14.2 Worked Examples	298
12.14.3 Cutoff Phenomenon	299
12.14.4 Exercises	299
12.15 Electrical Networks and Effective Resistance	300
12.15.1 Effective Resistance	300
12.15.2 Spanning Trees and Resistance	300
12.15.3 Hitting and Commute Times	300
12.15.4 Circle Packings	301
12.15.5 Exercises	301
12.16 Crystal Graphs and the Plactic Monoid	301
12.16.1 Crystals of Type A_{n-1}	302
12.16.2 The Plactic Monoid	302
12.16.3 Robinson–Schensted as a Graph Homomorphism	302
12.16.4 Application: Littlewood–Richardson Rule	302
12.16.5 Connection to Graph Theory	303
12.16.6 Exercises	303
12.17 Further Reading and Pointers	303

This page is intentionally left blank.

Part I

Foundations

1 Basic Classes of Graphs

1.1 Cycles

Definition 1. A cycle of length n ($n \geq 3$) is the graph $C_n = (V, E)$ where:

$$V = \{v_0, v_1, \dots, v_{n-1}\}$$

$$E = \{\{v_i, v_{i+1 \bmod n}\} : 0 \leq i < n\}$$

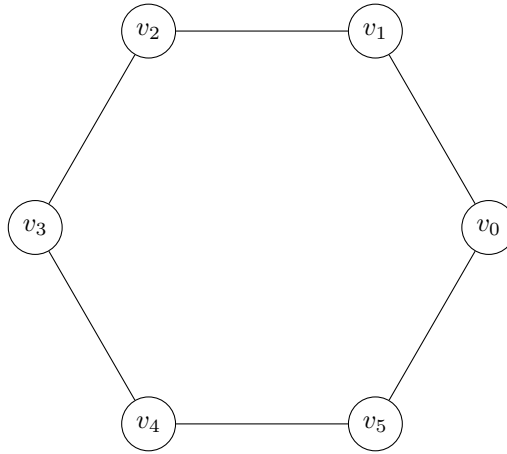


Figure 1: A cycle graph C_6 with 6 vertices.

1.2 Paths

Definition 2. A path of length k (that is, a path on $k + 1$ vertices) is the graph $P_{k+1} = (V, E)$ where:

$$V = \{v_0, v_1, \dots, v_k\}$$

$$E = \{\{v_i, v_{i+1}\} : 0 \leq i < k\}$$

In general, P_n denotes the path on n vertices; it has $n - 1$ edges.

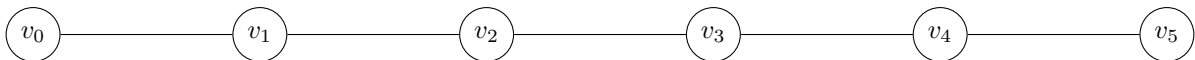


Figure 2: A path graph P_6 with 6 vertices (a path of length 5).

1.3 Degrees and Neighborhoods

Definition 3. If G is a graph and $v \in V(G)$, then:

- The neighborhood of v is $N_G(v) = \{w \in V(G) : \{v, w\} \in E(G)\}$
- The degree of v is $d(v) = d_G(v) = |N_G(v)|$

$$d_G(v) = |N_G(v)| = 4$$

$$N_G(v) = \{w_1, w_2, w_3, w_4\}$$

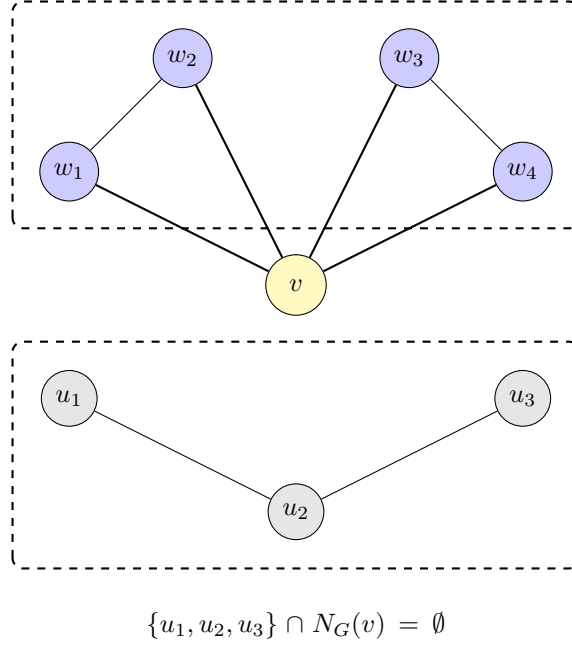


Figure 3: Example of neighborhood and degree of a vertex in a graph.

Definition 4 (Incidence). In graph theory, incidence refers to the relationship between vertices and edges:

- A vertex v and an edge e are *incident* with each other if v is an endpoint of e .
- If $e = \{u, v\}$, then:
 - Edge e is incident with vertices u and v
 - Vertex u is incident with edge e
 - Vertex v is incident with edge e
- The incidence relation forms the fundamental connection between vertices and edges in a graph.

1.4 Examples of Graphs and Their Vertex Degrees

1.4.1 Complete Graph K_n

In a complete graph with n vertices, every vertex is connected to all other vertices.

- For any vertex v in K_n : $\deg(v) = n - 1$
- Example: In K_5 , every vertex has $\deg(v) = 4$
- Total edges: $|E| = \frac{n(n-1)}{2} = \frac{5 \cdot 4}{2} = 10$

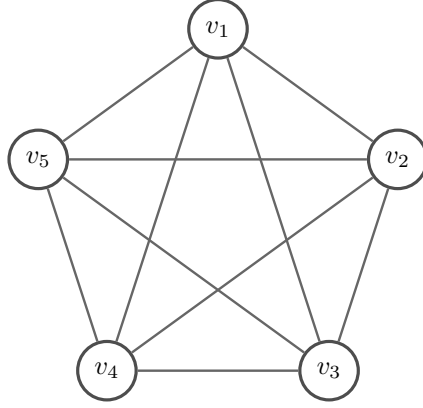
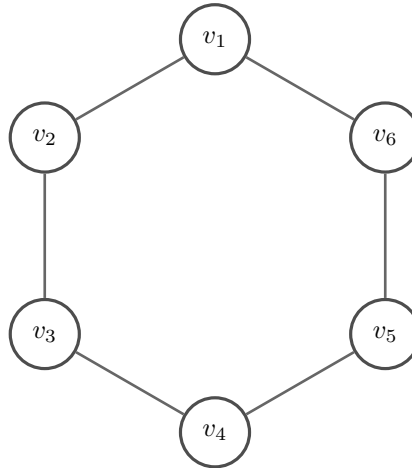


Figure 4: In this complete graph K_5 with 5 vertices, each vertex v_i has degree 4. Every vertex is connected to all other vertices in the graph. Total number of edges = $\frac{n(n-1)}{2} = \frac{5 \cdot 4}{2} = 10$

1.4.2 Cycle Graph C_n

A cycle graph with n vertices forms a closed loop.

- For any vertex v in C_n : $\deg(v) = 2$
- Example: In a hexagon C_6 , every vertex has $\deg(v) = 2$
- Total edges: $|E| = n = 6$



$$\forall i \in \{1, 2, 3, 4, 5, 6\} : \deg(v_i) = 2 \quad \text{and} \quad |E| = |V| = 6$$

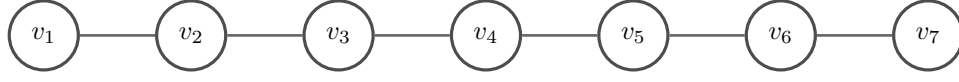
Figure 5: In this cycle graph C_6 with 6 vertices, each vertex v_i has degree 2. Every vertex is connected to exactly two other vertices in the graph. Total number of edges = $n = 6$

1.4.3 Path Graph P_n

A path graph with n vertices forms a single path.

- For endpoints: $\deg(v) = 1$
- For internal vertices: $\deg(v) = 2$

- Example: In P_7 , the degree sequence is $(1, 2, 2, 2, 2, 2, 1)$
- Total edges: $|E| = n - 1 = 6$



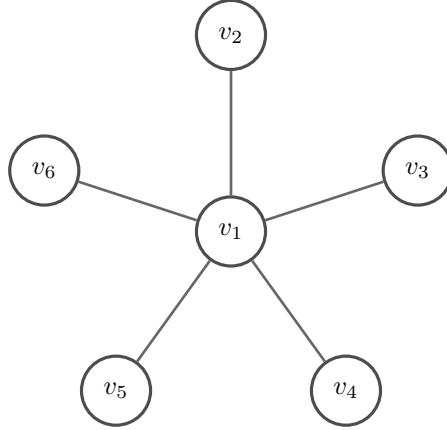
$$\deg(v_1) = \deg(v_7) = 1 \quad \text{and} \quad \deg(v_i) = 2 \quad \forall i \in \{2, 3, 4, 5, 6\}$$

Figure 6: In this path graph P_7 with 7 vertices, the endpoint vertices v_1 and v_7 have degree 1, while all internal vertices have degree 2. Total number of edges = $n - 1 = 6$

1.4.4 Star Graph S_n

A star graph has one central vertex connected to $n - 1$ leaf vertices.

- For central vertex c : $\deg(c) = n - 1$
- For any leaf vertex v : $\deg(v) = 1$
- Example: In S_6 , the central vertex has $\deg(c) = 5$, and the 5 leaf vertices each have $\deg(v) = 1$
- Total edges: $|E| = n - 1 = 5$



$$\deg(v_1) = 5 \quad \text{and} \quad \deg(v_i) = 1 \quad \forall i \in \{2, 3, 4, 5, 6\}$$

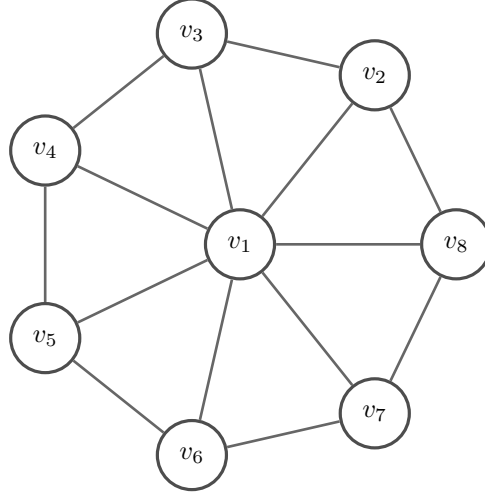
Figure 7: In this star graph S_6 with 6 vertices, the central vertex v_1 has degree 5, while all leaf vertices v_2 through v_6 have degree 1. Total number of edges = $n - 1 = 5$

1.4.5 Wheel Graph W_n

A wheel graph W_n ($n \geq 3$) is formed by connecting a single new vertex to all vertices of a cycle C_n , so W_n has $n + 1$ vertices.

- For central vertex c : $\deg(c) = n$
- For vertices on the cycle: $\deg(v) = 3$
- Example: In W_7 , the central vertex has $\deg(c) = 7$, and the 7 outer vertices each have $\deg(v) = 3$

- Total edges: $|E| = 2n = 14$



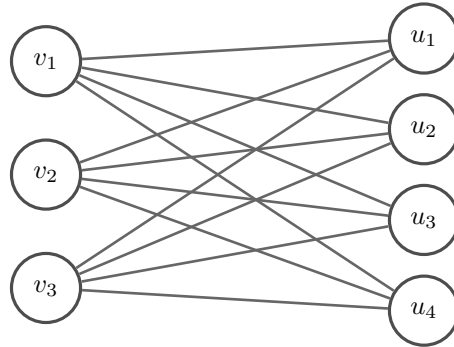
$$\deg(v_1) = 7 \quad \text{and} \quad \deg(v_i) = 3 \quad \forall i \in \{2, 3, 4, 5, 6, 7, 8\}$$

Figure 8: In this wheel graph W_7 with 8 vertices, the central vertex v_1 has degree 7, while all cycle vertices v_2 through v_8 have degree 3. Total number of edges = $2n = 14$

1.4.6 Complete Bipartite Graph $K_{m,n}$

A complete bipartite graph $K_{m,n}$ has two disjoint sets of vertices (with m and n vertices) where every vertex in the first set is connected to every vertex in the second set, and no edge joins two vertices in the same set.

- For vertices in the first set: $\deg(v) = n$
- For vertices in the second set: $\deg(v) = m$
- Example: In $K_{3,4}$, the 3 vertices in the first set each have $\deg(v) = 4$, and the 4 vertices in the second set each have $\deg(v) = 3$
- Total edges: $|E| = m \cdot n = 12$



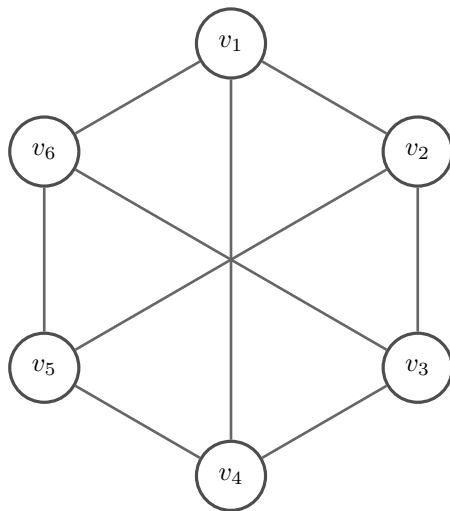
$$\deg(v_i) = 4 \quad \forall i \in \{1, 2, 3\} \quad \text{and} \quad \deg(u_j) = 3 \quad \forall j \in \{1, 2, 3, 4\}$$

Figure 9: In this complete bipartite graph $K_{3,4}$ with 7 vertices, each vertex v_i in the first set has degree 4, while each vertex u_j in the second set has degree 3. Total number of edges = $m \cdot n = 3 \cdot 4 = 12$

1.4.7 Regular Graph

In a k -regular graph, every vertex has the same degree k .

- For any vertex v : $\deg(v) = k$
- Example: In a 3-regular graph (cubic graph), every vertex has $\deg(v) = 3$
- Total edges in a k -regular graph with n vertices: $|E| = \frac{kn}{2}$



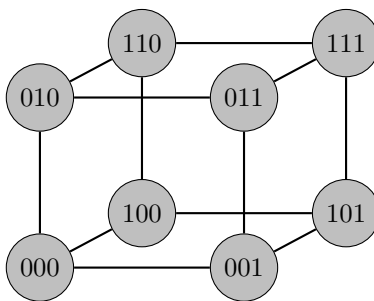
$$\deg(v_i) = 3 \quad \forall i \in \{1, 2, 3, 4, 5, 6\}$$

Figure 10: In this 3-regular graph with 6 vertices, each vertex v_i has exactly 3 incident edges. Total number of edges = $\frac{kn}{2} = \frac{3 \cdot 6}{2} = 9$

1.4.8 Hypercube Graph Q_n

In an n -dimensional hypercube, every vertex has degree n .

- For any vertex v in Q_n : $\deg(v) = n$
- Example: In Q_3 (3-dimensional cube), every vertex has $\deg(v) = 3$
- Total vertices: $|V| = 2^n$
- Total edges: $|E| = n \cdot 2^{n-1}$



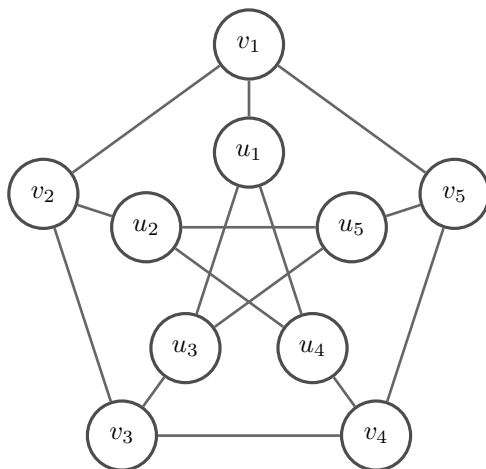
$$\deg(v) = 3 \quad \forall v \in V(Q_3)$$

Figure 11: Hypercube graph Q_3 with vertices labeled by binary strings. Each vertex has degree 3, with connections to neighbors that differ in exactly one bit position.

1.4.9 Petersen Graph

The Petersen graph is a specific 3-regular graph with 10 vertices.

- For any vertex v : $\deg(v) = 3$
- Total edges: $|E| = 15$



$$\deg(v_i) = \deg(u_i) = 3 \quad \forall i \in \{1, 2, 3, 4, 5\}$$

Figure 12: Petersen graph with 10 vertices. Each vertex has degree 3, with a total of 15 edges. This is a famous non-Hamiltonian 3-regular graph.

1.5 Mathematical Properties Related to Degrees

1.5.1 Handshaking Lemma

The sum of all degrees equals twice the number of edges:

$$\sum_{v \in V} \deg(v) = 2|E| \quad (1)$$

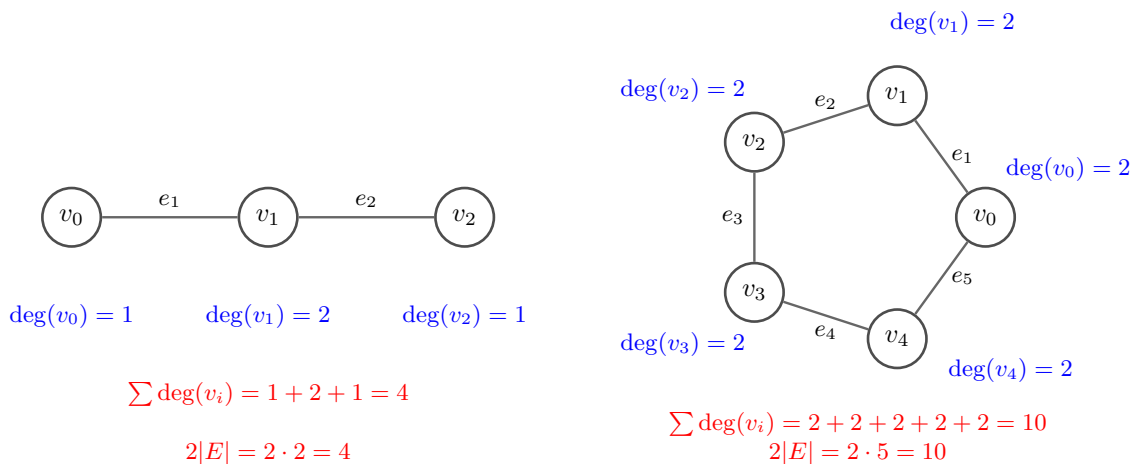


Figure 13: Illustration of the Handshaking Lemma. Left: A path graph with 3 vertices and 2 edges. Right: A cycle graph with 5 vertices and 5 edges. In both cases, the sum of vertex degrees equals twice the number of edges.

Example 1. Consider a graph with vertex set $V = \{1, 2, 3, 4\}$ and edge set $E = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{3, 4\}\}$.

$$\begin{aligned}\sum_{v \in V} d(v) &= d(1) + d(2) + d(3) + d(4) \\ &= 3 + 2 + 3 + 2 \\ &= 10\end{aligned}$$

And $2|E| = 2 \cdot 5 = 10$.

1.5.2 Degree Sequence

The non-increasing sequence of vertex degrees.

- Example: A graph with degree sequence $(4, 3, 3, 2, 2, 1, 1)$ has 7 vertices with varying connectivity
- Valid degree sequences must sum to an even number

1.5.3 Average Degree

The average degree of vertices in a graph $G = (V, E)$ is:

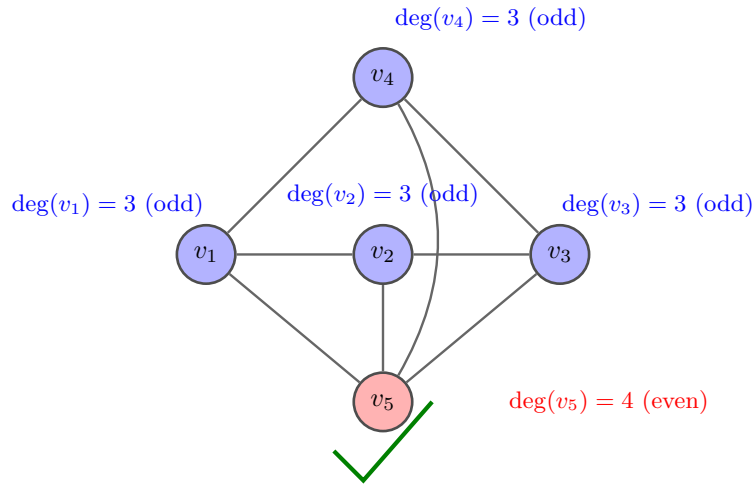
$$\text{avg}(\text{deg}) = \frac{\sum_{v \in V} \text{deg}(v)}{|V|} = \frac{2|E|}{|V|} \quad (2)$$

1.5.4 Maximum and Minimum Degree

$$\Delta(G) = \max\{\text{deg}(v) \mid v \in V\} \quad (3)$$

$$\delta(G) = \min\{\text{deg}(v) \mid v \in V\} \quad (4)$$

Corollary 1. In any graph, there are an even number of vertices of odd degree.

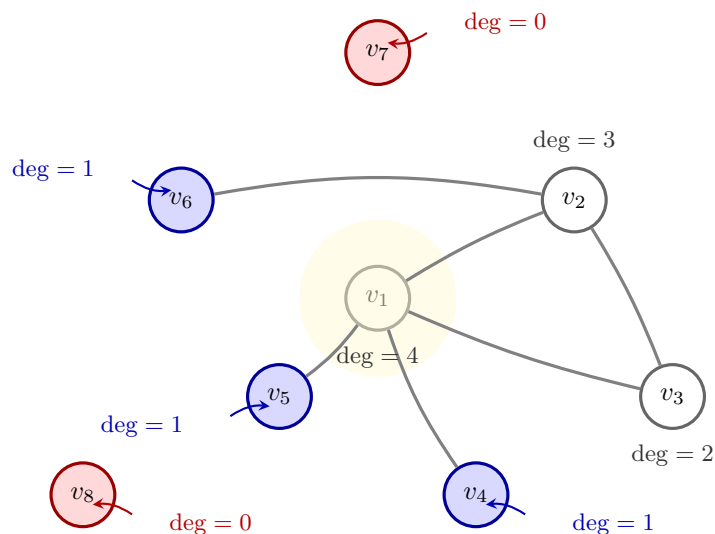


There are 4 odd-degree vertices (even number)

Figure 14: A graph illustrating the handshaking lemma's corollary that any graph must contain an even number of odd-degree vertices. This example has 4 vertices with odd degree (shown in blue) and 1 vertex with even degree (shown in red).

1.5.5 Isolated and Pendant Vertices

- Isolated vertex: $\deg(v) = 0$
- Pendant vertex (leaf): $\deg(v) = 1$
- Example: A tree with $n \geq 2$ vertices has at least 2 leaves, and hence at most $n - 2$ vertices of degree ≥ 2 (exactly $n - 2$ only when the tree is a path)



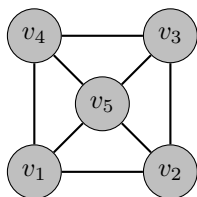
- **Isolated vertex:** $\deg(v) = 0$
- **Pendant vertex:** $\deg(v) = 1$
- **Regular vertex:** $\deg(v) \geq 2$

Figure 15: Illustration of special vertex types based on degree. The center vertex v_1 (highlighted) connects to both regular vertices and pendant vertices, while isolated vertices remain disconnected from the graph.

1.6 Subgraphs

Definition 5. A graph F is a subgraph of graph G if $V(F) \subseteq V(G)$ and $E(F) \subseteq E(G)$, written $F \subseteq G$.

Original Graph G



Subgraph $F \subseteq G$

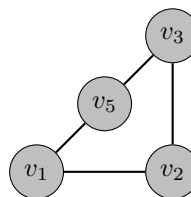


Figure 16: Illustration of a subgraph relationship. The left diagram shows the original graph G with 5 vertices and 8 edges. The right diagram shows a subgraph $F \subseteq G$ where vertex v_4 has been removed, along with its incident edges. Additionally, the edge v_2-v_5 between two remaining vertices has been removed, demonstrating that a subgraph maintains subset relationships for both vertex and edge sets.

Definition 6. A subgraph F of G is a spanning subgraph of G if $V(F) = V(G)$.

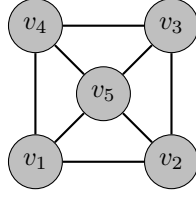
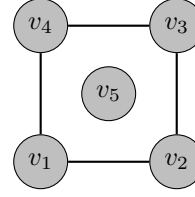
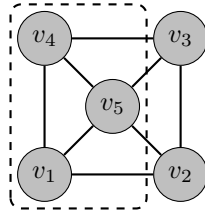
Original Graph G Spanning Subgraph F of G 

Figure 17: Illustration of a spanning subgraph. The original graph G (left) has 5 vertices and 8 edges, including connections from central vertex v_5 to all other vertices. The spanning subgraph F (right) contains all vertices of G but only a subset of edges, specifically removing all connections to the central vertex v_5 . A spanning subgraph maintains the same vertex set as the original graph ($V(F) = V(G)$) but may have fewer edges ($E(F) \subseteq E(G)$).

Definition 7. If $X \subseteq V(G)$, the subgraph induced by X is:

$$G[X] = (X, \{\{u, v\} \in E(G) : u, v \in X\})$$

Original Graph G 

$$X = \{v_1, v_4, v_5\}$$

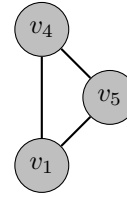
Induced Subgraph $G[X]$ 

Figure 18: Illustration of an induced subgraph. The original graph G (left) has a subset of vertices $X = \{v_1, v_4, v_5\}$ highlighted by a dashed rectangle. The induced subgraph $G[X]$ (right) contains exactly these vertices along with all edges from the original graph that connect vertices within this set. Unlike a general subgraph, an induced subgraph must include every edge from the original graph whose endpoints are both in the selected vertex subset.

1.7 Walks and Paths in Graphs

Definition 8. A walk in a graph is a sequence of vertices and edges:

$$v_0, \{v_0, v_1\}, v_1, \{v_1, v_2\}, v_2, \dots, \{v_{k-1}, v_k\}, v_k$$

often abbreviated as $v_0, v_1, v_2, \dots, v_k$. The length of a walk is the number of edges in that walk.

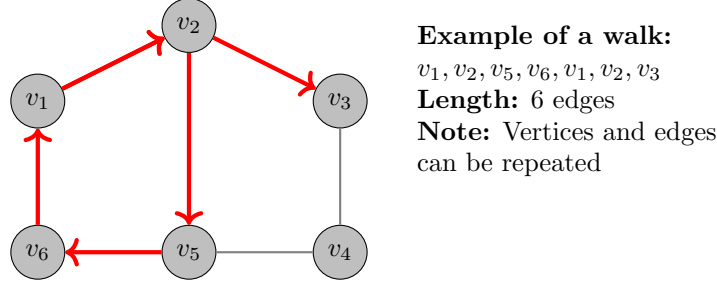


Figure 19: Illustration of a walk in graph theory. A walk is a sequence of vertices and edges where each consecutive pair of vertices in the sequence is connected by an edge. This example shows a walk of length 6 (red arrows) from v_1 to v_3 that passes through the sequence $v_1, v_2, v_5, v_6, v_1, v_2, v_3$. Unlike paths or trails, walks can repeat both vertices and edges. This walk repeats vertex v_1 and v_2 , and also repeats the edge between them.

Definition 9. A path of length k is a walk:

$$v_0, v_1, v_2, \dots, v_k$$

where all vertices are distinct.

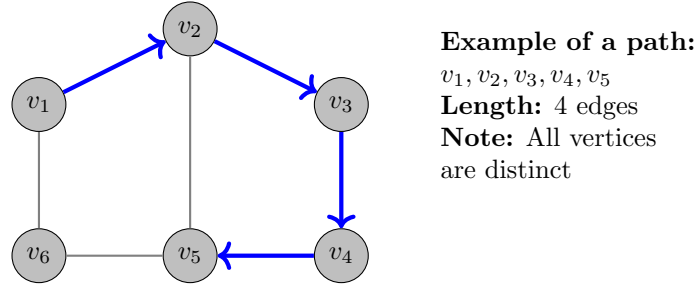


Figure 20: Illustration of a path in graph theory. A path is a walk where no vertex is repeated, ensuring each vertex appears at most once in the sequence. This example shows a path of length 4 (blue arrows) from v_1 to v_5 through the sequence v_1, v_2, v_3, v_4, v_5 . Paths are fundamental structures in graph theory and are used to define important concepts such as connectivity, distance, and shortest paths between vertices. Unlike walks, the no-repetition constraint makes paths more restrictive but also more useful for many graph algorithms.

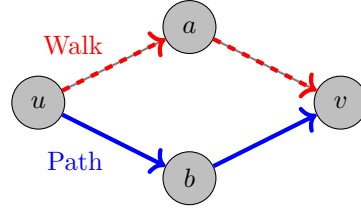
Definition 10 (Connected Graph). A graph $G = (V, E)$ is said to be *connected* if for every pair of distinct vertices $u, v \in V$, there exists a path in G from u to v . In other words, a graph is connected if and only if there is at least one path between any two vertices in the graph.

Lemma 1. A graph is connected if and only if every pair of vertices are the ends of some walk.

Definition 11 (Disconnected Graph). A graph $G = (V, E)$ is said to be *disconnected* if there exists at least one pair of distinct vertices $u, v \in V$ such that there is no path in G from u to v . Equivalently, a graph is disconnected if and only if its vertex set can be partitioned into two nonempty subsets such that there is no edge connecting any vertex in the first subset to any vertex in the second subset.

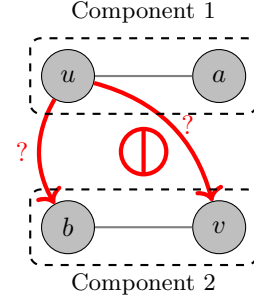
Definition 12 (Connected Component). A *connected component* of a graph $G = (V, E)$ is a maximal connected subgraph of G . That is, it is a connected subgraph $H = (V_H, E_H)$ of G such that no larger connected subgraph of G contains H as a proper subgraph. A connected graph has exactly one connected component, while a disconnected graph has two or more connected components.

Connected Graph



Every pair of vertices
can be connected by a path

Disconnected Graph



Some pairs of vertices
cannot be connected by any path

Figure 21: Comparison of connected and disconnected graphs. In the connected graph (left), there exists at least one path between any pair of vertices, as shown by the example paths between vertices u and v . The red dashed line represents one possible walk, while the blue solid line represents a path. In a disconnected graph (right), there exist pairs of vertices that cannot be connected by any path, as indicated by the crossed-out paths. The disconnected graph has two connected components (outlined by dashed boxes), which are maximal connected subgraphs.

Proof of Lemma 1. Every path is a walk, so in a connected graph every pair of vertices are the ends of some walk. Conversely, suppose vertices $u, v \in V(G)$ are the ends of a walk:

$$u = v_0, v_1, \dots, v_k = v$$

Take a shortest walk from u to v . We can cut out any loops in the walk. If some v_i is visited more than once, say:

$$u \rightarrow v_i \rightarrow \dots \rightarrow v_i \rightarrow v$$

Then we can use the segment of the walk from u to v_i followed by the segment from v_i to v :

$$u \rightarrow v_i \rightarrow v$$

which is shorter, a contradiction. So a shortest walk from u to v is a path. Hence, if every pair of vertices are the ends of some walk, then every pair of vertices is joined by a path, and G is connected. $\square$

Converting a Walk to a Path

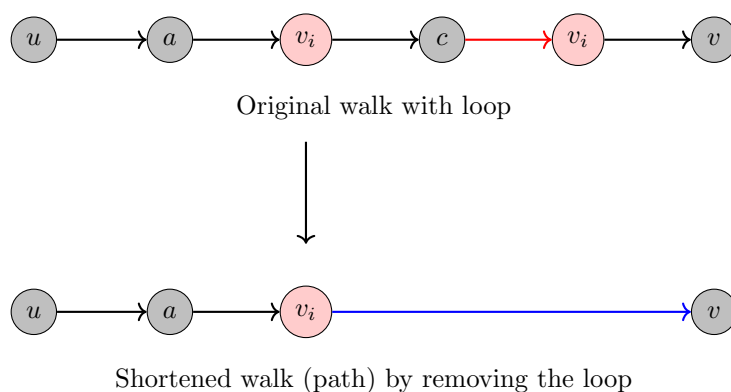
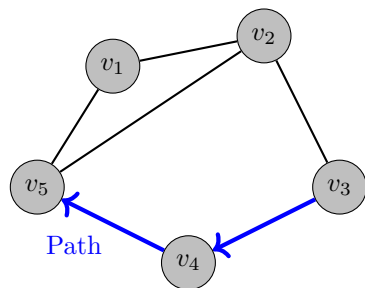


Figure 22: Illustration of the process of converting a walk to a path by removing loops. In the original walk (top), vertex v_i appears twice, creating a loop in the walk. By removing the portion of the walk between the two occurrences of v_i (marked in red), we obtain a shorter walk from u to v that doesn't repeat any vertices (bottom), thus forming a path. This technique is fundamental for proving that if there exists a walk between two vertices in a graph, then there must also exist a path between them.

1.7.1 More Examples of Walks, Paths, and Components

Connected Graph



Disconnected Graph

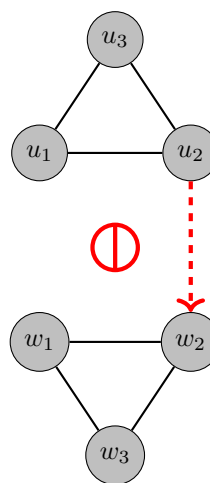


Figure 23: Comparison of connected and disconnected graphs. The connected graph (left) has a single component where a path exists between any pair of vertices, demonstrated by the blue path v_3, v_4, v_5 from v_3 to v_5 . The disconnected graph (right) contains two triangular structures that have no paths between them, as shown by the crossed-out red line. This fundamental distinction affects many graph properties, including traversability, algorithmic complexity, and network reliability.

Two separated components Three separated components Isolated vertices

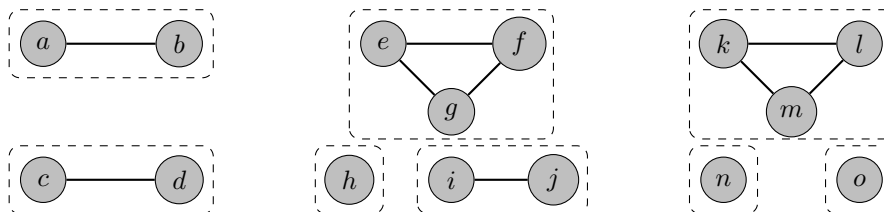


Figure 24: Various forms of disconnected graphs. Left: A disconnected graph with two separate edge components, where each component is a single edge. Middle: A graph with three distinct components - a triangular component, a single isolated vertex, and a path component with two vertices. Right: A graph with a triangular component and two isolated vertices. Each connected component (highlighted by dashed rectangles) forms a maximal connected subgraph. In graph theory, isolated vertices are considered separate connected components.

Graph Components

Disconnected Graph with 3 Components

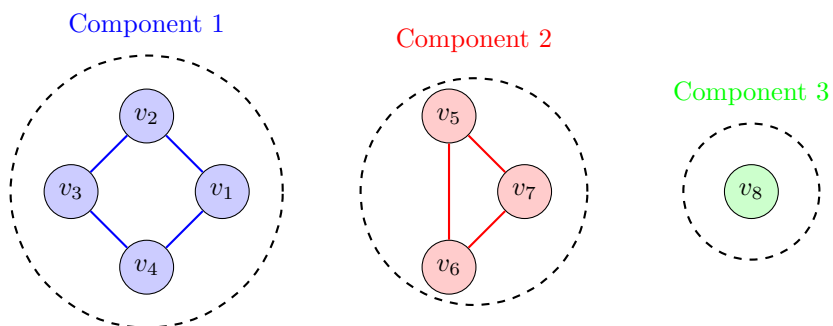
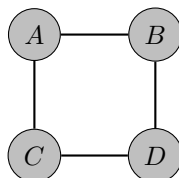


Figure 25: Illustration of a disconnected graph with three distinct components. Component 1 (blue) is a cycle graph with 4 vertices. Component 2 (red) is a triangle with 3 vertices. Component 3 (green) consists of a single isolated vertex. Each component is a maximal connected subgraph, meaning that there exists a path between any two vertices within the same component, but no path between vertices from different components. The number of components in a graph is an important invariant that affects connectivity, traversability, and many graph algorithms.

1.8 Chapter 1 Exercises

1.8.1 Easy Questions

1. Define the degree of a vertex in a graph and calculate the sum of all vertex degrees in the cycle graph C_5 .
2. Draw all possible non-isomorphic simple graphs with exactly 3 vertices.
3. Consider the following graph:

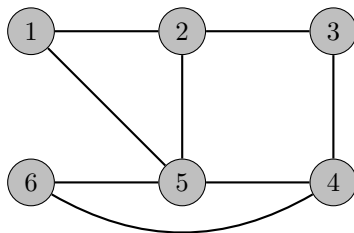


List all paths from vertex A to vertex D .

4. What is the difference between a walk and a path in a graph? Give an example of a walk that is not a path.
5. For each of the following graphs, determine if it is connected or disconnected:
 - A star graph S_6
 - A path graph P_4
 - A graph with vertices $\{a, b, c, d\}$ and edges $\{\{a, b\}, \{c, d\}\}$

1.8.2 Intermediate Questions

6. Draw a graph with degree sequence $(3, 3, 2, 2, 2, 0)$. Is there more than one non-isomorphic graph with this degree sequence?
7. Prove that a connected graph with n vertices must have at least $n - 1$ edges.
8. Consider a graph G with vertex set $V = \{1, 2, 3, 4, 5, 6\}$ and edges $\{1, 2\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 4\}, \{4, 5\}, \{4, 6\}, \{5, 6\}$.



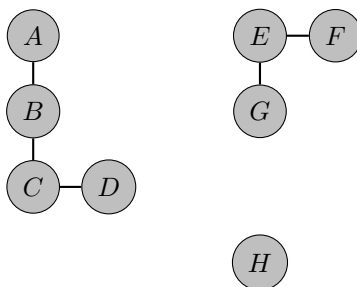
- Find an induced subgraph with vertex set $\{1, 2, 5, 6\}$
 - Find a spanning subgraph with exactly 6 edges
9. The Petersen graph is shown in your textbook (Figure 12). Find the longest path in this graph. Prove that your path is indeed the longest possible.
 10. Show that any graph with at least two vertices must have at least two vertices with the same degree.

1.8.3 Hard Questions

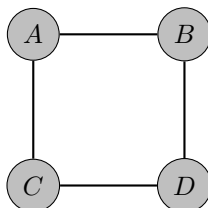
11. Prove that if G is a simple graph with n vertices and more than $\frac{(n-1)(n-2)}{2}$ edges, then G must be connected.
12. Let G be a graph with n vertices. If each vertex has degree at least $\frac{n-1}{2}$, prove that G is connected.
13. A graph is called bipartite if its vertices can be divided into two disjoint sets such that every edge connects a vertex in the first set to one in the second set.
 - Prove that a graph is bipartite if and only if it contains no cycles of odd length.
 - Which of the following graphs are bipartite: C_4 , C_5 , $K_{3,3}$, the Petersen graph?
14. Design a graph that that:
 - Has exactly 9 vertices
 - Is connected
 - Has exactly 2 vertices of degree 3, and the rest of degree 2
 - Contains exactly one cycle of length 4 and one cycle of length 5
15. Consider the hypercube graph Q_4 .
 - How many vertices and edges does Q_4 have?
 - What is the degree of each vertex?
 - Devise an algorithm to determine if there exists a path of length exactly k between any two vertices in Q_4 .

1.8.4 Questions with Diagrams

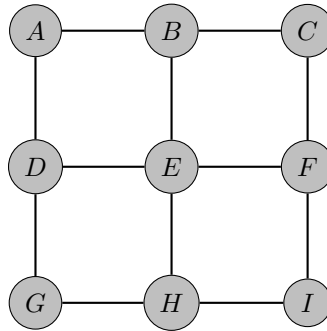
16. For the graph below, identify:
 - All isolated vertices
 - All pendant vertices
 - The degree sequence
 - The number of connected components



17. Draw all possible spanning trees of the following graph: (Hint 1: Refer to the definition of a spanning tree in Section 5.2)

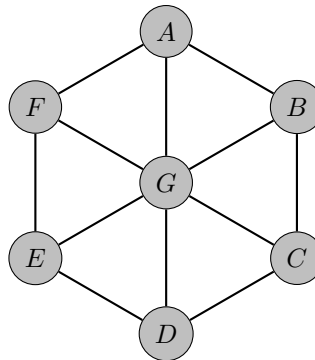


18. Consider the graph G shown below:

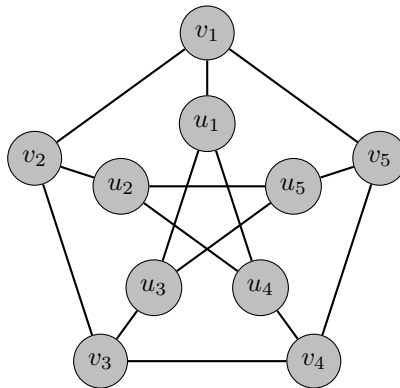


- Draw an induced subgraph $G[\{A, B, D, E\}]$
 - Find a subgraph that is not an induced subgraph
 - Draw a spanning subgraph that is a tree
 - What is the maximum number of edges that can be removed while keeping the graph connected?
19. In the wheel graph W_6 (shown below), find:

- The shortest path from any outer vertex to the vertex diametrically opposite on the cycle
- The number of distinct cycles of length 3
- The number of distinct cycles of length 4



20. Given the Petersen graph:



- Prove that it does not contain a cycle of length 3
- Find all cycles of length 5
- Show that the graph is not bipartite (Hint : Section 5.9)

2 Eulerian Graphs

2.1 Trails and Tours

Definition 13. A trail in a graph is a sequence of vertices:

$$v_1, v_2, \dots, v_n$$

such that $\{v_i, v_{i+1}\} \in E(G)$ and no edge appears twice. Unlike a path, a trail may visit the same vertex multiple times, but it cannot use the same edge more than once.

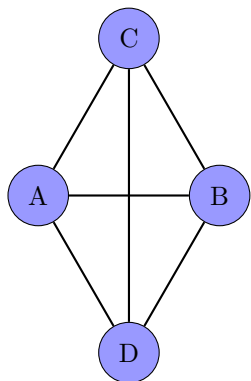
Definition 14. A tour is a trail $v_1, v_2, \dots, v_n$ with $v_n = v_1$. That is, a tour is a closed trail that starts and ends at the same vertex. A tour is also sometimes called a circuit.

Definition 15. An Eulerian trail is a trail that traverses every edge of the graph exactly once.

Definition 16. An Eulerian tour (or Eulerian circuit) is a tour that traverses every edge of the graph exactly once.

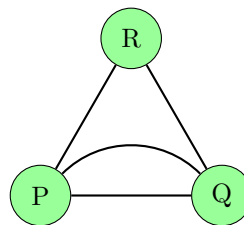
Theorem 1. Let G be a connected graph (or multigraph), and let $u, v \in V(G)$ with $u \neq v$. Then:

1. The graph has an Eulerian tour if and only if all vertices have even degree.
2. The graph has an Eulerian trail from u to v if and only if u and v have odd degree and all other vertices have even degree.



Complete graph K_4

No Eulerian tour or trail



Graph with 2 odd vertices

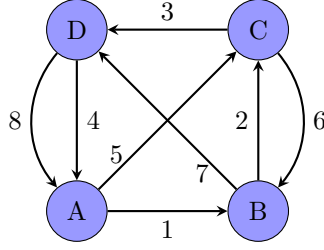
Eulerian trail exists, but no tour

Figure 26: Example graphs illustrating Eulerian concepts. Left: Complete graph K_4 , where all four vertices have degree 3 (odd). Since it has four vertices of odd degree, it has neither an Eulerian tour nor an Eulerian trail. Right: A graph with exactly two odd-degree vertices (P and Q) that has an Eulerian trail but no Eulerian tour.

Proof of (1). ($\Rightarrow$) Suppose there is an Eulerian tour $C = v_1, v_2, \dots, v_m, v_{m+1}$ with $v_{m+1} = v_1$, so that G has m edges. We prove that all vertices have even degree by strong induction on $m = |E(G)|$. Connectivity is not needed for this direction, so the induction runs over all graphs (or multigraphs) that have an Eulerian tour.

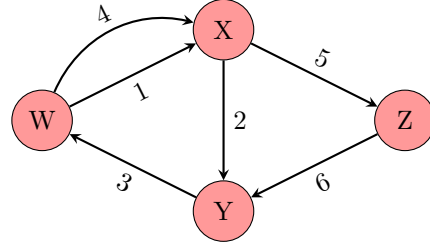
Base case: $m = 0$. Every vertex has degree 0, which is even.

Suppose $m \geq 1$ and the statement is true for all graphs with fewer than m edges. Let j be the smallest index $i > 1$ such that $v_i = v_k$ for some $k < i$, and fix this k . (Such an index exists because $v_{m+1} = v_1$.) By the minimality of j , the vertices $v_k, v_{k+1}, \dots, v_{j-1}$ are distinct, so the closed trail $D = v_k, v_{k+1}, \dots, v_j$



Eulerian tour: A-B-C-D-A-C-B-D-A

All vertices have degree 4 (even)



Eulerian trail: W-X-Y-W-X-Z-Y

W and Y have degree 3 (odd)

X and Z have degree 4 and 2 (even)

Figure 27: Explicit examples of Eulerian tours and trails. Left: A multigraph (with two parallel edges between B and C and two between D and A) where every vertex has even degree (4), with a complete Eulerian tour shown by numbered edges. The tour visits every edge exactly once and returns to the starting vertex A. Right: A multigraph (with two parallel edges between W and X) with exactly two odd-degree vertices (W and Y), showing an Eulerian trail that traverses every edge exactly once but doesn't form a closed circuit.

(where $v_j = v_k$) is a cycle; in a multigraph it may also be a loop or a pair of parallel edges. Hence every vertex is incident with exactly 0 or 2 edges of D (a loop counted twice).

If D uses all m edges, then every vertex of G has degree 0 or 2. Otherwise, remove the edges of D . The remaining edges form the closed trail

$$C' = v_1, \dots, v_k, v_{j+1}, \dots, v_{m+1},$$

which is an Eulerian tour of the graph H with vertex set $V(G)$ and the remaining edges. Since H has fewer than m edges, the induction hypothesis shows that every vertex has even degree in H . For every vertex x we have $d_G(x) = d_H(x) + d_D(x)$ with $d_D(x) \in \{0, 2\}$. Therefore, all vertices in the original graph have even degree.

($\Leftarrow$) Suppose all vertices have even degree, and let $C = v_1, v_2, \dots, v_n$ be a longest trail in the graph.

Case 1: Suppose $v_n \neq v_1$. Each time C passes through v_n before its last step it uses two edges at v_n , and its last edge uses one more, so C uses an odd number of the edges at v_n . Since v_n has even degree, we know an edge $\{v_n, v_{n+1}\}$ not in the trail exists. Now $C' = v_1, v_2, \dots, v_n, v_{n+1}$ is a longer trail, a contradiction. Hence $v_n = v_1$.

Case 2: $v_n = v_1$. Suppose some edge of G is not used by the trail. Since G is connected, some unused edge $\{u, v_i\}$ has an endpoint v_i on the trail. (If an unused edge xy has no endpoint on the trail, then a path in G from v_1 to x contains an edge with exactly one endpoint on the trail, and this edge is unused.) Then

$$u, v_i, v_{i+1}, \dots, v_n = v_1, v_2, \dots, v_i$$

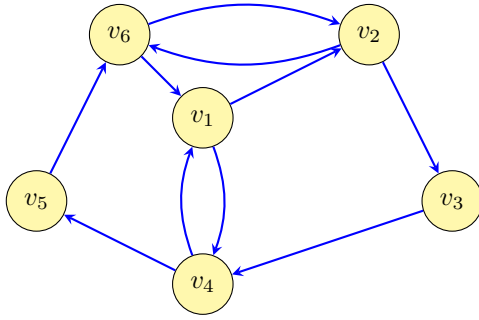
is a trail with one more edge than C , a contradiction. Therefore the trail uses every edge, and the graph has an Eulerian tour. $\square$

Proof of (2). ($\Rightarrow$) Suppose G has an Eulerian trail from u to v with $u \neq v$. Add an edge e between u and v . The resulting graph $G' = G \cup \{e\}$ has an Eulerian tour. By part (1), all vertices in G' have even degree.

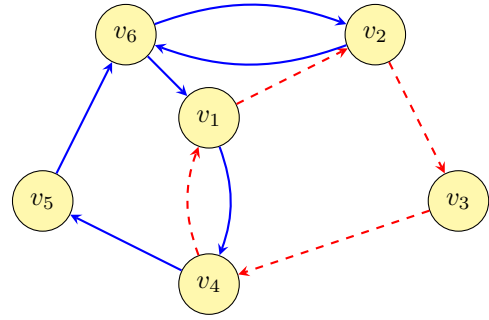
In the original graph G , the degrees of u and v are both reduced by 1, making them odd, while all other vertices maintain their even degree.

($\Leftarrow$) Suppose u and v have odd degree, and all other vertices have even degree. Add an edge e between u and v . In the resulting graph $G' = G \cup \{e\}$, all vertices now have even degree.

By part (1), G' has an Eulerian tour. This tour must use the edge e at some point. Remove e from the tour, and we obtain an Eulerian trail in G from u to v . $\square$

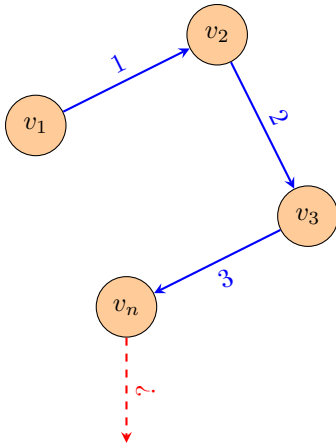


Eulerian tour: each pass through
a vertex uses two of its edges

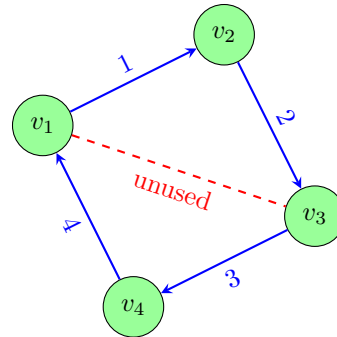


Induction step: remove the cycle
 $v_1v_2v_3v_4v_1$ (dashed)

Figure 28: Visualization of the forward direction proof - showing why Eulerian tours imply even degrees. Left: A multigraph (v_1 and v_4 are joined by two edges, as are v_2 and v_6) with the Eulerian tour $v_1, v_2, v_3, v_4, v_1, v_4, v_5, v_6, v_2, v_6, v_1$ (blue arrows). Each time the tour passes through a vertex it enters along one edge and leaves along another, so every vertex has even degree: v_3 and v_5 are passed once and have degree 2, and the other vertices are passed twice and have degree 4. Right: The induction step for this tour. The first repeated vertex is the fifth one, v_1 (so $j = 5$ and $k = 1$), so the cycle v_1, v_2, v_3, v_4, v_1 (red, dashed) is removed; the remaining edges form the closed trail $v_1, v_4, v_5, v_6, v_2, v_6, v_1$ (blue), which is an Eulerian tour of a graph with fewer edges.

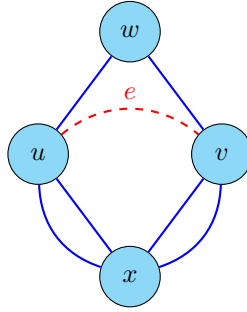


Case 1: $v_n \neq v_1$ and $d(v_n)$ is even
 $\Rightarrow$ Unused edge exists (contradiction)



Case 2: $v_n = v_1$ but not all edges used
 $\Rightarrow$ Can extend trail (contradiction)

Figure 29: Visualization of the reverse direction proof - showing why even degrees imply Eulerian tours. Left: If the longest trail doesn't end at its starting point and the final vertex has even degree, an unused edge must exist (contradiction). Right: If the trail forms a tour but doesn't use all edges, the graph has another edge that could extend the tour (contradiction).



Proof of (2): Add edge e between u and v
 This creates a graph where all vertices have even degree
 $\Rightarrow$ It has an Eulerian tour by part (1)

Figure 30: Visualization of the proof for part (2) - Converting between Eulerian trails and tours. In this multigraph, u and v have degree 3, w has degree 2 and x has degree 4, and u, w, v, x, u, x, v is an Eulerian trail. By adding a new edge e (red dashed line) between the two odd-degree vertices u and v , we transform the graph into one where every vertex has even degree. This creates a graph with an Eulerian tour, and removing edge e gives us the original Eulerian trail.

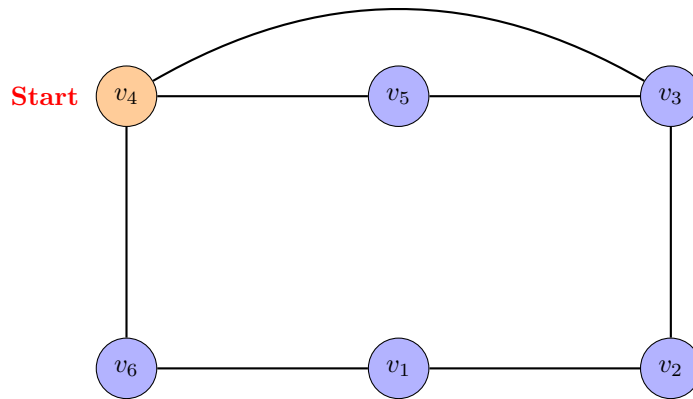


Figure 31: Graph for applying Fleury's Algorithm. This figure displays the initial connected graph with six vertices labeled v_1 through v_6 and seven edges. The vertices v_3 and v_4 have degree 3 and all other vertices have degree 2, so the graph has an Eulerian trail but no Eulerian tour, and every Eulerian trail starts at one of v_3, v_4 and ends at the other. Vertex v_4 is highlighted in orange as the starting point, indicated by the red "Start" label.

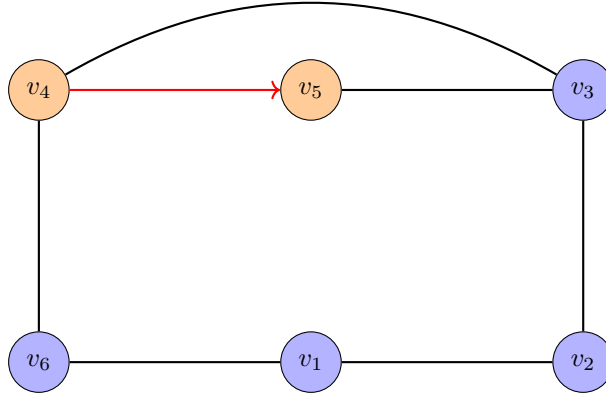


Figure 32: Step 1: Choose edge $\{v_4, v_5\}$. This figure shows the first step of Fleury's Algorithm. None of the three edges at v_4 is a bridge, so any of them may be chosen. Vertices v_4 and v_5 are both highlighted in orange, with a red directed arrow indicating the traversal from v_4 to v_5 . This represents the first edge chosen in the Eulerian trail.

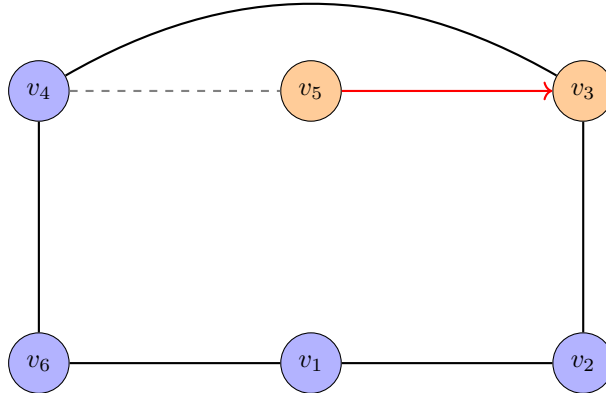


Figure 33: Step 2: Choose edge $\{v_5, v_3\}$. Vertices v_5 and v_3 are highlighted in orange. The previously traversed edge (from v_4 to v_5) is shown as a dashed gray line. Since $\{v_5, v_3\}$ is now the only unused edge at v_5 , the algorithm must take it; the red directed arrow from v_5 to v_3 shows the second edge of the trail.

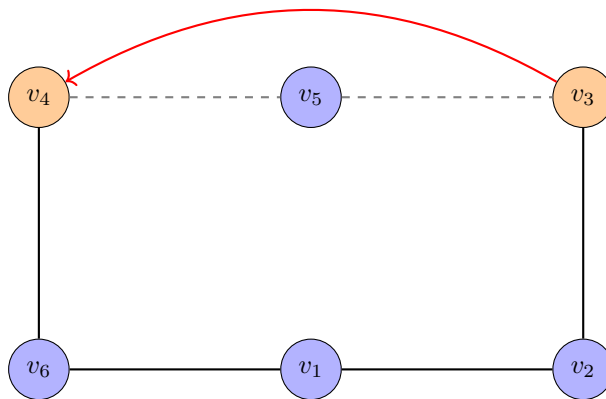


Figure 34: Step 3: Choose edge $\{v_3, v_4\}$. Vertices v_3 and v_4 are highlighted in orange. Previously traversed edges (v_4-v_5 and v_5-v_3) are shown as dashed gray lines. The unused edges form the cycle $v_3, v_4, v_6, v_1, v_2, v_3$, so neither unused edge at v_3 is a bridge and either may be chosen; the red directed arrow from v_3 to v_4 shows the choice made here.

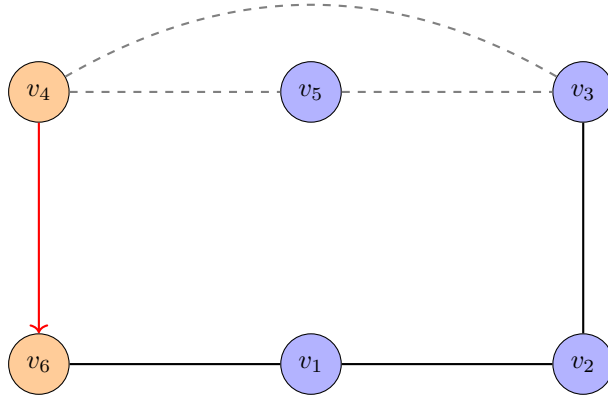


Figure 35: Step 4: Choose edge $\{v_4, v_6\}$. Vertices v_4 and v_6 are highlighted in orange. Three edges have now been traversed and appear as dashed gray lines. The only unused edge at v_4 is $\{v_4, v_6\}$, shown with a red directed arrow from v_4 to v_6 .

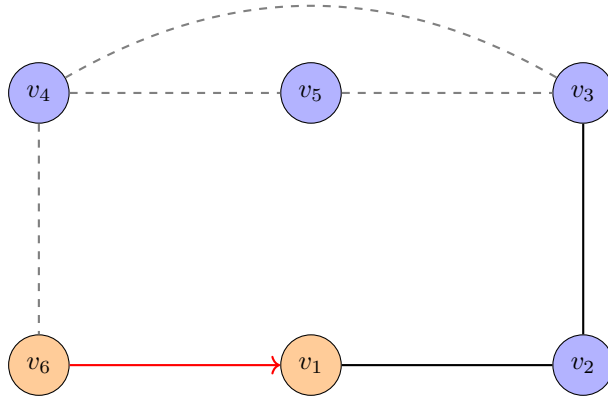


Figure 36: Step 5: Choose edge $\{v_6, v_1\}$. Vertices v_6 and v_1 are highlighted in orange. Four previously traversed edges are shown as dashed gray lines. The current edge being traversed from v_6 to v_1 is indicated by a red directed arrow.

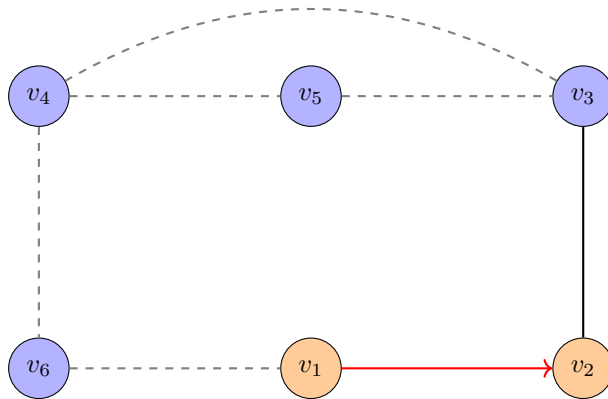


Figure 37: Step 6: Choose edge $\{v_1, v_2\}$. Vertices v_1 and v_2 are highlighted in orange. Most edges have been traversed (shown as dashed gray lines). The current traversal is from v_1 to v_2 , indicated by a red directed arrow.

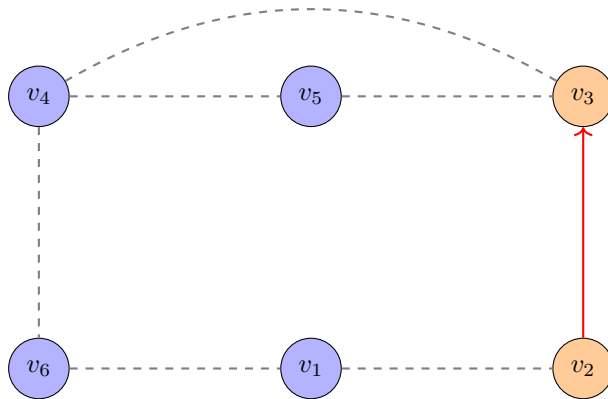


Figure 38: Step 7: Choose final edge $\{v_2, v_3\}$ to complete the trail. Vertices v_2 and v_3 are highlighted in orange with a red directed arrow from v_2 to v_3 . All seven edges have now been traversed exactly once. The trail ends at v_3 , the other vertex of odd degree, and not at the starting vertex: since the graph has two vertices of odd degree, the result is an Eulerian trail, not an Eulerian tour.

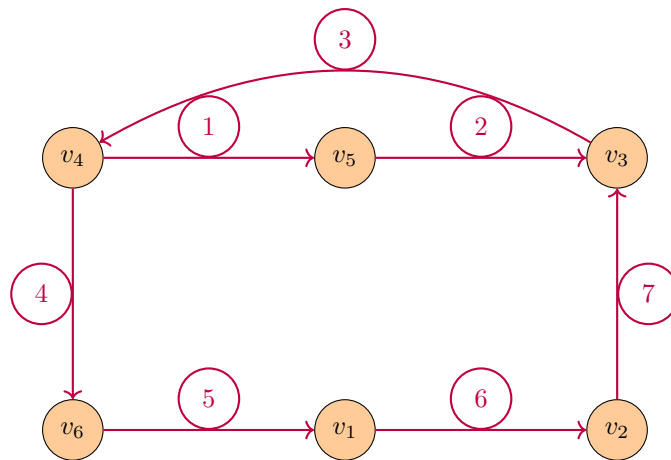


Figure 39: Complete Eulerian trail found by Fleury's Algorithm: $v_4 \rightarrow v_5 \rightarrow v_3 \rightarrow v_4 \rightarrow v_6 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$. All vertices are highlighted in orange, and all seven edges are shown as purple directed arrows with numbers indicating the order of traversal (1-7). This figure displays the entire Eulerian trail found by the algorithm, showing how every edge in the graph is traversed exactly once; the trail starts at v_4 and ends at v_3 , the two vertices of odd degree.

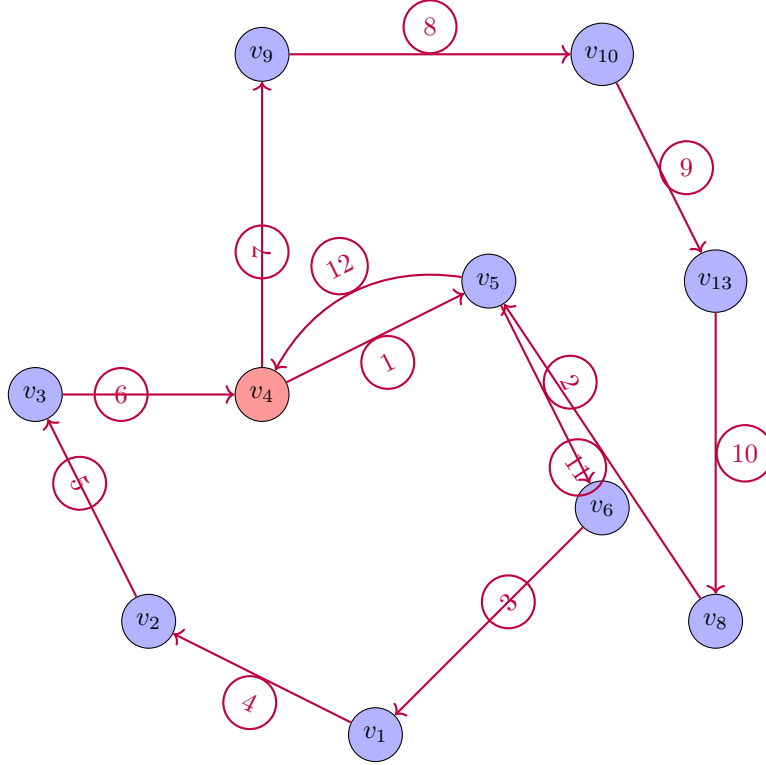


Figure 40: Example Eulerian tour: $v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_9 \rightarrow v_{10} \rightarrow v_{13} \rightarrow v_8 \rightarrow v_5 \rightarrow v_4$. This figure shows a more complex example of an Eulerian tour on a larger multigraph with 10 vertices, in which v_4 and v_5 are joined by two parallel edges (edges 1 and 12), so that v_4 and v_5 have degree 4 and all other vertices have degree 2. The starting vertex v_4 is highlighted in red, and the complete tour is shown with purple directed arrows numbered 1-12. This demonstrates how Fleury's Algorithm scales to larger graphs while still finding a valid Eulerian tour that traverses each edge exactly once before returning to the starting vertex.

Algorithm 1 Fleury's Algorithm for finding an Eulerian trail or circuit

Require: G is connected and has 0 or 2 vertices of odd degree; if it has two, $start$ is one of them

```

1: procedure FLEURY( $G(V, E)$ ,  $start$ )
2:   Let  $currentVertex \leftarrow start$ 
3:   Initialize empty list  $path$ 
4:   while  $E$  is not empty do
5:     if degree of  $currentVertex$  is 1 then
6:       Let  $nextEdge$  be the only edge incident to  $currentVertex$ 
7:     else
8:       Let  $nextEdge$  be any edge incident to  $currentVertex$  that is not a bridge of the current graph
9:     end if
10:    Let  $nextVertex$  be the vertex at the other end of  $nextEdge$ 
11:    Add  $currentVertex$  to  $path$ 
12:    Remove  $nextEdge$  from  $E$ 
13:     $currentVertex \leftarrow nextVertex$ 
14:  end while
15:  Add  $currentVertex$  to  $path$  ▷ Add the final vertex
16:  return  $path$ 
17: end procedure

```

2.2 Eulerian Digraphs and De Bruijn Sequences

2.2.1 Eulerian Digraphs

Definition 17 (Eulerian Directed Graphs). A directed graph (digraph) is Eulerian if there exists a closed walk that traverses each edge exactly once. The following properties characterize Eulerian digraphs without isolated vertices (here “connected” means that the underlying undirected graph is connected):

1. A digraph is Eulerian if and only if it is connected and the in-degree equals the out-degree for every vertex.
2. A digraph has an Eulerian trail (but not necessarily a closed one) if and only if it is connected and:
 - At most one vertex has $(\text{out-degree}) - (\text{in-degree}) = 1$
 - At most one vertex has $(\text{in-degree}) - (\text{out-degree}) = 1$
 - For all other vertices, in-degree equals out-degree

For a directed graph $G = (V, E)$, define:

$$d^+(v) = \text{out-degree of vertex } v \quad (5)$$

$$d^-(v) = \text{in-degree of vertex } v \quad (6)$$

Then a digraph G without isolated vertices is Eulerian if and only if:

1. G is connected (or more precisely, the underlying undirected graph of G is connected)
2. $\forall v \in V, d^+(v) = d^-(v)$

2.2.2 De Bruijn Sequences

Definition 18 (De Bruijn Sequence). A De Bruijn sequence $B(k, n)$ is a cyclic sequence of length k^n over an alphabet A of size k such that each possible string of length n appears exactly once as a contiguous substring.

Properties 1 (De Bruijn Sequences).

1. The length of a De Bruijn sequence $B(k, n)$ is k^n .
2. Every string of length n over the alphabet appears exactly once as a substring.
3. A De Bruijn sequence $B(k, n)$ can be viewed as a Hamiltonian cycle in the De Bruijn graph $G(k, n)$, whose vertices are the strings of length n (equivalently, as an Eulerian circuit in $G(k, n-1)$; see below).
4. The number of distinct De Bruijn sequences $B(k, n)$ is given by:

$$\frac{(k!)^{k^{n-1}}}{k^n} \quad (7)$$

5. More specifically, the number of binary De Bruijn sequences $B(2, n)$ is:

$$2^{2^{n-1} - n} \quad (8)$$

Properties 2 (Connection to Eulerian Digraphs). *De Bruijn sequences have a fundamental connection to Eulerian digraphs:*

1. A De Bruijn graph $G(k, n-1)$ is a directed graph with k^{n-1} vertices, each labeled with a string of length $n-1$.

2. There is an edge from vertex u to vertex v if the last $n - 2$ symbols of u match the first $n - 2$ symbols of v .
3. For every vertex in $G(k, n - 1)$, the in-degree equals the out-degree (both equal to k).
4. $G(k, n - 1)$ is strongly connected: from any vertex u one can reach any vertex v in $n - 1$ steps by appending the symbols of v one at a time.
5. Therefore, $G(k, n - 1)$ is an Eulerian digraph.
6. A De Bruijn sequence $B(k, n)$ corresponds to an Eulerian circuit in $G(k, n - 1)$.

Properties 3 (Mathematical Structure). The adjacency matrix A of the De Bruijn graph $G(k, n)$ has the following properties:

1. A is a $k^n \times k^n$ matrix.
2. $A_{i,j} = 1$ if the last $n - 1$ symbols of vertex i match the first $n - 1$ symbols of vertex j .
3. $A_{i,j} = 0$ otherwise.
4. The number of Eulerian circuits (and thus De Bruijn sequences) can be computed using the BEST theorem:

$$\text{Number of Eulerian circuits} = t_w \prod_{v \in V} (d^+(v) - 1)! \quad (9)$$

where t_w is the number of oriented spanning trees rooted at vertex w (see the definition of a spanning tree in Chapter 5).

Properties 4 (Combinatorial Aspects). The combinatorial aspects of De Bruijn sequences $B(k, n)$ have the following properties

1. A De Bruijn sequence can be constructed from a Eulerian circuit in the De Bruijn graph $G(k, n - 1)$.
2. Every De Bruijn sequence $B(k, n)$ contains k^n distinct substrings of length n .
3. The De Bruijn sequence has a feedback function f that generates the next bit:

$$a_{i+n} = f(a_i, a_{i+1}, \dots, a_{i+n-1}) \quad (10)$$

Theorem 2 (Generating Function). For a binary De Bruijn sequence, the generating function can be expressed as:

$$G(x) = \sum_{i=0}^{2^n-1} a_i x^i \mod (x^{2^n} - 1) \quad (11)$$

where a_i represents the i -th element in the sequence.

Theorem 3 (LFSR Representation). A binary Linear Feedback Shift Register (LFSR) of length n with the characteristic polynomial:

$$p(x) = x^n + c_{n-1}x^{n-1} + \dots + c_1x + c_0 \quad (12)$$

generates a sequence satisfying the recurrence relation:

$$a_{i+n} = c_0 a_i \oplus c_1 a_{i+1} \oplus \dots \oplus c_{n-1} a_{i+n-1} \quad (13)$$

where $\oplus$ represents the XOR operation. The all-zero state is mapped to itself, so the period of the output is at most $2^n - 1$; it equals $2^n - 1$ when $p(x)$ is primitive and the initial state is nonzero, and then every nonzero binary string of length n appears exactly once per period (an m -sequence). In particular, an LFSR can never generate a binary De Bruijn sequence. Inserting one extra 0 into the unique run of $n - 1$ zeros of an m -sequence gives a De Bruijn sequence $B(2, n)$, but the resulting feedback function is nonlinear.

Theorem 4 (Counting Formula). The number of distinct De Bruijn sequences $B(k, n)$ is:

$$N(k, n) = \frac{(k!)^{k^{n-1}}}{k^n} \quad (14)$$

For the binary case $B(2, n)$, the number of distinct sequences is:

$$N(2, n) = 2^{2^{n-1}-n} \quad (15)$$

Example 2 (Small Values). The number of distinct De Bruijn sequences for small parameters:

$$N(2, 2) = 1 \quad (16)$$

$$N(2, 3) = 2 \quad (17)$$

$$N(2, 4) = 16 \quad (18)$$

$$N(2, 5) = 2048 \quad (19)$$

$$N(3, 2) = 24 \quad (20)$$

$$N(3, 3) = 373248 \quad (21)$$

$$N(4, 2) = 20736 \quad (22)$$

Theorem 5 (FKM Algorithm). The FKM (Fredricksen, Kessler, Maiorana) algorithm constructs a binary De Bruijn sequence $B(2, n)$ by generating all necklaces of length n , ordering them lexicographically, replacing each with its aperiodic prefixes, and concatenating the last bits of these prefixes in order.

Algorithm 2 FKM Algorithm for $B(2, n)$

Require: Integer $n \geq 2$

Ensure: Binary De Bruijn sequence $B(2, n)$

```

1:  $S \leftarrow$  empty sequence
2: function GENERATENECKLACES( $n$ )
3:    $N \leftarrow \emptyset$ ;  $\text{seen} \leftarrow \emptyset$ 
4:   for all binary strings  $b$  of length  $n$  in lexicographic order do
5:     if  $b \notin \text{seen}$  then
6:       necklace  $\leftarrow$  smallest rotation of  $b$ 
7:        $N \leftarrow N \cup \{\text{necklace}\}$ ;  $\text{seen} \leftarrow \text{seen} \cup \{\text{all rotations of } b\}$ 
8:     end if
9:   end for
10:  return  $N$ 
11: end function
12: function GETAPERIODICPREFIXES(necklace)
13:   $\text{prefixes} \leftarrow$  empty list;  $p \leftarrow$  period of necklace
14:  for  $i \leftarrow 1$  to  $p$  do
15:    Append the first  $i$  bits of necklace to  $\text{prefixes}$ 
16:  end for
17:  return  $\text{prefixes}$ 
18: end function
19:  $\text{necklaces} \leftarrow \text{Sort}(\text{GenerateNecklaces}(n))$ 
20: for each  $\text{necklace}$  in  $\text{necklaces}$  do
21:   for each  $\text{prefix}$  in  $\text{GetAperiodicPrefixes}(\text{necklace})$  do
22:      $S \leftarrow S \cdot \text{last bit of } \text{prefix}$ 
23:   end for
24: end for
25: return  $S$ 

```

Remark 1. *The FKM algorithm can also be implemented using Lyndon words: the aperiodic prefix of a necklace of length n is a Lyndon word whose length divides n , and every such Lyndon word arises in this way. Equivalently, the sequence produced by the FKM algorithm is the concatenation, in lexicographic order, of all binary Lyndon words whose length divides n .*

Algorithm 3 Lyndon-based FKM Algorithm

Require: Integer $n \geq 2$

Ensure: Binary De Bruijn sequence $B(2, n)$

```

1: function ISNECKLACE( $w$ )
2:   for  $i \leftarrow 1$  to  $|w| - 1$  do
3:     if  $w >$  rotation of  $w$  by  $i$  then
4:       return false
5:     end if
6:   end for
7:   return true
8: end function
9: function PERIOD( $w$ )
10:  for  $i \leftarrow 1$  to  $|w|$  do
11:    if  $w =$  rotation of  $w$  by  $i$  then return  $i$ 
12:    end if
13:  end for
14:  return  $|w|$ 
15: end function
16:  $S \leftarrow$  empty sequence
17: for each binary string  $w$  of length  $n$  in lexicographic order do
18:   if IsNecklace( $w$ ) then
19:      $period \leftarrow$  Period( $w$ )  $\triangleright$  the first  $period$  bits of  $w$  form a Lyndon word
20:     for  $i \leftarrow 1$  to  $period$  do
21:        $S \leftarrow S \cdot$  last bit of first  $i$  bits of  $w$ 
22:     end for
23:   end if
24: end for
25: return  $S$ 

```

Theorem 6 (Prefer-1 Algorithm). The Prefer-1 algorithm generates a binary De Bruijn sequence $B(2, n)$ by starting with n zeros and repeatedly appending either a 1 (preferred) or a 0, choosing whichever creates a new n -tuple that hasn't appeared before in the sequence, and finally deleting the last $n - 1$ bits, which are all zeros.

Theorem 7 (Correctness). The Prefer-1 algorithm correctly generates a binary De Bruijn sequence of order n with length 2^n .

Remark 2. *The Prefer-1 algorithm can be viewed as a greedy approach that traverses a De Bruijn graph by always choosing the edge labeled 1 when possible. It represents a special case of the more general Prefer- d algorithm for arbitrary alphabet sizes.*

Theorem 8 (Prefer-Opposite Algorithm). The Prefer-Opposite algorithm generates a binary De Bruijn sequence $B(2, n)$ by starting with n zeros and repeatedly appending the opposite of the last bit if this creates a new n -tuple that hasn't appeared before in the sequence, and otherwise the same as the last bit, until the last $n - 1$ bits are all 1. It then appends a 1 followed by $n - 1$ zeros and deletes the first $n - 1$ bits. Without this final step the all-ones n -tuple would never be produced.

Theorem 9 (Correctness). The Prefer-Opposite algorithm correctly generates a binary De Bruijn sequence of order n with length 2^n .

Algorithm 4 Prefer-1 Algorithm for $B(2, n)$

Require: Integer $n \geq 2$ **Ensure:** Binary De Bruijn sequence $B(2, n)$

```
1:  $R \leftarrow$  n-bit register initialized to all zeros
2:  $S \leftarrow$  sequence of  $n$  zeros ▷ Output the initial  $n$  zeros
3:  $seen \leftarrow \{R\}$  ▷ Set of seen  $n$ -tuples, initially contains all zeros
4: while  $|seen| < 2^n$  do
5:    $R_{next1} \leftarrow$  last  $n - 1$  bits of  $R$  followed by 1
6:   if  $R_{next1} \notin seen$  then
7:      $R \leftarrow R_{next1}$  ▷ Shift in a 1
8:      $S \leftarrow S \cdot 1$  ▷ Append 1 to output
9:   else
10:     $R \leftarrow$  last  $n - 1$  bits of  $R$  followed by 0 ▷ Shift in a 0
11:     $S \leftarrow S \cdot 0$  ▷ Append 0 to output
12:   end if
13:    $seen \leftarrow seen \cup \{R\}$ 
14: end while
15: Delete the last  $n - 1$  bits of  $S$  ▷ They are all zeros; now  $|S| = 2^n$ 
16: return  $S$ 
```

Algorithm 5 Prefer-Opposite Algorithm for $B(2, n)$

Require: Integer $n \geq 2$ **Ensure:** Binary De Bruijn sequence $B(2, n)$

```
1:  $R \leftarrow$  n-bit register initialized to all zeros
2:  $S \leftarrow$  sequence of  $n$  zeros ▷ Output the initial  $n$  zeros
3:  $last \leftarrow 0$  ▷ Last bit output
4:  $seen \leftarrow \{R\}$  ▷ Set of seen  $n$ -tuples, initially contains all zeros
5: while the last  $n - 1$  bits of  $S$  are not all 1 do
6:    $opposite \leftarrow 1 - last$  ▷ Opposite of the last bit
7:    $R_{next} \leftarrow$  last  $n - 1$  bits of  $R$  followed by  $opposite$ 
8:   if  $R_{next} \notin seen$  then
9:      $last \leftarrow opposite$  ▷ Prefer the opposite
10:  else
11:     $R_{next} \leftarrow$  last  $n - 1$  bits of  $R$  followed by  $last$ 
12:  end if
13:   $R \leftarrow R_{next}$  ▷ Update register
14:   $S \leftarrow S \cdot last$  ▷ Append bit to output
15:   $seen \leftarrow seen \cup \{R\}$ 
16: end while
17:  $S \leftarrow S \cdot 1 \cdot 0^{n-1}$  ▷ Append 1 followed by  $n - 1$  zeros
18: Delete the first  $n - 1$  bits of  $S$  ▷ Now  $|S| = 2^n$ 
19: return  $S$ 
```

Remark 3. *Every binary De Bruijn sequence of order n contains exactly 2^{n-1} zeros and 2^{n-1} ones (its bits are the first bits of its 2^n windows, and exactly half of all n -bit strings begin with 1), so the two algorithms do not differ in the overall balance of 0s and 1s. They differ in how the bits are arranged: the Prefer-1 sequence begins with the long runs $0^n 1^n$, while the Prefer-Opposite sequence begins with n alternating bits $0101 \dots$; in fact its run-length encoding is the lexicographically smallest among all binary De Bruijn sequences of order n that begin with 0. This approach can be generalized to larger alphabets by defining a suitable notion of “opposite.”*

Theorem 10 (Cycle-Joining Algorithm). The cycle-joining method constructs a De Bruijn sequence $B(k, n)$ by first generating all disjoint cycles of a nonsingular feedback shift register (for example the pure cycling register, whose cycles are the necklaces of length n), then identifying a spanning tree (see the definition of a spanning tree in Chapter 5) of the cycle graph, and systematically joining cycles according to the spanning tree to form a single cycle containing all states. Two cycles are joined at a pair of conjugate states, that is, two states that differ only in their first symbol and lie on different cycles: interchanging the successors of these two states merges the two cycles into one. Huang (1990) gave a construction of this kind for binary De Bruijn sequences, based on the cycles of the complemented cycling register.

Theorem 11 (Correctness). If the feedback shift register is nonsingular and its cycle graph is connected (as it is for the pure cycling register), the Cycle-Joining algorithm correctly generates a De Bruijn sequence of order n over an alphabet of size k with length k^n .

Remark 4. *The cycle-joining method provides a general approach that works for any alphabet size and offers flexibility in how cycles are joined, enabling the construction of De Bruijn sequences with specific properties by choosing different spanning trees of the cycle graph.*

Remark 5 (On Spanning Trees). *The foundational concepts of spanning trees are addressed in Section 5.2 as part of our characterization of trees. Readers may find this material particularly relevant for understanding the tree-based algorithms presented in subsequent sections, including breadth-first search and depth-first search (Section 5.3), as well as Prim’s and Kruskal’s algorithms (Sections 5.4 and 5.5). Those unfamiliar with tree structures may wish to review the fundamental properties outlined in Section 5.1 before proceeding.*

Remark 6 (Non-uniqueness). *Binary De Bruijn sequences are not unique for $n > 2$. The number of distinct De Bruijn sequences $B(2, n)$ is given by $2^{2^{n-1}-n}$, which grows rapidly with n . For example, there are 2 distinct $B(2, 3)$ sequences and 16 distinct $B(2, 4)$ sequences.*

Table 1: De Bruijn Sequence Algorithm Comparison

	Cycle-joining	Prefer-Opp.	Prefer-1	Lyndon
Purpose	General	Binary	Binary	Mathematical
Time	$O(k^n)$	$O(2^n)$	$O(2^n)$	$O(2^n)$
Space	$O(k^n)$	$O(2^n)$	$O(2^n)$	$O(n)$
Key Feature	Any k	Greedy	Simplest	Elegant

2.3 De Bruijn Graphs

Definition 19 (De Bruijn Graph). The De Bruijn graph $G(k, n)$ of order n over an alphabet of size k (also written $B(k, n)$) is a directed graph representing all possible sequences of a given length over a given alphabet:

1. The graph has k^n vertices, where k is the alphabet size.
2. Each vertex represents a sequence of length n over the alphabet.
3. There is a directed edge from vertex u to vertex v if the last $n - 1$ symbols of u match the first $n - 1$ symbols of v .

Algorithm 6 Cycle-Joining Algorithm for $B(k, n)$

Require: Integer $n \geq 2$, alphabet size $k \geq 2$ **Ensure:** De Bruijn sequence $B(k, n)$

```
1: function GENERATEFSRCYCLES( $n, k$ )
2:   Generate all disjoint cycles formed by a nonsingular feedback shift register (e.g., the pure cycling
   register)
3:   return set of all cycles  $C = \{C_1, C_2, \dots, C_m\}$ 
4: end function
5: function BUILDCYCLEGRAPH( $C$ )
6:    $G \leftarrow$  empty graph with vertices representing cycles
7:   for each cycle  $C_i$  in  $C$  do
8:     Add a vertex  $v_i$  to  $G$  representing cycle  $C_i$ 
9:   end for
10:  for each pair of conjugate states  $s \in C_i, \hat{s} \in C_j$  with  $i \neq j$  do
11:    Add an edge  $(v_i, v_j)$  to  $G$  labeled with the joining state  $s$ 
12:  end for
13:  return  $G$ 
14: end function
15: function JOINCYCLES( $C_i, C_j, s$ )
16:    $s$  is a state of  $C_i$  whose conjugate  $\hat{s}$  (same last  $n - 1$  symbols, different first symbol) lies on  $C_j$ 
17:   Interchange the successors of  $s$  and  $\hat{s}$ , which merges  $C_i$  and  $C_j$  into a single cycle
18:   return the merged cycle
19: end function
20:  $cycles \leftarrow$  GenerateFSRCycles( $n, k$ )
21:  $G \leftarrow$  BuildCycleGraph( $cycles$ )
22:  $connectedCycles \leftarrow \emptyset$  ▷ Set of cycles already joined
23:  $joinEdges \leftarrow \emptyset$  ▷ Set of edges used to join cycles
24:  $result \leftarrow$  first cycle in  $cycles$ 
25:  $connectedCycles \leftarrow connectedCycles \cup \{result\}$ 
26: while  $|connectedCycles| < |cycles|$  do
27:    $bestEdge \leftarrow$  null
28:   for each edge  $(v_i, v_j)$  in  $G$  where  $v_i$  represents a cycle in  $connectedCycles$  and  $v_j$  doesn't do
29:     if  $bestEdge$  is null or edge  $(v_i, v_j)$  is preferred for joining then
30:        $bestEdge \leftarrow (v_i, v_j)$ 
31:     end if
32:   end for
33:   Add cycle represented by  $v_j$  to  $connectedCycles$ 
34:    $joinEdges \leftarrow joinEdges \cup \{bestEdge\}$ 
35:    $s \leftarrow$  joining state associated with  $bestEdge$ 
36:    $result \leftarrow$  JoinCycles( $result$ , cycle for  $v_j, s$ )
37: end while
38: return sequence representation of  $result$ 
```

4. Each edge is labeled with the symbol that extends the sequence represented by u to obtain the sequence for the destination vertex v .
5. The graph has exactly k^{n+1} edges.
6. For every vertex, the in-degree equals the out-degree (both equal to k).
7. A De Bruijn sequence $B(k, n + 1)$ corresponds to an Eulerian cycle in this graph.

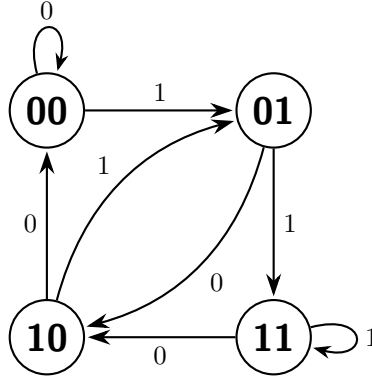


Figure 41: De Bruijn Graph $B(2, 2)$. This directed graph has 4 vertices representing all 2-bit sequences, and 8 edges labeled by the appended bit. An Eulerian circuit on this graph yields a de Bruijn sequence of order 3 over a binary alphabet.

Lemma 2 (Connection to Eulerian Circuits). A De Bruijn sequence $B(k, n)$ corresponds exactly to an Eulerian circuit in the De Bruijn graph $B(k, n - 1)$.

Proof sketch. Each edge in the De Bruijn graph $B(k, n - 1)$ represents the transition from one $(n - 1)$ -length sequence to another by appending a symbol and removing the first symbol. Following an Eulerian circuit through all edges generates each possible n -length sequence exactly once, which is precisely the definition of a De Bruijn sequence. $\square$

Lemma 3 (Fundamental Property of De Bruijn Graphs). For any alphabet size k and order n , the De Bruijn graph $B(k, n)$ has exactly k^n vertices and k^{n+1} edges.

Proof. Each vertex in $B(k, n)$ represents a sequence of n symbols from an alphabet of size k , giving exactly k^n possible vertices. From each vertex, there are k possible transitions (one for each symbol in the alphabet), resulting in exactly $k \cdot k^n = k^{n+1}$ edges. $\square$

Remark 7 (Relationship to Universal Cycles). *De Bruijn sequences are a special case of universal cycles, which are cyclic sequences where each object from a set appears exactly once as a consecutive substring. When the objects are all possible n -tuples from a k -symbol alphabet, the universal cycle is precisely a De Bruijn sequence of order n .*

Theorem 12 (Line Graph Relationship). The De Bruijn graph $B(k, n + 1)$ is isomorphic to the line graph of $B(k, n)$.

Proof. In the line graph of $B(k, n)$, each vertex corresponds to an edge in $B(k, n)$. An edge in $B(k, n)$ represents a transition from an n -length sequence to another by appending a symbol and removing the first symbol, which results in an $(n + 1)$ -length sequence. This exactly corresponds to a vertex in $B(k, n + 1)$. Two vertices in the line graph are adjacent if their corresponding edges in $B(k, n)$ form a directed path of length 2, which precisely matches the adjacency condition in $B(k, n + 1)$. $\square$

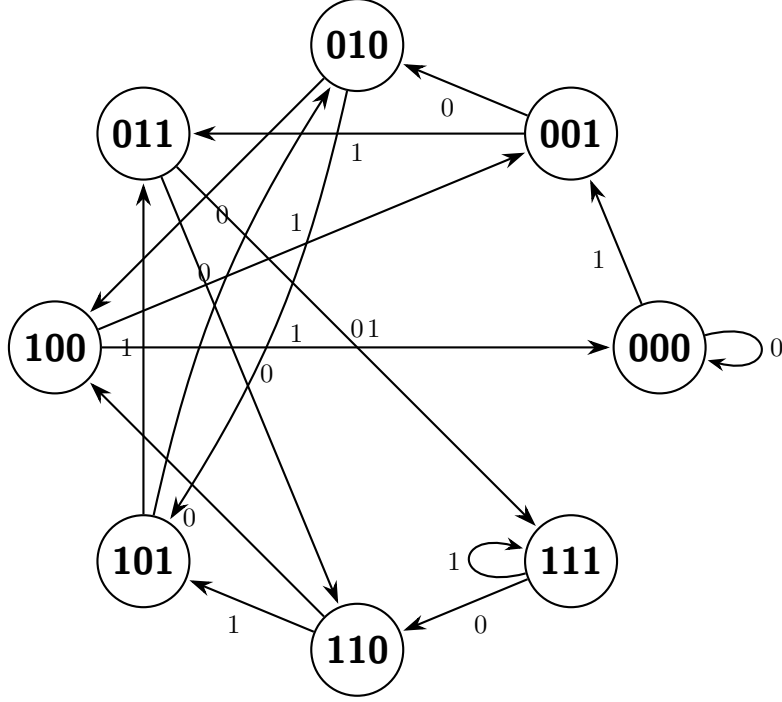


Figure 42: De Bruijn Graph $B(2, 3)$. This directed graph has 8 vertices representing all possible 3-bit binary sequences (000 through 111). Each edge is labeled with the bit (0 or 1) that is appended when transitioning between states. The graph has 16 edges in total, and a De Bruijn sequence $B(2, 4)$ corresponds to an Eulerian circuit traversing all edges exactly once.

Theorem 13 (Shift Register Implementation). Every binary De Bruijn sequence of order n can be generated by a feedback shift register of length n with an appropriate feedback function. This feedback function is necessarily nonlinear.

Proof outline. Consider an n -bit shift register with bits $(b_1, b_2, \dots, b_n)$. For each state of the register, we compute the next bit b_{n+1} using a feedback function $f(b_1, b_2, \dots, b_n)$. This bit is then shifted into the register, and b_1 is discarded.

Let $s_0 s_1 \dots s_{2^n-1}$ be a binary De Bruijn sequence of order n , with indices taken modulo 2^n . Since every binary string of length n occurs exactly once as a window $(s_i, s_{i+1}, \dots, s_{i+n-1})$, the rule

$$f(s_i, s_{i+1}, \dots, s_{i+n-1}) = s_{i+n}$$

defines f unambiguously on all 2^n states. Started in the state $(s_0, s_1, \dots, s_{n-1})$, the register outputs $s_0, s_1, s_2, \dots$ and cycles through all 2^n possible states before repeating.

The function f cannot be linear: a linear f satisfies $f(0, 0, \dots, 0) = 0$, so the state $(0, 0, \dots, 0)$ would be followed by itself, whereas in a De Bruijn sequence the window $00 \dots 0$ occurs only once in a period of length $2^n > 1$. In particular, $f(0, 0, \dots, 0) = 1$. For example, for $n = 3$ the linear recurrence $a_{i+3} = a_i \oplus a_{i+1}$ (characteristic polynomial $x^3 + x + 1$) produces the maximum-length sequence 0010111 of period 7, and the modified, nonlinear feedback function

$$f(b_1, b_2, b_3) = b_1 \oplus b_2 \oplus ((1 \oplus b_2)(1 \oplus b_3))$$

where $\oplus$ is XOR, produces the De Bruijn sequence 00010111, which includes all 2^3 possible states. $\square$

2.4 Chapter 2 Exercises

2.4.1 Easy Problems: Eulerian Graphs

1. Determine whether the following graph is Eulerian, semi-Eulerian, or neither: A graph with vertices $\{A, B, C, D, E\}$ and edges $\{(A, B), (B, C), (C, D), (D, E), (E, A), (A, C), (C, E)\}$
2. Prove that a connected graph has an Eulerian trail if and only if it has exactly 0 or 2 vertices with odd degree.
3. Construct a connected simple graph with 6 vertices that has an Eulerian circuit. How many edges must such a graph have at minimum?
4. Given a connected graph where all vertices have even degree, prove that it has an Eulerian circuit.

2.4.2 Medium Problems: Eulerian Graphs

5. A town is divided by a river into four land areas A, B, C, D , joined by 7 bridges: two between A and B , two between A and C , and one each between A and D , B and D , and C and D (the bridges of Königsberg). Is it possible to walk through the town crossing each bridge exactly once? If so, find such a walk. If not, explain why.
6. Show that if G is a connected graph where every vertex has even degree, then removing any edge results in a graph that has an Eulerian trail but not an Eulerian circuit.
7. Given a connected graph G with $2k$ vertices of odd degree ($k \geq 1$), what is the minimum number of edge-disjoint trails needed to cover all edges of G exactly once?
8. Prove that if G is a connected graph with exactly two vertices of odd degree, say u and v , then there exists an Eulerian trail from u to v .

2.4.3 Hard Problems: Eulerian Graphs

9. Prove that a connected graph G has an Eulerian circuit if and only if its edge set can be partitioned into simple cycles.
10. Consider a complete graph K_n . For which values of n does K_n have an Eulerian circuit? Provide a proof for your answer.
11. Let G be a connected graph. Prove that the edges of G can be partitioned into paths of length 2 if and only if G has an even number of edges.
12. For a graph G , let $o(G)$ denote the number of vertices with odd degree. If $G_1, G_2, \dots, G_k$ are the connected components of G , prove that $\sum_{i=1}^k o(G_i)$ is even.

2.4.4 Easy Problems: De Bruijn Sequences

13. Construct a binary De Bruijn sequence of order 3 (i.e., a circular sequence where every possible 3-length binary string appears exactly once as a substring).
14. How many distinct binary De Bruijn sequences of order 4 exist? Explain your reasoning.
15. For a ternary alphabet $\{0, 1, 2\}$, what is the length of a De Bruijn sequence of order 2? List all possible substrings that must appear exactly once.

2.4.5 Medium Problems: De Bruijn Sequences

16. Prove that for any alphabet of size k , a De Bruijn sequence of order n has length k^n .
17. Construct a De Bruijn sequence of order 2 for an alphabet of size 3. Then describe an algorithm to generate such sequences for any alphabet size.
18. Show how to construct a De Bruijn sequence of order $n + 1$ from a De Bruijn sequence of order n .
19. Given a binary De Bruijn sequence of order 4, describe how many different such sequences exist and explain an algorithm to generate them.

2.4.6 Hard Problems: De Bruijn Sequences

20. Prove that any De Bruijn sequence of order n for an alphabet of size k can be represented as an Eulerian circuit in a directed graph with k^{n-1} vertices.
21. Given a De Bruijn sequence of order n over alphabet $\{0, 1\}$, describe an algorithm to find the position of any n -length binary string within the sequence without examining the entire sequence.
22. Prove that for any binary De Bruijn sequence of order n , there exists a corresponding binary necklace of length $2^n - 1$.

2.4.7 Easy Problems: De Bruijn Graphs

23. Draw the De Bruijn Graph of order 1 for the binary alphabet $\{0, 1\}$. How many vertices and edges does it have?
24. For a ternary alphabet $\{0, 1, 2\}$, construct the De Bruijn Graph of order 1. How many vertices and edges does it have?
25. Draw the De Bruijn Graph of order 2 for the binary alphabet $\{0, 1\}$. Then trace an Eulerian circuit in this diagram that generates a De Bruijn sequence of order 3.
26. Draw a binary De Bruijn Graph of order 1 (with vertices 0 and 1). Then draw the corresponding De Bruijn Graph of order 2 using the line graph construction.
27. Draw a De Bruijn Graph for a ternary alphabet $\{0, 1, 2\}$ of order 1. Find an Eulerian circuit in this diagram and write down the corresponding De Bruijn sequence.

2.4.8 Medium Problems: De Bruijn Graphs

28. Prove that the De Bruijn Graph $B(k, n)$ has k^n vertices and k^{n+1} edges.
29. Consider a modified De Bruijn graph where certain transitions are forbidden. Specifically, in a binary De Bruijn graph of order 3, transitions to states containing "111" are prohibited. Is it still possible to find an Eulerian trail in this modified diagram? If so, construct one. If not, explain why.

2.4.9 Hard Problems: De Bruijn Graphs

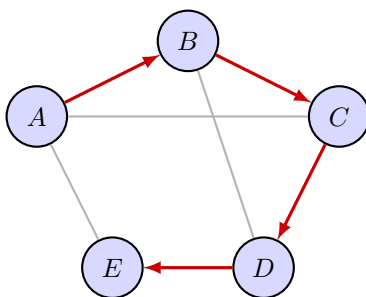
30. Let $B(k, n)$ be the De Bruijn graph of order n over an alphabet of size k . Prove that the line graph of $B(k, n)$ is isomorphic to $B(k, n + 1)$.
31. For a De Bruijn Graph $B(2, n)$, determine the chromatic number of the underlying undirected simple graph (ignore edge directions, delete the loops at $00 \cdots 0$ and $11 \cdots 1$, and merge parallel edges) and prove your answer.

3 Hamiltonian Graphs

3.1 Hamiltonian Walks, Paths, Cycles

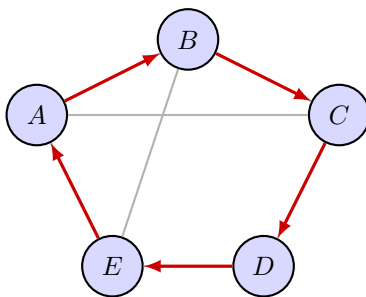
Definition 20 (Hamiltonian Cycle). A Hamiltonian cycle in a graph G is a cycle that visits each vertex of G exactly once (except for the starting vertex, which is also the ending vertex). Equivalently, a Hamiltonian cycle is a spanning cycle of the graph, meaning $V(C) = V(G)$. A graph is called Hamiltonian if it contains a Hamiltonian cycle.

Definition 21 (Hamiltonian Path). A Hamiltonian path in a graph G is a path that visits each vertex of G exactly once. Equivalently, a Hamiltonian path is a spanning path in the graph, meaning $V(P) = V(G)$. A graph is called traceable if it contains a Hamiltonian path.



Hamiltonian Path: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

Figure 43: Example of a Hamiltonian path. The red path visits every vertex of the graph exactly once, making it a Hamiltonian path. Unlike Eulerian paths which must traverse every edge exactly once, a Hamiltonian path must visit every vertex exactly once.



Hamiltonian Cycle: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$

Figure 44: Example of a Hamiltonian cycle. The red cycle visits every vertex of the graph exactly once before returning to the starting vertex, forming a closed loop.

Lemma 4 (Difference Between Hamiltonian and Eulerian). There exists no known simple necessary and sufficient condition for a graph to be Hamiltonian, unlike Eulerian graphs which can be characterized by the parity of vertex degrees.

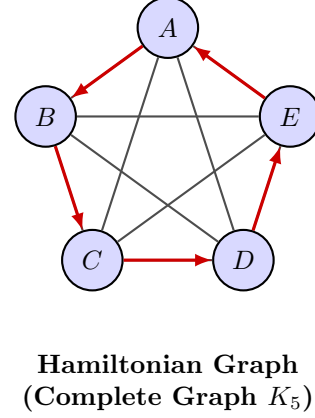
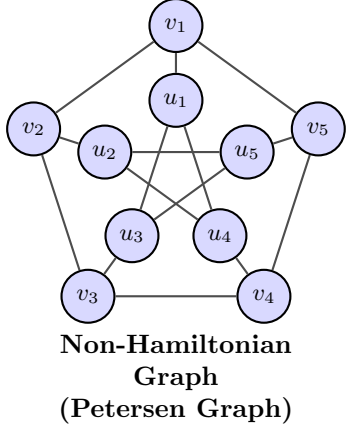


Figure 45: Comparison of a non-Hamiltonian graph (Petersen graph) and a Hamiltonian graph (complete graph K_5). The complete graph contains a Hamiltonian cycle (shown with red arrows) that visits every vertex exactly once before returning to the starting vertex.

Theorem 14. The Petersen graph is not Hamiltonian.

Proof. We present a proof by contradiction. Suppose that the Petersen graph contains a Hamiltonian cycle C .

Let us recall that the Petersen graph has 10 vertices and 15 edges, where each vertex has exactly 3 neighbors. The graph can be constructed as follows:

- Create an outer 5-cycle with vertices labeled $\{v_1, v_2, v_3, v_4, v_5\}$
- Create an inner 5-pointed star with vertices labeled $\{u_1, u_2, u_3, u_4, u_5\}$
- Connect each outer vertex v_i to the corresponding inner vertex u_i

Now, if a Hamiltonian cycle C exists, it must use exactly 10 edges of the graph to visit all 10 vertices exactly once. Since the Petersen graph has 15 edges in total, the cycle C omits exactly 5 edges.

Let us classify the edges of the Petersen graph into three types:

1. Type A: The 5 edges of the outer 5-cycle $\{v_1, v_2, v_3, v_4, v_5\}$
2. Type B: The 5 "spoke" edges connecting v_i to u_i for $i \in \{1, 2, 3, 4, 5\}$
3. Type C: The 5 edges of the inner star connecting u_i to $u_{i \pm 2}$ (with indices taken modulo 5)

For a Hamiltonian cycle to exist, it must use exactly 2 edges incident to each vertex (entering and exiting). Since each vertex has degree 3, exactly one edge at each vertex must be omitted from the cycle. This means that the 5 omitted edges form a perfect matching (or 1-factor) of the Petersen graph.

Now consider the Type B edges (the spokes). Deleting them separates the outer vertices from the inner vertices, so each time C passes from the outer 5-cycle to the inner star along a spoke, it must later return along another spoke. Hence C uses an even number of spokes, and it uses at least one, since otherwise it would never reach the inner vertices. So C uses exactly 2 or exactly 4 spokes (equivalently, the omitted perfect matching contains exactly 3 spokes or exactly 1 spoke).

Case 1: C uses exactly two spokes, $v_i u_i$ and $v_j u_j$. Each of the other three outer vertices does not use its spoke, so both of its outer edges lie in C , while v_i and v_j each have exactly one outer edge in C . Thus the outer edges of C are four of the five edges of the outer 5-cycle, forming a path through all five outer vertices with ends v_i and v_j . The omitted outer edge joins these two ends, so v_i and v_j are adjacent on the outer 5-cycle, i.e., $j \equiv i \pm 1 \pmod{5}$. The same argument applied to the inner vertices shows that u_i and u_j are adjacent in the inner star, i.e., $j \equiv i \pm 2 \pmod{5}$. These two conditions cannot both hold, a contradiction.

Case 2: C uses exactly four spokes; say $v_i u_i$ is the one it omits (indices modulo 5). Since v_i has degree 3 and its spoke is not used, both outer edges $v_{i-1} v_i$ and $v_i v_{i+1}$ lie in C ; likewise both inner edges $u_i u_{i+2}$ and

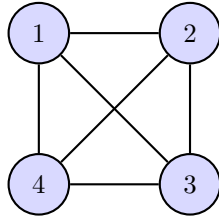
$u_i u_{i-2}$ lie in C . The vertex v_{i+1} already has two edges of C (namely $v_i v_{i+1}$ and its spoke), so $v_{i+1} v_{i+2} \notin C$. Hence v_{i+2} must use its other two edges, the spoke $v_{i+2} u_{i+2}$ and the outer edge $v_{i+2} v_{i+3}$. Since $i+3 \equiv i-2 \pmod{5}$, the cycle C now contains the five edges

$$v_{i+2} v_{i-2}, \quad v_{i-2} u_{i-2}, \quad u_{i-2} u_i, \quad u_i u_{i+2}, \quad u_{i+2} v_{i+2},$$

which form a cycle through only five vertices. A Hamiltonian cycle on 10 vertices cannot contain a shorter cycle, a contradiction.

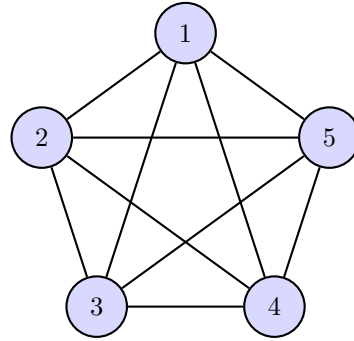
Both cases are impossible, so the Petersen graph cannot contain a Hamiltonian cycle. $\square$

Remark 8 (Complete Graphs). Recall that a complete graph K_n , introduced in Chapter 1, is a simple graph where every pair of vertices is connected by an edge. This property makes complete graphs particularly important for studying Hamiltonian cycles.



K_4 : Complete graph
on 4 vertices

Each vertex has degree 3 ($n-1$)
Total of 6 edges $\binom{4}{2}$



K_5 : Complete graph
on 5 vertices

Each vertex has degree 4 ($n-1$)
Total of 10 edges $\binom{5}{2}$

Figure 46: Examples of complete graphs K_4 and K_5 . In a complete graph, every pair of distinct vertices is connected by exactly one edge. Each vertex has degree $n-1$, and the total number of edges is $\binom{n}{2} = \frac{n(n-1)}{2}$.

Corollary 2. Every complete graph K_n with $n \geq 3$ is Hamiltonian.

Remark 9 (Applications of Hamiltonian Cycles in Complete Graphs). Complete graphs have numerous practical applications involving Hamiltonian cycles. In particular, the Traveling Salesperson Problem (TSP) can be modeled using complete graphs, where vertices represent cities and edges represent paths between them, with weights indicating distances. Finding a minimum-weight Hamiltonian cycle in this complete graph corresponds to finding the optimal route for the salesperson.

Theorem 15 (NP-Completeness of Hamiltonian Cycle Problem). The problem of determining whether a given graph contains a Hamiltonian cycle is NP-complete. However, for complete graphs, the existence of a Hamiltonian cycle is guaranteed for $n \geq 3$, though finding the minimum-weight Hamiltonian cycle remains NP-hard.

Example 3 (Number of Hamiltonian Cycles). The number of distinct Hamiltonian cycles in a complete graph K_n is $\frac{(n-1)!}{2}$. For example:

- K_3 has exactly 1 Hamiltonian cycle (up to equivalence)
- K_4 has exactly 3 Hamiltonian cycles
- K_5 has exactly 12 Hamiltonian cycles

- K_{10} has exactly 181,440 distinct Hamiltonian cycles

This rapid growth illustrates why brute-force approaches to the TSP quickly become impractical as the number of vertices increases.

3.2 Dirac's Theorem

Theorem 16 (Dirac, 1952). Let G be a simple graph with $n \geq 3$ vertices. If every vertex has degree at least $\frac{n}{2}$ (equivalently, if the minimum degree $\delta(G) \geq \frac{n}{2}$), then G contains a Hamiltonian cycle (i.e., G is Hamiltonian).

Proof. Suppose the theorem is false. So there exists an n -vertex graph G with minimum degree at least $\frac{n}{2}$ and no Hamiltonian cycle. Let G be a maximal counterexample to the theorem, i.e., with as many edges as possible. Note: K_n is Hamiltonian, so $G \neq K_n$. Thus there are non-adjacent vertices u and v . Adding an edge cannot decrease the minimum degree, so by the maximality of G , the graph $G + \{u, v\}$ is Hamiltonian. Since G is not Hamiltonian, every Hamiltonian cycle of $G + \{u, v\}$ uses the edge $\{u, v\}$, and deleting this edge leaves a Hamiltonian path $P = (x_1, x_2, \dots, x_n)$ of G from $x_1 = u$ to $x_n = v$. For a vertex $w \in V \setminus \{u, v\}$, let w^+ be the vertex after w on the path from u to v . Let $S = \{w^+ : w \in N_G(v)\}$. We want $S \cap N_G(u) \neq \emptyset$. Since u and v are not adjacent, $N_G(u)$ and $N_G(v)$ are subsets of $V \setminus \{u, v\}$. The map $w \mapsto w^+$ is one-to-one and never takes the value $u = x_1$, so $|S| = |N_G(v)|$ and $S \cup N_G(u) \subseteq V \setminus \{u\}$, which gives $|S \cup N_G(u)| \leq n - 1$. Since $|N_G(u)| \geq \frac{n}{2}$ and $|S| = |N_G(v)| \geq \frac{n}{2}$, we have:

$$\begin{aligned} |S \cap N_G(u)| &= |S| + |N_G(u)| - |S \cup N_G(u)| \\ &\geq \frac{n}{2} + \frac{n}{2} - (n - 1) \\ &= 1 \end{aligned}$$

Therefore, there is an index i with $x_{i+1} \in S \cap N_G(u)$; that is, $x_i \in N_G(v)$ and $x_{i+1} \in N_G(u)$. Now let C be the cycle $(x_1, x_2, \dots, x_i, x_n, x_{n-1}, \dots, x_{i+1}, x_1)$: it follows P from u to x_i , uses the edge $\{x_i, v\}$, follows P backwards from v to x_{i+1} , and returns to u along the edge $\{x_{i+1}, u\}$. This creates a Hamiltonian cycle in G , a contradiction. $\square$

Remark 10 (Algorithmic Verification). *Dirac's condition is particularly valuable because it can be verified in $O(n)$ time by simply checking the minimum degree of the graph, whereas determining if a general graph has a Hamiltonian cycle is NP-complete.*

Algorithm 7 Verify Dirac's Condition

Require: Graph $G = (V, E)$ with $n \geq 3$ vertices

- 1: `min_degree` $\leftarrow n$
 - 2: **for** each $v \in V$ **do**
 - 3: `min_degree` $\leftarrow \min(\text{min_degree}, |N(v)|)$
 - 4: **end for**
 - 5: `dirac_satisfied` $\leftarrow \text{min_degree} \geq \lceil n/2 \rceil$
 - 6: **return** `dirac_satisfied`
-

Remark 11 (Algorithmic Efficiency). *The algorithm's time complexity is $O(n)$ assuming the graph is represented with adjacency lists and degree information is readily available. This is significantly more efficient than directly testing for Hamiltonicity, which is an NP-complete problem.*

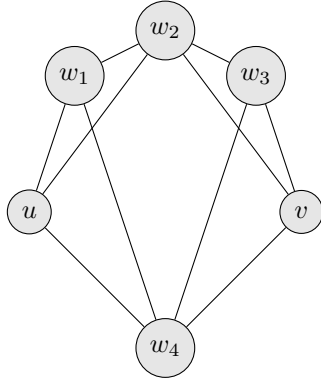
Remark 12 (Sufficiency vs. Necessity). *Dirac's condition is sufficient but not necessary for a graph to be Hamiltonian. Many graphs with minimum degree less than $n/2$ still contain Hamiltonian cycles.*

Theorem 17 (Disjoint Union Property). If G_1 and G_2 are two graphs that satisfy Dirac's condition individually, their disjoint union $G_1 \cup G_2$ does not necessarily satisfy Dirac's condition, highlighting that Hamiltonicity is not preserved under disjoint union.

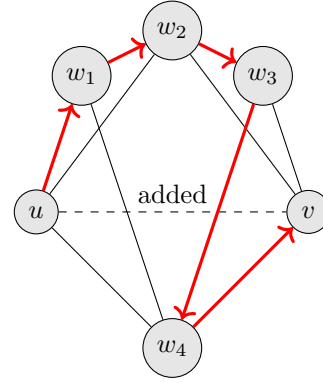
Remark 13 (Implementation Note). *When implementing this algorithm, special care should be taken with disconnected graphs, as no disconnected graph can have a Hamiltonian cycle regardless of minimum degree.*

Lemma 5 (Approximating Hamiltonicity). When $\delta(G) \geq \frac{n}{2} - \epsilon$ for small ϵ , even though Dirac's condition isn't satisfied, the graph may still contain a path or cycle covering most vertices.

3.3 Visual Illustration of Dirac's Theorem Proof

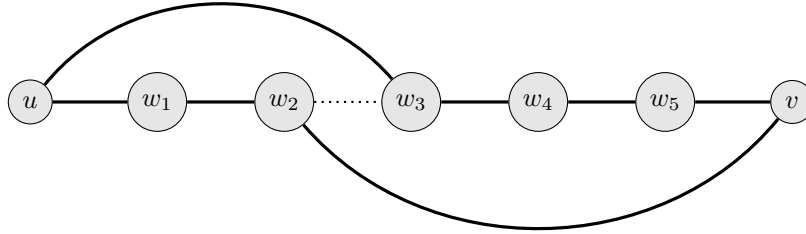


G (with $\{u, v\} \notin E(G)$)



$G + \{u, v\}$ with Hamiltonian path

Figure 47: Illustration of the graph closure concept for Hamiltonicity. Left: Original graph G where vertices u and v are not adjacent but have combined degree $\deg(u) + \deg(v) \geq n$. Right: After adding edge $\{u, v\}$ (shown as dashed), a Hamiltonian path (shown in red) exists from u to v . The Bondy-Chvátal theorem proves that if such vertices have sufficient combined degree, adding the edge between them preserves the Hamiltonian property.



Hamiltonian cycle $u, w_1, w_2, v, w_5, w_4, w_3, u$ in G

Figure 48: The key step in the proof of Dirac's Theorem. The path $u, w_1, \dots, w_5, v$ is a Hamiltonian path of G between the non-adjacent vertices u and v ; only the edges used in the argument are drawn. Since $\deg(u) + \deg(v) \geq n$, some neighbor of v is immediately followed on the path by a neighbor of u : here v is adjacent to w_2 and u is adjacent to $w_3 = w_2^+$. Deleting the path edge $\{w_2, w_3\}$ (dotted) and adding the edges $\{w_2, v\}$ and $\{u, w_3\}$ gives the Hamiltonian cycle $u, w_1, w_2, v, w_5, w_4, w_3, u$ (thick edges). A common neighbor w of u and v would not suffice: the closed walk that goes from u to w to v and then back along the path to u passes through w twice.

Theorem 18 (Closure Properties). Let G be a graph with n vertices and let $c(G)$ be its closure. Then:

1. G is Hamiltonian if and only if $c(G)$ is Hamiltonian.
2. If G satisfies Dirac's condition (minimum degree $\delta(G) \geq n/2$), then $c(G)$ is a complete graph.
3. The closure operation is uniquely defined, independent of the order in which edges are added.

Lemma 6. Let G be a simple graph and let u, v be non-adjacent vertices in G . If there exists a Hamiltonian path from u to v in G , then $G + \{u, v\}$ has a Hamiltonian cycle.

Lemma 7 (Crossing Edges). Let $P = (v_1, v_2, \dots, v_n)$ be a Hamiltonian path in a graph G with $n \geq 3$. If there is an index i with $1 \leq i \leq n-1$ such that $v_1 v_{i+1} \in E(G)$ and $v_i v_n \in E(G)$, then G has the Hamiltonian cycle $(v_1, \dots, v_i, v_n, v_{n-1}, \dots, v_{i+1}, v_1)$.

A common neighbor of v_1 and v_n is not enough: the path (v_1, v_2, v_3) has $N(v_1) \cap N(v_3) = \{v_2\}$ but no Hamiltonian cycle.

Corollary 3. If G has $n \geq 3$ vertices and a Hamiltonian path whose endpoints have degree sum at least n , then G has a Hamiltonian cycle.

Proof. Let $P = (v_1, v_2, \dots, v_n)$ be the path. The sets $\{i : v_1 v_{i+1} \in E(G)\}$ and $\{i : v_i v_n \in E(G)\}$ are subsets of $\{1, \dots, n-1\}$ with $\deg(v_1)$ and $\deg(v_n)$ elements, respectively. Since $\deg(v_1) + \deg(v_n) \geq n > n-1$, they have a common element i , and the Crossing Edges lemma gives a Hamiltonian cycle. $\square$

3.4 Alternative Form of Dirac's Theorem

Theorem 19 (Dirac, 1952). Let G be a simple graph with $n \geq 3$ vertices. If every vertex has degree at least $\frac{n}{2}$ (equivalently, if the minimum degree $\delta(G) \geq \frac{n}{2}$), then G contains a Hamiltonian cycle (i.e., G is Hamiltonian).

Proof Sketch. First, G is connected: otherwise a smallest component has at most $\frac{n}{2}$ vertices, so its vertices have degree at most $\frac{n}{2} - 1 < \frac{n}{2}$. Consider a longest path $P = (v_1, \dots, v_k)$. By the maximality of P , every neighbor of v_1 and every neighbor of v_k lies on P . So each of the at least $\frac{n}{2}$ neighbors of v_1 is the right end v_{i+1} of an edge $v_i v_{i+1}$ of P , and at least $\frac{n}{2}$ edges $v_i v_{i+1}$ of P have their left end v_i adjacent to v_k . Since P has only $k-1 < n$ edges, by the pigeonhole principle some edge $v_i v_{i+1}$ of P has both properties, i.e., $v_1 v_{i+1}, v_k v_i \in E(G)$. This creates the cycle $C = (v_1, \dots, v_i, v_k, \dots, v_{i+1}, v_1)$ through all vertices of P . If some vertex of G were not on C , then, since G is connected, some vertex $x \notin C$ would be adjacent to a vertex y of C , and x followed by the path around C starting at y would be a path longer than P , contradicting P 's maximality. Hence C is a Hamiltonian cycle. $\square$

Lemma 8. The degree condition in Dirac's theorem is tight, meaning that there exist graphs with minimum degree $\delta(G) = \frac{n}{2} - 1$ that are not Hamiltonian.

Proof. Let $n = 2k$ with $k \geq 2$, and let $G = K_{k-1, k+1}$. Every vertex of G has degree $k-1$ or $k+1$, so $\delta(G) = k-1 = \frac{n}{2} - 1$. A cycle in a bipartite graph alternates between the two partite sets, so it contains equally many vertices from each; a Hamiltonian cycle of G would therefore force $k-1 = k+1$, which is impossible. (Alternatively, the disjoint union of two copies of K_k has minimum degree $k-1 = \frac{n}{2} - 1$ and, being disconnected, is not Hamiltonian.) $\square$

Example 4 (Tightness of Dirac's Condition). Consider the complete bipartite graph $K_{m, m+1}$ with $m \geq 2$. This graph has $n = 2m+1$ vertices, and each vertex has degree either m or $m+1$. The minimum degree is $\delta(G) = m = \frac{n-1}{2} = \frac{n}{2} - \frac{1}{2}$, which is just below Dirac's threshold. This graph cannot contain a Hamiltonian cycle because any cycle must alternate between the two partite sets, which is impossible when the sets have different sizes.

Remark 14 (Applications). *Dirac's Theorem provides a simple sufficient condition for Hamiltonicity that can be verified in linear time, making it valuable in network design applications where robustness requires the existence of a cycle visiting all nodes.*

Remark 15 (Related Results). *Dirac's Theorem was generalized by Ore (1960), who showed that a graph with $n \geq 3$ vertices is Hamiltonian if, for every pair of non-adjacent vertices u and v , we have $\deg(u) + \deg(v) \geq n$. This is a strictly stronger result, as there exist graphs that satisfy Ore's condition but not Dirac's.*

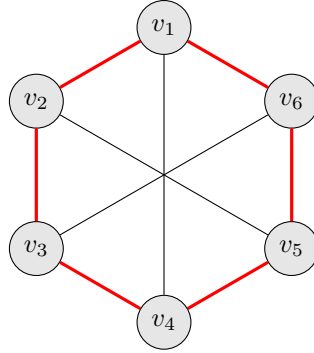


Figure 49: A 3-regular graph on 6 vertices that satisfies Dirac's condition. Each vertex has degree 3, which equals $n/2$. The red cycle highlights one of the Hamiltonian cycles in this graph. This example demonstrates that cubic (3-regular) graphs can be Hamiltonian when they meet the minimum degree threshold.

3.5 Examples of Dirac's Theorem

Example 5 (Minimum degree exactly $\frac{n}{2}$). Consider a graph on $n = 6$ vertices where each vertex has degree exactly $\frac{n}{2} = 3$.

Lemma 9 (Cubic Graph Hamiltonicity). Not all cubic graphs are Hamiltonian. The Petersen graph is a well-known 3-regular graph with 10 vertices that has no Hamiltonian cycle, demonstrating that regularity alone is insufficient for Hamiltonicity.

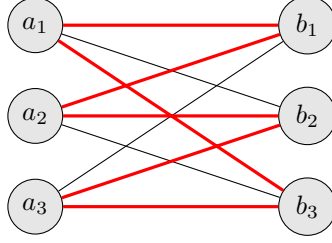


Figure 50: The complete bipartite graph $K_{3,3}$ with $n = 6$ vertices. Each vertex has degree exactly 3, which equals $\frac{n}{2}$, satisfying Dirac's condition. A Hamiltonian cycle is highlighted in red: $a_1 - b_1 - a_2 - b_2 - a_3 - b_3 - a_1$. This example illustrates that bipartite graphs can be Hamiltonian when they have equal-sized partitions.

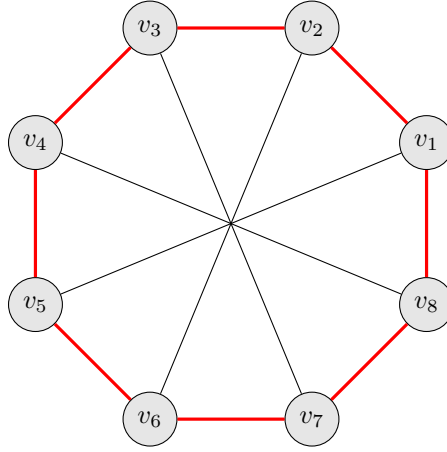


Figure 51: A graph on $n = 8$ vertices (the Wagner graph, also called the Möbius ladder on 8 vertices) in which each vertex has degree exactly 3, one less than $\frac{n}{2} = 4$; so this graph does *not* satisfy Dirac's minimum degree condition. Nevertheless, it contains a Hamiltonian cycle (highlighted in red) along the outer cycle, which shows once more that Dirac's condition is sufficient but not necessary. Connecting each vertex of a regular n -gon to its opposite vertex always produces a 3-regular graph, so this construction reaches Dirac's threshold $\frac{n}{2}$ only for $n \leq 6$ (for $n = 6$ it gives $K_{3,3}$, as in Figure 49).

Definition 22 (Circulant Graph). A circulant graph $\text{Circ}(n; S)$ is a graph with vertex set $\{0, 1, 2, \dots, n-1\}$ and an edge between vertices i and j if and only if $i - j \equiv \pm s \pmod{n}$ for some s in the connection set $S \subseteq \{1, 2, \dots, \lfloor n/2 \rfloor\}$. In other words, it's a graph where vertices are arranged in a circle, and each vertex is connected to specific vertices at fixed distances around the circle.

Definition 23 (Graph Toughness). The toughness $t(G)$ of a graph G is defined as:

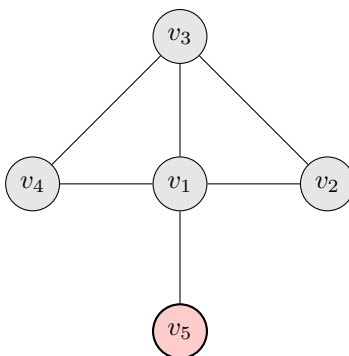
$$t(G) = \min \left\{ \frac{|S|}{c(G-S)} : S \subset V(G), c(G-S) > 1 \right\}$$

where $c(G-S)$ is the number of connected components in the graph after removing vertex set S . Intuitively, toughness measures how difficult it is to disconnect a graph by removing vertices. (A complete graph has no such set S ; by convention $t(K_n) = \infty$.)

Theorem 20 (Circulant Graphs and Hamiltonicity). The graph shown in Figure 51 is an example of a circulant graph $\text{Circ}(8; \{1, 4\})$. A circulant graph with $n \geq 3$ vertices is Hamiltonian if and only if it is connected.

Problem 1. Show that in the graph of Figure 51, there are exactly 5 distinct Hamiltonian cycles. Hint: Count the cycles that use only the outer edges versus those that must use the "diagonal" connections.

Lemma 10 (Toughness of Regular Graphs). The graph in Figure 51 has toughness exactly $\frac{4}{3}$: removing $S = \{v_1, v_2, v_4, v_7\}$ leaves the three components $\{v_3\}$, $\{v_8\}$, and $\{v_5, v_6\}$, and no vertex set gives a smaller ratio. In general, toughness is bounded *above* by the degrees: if G is not complete and v is a vertex of minimum degree, then removing the $\delta(G)$ neighbors of v leaves v isolated together with at least one other component, so $t(G) \leq \frac{\delta(G)}{2}$. In particular, a non-complete k -regular graph has toughness at most $\frac{k}{2}$, and it can be much smaller (for example, a disconnected k -regular graph has toughness 0).



$$\deg(v_5) = 1 < \frac{n}{2} = 2.5$$

Figure 52: A graph that fails to satisfy Dirac's condition. This graph has $n = 5$ vertices, with vertex v_5 (highlighted in red) having degree 1, which is less than $\frac{n}{2} = 2.5$. The graph cannot contain a Hamiltonian cycle because any cycle including v_5 must use both its only edge (to v_1) as entry and exit, making it impossible to continue the cycle. This demonstrates why the minimum degree requirement in Dirac's theorem is necessary.

Lemma 11 (Connectivity Requirement). If a graph G has a Hamiltonian cycle, then G is 2-connected.

Proof. Suppose G has a Hamiltonian cycle C . If we remove any vertex v from G , the cycle C becomes a path that connects all remaining vertices. Therefore, $G - v$ is connected, meaning G is 2-connected. $\square$

Theorem 21 (Pendant Vertices). Any graph G containing a pendant vertex cannot contain a Hamiltonian cycle.

Proof. Let v be a pendant vertex in G with its only neighbor u . Any cycle containing v must use the edge $\{u, v\}$ twice: once to enter v and once to exit. Since a cycle can use each edge at most once, this creates a contradiction. $\square$

Lemma 12 (Cut Vertices). The vertex v_1 in Figure 52 is a cut vertex; hence no Hamiltonian cycle can exist in this graph.

Proof. Removing v_1 leaves two components, the path $v_2v_3v_4$ and the isolated vertex v_5 . By the Connectivity Requirement lemma above, a graph with a Hamiltonian cycle is 2-connected and so has no cut vertex; hence no Hamiltonian cycle can exist in this graph. $\square$

Theorem 22 (Relaxed Degree Condition). Let G be a simple graph with $n \geq 3$ vertices. If $\delta(G) \geq \frac{n-1}{2}$, then G is Hamiltonian unless $n = 2m + 1$ is odd and G is one of the following graphs:

1. two copies of K_{m+1} sharing exactly one vertex; or

2. a graph whose vertex set is partitioned into an independent set I of $m + 1$ vertices and a set J of m vertices, with every vertex of I adjacent to every vertex of J (the edges inside J being arbitrary); for example, $K_{m,m+1}$.

Remark 16 (Wheel-Graph Relation). *The graph in Figure 52 can be viewed as the wheel graph W_4 (the hub v_1 joined to every vertex of the 4-cycle $v_2v_3v_4v_5$) with the two rim edges $\{v_4, v_5\}$ and $\{v_5, v_2\}$ removed. Removing only one rim edge would not suffice, since W_4 minus a single rim edge is still Hamiltonian; removing both leaves v_5 as a pendant vertex, which destroys the Hamiltonian property.*

Remark 17 (Dirac's Condition Is Not Necessary). *Adding the edge $\{v_2, v_5\}$ makes the graph Hamiltonian (for example, $v_1, v_4, v_3, v_2, v_5, v_1$ is a Hamiltonian cycle), even though the minimum degree then rises only to $2 < \frac{n}{2}$.*

Problem 2. Determine which pairs of edges can be added to Figure 52 to meet Dirac's condition, and list all such possible pairs.

Lemma 13 (Dirac's Condition and Pendant Vertices). Let G be the graph in Figure 52 with a pendant vertex v_5 of degree 1. Then:

1. G fails Dirac's condition since $\deg(v_5) = 1 < \frac{n}{2} = 2.5$
2. G cannot contain a Hamiltonian cycle
3. Adding the edge $\{v_2, v_5\}$ would increase $\deg(v_5)$ to 2, but this still falls short of Dirac's threshold
4. G requires at least two additional edges to satisfy Dirac's condition

Proof. A Hamiltonian cycle requires each vertex to have degree at least 2, which v_5 fails to satisfy. Furthermore, v_1 is a cut vertex, dividing the graph into two components upon its removal, which prevents any Hamiltonian cycle from existing. $\square$

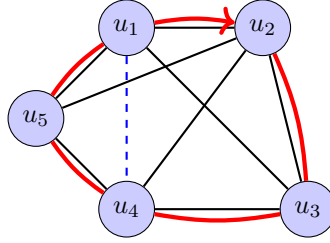
3.6 Related Results and Extensions

3.6.1 Ore's Theorem

Theorem 23 (Ore's Theorem, 1960). Let G be a simple graph with $n \geq 3$ vertices. If for every pair of non-adjacent vertices u and v in G ,

$$\deg(u) + \deg(v) \geq n,$$

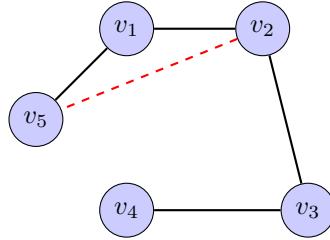
then G is Hamiltonian.



$$\deg(u_1) = 3, \deg(u_2) = 4, \deg(u_3) = 3, \deg(u_4) = 3, \deg(u_5) = 3$$

For non-adjacent vertices u_1 and u_4 : $\deg(u_1) + \deg(u_4) = 3 + 3 = 6 > 5 = n$

Figure 53: A graph with $n = 5$ vertices satisfying Ore's condition. The red path shows a Hamiltonian cycle, and the blue dashed line indicates a non-adjacent vertex pair whose degree sum exceeds n .



$$\deg(v_1) = 2, \deg(v_2) = 2, \deg(v_3) = 2, \deg(v_4) = 1, \deg(v_5) = 1$$

For non-adjacent vertices v_2 and v_5 : $\deg(v_2) + \deg(v_5) = 2 + 1 = 3 < 5 = n$

No Hamiltonian cycle exists in this graph

Figure 54: A graph with $n = 5$ vertices that fails both Dirac's and Ore's conditions. The red dashed line shows a non-adjacent vertex pair whose degree sum is less than n .

Lemma 14 (Ore's Condition and Path Graphs). Let P_n be a path graph with $n \geq 4$ vertices. Then:

1. For any non-adjacent vertices u and v in P_n , $\deg(u) + \deg(v) < n$
2. P_n fails to satisfy Ore's condition
3. The graph in Figure 54 is itself the path $P_5 = (v_5, v_1, v_2, v_3, v_4)$; in particular, it contains P_4 as a subgraph, e.g., (v_1, v_2, v_3, v_4)
4. Adding any single edge to Figure 54 is insufficient to satisfy Ore's condition

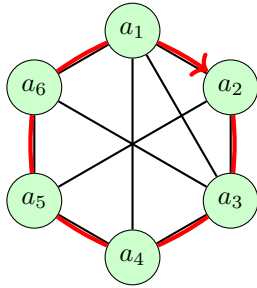
Proof. In any path graph P_n the maximum degree of any vertex is 2, and the two end vertices have degree 1. If $n \geq 5$, any two non-adjacent vertices have degree sum at most $2 + 2 = 4 < n$. If $n = 4$, the two vertices of degree 2 are adjacent, so every non-adjacent pair contains an end vertex and has degree sum at most $1 + 2 = 3 < 4$. Thus P_n always fails Ore's condition. (Failing Ore's condition does not by itself rule out a Hamiltonian cycle, since the condition is only sufficient; P_n has no Hamiltonian cycle because its end vertices have degree 1.) The graph in Figure 54 is the path $(v_5, v_1, v_2, v_3, v_4)$, with v_4 and v_5 of degree only 1. If a single edge is added and it does not join v_4 and v_5 , then one of v_4, v_5 still has degree 1 and is non-adjacent to three vertices, each of degree at most 3, so some non-adjacent pair has degree sum at most $4 < 5$; if the new edge joins v_4 and v_5 , the result is C_5 , whose non-adjacent pairs have degree sum $4 < 5$. $\square$

3.6.2 Chvátal's Theorem

Theorem 24 (Chvátal, 1972). Let G be a simple graph with $n \geq 3$ vertices and degree sequence $d_1 \leq d_2 \leq \dots \leq d_n$. If for all $i < \frac{n}{2}$,

$$d_i \leq i \implies d_{n-i} \geq n - i,$$

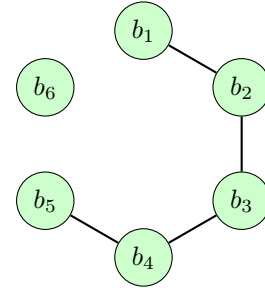
then G is Hamiltonian.



Example: Satisfies Chvátal's condition

$\deg(a_1) = 4, \deg(a_2) = 3, \deg(a_3) = 4,$
 $\deg(a_4) = 3, \deg(a_5) = 3, \deg(a_6) = 3$
 Degree sequence: 3, 3, 3, 3, 4, 4

i	$d_i \leq i?$	If yes, $d_{n-i} \geq n - i?$
1	$3 > 1$	Not needed
2	$3 > 2$	Not needed



Counter-example: Fails Chvátal's condition

$\deg(b_1) = 1, \deg(b_2) = 2, \deg(b_3) = 2,$
 $\deg(b_4) = 2, \deg(b_5) = 1, \deg(b_6) = 0$
 Degree sequence: 0, 1, 1, 2, 2, 2

i	$d_i \leq i?$	If yes, $d_{n-i} \geq n - i?$
1	$0 < 1$	$d_5 = 2 < 5$ (Fails)
2	$1 < 2$	$d_4 = 2 < 4$ (Fails)

Figure 55: Two examples illustrating Chvátal's condition: the left graph satisfies the condition and contains a Hamiltonian cycle (shown in red), while the right graph fails the condition and has no Hamiltonian cycle.

Lemma 15 (Bondy-Chvátal Closure). Let G be a graph with n vertices. The *Bondy-Chvátal closure* (or simply the *closure*) of G , denoted $cl(G)$, is obtained by repeatedly adding edges between non-adjacent vertices u and v whenever $\deg(u) + \deg(v) \geq n$. This process is continued until no more edges can be added.

Corollary 4 (Bondy-Chvátal Theorem). A graph G with n vertices is Hamiltonian if and only if its closure $cl(G)$ is Hamiltonian.

Proof. The key insight is that adding an edge between vertices u and v where $\deg(u) + \deg(v) \geq n$ preserves the Hamiltonian property. That is, G has a Hamiltonian cycle if and only if $G + uv$ has a Hamiltonian cycle. By repeatedly applying this operation until no more edges can be added, we obtain $cl(G)$, which is Hamiltonian if and only if G is Hamiltonian. $\square$

Corollary 5 (Dirac's Theorem). Let G be a simple graph with $n \geq 3$ vertices. If $\delta(G) \geq \frac{n}{2}$, where $\delta(G)$ denotes the minimum degree of G , then G is Hamiltonian.

Proof. Suppose $\delta(G) \geq \frac{n}{2}$. Then for every vertex $v \in G$, we have $\deg(v) \geq \frac{n}{2}$. Consider the degree sequence $d_1 \leq d_2 \leq \dots \leq d_n$. Since the minimum degree $d_1 \geq \frac{n}{2} > i$ for all $i < \frac{n}{2}$, the condition in Chvátal's theorem is satisfied trivially (the premise is false). Therefore, G is Hamiltonian. $\square$

Lemma 16 (Cycle Extension). Let C be a cycle in a graph G . If there exists a vertex $v \notin C$ that is adjacent to two consecutive vertices of C , then there exists a cycle C' in G with $V(C') = V(C) \cup \{v\}$.

Proof. Label C as $v_1, v_2, \dots, v_k, v_1$ so that the two consecutive neighbors of v on C are v_k and v_1 . Then $C' = v_1, v_2, \dots, v_k, v, v_1$ is a cycle which includes all vertices of C plus v . The hypothesis that the two neighbors of v are consecutive on C cannot be dropped: if $C = v_1 v_2 v_3 v_4 v_1$ and v is adjacent only to v_1 and v_3 , the resulting graph is $K_{2,3}$, which has no cycle through all five vertices. $\square$

Corollary 6 (Fan's Theorem). Let G be a 2-connected graph with n vertices. If $\max\{\deg(u), \deg(v)\} \geq \frac{n}{2}$ for every pair of vertices u and v with distance $d(u, v) = 2$, then G is Hamiltonian.

Lemma 17 (Pancyclicity). If G is a graph with n vertices and contains a Hamiltonian cycle, and if $\deg(u) + \deg(v) \geq n + 1$ for every pair of non-adjacent vertices u and v , then G is pancyclic (i.e., contains cycles of all lengths from 3 to n).

Corollary 7 (Hamiltonian-Connected). If G is a graph with n vertices and $\delta(G) \geq \frac{n+1}{2}$, then G is Hamiltonian-connected (i.e., for any two vertices u and v , there exists a Hamiltonian path from u to v).

Remark 18. *These sufficient conditions create a hierarchy:*

$$\text{Dirac} \Rightarrow \text{Ore} \Rightarrow \text{Chvátal}$$

That is, Dirac's condition implies Ore's condition, which in turn implies Chvátal's condition.

Remark 19 (Sharpness). *All of these conditions are sharp, in the sense that they cannot be improved. For each theorem, there exist non-Hamiltonian graphs that almost satisfy the given condition but fail by the smallest possible margin.*

Remark 20 (Computational Complexity). *Despite these sufficient conditions, determining whether an arbitrary graph is Hamiltonian remains NP-complete. These theorems provide easily verifiable sufficient conditions but not necessary conditions for Hamiltonicity.*

Remark 21 (Closure Algorithm). *The Bondy-Chvátal closure can be computed in $O(n^3)$ time, where n is the number of vertices, by repeatedly checking all pairs of non-adjacent vertices and adding edges when the degree sum condition is met.*

Theorem 25 (Necessary Conditions). Let G be a graph. If G contains a Hamiltonian cycle, then the following must be true:

1. G is connected.
2. G is 2-connected (has no cut vertices).
3. G is 1-tough. That is, for any nonempty set S of vertices, removing S from G results in at most $|S|$ connected components.
4. If G is bipartite with partite sets of sizes a and b , then $a = b$.
5. No vertex in G has degree less than 2.

Remark 22. *While these conditions are necessary, they are not sufficient. There exist graphs satisfying all these conditions that still do not contain a Hamiltonian cycle.*

Corollary 8. A vertex of degree 1 cannot be part of any Hamiltonian cycle.

Definition 24. The **connectivity** of a graph G , denoted $\kappa(G)$, is the minimum number of vertices whose removal disconnects the graph. If G is a complete graph K_n , then $\kappa(G) = n - 1$.

Definition 25. The **independence number** of a graph G , denoted $\alpha(G)$, is the size of the largest independent set in G , i.e., the maximum number of vertices such that no two are adjacent.

Theorem 26 (Chvátal-Erdős Theorem). Let G be a graph with $n \geq 3$ vertices. If the connectivity $\kappa(G)$ and independence number $\alpha(G)$ satisfy $\kappa(G) \geq \alpha(G)$, then G contains a Hamiltonian cycle.

Sketch. If $\alpha(G) = 1$, then G is complete and hence Hamiltonian, so assume $\alpha(G) \geq 2$. Then $\kappa(G) \geq 2$, so every vertex has degree at least 2 and G contains a cycle. Let C be a longest cycle of G , oriented in one direction, and suppose some vertex lies outside C . Let D be a component of $G - V(C)$, and let U be the set of vertices of C that have a neighbor in D . For $x \in U$ let x^+ be the successor of x on C , and let $U^+ = \{x^+ : x \in U\}$. By the maximality of C , no vertex of U^+ lies in U , and no two vertices of U^+ are adjacent (otherwise C could be rerouted through D to give a longer cycle). Hence, for any vertex $d \in D$, the set $U^+ \cup \{d\}$ is independent, so $\alpha(G) \geq |U| + 1$. On the other hand, U separates D from U^+ , so $|U| \geq \kappa(G)$. Therefore $\alpha(G) \geq \kappa(G) + 1$, contradicting $\kappa(G) \geq \alpha(G)$. Hence C is a Hamiltonian cycle. $\square$

Remark 23. Chvátal and Erdős also showed that if $\kappa(G) > \alpha(G)$, then G is Hamiltonian-connected. Jackson and Ordaz (1990) conjectured that $\kappa(G) > \alpha(G)$ even implies that G is pancyclic (contains cycles of all lengths from 3 to n); Letzter (2023) proved this conjecture for all sufficiently large graphs.

Remark 24. Intuitively, the Chvátal-Erdős theorem states that if a graph is "highly connected" relative to how many vertices can be independent, then it must contain a Hamiltonian cycle. High connectivity prevents the graph from being disconnected too easily, while a low independence number ensures sufficient edge density.

Remark 25. This theorem is particularly useful because it can identify Hamiltonian graphs that might not satisfy other conditions like those of Dirac or Ore. For example, the 5-cycle C_5 has minimum degree $2 < \frac{5}{2}$, and every pair of non-adjacent vertices has degree sum $4 < 5$, so neither Dirac's nor Ore's condition holds. However, $\kappa(C_5) = 2$ and $\alpha(C_5) = 2$, so C_5 satisfies the Chvátal-Erdős condition. (By contrast, $K_{n,n}$, with $2n$ vertices and minimum degree n , already satisfies Dirac's condition.)

Remark 26. The lack of simple necessary and sufficient conditions for Hamiltonicity is connected to the computational complexity of the problem. The Hamiltonian cycle problem is NP-complete, which suggests that no easily verifiable characterization exists unless $P = NP$.

Theorem 27 (Hierarchy of Sufficient Conditions). The following implications hold for sufficient conditions of Hamiltonicity:

$$\text{Dirac} \Rightarrow \text{Ore} \Rightarrow \text{Chvátal}, \quad \text{Ore} \Rightarrow \text{Chvátal-Erdős}.$$

The last implication is due to Bondy (1978). Neither of the Chvátal and Chvátal-Erdős conditions implies the other: C_5 has $\kappa(C_5) = \alpha(C_5) = 2$, but its degree sequence $2, 2, 2, 2, 2$ violates Chvátal's condition at $i = 2$; conversely, the graph obtained from a triangle abc by adding vertices x, y, z with x adjacent to a, b , y adjacent to b, c , and z adjacent to c, a has degree sequence $2, 2, 2, 4, 4$, which satisfies Chvátal's condition, while $\kappa = 2 < 3 = \alpha$.

Remark 27. While there are many sufficient conditions for Hamiltonicity, finding a complete characterization remains an open problem in graph theory.

3.7 Traveling Salesman Problem

Definition 26 (NP-hard). A problem X is NP-hard if every problem $Y \in \mathbf{NP}$ can be reduced to X in polynomial time. Formally, for all $Y \in \mathbf{NP}$, $Y \leq_p X$, where $\leq_p$ denotes polynomial-time reduction.

In graph-theoretic context, a graph problem X is NP-hard if there exists a polynomial-time reduction from every problem in $\mathbf{NP}$ to X . This means that if we could solve X efficiently, we could solve all problems in $\mathbf{NP}$ efficiently.

Theorem 28. Many fundamental graph problems are NP-hard, including:

1. Determining if a graph contains a Hamiltonian cycle
2. Finding the minimum number of colors needed for proper vertex coloring

3. Finding the maximum clique in a graph
4. Computing the minimum vertex cover

Definition 27 (NP-complete). A problem X is *NP-complete* if:

1. $X \in \mathbf{NP}$, i.e., solutions to X can be verified in polynomial time, and
2. X is NP-hard, i.e., every problem in $\mathbf{NP}$ can be reduced to X in polynomial time.

A graph problem is NP-complete if it is both in $\mathbf{NP}$ and NP-hard. This means that a solution can be verified efficiently, but finding such a solution is believed to require exponential time in the worst case.

Remark 28. *The distinction between NP-hard and NP-complete is that NP-complete problems must be in $\mathbf{NP}$, while NP-hard problems may or may not be in $\mathbf{NP}$. For example, the graph isomorphism problem is in $\mathbf{NP}$, while the halting problem is NP-hard but not in $\mathbf{NP}$ (it is not even decidable). Likewise, computing a minimum-cost Hamiltonian cycle (the optimization version of the TSP) is NP-hard, but it is not a decision problem and so does not belong to $\mathbf{NP}$.*

Corollary 9 (Hamiltonian Path vs. Cycle). The Hamiltonian path problem and Hamiltonian cycle problem are polynomially equivalent; either both are in $\mathbf{P}$ or both are NP-complete. In fact, both are NP-complete.

Theorem 29 (Traveling Salesman Problem (TSP)). The **Traveling Salesman Problem (TSP)** asks for a minimum cost Hamiltonian cycle in a weighted graph. This problem is computationally hard: it is NP-hard, so it has no polynomial-time algorithm unless $\mathbf{P} = \mathbf{NP}$, and the fastest known exact algorithms on an n -vertex graph, such as the Held-Karp algorithm below, run in exponential time.

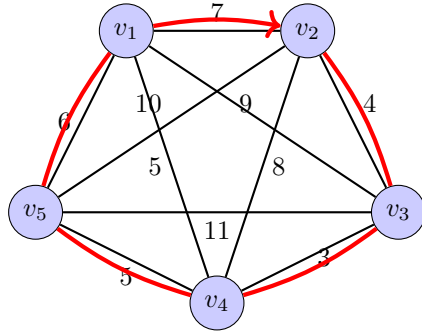


Figure 56: Complete graph with edge weights. The red arrows indicate an optimal TSP tour with cost $7 + 4 + 3 + 5 + 6 = 25$.

$$\text{Minimize } \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (23)$$

$$\text{Subject to } \sum_{j=1}^n x_{ij} = 1 \quad \forall i \in \{1, 2, \dots, n\} \quad (24)$$

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j \in \{1, 2, \dots, n\} \quad (25)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad \forall S \subset \{1, 2, \dots, n\}, 2 \leq |S| < n \quad (26)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in \{1, 2, \dots, n\} \quad (27)$$

where $x_{ij} = 1$ if the tour includes the edge from vertex i to vertex j , and 0 otherwise.

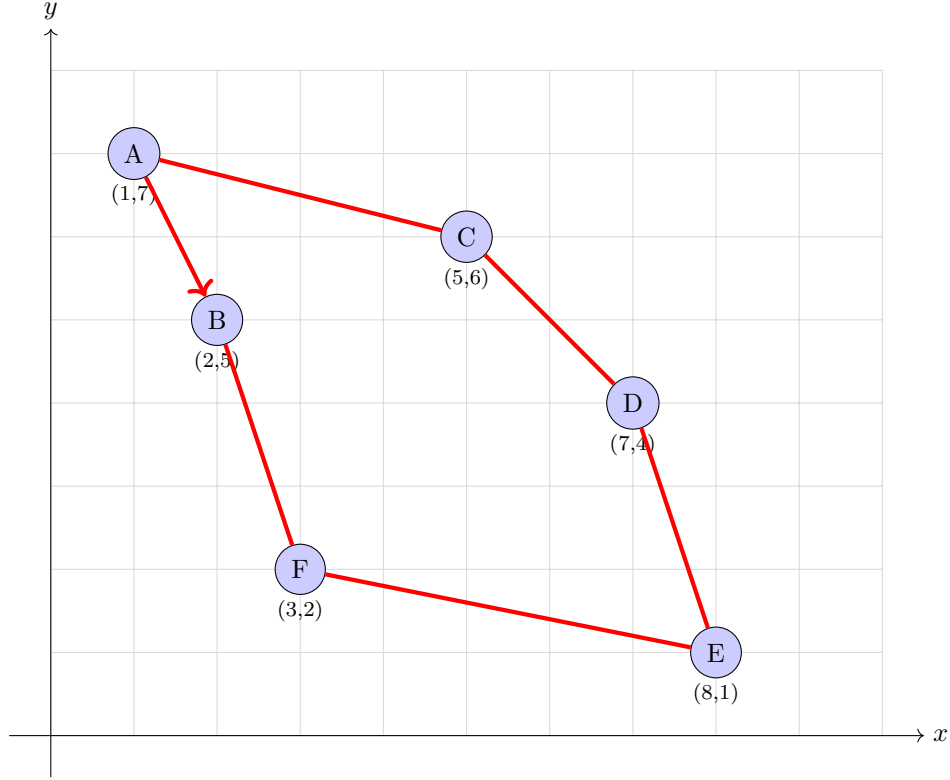


Figure 57: Euclidean TSP instance where cities are points in the plane and distances are Euclidean. The red path shows an optimal tour.

Corollary 10. The Traveling Salesman Problem is a direct extension of the Hamiltonian cycle problem, which is NP-complete.

Proof. The Hamiltonian cycle problem can be reduced to TSP by setting all edge weights to 1 for existing edges and ∞ for non-existing edges. Finding the minimum weight Hamiltonian cycle would then be equivalent to determining if a Hamiltonian cycle exists. $\square$

3.7.1 Approximation Algorithms

Theorem 30 (2-Approximation using MST). For metric TSP (where edge weights satisfy the triangle inequality), a 2-approximation can be obtained by:

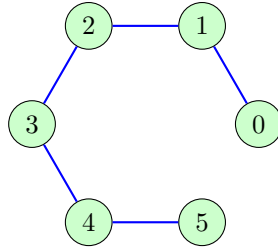
1. Finding a minimum spanning tree T of the graph
2. Performing a preorder traversal of T to get a tour

This tour has cost at most twice the optimal tour.

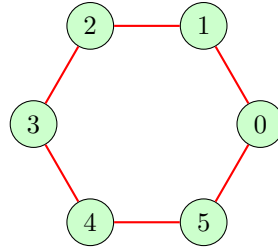
Corollary 11. The Traveling Salesman Problem is a direct extension of the Hamiltonian cycle problem, which is NP-complete. Thus, TSP is also NP-hard.

Lemma 18 (MST Lower Bound). For any graph G with metric distances, the cost of a minimum spanning tree $MST(G)$ is a lower bound on the cost of an optimal TSP tour $OPT(G)$.

Proof. Let T^* be an optimal TSP tour of G . Removing any single edge from T^* produces a spanning path, which is a spanning tree. Since $MST(G)$ is the minimum-cost spanning tree, we have $c(MST(G)) \leq c(T^* - e) \leq c(T^*) = c(OPT(G))$, since edge costs are non-negative. $\square$



Minimum
Spanning Tree



Approximated
TSP Tour

Relationship

$$\begin{aligned} c(MST) &\leq c(OPT) \\ c(APPROX) &\leq 2 \cdot c(MST) \\ \Rightarrow c(APPROX) &\leq 2 \cdot c(OPT) \end{aligned}$$

Figure 58: 2-approximation algorithm for TSP using Minimum Spanning Tree. **Left:** A minimum spanning tree; here it is the path 0, 1, 2, 3, 4, 5 with $n - 1 = 5$ edges (like every spanning tree, it contains no cycle). **Right:** The approximated TSP tour visits the vertices in the order of a preorder traversal of the MST starting at vertex 0, namely 0, 1, 2, 3, 4, 5, and then returns to 0; in this example the tour is the MST plus the single edge $\{5, 0\}$. The approximation guarantee works because (1) the MST cost is a lower bound on the optimal TSP tour cost since removing any edge from a tour produces a spanning tree, and (2) doubling the MST and shortcutting (via triangle inequality) produces a tour of cost at most twice optimal.

Theorem 31 (Christofides' Algorithm). For metric TSP, Christofides' algorithm achieves a $\frac{3}{2}$ -approximation by:

1. Computing an MST of the graph
2. Finding a minimum-weight perfect matching of odd-degree vertices in the MST
3. Combining the MST and matching to form an Eulerian graph
4. Converting the Eulerian tour to a Hamiltonian cycle using shortcuts

Corollary 12 (Approximation Hierarchy). For metric TSP, the following bounds hold:

$$c(MST) \leq c(OPT) \leq c(Christofides) \leq \frac{3}{2} c(OPT), \quad (28)$$

$$c(OPT) \leq c(MST\text{-preorder}) \leq 2 c(MST) \leq 2 c(OPT). \quad (29)$$

In particular, $1 \leq \frac{c(Christofides)}{c(OPT)} \leq \frac{3}{2}$ and $1 \leq \frac{c(MST\text{-preorder})}{c(OPT)} \leq 2$. These guarantees are relative to $c(OPT)$, not to $c(MST)$: since $c(OPT)$ can be as large as $2 c(MST)$, the ratio $\frac{c(Christofides)}{c(MST)}$ can be as large as 2.

Remark 29. For more than four decades after Christofides' algorithm (1976; found independently by Serdyukov), $\frac{3}{2}$ was the best known approximation ratio for metric TSP, despite significant research efforts. In 2020, Karlin, Klein, and Oveis Gharan gave a randomized $(\frac{3}{2} - \epsilon)$ -approximation algorithm for some $\epsilon > 10^{-36}$; the improvement is tiny, but it shows that $\frac{3}{2}$ is not the limit.

Remark 30. For the Euclidean TSP (where cities are points in the plane and distances are Euclidean), a polynomial-time approximation scheme (PTAS) exists, allowing approximation to within $(1 + \epsilon)$ of optimal for any $\epsilon > 0$.

Lemma 19 (Triangle Inequality). The triangle inequality is essential for the 2-approximation guarantee. Without it, TSP cannot be approximated within any constant factor unless $P = NP$.

Remark 31 (Special Case). The diagram shows a special case where the MST is a path, so a preorder traversal starting at an end of the path visits the vertices in path order, and the tour is the MST plus the single closing edge. In general, the MST has $n - 1$ edges and may branch; listing its vertices in preorder and then returning to the start amounts to walking around the tree (using each tree edge twice) while skipping vertices that have already been visited (shortcutting), which yields a valid Hamiltonian cycle.

Theorem 32 (Tight Example). The factor-2 approximation of the MST-based algorithm is tight: there exist graphs where the approximation produces a tour whose cost is arbitrarily close to twice the optimal.

Remark 32 (Implementation Insight). *In practice, the MST-based approximation usually performs much better than its worst-case factor-2 guarantee, but it is not competitive with other simple heuristics: in the survey of Johnson and McGeoch it is dominated in practice by nearest neighbor and the greedy heuristic, and on random Euclidean instances nearest-neighbor tours are already about 25% longer than optimal on average, compared with about 10% for Christofides' algorithm.*

While the TSP is NP-hard, several approximation algorithms provide practical solutions with guaranteed bounds on the quality of the result. We begin with an exact algorithm before exploring approximation techniques.

Definition 28 (Held-Karp Algorithm). The Held-Karp algorithm is an exact dynamic programming approach to solving the Traveling Salesman Problem. For a graph $G = (V, E)$ with edge weights w , it computes the minimum-cost tour by:

1. Defining $C(S, i)$ as the cost of the shortest path starting at vertex 1, visiting all vertices in S exactly once, and ending at vertex $i \in S$
2. Computing values using the recurrence relation:

$$C(S, i) = \begin{cases} w(1, i) & \text{if } S = \{i\} \\ \min_{j \in S, j \neq i} \{C(S \setminus \{i\}, j) + w(j, i)\} & \text{otherwise} \end{cases} \quad (30)$$

3. Finding the optimal tour cost as $\min_{i \in V \setminus \{1\}} \{C(V \setminus \{1\}, i) + w(i, 1)\}$

The algorithm has time complexity $O(n^2 \cdot 2^n)$ and space complexity $O(n \cdot 2^n)$, making it tractable only for instances with approximately 20-25 cities.

Algorithm 8 Held-Karp Algorithm

Require: Complete graph $G = (V, E)$ with edge weights w

Ensure: Optimal TSP tour and its cost

```

1: function HELDKARP( $G = (V, E), w$ )
2:   Initialize  $C[\{i\}][i] \leftarrow w(1, i)$  for all  $i \in \{2, \dots, n\}$ 
3:   for  $s \leftarrow 2$  to  $n - 1$  do
4:     for all subsets  $S \subset \{2, \dots, n\}$  with  $|S| = s$  do
5:       for each  $i \in S$  do
6:          $C[S][i] \leftarrow \min_{j \in S, j \neq i} \{C[S \setminus \{i\}][j] + w(j, i)\}$ 
7:       end for
8:     end for
9:   end for
10:   $opt\_cost \leftarrow \min_{i=2}^n \{C[\{2, \dots, n\}][i] + w(i, 1)\}$ 
11:   $tour \leftarrow$  Reconstruct tour using parent pointers
12:  return ( $tour, opt\_cost$ )
13: end function

```

Definition 29 (Approximation Algorithms for TSP). For instances where exact solutions are impractical, the following approximation algorithms offer efficient alternatives:

1. **Christofides Algorithm:** Provides a $\frac{3}{2}$ -approximation for metric TSP by augmenting a minimum spanning tree with a minimum-weight perfect matching on odd-degree vertices.
2. **Double-Tree Algorithm:** Achieves a 2-approximation for metric TSP by doubling each edge in a minimum spanning tree to create an Eulerian circuit, then shortcutting to form a tour.

3. **Nearest Neighbor:** A greedy heuristic without general approximation guarantees that iteratively selects the closest unvisited vertex.
4. **k-opt:** A local search method that improves tours by replacing k edges with k different edges, with no general approximation ratio but good practical performance.

3.8 Chapter 3 Exercises

3.8.1 Easy Problems

1. Explain the difference between a Hamiltonian path and a Hamiltonian cycle using a simple example.
2. Prove that every complete graph K_n with $n \geq 3$ is Hamiltonian.
3. Explain why a bipartite graph with unequal partition sizes cannot contain a Hamiltonian cycle.
4. Consider a graph with 8 vertices where each vertex has degree 3. Does Dirac's theorem guarantee that this graph is Hamiltonian? Explain your reasoning.
5. Prove that a graph with a pendant vertex (degree 1) cannot have a Hamiltonian cycle.
6. Design an algorithm to verify if a graph satisfies Dirac's condition in $O(n)$ time.
7. If a graph has minimum degree $k = 4$, what is the minimum length of a cycle guaranteed to exist in the graph?
8. How many distinct Hamiltonian cycles does K_5 contain?

3.8.2 Medium Problems

9. Consider a graph with 9 vertices. For every pair of non-adjacent vertices, the sum of their degrees is at least 8. Apply Ore's theorem to determine if this graph is Hamiltonian.
10. Given a graph with degree sequence $(2, 2, 3, 3, 3, 5)$, determine whether it satisfies Chvátal's condition for Hamiltonicity.
11. Prove that the Petersen graph is not Hamiltonian using the concept of matchings.
12. Given a graph G , explain the process of computing its closure $\text{cl}(G)$ and how this relates to Hamiltonicity.
13. For a graph with 10 vertices, which is stronger: Dirac's condition or Ore's condition? Give an example of a graph that satisfies one but not the other.
14. Prove that if a graph with $n \geq 3$ vertices has a Hamiltonian path with endpoints u and v , and $\{u, v\} \in E(G)$, then G has a Hamiltonian cycle.
15. Explain the relationship between a graph's toughness and its Hamiltonian properties.
16. Let G be a graph with a cycle C of length $k < n$. Give sufficient conditions for when G contains a cycle of length $k + 1$ that includes all vertices of C .
17. Prove that the circulant graph $\text{Circ}(n; \{1, 2\})$ is Hamiltonian for all $n \geq 3$.
18. Describe how to construct a Hamiltonian cycle in the complete bipartite graph $K_{3,3}$.

3.8.3 Hard Problems

19. Construct a graph that has minimum degree $\lfloor \frac{n}{2} \rfloor - 1$ and is not Hamiltonian.
20. Prove that if a graph G with $n \geq 3$ vertices has connectivity $\kappa(G) \geq \alpha(G)$ (independence number), then G is Hamiltonian.
21. Prove that the factor-2 approximation of the MST-based algorithm for metric TSP is tight by constructing an example.
22. If a graph G on n vertices is Hamiltonian and $\delta(G) \geq \frac{n+1}{2}$, prove that G contains cycles of all lengths from 3 to n .

23. Prove that a graph G is Hamiltonian if and only if its closure $\text{cl}(G)$ is Hamiltonian.
24. Give a complexity analysis for the Held-Karp algorithm for the TSP, explaining why the dynamic programming approach requires $O(n^2 \cdot 2^n)$ time.
25. Prove that an 8×8 chessboard has a closed knight's tour (a Hamiltonian cycle in the knight's graph).
26. Analyze why Christofides' algorithm achieves a $\frac{3}{2}$ -approximation for metric TSP rather than a 2-approximation.

3.8.4 Diagram Problems

27. Draw a non-Hamiltonian graph with 6 vertices where each vertex has degree at least 2.
28. Draw a graph that is Hamiltonian but not Eulerian, and another graph that is Eulerian but not Hamiltonian. Explain why each has one property but not the other.
29.
 - (a) Create a weighted graph with 5 vertices.
 - (b) Find the optimal TSP tour.
 - (c) Apply the MST-based 2-approximation algorithm.
 - (d) Compare the results of the optimal solution and the approximation.
30. Draw a diagram illustrating the key step in Dirac's theorem proof: a Hamiltonian path $(x_1, x_2, \dots, x_n)$ from $x_1 = u$ to $x_n = v$ between non-adjacent vertices u and v , together with an index i such that u is adjacent to x_{i+1} and v is adjacent to x_i . Show how this leads to a Hamiltonian cycle, and explain why a common neighbor of u and v alone would not suffice.

4 Long Cycles

4.1 Long Cycles and Hamiltonian

The study of long cycles in graph theory extends our understanding beyond the well-established theory of Hamiltonian cycles, offering both theoretical insights and practical applications:

- **Extension of Hamiltonian theory:** While Dirac's Theorem and the Bondy-Chvátal Closure Theorem provide sufficient conditions for Hamiltonicity, many graphs fail to meet these conditions. Long cycle theory addresses the natural follow-up question: what is the best we can do when a Hamiltonian cycle doesn't exist?
- **Connection to the Traveling Salesman Problem:** The TSP seeks optimal Hamiltonian cycles in weighted graphs. However, in contexts where Hamiltonian cycles don't exist, finding the longest possible cycle becomes the natural alternative objective.
- **Structural guarantees:** The theorem establishing that graphs with minimum degree $k \geq 2$ must contain a cycle of length at least $k + 1$ provides a fundamental relationship between local properties (vertex degrees) and global structures (cycle lengths).
- **Optimality and tightness of bounds:** The complete graph K_{k+1} demonstrates that the bound of $k + 1$ is tight—we cannot guarantee longer cycles based solely on minimum degree. This reveals both the power and limitations of the minimum degree parameter.
- **Bridge to extremal graph theory:** The study of longest cycles connects to broader questions in extremal graph theory: what structural properties guarantee the existence of specific subgraphs or patterns?

This exploration creates a spectrum between graphs containing no cycles and those with Hamiltonian cycles, offering a more nuanced understanding of cycle structures in graphs.

Theorem 33 (Minimum Degree and Cycle Length). If G is a graph with minimum degree $k \geq 2$, it contains a cycle of length at least $k + 1$.

Proof. Let P be a longest path in the graph, with endpoints u and v . We first observe that:

$$N_G(u) \subseteq V(P) \quad \text{and} \quad N_G(v) \subseteq V(P) \quad (31)$$

This must be true, as any neighbor outside P would allow us to extend the path, contradicting its maximality. Since G has minimum degree at least k , we know that $|N_G(v)| \geq k$.

Let $N_G(v) = \{v_1, v_2, \dots, v_\ell\}$ where $\ell \geq k$. We can list these neighbors in the order they appear along the path P from u to v :

$$v_\ell, v_{\ell-1}, \dots, v_2, v_1 \quad (32)$$

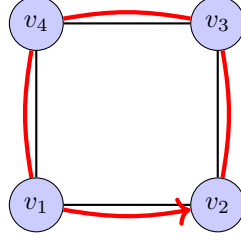
Now, consider the cycle C' formed by:

- The subpath of P from v_k to v (which passes through all vertices $v_{k-1}, \dots, v_1$)
- The edge $\{v_k, v\}$

This cycle C' contains at least the vertices $\{v, v_1, v_2, \dots, v_k\}$, so $|V(C')| \geq k + 1$, and therefore C' has length at least $k + 1$.

To show this bound is tight, consider the complete graph K_{k+1} . It has minimum degree k and its longest cycle has length exactly $k + 1$, demonstrating that our bound cannot be improved in general. $\square$

This C_4 example shows that while the theorem guarantees a cycle of length at least $k + 1$ (which would be 3 in this case), we can sometimes find longer cycles. Here, the minimum degree is $k = 2$, but the cycle has length 4. This illustrates that the theorem provides a lower bound that is tight in general, but specific graph structures may contain even longer cycles.



Edges: $(v_1, v_2) = e_1$, $(v_2, v_3) = e_2$, $(v_3, v_4) = e_3$, $(v_4, v_1) = e_4$
 Path: $v_1 \xrightarrow{e_1} v_2 \xrightarrow{e_2} v_3 \xrightarrow{e_3} v_4 \xrightarrow{e_4} v_1$

Figure 59: A cycle C_4 where each vertex has degree $k = 2$. The cycle length is $4 = k + 2$, demonstrating that cycles longer than $k + 1$ are possible in specific graphs.

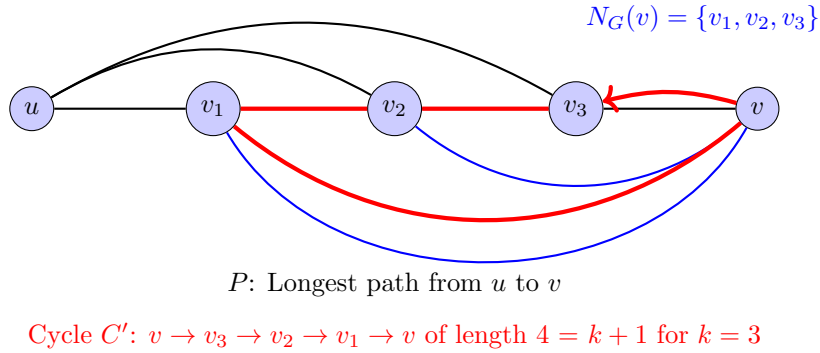


Figure 60: Applying the long cycle theorem to a graph with minimum degree 3.

Example 6 (Applying the Proof to a Concrete Graph). In this example, we illustrate the proof technique on a concrete graph. The vertex v has three neighbors on the longest path P : v_1 , v_2 , and v_3 . Following the construction in our proof, we form cycle C' by taking the path from v_1 to v along P , and then adding the edge $\{v_1, v\}$.

The resulting cycle C' contains vertices $\{v, v_3, v_2, v_1\}$ and thus has length 4, which is exactly $k + 1$ where $k = 3$ is the minimum degree of the graph. This demonstrates the theorem's guarantee in action, showing that the minimum degree directly determines the length of the longest guaranteed cycle.

Theorem 34 (Long Cycle Theorem). If G is a graph with minimum degree at least $k \geq 2$, then G contains a cycle of length at least $k + 1$.

Lemma 20 (Path Extension). Let P be a path in a graph G . If an endpoint v of P has a neighbor $w \notin V(P)$, then there exists a path P' that is longer than P .

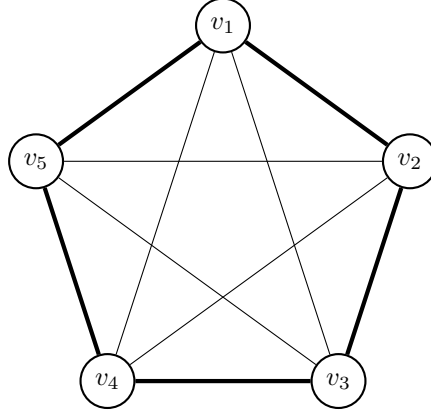
Proof. We can construct P' by appending the vertex w and the edge $\{v, w\}$ to P at its endpoint v . Since $w \notin V(P)$, this creates a strictly longer path. $\square$

Lemma 21 (Endpoint Neighborhood Containment). If P is a longest path in a graph G with endpoints u and v , then $N_G(u) \subseteq V(P)$ and $N_G(v) \subseteq V(P)$.

Corollary 13 (Degree-Length Relationship). If G has minimum degree k and P is a longest path with endpoint v , then P contains at least k vertices adjacent to v .

Remark 33. The Long Cycle Theorem is tight in the sense that for any $k \geq 2$, there exists a graph with minimum degree k whose longest cycle has length exactly $k + 1$. The complete graph K_{k+1} demonstrates this, as shown in Figure 61 for $k = 4$.

Remark 34. While the theorem guarantees a cycle of length at least $k + 1$, specific graph structures may contain longer cycles. For instance, in our example with minimum degree $k = 3$, the proof produces a cycle of length 4, which equals $k + 1$. However, the same graph also contains the cycle (u, v_1, v, v_3, v_2, u) of length 5, and other graphs with minimum degree 3 may contain even longer cycles.



K_5 (Complete Graph)

Degree Information: $d(v_i) = 4$ for all i , Minimum degree $k = 4$

Cycle: $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_1$ has length $5 = k + 1$

Figure 61: The complete graph K_5 demonstrating the tightness of the Long Cycle Theorem. The thick cycle $C = (v_1, v_2, v_3, v_4, v_5, v_1)$ has length 5, which equals $k + 1$ where $k = 4$ is the minimum degree. This shows the theorem's bound is tight, as no cycle longer than $k + 1$ exists in this graph.

Example (Complete Graph - Tight Bound Example). This example demonstrates that the Long Cycle Theorem's bound cannot be improved in general. In the complete graph K_5 :

- Every vertex is adjacent to the other four vertices, so each vertex has degree 4
- Hence the minimum degree is $k = 4$, and the theorem guarantees a cycle of length at least $k + 1 = 5$
- A cycle cannot repeat vertices, so no cycle in K_5 has length greater than 5; the longest cycles are the Hamiltonian cycles, of length 5
- This cycle length equals $k + 1 = 5$, showing we cannot improve the theorem to guarantee cycles longer than $k + 1$

While some graphs with minimum degree k may contain cycles of length greater than $k + 1$, this example (and, more generally, the complete graph K_{k+1} for every $k \geq 2$) proves that $k + 1$ is the best possible universal guarantee for all graphs with minimum degree k .

Remark 35 (Star Graphs and the Hypothesis $k \geq 2$). The star graph $K_{1,n}$ shows why the Long Cycle Theorem requires $k \geq 2$. For any star graph, the minimum degree is 1 (at the leaves), and $K_{1,n}$ contains no cycle at all: a cycle uses two edges at each of its vertices, so it cannot pass through a leaf, and the center alone cannot form a cycle. Thus minimum degree 1 does not guarantee any cycle, regardless of how many leaves are added.

Theorem 35 (Improved Bound for 2-Connected Graphs; Dirac, 1952). If G is a 2-connected graph with minimum degree $k \geq 2$, then G contains a cycle of length at least $\min\{2k, |V(G)|\}$.

Sketch. Write $n = |V(G)|$ and suppose, for a contradiction, that every cycle of G has length less than $\min\{2k, n\}$. Among all pairs (C, P) , where C is a longest cycle and $P = u_1 u_2 \cdots u_{\ell+1}$ is a path that meets C only in its first vertex u_1 , choose one with ℓ as large as possible, and write $C = v_1 v_2 \cdots v_c v_1$ with $v_c = u_1$.

Then $c < n$ and $c \leq 2k - 1$. Since $c < n$ and G is connected, $\ell \geq 1$. Let $y = u_{\ell+1}$. By the maximality of ℓ , every neighbor of y lies on C or on P .

Case 1: y is adjacent to some v_i with $1 \leq i \leq c - 1$. The cycle $v_i v_{i+1} \cdots v_c u_2 u_3 \cdots u_{\ell+1} v_i$ has length $c - i + \ell + 1$, so the maximality of C forces $i \geq \ell + 1$; symmetrically, $i \leq c - \ell - 1$. Moreover, y is not adjacent to two consecutive vertices v_i, v_{i+1} of C , since otherwise replacing the edge $\{v_i, v_{i+1}\}$ by the path $v_i y v_{i+1}$ would give a longer cycle. Hence y has at most $\lceil (c - 1 - 2\ell)/2 \rceil$ neighbors among $v_1, \dots, v_{c-1}$ and at most ℓ neighbors among $u_1, \dots, u_\ell$, so

$$k \leq d(y) \leq \ell + \left\lceil \frac{c - 1 - 2\ell}{2} \right\rceil = \left\lceil \frac{c - 1}{2} \right\rceil \leq k - 1,$$

a contradiction.

Case 2: every neighbor of y lies on P . This case uses a refinement, due to Dirac, of the fact that any two vertices of a 2-connected graph are joined by two internally disjoint paths: the two paths can be chosen so that each of them meets a prescribed path between the two vertices in the same order as that path does. We omit the details; see G. A. Dirac, *Some theorems on abstract graphs*, Proc. London Math. Soc. (3) 2 (1952), 69–81 (Theorem 4), or A. Kostochka, R. Luo, and G. McCourt, *On a property of 2-connected graphs and Dirac's theorem*, Discrete Math. 347 (2024), 114153, for a short complete proof. $\square$

Theorem 36 (Extremal Graphs for the Long Cycle Theorem). For any $k \geq 2$, the complete graph K_{k+1} is an extremal graph for the Long Cycle Theorem in the following sense: it has minimum degree k and its longest cycle has length exactly $k + 1$.

Proof. Every vertex of K_{k+1} is adjacent to the other k vertices, so $\delta(K_{k+1}) = k$. A cycle cannot repeat vertices, so no cycle of K_{k+1} has length greater than $k + 1$; on the other hand, listing the vertices in any order $v_1, v_2, \dots, v_{k+1}$ gives the cycle $(v_1, v_2, \dots, v_{k+1}, v_1)$ of length exactly $k + 1$. $\square$

4.2 Algorithm for Finding Long Cycles

A simple algorithm to find a cycle of length at least $k + 1$ in a graph with minimum degree $k \geq 2$:

Remark 36. *DFS and BFS will be covered later in this book. Come back to this section after we cover DFS and BFS to gain a deeper understanding of the path-finding technique employed.*

Remark 37. *While the algorithm is presented using DFS for finding a longest path, it's important to note that finding a truly longest path in a general graph is NP-hard. However, any maximal path (one that cannot be extended further) provides sufficient properties for the algorithm to work correctly.*

Remark 38. *The correctness of this algorithm relies on the theorem that in a graph with minimum degree $k \geq 2$, there exists a cycle of length at least $k + 1$. The proof of this theorem follows from the properties of longest paths and the pigeonhole principle.*

Remark 39. *For practical implementations, approximation algorithms for longest paths may be substituted. Even a greedy path-building approach will satisfy our needs, as long as the resulting path is maximal (cannot be extended).*

4.3 Chapter 4 Exercises

4.3.1 Easy Problems

1. State the theorem on long cycles in a graph with minimum degree $k \geq 2$. What length of a longest cycle is guaranteed?
2. In a graph with minimum degree 5, what is the minimum cycle length guaranteed by the long cycle theorem?
3. Prove that a graph with maximum degree 2 cannot have a cycle of length greater than the number of vertices.

Algorithm 9 FindLongCycle: Finding a cycle of length at least $k + 1$

Require: Graph $G = (V, E)$ with minimum degree $k \geq 2$

Ensure: A cycle C of length at least $k + 1$

```

1: function FINDLONGCYCLE( $G = (V, E)$ )
2:   /* Phase 1: Find longest path */
3:    $P \leftarrow$  longest path  $(v_1, \dots, v_\ell)$  found via DFS from arbitrary start
4:   /* Phase 2: Find neighbors of endpoint */
5:    $N \leftarrow \{v_j \in P \setminus \{v_\ell\} : \{v_j, v_\ell\} \in E\}$   $\triangleright |N| \geq k$ 
6:   /* Phase 3: Sort neighbors by position */
7:   Build position map  $pos$  where  $pos[v_i] = i$  for all  $v_i \in P$ 
8:   Sort  $N$  by increasing position value
9:    $v_{earliest} \leftarrow$  first vertex in sorted  $N$ 
10:   $j \leftarrow pos[v_{earliest}]$ 
11:  /* Phase 4: Build and return cycle */
12:   $C \leftarrow (v_j, v_{j+1}, \dots, v_\ell, v_j)$ 
13:  return  $C$   $\triangleright |C| = \ell - j + 1 \geq k + 1$ 
14: end function
15: function FINDLONGESTPATH( $G, start$ )
16:   Initialize  $visited \leftarrow \{start\}$ ,  $curr \leftarrow (start)$ ,  $best \leftarrow (start)$ 
17:   Call DFSPATHHELPER( $G, start, visited, curr, best$ )
18:   return  $best$ 
19: end function
20: function DFSPATHHELPER( $G, v, visited, curr, best$ )
21:   for each neighbor  $w$  of  $v$  in  $G$  do
22:     if  $w \notin visited$  then
23:       Add  $w$  to  $visited$  and append to  $curr$ 
24:       DFSPATHHELPER( $G, w, visited, curr, best$ )
25:       Remove  $w$  from  $visited$  and  $curr$   $\triangleright$  Backtrack
26:     end if
27:   end for
28:   if  $|curr| > |best|$  then
29:     Update  $best \leftarrow$  copy of  $curr$ 
30:   end if
31: end function

```

4. What cycle length does the long cycle theorem guarantee in a 3-regular graph with 8 vertices? Is this bound tight? Explain.
5. Explain why the minimum cycle length in a graph must be at least 3.
6. Find the length of the longest cycle in the graph $K_{2,4}$ (complete bipartite graph with partite sets of sizes 2 and 4).
7. Prove that any graph with minimum degree $\delta(G) \geq 2$ must contain at least one cycle.

4.3.2 Medium Problems

8. Let P be a longest path in a graph G with minimum degree $k \geq 2$. Prove that the endpoints of P have all their neighbors on P .
9. Explain why the bound in the long cycle theorem is tight by constructing an example of a graph with minimum degree k where the longest cycle has length exactly $k + 1$.
10. Prove that in a 2-connected graph with minimum degree $k \geq 2$, there exists a cycle of length at least $\min\{2k, n\}$.
11. A graph G has minimum degree $k = 3$. Describe an algorithm to find a cycle of length at least $k + 1 = 4$ in G .
12. Let G be a graph with minimum degree $\delta(G) = 3$. If G has exactly 6 vertices, show that G must contain a cycle of length at least 4.
13. Find the relationship between the length of the longest cycle in a graph G and the length of the longest cycle in its complement $\overline{G}$.
14. Prove that a graph G on $n \geq 3$ vertices is Hamiltonian if and only if the length of its longest cycle is exactly n .

4.3.3 Hard Problems

15. Prove or disprove: if G is a graph on n vertices with minimum degree $\delta(G) \geq \frac{n}{r+1} + 1$, where $r \geq 2$, then G contains a cycle of length at least $r + 2$. Then show that the statement is true under the additional assumption $n \geq r(r + 1)$.
16. Prove or disprove: if G is a k -connected graph with minimum degree $\delta(G) \geq k + 1$, then G contains a cycle of length at least $2k + 2$.
17. Let G be a graph with minimum degree $k \geq 2$. Prove that if G has girth g (the length of its shortest cycle), then G has a cycle of length at least $(k - 1)(g - 2) + 2$. (For $g = 3$ this is the Long Cycle Theorem.)
18. Design an algorithm to find the longest cycle in a graph G . Analyze its time complexity and explain why finding the longest cycle is generally an NP-hard problem.
19. Let G be a graph with n vertices and let $\lambda(G)$ denote the length of the longest cycle in G . Prove that if $\delta(G) > \frac{n}{2}$, then $\lambda(G) \geq 2\delta(G)$ unless G is Hamiltonian.
20. Prove or disprove: if G is a 3-connected cubic graph (3-regular) on n vertices, then G contains a cycle of length at least $4 \log_2 n$.

4.3.4 Diagram Problems

21. Draw a graph with 8 vertices and minimum degree 3 that contains a cycle of length exactly 4 but no longer cycles. Mark the cycle in your diagram.
22. Illustrate the proof technique of the long cycle theorem by drawing:
 - (a) A longest path P in a graph with endpoints u and v
 - (b) The neighbors of v on the path P
 - (c) The earliest neighbor of v on the path
 - (d) The resulting cycle formed by the theorem

Label all relevant vertices and explain each step.

23. Draw a star graph $K_{1,4}$ and explain why it contains no cycle at all. Then add one edge joining two leaves and explain why the resulting graph has exactly one cycle, and why that cycle has length 3. Finally, add a second edge so that the graph contains a cycle of length 4, and mark it.
24. Create a sequence of graphs G_3, G_4, G_5 where each G_k has minimum degree k and contains a cycle of length exactly $k + 1$. Draw each graph and highlight the longest cycle in each.
25. Draw the Petersen graph and identify the longest cycle in it. Explain why this graph does not have a Hamiltonian cycle despite having minimum degree 3.
26. Draw a graph where the bound from the long cycle theorem is not tight, i.e., a graph with minimum degree k where the longest cycle has length strictly greater than $k + 1$. Explain why this happens in your example.
27. Illustrate the relationship between the minimum degree and cycle length by drawing three different graphs with 6 vertices that have minimum degrees 2, 3, and 4 respectively. For each graph, highlight the longest cycle and discuss how the minimum degree affects the cycle length.

5 Trees and Forests

5.1 Trees, Forests, and Bridges

Definition 30 (Tree). A tree is a connected graph with no cycles.

Definition 31 (Acyclic Graph). A graph G is called *acyclic* if it contains no cycles.

Definition 32 (Forest). A forest is an acyclic graph, i.e., a graph with no cycles. Equivalently, a forest is a disjoint union of trees.

Definition 33 (Root). In a rooted tree, the *root* is a distinguished vertex that serves as the reference point for the hierarchical structure of the tree. From the root, there is a unique path to every other vertex in the tree. The root is the only vertex in a rooted tree that has no parent.

Lemma 22 (Tree Edge Count). Let T be a tree with n vertices. Then T has exactly $n - 1$ edges.

Proof. We proceed by induction on n . For $n = 1$, a tree with one vertex has 0 edges, and $1 - 1 = 0$.

Assume that for some $k \geq 1$, every tree with at most k vertices has exactly one fewer edge than vertices (we use strong induction). Consider a tree T with $k + 1$ vertices. Choose any edge $e = \{u, v\}$ in T . Removing e disconnects T into two components, each of which is a tree: if u and v were still joined by a path in $T - e$, that path together with e would form a cycle. Let T_1 have n_1 vertices and T_2 have n_2 vertices, where $n_1 + n_2 = k + 1$ and $n_1, n_2 \geq 1$, so that $n_1, n_2 \leq k$. By the induction hypothesis, T_1 has $n_1 - 1$ edges and T_2 has $n_2 - 1$ edges. Therefore, T has $(n_1 - 1) + (n_2 - 1) + 1 = n_1 + n_2 - 1 = (k + 1) - 1 = k$ edges. $\square$

Lemma 23 (Unique Path in Trees). In a tree, there exists exactly one path between any two vertices.

Proof. Let T be a tree and let u and v be two distinct vertices in T . Since T is connected, there exists at least one path between u and v . Suppose for contradiction that there exist two distinct paths P_1 and P_2 from u to v . Then $P_1 \cup P_2$ contains a cycle, which contradicts the definition of a tree. Therefore, there can be only one path between any two vertices in a tree. $\square$

Lemma 24 (Leaf Existence). Every tree with at least two vertices has at least two leaves (vertices of degree 1).

Proof. Let T be a tree with $n \geq 2$ vertices. The sum of all vertex degrees in T is $2(n - 1)$ since T has $n - 1$ edges and each edge contributes 2 to the total degree sum. If every vertex had degree at least 2, then the sum of degrees would be at least $2n$, which is a contradiction. Therefore, T must have at least one vertex of degree 1.

Now consider a longest path P in T ; since T is connected and has $n \geq 2$ vertices, P has at least one edge, so its two endpoints are distinct. Both endpoints of P must have degree 1 in T . Indeed, suppose an endpoint x of P had a neighbor y other than its neighbor on P . If $y \in V(P)$, then the subpath of P from x to y together with the edge $\{x, y\}$ would form a cycle, contradicting that T is acyclic. If $y \notin V(P)$, we could extend P by y to a longer path, contradicting our choice of P . Therefore, T has at least two leaves. $\square$

Lemma 25 (Removal of Leaf). If T is a tree and v is a leaf in T , then $T - v$ is also a tree.

Proof. Let T be a tree and v a leaf in T . Since v has degree 1, it is connected to exactly one other vertex. Removing v and its incident edge cannot create a cycle. Moreover, $T - v$ remains connected since v is not on any path between other vertices in T . Therefore, $T - v$ is a tree. $\square$

Lemma 26 (Equivalent Tree Characterizations). For a graph G with n vertices, the following statements are equivalent:

1. G is a tree.
2. G is connected and has $n - 1$ edges.
3. G is acyclic and has $n - 1$ edges.
4. G is acyclic, and adding any edge creates exactly one cycle.
5. G is connected, and removing any edge disconnects the graph.

Lemma 27 (Forest Component Count). If F is a forest with n vertices, m edges, and c components, then $n - m = c$.

Proof. Let F be a forest with c components. Each component is a tree. If the i -th component has n_i vertices, then it has $n_i - 1$ edges. The total number of vertices is $n = \sum_{i=1}^c n_i$, and the total number of edges is $m = \sum_{i=1}^c (n_i - 1) = \sum_{i=1}^c n_i - c = n - c$. Rearranging, we get $n - m = c$. $\square$

Lemma 28 (Bipartiteness of Trees). Every tree is bipartite.

Proof. Let T be a tree. Choose any vertex r in T as a "root." Partition the vertices of T into two sets: A consists of vertices whose distance from r is even, and B consists of vertices whose distance from r is odd. Since T has a unique path from r to any other vertex, this partition is well-defined. Now, consider any edge $\{u, v\}$ in T . The distances of u and v from r differ by exactly 1, so one is even and one is odd. Hence, one vertex is in A and the other is in B . Therefore, T is bipartite with partite sets A and B . $\square$

Lemma 29 (Spanning Trees). Every connected graph has at least one spanning tree.

Proof. Let G be a connected graph. If G is acyclic, then it is already a tree and thus its own spanning tree. If G contains cycles, we can repeatedly remove edges from cycles until no cycles remain. This process preserves connectivity because removing an edge from a cycle does not disconnect the graph. The resulting subgraph is a tree that spans all vertices of G , i.e., a spanning tree of G . $\square$

Lemma 30 (Forest Edge Bound). A graph with n vertices and fewer than $n - 1$ edges is not connected. A graph with n vertices and more than $n - 1$ edges contains at least one cycle.

Proof. Let G be a graph with n vertices and m edges. Suppose $m < n - 1$, and suppose for contradiction that G is connected. By the Spanning Trees lemma, G has a spanning tree T , and by the Tree Edge Count lemma, T has exactly $n - 1$ edges. Since $E(T) \subseteq E(G)$, this gives $m \geq n - 1$, a contradiction. Therefore G is not connected.

If $m > n - 1$, suppose for contradiction that G is acyclic. Then G is a forest with c components, where $c \geq 1$. By the formula $n - m = c$, we have $c = n - m < n - (n - 1) = 1$, which is a contradiction. Therefore, G must contain at least one cycle. $\square$

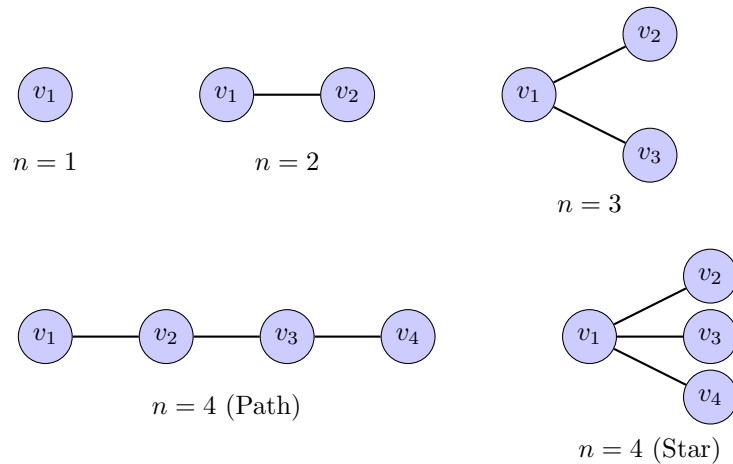


Figure 62: Examples of all possible tree structures with 1 to 4 vertices. Notice that each tree with n vertices has exactly $n - 1$ edges. As the number of vertices increases, the variety of possible tree structures grows rapidly.

Example 7 (Trees with at most four vertices). This figure shows the fundamental tree structures with small numbers of vertices. For $n = 1$, there is only one possible tree: a single isolated vertex. For $n = 2$, there is again only one possible structure: two vertices connected by a single edge. For $n = 3$, there is still only one

tree up to isomorphism: the path with three vertices, which is the same graph as the star with one central vertex connected to two leaves. Starting from $n = 4$, we see multiple possible configurations emerging: up to isomorphism there are exactly two trees with four vertices, a path of length 3 and a star with one central vertex connected to three leaves, and both are shown. (There are three trees with five vertices.) Each of these structures satisfies the definition of a tree: they are connected and acyclic. No matter how complex a tree becomes, it will always have exactly $n - 1$ edges, where n is the number of vertices.

Definition 34 (Bridge). A bridge in a graph G is an edge $e \in E(G)$ such that $G - e$ has more components than G . If G is connected, then a bridge is an edge whose removal disconnects the graph.

Corollary 14. If G is a tree, then every edge is a bridge.

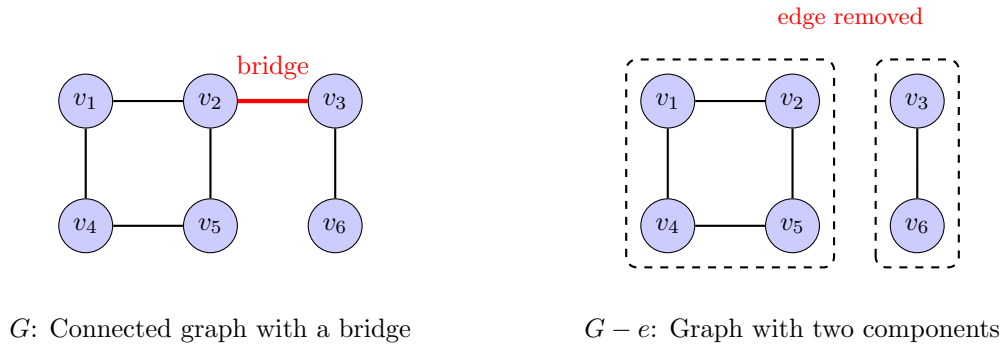


Figure 63: Illustration of a bridge in a graph. The left shows a connected graph G with a bridge highlighted in red between vertices v_2 and v_3 . The right shows the result after removing this bridge: the graph $G - e$ becomes disconnected with two separate components (outlined by dashed boxes). In a tree, every edge is a bridge because removing any edge would disconnect the tree.

Remark 40. A bridge is also sometimes called a cut-edge or isthmus. The presence of bridges in a graph indicates potential vulnerabilities in network connectivity. In applications such as communication networks, bridges represent critical connections whose failure would partition the network.

5.2 Characterization of Trees

Lemma 31 (Tree Characterization). For a graph G , the following conditions are equivalent:

1. G is a tree.
2. G is connected and acyclic.
3. G is connected and every edge is a bridge.
4. G is connected and has exactly $|V(G)| - 1$ edges.
5. G is acyclic and has exactly $|V(G)| - 1$ edges.
6. G is acyclic, and adding any edge creates exactly one cycle.

Proof. We'll prove the equivalence by showing $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (1)$ and $(5) \Rightarrow (6) \Rightarrow (2)$.

1. $(1) \Rightarrow (2)$: This follows directly from the definition of a tree.
2. $(2) \Rightarrow (3)$: If G is connected and acyclic, then for any edge $e = \{u, v\}$, removing e would disconnect u and v . If there were still a path from u to v after removing e , then this path together with e would form a cycle, contradicting that G is acyclic.

3. (3) $\Rightarrow$ (4): Suppose G is connected and every edge is a bridge.
 - G is acyclic: if G contained a cycle C and $e = \{u, v\}$ were an edge of C , then $C - e$ would be a u, v -path in $G - e$, so $G - e$ would still be connected and e would not be a bridge.
 - Hence G is connected and acyclic, i.e., G is a tree.
 - By the Tree Edge Count Lemma, G has exactly $|V(G)| - 1$ edges.
4. (4) $\Rightarrow$ (5): We need to show that if G is connected with $|V(G)| - 1$ edges, then G is acyclic.
 - Suppose G contains a cycle.
 - Removing any edge e from this cycle would preserve connectivity.
 - Thus, $G - e$ would be connected with $|V(G)| - 2$ edges.
 - This contradicts the fact that a connected graph on $|V(G)|$ vertices needs at least $|V(G)| - 1$ edges.
5. (5) $\Rightarrow$ (1): If G is acyclic with $|V(G)| - 1$ edges, then G has exactly the right number of edges to be a tree.
 - We need to show that G is connected.
 - If G were not connected, with $c > 1$ components, then each component would be a tree.
 - If component i has n_i vertices, it would have $n_i - 1$ edges.
 - The total number of edges would be $\sum_{i=1}^c (n_i - 1) = |V(G)| - c < |V(G)| - 1$, a contradiction.
6. (5) $\Rightarrow$ (6): If G is acyclic with $|V(G)| - 1$ edges, adding any edge $e = \{u, v\}$ would create at most one cycle.
 - This cycle would be formed by e and the unique path from u to v in G .
 - We need to show that G is connected so that this path exists.
 - The proof is the same as for (5) $\Rightarrow$ (1).
7. (6) $\Rightarrow$ (2): If G is acyclic and adding any edge creates exactly one cycle, then for any two vertices u and v not adjacent in G :
 - Adding the edge $\{u, v\}$ creates a cycle.
 - This means there must already be a path from u to v in G .
 - Since this holds for any two vertices, G is connected.

□

Definition 35. (Spanning Tree) A spanning tree of a graph G is a spanning subgraph of G that is a tree.

Lemma 32 (Existence of Spanning Trees). Every connected graph has a spanning tree.

Proof. Let G be a connected graph. If G has no cycles, then it is already a tree and thus its own spanning tree. If G contains cycles, we can repeatedly remove edges from cycles until no cycles remain. This process preserves connectivity because removing an edge from a cycle does not disconnect the graph. The resulting subgraph is a tree that spans all vertices of G , i.e., a spanning tree of G . □

Example 8 (Finding a Spanning Tree). To find a spanning tree in the graph shown in Figure 65:

1. We start with the connected graph on the left, which contains cycles.
2. First, we identify the cycle formed by vertices a , b , and c , and remove the edge $\{b, c\}$.
3. Next, we identify the remaining cycle formed by vertices a , b , and d , and remove the edge $\{b, d\}$.

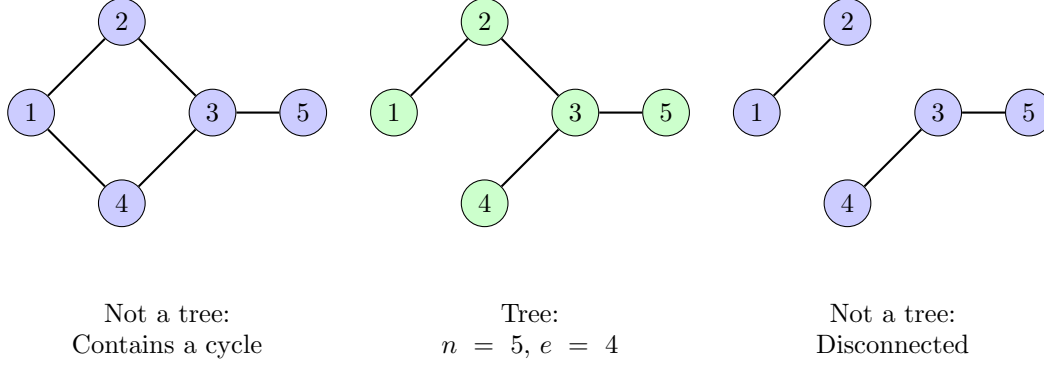


Figure 64: Illustration of tree characterization. Left: A connected graph with a cycle is not a tree. Center: A connected acyclic graph is a tree with $n - 1$ edges. Right: A disconnected acyclic graph is a forest, not a tree. The center graph satisfies all conditions from the lemma: it is connected, acyclic, has $n - 1$ edges, and every edge is a bridge.

Algorithm 10 Spanning Tree Construction

Require: A connected graph $G = (V, E)$

Ensure: A spanning tree $T = (V, E')$ of G

- 1: Initialize $T \leftarrow G$
 - 2: **while** T contains a cycle C **do**
 - 3: Choose any edge $e \in C$
 - 4: Remove e from T
 - 5: **end while**
 - 6: **return** T
-

4. After removing these edges, no cycles remain in the graph.

5. The resulting structure is a spanning tree of the original graph, with $n - 1 = 3$ edges for $n = 4$ vertices.

Remark 41. *The spanning tree of a graph is not necessarily unique. Different choices of which edges to remove from cycles can lead to different spanning trees of the same graph.*

Theorem 37 (Spanning Tree Edge Properties). If T is a spanning tree of graph G , then:

1. For any edge $e \in E(G) \setminus E(T)$, the graph $T + e$ contains exactly one cycle.
2. For any edge $e \in E(T)$, the graph $T - e$ has exactly two components.

5.3 Breadth-First Search and Depth-First Search Algorithms

Definition 36. If $u, v \in V(G)$, then the distance between u and v , denoted $d_G(u, v)$, is the length of a shortest u, v -path, or infinity if u and v are in different components of G .

Definition 37 (Diameter). The diameter of a connected graph G is:

$$\text{diam}(G) = \max\{d_G(u, v) : u, v \in V(G)\}$$

Definition 38 (Radius). The radius of a connected graph G is:

$$\text{rad}(G) = \min_{v \in V(G)} \max_{u \in V(G)} d_G(u, v)$$

Lemma 33 (Relationship Between Diameter and Radius). For any connected graph G :

$$\text{rad}(G) \leq \text{diam}(G) \leq 2 \cdot \text{rad}(G)$$

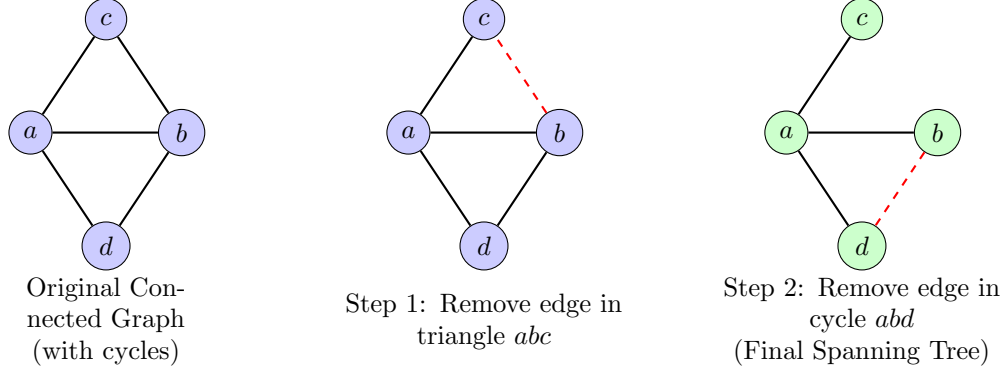


Figure 65: Construction of a spanning tree from a connected graph. Left: Original connected graph with three cycles (abc , abd , and $acbd$). Center: After removing edge bc from the first cycle. Right: Final spanning tree after removing edge bd from the second cycle. The result is a connected acyclic subgraph that includes all vertices of the original graph.

Remark 42. Some common diameter and radius values for standard graphs:

- Complete Graph K_n : $\text{diam}(K_n) = 1$, $\text{rad}(K_n) = 1$
- Path P_n : $\text{diam}(P_n) = n - 1$, $\text{rad}(P_n) = \lceil \frac{n-1}{2} \rceil$
- Cycle C_n : $\text{diam}(C_n) = \lfloor \frac{n}{2} \rfloor$, $\text{rad}(C_n) = \lfloor \frac{n}{2} \rfloor$ (every vertex of C_n has eccentricity $\lfloor \frac{n}{2} \rfloor$)
- Star $K_{1,n-1}$: $\text{diam}(K_{1,n-1}) = 2$, $\text{rad}(K_{1,n-1}) = 1$

Remark 43 (Applications). The diameter and radius have important applications in network analysis:

- Network Efficiency: Lower diameter and radius indicate more efficient communication networks
- Facility Location: The center of a graph (vertices with eccentricity equal to the radius) represents optimal locations for facilities to minimize maximum distance
- Social Networks: Diameter measures the maximum number of intermediaries needed to connect any two people
- Transportation Networks: Radius helps identify central hubs that provide efficient access to all locations

Definition 39 (Distance). The distance between vertices $u, v \in V(G)$, denoted $d_G(u, v)$, is the length of a shortest u, v -path, or infinity if u and v are in different components of G .

Definition 40 (Eccentricity). The eccentricity of a vertex v in a connected graph G , denoted $\text{ecc}(v)$, is the maximum distance from v to any other vertex:

$$\text{ecc}(v) = \max_{u \in V(G)} d_G(u, v)$$

Definition 41. The diameter of a connected graph G is:

$$\text{diam}(G) = \max\{d_G(u, v) : u, v \in V(G)\} = \max_{v \in V(G)} \text{ecc}(v)$$

In other words, the diameter is the maximum eccentricity of any vertex in G .

Definition 42. The radius of a connected graph G is:

$$\text{rad}(G) = \min_{v \in V(G)} \text{ecc}(v)$$

That is, the radius is the minimum eccentricity of any vertex in G .

5.3.1 BFS Algorithm and BFS Tree

Given a graph G and a starting vertex v_1 , we can construct a breadth-first search (BFS) tree as follows:

1. Designate v_1 as the root of the tree.
2. Initialize a queue Q with v_1 and mark v_1 as visited.
3. While Q is not empty:
 - (a) Dequeue a vertex u from Q .
 - (b) For each unvisited neighbor v of u :
 - i. Mark v as visited.
 - ii. Add edge $\{u, v\}$ to the BFS tree T .
 - iii. Enqueue v to Q .

If G is connected, the resulting tree T is a BFS spanning tree of G (in general, T spans the component containing v_1).

Example. Finding a BFS tree in the graph of Figure 66, starting from vertex a :

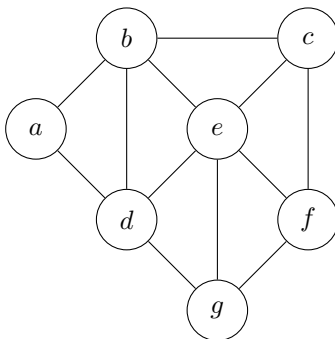


Figure 66: Example diagram for BFS Tree.

1. a is the root, $V(T) = \{a\}$, BFS ordering: (a)
2. Add b : $V(T) = \{a, b\}$, $E(T) = \{\{a, b\}\}$, BFS ordering: (a, b)
3. Add d : $V(T) = \{a, b, d\}$, $E(T) = \{\{a, b\}, \{a, d\}\}$, BFS ordering: (a, b, d)
4. Add c : $V(T) = \{a, b, d, c\}$, $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}\}$
BFS ordering: (a, b, d, c)
5. Add e : $V(T) = \{a, b, d, c, e\}$, $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}\}$
BFS ordering: (a, b, d, c, e)
6. Add g : $V(T) = \{a, b, d, c, e, g\}$
 $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}, \{d, g\}\}$
BFS ordering: (a, b, d, c, e, g)
7. Add f : $V(T) = \{a, b, d, c, e, g, f\}$
 $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}, \{d, g\}, \{c, f\}\}$
BFS ordering: (a, b, d, c, e, g, f)

Final BFS ordering: (a, b, d, c, e, g, f)

Note that some edges from the original graph, such as $\{b, d\}$, $\{c, e\}$, $\{e, f\}$, $\{e, g\}$, and $\{f, g\}$, are not part of the BFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

Example. Finding a BFS tree in the graph of Figure 67, starting from vertex 2:

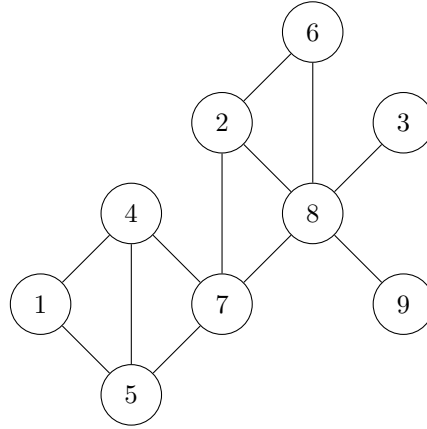


Figure 67: Example diagram for BFS Tree.

As in the previous example, the neighbors of each vertex are examined in increasing (here, numerical) order.

1. 2 is the root, $V(T) = \{2\}$, BFS ordering: (2)
2. Add 6: $V(T) = \{2, 6\}$, $E(T) = \{\{2, 6\}\}$
BFS ordering: (2, 6)
3. Add 7: $V(T) = \{2, 6, 7\}$, $E(T) = \{\{2, 6\}, \{2, 7\}\}$
BFS ordering: (2, 6, 7)
4. Add 8: $V(T) = \{2, 6, 7, 8\}$, $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}\}$
BFS ordering: (2, 6, 7, 8)
5. Add 4: $V(T) = \{2, 6, 7, 8, 4\}$, $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}\}$
BFS ordering: (2, 6, 7, 8, 4)
6. Add 5: $V(T) = \{2, 6, 7, 8, 4, 5\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{7, 5\}\}$
BFS ordering: (2, 6, 7, 8, 4, 5)
7. Add 3: $V(T) = \{2, 6, 7, 8, 4, 5, 3\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{7, 5\}, \{8, 3\}\}$
BFS ordering: (2, 6, 7, 8, 4, 5, 3)
8. Add 9: $V(T) = \{2, 6, 7, 8, 4, 5, 3, 9\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{7, 5\}, \{8, 3\}, \{8, 9\}\}$
BFS ordering: (2, 6, 7, 8, 4, 5, 3, 9)
9. Add 1: $V(T) = \{2, 6, 7, 8, 4, 5, 3, 9, 1\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{7, 5\}, \{8, 3\}, \{8, 9\}, \{4, 1\}\}$
BFS ordering: (2, 6, 7, 8, 4, 5, 3, 9, 1)

Final BFS ordering: (2, 6, 7, 8, 4, 5, 3, 9, 1)

Note that edges $\{1, 5\}$, $\{4, 5\}$, $\{6, 8\}$, and $\{7, 8\}$ are not part of the BFS tree because their endpoints were already discovered when these edges were encountered. The BFS tree provides a minimum-distance path from the root (vertex 2) to every other vertex in the graph.

Lemma 34. If T is a BFS tree in a connected graph rooted at v , then:

$$d_T(v, w) = d_G(v, w)$$

i.e., BFS tree preserves distances from the root.

Definition 43 (BFS Layers). Let G be a graph and T be a BFS tree of G rooted at vertex v . The **layers** of T are defined as:

- Layer $L_0(v) = \{v\}$ (the root)
- Layer $L_i(v) = \{u \in V(G) : d_G(v, u) = i\}$ for $i \geq 1$

That is, the i^{th} layer consists of all vertices at distance exactly i from the root v .

The following properties hold:

1. All edges of G connect vertices either within the same layer or in adjacent layers.
2. The **height** of the BFS tree is the index of the largest non-empty layer.
3. The **eccentricity** of vertex v is equal to the height of the BFS tree rooted at v .
4. The **radius** of G is the minimum eccentricity among all vertices in G .

Graph with vertices $\{a, b, c, d, e, f, g\}$:

Let's calculate the eccentricity of each vertex by finding the height of the BFS tree rooted at that vertex:

Vertex	BFS Layers	Height	Eccentricity
a	$L_0 = \{a\}, L_1 = \{b, d\}, L_2 = \{c, e, g\}, L_3 = \{f\}$	3	3
b	$L_0 = \{b\}, L_1 = \{a, c, d, e\}, L_2 = \{g, f\}$	2	2
c	$L_0 = \{c\}, L_1 = \{b, e, f\}, L_2 = \{a, d, g\}$	2	2
d	$L_0 = \{d\}, L_1 = \{a, b, e, g\}, L_2 = \{c, f\}$	2	2
e	$L_0 = \{e\}, L_1 = \{b, c, d, f, g\}, L_2 = \{a\}$	2	2
f	$L_0 = \{f\}, L_1 = \{c, e, g\}, L_2 = \{b, d\}, L_3 = \{a\}$	3	3
g	$L_0 = \{g\}, L_1 = \{d, e, f\}, L_2 = \{a, b, c\}$	2	2

Properties of Figure 66:

- **Radius:** $\text{rad}(G) = \min\{\text{ecc}(v) : v \in V(G)\} = 2$
- **Diameter:** $\text{diam}(G) = \max\{\text{ecc}(v) : v \in V(G)\} = 3$
- **Central Vertices:** $\{b, c, d, e, g\}$ (vertices with eccentricity equal to radius)
- **Peripheral Vertices:** $\{a, f\}$ (vertices with eccentricity equal to diameter)

Graph with vertices $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$:

Let's examine key vertices to determine the graph's properties:

Vertex	BFS Layers	Height	Eccentricity
1	$L_0 = \{1\}, L_1 = \{4, 5\}, L_2 = \{7\}, L_3 = \{2, 8\}, L_4 = \{3, 6, 9\}$	4	4
2	$L_0 = \{2\}, L_1 = \{6, 7, 8\}, L_2 = \{3, 4, 5, 9\}, L_3 = \{1\}$	3	3
7	$L_0 = \{7\}, L_1 = \{2, 4, 5, 8\}, L_2 = \{1, 3, 6, 9\}$	2	2
8	$L_0 = \{8\}, L_1 = \{2, 3, 6, 7, 9\}, L_2 = \{4, 5\}, L_3 = \{1\}$	3	3

A full analysis would include all vertices, but we've computed enough to determine the key properties: no vertex is adjacent to all others, so every eccentricity is at least 2, and $\text{ecc}(7) = 2$; hence $\text{rad}(G) = 2$, so $\text{diam}(G) \leq 2 \cdot \text{rad}(G) = 4$, and $\text{ecc}(1) = 4$ shows $\text{diam}(G) = 4$.

Properties of Figure 67:

- **Radius:** $\text{rad}(G) = 2$ (achieved by vertex 7)
- **Diameter:** $\text{diam}(G) = 4$ (for example, $d_G(1, 3) = 4$, realized by the path 1-4-7-8-3)
- **A Central Vertex:** Vertex 7 has minimum eccentricity (it is the only central vertex)
- **BFS Tree Height from vertex 2:** 3

Remark 44. Both graphs share interesting structural properties:

- Both have radius 2; Figure 66 has diameter 3 and Figure 67 has diameter 4
- Both satisfy the theorem that $\text{rad}(G) \leq \text{diam}(G) \leq 2 \cdot \text{rad}(G)$; equality holds on the right for Figure 67 ($4 = 2 \cdot 2$) but not for Figure 66 ($3 < 4$)
- Figure 66 has multiple central vertices (5 out of 7 vertices), while Figure 67 has a single central vertex (vertex 7)
- The BFS trees of both graphs efficiently reveal their distance properties

Algorithm 11 Breadth-First Search (BFS)

Require: Graph $G = (V, E)$, starting vertex s

Ensure: Vertices discovered in BFS order and a BFS tree T

```

1: procedure BFS( $G, s$ )
2:   Mark all vertices as unvisited
3:   Create an empty queue  $Q$ 
4:   Create an empty tree  $T$  with  $s$  as its root
5:   Mark  $s$  as visited
6:   Enqueue  $s$  into  $Q$ 
7:   while  $Q$  is not empty do
8:     Dequeue a vertex  $u$  from  $Q$ 
9:     for each neighbor  $v$  of  $u$  do
10:      if  $v$  is unvisited then
11:        Mark  $v$  as visited
12:        Add edge  $(u, v)$  to tree  $T$ 
13:        Enqueue  $v$  into  $Q$ 
14:      end if
15:    end for
16:  end while
17:  return discovered vertices and tree  $T$ 
18: end procedure

```

5.3.2 DFS Algorithm and DFS Tree

Given a graph G and a starting vertex v_1 , we can construct a depth-first search (DFS) tree as follows:

1. Designate v_1 as the root of the tree.
2. Call the recursive procedure DFS-VISIT(v_1):
 - (a) Mark vertex v_1 as visited.
 - (b) For each unvisited neighbor w of v_1 :
 - i. Add edge $\{v_1, w\}$ to the DFS tree T .
 - ii. Recursively call DFS-VISIT(w).

If G is connected, the resulting tree T is a DFS spanning tree of G (in general, T spans the component containing v_1).

DFS Tree Construction for Figure 66, starting from vertex a : (as in the BFS examples, the neighbors of each vertex are examined in alphabetical order)

1. a is the root, $V(T) = \{a\}$, DFS ordering: (a)
2. Add b : $V(T) = \{a, b\}$, $E(T) = \{\{a, b\}\}$, DFS ordering: (a, b)

3. Add c : $V(T) = \{a, b, c\}$, $E(T) = \{\{a, b\}, \{b, c\}\}$
DFS ordering: (a, b, c)
4. Add e : $V(T) = \{a, b, c, e\}$, $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}\}$
DFS ordering: (a, b, c, e)
5. Add d : $V(T) = \{a, b, c, e, d\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, d\}\}$
DFS ordering: (a, b, c, e, d)
6. Add g : $V(T) = \{a, b, c, e, d, g\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, d\}, \{d, g\}\}$
DFS ordering: (a, b, c, e, d, g)
7. Add f : $V(T) = \{a, b, c, e, d, g, f\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, d\}, \{d, g\}, \{g, f\}\}$
DFS ordering: (a, b, c, e, d, g, f)

Final DFS ordering: (a, b, c, e, d, g, f)

Note that the edges $\{a, d\}$, $\{b, d\}$, $\{b, e\}$, $\{c, f\}$, $\{e, f\}$, and $\{e, g\}$ are not part of the DFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

DFS Tree Construction for Figure 67, starting from vertex 2: (neighbors are examined in increasing numerical order)

1. 2 is the root, $V(T) = \{2\}$, DFS ordering: (2)
2. Add 6: $V(T) = \{2, 6\}$, $E(T) = \{\{2, 6\}\}$, DFS ordering: $(2, 6)$
3. Add 8: $V(T) = \{2, 6, 8\}$, $E(T) = \{\{2, 6\}, \{6, 8\}\}$
DFS ordering: $(2, 6, 8)$
4. Add 3: $V(T) = \{2, 6, 8, 3\}$, $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}\}$
DFS ordering: $(2, 6, 8, 3)$
5. Add 7 (after backtracking from 3 to 8): $V(T) = \{2, 6, 8, 3, 7\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 7\}\}$
DFS ordering: $(2, 6, 8, 3, 7)$
6. Add 4: $V(T) = \{2, 6, 8, 3, 7, 4\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 7\}, \{7, 4\}\}$
DFS ordering: $(2, 6, 8, 3, 7, 4)$
7. Add 1: $V(T) = \{2, 6, 8, 3, 7, 4, 1\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 7\}, \{7, 4\}, \{4, 1\}\}$
DFS ordering: $(2, 6, 8, 3, 7, 4, 1)$
8. Add 5: $V(T) = \{2, 6, 8, 3, 7, 4, 1, 5\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 7\}, \{7, 4\}, \{4, 1\}, \{1, 5\}\}$
DFS ordering: $(2, 6, 8, 3, 7, 4, 1, 5)$
9. Add 9 (after backtracking from 5 to 8): $V(T) = \{2, 6, 8, 3, 7, 4, 1, 5, 9\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 7\}, \{7, 4\}, \{4, 1\}, \{1, 5\}, \{8, 9\}\}$
DFS ordering: $(2, 6, 8, 3, 7, 4, 1, 5, 9)$

Final DFS ordering: $(2, 6, 8, 3, 7, 4, 1, 5, 9)$

Note that the edges $\{2, 7\}$, $\{2, 8\}$, $\{4, 5\}$, and $\{5, 7\}$ are not part of the DFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

Remark 45 (DFS Properties Comparison). *Both graphs exhibit interesting structural properties when analyzed through DFS traversal:*

- The DFS trees of both graphs have maximum depth 6: vertex f is at depth 6 in Figure 66, and vertex 5 is at depth 6 in Figure 67
- The DFS trees reveal the connectivity patterns differently than BFS, highlighting branch-like structures
- In Figure 66, DFS from vertex a never needs to backtrack before every vertex is discovered: the DFS tree is the single path $a-b-c-e-d-g-f$
- In Figure 67, DFS from vertex 2 follows 2-6-8-3, backtracks to 8 to explore the branch 8-7-4-1-5, and finally backtracks to 8 again to reach 9
- Both DFS trees contain exactly $n - 1$ edges where n is the number of vertices
- The back edges (edges not in the DFS tree) reveal the cycle structure in both graphs
- While BFS trees minimize the distance from the root, DFS trees tend to create longer paths from the root to certain vertices

Algorithm 12 Depth-First Search (DFS) - Recursive

Require: Graph $G = (V, E)$, starting vertex s

Ensure: Vertices discovered in DFS order and a DFS tree T

```

1: procedure DFS( $G, s$ )
2:   Mark all vertices as unvisited
3:   Create an empty tree  $T$  with  $s$  as its root
4:   DFS-VISIT( $G, s, T$ )
5:   return discovered vertices and tree  $T$ 
6: end procedure
7: procedure DFS-VISIT( $G, u, T$ )
8:   Mark  $u$  as visited
9:   for each neighbor  $v$  of  $u$  do
10:    if  $v$  is unvisited then
11:      Add edge  $(u, v)$  to tree  $T$ 
12:      DFS-VISIT( $G, v, T$ )
13:    end if
14:  end for
15: end procedure

```

5.3.3 Comparison of BFS and DFS Traversals

Graph	BFS Traversal	DFS Traversal
Figure 66	(a, b, d, c, e, g, f)	(a, b, c, e, d, g, f)
Figure 67	$(2, 6, 7, 8, 4, 5, 3, 9, 1)$	$(2, 6, 8, 3, 7, 4, 1, 5, 9)$

Table 2: Comparison of BFS and DFS traversal orders starting from vertices a and 2

Remark 46 (Traversal Pattern Differences). *The BFS and DFS traversals reveal fundamentally different exploration strategies:*

- **Exploration Pattern:** BFS explores vertices in breadth (layer by layer), while DFS explores in depth (branch by branch).
- **Figure 66 Patterns:**
 - In BFS, vertices b and d are discovered first (immediate neighbors of a), then the next layer (c, e, g), and finally f .

- In DFS, the algorithm follows the single branch $a-b-c-e-d-g-f$ without backtracking until every vertex has been discovered; the DFS tree is a path.

- **Figure 67 Patterns:**

- In BFS, immediate neighbors of 2 (vertices 6, 7, 8) are explored first, then their neighbors, creating a layered exploration.
- In DFS, the search follows 2-6-8-3, backtracks to 8 to explore the branch 8-7-4-1-5, and then backtracks to 8 again to discover 9.

Remark 47 (Structural Insights from Traversals). *The traversal trees reveal important structural differences:*

- **Tree Shape:** *BFS trees tend to be broader and less deep, while DFS trees are typically deeper with fewer branches.*
- **Path Lengths:**
 - In Figure 66, the BFS tree guarantees shortest paths from a to all vertices, while the DFS tree reaches d only along the path $a-b-c-e-d$ of length 4, although d is adjacent to a .
 - In Figure 67, the BFS path from 2 to 5 is 2-7-5, of length $2 = d_G(2, 5)$, while the DFS path 2-6-8-7-4-1-5 has length 6.
- **Edge Selection:**
 - BFS includes edges that minimize distance from the root.
 - DFS includes edges that extend the current exploration path as far as possible.
 - For example, in Figure 66, BFS includes edge $\{a, d\}$ while DFS includes $\{f, g\}$.
- **Cycle Detection:** *Both traversals identify different sets of non-tree edges (edges not in the spanning tree), each of which closes a cycle with the tree.*

Remark 48 (Application Implications). *The different traversal orders suggest optimal algorithms for different applications:*

- **BFS Applications:** *Finding shortest paths, computing graph distances and eccentricity, detecting layers/levels in networks.*
- **DFS Applications:** *Detecting cycles, finding connected components, topological sorting, solving maze-like problems.*
- *For Figure 66, BFS would be more appropriate for determining central vertices, while DFS would be better for detecting cycles (for example, the non-tree edge $\{a, d\}$ closes the cycle $a-b-c-e-d-a$ with the DFS tree).*
- *For Figure 67, BFS would be better for finding the shortest path from vertex 2 to vertex 5, while DFS would be a natural choice for detecting cycles, since each non-tree edge found by DFS closes a cycle with the DFS tree (for example, $\{4, 5\}$ closes the cycle 4-1-5-4).*

5.4 Prim's Algorithm

Definition 44 (Subtree). Given a tree $T = (V, E)$, a subtree $T' = (V', E')$ is a connected subgraph of T that is itself a tree. Formally, T' is a subtree of T if:

1. $V' \subseteq V$ (the vertices of T' are a subset of the vertices of T)
2. $E' \subseteq E$ (the edges of T' are a subset of the edges of T)
3. T' is connected
4. T' has no cycles

In other words, a subtree is formed by taking a connected subset of vertices and edges from the original tree while maintaining the tree property.

Remark 49. Any connected subgraph of a tree is itself a tree. This is because removing edges or vertices from a tree cannot create cycles where none existed before.

Remark 50. For a tree with n vertices, if a subtree has k vertices, then it must have exactly $k - 1$ edges.

Example 9. Consider a tree T with vertices $V = \{a, b, c, d, e\}$ and edges $E = \{(a, b), (b, c), (b, d), (d, e)\}$. Examples of valid subtrees include:

- The subtree with vertices $\{a, b, d\}$ and edges $\{(a, b), (b, d)\}$
- The subtree with vertices $\{b, c, d, e\}$ and edges $\{(b, c), (b, d), (d, e)\}$
- The subtree with vertices $\{d, e\}$ and edge $\{(d, e)\}$
- Any single vertex, which forms a trivial subtree

Definition 45 (Prim's Algorithm). Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. It grows the minimum spanning tree one vertex at a time, starting from an arbitrary vertex and always adding the lowest-weight edge that connects a vertex in the tree to a vertex outside the tree. The algorithm proceeds as follows:

1. Select an arbitrary vertex s as the starting point and include it in the MST.
2. Initialize a set S to contain only the starting vertex s : $S = \{s\}$.
3. Initialize a set T of edges to be empty: $T = \emptyset$.
4. While $S \neq V$ (not all vertices are included in the MST):
 - (a) Consider all edges (u, v) where $u \in S$ and $v \in V \setminus S$.
 - (b) Choose the edge (u, v) with minimum weight among all crossing edges.
 - (c) Add vertex v to set S : $S = S \cup \{v\}$.
 - (d) Add edge (u, v) to set T : $T = T \cup \{(u, v)\}$.
5. The set T of edges forms a minimum spanning tree.

This algorithm can be efficiently implemented using a priority queue to select the minimum-weight edge at each step.

Remark 51. Unlike Kruskal's algorithm, Prim's algorithm maintains a single connected subtree at all times. It grows this tree by adding one vertex and one edge at each step, always ensuring connectivity.

Theorem 38 (Prim's Theorem). Let $G = (V, E)$ be a connected, undirected graph with positive edge weights $w : E \rightarrow \mathbb{R}^+$. The tree T constructed by Prim's Algorithm, which iteratively adds the minimum-weight edge connecting a vertex in the current tree to a vertex not yet in the tree until all vertices are included, is a minimum weight spanning tree of G .

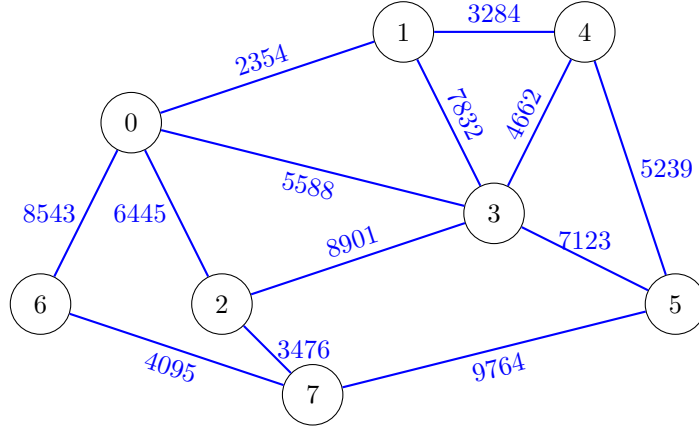


Figure 68: Example of a weighted graph

1. **Initialize:** Start with vertex 0.
 - MST vertices: $\{0\}$
 - MST edges: $\emptyset$
2. **Iteration 1:** Consider all edges from vertex 0 to unvisited vertices.
 - Edge (0,1) with weight 2354
 - Edge (0,2) with weight 6445
 - Edge (0,3) with weight 5588
 - Edge (0,6) with weight 8543
 - **Add** the minimum weight edge: (0,1) with weight 2354
 - MST vertices: $\{0,1\}$
 - MST edges: $\{(0,1)\}$
3. **Iteration 2:** Consider all edges from vertices $\{0,1\}$ to unvisited vertices.
 - Edge (0,2) with weight 6445
 - Edge (0,3) with weight 5588
 - Edge (0,6) with weight 8543
 - Edge (1,3) with weight 7832
 - Edge (1,4) with weight 3284
 - **Add** the minimum weight edge: (1,4) with weight 3284
 - MST vertices: $\{0,1,4\}$
 - MST edges: $\{(0,1), (1,4)\}$
4. **Iteration 3:** Consider all edges from vertices $\{0,1,4\}$ to unvisited vertices.
 - Edge (0,2) with weight 6445
 - Edge (0,3) with weight 5588
 - Edge (0,6) with weight 8543
 - Edge (1,3) with weight 7832
 - Edge (4,3) with weight 4662

- Edge (4,5) with weight 5239
- **Add** the minimum weight edge: (4,3) with weight 4662
- MST vertices: $\{0, 1, 4, 3\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3)\}$

5. **Iteration 4:** Consider all edges from vertices $\{0, 1, 4, 3\}$ to unvisited vertices.

- Edge (0,2) with weight 6445
- Edge (0,6) with weight 8543
- Edge (3,2) with weight 8901
- Edge (3,5) with weight 7123
- Edge (4,5) with weight 5239
- **Add** the minimum weight edge: (4,5) with weight 5239
- MST vertices: $\{0, 1, 4, 3, 5\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5)\}$

6. **Iteration 5:** Consider all edges from vertices $\{0, 1, 4, 3, 5\}$ to unvisited vertices.

- Edge (0,2) with weight 6445
- Edge (0,6) with weight 8543
- Edge (3,2) with weight 8901
- Edge (5,7) with weight 9764
- **Add** the minimum weight edge: (0,2) with weight 6445
- MST vertices: $\{0, 1, 4, 3, 5, 2\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2)\}$

7. **Iteration 6:** Consider all edges from vertices $\{0, 1, 4, 3, 5, 2\}$ to unvisited vertices.

- Edge (0,6) with weight 8543
- Edge (2,7) with weight 3476
- Edge (5,7) with weight 9764
- **Add** the minimum weight edge: (2,7) with weight 3476
- MST vertices: $\{0, 1, 4, 3, 5, 2, 7\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7)\}$

8. **Iteration 7:** Consider all edges from vertices $\{0, 1, 4, 3, 5, 2, 7\}$ to unvisited vertices.

- Edge (0,6) with weight 8543
- Edge (6,7) with weight 4095
- **Add** the minimum weight edge: (6,7) with weight 4095
- MST vertices: $\{0, 1, 4, 3, 5, 2, 7, 6\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7), (6, 7)\}$

Result: The minimum spanning tree has the following edges:

- Edge (0,1) with weight 2354
- Edge (1,4) with weight 3284
- Edge (4,3) with weight 4662

- Edge (4,5) with weight 5239
- Edge (0,2) with weight 6445
- Edge (2,7) with weight 3476
- Edge (6,7) with weight 4095

The total weight of the MST is: $2354 + 3284 + 4662 + 5239 + 6445 + 3476 + 4095 = 29,555$.

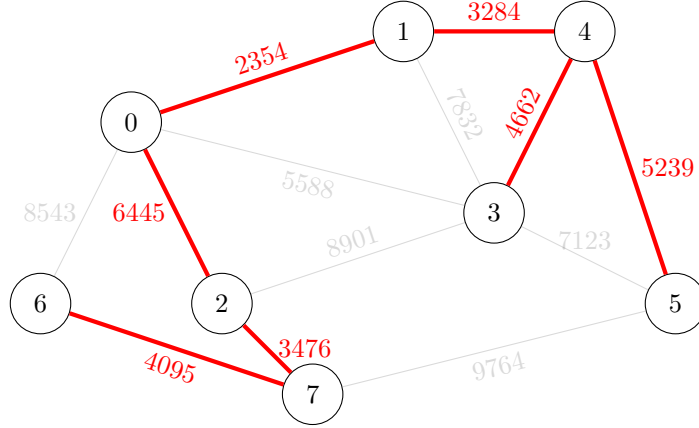


Figure 69: Finding the shortest spanning tree using Prim's Algorithm

Algorithm 13 Prim's Algorithm for Minimum Spanning Tree

Require: Connected, undirected graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$

Ensure: Minimum spanning tree $T = (V, E_T)$

```

1: procedure PRIM( $G, w$ )
2:   Select an arbitrary vertex  $s \in V$  as the starting vertex
3:    $V_T \leftarrow \{s\}$  ▷ Vertices in the MST
4:    $E_T \leftarrow \emptyset$  ▷ Edges in the MST
5:    $PQ \leftarrow \emptyset$  ▷ Priority queue of edges based on weight
6:   for each edge  $(s, x) \in E$  do
7:     Insert edge  $(s, x)$  into  $PQ$  with priority  $w(s, x)$ 
8:   end for
9:   while  $V_T \neq V$  do
10:     $(u, v) \leftarrow \text{Extract-Min}(PQ)$  ▷ Extract edge with minimum weight;  $u \in V_T$ 
11:    if  $v \notin V_T$  then ▷ Otherwise both ends are already in the tree: discard the edge
12:       $V_T \leftarrow V_T \cup \{v\}$  ▷ Add vertex  $v$  to the MST
13:       $E_T \leftarrow E_T \cup \{(u, v)\}$  ▷ Add edge  $(u, v)$  to the MST
14:      for each edge  $(v, x) \in E$  with  $x \notin V_T$  do
15:        Insert edge  $(v, x)$  into  $PQ$  with priority  $w(v, x)$ 
16:      end for
17:    end if
18:  end while
19:  return  $T = (V, E_T)$ 
20: end procedure

```

Lemma 35 (Cut Property for Prim's Algorithm). At each step of Prim's algorithm, the edge selected is a minimum-weight edge crossing the cut between the set S of vertices already in the tree and $V \setminus S$. If the set E_T of edges chosen so far is contained in some minimum spanning tree of the graph, then E_T together with the selected edge is also contained in some minimum spanning tree.

Proof. Let $e = (u, v)$ be the selected edge, with $u \in S$ and $v \in V \setminus S$, and let T be a minimum spanning tree with $E_T \subseteq E(T)$. If $e \in E(T)$, we are done. Otherwise, adding e to T creates a cycle, and this cycle must contain another edge e' that crosses the cut $(S, V \setminus S)$, because the cycle leaves S through e and must return to S . Every edge of E_T has both endpoints in S , so $e' \notin E_T$. Removing e' and adding e gives a graph $T' = T - e' + e$ that is connected (since e' lies on a cycle of $T + e$) and has $|V| - 1$ edges, so T' is a spanning tree containing $E_T \cup \{e\}$. Since e has minimum weight among all crossing edges, $w(e) \leq w(e')$, and hence $w(T') = w(T) - w(e') + w(e) \leq w(T)$. Therefore T' is also a minimum spanning tree, and it contains $E_T \cup \{e\}$. (If all edge weights are distinct, then $w(e) < w(e')$, so in fact e lies in every minimum spanning tree.) $\square$

Proof of Prim's Theorem. Initially $E_T = \emptyset$, which is contained in every minimum spanning tree. By the Cut Property and induction on the number of iterations, after every iteration the set E_T is contained in some minimum spanning tree. Since G is connected, each iteration adds one new vertex together with one edge joining it to the current tree, so the algorithm stops after $|V| - 1$ iterations with a spanning tree (V, E_T) . This tree is contained in some minimum spanning tree T^* , and both have $|V| - 1$ edges, so $(V, E_T) = T^*$ is a minimum spanning tree. $\square$

Lemma 36 (Uniqueness Condition). If all edge weights in a connected graph are distinct, then the minimum spanning tree is unique and will be found by Prim's algorithm regardless of the starting vertex.

5.5 Kruskal's Algorithm

Definition 46 (Kruskal's Algorithm). Kruskal's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. It builds the minimum spanning tree by adding edges in order of increasing weight, skipping edges that would create a cycle. The algorithm proceeds as follows:

1. Sort all edges of the graph $G = (V, E)$ in non-decreasing order of their weights.
2. Initialize a forest F where each vertex in the graph is a separate tree.
3. For each edge (u, v) in the sorted order:
 - (a) If vertices u and v belong to different trees in the forest F :
 - i. Add edge (u, v) to the forest F , merging the trees containing u and v .
 - (b) Otherwise, discard the edge (as it would create a cycle).
4. The forest F is the minimum spanning tree (or forest if the graph is not connected).

This algorithm uses the disjoint-set data structure to efficiently track which tree each vertex belongs to and to perform the merging operations.

Remark 52. *Kruskal's algorithm, unlike Prim's algorithm, does not necessarily build a connected subtree at each step. Instead, it may create a forest of trees that eventually merges into a single minimum spanning tree if the graph is connected.*

Demonstration of Kruskal's Algorithm for Finding a Minimum Spanning Tree

Starting with the weighted graph shown in Figure 68, we apply Kruskal's algorithm to find the minimum spanning tree (MST):

1. **Initialize:** Start with a forest where each vertex is a separate tree.
 - Forest trees: $\{\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\emptyset$
2. **Sort edges:** Arrange all edges by weight in ascending order.
 - Edge (0,1) with weight 2354
 - Edge (1,4) with weight 3284
 - Edge (2,7) with weight 3476
 - Edge (6,7) with weight 4095
 - Edge (3,4) with weight 4662
 - Edge (4,5) with weight 5239
 - Edge (0,3) with weight 5588
 - Edge (0,2) with weight 6445
 - Edge (3,5) with weight 7123
 - Edge (1,3) with weight 7832
 - Edge (6,0) with weight 8543
 - Edge (2,3) with weight 8901
 - Edge (7,5) with weight 9764
3. **Iteration 1:** Consider edge (0,1) with weight 2354.
 - Vertices 0 and 1 are in different trees.
 - **Add** this edge to the MST.

- Forest trees: $\{\{0, 1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\{(0, 1)\}$
4. **Iteration 2:** Consider edge (1,4) with weight 3284.
- Vertices 1 and 4 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2\}, \{3\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\{(0, 1), (1, 4)\}$
5. **Iteration 3:** Consider edge (2,7) with weight 3476.
- Vertices 2 and 7 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2, 7\}, \{3\}, \{5\}, \{6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7)\}$
6. **Iteration 4:** Consider edge (6,7) with weight 4095.
- Vertices 6 and 7 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2, 7, 6\}, \{3\}, \{5\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7)\}$
7. **Iteration 5:** Consider edge (3,4) with weight 4662.
- Vertices 3 and 4 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3\}, \{2, 7, 6\}, \{5\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4)\}$
8. **Iteration 6:** Consider edge (4,5) with weight 5239.
- Vertices 4 and 5 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3, 5\}, \{2, 7, 6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5)\}$
9. **Iteration 7:** Consider edge (0,3) with weight 5588.
- Vertices 0 and 3 are already in the same tree.
 - **Skip** this edge (would create a cycle).
 - Forest trees remain: $\{\{0, 1, 4, 3, 5\}, \{2, 7, 6\}\}$
10. **Iteration 8:** Consider edge (0,2) with weight 6445.
- Vertices 0 and 2 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3, 5, 2, 7, 6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5), (0, 2)\}$
11. **Iteration 9:** All vertices are now in one tree (MST complete).

- Skip remaining edges: (3,5), (1,3), (6,0), (2,3), (7,5)

Result: The minimum spanning tree has the following edges:

- Edge (0,1) with weight 2354
- Edge (1,4) with weight 3284
- Edge (2,7) with weight 3476
- Edge (6,7) with weight 4095
- Edge (3,4) with weight 4662
- Edge (4,5) with weight 5239
- Edge (0,2) with weight 6445

The total weight of the MST is: $2354 + 3284 + 3476 + 4095 + 4662 + 5239 + 6445 = 29,555$.

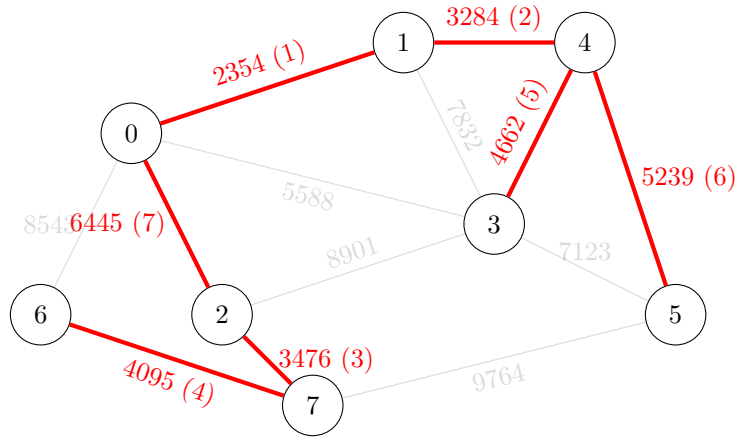


Figure 70: Finding the minimum spanning tree using Kruskal's Algorithm. Numbers in parentheses indicate the order in which edges were added to the MST.

Algorithm 14 Kruskal's Algorithm

```

1: procedure KRUSKAL( $G = (V, E)$ )
2:    $A \leftarrow \emptyset$  ▷ Initialize an empty set for the minimum spanning tree
3:   Create  $|V|$  trees, each containing a single vertex
4:   Sort all edges in  $E$  by weight in non-decreasing order
5:   for each edge  $e = (u, v) \in E$  in sorted order do
6:     if  $u$  and  $v$  are in different trees then
7:        $A \leftarrow A \cup \{e\}$  ▷ Add edge to the minimum spanning tree
8:       Merge the trees containing  $u$  and  $v$ 
9:     end if
10:  end for
11:  return  $A$  ▷ Return the minimum spanning tree
12: end procedure

```

Both Prim's and Kruskal's algorithms are greedy algorithms that find the minimum spanning tree (MST) of a connected, weighted graph. While they produce the same result when edge weights are unique, they differ in their approach and performance characteristics.

5.5.1 Key Similarities

- Both algorithms find a minimum spanning tree with the same total weight
- Both are greedy algorithms, making locally optimal choices at each step
- Both have the same time complexity of $O(E \log V)$ when implemented with appropriate data structures
- Both guarantee an optimal solution

Characteristic	Prim's Algorithm	Kruskal's Algorithm
Strategy	Grows a single tree	Maintains a forest of trees
Starting Point	Begins with a single vertex	Begins with all vertices as separate trees
Edge Selection	Selects the minimum-weight edge connecting the current tree to a new vertex	Selects the minimum-weight edge overall that doesn't create a cycle
Data Structure	Priority queue of edges or vertices	Disjoint-set data structure + sorted edges
Iteration Order	Based on graph structure	Based solely on edge weights
Optimal For	Dense graphs	Sparse graphs
Intermediate State	Always maintains a connected subtree	Maintains a forest that gradually merges

Table 3: Comparison of Prim's and Kruskal's Algorithms

As demonstrated in Figures 69 and 70:

1. Edge Addition Order:

- Prim's algorithm added edges: (0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7), (6, 7)
- Kruskal's algorithm added edges: (0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5), (0, 2)

2. Intermediate Structures:

- Prim's maintained a single connected component throughout
- Kruskal's initially had multiple components: {0, 1, 4}, {2, 7}, etc.

3. Edge Rejection Reasoning:

- Prim's rejected edges that didn't connect to new vertices
- Kruskal's rejected edges that would create cycles

4. Final Result:

- Both algorithms produced the same MST with total weight 29,555
- The resulting tree structure is identical
- Prim's algorithm requires efficient priority queue operations
- Kruskal's algorithm requires efficient disjoint-set operations with union-find
- For graphs with $|V|$ vertices and $|E|$ edges:

- Prim’s with binary heap: $O(E \log V)$
 - Prim’s with Fibonacci heap: $O(E + V \log V)$
 - Kruskal’s with efficient sorting and union-find: $O(E \log E) \approx O(E \log V)$
- Prim’s algorithm is often preferred for dense graphs
 - Kruskal’s algorithm is often preferred for sparse graphs
 - Both are used in network design, clustering algorithms, and approximation algorithms for other problems

Theorem 39 (MST Uniqueness). If all edge weights in a connected graph are distinct, then the minimum spanning tree is unique and will be found by both Prim’s and Kruskal’s algorithms.

Remark 53. *The choice between these algorithms typically depends on the specific graph characteristics and available data structures in the implementation environment.*

Corollary 15 (Exchange Property). Let T be an MST and let f be an edge not in T . Adding f to T creates a unique cycle C , and $w(f) \geq w(e)$ for every edge e of C . Moreover, if $e \neq f$ is an edge of C with $w(e) = w(f)$, then $(T \setminus \{e\}) \cup \{f\}$ is also an MST.

Proof. Let $e \neq f$ be an edge of C , and write $w(H)$ for the total weight of the edges of a subgraph H . The graph $T' = (T \setminus \{e\}) \cup \{f\}$ is connected, because e lies on the cycle C of $T \cup \{f\}$, and it has $|V| - 1$ edges, so T' is a spanning tree with $w(T') = w(T) - w(e) + w(f)$. Since T is an MST, $w(T') \geq w(T)$, that is, $w(f) \geq w(e)$. If $w(e) = w(f)$, then $w(T') = w(T)$, so T' is also an MST. $\square$

Corollary 16 (Growth Property). Both algorithms build the MST by adding one edge at a time, never removing an edge once added.

Aspect	Prim’s Algorithm	Kruskal’s Algorithm
Best for	Dense graphs	Sparse graphs
Time Complexity	$O(E \log V)$ with binary heap	$O(E \log E)$ or $O(E \log V)$
Key Data Structure	Priority Queue	Disjoint-Set (Union-Find)
Growth Pattern	Single tree	Forest of trees

Table 4: Comparison of Prim’s and Kruskal’s Algorithms

Remark 54 (Kruskal’s Algorithm Optimizations). • *Using Union-Find with path compression and union by rank gives near-constant time operations*

- *Pre-sorting edges can be implemented efficiently using counting sort if weights are integers within a small range*

Remark 55 (Prim’s Algorithm Optimizations). • *Using a Fibonacci heap reduces time complexity to $O(E + V \log V)$*

- *For dense graphs, a simple array implementation of the priority queue can be more efficient than a binary heap*

Remark 56 (Algorithm Selection). • *Use Prim’s when $|E|$ is close to $|V|^2$*

- *Use Kruskal’s when $|E|$ is close to $|V|$*

5.6 Dijkstra's Algorithm

Definition 47 (Dijkstra's Algorithm). Let $G = (V, E)$ be a directed graph with a non-negative weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$, and let $s \in V$ be a source vertex. Dijkstra's algorithm computes $\delta(s, v)$, the weight of the shortest path from s to each vertex $v \in V$, as follows:

1. Initialize $d[s] = 0$ and $d[v] = \infty$ for all other vertices $v \in V$.
2. Initialize a priority queue Q containing all vertices, with priority values equal to their d values.
3. Initialize the set of finalized vertices $S = \emptyset$.
4. While $Q \neq \emptyset$:
 - (a) Extract the vertex u with minimum distance value $d[u]$ from Q .
 - (b) Add u to S .
 - (c) For each edge $(u, v) \in E$:
 - i. If $d[v] > d[u] + w(u, v)$, then set $d[v] = d[u] + w(u, v)$ and update v 's position in Q .
5. Upon termination, $d[v] = \delta(s, v)$ for all $v \in V$.

The correctness of the algorithm follows from the invariant that at each iteration, $d[v] = \delta(s, v)$ for all $v \in S$.

Demonstration of Dijkstra's Algorithm for Finding Shortest Paths

Starting with the weighted graph shown in Figure 71, we apply Dijkstra's algorithm to find the shortest paths from source vertex 0 to all other vertices:

1. **Initialize:** Set distance to source vertex 0 as 0, and all other vertices as infinity.
 - Distance values: $d[0] = 0, d[1] = \infty, d[2] = \infty, d[3] = \infty, d[4] = \infty, d[5] = \infty, d[6] = \infty, d[7] = \infty$
 - Previous vertices: $prev[i] = null$ for all i
 - Priority queue $Q = \{0(0), 1(\infty), 2(\infty), 3(\infty), 4(\infty), 5(\infty), 6(\infty), 7(\infty)\}$
 - Visited set $S = \emptyset$
2. **Iteration 1:** Extract vertex 0 (smallest distance).
 - Remove vertex 0 from Q .
 - $S = \{0\}$
 - Update neighbors of vertex 0:
 - Edge (0, 1) with weight 2354: $d[1] = \min(\infty, 0 + 2354) = 2354, prev[1] = 0$
 - Edge (0, 2) with weight 6445: $d[2] = \min(\infty, 0 + 6445) = 6445, prev[2] = 0$
 - Edge (0, 3) with weight 5588: $d[3] = \min(\infty, 0 + 5588) = 5588, prev[3] = 0$
 - Edge (0, 6) with weight 8543: $d[6] = \min(\infty, 0 + 8543) = 8543, prev[6] = 0$
 - Updated priority queue: $Q = \{1(2354), 3(5588), 2(6445), 6(8543), 4(\infty), 5(\infty), 7(\infty)\}$
3. **Iteration 2:** Extract vertex 1 (smallest distance in Q).
 - Remove vertex 1 from Q .
 - $S = \{0, 1\}$
 - Update neighbors of vertex 1:
 - Edge (1, 3) with weight 7832: $d[3] = \min(5588, 2354 + 7832) = 5588$, no change
 - Edge (1, 4) with weight 3284: $d[4] = \min(\infty, 2354 + 3284) = 5638, prev[4] = 1$
 - Updated priority queue: $Q = \{3(5588), 4(5638), 2(6445), 6(8543), 5(\infty), 7(\infty)\}$
4. **Iteration 3:** Extract vertex 3 (smallest distance in Q).

- Remove vertex 3 from Q .
 - $S = \{0, 1, 3\}$
 - Update neighbors of vertex 3:
 - Edge (3, 2) with weight 8901: $d[2] = \min(6445, 5588 + 8901) = 6445$, no change
 - Edge (3, 4) with weight 4662: $d[4] = \min(5638, 5588 + 4662) = 5638$, no change
 - Edge (3, 5) with weight 7123: $d[5] = \min(\infty, 5588 + 7123) = 12711$, $prev[5] = 3$
 - Updated priority queue: $Q = \{4(5638), 2(6445), 6(8543), 5(12711), 7(\infty)\}$
5. **Iteration 4:** Extract vertex 4 (smallest distance in Q).
- Remove vertex 4 from Q .
 - $S = \{0, 1, 3, 4\}$
 - Update neighbors of vertex 4:
 - Edge (4, 5) with weight 5239: $d[5] = \min(12711, 5638 + 5239) = 10877$, $prev[5] = 4$
 - Updated priority queue: $Q = \{2(6445), 6(8543), 5(10877), 7(\infty)\}$
6. **Iteration 5:** Extract vertex 2 (smallest distance in Q).
- Remove vertex 2 from Q .
 - $S = \{0, 1, 3, 4, 2\}$
 - Update neighbors of vertex 2:
 - Edge (2, 7) with weight 3476: $d[7] = \min(\infty, 6445 + 3476) = 9921$, $prev[7] = 2$
 - Updated priority queue: $Q = \{6(8543), 7(9921), 5(10877)\}$
7. **Iteration 6:** Extract vertex 6 (smallest distance in Q).
- Remove vertex 6 from Q .
 - $S = \{0, 1, 3, 4, 2, 6\}$
 - Update neighbors of vertex 6:
 - Edge (6, 7) with weight 4095: $d[7] = \min(9921, 8543 + 4095) = 9921$, no change
 - Updated priority queue: $Q = \{7(9921), 5(10877)\}$
8. **Iteration 7:** Extract vertex 7 (smallest distance in Q).
- Remove vertex 7 from Q .
 - $S = \{0, 1, 3, 4, 2, 6, 7\}$
 - Update neighbors of vertex 7:
 - Edge (7, 5) with weight 9764: $d[5] = \min(10877, 9921 + 9764) = 10877$, no change
 - Updated priority queue: $Q = \{5(10877)\}$
9. **Iteration 8:** Extract vertex 5 (smallest distance in Q).
- Remove vertex 5 from Q .
 - $S = \{0, 1, 3, 4, 2, 6, 7, 5\}$
 - No more updates needed (all vertices visited).

Result: The shortest paths from vertex 0 to all other vertices:

- Path to vertex 1: $0 \rightarrow 1$ with distance 2354
- Path to vertex 2: $0 \rightarrow 2$ with distance 6445

- Path to vertex 3: $0 \rightarrow 3$ with distance 5588
- Path to vertex 4: $0 \rightarrow 1 \rightarrow 4$ with distance 5638
- Path to vertex 5: $0 \rightarrow 1 \rightarrow 4 \rightarrow 5$ with distance 10877
- Path to vertex 6: $0 \rightarrow 6$ with distance 8543
- Path to vertex 7: $0 \rightarrow 2 \rightarrow 7$ with distance 9921

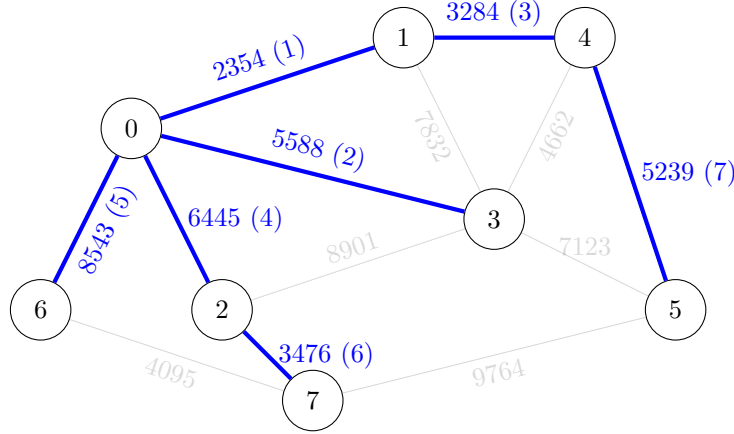


Figure 71: Finding the shortest paths using Dijkstra's Algorithm. Numbers in parentheses indicate the order in which vertices were processed after the source 0; the number for each vertex is written on the tree edge joining it to its predecessor.

Algorithm 15 Dijkstra's Algorithm

```

1: procedure DIJKSTRA( $G = (V, E), w, s$ )
2:   Input: Graph  $G$  with vertices  $V$  and edges  $E$ , weight function  $w : E \rightarrow \mathbb{R}_{\geq 0}$ , source vertex  $s \in V$ 
3:   Output: For each vertex  $v \in V$ , the shortest path distance  $d[v]$  from  $s$  to  $v$ 
4:    $d[v] \leftarrow \infty$  and  $\pi[v] \leftarrow \text{NIL}$  for all  $v \in V$  ▷ Initialize distances and predecessors
5:    $d[s] \leftarrow 0$  ▷ Distance from source to itself is 0
6:    $Q \leftarrow V$  ▷ Initialize priority queue with all vertices
7:    $S \leftarrow \emptyset$  ▷ Set of vertices with finalized shortest path
8:   while  $Q \neq \emptyset$  do
9:      $u \leftarrow \text{Extract-Min}(Q)$  ▷ Extract vertex with minimum distance
10:     $S \leftarrow S \cup \{u\}$  ▷ Add  $u$  to set of processed vertices
11:    for each vertex  $v \in \text{Adj}[u]$  do
12:      if  $d[v] > d[u] + w(u, v)$  then ▷ Relaxation step
13:         $d[v] \leftarrow d[u] + w(u, v)$  ▷ Update distance
14:         $\pi[v] \leftarrow u$  ▷ Update predecessor
15:        Decrease-Key( $Q, v, d[v]$ ) ▷ Update  $v$ 's position in  $Q$ 
16:      end if
17:    end for
18:  end while
19:  return  $d, \pi$  ▷ Return distances and predecessors
20: end procedure

```

Demonstration of Dijkstra's Algorithm for Finding Shortest Paths

Important Note for Readers

Dijkstra's algorithm and Kruskal's algorithm solve fundamentally different graph problems:

- **Kruskal's algorithm** finds a Minimum Spanning Tree (MST) that connects all vertices with the minimum possible total edge weight (29,555 in our example).
- **Dijkstra's algorithm** finds the shortest paths from a single source vertex to all other vertices, creating a Shortest Path Tree (SPT).

These algorithms serve different purposes and will generally produce different tree structures with different total weights. The total weight of the MST (29,555) will not match the sum of the individual shortest path distances.

Theorem 40 (Correctness of Dijkstra's Algorithm). Let $G = (V, E)$ be a weighted graph with non-negative edge weights and $s \in V$ be a source vertex. Upon termination of Dijkstra's algorithm, for each vertex $v \in V$, the distance value $d[v]$ equals $\delta(s, v)$, the weight of the shortest path from s to v .

Theorem 41 (Optimality Principle). Any subpath of a shortest path is itself a shortest path between its endpoints.

Theorem 42 (Triangle Inequality). For any three vertices u , v , and w in a graph with non-negative edge weights, $\delta(u, w) \leq \delta(u, v) + \delta(v, w)$.

Theorem 43 (Negative Edge Weight Limitation). Dijkstra's algorithm may fail to find correct shortest paths if the graph contains any edges with negative weights.

Lemma 37 (Relaxation Property). If at any point during the execution of the algorithm, $d[u] = \delta(s, u)$ for some vertex u , then this equality will persist until termination of the algorithm.

Lemma 38 (Path Relaxation). If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from source $s = v_0$ to vertex v_k , and we relax the edges of p in order, then $d[v_k] = \delta(s, v_k)$ afterward, regardless of any other relaxation steps that occur.

Lemma 39 (Upper-Bound Property). For each vertex $v \in V$, we have $d[v] \geq \delta(s, v)$ at all times during the execution of the algorithm, and once $d[v] = \delta(s, v)$, it never changes.

Corollary 17 (Greedy Choice Property). The vertex u with the minimum distance value $d[u]$ among all unvisited vertices has $d[u] = \delta(s, u)$.

Corollary 18 (Subgraph Property). The shortest paths from a source vertex s to all vertices reachable from s can be chosen so that together they form a tree rooted at s , called a *shortest-path tree*; for example, the predecessor pointers π computed by Dijkstra's algorithm give such a tree. Since shortest paths need not be unique, an arbitrary collection of shortest paths need not form a tree.

Corollary 19 (Convergence Time). For a graph with n vertices, Dijkstra's algorithm converges in exactly n iterations of the main loop.

Remark 57 (Time Complexity). *The standard implementation of Dijkstra's algorithm using a binary heap has time complexity $O((V + E) \log V)$ for a graph with V vertices and E edges, which is $O(E \log V)$ when every vertex is reachable from the source. Using a Fibonacci heap can reduce this to $O(V \log V + E)$.*

Remark 58 (Bidirectional Search). *For finding the shortest path between a single source and a single target, bidirectional Dijkstra's algorithm (which runs the algorithm simultaneously from both endpoints) can significantly reduce the search space.*

Remark 59 (A* Extension). *Dijkstra's algorithm can be extended to the A* algorithm by incorporating a heuristic function that estimates the distance from each vertex to the target, potentially improving performance for specific path-finding problems.*

5.7 Floyd–Warshall Algorithm

Definition 48 (Floyd-Warshall Algorithm). Let $G = (V, E)$ be a weighted directed graph with n vertices, where $V = \{1, 2, \dots, n\}$, whose edge weights may be negative but which contains no negative-weight cycle. The Floyd-Warshall algorithm computes the shortest path distances between all pairs of vertices as follows:

1. Initialize an $n \times n$ matrix $D^{(0)}$ where:
 - $D_{ij}^{(0)} = w(i, j)$ if there is an edge from i to j
 - $D_{ij}^{(0)} = \infty$ if there is no edge from i to j and $i \neq j$
 - $D_{ii}^{(0)} = 0$ for all $i \in V$
2. For $k = 1$ to n :
 - For $i = 1$ to n :
 - For $j = 1$ to n :
 - * $D_{ij}^{(k)} = \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$
3. The final matrix $D^{(n)}$ contains the shortest path distances, where $D_{ij}^{(n)}$ is the length of the shortest path from vertex i to vertex j .

The algorithm has time complexity $\Theta(n^3)$ and space complexity $\Theta(n^2)$, where n is the number of vertices in the graph.

Demonstration of Floyd-Warshall Algorithm for All-Pairs Shortest Paths

Starting with the weighted graph shown in Figure 72, we apply the Floyd-Warshall algorithm to find the shortest paths between all pairs of vertices:

1. **Initialize:** Create the distance matrix $D^{(0)}$ based on the direct edge weights.

- $D_{ii}^{(0)} = 0$ for all i (distance from a vertex to itself is 0)
- $D_{ij}^{(0)} = w(i, j)$ if there is an edge from i to j
- $D_{ij}^{(0)} = \infty$ if there is no edge from i to j and $i \neq j$

Initial distance matrix $D^{(0)}$:

$$\begin{pmatrix} 0 & 2354 & 6445 & 5588 & \infty & \infty & 8543 & \infty \\ \infty & 0 & \infty & 7832 & 3284 & \infty & \infty & \infty \\ \infty & \infty & 0 & 8901 & \infty & \infty & \infty & 3476 \\ \infty & \infty & \infty & 0 & 4662 & 7123 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 5239 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 & 4095 \\ \infty & \infty & \infty & \infty & \infty & 9764 & \infty & 0 \end{pmatrix}$$

2. **Iteration** $k = 0$ to $k = 1$: Consider vertex 0 as an intermediate vertex.

- For each pair of vertices (i, j) , check if path $i \rightarrow 0 \rightarrow j$ is shorter than current path $i \rightarrow j$
- $D_{ij}^{(1)} = \min(D_{ij}^{(0)}, D_{i0}^{(0)} + D_{0j}^{(0)})$

Updated significant values in $D^{(1)}$:

- $D_{12}^{(1)} = \min(\infty, \infty + 6445) = \infty$
- $D_{13}^{(1)} = \min(7832, \infty + 5588) = 7832$
- $D_{16}^{(1)} = \min(\infty, \infty + 8543) = \infty$

(No entry changes, since no edge enters vertex 0.)

3. **Iteration** $k = 1$ to $k = 2$: Consider vertex 1 as an intermediate vertex.

- For each pair of vertices (i, j) , check if path $i \rightarrow 1 \rightarrow j$ is shorter than current path $i \rightarrow j$
- $D_{ij}^{(2)} = \min(D_{ij}^{(1)}, D_{i1}^{(1)} + D_{1j}^{(1)})$

Updated significant values in $D^{(2)}$:

- $D_{04}^{(2)} = \min(\infty, 2354 + 3284) = 5638$
- $D_{13}^{(2)} = \min(7832, 0 + 7832) = 7832$

(Most entries remain unchanged from $D^{(1)}$)

4. **Iteration** $k = 2$ to $k = 3$: Consider vertex 2 as an intermediate vertex.

- Calculate $D_{ij}^{(3)} = \min(D_{ij}^{(2)}, D_{i2}^{(2)} + D_{2j}^{(2)})$
- Notable update: $D_{07}^{(3)} = \min(\infty, 6445 + 3476) = 9921$

5. **Iteration** $k = 3$ to $k = 4$: Consider vertex 3 as an intermediate vertex.

- Calculate $D_{ij}^{(4)} = \min(D_{ij}^{(3)}, D_{i3}^{(3)} + D_{3j}^{(3)})$
- Notable updates: $D_{05}^{(4)} = \min(\infty, 5588 + 7123) = 12711$, $D_{15}^{(4)} = \min(\infty, 7832 + 7123) = 14955$,
 $D_{24}^{(4)} = \min(\infty, 8901 + 4662) = 13563$, and $D_{25}^{(4)} = \min(\infty, 8901 + 7123) = 16024$

6. **Iteration** $k = 4$ to $k = 5$: Consider vertex 4 as an intermediate vertex.

- Calculate $D_{ij}^{(5)} = \min(D_{ij}^{(4)}, D_{i4}^{(4)} + D_{4j}^{(4)})$
- Notable updates: $D_{05}^{(5)} = \min(12711, 5638 + 5239) = 10877$ and $D_{15}^{(5)} = \min(14955, 3284 + 5239) = 8523$

7. **Iterations** $k = 6$ through $k = 8$: Consider remaining vertices as intermediates.

- Continue applying the recurrence relation for remaining vertices
- Vertices 5 and 6 produce no changes; vertex 7 gives $D_{25}^{(8)} = \min(16024, 3476 + 9764) = 13240$ and $D_{65}^{(8)} = \min(\infty, 4095 + 9764) = 13859$
- The algorithm completes after considering all vertices as potential intermediates

Final Distance Matrix $D^{(8)}$:

$$\begin{pmatrix} 0 & 2354 & 6445 & 5588 & 5638 & 10877 & 8543 & 9921 \\ \infty & 0 & \infty & 7832 & 3284 & 8523 & \infty & \infty \\ \infty & \infty & 0 & 8901 & 13563 & 13240 & \infty & 3476 \\ \infty & \infty & \infty & 0 & 4662 & 7123 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 5239 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 13859 & 0 & 4095 \\ \infty & \infty & \infty & \infty & \infty & 9764 & \infty & 0 \end{pmatrix}$$

Result: The shortest path distances between all pairs of vertices are given in the final matrix $D^{(8)}$. For example:

- The shortest path from vertex 0 to vertex 5 has distance 10877.
- The shortest path from vertex 1 to vertex 5 has distance 8523.
- The shortest path from vertex 2 to vertex 5 has distance 13240 (along $2 \rightarrow 7 \rightarrow 5$).
- The shortest path from vertex 7 to vertex 5 has distance 9764.
- Many vertex pairs have no path between them (indicated by ∞).

Algorithm 16 Floyd-Warshall Algorithm

```
1: procedure FLOYDWARSHALL( $G = (V, E), w$ )
2:   Input: Graph  $G$  with vertices  $V = \{1, 2, \dots, n\}$  and edges  $E$ , weight function  $w$ 
3:   Output: Matrix  $D$  where  $D[i, j]$  is the shortest path distance from  $i$  to  $j$ 
4:   Initialize distance matrix  $D$  and successor matrix  $P$ :
5:   for each  $i \in V$  do
6:     for each  $j \in V$  do
7:       if  $i = j$  then
8:          $D[i, j] \leftarrow 0$ ;  $P[i, j] \leftarrow i$ 
9:       else if  $(i, j) \in E$  then
10:         $D[i, j] \leftarrow w(i, j)$ ;  $P[i, j] \leftarrow j$   $\triangleright P[i, j]$ : next vertex after  $i$ 
11:       else
12:         $D[i, j] \leftarrow \infty$ ;  $P[i, j] \leftarrow \text{NIL}$ 
13:       end if
14:     end for
15:   end for
16:   Main algorithm:
17:   for  $k \leftarrow 1$  to  $n$  do
18:     for  $i \leftarrow 1$  to  $n$  do
19:       for  $j \leftarrow 1$  to  $n$  do
20:         if  $D[i, j] > D[i, k] + D[k, j]$  then
21:            $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
22:            $P[i, j] \leftarrow P[i, k]$   $\triangleright$  Optional: store path information
23:         end if
24:       end for
25:     end for
26:   end for
27:   Negative cycle detection (optional):
28:   for  $i \leftarrow 1$  to  $n$  do
29:     if  $D[i, i] < 0$  then
30:       return "Graph contains a negative cycle"
31:     end if
32:   end for
33:   return  $D$   $\triangleright$  Return the matrix of shortest path distances
34: end procedure
```

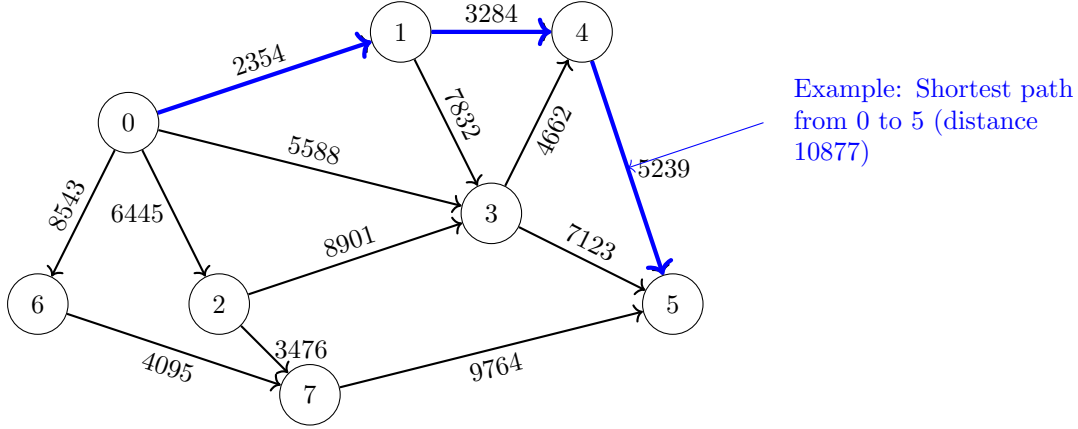


Figure 72: Directed graph for the Floyd-Warshall algorithm. The blue arrows highlight an example shortest path.

5.8 Bellman-Ford Algorithm

Definition 49 (Bellman-Ford Algorithm). Let $G = (V, E)$ be a weighted directed graph with n vertices and m edges, where edge weights may be negative. Let $s \in V$ be a source vertex. The Bellman-Ford algorithm computes the shortest path distances from s to all other vertices in V as follows:

1. Initialize distance values: $d[s] = 0$ and $d[v] = \infty$ for all other vertices $v \in V$.
2. Repeat the following relaxation step $n - 1$ times:
 - For each edge $(u, v) \in E$ with weight $w(u, v)$:
 - If $d[v] > d[u] + w(u, v)$, then $d[v] = d[u] + w(u, v)$ and $prev[v] = u$.
3. Check for negative weight cycles:
 - For each edge $(u, v) \in E$ with weight $w(u, v)$:
 - If $d[v] > d[u] + w(u, v)$, then report that the graph contains a negative weight cycle.

The algorithm has time complexity $O(nm)$ and space complexity $O(n)$, where n is the number of vertices and m is the number of edges in the graph.

Starting with the weighted graph shown in Figure 73, we apply the Bellman-Ford algorithm to find the shortest paths from source vertex 0 to all other vertices:

1. **Initialize:** Set distance to source vertex 0 as 0, and all other vertices as infinity.
 - Distance values: $d[0] = 0, d[1] = \infty, d[2] = \infty, d[3] = \infty, d[4] = \infty, d[5] = \infty, d[6] = \infty, d[7] = \infty$
 - Previous vertices: $prev[i] = null$ for all i
2. **Relaxation Iterations:** Apply relaxation step to all edges $n - 1 = 7$ times.
3. **Iteration 1:**
 - Edge (0, 1) with weight 2354: $d[1] = \min(\infty, 0 + 2354) = 2354, prev[1] = 0$
 - Edge (0, 2) with weight 6445: $d[2] = \min(\infty, 0 + 6445) = 6445, prev[2] = 0$
 - Edge (0, 3) with weight 5588: $d[3] = \min(\infty, 0 + 5588) = 5588, prev[3] = 0$
 - Edge (0, 6) with weight 8543: $d[6] = \min(\infty, 0 + 8543) = 8543, prev[6] = 0$
 - Edge (1, 3) with weight 7832: $d[3] = \min(5588, 2354 + 7832) = 5588$, no change

- Edge (1, 4) with weight 3284: $d[4] = \min(\infty, 2354 + 3284) = 5638$, $prev[4] = 1$
- Edge (2, 3) with weight 8901: $d[3] = \min(5588, 6445 + 8901) = 5588$, no change
- Edge (2, 7) with weight 3476: $d[7] = \min(\infty, 6445 + 3476) = 9921$, $prev[7] = 2$
- Edge (3, 4) with weight 4662: $d[4] = \min(5638, 5588 + 4662) = 5638$, no change
- Edge (3, 5) with weight 7123: $d[5] = \min(\infty, 5588 + 7123) = 12711$, $prev[5] = 3$
- Edge (4, 5) with weight 5239: $d[5] = \min(12711, 5638 + 5239) = 10877$, $prev[5] = 4$
- Edge (6, 7) with weight 4095: $d[7] = \min(9921, 8543 + 4095) = 9921$, no change
- Edge (7, 5) with weight 9764: $d[5] = \min(10877, 9921 + 9764) = 10877$, no change

4. **Updated distances after Iteration 1:**

- $d[0] = 0, d[1] = 2354, d[2] = 6445, d[3] = 5588, d[4] = 5638, d[5] = 10877, d[6] = 8543, d[7] = 9921$

5. **Iteration 2:** Perform the same relaxation steps for all edges.

- No distance values change in this iteration.

6. **Iterations 3-7:** Repeat the relaxation process.

- No further changes in distance values occur in these iterations.

7. **Negative Cycle Check:** Check for negative weight cycles.

- For each edge $(u, v) \in E$, verify that $d[v] \leq d[u] + w(u, v)$
- All edges satisfy this condition, so there are no negative weight cycles.

Result: The shortest paths from vertex 0 to all other vertices:

- Path to vertex 1: $0 \rightarrow 1$ with distance 2354
- Path to vertex 2: $0 \rightarrow 2$ with distance 6445
- Path to vertex 3: $0 \rightarrow 3$ with distance 5588
- Path to vertex 4: $0 \rightarrow 1 \rightarrow 4$ with distance 5638
- Path to vertex 5: $0 \rightarrow 1 \rightarrow 4 \rightarrow 5$ with distance 10877
- Path to vertex 6: $0 \rightarrow 6$ with distance 8543
- Path to vertex 7: $0 \rightarrow 2 \rightarrow 7$ with distance 9921

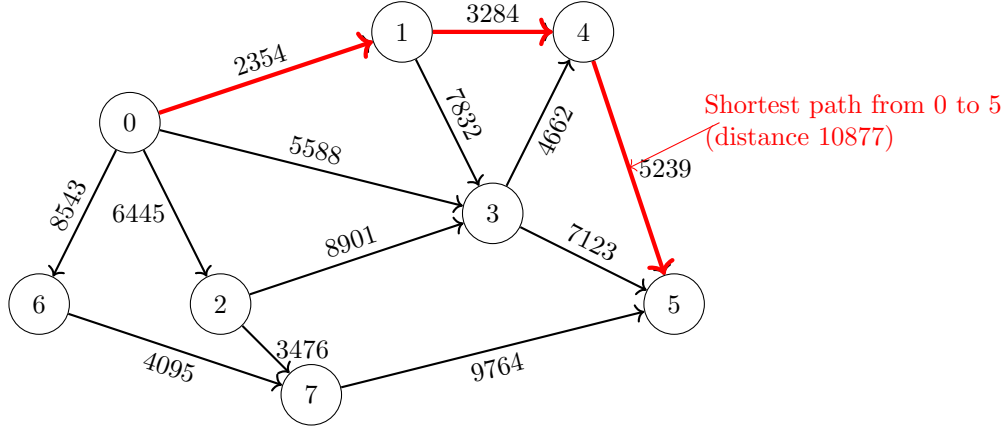


Figure 73: Graph for the Bellman-Ford algorithm. The red arrows highlight the shortest path from vertex 0 to vertex 5.

Algorithm 17 Bellman-Ford Algorithm

```

1: procedure BELLMANFORD( $G = (V, E), w, s$ )
2:   Input: Graph  $G$  with vertices  $V = \{1, 2, \dots, n\}$ , edges  $E$ , weight function  $w$ , source vertex  $s$ 
3:   Output: Array  $d$  where  $d[v]$  is the shortest path distance from  $s$  to  $v$ , or indication of a negative
      cycle
4:   Initialize:
5:   for each vertex  $v \in V$  do
6:      $d[v] \leftarrow \infty$  ▷ Set initial distance to infinity
7:      $predecessor[v] \leftarrow \text{NIL}$  ▷ Initialize predecessor array
8:   end for
9:    $d[s] \leftarrow 0$  ▷ Distance from source to itself is 0
10:  Relaxation phase:
11:  for  $i \leftarrow 1$  to  $|V| - 1$  do ▷ Repeat  $|V| - 1$  times
12:    for each edge  $(u, v) \in E$  do
13:      if  $d[v] > d[u] + w(u, v)$  then ▷ Relaxation step
14:         $d[v] \leftarrow d[u] + w(u, v)$ 
15:         $predecessor[v] \leftarrow u$ 
16:      end if
17:    end for
18:  end for
19:  Negative cycle detection:
20:  for each edge  $(u, v) \in E$  do
21:    if  $d[v] > d[u] + w(u, v)$  then
22:      return "Graph contains a negative weight cycle reachable from source"
23:    end if
24:  end for
25:  return  $d, predecessor$  ▷ Return shortest distances and predecessors
26: end procedure

```

Key Theoretical Results for Bellman-Ford Algorithm

Theorem 44 (Correctness of Bellman-Ford). If a graph contains no negative weight cycles reachable from the source vertex s , then after the Bellman-Ford algorithm terminates, $d[v] = \delta(s, v)$ for all vertices $v \in V$, where $\delta(s, v)$ is the weight of the shortest path from s to v .

Theorem 45 (Negative Cycle Detection). If the graph contains a negative weight cycle reachable from the source vertex s , then the Bellman-Ford algorithm will detect this after the relaxation phase.

Lemma 40 (Path Relaxation Lemma). Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k . If we relax the edges of this path in order, then $d[v_k] = \delta(s, v_k)$ afterward, regardless of any other relaxation steps that occur.

Corollary 20 (Convergence Property). If the graph contains no negative weight cycles reachable from s , then after i iterations of the outer loop, $d[v] = \delta(s, v)$ for every vertex v that has a shortest path from s with at most i edges. (In general, after i iterations $d[v]$ is at most the weight of a lightest s - v path with at most i edges, and it may already be smaller.)

Remark 60 (Comparison with Dijkstra's Algorithm). *While Dijkstra's algorithm is more efficient for graphs with non-negative edge weights ($O(E \log V)$ vs. $O(VE)$ for Bellman-Ford), the Bellman-Ford algorithm can handle graphs with negative edge weights and detect negative cycles, which Dijkstra's algorithm cannot.*

5.8.1 Algorithm Characteristics

Characteristic	Bellman-Ford Algorithm	Floyd-Warshall Algorithm
Primary purpose	Single-source shortest paths	All-pairs shortest paths
Time complexity	$O(V \cdot E)$	$O(V ^3)$
Space complexity	$O(V)$	$O(V ^2)$
Handles negative weights	Yes	Yes
Detects negative cycles	Yes	Yes (with additional checks)
Approach	Edge relaxation	Dynamic programming
Implementation complexity	Moderate	Simple

Table 5: Comparison of key characteristics between Bellman-Ford and Floyd-Warshall algorithms

- **Bellman-Ford Algorithm:** Uses the principle of edge relaxation, where each edge is "relaxed" to potentially improve the shortest path estimate. This process is repeated $|V| - 1$ times to ensure that shortest paths with up to $|V| - 1$ edges are found.
- **Floyd-Warshall Algorithm:** Uses dynamic programming to iteratively find the shortest path between all pairs of vertices. In each iteration k , it considers whether the path from vertex i to vertex j can be improved by going through vertex k .
- **Scalability:**
 - Bellman-Ford performs better on sparse graphs (where $|E| \approx |V|$)
 - Floyd-Warshall performs better when the graph is dense (where $|E| \approx |V|^2$)
- **Multiple Source Queries:**
 - If shortest paths are needed from a single source, Bellman-Ford is more efficient
 - If shortest paths are needed between many or all pairs of vertices, Floyd-Warshall is more efficient than running Bellman-Ford multiple times
- **Incremental Computation:**
 - Bellman-Ford can be terminated early if only short paths are of interest
 - Floyd-Warshall requires the full computation to complete before any results are available

Theorem 46 (Bellman-Ford Relaxation Property). If a graph $G = (V, E)$ contains no negative weight cycles reachable from the source vertex s , then after the Bellman-Ford algorithm performs $|V| - 1$ relaxation passes, $d[v] = \delta(s, v)$ for all vertices $v \in V$.

Theorem 47 (Floyd-Warshall Optimality Property). If G contains no negative-weight cycle, then after the k -th iteration of the Floyd-Warshall algorithm, $d[i, j]$ equals the length of a shortest path from vertex i to vertex j using only vertices in $\{1, 2, \dots, k\}$ as intermediate vertices.

Scenario	Recommendation	Rationale
Single source, sparse graph	Bellman-Ford	$O(V \cdot E)$ is better than $O(V ^3)$ when $ E $ is small
All pairs, dense graph	Floyd-Warshall	Cubic time is acceptable and implementation is simpler
Multiple sources, sparse graph	Multiple Bellman-Ford runs	May be faster than Floyd-Warshall if number of sources is small
Negative cycles detection	Bellman-Ford	More direct detection of negative cycles
Limited memory	Bellman-Ford	$O(V)$ space vs. $O(V ^2)$ space
Simple implementation	Floyd-Warshall	Conceptually simpler with fewer implementation pitfalls

Table 6: Practical recommendations for algorithm selection

5.9 Bipartiteness

Definition 50 (Bipartite Graph). A graph $G = (V, E)$ is bipartite if its vertex set V can be partitioned into two disjoint sets U and W such that every edge in E connects a vertex in U to a vertex in W . Equivalently, a graph is bipartite if and only if it does not contain any cycles of odd length.

Theorem 48 (Characterization of Bipartite Graphs). For a graph G , the following are equivalent:

- (i) G is bipartite;
- (ii) G is 2-colorable, i.e., its vertices can be colored using two colors such that no adjacent vertices share the same color;
- (iii) G contains no cycle of odd length.

Proof. (i) $\Leftrightarrow$ (ii): If G is bipartite with partite sets U and W , color every vertex of U with the first color and every vertex of W with the second; since every edge joins U to W , adjacent vertices receive different colors. Conversely, the two color classes of a proper 2-coloring partition V , and no edge joins two vertices of the same class, so the color classes are partite sets.

(i) $\Rightarrow$ (iii): Let $C = v_0 v_1 \cdots v_{\ell-1} v_0$ be a cycle of G . Consecutive vertices of C lie in different partite sets, so v_i lies in the same partite set as v_0 if and only if i is even. Since $v_{\ell-1}$ is adjacent to v_0 , the index $\ell - 1$ is odd, so ℓ is even.

(iii) $\Rightarrow$ (i): Suppose G has no odd cycle. A graph is bipartite if each of its components is, so we may assume that G is connected. Fix a vertex z and let $U = \{x \in V : d(z, x) \text{ is even}\}$ and $W = \{x \in V : d(z, x) \text{ is odd}\}$; these are exactly the two classes produced by a breadth-first search from z . Suppose, for a contradiction, that some edge xy has both ends in U or both ends in W . Since $|d(z, x) - d(z, y)| \leq 1$ for adjacent x and y , and $d(z, x)$ and $d(z, y)$ have the same parity, we get $d(z, x) = d(z, y) = m$. Let P be a shortest z - x path, let Q be a shortest z - y path, and let u be the last vertex of P that also lies on Q . The z - u portions of P and of Q are shortest z - u paths, so the u - x portion P' of P and the u - y portion Q' of Q both have length $m - d(z, u)$, and by the choice of u they have only the vertex u in common. Moreover $u \neq x$, for otherwise x would be the vertex of Q at distance m from z , namely y . Hence P' , the edge xy , and Q' together form a cycle of length $2(m - d(z, u)) + 1$, which is odd, a contradiction. Therefore every edge joins U and W , and G is bipartite. $\square$

Theorem 49 (Bipartite Edge Bound). A bipartite graph with n vertices can have at most $\lfloor n^2/4 \rfloor$ edges. This bound is tight and achieved by the complete bipartite graph $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$.

Proof. Let $G = (U \cup W, E)$ be a bipartite graph with $|U| = a$, $|W| = b$, and $a + b = n$. The maximum number of edges is $a \cdot b$, which corresponds to a complete bipartite graph. To maximize $a \cdot b$ subject to $a + b = n$, we need $a \approx b \approx n/2$. More precisely, the maximum occurs at $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$, giving $\lfloor n/2 \rfloor \cdot \lceil n/2 \rceil = \lfloor n^2/4 \rfloor$ edges. $\square$

The bipartiteness of a graph can be tested using either Breadth-First Search (BFS) or Depth-First Search (DFS). The following algorithm uses BFS to determine if a graph is bipartite:

1. Initialize:

- Create a color array for all vertices, initially set to "uncolored"
- Choose an arbitrary starting vertex s and color it (e.g., "red")
- Initialize a queue Q with vertex s

2. BFS Traversal:

- While queue Q is not empty:
 - Dequeue a vertex u from Q
 - For each neighbor v of u :
 - * If v is uncolored, assign it the opposite color of u and enqueue it

* If v is colored and has the same color as u , then the graph is not bipartite

3. **Repeat for Disconnected Components:**

- If there are uncolored vertices remaining, pick one and repeat the BFS process

4. **Result:**

- If no conflicts were found, the graph is bipartite
- Otherwise, the graph is not bipartite

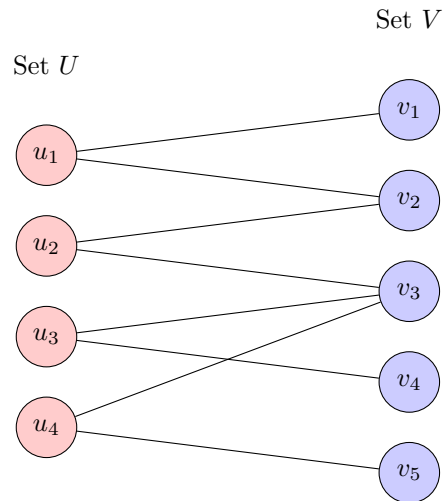


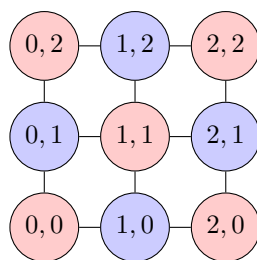
Figure 74: A bipartite graph with partite sets U and V

Example 10. A grid graph $P_m \times P_n$ is bipartite for any positive integers m and n .

Proof. A grid graph can be partitioned into two sets based on the parity of the sum of the row and column indices:

- Set A : Vertices (i, j) where $i + j$ is even
- Set B : Vertices (i, j) where $i + j$ is odd

Any edge in the grid connects a vertex (i, j) to either $(i + 1, j)$ or $(i, j + 1)$. In either case, the sum of the indices changes parity, meaning the edge connects a vertex in set A to a vertex in set B . Thus, there are no edges within set A or within set B , which proves the graph is bipartite. $\square$



3×3 Grid Graph

Figure 75: A 3×3 grid graph with bipartite coloring. Vertices are labeled with their coordinates (i, j) , and colored based on whether $i + j$ is even (red) or odd (blue).

Lemma 41. A cycle graph C_n is bipartite if and only if n is even.

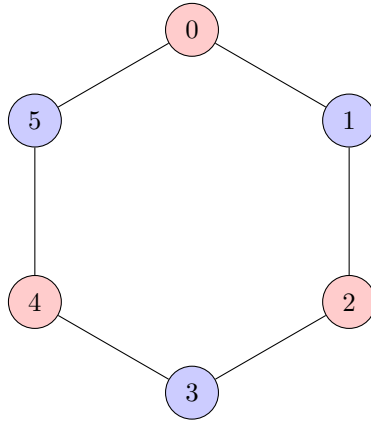
Proof. For a cycle with even length n , we can alternate colors around the cycle, placing adjacent vertices in different sets:

- Set A : Vertices at even positions (0, 2, 4, ...)
- Set B : Vertices at odd positions (1, 3, 5, ...)

Since n is even, when we return to the starting vertex after traversing the cycle, we will have used exactly $n/2$ vertices from set A and $n/2$ vertices from set B . The last vertex (at position $n - 1$) connects to the first vertex (at position 0), and they belong to different sets because $n - 1$ has different parity than 0 when n is even.

For an odd cycle, it's impossible to create such a bipartition because the last vertex would need to connect to the first vertex, but they would have the same parity, forcing them into the same set—which violates the bipartite property. $\square$

Definition 51 (Biregular Graph). A bipartite graph $G = (X \cup Y, E)$ is *biregular* (also called *semiregular*) if all vertices in X have degree k and all vertices in Y have degree l , where generally $k \neq l$; we then say that G is (k, l) -biregular.



Even Cycle C_6

Figure 76: An even cycle (C_6) with bipartite coloring. Even-indexed vertices are colored red, and odd-indexed vertices are colored blue.

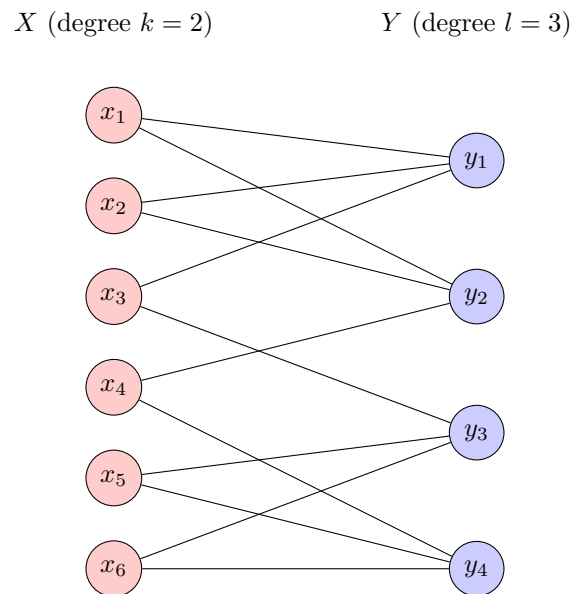
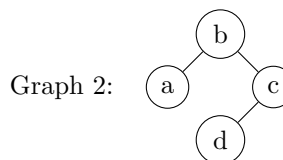
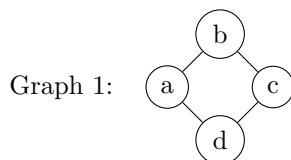


Figure 77: A $(2, 3)$ -biregular bipartite graph with $|X| = 6$ and $|Y| = 4$

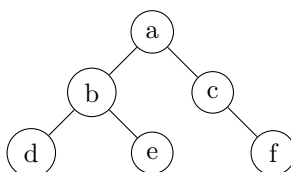
5.10 Chapter 5 Exercises

5.10.1 Visual Questions

- For each of the following graphs, determine whether it is a tree and explain your reasoning:

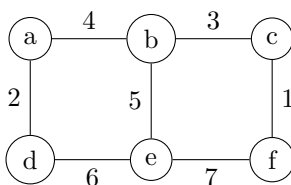


- Consider the following tree:



Find all leaves, identify the center of the tree, and determine its diameter.

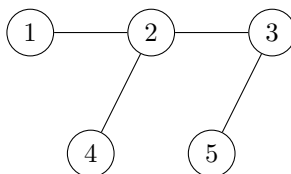
- Draw all non-isomorphic trees with exactly 5 vertices.
- Given the following weighted graph, find a minimum spanning tree and calculate its total weight:



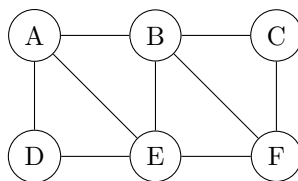
- Determine the Prüfer code for the following tree:

(Hint 1: The Prüfer algorithm finds a unique sequence of $n-2$ integers for a labeled tree with n vertices. To construct the Prüfer code: repeatedly find the leaf with the smallest label, add its neighbor's label to the code, and remove the leaf from the tree. Continue until only 2 vertices remain.)

(Hint 2: The Prüfer algorithm's pseudo code is given by: 1. Initialize an empty sequence P 2. While the tree has more than 2 vertices: a. Find the leaf with the smallest label, call it v b. Let u be the only neighbor of v c. Add the label of u to sequence P d. Remove vertex v and its incident edge from the tree 3. Return the sequence P)

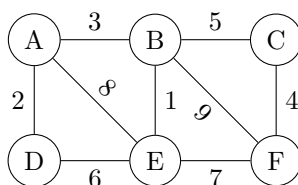


- Draw a graph with 6 vertices that shows the difference between a spanning tree and a minimum spanning tree. Label the edge weights clearly.
- Given the following graph:

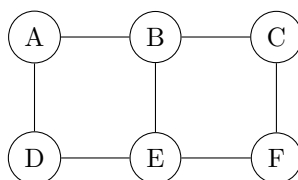


Illustrate step by step how Kruskal's algorithm would find a minimum spanning tree. Assume all edges have weight 1.

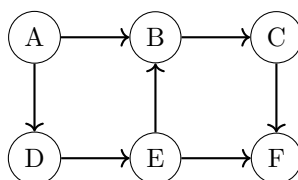
8. Draw a graph with exactly one bridge and explain how to identify it using the depth-first search algorithm. Show the discovery time and low values for each vertex.
9. Construct a graph with 7 vertices where the removal of some pair of vertices disconnects the graph, but the removal of any single vertex does not. Explain how this relates to vertex connectivity.
10. Illustrate Prim's algorithm step-by-step on the following weighted graph, starting from vertex A:



11. For the following graph, perform both BFS and DFS starting from vertex A. Show the resulting traversal order and the BFS/DFS trees:

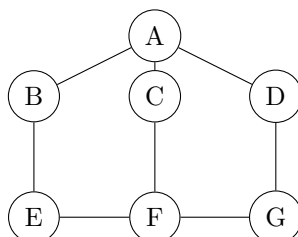


12. Consider the following directed graph:

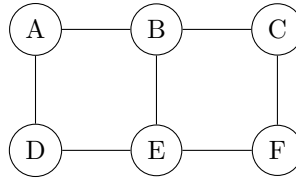


Perform a DFS traversal starting from vertex A and classify each edge as a tree edge, back edge, forward edge, or cross edge.

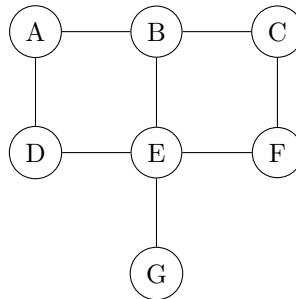
13. Explain the difference between BFS and DFS traversals by showing both traversal trees for the following graph, starting from vertex A:



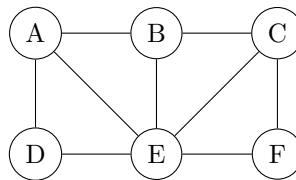
14. Use BFS to determine if the following graph is bipartite. If it is, show the two partite sets with different colors:



15. Find all articulation points in the following graph using the DFS algorithm. Show the discovery time and low value for each vertex:



16. Draw a directed graph with at least 8 vertices that has exactly 3 strongly connected components. Use DFS to identify these components.
17. Create a graph where the BFS tree and DFS tree (starting from the same vertex) are as different as possible. Highlight both trees in your diagram, and explain why the two trees must always share at least one edge.
18. Illustrate the concept of DFS numbering and low values by drawing a graph with at least one bridge and one articulation point. Show the DFS tree with discovery times and low values for each vertex.
19. Draw an example showing how BFS can be used to find the shortest path between two vertices in an unweighted graph. Label the distances from the source vertex.
20. For the following graph, demonstrate how to detect cycles using DFS:



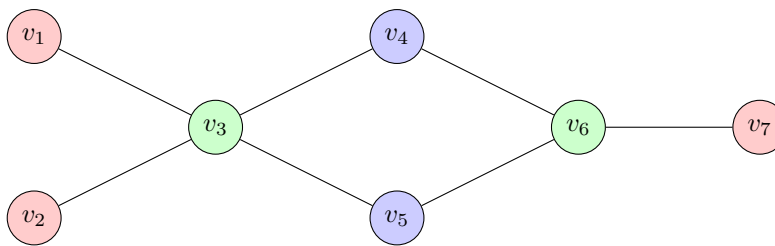
6 Structure of Connected Graphs

6.1 Blocks, Cut Vertices, and Connectedness

Definition 52 (Connected Graphs). A graph G is *connected* if for every pair of vertices $u, v \in V(G)$, there exists a path from u to v in G .

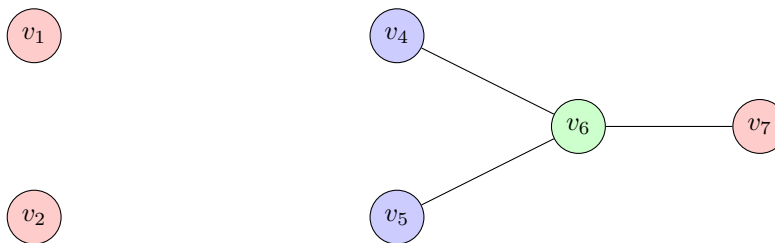
Definition 53 (Disconnected Graphs). A graph G is *disconnected* if there exist vertices $u, v \in V(G)$ such that there is no path from u to v in G .

Definition 54 (Cut Vertex). Let G be a connected graph. A vertex $v \in V(G)$ is called a *cut vertex* if the graph $G - \{v\}$ is disconnected.



Graph G with cut vertices v_3 and v_6

Figure 78: A graph G with cut vertices v_3 and v_6 (colored green). Removing either vertex disconnects the graph.



$G - \{v_3\}$ is disconnected

Figure 79: When vertex v_3 is removed, the graph becomes disconnected, with three components $\{v_1\}$, $\{v_2\}$, and $\{v_4, v_5, v_6, v_7\}$.

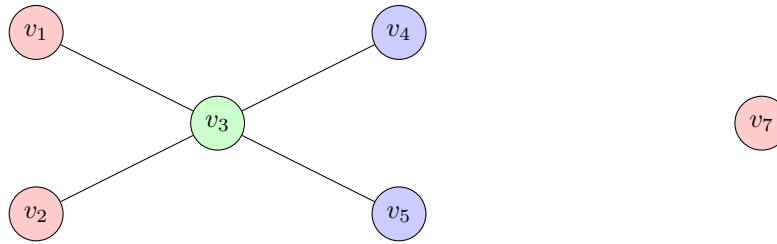
Definition 55 (Block). A block in a graph G is a maximal connected subgraph with no cut vertex.

Lemma 42. Every block of a graph is either a maximal 2-connected subgraph, or a bridge (an edge whose removal increases the number of components), or an isolated vertex.

Remark 61. The blocks of a graph form a partition of its edges, meaning each edge belongs to exactly one block.

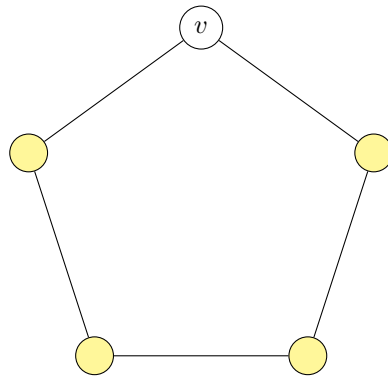
Remark 62. A vertex can belong to multiple blocks, and such a vertex must be a cut vertex of the graph.

Example 11. Consider the cycle graph C_5 shown below:



$G - \{v_6\}$ is disconnected

Figure 80: When vertex v_6 is removed, the graph becomes disconnected, isolating vertex v_7 .



C_5 has no cut vertices.

$C_5 - \{v\}$ is P_4

Figure 81: Cycle graph C_5 with vertex v marked; the other four (highlighted) vertices form $C_5 - \{v\}$. Removing any vertex from C_5 results in a path graph P_4 .

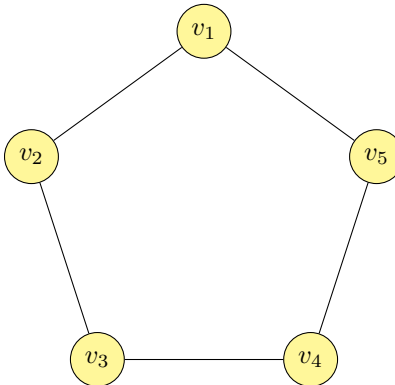


Figure 82: C_5 : The entire graph is a single block
For the cycle graph C_5 :

- C_5 has no cut vertices.
- The entire graph is a single block (highlighted in yellow).
- For any vertex v in C_5 , the graph $C_5 - \{v\}$ is a path P_4 , which is connected.

Lemma 43. In a cycle graph C_n with $n \geq 3$, the entire graph forms a single block.

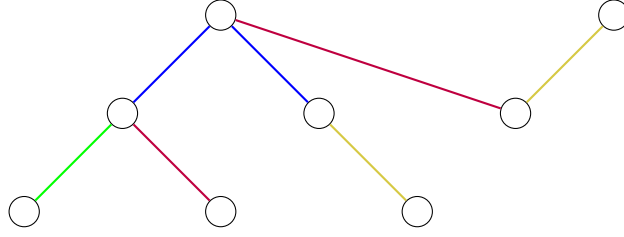
Remark 63. The blocks of a connected graph are the "building blocks" that can be used to reconstruct the

original graph. Understanding the block structure of a graph is fundamental to many graph algorithms and applications.

For a general connected graph G , we can identify its blocks as follows:

1. Identify all cut vertices of G .
2. Remove the cut vertices (one at a time) to find the connected components.
3. Each maximal subgraph with no cut vertices is a block.

Example 12. If T is a tree with at least two vertices, then all blocks are K_2 (i.e., single edges).



Graph T

Figure 83: This graph T illustrates a fundamental property of trees in graph theory. It is a connected acyclic graph with multiple vertices joined by edges that are highlighted in different colors (blue, purple, green, and yellow). Each colored edge represents a distinct block in the graph.

Definition 56 (Block-Cutpoint Graph). Given a connected graph G , with blocks $B_1, B_2, \dots, B_k$ and cut vertices $v_1, v_2, \dots, v_c$, we form a new graph $\mathcal{B}(G)$ (also called the block-cutpoint graph of G) where:

$$V(\mathcal{B}(G)) = \{B_1, B_2, \dots, B_k, v_1, \dots, v_c\} \quad (33)$$

$$E(\mathcal{B}(G)) = \{\{B_i, v_j\} : v_j \in B_i\} \quad (34)$$

That is, $\mathcal{B}(G)$ has vertices corresponding to the blocks and cut vertices of G , and there is an edge between a block vertex B_i and a cut vertex v_j if and only if the cut vertex v_j belongs to the block B_i in the original graph G .

Theorem 50. For any connected graph G , its block-cutpoint graph $\mathcal{B}(G)$ is a tree.

Example 13. Consider a graph G that resembles a dumbbell (two triangles connected by a path). The block-cutpoint graph $\mathcal{B}(G)$ would have vertices representing the two triangular blocks, the edges of the connecting path as blocks, and the cut vertices that connect these blocks.

Example 14. Consider the following graph G with three blocks and two cut vertices:

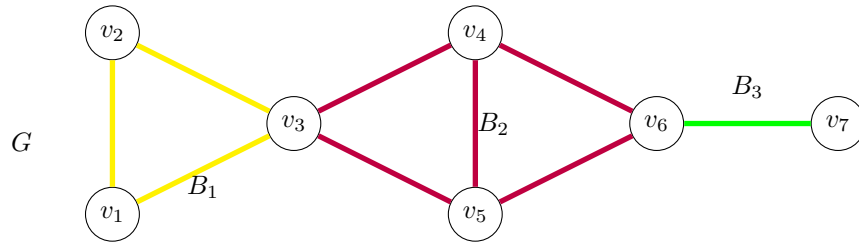


Figure 84: The cut vertices are $\{v_3, v_6\}$. The blocks are B_1, B_2, B_3 . The block-cutpoint graph $\mathcal{B}(G)$ is shown below.

Remark 64. The block-cutpoint graph $\mathcal{B}(G)$ is a bipartite graph, with one partition consisting of the blocks and the other partition consisting of the cut vertices.

Remark 65. The block-cutpoint graph is a useful tool for analyzing the structure of a graph. Many graph algorithms, such as finding bridges and articulation points, can be simplified by working with the block-cutpoint graph.

Theorem 51. A graph G is 2-connected if and only if it has at least 3 vertices and $\mathcal{B}(G)$ consists of a single block vertex (with no cut vertices).

Corollary 21. A graph G has no cut vertices (no vertex whose removal increases the number of components) if and only if every component H of G is K_1 , K_2 , or 2-connected; equivalently, if and only if $\mathcal{B}(H)$ consists of a single vertex for every component H of G .

Block-Cutpoint Graph $\mathcal{B}(G)$

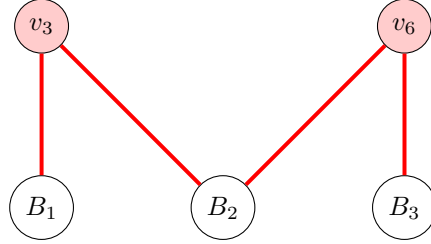


Figure 85: The block-cutpoint graph represents the structure of G by showing how its blocks connect through cut vertices. Note that $\mathcal{B}(G)$ forms a tree with block vertices (white) and cut vertices (red).

Theorem 52 (Tree Structure). The block-cutpoint graph $\mathcal{B}(G)$ of a connected graph G is always a tree.

Theorem 53 (Bipartite Property). The block-cutpoint graph $\mathcal{B}(G)$ is bipartite, with one partition consisting of block vertices and the other of cut vertices.

Lemma 44 (Leaf Property). Every leaf vertex in $\mathcal{B}(G)$ corresponds to a block in G , never to a cut vertex.

Lemma 45 (Path Uniqueness). For any two blocks B_i and B_j in G , there exists a unique path between the corresponding block vertices in $\mathcal{B}(G)$.

Remark 66 (Block Separation). If v is a cut vertex in G , then removing v from G increases the number of connected components by exactly the number of blocks containing v minus 1.

Remark 67 (Reconstruction). The original graph G can be reconstructed from its block-cutpoint graph $\mathcal{B}(G)$ if we know the structure of each block.

Corollary 22 (2-Connectivity). A graph G is 2-connected if and only if G has at least three vertices and $\mathcal{B}(G)$ consists of a single block vertex.

Corollary 23 (Distance Property). The distance between any two blocks B_i and B_j in $\mathcal{B}(G)$ is equal to twice the number of cut vertices that every path from a vertex in B_i to a vertex in B_j in G must pass through.

Theorem 54. For any connected graph G , the block graph $\mathcal{B}(G)$ is a tree.

Proof. We need the block graph $\mathcal{B}(G)$ to be connected. If $\mathcal{B}(G)$ has two components H_1 and H_2 , then for any block $B \in H_1$ and any block $B' \in H_2$, pick any vertex $v \in V(B)$ and $v' \in V(B')$, there is no v - v' path in G . So v and v' are in different components of G , contradicting that G is connected.

To show $\mathcal{B}(G)$ is a tree, we must show $\mathcal{B}(G)$ has no cycles.

Suppose C is a cycle in $\mathcal{B}(G)$. Then

$$\begin{aligned} V(C) &= \{v_1, B_1, v_2, B_2, \dots, v_r, B_r\} \\ E(C) &= \{\{v_1, B_1\}, \{B_1, v_2\}, \{v_2, B_2\}, \dots, \{v_r, B_r\}, \{B_r, v_1\}\} \end{aligned}$$

We claim $B = B_1 \cup B_2 \cup \dots \cup B_r \subseteq G$ has no cut vertex. This is a contradiction since each B_i is a block, but properly contained in B (note that $r \geq 2$, because the bipartite graph $\mathcal{B}(G)$ has no multiple edges, so C has length at least 4).

Indeed, B is connected, since consecutive blocks B_{i-1} and B_i share the vertex v_i (indices modulo r). Let x be any vertex of B . Each $B_i - x$ is connected, because a block has no cut vertex. If x is not one of $v_1, \dots, v_r$, then consecutive blocks still share their common vertices, so $B - x$ is connected. If $x = v_i$, then the blocks $B_i, B_{i+1}, \dots, B_r, B_1, \dots, B_{i-1}$ are still linked in this order by the remaining vertices $v_{i+1}, \dots, v_r, v_1, \dots, v_{i-1}$, so again $B - x$ is connected. So B has no cut vertex.

So $\mathcal{B}(G)$ has no cycles; it is a tree. □

intuition: This proof demonstrates why the block graph must be a tree. When we construct a block graph from our original graph, we’re creating a structure that shows how the blocks and cut vertices are connected to each other.

First, the block graph must be connected because the original graph is connected. If the block graph weren’t connected, that would mean some blocks in the original graph couldn’t be reached from others, contradicting our assumption that the original graph is connected. The key insight is understanding why the block graph cannot contain cycles. If it did contain a cycle, this cycle would alternate between blocks and cut vertices (since in the block graph, blocks connect only to cut vertices and vice versa). Following this cycle in the original graph would give us a sequence of blocks $B_1, B_2, \dots, B_r$, where consecutive pairs share a cut vertex. If we take the union of all these blocks, $B = B_1 \cup B_2 \cup \dots \cup B_r$, we’d get a larger subgraph. The contradiction arises because this union B would have no cut vertex! Any vertex that was a cut vertex for one of the original blocks is no longer a cut vertex in the union, because the cycle structure provides an alternate path around any potential cut vertex. This contradicts the maximality in the definition of a block (a maximal connected subgraph without a cut vertex). We’ve found a strictly larger connected subgraph without a cut vertex containing one of our blocks, which is impossible by the definition of maximality. Since we’ve reached a contradiction, our assumption must be wrong—the block graph cannot contain any cycles. And a connected graph with no cycles is precisely the definition of a tree. In essence, the tree structure of the block graph reflects how the original graph is “held together” by its cut vertices. Each block is like a solid component, and the cut vertices are the connectors that bind these components together in a tree-like arrangement.

A **tree of cycles** is a connected graph whose blocks are all cycles or K_2 .

Theorem 55. Let G be a connected graph with at least two vertices containing no Θ -graph. Then G is a tree of cycles; that is, every block of G is a cycle or K_2 . (Such a graph need not be a tree or a cycle: two triangles sharing one vertex contain no Θ -graph.)

A **Θ -graph** is a graph consisting of three uv -paths P , Q and R such that

$$\begin{aligned} V(P) \cap V(Q) &= \{u, v\} \\ V(Q) \cap V(R) &= \{u, v\} \\ V(R) \cap V(P) &= \{u, v\} \end{aligned}$$

Proof. Since G is connected and has at least two vertices, no block of G is an isolated vertex, so every block of G is K_2 or 2-connected. By the definition of a tree of cycles, it is enough to prove that every block of G is a cycle or K_2 .

Let B be a block of G . If $B \neq K_2$ and B is not a cycle, then B is 2-connected, so every vertex has degree at least 2 in B ; since a connected graph in which every vertex has degree 2 is a cycle, B has a vertex u with $d_B(u) \geq 3$.

Block B has a cycle C containing u (a 2-connected graph has no bridge, and an edge that is not a bridge lies on a cycle). Only two edges of C meet u , so there is an edge $uw \in E(B) \setminus E(C)$.

If $w \in V(C)$, then the edge uw and the two uw -paths Q and R with $Q \cup R = C$ form a Θ -graph. This is a contradiction.

If $w \notin V(C)$, then, since u is not a cut vertex of B , $B - \{u\}$ is connected. So there is a path in $B - \{u\}$ from w to a vertex of $C - \{u\}$; let P' be a shortest such path and let v be its last vertex. Then P' meets C only in v , so the path P consisting of the edge uw followed by P' is a uv -path that meets C only in u and v .

Now P , Q , R form a Θ -graph, where $Q \cup R = C$ and Q and R are uv -paths. This is a contradiction.

So $B = K_2$ or B is a cycle. □

In other words: This proposition reveals something fascinating about graphs with a certain structural constraint. It tells us that if our graph doesn’t contain a specific forbidden pattern (three internally disjoint paths connecting the same two points), then every block of the graph must be very simple - either just a single edge or a single cycle.

Think about the forbidden pattern this way: imagine two cities connected by three completely separate roads. This creates a structure with multiple alternate routes between the same two points. What our Theorem 55 says is that if a network doesn’t have any such triple-path connections anywhere, then the

network is built from single links and simple loops, glued together at cut vertices in a tree-like fashion. The network need not be a single tree or a single loop: two triangles sharing one vertex contain no triple-path connection, yet form neither a tree nor a cycle.

The proof uses the block structure of the graph. Remember that the blocks of a graph are its maximal connected subgraphs without a cut vertex; in a connected graph with at least two vertices, every block is either a single edge or a maximal 2-connected piece of the graph. The key insight is showing that each block in our graph must be either just a single edge or a cycle - nothing more complex is possible.

Here's why: If any block were more complex than just an edge or cycle, it would contain a vertex with at least three neighbors inside the block. This high-degree vertex would be part of some cycle of the block, and one of its edges would not be on this cycle. If that edge joins two vertices of the cycle, it is a chord, and the chord together with the two arcs of the cycle forms the triple-path pattern. Otherwise the edge leads to an "outside neighbor"; since the block is 2-connected, we could find a path that avoids the high-degree vertex, connecting its "outside neighbor" to the original cycle. But this would create exactly the triple-path pattern we're not allowed to have!

This contradiction tells us each block must be just an edge or a cycle. When we connect these simple blocks together at cut vertices, we get what's called a "tree of cycles" - essentially a tree-like structure where the "nodes" are either single edges or cycles.

Since the block graph forms a tree (from our previous theorem), the entire graph is either a single block (so just a cycle or a single edge) or several such blocks arranged in a tree structure. In particular, the graph is a tree exactly when all of its blocks are single edges, and it is a cycle exactly when it consists of a single block that is a cycle; in general it mixes both kinds of blocks.

Conversely, a tree of cycles contains no Θ -graph: a Θ -graph has no cut vertex, so it would have to lie inside a single block, and neither an edge nor a cycle contains a Θ -graph. So this result completely characterizes the connected graphs with at least two vertices that avoid the triple-path pattern: they are exactly the trees of cycles.

6.2 Menger's Theorem

Theorem 56 (Menger's Theorem). Let G be a graph and u, v be two non-adjacent vertices in G . Then the minimum number of vertices whose removal disconnects u from v equals the maximum number of pairwise internally vertex-disjoint u - v paths in G .

Theorem 57 (Menger's Theorem - Vertex Form). For any graph G , and non-adjacent $u, v \in V(G)$, $\kappa(u, v)$ is equal to the maximum number of uv -paths $P_1, P_2, \dots, P_k$ such that

$$V(P_i) \cap V(P_j) = \{u, v\}$$

for all $i \neq j$.

Proof. Let $\kappa(u, v)$ be the minimum size of a uv -separator, and let $\pi(u, v)$ be the maximum number of internally vertex-disjoint uv -paths. We need to show that $\kappa(u, v) = \pi(u, v)$.

First, we prove that $\kappa(u, v) \geq \pi(u, v)$.

Consider any set of $\pi(u, v)$ internally vertex-disjoint uv -paths. Any uv -separator S must include at least one vertex from each path (excluding u and v), otherwise there would still be a uv -path after removing S . Since the paths are internally vertex-disjoint, S must contain at least $\pi(u, v)$ vertices. Therefore, $\kappa(u, v) \geq \pi(u, v)$.

Next, we prove that $\kappa(u, v) \leq \pi(u, v)$. It is convenient to prove a more general statement about sets of vertices. Let H be a graph and let $A, B \subseteq V(H)$. An A - B path is a path whose first vertex is its only vertex in A and whose last vertex is its only vertex in B (a single vertex of $A \cap B$ is an A - B path of length 0). A set $X \subseteq V(H)$ separates A from B if every A - B path in H contains a vertex of X .

Claim. If every set separating A from B in H has at least k vertices, then H contains k pairwise vertex-disjoint A - B paths.

We prove the claim by induction on $|E(H)|$, for all A, B and k at once. (This is the first proof of Menger's Theorem in Diestel's *Graph Theory*, Theorem 3.3.1.)

Base case: If H has no edges, then the A - B paths are exactly the single vertices of $A \cap B$, so $A \cap B$ separates A from B . Hence $|A \cap B| \geq k$, and these single vertices give k disjoint A - B paths.

Inductive step: Let $e = xy$ be an edge of H , and suppose, for a contradiction, that H does not contain k disjoint A – B paths. Let H/e be the graph obtained by contracting e , that is, by replacing x and y with a single new vertex v_e that is adjacent to every other vertex adjacent to x or to y ; the vertex v_e counts as a vertex of A (of B) if x or y lies in A (in B). If H/e contained k disjoint A – B paths, at most one of them would contain v_e . Replacing v_e on that path by x , by y , or by the edge xy (suitably chosen) gives a path in H that starts in A and ends in B , and hence contains an A – B path, while the other paths avoid x and y and are A – B paths in H . This would give k disjoint A – B paths in H , so H/e does not contain k disjoint A – B paths either. Since H/e has fewer edges than H , the induction hypothesis gives a set Y of fewer than k vertices separating A from B in H/e . The set Y must contain v_e , since otherwise Y would separate A from B in H , which is impossible because $|Y| < k$. Therefore $X = (Y \setminus \{v_e\}) \cup \{x, y\}$ separates A from B in H , and $|X| \leq k$; by hypothesis, $|X| = k$.

Now consider $H - e$. Every A – B path P in H meets X , and the initial segment of P up to its first vertex in X does not use e , because both ends of e lie in X . So every set separating A from X in $H - e$ also separates A from B in H , and therefore has at least k vertices. Since $H - e$ has fewer edges than H , the induction hypothesis gives k disjoint A – X paths in $H - e$, and in the same way k disjoint X – B paths in $H - e$. As $|X| = k$, each vertex of X is an end of exactly one path of each family. A path of the first family and a path of the second family meet only in X , since a common vertex outside X would give an A – B walk in H avoiding X . Joining the two families at the vertices of X yields k disjoint paths in H , each starting in A and ending in B ; each of them contains an A – B path, so H contains k disjoint A – B paths, a contradiction. This proves the claim.

Finally, let u and v be non-adjacent vertices of G and let $k = \kappa(u, v)$. Apply the claim to $H = G - \{u, v\}$ with $A = N(u)$ and $B = N(v)$. If $X \subseteq V(H)$ separates A from B in H , then X is a uv -separator in G : a uv -path in G has the form $ux_1x_2 \cdots x_mv$ with $x_1 \in A$ and $x_m \in B$, and if x_j is its first vertex in B and x_i is the last vertex of A among $x_1, \dots, x_j$, then $x_ix_{i+1} \cdots x_j$ is an A – B path, which meets X . Hence every set separating A from B in H has at least k vertices, and the claim gives k disjoint A – B paths $Q_1, \dots, Q_k$ in H . Adding u at the beginning and v at the end of each Q_i gives k internally vertex-disjoint uv -paths in G . Therefore $\pi(u, v) \geq \kappa(u, v)$, and so $\kappa(u, v) = \pi(u, v)$ for all graphs. $\square$

Definition 57. For $u, v \in V(G)$, a uv -edge-separator is a set $L \subseteq E(G)$ such that u and v are in different components of $G - L$.

Theorem 58 (Menger's Theorem - Edge Form). Let G be a graph and u, v be two distinct vertices in G . Then the minimum number of edges whose removal disconnects u from v equals the maximum number of pairwise edge-disjoint u - v paths in G .

Definition 58. A graph G is k -connected if and only if $|V(G)| > k$ and for every set $S \subset V(G)$ with $|S| < k$, the graph $G - S$ is connected.

Lemma 46. A block is a 2-connected graph, except for K_1 (an isolated vertex) and K_2 (a bridge with its ends).

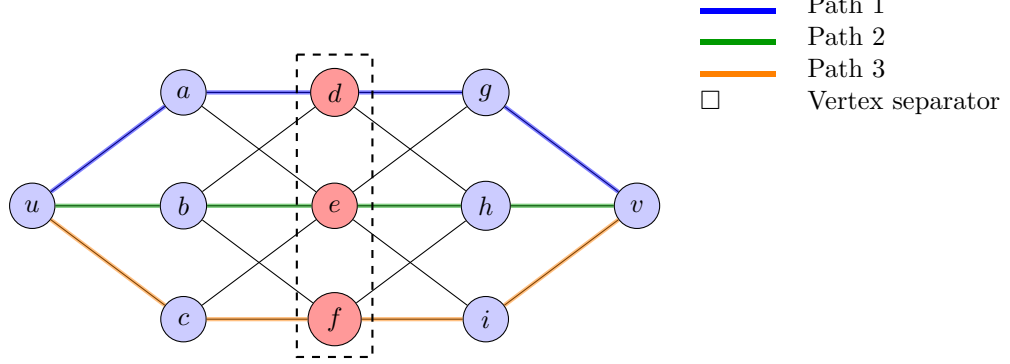
Definition 59. A uv -separator in a graph G with non-adjacent vertices $u, v \in V(G)$ is a set $X \subseteq V(G) \setminus \{u, v\}$ such that u and v are in different components of $G - X$.

Definition 60. An **edge-cut** of a graph G is a collection of edges from $E(G)$ whose removal disconnects the graph.

Remark 68. Intuitively, an edge-cut is a collection of edges that, when removed from the graph, breaks it into separate pieces. Think of these edges as bridges between different regions of the graph—remove these bridges, and you can no longer travel between the regions. In real-world networks, edge-cuts represent vulnerable connection points; for example, in a communication network, an edge-cut might represent a set of communication links whose failure would divide the network into isolated parts.

Remark 69. To understand a uv -edge-separator, imagine two specific locations in a transportation network (vertices u and v). A minimum uv -edge-separator is a smallest set of roads (edges) that, if closed, would make it impossible to travel between these two locations. The size of this separator tells us how robust the

Menger's Theorem - Example



This graph illustrates Menger's Theorem with $\kappa(u, v) = \pi(u, v) = 3$.
 The three colored paths are internally vertex-disjoint.
 The minimum vertex separator (vertices d, e, f) has size 3.

Figure 86: An example illustrating Menger's Theorem with three internally vertex-disjoint paths and a minimum separator of size 3.

connection is between u and v —a larger separator means more alternative routes exist. Menger's Theorem elegantly shows that this robustness measure exactly equals the maximum number of separate paths we can find between u and v .

Lemma 47. If S is a minimal uv -separator in a graph G , then every vertex in S has at least one neighbor in each component of $G - S$ containing either u or v .

Remark 70. This lemma reveals an important property of minimal separators: every vertex in a minimal separator plays a critical role in the disconnection. If any vertex in the separator didn't have neighbors in both the u -component and v -component, we could remove it and still maintain the separation, contradicting minimality.

Corollary 24. Let u and v be non-adjacent vertices in a graph G , let S be a minimum uv -separator, and let $P_1, \dots, P_k$ be a maximum family of internally disjoint uv -paths. Then each path P_i contains exactly one vertex of S , and each vertex of S lies on exactly one of the paths P_i .

Lemma 48 (Block Characterization). A graph is 2-connected if and only if it can be constructed from a cycle by successively adding an ear (a path that connects two existing vertices and is otherwise disjoint from the graph).

Corollary 25 (Cut Vertex Characterization). A vertex v in a connected graph G is a cut vertex if and only if there exist vertices u and w different from v such that v belongs to every uw -path in G .

Definition 61. For any vertices $u, v \in V(G)$, non-adjacent, let $\kappa(u, v)$ be the minimum size of a uv -separator.

Definition 62. A graph G that is not complete is k -connected if and only if $\kappa(u, v) \geq k$ for all non-adjacent u, v in the graph; the complete graph K_n is k -connected if and only if $n \geq k + 1$.

Definition 63. A graph with at least two vertices is k -connected if and only if it is K_n , $n \geq k + 1$, or for any two distinct vertices u, v in the graph, there exist k internally disjoint uv -paths.

Lemma 49 (Whitney's Theorem). A graph with at least 3 vertices is 2-connected if and only if any two distinct vertices are joined by at least two internally disjoint paths.

Lemma 50 (Minimum Degree Bound). For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

Menger's Theorem - Before and After Removing Separator

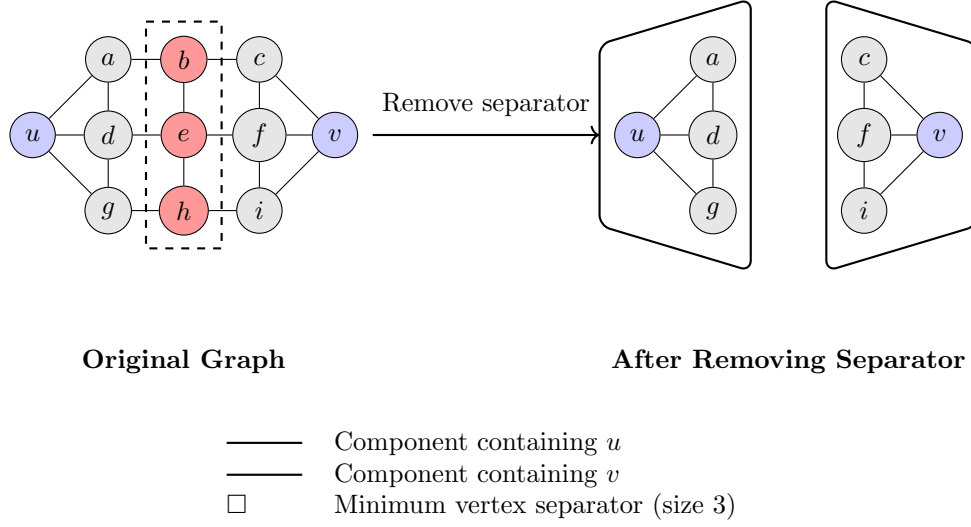


Figure 87: Illustration of a vertex separator in Menger's Theorem. When the minimum vertex separator (vertices b, e, h) is removed, vertices u and v become disconnected. The size of this separator equals the maximum number of internally vertex-disjoint paths between u and v .

Lemma 51 (Fan Lemma). If G is a k -connected graph and X is a set of at least k vertices, then from any vertex v not in X , there exist k internally disjoint paths from v to X (each path ends at a different vertex of X).

Lemma 52 (Expansion Lemma). If G is a k -connected graph and G' is obtained from G by adding a new vertex y with at least k neighbors in G , then G' is k -connected.

Definition 64. The *edge-connectivity* between vertices u and v , denoted by $\lambda(u, v)$, is the minimum size of a uv -edge-separator.

Definition 65. Two paths are *edge-disjoint* if they do not share any edges (though they may share vertices).

Theorem 59 (Menger's Theorem (Edge Form)). For any distinct vertices $u, v \in V(G)$, the minimum size $\lambda(u, v)$ of a uv -edge-separator equals the maximum number of edge-disjoint uv -paths in G .

This theorem has several important implications:

Corollary 26. A graph G is k -edge-connected if and only if for every pair of distinct vertices $u, v \in V(G)$, there exist at least k edge-disjoint paths between u and v .

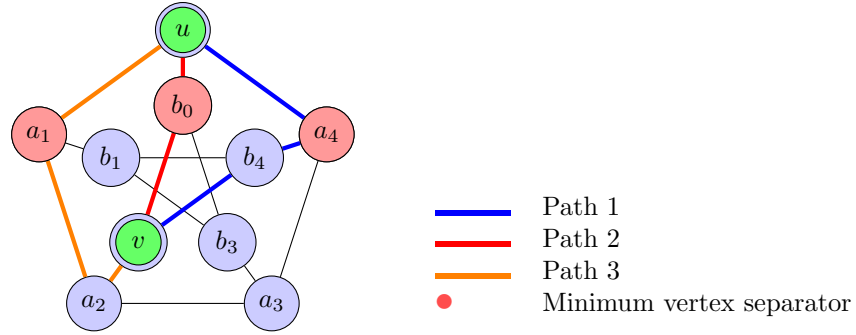


Figure 88: Application of Menger's Theorem to the Petersen graph. Three internally vertex-disjoint paths are highlighted between vertices u and v . The minimum vertex separator $\{a_1, a_4, b_0\}$ (red vertices; these are the three neighbors of u) has size 3, confirming that $\kappa(u, v) = \pi(u, v) = 3$.

Example of a uv -separator

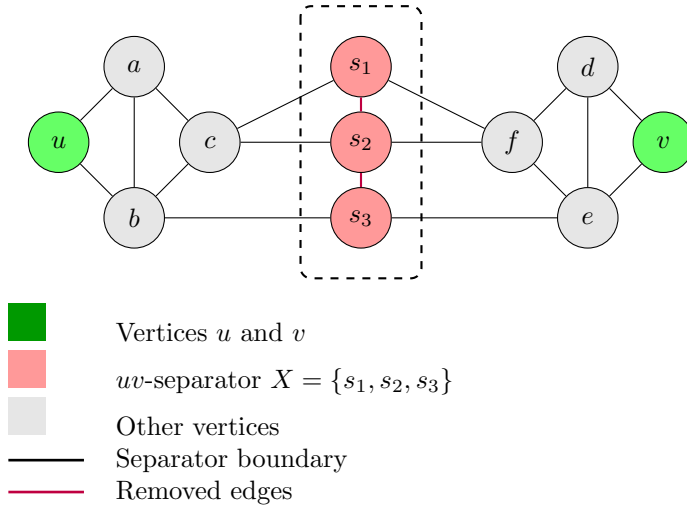


Figure 89: Example of a uv -separator $X = \{s_1, s_2, s_3\}$. When the vertices in X are removed, vertices u and v end up in different connected components of the resulting graph $G - X$.

Example 15 (Cycle Graph). Consider a cycle graph C_n with $n \geq 3$ vertices. For any two vertices u and v on the cycle, there are exactly 2 edge-disjoint paths connecting them: one going clockwise and one going counterclockwise. Therefore, $\lambda(u, v) = 2$ for any u, v , and the cycle is 2-edge-connected.

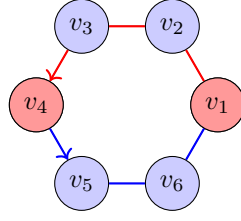


Figure 90: C_6 with two edge-disjoint paths between v_1 and v_4

Example 16 (Complete Graph). In a complete graph K_n with $n \geq 2$ vertices, any two vertices u and v are connected by $n - 1$ edge-disjoint paths: one direct edge (u, v) and $n - 2$ paths of the form (u, w, v) for each vertex $w \neq u, v$. Therefore, $\lambda(u, v) = n - 1$ for any u, v , and K_n is $(n - 1)$ -edge-connected.

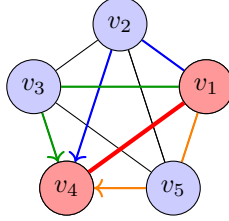


Figure 91: K_5 with four edge-disjoint paths between v_1 and v_4

It's important to note the relationship between the edge and vertex forms of Menger's Theorem:

Theorem 60 (Relationship Between Edge and Vertex Connectivity). For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

Example 17 (Path Graph). Consider a path graph P_n with $n \geq 3$ vertices.

- Vertex connectivity: $\kappa(P_n) = 1$, since removing any internal vertex disconnects the graph.
- Edge connectivity: $\lambda(P_n) = 1$, since removing any edge disconnects the graph.

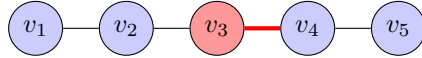


Figure 92: Removing vertex v_3 (in red) or edge (v_3, v_4) (red) disconnects the path graph, showing $\kappa(P_5) = \lambda(P_5) = 1$.

Example 18 (Cycle Graph). Consider a cycle graph C_n with $n \geq 3$ vertices.

- Vertex connectivity: $\kappa(C_n) = 2$, since removing one vertex leaves a path, while for $n \geq 4$ removing two non-adjacent vertices disconnects the graph (for $n = 3$, $C_3 = K_3$ and $\kappa(K_3) = 2$ by the convention $\kappa(K_n) = n - 1$).
- Edge connectivity: $\lambda(C_n) = 2$, since we need to remove at least 2 edges to disconnect the graph.

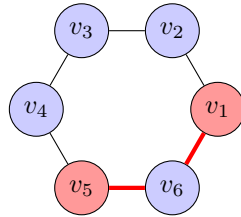


Figure 93: Removing the non-adjacent vertices v_1 and v_5 (in red), which isolates v_6 , or the edges (v_5, v_6) and (v_6, v_1) (red) disconnects the cycle graph, showing $\kappa(C_6) = \lambda(C_6) = 2$.

Example 19 (Wheel Graph). Consider a wheel graph W_n with $n + 1$ vertices (a cycle with n vertices plus a central vertex connected to all cycle vertices).

- Vertex connectivity: $\kappa(W_n) = 3$ for $n \geq 4$, since removing the central vertex and two non-adjacent cycle vertices disconnects the graph, while removing any two vertices leaves a connected graph.
- Edge connectivity: $\lambda(W_n) = 3$ for $n \geq 4$, which equals the minimum degree of the graph.

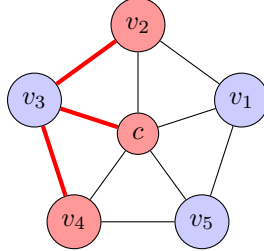


Figure 94: Removing the center c and the non-adjacent vertices v_2 and v_4 (in red) isolates v_3 and so disconnects the wheel graph. Removing the three red edges (the three edges at v_3) also disconnects the graph, showing $\kappa(W_5) = \lambda(W_5) = 3$.

Example 20 (Petersen Graph). The Petersen graph is a 3-regular graph with 10 vertices.

- Vertex connectivity: $\kappa = 3$, since removing the 3 neighbors of any vertex disconnects the graph, but no fewer vertices will do so.
- Edge connectivity: $\lambda = 3$, matching the minimum degree.

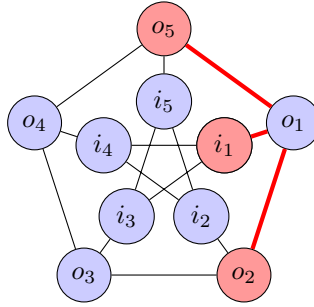


Figure 95: Removing the vertices o_2 , o_5 , and i_1 (in red), which are the three neighbors of o_1 , isolates o_1 and so disconnects the Petersen graph. Removing the three red edges (the three edges at o_1) also disconnects the graph, showing $\kappa = \lambda = 3$.

Example 21 (Grid Graph $P_2 \times P_n$). Consider a $2 \times n$ grid graph (ladder graph) with $n \geq 2$.

- Vertex connectivity: $\kappa = 2$, since removing the two ends of an interior rung (when $n \geq 3$), or the two neighbors of a corner vertex, disconnects the graph, while removing a single vertex never does.
- Edge connectivity: $\lambda = 2$, since removing the two edges at a corner vertex, or the two rail edges between the same pair of consecutive rungs, disconnects the graph, while removing a single edge never does.

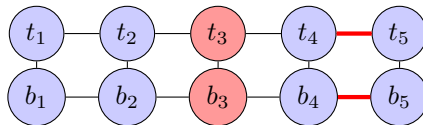


Figure 96: Removing the vertices t_3 and b_3 of the middle rung (in red) disconnects the grid graph. Removing the edges (t_4, t_5) and (b_4, b_5) (red) also disconnects the graph, showing $\kappa = \lambda = 2$.

Example 22 (Graph with $\kappa < \lambda$). Consider the graph formed by two copies of K_4 , on the vertex sets $\{a, b, c, d\}$ and $\{w, x, y, z\}$, joined by the three edges aw , dw , and bx .

- Vertex connectivity: $\kappa = 2$, since removing b and w , which together meet all three connecting edges, disconnects the graph, while removing a single vertex always leaves a connecting edge.
- Edge connectivity: $\lambda = 3$, since the three connecting edges form an edge cut, while any edge cut either separates two vertices of the same copy of K_4 , and then contains at least 3 edges of that copy, or separates the two copies, and then contains all three connecting edges.

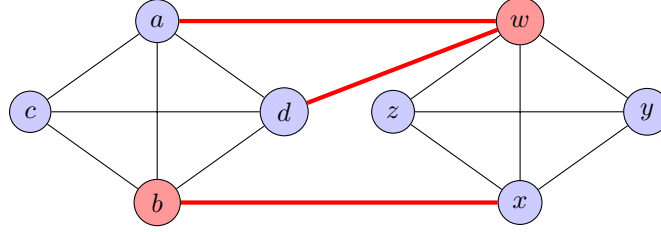


Figure 97: Removing vertices b and w (in red) disconnects the graph, showing $\kappa = 2$. Removing the three red edges aw , dw , and bx disconnects the graph, showing $\lambda = 3$.

Remark 71 (Connectivity Inequality). For any graph G , we have $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\delta(G)$ is the minimum degree. This inequality is tight, as demonstrated by complete graphs where $\kappa(K_n) = \lambda(K_n) = \delta(K_n) = n - 1$.

Corollary 27 (Characterization of k -Connectivity). A graph G with at least $k + 1$ vertices is k -connected if and only if for every pair of distinct vertices u and v , there exist at least k internally vertex-disjoint paths between u and v .

Proof. Suppose first that for every pair of distinct vertices u and v there are k internally vertex-disjoint uv -paths, and let S be a set of fewer than k vertices. For any two vertices $u, v \notin S$, each vertex of S is an internal vertex of at most one of these k paths, so at least one of the paths avoids S . Hence $G - S$ is connected, and since G has at least $k + 1$ vertices, G is k -connected.

Conversely, let G be k -connected and let $u \neq v$. If u and v are non-adjacent, then every uv -separator disconnects G , so $\kappa(u, v) \geq k$, and Menger's Theorem gives k internally vertex-disjoint uv -paths. Now let u and v be adjacent, and let $G' = G - uv$ be the graph obtained by deleting the edge uv . Together with the edge uv , any $k - 1$ internally disjoint uv -paths in G' give k such paths in G . Suppose G' had no $k - 1$ internally disjoint uv -paths. Since u and v are non-adjacent in G' , Menger's Theorem gives a uv -separator X in G' with $|X| \leq k - 2$. Since G has at least $k + 1$ vertices, there is a vertex $w \notin X \cup \{u, v\}$. As u and v lie in different components of $G' - X$, the vertex w lies in a different component from one of them, say from u (the other case is symmetric). Then u and w lie in different components of $G - (X \cup \{v\})$, because the only edge of G missing from G' is uv , and it is deleted together with v . So removing the set $X \cup \{v\}$ of at most $k - 1$ vertices disconnects G , contradicting that G is k -connected. $\square$

Theorem 61 (Menger's Theorem - Edge Form). For distinct $u, v \in V(G)$, the minimum size $\lambda(u, v)$ of a uv -edge-separator equals the maximum number of edge-disjoint uv -paths in G .

Theorem 62. If G is a connected graph with a bridge, then $\lambda(u, v) = 1$ when $\{u, v\}$ is the bridge.

Corollary 28. If G is connected with no bridge, then $\lambda(u, v) \geq 2$ for all distinct $u, v \in V(G)$.

Definition 66. An **edge-cut** of a graph G is a set $L \subseteq E(G)$ such that $G - L$ is disconnected.

Definition 67. A graph G is k -edge-connected if and only if every edge cut in G has size at least k .

Theorem 63. The following statements are equivalent:

1. A graph is connected.

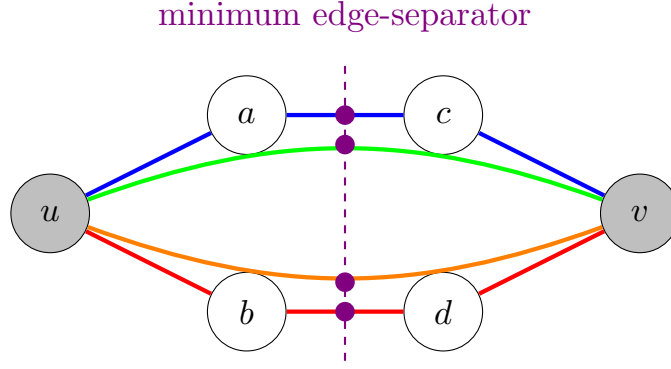


Figure 98: Illustration of Menger's Theorem (Edge Form): The graph contains 4 edge-disjoint paths between u and v (shown in different colors); the two curved paths are two parallel edges joining u and v , so this example is a multigraph (the edge form of Menger's Theorem also holds for multigraphs: subdividing every edge once reduces it to the simple case). The minimum edge-separator (violet dashed line) cuts exactly 4 edges, demonstrating that $\lambda(u, v) = 4$.

2. A graph is 1-edge-connected.

Theorem 64. The following statements are equivalent:

1. A graph is 2-edge-connected.
2. A graph is connected and has no bridge.
3. A graph is connected and every edge in the graph is part of at least one cycle.

Definition 68. If G is a graph, then the vertex connectivity $\kappa(G)$ is $n - 1$ if $G = K_n$, otherwise it is the minimum size of a vertex cut in G .

Definition 69. The edge-connectivity $\lambda(G)$ is the minimum size of an edge cut in G .

Example 23. For the cycle graph G_1 shown:

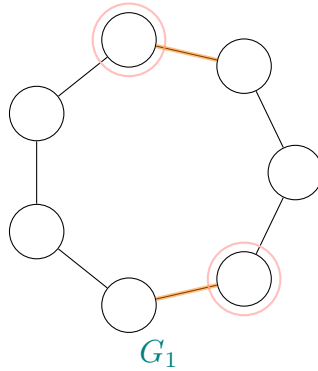


Figure 99: A cycle graph with $\lambda(G_1) = 2$ and $\kappa(G_1) = 2$. The minimum vertex cut (two vertices highlighted in pink) and minimum edge cut (two edges highlighted in orange) both have size 2.

$$\lambda(G_1) = 2 \quad \kappa(G_1) = 2$$

For the graph G_2 obtained from two disjoint copies of K_{12} by adding 7 edges between them, where the added edges have only 5 distinct ends $a_1, \dots, a_5$ in the first copy and 7 distinct ends $b_1, \dots, b_7$ in the second copy (the added edges are $a_i b_i$ for $1 \leq i \leq 5$, together with $a_5 b_6$ and $a_5 b_7$):

$$\text{For this graph } G_2: \lambda(G_2) = 7 \quad \kappa(G_2) = 5 \quad \delta(G_2) = 11$$

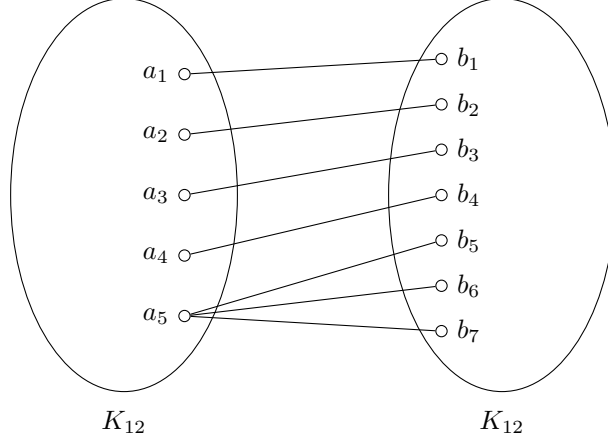


Figure 100: The graph G_2 : two copies of K_{12} (drawn schematically) joined by 7 edges whose ends are the 5 vertices $a_1, \dots, a_5$ of the first copy and the 7 vertices $b_1, \dots, b_7$ of the second copy. The set $\{a_1, \dots, a_5\}$ is a minimum vertex cut (size 5), the 7 added edges form a minimum edge cut (size 7), and every vertex that is not an end of an added edge has degree 11.

Corollary 29. For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Lemma 53. If G is not a complete graph, then $\kappa(G) = \min\{|X| : X \subseteq V(G) \text{ is a vertex cut}\}$, where a vertex cut is a set X such that $G - X$ is disconnected. (A complete graph K_n has no vertex cut, and $\kappa(K_n) = n - 1$.)

Remark 72. Recall that a graph G is k -connected if and only if $\kappa(G) \geq k$.

Lemma 54. The edge-connectivity $\lambda(G)$ is $\min\{|L| : L \subseteq E(G) \text{ is an edge cut}\}$.

Remark 73. Recall that a graph G is k -edge-connected if and only if $\lambda(G) \geq k$.

Proof of Lemma 50 (Minimum Degree Bound). Recall that for any graph G , we need to prove that $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

We first prove $\lambda(G) \leq \delta(G)$. Consider any vertex $v \in V(G)$ with $d(v) = \delta(G)$. Let $L = \{\{v, w\} : w \in N(v)\}$ be the set of all edges incident to v . This is an edge cut of size $\delta(G)$. Therefore, $\lambda(G) \leq \delta(G)$.

For $\kappa(G) \leq \lambda(G)$, we use Menger's theorem.

Recall Menger's Theorem in vertex form, which states that for non-adjacent vertices u, v in a graph G :

$$\kappa(u, v) = \max\{k : \text{there are } k \text{ internally disjoint } uv\text{-paths}\}.$$

Note that internally disjoint uv -paths are also edge-disjoint.

Let L be an edge cut with $|L| = \lambda(G)$, let S be the vertex set of one component of $G - L$, and let $\bar{S} = V(G) \setminus S$. Every edge joining S to $\bar{S}$ lies in L , and these edges alone already form an edge cut; so, by the minimality of $|L|$, L is exactly the set of edges joining S to $\bar{S}$.

Case 1: Some $u \in S$ and $v \in \bar{S}$ are non-adjacent. Every uv -path uses an edge of L , so there are at most $|L|$ edge-disjoint uv -paths, and hence at most $|L|$ internally disjoint uv -paths. Since a minimum uv -separator is a vertex cut of G , Menger's Theorem gives

$$\kappa(G) \leq \kappa(u, v) \leq |L| = \lambda(G).$$

Case 2: Every vertex of S is adjacent to every vertex of $\bar{S}$. Then, with $n = |V(G)|$,

$$\lambda(G) = |L| = |S| \cdot |\bar{S}| \geq |S| + |\bar{S}| - 1 = n - 1 \geq \kappa(G),$$

where $|S| \cdot |\bar{S}| \geq |S| + |\bar{S}| - 1$ because $(|S| - 1)(|\bar{S}| - 1) \geq 0$, and $\kappa(G) \leq n - 1$ holds for every graph.

Therefore, $\kappa(G) \leq \lambda(G)$.

This completes the proof. $\square$

Theorem 65. For any integers k, ℓ, d with $1 \leq k \leq \ell \leq d$, there exists a graph G with $\kappa(G) = k$, $\lambda(G) = \ell$, and $\delta(G) = d$.

Proof. We construct a graph G as follows (the theorem is due to Chartrand and Harary, 1968):

1. Start with two disjoint complete graphs K_{d+1} , which we will call X and Y .
2. Select k vertices $\{x_1, x_2, \dots, x_k\}$ from X and ℓ vertices $\{y_1, y_2, \dots, y_\ell\}$ from Y ; this is possible since $k \leq \ell \leq d < d + 1$.
3. Connect these selected vertices with edges such that:
 - x_i is joined to y_i for $i = 1, 2, \dots, k$
 - x_k is joined to y_i for $i = k + 1, \dots, \ell$

Thus exactly ℓ edges join X to Y , every selected vertex is incident with at least one of them, and the k edges $x_1y_1, \dots, x_ky_k$ are pairwise disjoint.

Now, we verify the three parameters:

For minimum degree $\delta(G)$:

Every vertex has degree d inside its own copy of K_{d+1} , and the added edges only increase degrees. Since $k \leq d < d + 1$, some vertex of X is not selected, and its degree is exactly d . Hence $\delta(G) = d$.

For edge connectivity $\lambda(G)$:

Removing the ℓ edges between X and Y disconnects the graph into two components (X and Y), so $\lambda(G) \leq \ell$. We cannot disconnect the graph by removing fewer than ℓ edges. Indeed, let L be an edge cut and let S be the vertex set of a component of $G - L$.

- If S contains some but not all vertices of X , say j of them with $1 \leq j \leq d$, then L contains the $j(d+1-j)$ edges of X joining $S \cap X$ to $X \setminus S$, and $j(d+1-j) \geq d \geq \ell$ because $j(d+1-j) - d = (j-1)(d-j) \geq 0$. The same holds for Y .
- Otherwise $S = X$ or $S = Y$, and L contains all ℓ edges between X and Y .

Therefore, $\lambda(G) = \ell$.

For vertex connectivity $\kappa(G)$:

If we remove the k vertices $\{x_1, x_2, \dots, x_k\}$ from X , the graph becomes disconnected because there are no remaining edges between X and Y , while both Y and $X - \{x_1, \dots, x_k\}$ (which has $d + 1 - k \geq 1$ vertices) remain. So $\kappa(G) \leq k$. We cannot disconnect the graph by removing a set S of fewer than k vertices because:

- $X - S$ and $Y - S$ are nonempty complete graphs, hence connected
- Each vertex of S is an end of at most one of the k disjoint edges $x_1y_1, \dots, x_ky_k$, so at least one of these edges survives and joins $X - S$ to $Y - S$

Therefore, since also $|V(G)| = 2d + 2 > k$, we have $\kappa(G) = k$.

This completes the proof, showing that we can construct a graph G with $\kappa(G) = k$, $\lambda(G) = \ell$, and $\delta(G) = d$ for any integers satisfying $1 \leq k \leq \ell \leq d$. $\square$

6.3 Structure of Connected Graphs Exercises

6.3.1 Easy Questions

1. State the definition of a cut vertex. Give an example of a graph with exactly two cut vertices.
2. Define the terms “block” and “block-cutpoint graph” in the context of graph theory.
3. If a graph G has exactly one block, what can you say about G ? Justify your answer.
4. True or False: Every cut vertex in a connected graph must belong to at least two blocks. Prove your answer.

5. If T is a tree, characterize all of its blocks. How many blocks does a tree with n vertices have?
6. State Menger's Theorem (vertex form). What is the relationship between the minimum number of vertices whose removal disconnects u from v and the maximum number of internally vertex-disjoint u - v paths?
7. Given a graph with $\kappa(G) = 2$, $\lambda(G) = 3$, and $\delta(G) = 4$, explain what each of these values represents.

6.3.2 Medium Questions

8. Let G be a connected graph with n vertices. If G has exactly $n - 1$ edges, prove that every edge of G is a block.
9. Prove that the block-cutpoint graph $\mathcal{B}(G)$ of a connected graph G is always a tree.
10. Let G be a graph with vertex connectivity $\kappa(G) = k$ and edge connectivity $\lambda(G) = \ell$. Prove that $k \leq \ell$.
11. If a graph G has minimum degree $\delta(G) = 3$ and vertex connectivity $\kappa(G) = 2$, construct a concrete example of such a graph with the minimum possible number of vertices.
12. Prove that in a 2-connected graph, any two vertices lie on a common cycle.
13. Let G be a connected graph with at least three vertices. Prove that G is 2-connected if and only if for any three distinct vertices u , v , and w , there exists a path from u to v that does not contain w .
14. A vertex v in a connected graph G is called an articulation point if removing v disconnects the graph. Prove that v is an articulation point if and only if there exist vertices a and b , both different from v , such that every path from a to b passes through v .

6.3.3 Hard Questions

15. Let G be a connected graph with at least two vertices containing no Θ -graph. Prove that every block of G is a cycle or a K_2 (so G is a tree of cycles), and give an example showing that G need not be a tree or a cycle.
16. Prove that if G is a k -connected graph, then for any set S of k vertices and any vertex $v \notin S$, there exist k internally vertex-disjoint paths from v to the vertices of S .
17. Given integers $1 \leq k \leq \ell \leq d$, construct a graph with vertex connectivity $\kappa(G) = k$, edge connectivity $\lambda(G) = \ell$, and minimum degree $\delta(G) = d$. Prove that your construction works.
18. Let G be a graph with $\lambda(G) = k$. Prove that if G has a vertex cut of size k , then G also has an edge cut of size k .
19. Let G be a graph with $n \geq 3$ vertices. Prove that if $\delta(G) > \frac{n-1}{2}$, then G is 2-connected.
20. Develop an algorithm to find all cut vertices in a connected graph G . Analyze its time complexity.
21. For a connected graph G , let $b(G)$ be the number of blocks and $c(G)$ be the number of cut vertices, and for a cut vertex v let $b(v)$ be the number of blocks containing v . Prove that $b(G) = 1 + \sum_v (b(v) - 1)$, where the sum runs over all cut vertices v of G . Deduce that $b(G) \geq c(G) + 1$, with equality if and only if every cut vertex lies in exactly two blocks. (For example, the star $K_{1,3}$ has $b(G) = 3$ and $c(G) = 1$.)

6.3.4 Diagram Questions

22. Construct a graph G with vertex connectivity $\kappa(G) = 2$, edge connectivity $\lambda(G) = 3$, and minimum degree $\delta(G) = 4$. Clearly label the graph and explain why it satisfies the required properties.
23. Draw a graph with at least 3 blocks. Identify all cut vertices and blocks. Then draw its block-cutpoint graph $\mathcal{B}(G)$.
24. Draw a 3-connected graph with 8 vertices where every vertex has degree exactly 3. Highlight three internally vertex-disjoint paths between two non-adjacent vertices to illustrate Menger's Theorem.
25. Construct a graph that has exactly 3 cut vertices and 4 blocks. Draw the corresponding block-cutpoint graph.
26. Consider the Petersen graph. Choose two non-adjacent vertices u and v , and illustrate Menger's Theorem by highlighting: (a) a minimum vertex separator between u and v , and (b) the maximum number of internally vertex-disjoint paths between u and v .
27. Draw a graph where $\kappa(G) < \lambda(G) < \delta(G)$. Clearly indicate the values of each parameter and explain how your graph satisfies these inequalities.
28. Draw an example of a connected graph that contains no Θ -graph and is neither a tree nor a cycle. Identify its blocks, and explain why your example is consistent with the relevant theorem.

6.3.5 Additional Questions

29. Let C be a cut set of vertices in a connected graph G . Prove that if C is minimal (i.e., no proper subset of C is a cut set), then every vertex in C has at least one neighbor in each connected component of $G - C$.
30. Prove that a k -connected graph on n vertices must have at least $\lceil \frac{kn}{2} \rceil$ edges.
31. For any two distinct non-adjacent vertices u and v in a graph G , define $\kappa(u, v)$ as the minimum number of vertices, other than u and v , whose removal disconnects u from v . Prove that if G is not a complete graph, then $\kappa(G) = \min \kappa(u, v)$, where the minimum is taken over all pairs of distinct non-adjacent vertices $u, v \in V(G)$.
32. If G is a k -connected graph and H is obtained from G by adding a new vertex adjacent to at least k vertices of G , prove that H is also k -connected.
33. Let G be a connected graph where every vertex has even degree. Prove that G has no bridges.
34. Let G be a graph where every vertex has degree at least k . Prove that G contains a path of length at least k .
35. Define a uv -separator in a graph G with non-adjacent vertices u and v as a set $X \subseteq V(G) \setminus \{u, v\}$ such that u and v are in different components of $G - X$. If S is a minimal uv -separator, prove that every vertex in S has at least one neighbor in the component of $G - S$ containing u and at least one neighbor in the component containing v .
36. Let G be a graph with vertex connectivity $\kappa(G) = k$. Prove that G has at least $k + 1$ vertices of degree at least k .
37. Prove that a graph G is 2-edge-connected if and only if it has an orientation such that the resulting directed graph is strongly connected.
38. Use Menger's Theorem to prove that if G is a k -connected graph and S is a subset of $V(G)$ with $|S| \geq k$, then for any vertex $v \notin S$, there exist k internally vertex-disjoint paths from v to k distinct vertices in S .

39. Let G be a 3-connected graph. Prove that any two cycles in G have a common vertex or there exists a path connecting them that is internally disjoint from both cycles.
40. A block-cut tree of a connected graph G is a tree where each vertex corresponds to either a block or a cut vertex of G , with edges connecting each cut vertex to the blocks containing it. Prove that removing a cut vertex v from G leaves exactly $\deg_T(v)$ components (so the number of components increases by exactly $\deg_T(v) - 1$), where $\deg_T(v)$ is the degree of v in the block-cut tree.

7 Matchings and Factors

Definition 70 (Matching). A matching in a graph is a set of vertex disjoint edges in the graph.

Definition 71 (Maximum Matching). A maximum matching in a graph G is a matching M such that $|E(M)|$ is a maximum.

Definition 72 (Perfect Matching). A perfect matching or 1-factor is a matching M such that

$$V(M) = V(G).$$

Definition 73 (1-Factor). A 1-factor of a graph G is a spanning 1-regular subgraph of G . In other words, it is a subgraph that includes all vertices of G where each vertex has degree exactly 1.

Theorem 66 (Tutte's 1-Factor Theorem). A graph $G = (V, E)$ has a perfect matching (also called a 1-factor) if and only if for every subset $S \subseteq V$, the number of odd-sized components in $G - S$ is at most $|S|$.

In other words, a graph G has a perfect matching if and only if:

$$\text{odd}(G - S) \leq |S| \quad \text{for all } S \subseteq V$$

where $\text{odd}(G - S)$ denotes the number of connected components in $G - S$ that have an odd number of vertices.

Corollary 30 (Tutte-Berge Formula). For any graph $G = (V, E)$, the size of a maximum matching equals:

$$\nu(G) = \frac{1}{2} \min_{S \subseteq V} \{|V| + |S| - \text{odd}(G - S)\}$$

where $\nu(G)$ is the matching number of G (the size of a maximum matching).

Remark 74. A 1-factor is equivalent to a perfect matching, as each vertex must be incident to exactly one edge, and the collection of these edges forms a matching that covers all vertices of the graph.

Remark 75. The Tutte-Berge formula generalizes Tutte's 1-factor theorem by quantifying the maximum matching size when a perfect matching does not exist. It shows that the "deficiency" is determined by the subset S that maximizes $\text{odd}(G - S) - |S|$.

Remark 76. For bipartite graphs, Tutte's condition simplifies to Hall's Marriage Theorem, which states that a bipartite graph $G = (A \cup B, E)$ with $|A| = |B|$ has a perfect matching if and only if for all subsets $S \subseteq A$, $|N(S)| \geq |S|$.

Remark 77. For a graph to have a 1-factor:

- The graph must have an even number of vertices (since each edge covers exactly two vertices)
- The graph must satisfy Tutte's condition: for any subset S of vertices, the number of odd components in $G - S$ is at most $|S|$

Example 24. The complete graph K_{2n} always has a 1-factor. In fact, it can be decomposed into $2n - 1$ disjoint 1-factors.

Example 25. Consider the following cycle graph G with 7 vertices:

The highlighted edges form a matching because they don't share any vertices. This matching is not maximum.

This graph does not have a perfect matching because it has 7 vertices (an odd number), and in a perfect matching, each edge covers exactly 2 vertices. With an odd number of vertices, at least one vertex must be left uncovered.

The Petersen graph shown above has 10 vertices, which is an even number, and it does have a perfect matching: the five spokes $o_1i_1, o_2i_2, \dots, o_5i_5$ are pairwise disjoint and together cover all ten vertices. (In

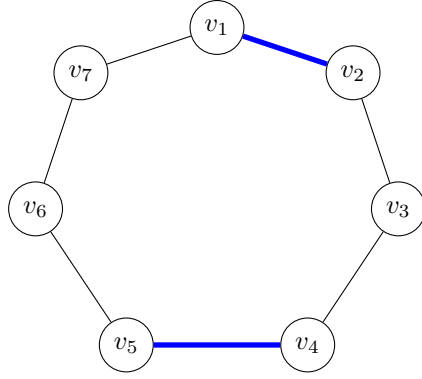


Figure 101: A matching with edges $\{(v_1, v_2), (v_4, v_5)\}$

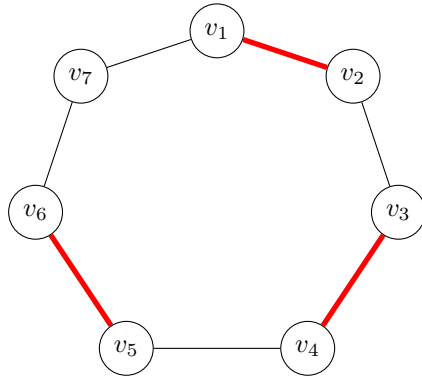
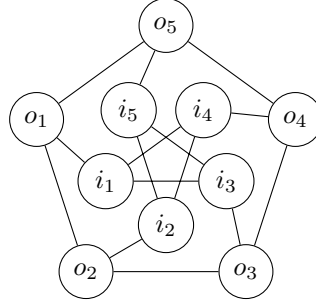


Figure 102: A maximum matching in G with 3 edges

fact, the Petersen graph has exactly six perfect matchings; what it lacks is a partition of its edge set into three perfect matchings.) An even number of vertices is necessary but not sufficient for a perfect matching: the star $K_{1,3}$ has 4 vertices, but deleting the set S consisting of its center leaves $3 > 1 = |S|$ odd components, so $K_{1,3}$ has no perfect matching. In general, a graph G has a perfect matching if and only if for every subset S of vertices, the number of connected components in $G - S$ that have an odd number of vertices is at most $|S|$ (Tutte's theorem).

Example 26. Here is an example of a graph with a perfect matching:

The complete bipartite graph $K_{3,3}$ has 6 vertices and the highlighted edges form a perfect matching that covers all vertices.



The five spokes $o_i i_i$ form a perfect matching

Figure 103: Petersen graph - has an even number of vertices (5+5)

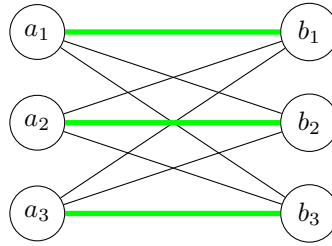


Figure 104: $K_{3,3}$ with a perfect matching (highlighted in green)

Problem 3. Suppose we have persons $p_1, \dots, p_n$ and jobs $j_1, j_2, \dots, j_m$.

Place an edge $\{p_i, j_k\}$ if person p_i can do job j_k .

What is the maximum number of jobs that can be done, if each person is assigned at most one job, and each job is done by at most one person?

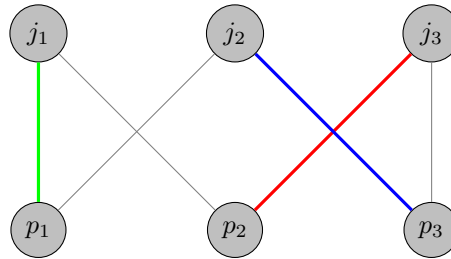


Figure 105: Example of a job assignment problem with a maximum matching (Each color represents a person-job assignment)

Remark 78. This is equivalent to finding a maximum matching in the bipartite graph $G = (P \cup J, E)$ where:

- $P = \{p_1, p_2, \dots, p_n\}$ is the set of people
- $J = \{j_1, j_2, \dots, j_m\}$ is the set of jobs
- $E = \{\{p_i, j_k\} : \text{person } p_i \text{ can do job } j_k\}$

Example 27. In the example shown in the figure, the maximum number of jobs that can be done is 3, as shown by the highlighted matching where:

- Person p_1 is assigned job j_1 (green edge)
- Person p_2 is assigned job j_3 (red edge)
- Person p_3 is assigned job j_2 (blue edge)

This is a perfect matching since all persons are assigned a job and all jobs are filled.

Remark 79. If $n \neq m$ (i.e., the number of people doesn't equal the number of jobs), then we can't have a perfect matching. The maximum number of assignments possible is at most $\min(n, m)$, and it can be smaller: for example, if $n = 2$, $m = 3$, and both people can do only job j_1 , then only one assignment is possible.

7.1 Independent Sets and Covers

Definition 74 (Independent Set). An independent set in a graph is a set of vertices containing no edge of the graph.

Definition 75 (Independence Number). Let $\alpha(G)$ be the size of a maximum independent set in G .

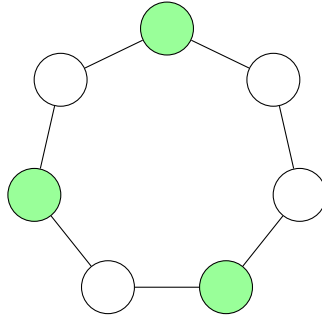


Figure 106: C_7 with a maximum independent set of size $\alpha(C_7) = 3$

Definition 76 (Matching Number). Let $\alpha'(G)$ be the number of edges in a maximum matching.

Example 28. For the cycle graph C_7 , we have $\alpha(C_7) = 3$ and $\alpha'(C_7) = 3$.

Definition 77 (Vertex Cover). A vertex cover of a graph G is a set $X \subseteq V(G)$ such that for every edge $e \in E(G)$, $e \cap X \neq \emptyset$.

Definition 78 (Vertex Cover Number). Let $\beta(G)$ be the smallest size of a vertex cover in G .

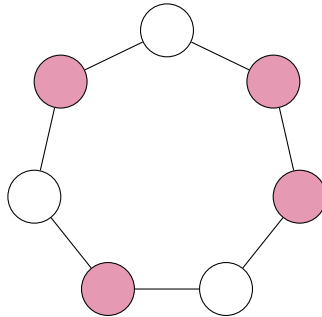


Figure 107: C_7 with a minimum vertex cover of size $\beta(C_7) = 4$

Definition 79 (Edge Cover). An edge cover is a set of edges of a graph G such that every vertex is contained in at least one of the edges.

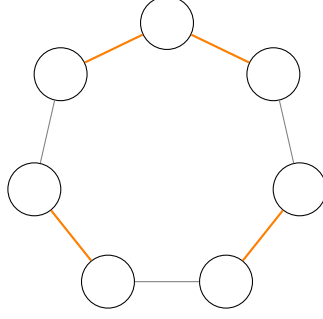


Figure 108: C_7 with a minimum edge cover of size $\beta'(C_7) = 4$

Definition 80 (Edge Cover Number). Let $\beta'(G)$ be the smallest size of an edge cover in G .

Remark 80. We can verify these relationships in our C_7 example:

$$\alpha(C_7) + \beta(C_7) = 3 + 4 = 7 = |V(C_7)| \quad (35)$$

$$\alpha'(C_7) + \beta'(C_7) = 3 + 4 = 7 = |V(C_7)| \quad (36)$$

Remark 81. An independent set S is a maximum independent set if and only if its complement $V(G) \setminus S$ is a minimum vertex cover. This follows directly from Gallai's Lemma below.

Lemma 55 (Gallai's Lemma). If G is a graph with no isolated vertices, then

$$\alpha'(G) + \beta'(G) = |V(G)|.$$

If G is any graph, then

$$\alpha(G) + \beta(G) = |V(G)|.$$

Proof. Let $I \subseteq V(G)$ be a maximum independent set in G ; hence $|I| = \alpha(G)$. Since I is independent, every edge of G has at least one endpoint in

$$V(G) \setminus I,$$

so $V(G) \setminus I$ is a vertex cover. By minimality of $\beta(G)$, we have

$$|V(G) \setminus I| \geq \beta(G).$$

But $|V(G) \setminus I| = |V(G)| - |I| = |V(G)| - \alpha(G)$, so

$$\alpha(G) + \beta(G) \leq |V(G)|.$$

Conversely, let $C \subseteq V(G)$ be a minimum vertex cover; hence $|C| = \beta(G)$. By definition of a vertex cover, no two vertices in

$$V(G) \setminus C$$

can be adjacent, so $V(G) \setminus C$ is an independent set. By maximality of $\alpha(G)$,

$$|V(G) \setminus C| \leq \alpha(G).$$

Again $|V(G) \setminus C| = |V(G)| - \beta(G)$, giving

$$\alpha(G) + \beta(G) \geq |V(G)|.$$

Combining the two inequalities yields

$$\alpha(G) + \beta(G) = |V(G)|,$$

which is the second identity of the lemma. The first identity, $\alpha'(G) + \beta'(G) = |V(G)|$ for graphs without isolated vertices, is proved later in this section. $\square$

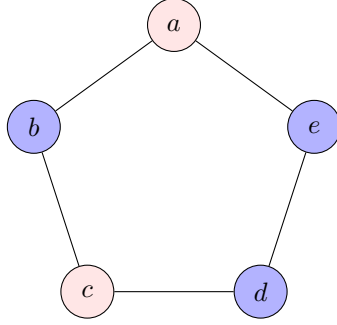


Figure 109: Graph $G = C_5$ with $\alpha(G) = 2$ and $\beta(G) = 3$. Note that $\alpha(G) + \beta(G) = 2 + 3 = 5 = |V(G)|$.

Example 29. Consider the cycle graph C_5 shown below:

In the figure above:

- Pink vertices form a maximum independent set of size $\alpha(G) = 2$
- Blue vertices form a minimum vertex cover of size $\beta(G) = 3$

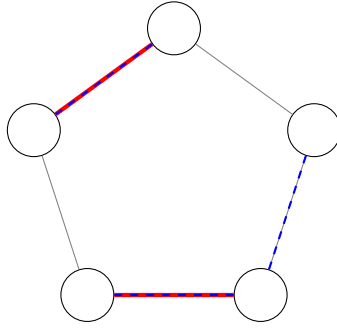


Figure 110: Graph $G = C_5$ with a maximum matching (solid red) and a minimum edge cover (dashed blue). $\alpha'(G) = 2$ and $\beta'(G) = 3$. $\alpha'(G) + \beta'(G) = 2 + 3 = 5 = |V(G)|$.

In the second diagram:

- The solid red edges form a maximum matching of size $\alpha'(G) = 2$
- The dashed blue edges form a minimum edge cover of size $\beta'(G) = 3$

Remark 82. The first part of Gallai's Lemma tells us that for any graph without isolated vertices, the sum of the size of a maximum matching and the size of a minimum edge cover equals the number of vertices in the graph.

The second part tells us that for any graph (even those with isolated vertices), the sum of the size of a maximum independent set and the size of a minimum vertex cover equals the number of vertices in the graph.

Remark 83. These relationships reflect the complementary nature of these graph parameters:

- A set $S \subseteq V(G)$ is independent if and only if its complement $V(G) \setminus S$ is a vertex cover
- Each vertex must be covered by at least one edge in an edge cover, and can be covered by at most one edge in a matching

Definition 81 (Independence Number). The *independence number* of a graph G , denoted by $\alpha(G)$, is the size of a maximum independent set in G .

Definition 82 (Matching Number). The *matching number* of a graph G , denoted by $\alpha'(G)$, is the size of a maximum matching in G .

Definition 83 (Vertex Cover Number). The *vertex cover number* of a graph G , denoted by $\beta(G)$, is the size of a minimum vertex cover in G .

Definition 84 (Edge Cover Number). The *edge cover number* of a graph G , denoted by $\beta'(G)$, is the size of a minimum edge cover in G .

Recall Gallai's Lemma, which establishes an important relationship between these graph parameters:
For any graph G :

$$\alpha(G) + \beta(G) = |V(G)| \quad (37)$$

And if G has no isolated vertices:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (38)$$

Proof that $\alpha(G) + \beta(G) = |V(G)|$:

Let I be a maximum independent set in G , so $|I| = \alpha(G)$.

Since I is an independent set, the remaining vertices $V(G) \setminus I$ must form a vertex cover. This gives us:

$$|V(G) \setminus I| \geq \beta(G) \quad (39)$$

Therefore:

$$|I| + |V(G) \setminus I| = |V(G)| \geq \alpha(G) + \beta(G) \quad (40)$$

Conversely, let C be a minimum vertex cover, so $|C| = \beta(G)$. Then $V(G) \setminus C$ is an independent set, so $|V(G) \setminus C| \leq \alpha(G)$.

We have:

$$|V(G)| = |C| + |V(G) \setminus C| \leq \beta(G) + \alpha(G) \quad (41)$$

Combining the last two displayed inequalities, we get:

$$\alpha(G) + \beta(G) = |V(G)| \quad (42)$$

Proof that $\alpha'(G) + \beta'(G) = |V(G)|$:

Suppose now G has no isolated vertices. Let M be a maximum matching in G , so $|E(M)| = \alpha'(G)$.

Then $V(G) \setminus V(M)$ is an independent set of G . This is because if there were an edge between two vertices in $V(G) \setminus V(M)$, we could add this edge to M to create a larger matching, contradicting the maximality of M .

Now, to form an edge cover, we can: 1. Pick one edge containing each vertex of $V(G) \setminus V(M)$ 2. Add all edges from M to this set

This forms an edge cover, as every vertex is covered by at least one edge. The total number of edges in this cover is at most:

$$|V(G) \setminus V(M)| + |E(M)| = |V(G)| - |V(M)| + |E(M)| \quad (43)$$

Since each edge in M covers exactly two vertices, $|V(M)| = 2|E(M)|$. Therefore:

$$|V(G)| - |V(M)| + |E(M)| = |V(G)| - 2|E(M)| + |E(M)| = |V(G)| - |E(M)| \quad (44)$$

So the size of this edge cover is at most $|V(G)| - \alpha'(G)$, which means:

$$\beta'(G) \leq |V(G)| - \alpha'(G) \quad (45)$$

Conversely, starting with a minimum edge cover, we can construct a matching to show:

$$\alpha'(G) \geq |V(G)| - \beta'(G) \quad (46)$$

Combining these inequalities:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (47)$$

This is the first part of Gallai's Lemma; the matching used for the second inequality is constructed in the proof below.

Proof of $\alpha'(G) + \beta'(G) = |V(G)|$ for graphs with no isolated vertices. Suppose G has no isolated vertices. Let M be a maximum matching in G , so $|E(M)| = \alpha'(G)$.

Then $V(G) \setminus V(M)$ is an independent set of G . This is because if there were an edge between two vertices in $V(G) \setminus V(M)$, we could add this edge to M to create a larger matching, contradicting the maximality of M .

Now, to form an edge cover, we can pick one edge containing each vertex of $V(G) \setminus V(M)$ and add all edges from M . Since G has no isolated vertices, such edges must exist. This gives us an edge cover of size at most:

$$\beta'(G) \leq |V(G) \setminus V(M)| + |E(M)| \quad (48)$$

$$= |V(G)| - |V(M)| + |E(M)| \quad (49)$$

$$= |V(G)| - 2|E(M)| + |E(M)| \quad (50)$$

$$= |V(G)| - |E(M)| \quad (51)$$

$$= |V(G)| - \alpha'(G) \quad (52)$$

So:

$$\alpha'(G) + \beta'(G) \leq |V(G)| \quad (1)$$

Conversely, let $F \subseteq E(G)$ be a minimum edge cover. Then every edge of F has an end which has degree 1 in F . This must be true, for otherwise we could remove an edge and still maintain an edge cover, contradicting the minimality of F .

This means that F is a disjoint union of stars: every component of F is a $K_{1,t}$ (a star graph with one central vertex connected to t leaf vertices).

If F has k components, then $\alpha'(G) \geq k$ (since picking one edge from each component gives k pairwise disjoint edges, that is, a matching of size k).

We have $|E(F)| = |V(G)| - k$ because in a disjoint union of k stars covering all vertices, the number of edges is exactly the number of vertices minus the number of stars. Therefore:

$$|E(F)| = \beta'(G) = |V(G)| - k \geq |V(G)| - \alpha'(G) \quad (53)$$

So:

$$\beta'(G) + \alpha'(G) \geq |V(G)| \quad (2)$$

From inequalities (1) and (2), we conclude:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (54)$$

This completes the proof of the first part of Gallai's Lemma. $\square$

Theorem 67 (Hall's Theorem - Formal Statement). Let G be a bipartite graph with bipartition (A, B) where $|A| = |B|$. Then G has a perfect matching if and only if for every subset $X \subseteq A$:

$$|N(X)| \geq |X|$$

Theorem 68 (Hall's Marriage Theorem). Hall's Theorem gives a necessary and sufficient condition for a bipartite graph to have a perfect matching.

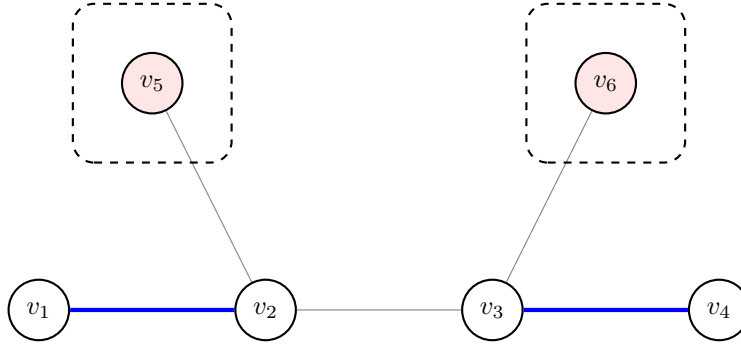
Remark 84. For bipartite graphs, Tutte's condition simplifies to Hall's Marriage Theorem, which states that a bipartite graph $G = (A \cup B, E)$ with $|A| = |B|$ has a perfect matching if and only if for all subsets $S \subseteq A$, $|N(S)| \geq |S|$.

Theorem 69 (König-Ore Formula). Let $G(A, B)$ be a bipartite graph. Then

$$\nu(G, A) = \max_{S \subseteq A} \{|S| - |N(S)|\}.$$

where:

Independent set $V(G) \setminus V(M) = \{v_5, v_6\}$ (pink vertices)



Maximum matching M (thick blue edges)

Figure 111: Illustration for Gallai's Lemma: A graph G with a maximum matching M of size $\alpha'(G) = 2$ (blue edges). The vertices not covered by the matching (v_5 and v_6 , highlighted in pink) form an independent set. For an edge cover, we need the two edges in M plus one edge incident to each of v_5 and v_6 , giving a minimum edge cover of size $\beta'(G) = 4$. Note that $\alpha'(G) + \beta'(G) = 2 + 4 = 6 = |V(G)|$.

- $\nu(G, A)$ is the deficiency of G with respect to A (the maximum number of vertices in A that must remain unmatched in any matching)
- $N(S)$ is the neighborhood of set S (all vertices adjacent to at least one vertex in S)
- The maximum is taken over all subsets S of A

Theorem 70 (Relationship to König-Ore Formula). For bipartite graphs $G = (A \cup B, E)$ with $|A| = |B|$, Tutte's condition becomes equivalent to Hall's condition (the case of deficiency 0 in the König-Ore formula), as:

$$\text{odd}(G - S) \leq |S| \quad \text{for all } S \subseteq V$$

reduces to Hall's condition:

$$|N(T)| \geq |T| \quad \text{for all } T \subseteq A$$

when applied to bipartite graphs.

Corollary 31. A bipartite graph $G(A, B)$ has a matching that covers all vertices in A if and only if for every subset $S \subseteq A$, $|N(S)| \geq |S|$.

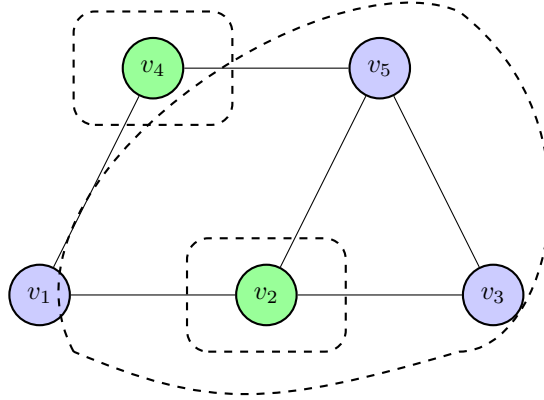
Remark 85. This is also known as Hall's Marriage Theorem, which states that a bipartite graph $G(A, B)$ has a matching covering all vertices in A if and only if for all $S \subseteq A$, $|N(S)| \geq |S|$.

Remark 86. In the context of the marriage problem, this means that a set of men can all be married if and only if any set of k men collectively know at least k women.

Remark 87. This condition is often called Hall's marriage condition and can be intuitively understood as: "A group of people can all get married if and only if any group of k people collectively know at least k potential spouses."

Example 30. Consider a bipartite graph representing possible marriages between n men and n women. Each edge represents that a particular man and woman would consider marrying each other. Hall's Theorem states that perfect matchings (where everyone gets married) are possible if and only if for any group of k men, there are at least k women who would consider marrying someone in that group.

Independent set $V(G) \setminus C = \{v_2, v_4\}$ (green vertices)



Vertex cover $C = \{v_1, v_3, v_5\}$ (blue vertices)

Figure 112: Illustrating $\alpha(G) + \beta(G) = |V(G)|$: This graph has a minimum vertex cover C of size $\beta(G) = 3$ (blue vertices) and a maximum independent set of size $\alpha(G) = 2$ (green vertices). The vertex cover and independent set partition the vertex set, and $\alpha(G) + \beta(G) = 2 + 3 = 5 = |V(G)|$.

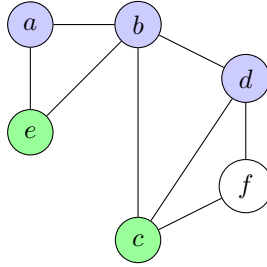


Figure 113: Illustration of neighborhood $N(X)$ in a graph. The set $X = \{e, c\}$ (green vertices) has neighborhood $N(X) = \{a, b, d, f\}$ (blue vertices). Note that $|N(X)| = 4 > 2 = |X|$. This graph is not bipartite (for example, a, b, e form a triangle), so it illustrates only the neighborhood of a set; Hall's condition and Hall's Theorem concern bipartite graphs.

Remark 88. The condition in Hall's Theorem is often easier to check by considering its contrapositive: a bipartite graph with bipartition (A, B) where $|A| = |B|$ has no perfect matching if and only if there exists a subset $X \subseteq A$ such that $|N(X)| < |X|$.

Corollary 32. If G is a k -regular bipartite graph, and $k \geq 1$, then G has a perfect matching.

Proof. Let G be a k -regular bipartite graph with bipartition (A, B) . Since G is k -regular, every vertex in A has exactly k neighbors in B , and every vertex in B has exactly k neighbors in A .

Let's verify Hall's condition. For any subset $X \subseteq A$, the vertices in X contribute exactly $k|X|$ edges to the graph. All these edges connect to vertices in $N(X) \subseteq B$. Since each vertex in $N(X)$ can be incident to at most k edges, we have:

$$k|X| \leq k|N(X)|$$

This simplifies to:

$$|X| \leq |N(X)|$$

Thus, Hall's condition is satisfied for any subset $X \subseteq A$. Moreover, counting the edges of G from each side gives $k|A| = |E(G)| = k|B|$, so $|A| = |B|$ because $k \geq 1$. Therefore, by Hall's Theorem, G has a perfect matching. $\square$

Example 31. Consider the following 3-regular bipartite graph G :

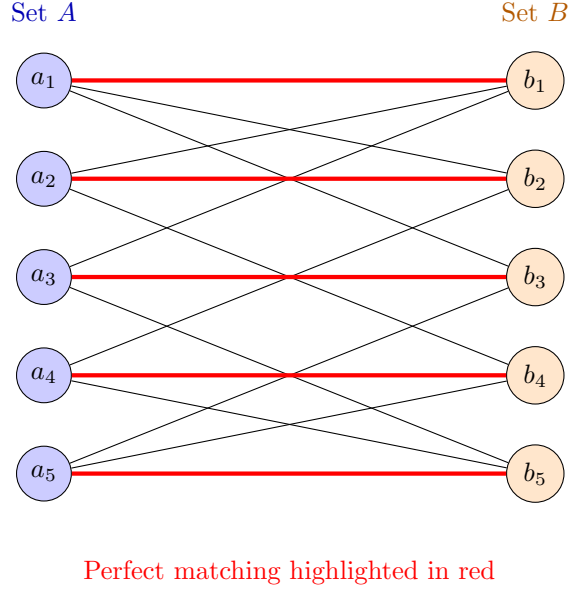


Figure 114: A 3-regular bipartite graph with a perfect matching. Each vertex has exactly 3 neighbors, and by the corollary, a perfect matching is guaranteed to exist.

In this 3-regular bipartite graph, each vertex is connected to exactly 3 vertices on the opposite side. According to the corollary, this graph must have a perfect matching. Indeed, a perfect matching is highlighted in red.

We can verify Hall's condition: for any subset X of the left partition, $|N(X)| \geq |X|$. For example:

- If $X = \{a_1\}$, then $N(X) = \{b_1, b_2, b_3\}$ and $|N(X)| = 3 > 1 = |X|$
- If $X = \{a_1, a_2\}$, then $N(X) = \{b_1, b_2, b_3, b_4\}$ and $|N(X)| = 4 > 2 = |X|$
- If $X = \{a_1, a_2, a_3, a_4, a_5\}$, then $N(X) = \{b_1, b_2, b_3, b_4, b_5\}$ and $|N(X)| = 5 = |X|$

Hall's condition is satisfied for all possible subsets of A , confirming the existence of a perfect matching.

Proof. We need to show if G has parts A and B , then for every $X \subseteq A$, $|N(X)| \geq |X|$.

Let $e(X, N(X))$ be the number of edges between X and $N(X)$.

Since G is k -regular, each vertex in X has exactly k neighbors in B . Therefore:

$$e(X, N(X)) = k|X|$$

On the other hand, each vertex in $N(X)$ is adjacent to at most k vertices in X (since the graph is k -regular):

$$e(N(X), X) \leq k|N(X)|$$

Since $e(X, N(X)) = e(N(X), X)$ (these represent the same set of edges), we have:

$$k|X| = e(X, N(X)) = e(N(X), X) \leq k|N(X)|$$

Dividing both sides by k (which is possible since $k \geq 1$):

$$|X| \leq |N(X)|$$

So $|N(X)| \geq |X|$ for all $X \subseteq A$.

By Hall's Theorem, there is a matching M covering A .

Since the graph is k -regular, the total number of edges in the graph is $k|A| = k|B|$, which implies $|A| = |B|$.

Therefore, M is a perfect matching. □

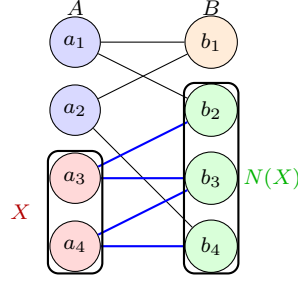


Figure 115: 2-regular bipartite graph with $X = \{a_3, a_4\}$ (red) and $N(X) = \{b_2, b_3, b_4\}$ (green). Each vertex in X has $k = 2$ neighbors, giving $e(X, N(X)) = k|X| = 4$ edges (blue). Since $|N(X)| = 3 > 2 = |X|$, Hall's condition is satisfied.

7.2 Hall's Theorem and Various Matching Algorithms

Theorem 71 (Hall's Theorem). Let G be a bipartite graph with parts A and B . Then G has a matching covering A if and only if for every $X \subseteq A$:

Hall's condition: $|N(X)| \geq |X|$

Proof. ($\Rightarrow$) If G has a matching M covering A , then for $X \subseteq A$ the vertices matched to X by M are $|X|$ distinct vertices of $N(X)$, so $|N(X)| \geq |X|$.

($\Leftarrow$) If $|N(X)| \geq |X| \forall X \subseteq A$, we have to find a matching covering A .

Proceed by induction on $|A|$:

Base case: $|A| = 1$. Then by Hall's condition $|N(A)| \geq |A|$, so a has a neighbor $b \in B$. Now $\{a, b\}$ gives a matching covering A .

Inductive step: Suppose $|A| \geq 2$ and the theorem is true for bipartite graphs with parts A' and B where $|A'| < |A|$.

Case 1: $|N(X)| > |X|$ for every nonempty proper subset $X \subsetneq A$

Pick any edge $ab \in E(G)$ where $a \in A$, $b \in B$. Let $G' = G - \{a\} - \{b\}$

Then G' has parts $A' = A - \{a\}$ and $B' = B - \{b\}$

For $X \subseteq A'$: $N_{G'}(X) \supseteq N_G(X) - \{b\}$. If $X \neq \emptyset$, then X is a nonempty proper subset of A , so $|N_{G'}(X)| \geq |N_G(X)| - 1 \geq |X|$ (and $X = \emptyset$ is trivial)

Therefore Hall's condition holds for G' . By induction, G' has a matching M' covering A' . Now $M = M' \cup \{ab\}$ is a matching covering A .

Case 2: $|N(X)| = |X|$ for some nonempty proper subset $X \subsetneq A$

Let $G_1 = G[X \cup N(X)]$, and let $G_2 = G - (X \cup N(X))$ be the bipartite graph with parts $A - X$ and $B - N(X)$.

If $Y \subseteq X$: $N_{G_1}(Y) = N_G(Y)$ and $|N_G(Y)| \geq |Y|$ by Hall's condition in G

So Hall's condition holds in G_1 .

Since $|X| < |A|$, by induction G_1 has a matching M_1 covering X .

Now let $Y \subseteq A - X$. Since $N_{G_2}(Y) = N_G(Y) - N_G(X) = N_G(Y \cup X) - N_G(X)$ and $N_G(X) \subseteq N_G(Y \cup X)$:

$$\begin{aligned} |N_{G_2}(Y)| &= |N_G(Y \cup X)| - |N_G(X)| \\ &\geq |Y \cup X| - |X| \\ &= |Y| \end{aligned}$$

So Hall's condition holds for G_2 . Since $|A - X| < |A|$, by induction there is a matching M_2 in G_2 covering $A - X$.

Now $M_1 \cup M_2$ is a matching in G covering A , because M_1 and M_2 have no vertex in common. $\square$

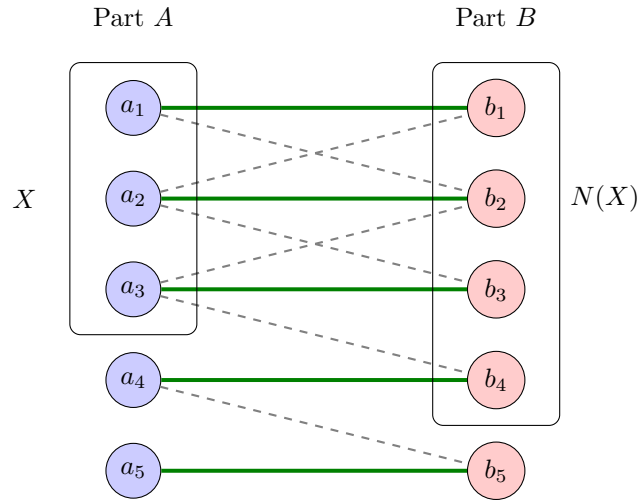


Figure 116: Illustration of Hall's Theorem. The green edges form a matching covering set A . The subset $X \subseteq A$ and its neighborhood $N(X)$ are outlined by boxes. Hall's condition requires that $|N(X)| \geq |X|$ for every $X \subseteq A$.

Case 1: $|N(X)| > |X|$ for all nonempty $X \subsetneq A$

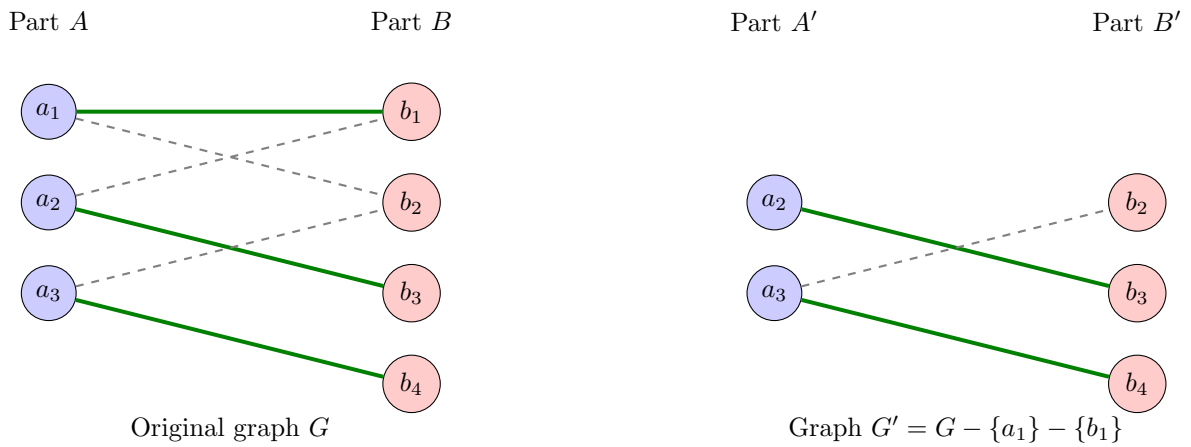


Figure 117: Illustration of Case 1 in the proof of Hall's Theorem. We remove an edge a_1b_1 and the vertices a_1 and b_1 to create G' .

Case 2: $|N(X)| = |X|$ for some nonempty $X \subsetneq A$

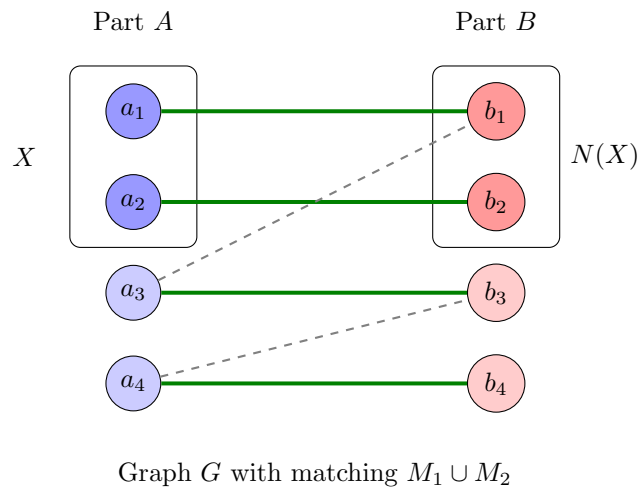


Figure 118: Illustration of Case 2 in the proof of Hall's Theorem. We have a nonempty proper subset $X \subsetneq A$ where $|N(X)| = |X|$. We find matchings M_1 in the subgraph induced by $X \cup N(X)$ and M_2 in the remaining graph.

Definition 85. Suppose we have people $p_1, p_2, \dots, p_n$ and committees $S_1, S_2, \dots, S_m$ which are sets of people.

A **System of Distinct Representatives (SDR)** is a selection of one person from each committee such that no person represents more than one committee.

For bipartite $G = (A \cup B, E)$ where:

- $A = \{S_1, S_2, \dots, S_m\}$
- $B = \{p_1, p_2, \dots, p_n\}$
- $(S_i, p_j) \in E$ if $p_j \in S_i$

An SDR in G is a matching covering A .

Theorem 72. Sets $S_1, S_2, \dots, S_m$ have an SDR if and only if for all $I \subseteq \{1, 2, \dots, m\}$:

$$\left| \bigcup_{i \in I} S_i \right| \geq |I| \quad (55)$$

Example 32. Suppose we have committees:

$$S_1 = \{p_1, p_2, p_3\}$$

$$S_2 = \{p_2, p_3, p_4\}$$

$$S_3 = \{p_1, p_4, p_5\}$$

Is there a system of distinct representatives?

Yes: $\{p_3, p_2, p_1\}$ where p_3 represents S_1 , p_2 represents S_2 , and p_1 represents S_3 .

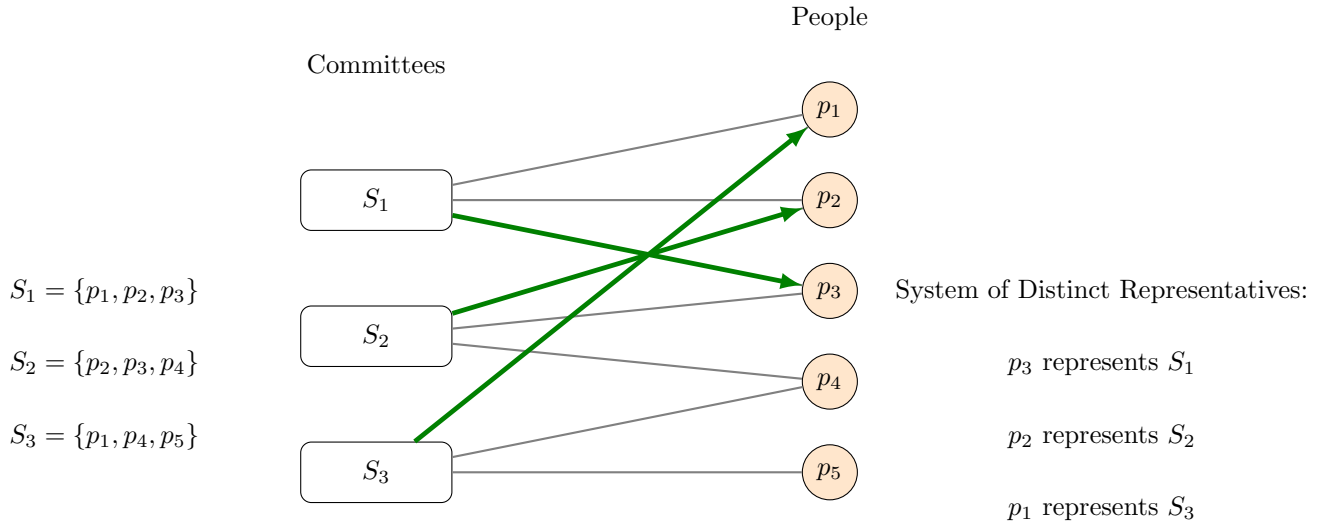


Figure 119: Illustration of a System of Distinct Representatives (SDR) for the given committees. The green arrows show the selected representatives, where p_3 represents S_1 , p_2 represents S_2 , and p_1 represents S_3 .

Example 33. Suppose we have committees:

$$S_1 = \{1, 4\}$$

$$S_2 = \{3, 5\}$$

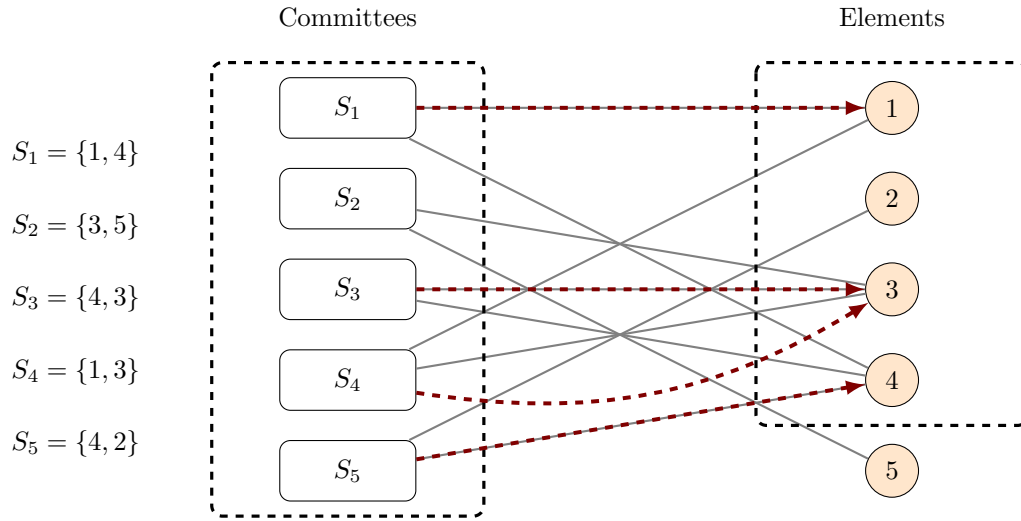
$$S_3 = \{4, 3\}$$

$$S_4 = \{1, 3\}$$

$$S_5 = \{4, 2\}$$

An SDR exists, even though Hall's condition holds with equality for some I : For $I = \{1, 3, 4, 5\}$ $|S_1 \cup S_3 \cup S_4 \cup S_5| = |\{1, 2, 3, 4\}| = 4 = |I|$

Equality is allowed by Hall's condition. One checks that $|\bigcup_{i \in I} S_i| \geq |I|$ for every $I \subseteq \{1, 2, 3, 4, 5\}$, so by the theorem an SDR exists: for example, 1, 5, 4, 3, 2 represent S_1, S_2, S_3, S_4, S_5 , respectively. (The only other SDR is 4, 5, 3, 1, 2.)



For the subset $I = \{S_1, S_3, S_4, S_5\}$, we have: $|S_1 \cup S_3 \cup S_4 \cup S_5| = |\{1, 2, 3, 4\}| = 4 = |I|$

Hall's condition is satisfied with equality for this I (and holds for every I), so an SDR exists. Equality only restricts the choices:

- Elements 1, 2, 3, 4 must all represent committees in I , so element 5 represents S_2
- Element 2 can only represent S_5
- Elements 1, 3, 4 then represent S_1, S_3, S_4 : either 1, 4, 3 or 4, 3, 1

The dashed arrows show a greedy attempt (1 for S_1 , 3 for S_3 , 4 for S_4 , 2 for S_5) that leaves nothing for S_2 ; choosing 4 for S_3 and 2 for S_5 instead frees 3 for S_4 .

Figure 120: A family of committees satisfying Hall's condition, with equality for $I = \{S_1, S_3, S_4, S_5\}$: the dashed box on the right marks their union $\{1, 2, 3, 4\}$, and $|I| = |\bigcup_{i \in I} S_i| = 4$. By Hall's theorem an SDR exists (e.g. 1, 5, 4, 3, 2 represent $S_1, \dots, S_5$); the dashed arrows show a greedy attempt that gets stuck and must be repaired.

Example 34. Suppose:

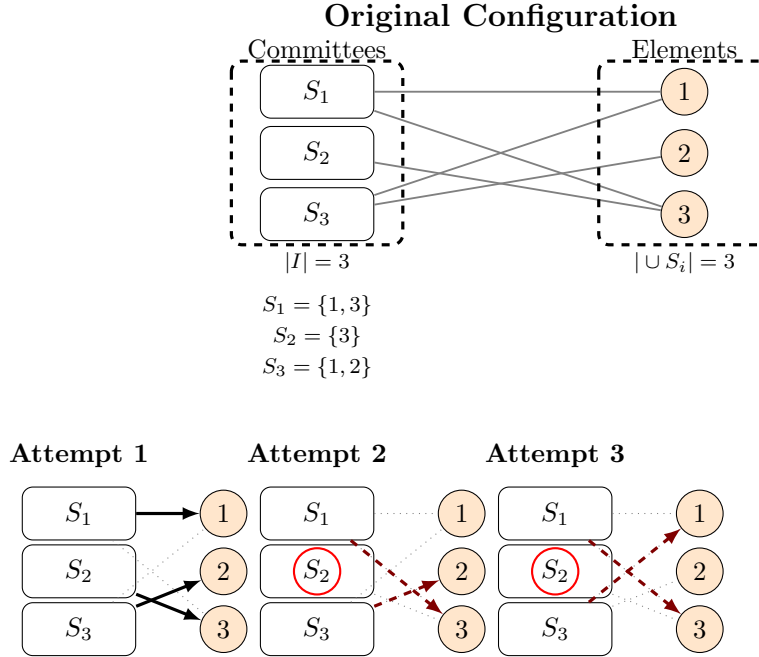
$$S_1 = \{1, 3\}$$

$$S_2 = \{3\}$$

$$S_3 = \{1, 2\}$$

An SDR exists, and it is unique. For $I = \{1, 2, 3\}$, we have: $|S_1 \cup S_2 \cup S_3| = |\{1, 2, 3\}| = 3 = |I|$

The condition is satisfied with equality here (and it holds for every I), so an SDR exists by the theorem. The structure forces it: 3 must represent S_2 , then 1 must represent S_1 , and 2 represents S_3 .



Explanation: For $I = \{S_1, S_2, S_3\}$, we have $|S_1 \cup S_2 \cup S_3| = |\{1, 2, 3\}| = 3 = |I|$. Hall's condition is satisfied (with equality for this I), so an SDR exists; here it is unique:

- **Attempt 1:** If we assign element 1 to S_1 and element 3 to S_2 , then element 2 is still available for S_3 , and we obtain an SDR.
- **Attempt 2:** If we assign element 3 to S_1 and element 2 to S_3 , then S_2 has no representative left.
- **Attempt 3:** If we assign element 3 to S_1 and element 1 to S_3 , then S_2 has no representative left.

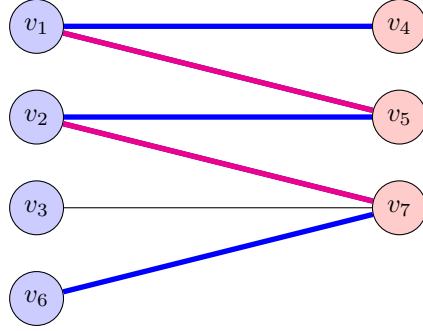
Since S_2 contains only element 3, element 3 must represent S_2 in any SDR. Then S_1 must be represented by element 1, and S_3 by element 2. Here Hall's condition holds with equality for $\{S_2\}$, $\{S_1, S_2\}$ and $\{S_1, S_2, S_3\}$, which forces every choice; equality does not prevent an SDR from existing.

Figure 121: A family that satisfies Hall's condition with equality ($|I| = |\cup_{i \in I} S_i| = 3$) and has exactly one System of Distinct Representatives. The top diagram shows the committee-element relationships, while the bottom row shows the unique SDR (Attempt 1) and two assignments that get stuck (Attempts 2 and 3).

Definition 86 (M-alternating path). Let G be a graph and M a matching in G . Then an **M-alternating path** is a path $P \subseteq G$ with

$$E(P) = \{\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{k-1}, v_k\}\}$$

such that edges are alternately in and not in M .

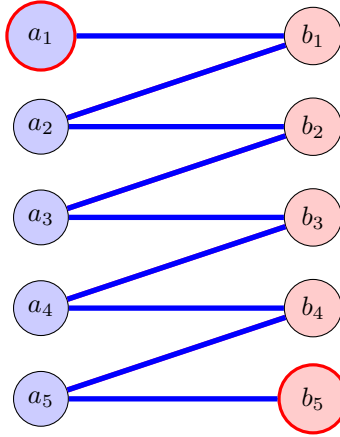


An M-alternating path: $P = (v_6, v_7, v_2, v_5, v_1, v_4)$

Edge sequence: not in M , in M , not in M , in M , not in M

Figure 122: Example of an M-alternating path. Matching edges are shown in magenta. The path alternates between edges in and not in the matching.

Definition 87 (M-augmenting path). An **M-augmenting path** is an M-alternating path starting and ending with edges not in M , and the first and last vertices of the path are exposed by M (i.e., not in any edge of M).



An M-augmenting path from exposed vertex a_1 to exposed vertex b_5

The path alternates between edges not in M and edges in M

Figure 123: Example of an M-augmenting path. The red circled vertices are exposed (not matched). The augmenting path starts and ends with exposed vertices.

Theorem 73 (Berge). If M is a matching in a graph G , then M is a maximum matching if and only if G contains no M-augmenting paths.

Hopcroft-Karp Algorithm Step by Step

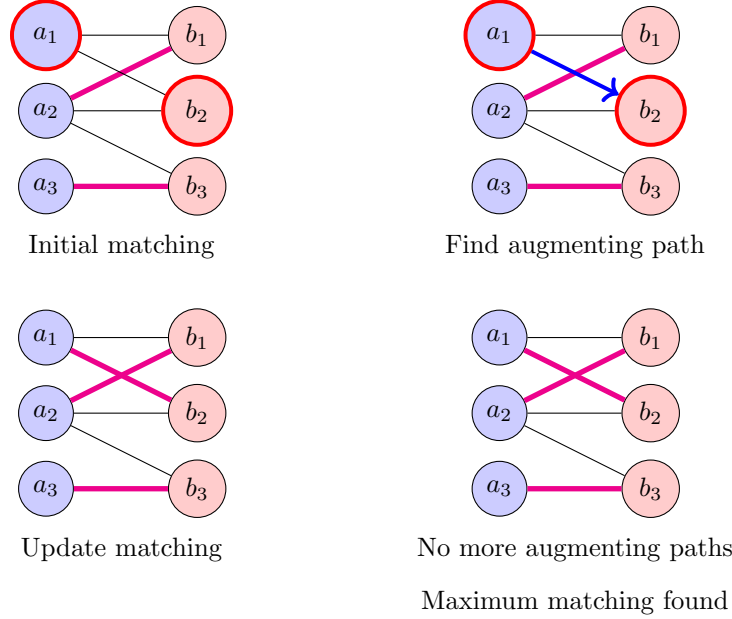


Figure 124: Complete execution of the Hopcroft-Karp algorithm on a small example. We start with a matching, find augmenting paths, update the matching, and continue until no more augmenting paths exist. At that point, we have found a maximum matching.

Algorithm 18 Hopcroft-Karp Algorithm

1: procedure HOPCROFTKARP($G = (U \cup V, E)$)	▷ G is a bipartite graph
2: $M \leftarrow \emptyset$	▷ Initialize empty matching
3: while true do	
4: $L \leftarrow \text{BuildLevelGraph}(G, M)$	▷ BFS to build level graph
5: if no augmenting path exists in L then	
6: return M	▷ No more augmenting paths, M is maximum
7: end if	
8: $P \leftarrow \emptyset$	▷ Set of vertex-disjoint augmenting paths
9: Mark all vertices as unvisited	
10: for all exposed vertices $u \in U$ do	
11: DFS(L, u, M, P)	▷ Look for one augmenting path starting at u
12: end for	
13: for all augmenting paths $p \in P$ do	
14: $M \leftarrow M \oplus p$	▷ Augment matching using symmetric difference
15: end for	
16: end while	
17: end procedure	

Algorithm 19 Build Level Graph

```
1: procedure BUILDLEVELGRAPH( $G = (U \cup V, E), M$ )
2:    $L \leftarrow$  empty graph
3:   level  $\leftarrow$  array of distances, initialized to  $\infty$ 
4:   queue  $\leftarrow$  all exposed vertices from  $U$ 
5:   for all exposed vertices  $u \in U$  do
6:     level[ $u$ ]  $\leftarrow 0$ 
7:   end for
8:   foundAugmentingPath  $\leftarrow$  false
9:   while queue is not empty AND not foundAugmentingPath do
10:     $u \leftarrow$  queue.dequeue()
11:    if  $u \in U$  then ▷ Vertex in left partition
12:      for all edges  $(u, v) \in E$  where  $(u, v) \notin M$  do ▷ Only non-matching edges
13:        if level[ $v$ ] =  $\infty$  then
14:          level[ $v$ ]  $\leftarrow$  level[ $u$ ] + 1
15:          queue.enqueue( $v$ )
16:        end if
17:        if level[ $v$ ] = level[ $u$ ] + 1 then
18:          Add edge  $(u, v)$  to  $L$  ▷ Keep all edges between consecutive layers
19:        end if
20:      end for
21:    else ▷  $u \in V$ , vertex in right partition
22:      if  $u$  is exposed then ▷ Found augmenting path
23:        foundAugmentingPath  $\leftarrow$  true
24:      else
25:        Let  $(v, u)$  be the unique edge in  $M$  containing  $u$ 
26:        if level[ $v$ ] =  $\infty$  then
27:          level[ $v$ ]  $\leftarrow$  level[ $u$ ] + 1
28:          Add edge  $(u, v)$  to  $L$  ▷ Add the matching edge to level graph
29:          queue.enqueue( $v$ )
30:        end if
31:      end if
32:    end if
33:  end while
34:  return  $L$ 
35: end procedure
```

Algorithm 20 DFS to Find Augmenting Path

```
1: procedure DFS( $L, u, M, P$ )
2:   Mark  $u$  as visited ▷ Keeps the paths in  $P$  vertex-disjoint
3:   if  $u$  is exposed in  $V$  then ▷ Found augmenting path ending at exposed vertex in  $V$ 
4:     Add current path to  $P$ 
5:     return true
6:   end if
7:   for all edges  $(u, v)$  in  $L$  with  $v$  unvisited do
8:     if DFS( $L, v, M, P$ ) then
9:       return true
10:    end if
11:  end for
12:  return false
13: end procedure
```

Definition 88 (Hopcroft-Karp Algorithm). The Hopcroft-Karp algorithm is a method for finding a maximum matching in a bipartite graph $G = (U \cup V, E)$ where U and V are disjoint vertex sets and all edges in E connect a vertex in U to a vertex in V . The algorithm iteratively increases the size of a matching by finding multiple shortest augmenting paths in each phase until no more augmenting paths exist. An augmenting path is defined as a path that starts and ends at unmatched vertices and alternates between edges not in the matching and edges in the matching.

1. **Multiple Augmenting Paths:** Unlike the standard augmenting path algorithm, Hopcroft-Karp finds multiple shortest augmenting paths in each phase.
2. **Level Graph:**
 - The algorithm builds a level graph using BFS where layers alternate between vertices in U and V .
 - Each layer contains vertices at the same distance from the source.
 - Paths in the level graph are guaranteed to be shortest augmenting paths.
3. **DFS for Finding Disjoint Paths:** After building the level graph, the algorithm uses DFS to find a maximal set of vertex-disjoint augmenting paths.
4. **Symmetric Difference:** For each augmenting path p , the matching is updated using symmetric difference: $M \oplus p = (M \setminus p) \cup (p \setminus M)$.
5. **Efficiency:** The algorithm performs at most $O(\sqrt{|V|})$ phases, each taking $O(|E|)$ time, yielding the overall $O(|E|\sqrt{|V|})$ time complexity.

Theorem 74 (Alternating Path Decomposition). For any two matchings M and M' in a graph G , the symmetric difference $M \oplus M'$ can be decomposed into alternating paths and alternating cycles.

Corollary 33. For any matching M that is not maximum, there exists an augmenting path P such that $M \oplus P$ is a matching with $|M \oplus P| = |M| + 1$.

Lemma 56 (Hopcroft-Karp). If no augmenting paths of length less than or equal to $2k - 1$ exist, then the matching M is at most $\frac{|M^*|}{k}$ edges away from a maximum matching M^* .

Lemma 57 (Monotonicity of Shortest Augmenting Paths). In successive phases of the Hopcroft-Karp algorithm, the length of the shortest augmenting paths is strictly increasing.

Lemma 58 (Phase Count). The Hopcroft-Karp algorithm performs at most $O(\sqrt{|V|})$ phases, where each phase increases the length of the shortest augmenting path.

Theorem 75 (Disjoint Augmenting Paths). Given a set of vertex-disjoint augmenting paths, applying all of them simultaneously results in a valid matching.

Lemma 59 (Length of Augmenting Paths). If M is a non-maximum matching and M^* is a maximum matching in graph G , then the symmetric difference $M \oplus M^*$ contains at least one M -augmenting path.

Theorem 76 (König–Egerváry). In any bipartite graph $G = (U \cup V, E)$, the size of a maximum matching equals the size of a minimum vertex cover:

$$\nu(G) = \tau(G).$$

Theorem 77 (Alternating Path Decomposition). For any two matchings M and M' in a graph G , the symmetric difference

$$M \oplus M' = (M \setminus M') \cup (M' \setminus M)$$

can be decomposed into a set of vertex-disjoint alternating paths and alternating cycles.

Corollary 34. If M is a non-maximum matching in G , then there exists an augmenting path P such that

$$|M \oplus P| = |M| + 1.$$

Lemma 60 (Hopcroft–Karp Distance Bound). Let M be the current matching and M^* a maximum matching in a bipartite graph G , and let $k \geq 1$. If there is no M -augmenting path of length $\leq 2k - 1$, then

$$|M| \geq \frac{k}{k+1} |M^*|.$$

Equivalently, $|M^*| - |M| \leq |M|/k$, so M is at most $|M|/k \leq |V|/(2k)$ edges short of optimal.

Lemma 61 (Phase Count). In the Hopcroft–Karp algorithm, the length of the shortest augmenting paths strictly increases between phases. Combined with Lemma 60 (applied after about $\sqrt{|V|}$ phases), this shows that the total number of phases is

$$O(\sqrt{|V|}).$$

Theorem 78 (Disjoint Augmenting Paths). Given a set of vertex-disjoint M -augmenting paths in the level graph, applying the symmetric-difference update on all of them simultaneously produces a valid matching of size increased by the number of paths.

Lemma 62 (Existence in Symmetric Difference). If M is a non-maximum matching and M^* is a maximum matching, then $M \oplus M^*$ contains at least one M -augmenting path.

Lemma 63 (Blocking-Flow Phase Bound). In a unit network, i.e. a network with unit capacities in which every vertex other than the source and the sink has a single entering edge or a single leaving edge (such as the flow network of a bipartite matching problem), each blocking-flow phase can be found in $O(|E|)$ time, and there are at most $O(\sqrt{|V|})$ such phases.

Definition 89 (Hungarian Algorithm). The Hungarian Algorithm (also known as the Kuhn–Munkres algorithm) is a combinatorial optimization method that solves the assignment problem in polynomial time. Given a weighted bipartite graph $G = (U \cup V, E)$ with weights $w : E \rightarrow \mathbb{R}$, the algorithm finds a perfect matching $M \subseteq E$ that minimizes (or maximizes) the sum of weights $\sum_{e \in M} w(e)$.

The algorithm works by iteratively improving a feasible vertex labeling and a matching until optimality is achieved. It maintains a set of feasible vertex labels $l : U \cup V \rightarrow \mathbb{R}$ that satisfy the condition $l(u) + l(v) \leq w(u, v)$ for all $(u, v) \in E$ (for minimization) or $l(u) + l(v) \geq w(u, v)$ (for maximization). The time complexity of the Hungarian Algorithm is $O(|V|^3)$.

Algorithm 21 Hungarian Algorithm (Minimization Version)

Require: Complete bipartite graph $G = (U \cup V, E)$ with $|U| = |V| = n$ and weights $w : E \rightarrow \mathbb{R}$

Ensure: Perfect matching M with minimum total weight

```

1: Initialize feasible labeling  $l : U \cup V \rightarrow \mathbb{R}$  where:
2:    $l(u) = \min_{v \in V} w(u, v)$  for all  $u \in U$ 
3:    $l(v) = 0$  for all  $v \in V$ 
4: Initialize matching  $M = \emptyset$ 
5: while  $M$  is not a perfect matching do
6:   Construct equality subgraph  $G_l = (U \cup V, E_l)$  where  $E_l = \{(u, v) \in E : l(u) + l(v) = w(u, v)\}$ 
7:   if there exists an augmenting path  $P$  in  $G_l$  then
8:      $M \leftarrow M \oplus P$  ▷ Augment matching using path  $P$ 
9:   else
10:     $Z \leftarrow \{ \text{vertices reachable from the unmatched vertices of } U \text{ through alternating paths in } G_l \}$ 
      (including these unmatched vertices)
11:     $S \leftarrow Z \cap U$  and  $T \leftarrow Z \cap V$ 
12:     $\alpha \leftarrow \min_{u \in S, v \in V \setminus T} \{w(u, v) - l(u) - l(v)\}$  ▷  $\alpha > 0$ 
13:    Update labels:
14:     $l(u) \leftarrow l(u) + \alpha$  for all  $u \in S$ 
15:     $l(v) \leftarrow l(v) - \alpha$  for all  $v \in T$ 
16:  end if
17: end while
18: return  $M$ 

```

Lemma 64 (Feasible Labeling Optimality). If l is a feasible labeling and M is a perfect matching in the equality graph G_l , then M is a minimum-weight perfect matching in the original graph G .

Lemma 65 (Complementary Slackness). For a feasible labeling l and a perfect matching M , if $(u, v) \in M$ implies $l(u) + l(v) = w(u, v)$, then M is an optimal (minimum-weight) perfect matching.

Lemma 66 (Labeling Update Invariant). Let $S \subseteq U$ and $T \subseteq V$ be the sets of vertices reachable from unmatched vertices of U by alternating paths in G_l . The update rule $l(u) \leftarrow l(u) + \alpha$ for $u \in S$ and $l(v) \leftarrow l(v) - \alpha$ for $v \in T$ (where $\alpha = \min_{u \in S, v \in V \setminus T} \{w(u, v) - l(u) - l(v)\}$) maintains the feasibility of the labeling and keeps every edge of M in the equality graph.

Lemma 67 (Labeling Value and Matching Weight). For any feasible labeling l and any perfect matching M , the sum of all vertex labels $\sum_{v \in U \cup V} l(v)$ provides a lower bound on the total weight of M .

Lemma 68 (Duality Gap). For a feasible labeling l and a perfect matching M , the difference between the weight of M and the sum of labels $\sum_{v \in U \cup V} l(v)$ is nonnegative, and it is zero if and only if every edge of M lies in the equality graph G_l ; in that case M is optimal. Conversely, an optimal M has zero gap with respect to every feasible labeling of maximum total value (see the Strong Duality theorem below), but not necessarily with respect to every feasible labeling.

Lemma 69 (Augmenting Path Property). If P is an augmenting path in the equality graph G_l , then applying the symmetric difference $M \oplus P$ strictly increases the size of the matching while maintaining the property that matched edges are in the equality graph.

Definition 90 (Kuhn-Munkres Algorithm). The Kuhn-Munkres algorithm (also known as the Hungarian algorithm) is a combinatorial optimization algorithm that solves the assignment problem in polynomial time. Given a weighted bipartite graph $G = (U \cup V, E)$ with $|U| = |V| = n$ and weight function $w : E \rightarrow \mathbb{R}$, it finds a perfect matching $M \subseteq E$ that minimizes the total weight $\sum_{e \in M} w(e)$.

The algorithm maintains a feasible vertex labeling $l : U \cup V \rightarrow \mathbb{R}$ and iteratively improves both the labeling and a partial matching until a perfect matching is found in the equality subgraph $G_l = (U \cup V, E_l)$ where $E_l = \{(u, v) \in E : l(u) + l(v) = w(u, v)\}$. By the theory of complementary slackness, when a perfect matching exists in G_l , it is guaranteed to be an optimal solution to the original assignment problem.

The time complexity of the Kuhn-Munkres algorithm is $O(n^3)$, where n is the number of vertices in each partition of the bipartite graph.

Theorem 79 (Kuhn-Munkres Optimality). If l is a feasible labeling and M is a perfect matching in the equality graph G_l , then M is an optimal assignment (minimum-weight perfect matching) for the original weighted bipartite graph G .

Lemma 70 (Feasible Labeling Lower Bound). For any feasible labeling l and any perfect matching M in G , the following inequality holds:

$$\sum_{u \in U} l(u) + \sum_{v \in V} l(v) \leq \sum_{(u, v) \in M} w(u, v) \quad (56)$$

Theorem 80 (Strong Duality). The minimum weight of a perfect matching equals the maximum value of $\sum_{u \in U} l(u) + \sum_{v \in V} l(v)$ over all feasible labelings l .

Lemma 71 (Invariant During Algorithm). Throughout the execution of the Kuhn-Munkres algorithm, for any edge $(u, v) \in M$, the equality $l(u) + l(v) = w(u, v)$ is maintained.

Lemma 72 (Labeling Update Property). The labeling update rule (applied when $N_l(S) = T$):

$$\alpha = \min_{u \in S, v \in V \setminus T} \{w(u, v) - l(u) - l(v)\} \quad (57)$$

$$l'(x) = \begin{cases} l(x) + \alpha & \text{if } x \in S \\ l(x) - \alpha & \text{if } x \in T \\ l(x) & \text{otherwise} \end{cases} \quad (58)$$

preserves the feasibility of the labeling and adds at least one new edge to the equality graph G_l .

Algorithm 22 Kuhn-Munkres Algorithm

Require: Complete bipartite graph $G = (U \cup V, E)$ with $|U| = |V| = n$ and weights $w : E \rightarrow \mathbb{R}$

Ensure: Perfect matching M with minimum total weight

```
1: Initialize feasible labeling  $l$ :
2:  $l(u) = \min_{v \in V} w(u, v)$  for all  $u \in U$ 
3:  $l(v) = 0$  for all  $v \in V$ 
4: Initialize  $M = \emptyset$  ▷ Current matching
5: Initialize  $S = \emptyset, T = \emptyset$  ▷ Vertex sets for alternating tree
6: while  $|M| < n$  do ▷ While matching is not perfect
7:   if  $S = \emptyset$  then
8:     Choose an unmatched vertex  $u \in U$ 
9:      $S \leftarrow \{u\}$ 
10:     $T \leftarrow \emptyset$ 
11:   end if
12:   if  $N_l(S) = T$  then ▷  $N_l(S)$  is the neighbors of  $S$  in the equality subgraph
13:      $\alpha \leftarrow \min_{u \in S, v \in V \setminus T} \{w(u, v) - l(u) - l(v)\}$  ▷  $\alpha > 0$ 
14:     Update labels:
15:      $l(u) \leftarrow l(u) + \alpha$  for all  $u \in S$ 
16:      $l(v) \leftarrow l(v) - \alpha$  for all  $v \in T$ 
17:   else
18:     Choose  $v \in N_l(S) \setminus T$ 
19:     if  $v$  is not matched in  $M$  then
20:       Augment  $M$  along the alternating path from unmatched  $u \in S$  to  $v$ 
21:        $S \leftarrow \emptyset, T \leftarrow \emptyset$  ▷ Reset alternating tree
22:     else
23:       Let  $(z, v) \in M$  be the edge matching  $v$ 
24:        $S \leftarrow S \cup \{z\}$ 
25:        $T \leftarrow T \cup \{v\}$ 
26:     end if
27:   end if
28: end while
29: return  $M$ 
```

Lemma 73 (Alternating Path Property). If P is an alternating path in the equality graph G_l from an unmatched vertex in U to an unmatched vertex in V , then $M \oplus P$ is a matching of size $|M| + 1$ that maintains the property that all matched edges belong to the equality graph.

Remark 89 (Relation to Linear Programming). *The assignment problem can be formulated as a linear program:*

$$\text{minimize} \quad \sum_{(u,v) \in E} w(u,v) \cdot x_{uv} \quad (59)$$

$$\text{subject to} \quad \sum_{v \in V} x_{uv} = 1 \quad \forall u \in U \quad (60)$$

$$\sum_{u \in U} x_{uv} = 1 \quad \forall v \in V \quad (61)$$

$$x_{uv} \geq 0 \quad \forall (u,v) \in E \quad (62)$$

The dual of this linear program corresponds to finding an optimal feasible vertex labeling.

Remark 90 (Integrality). *The constraint matrix of the assignment problem is totally unimodular, which guarantees that the linear program has an integer optimal solution, justifying the combinatorial approach of the Kuhn-Munkres algorithm.*

Remark 91 (Maximization Version). *The Kuhn-Munkres algorithm can be adapted to solve the maximum-weight assignment problem by negating all weights or by modifying the feasible labeling condition to $l(u) + l(v) \geq w(u,v)$.*

Remark 92 (Initialization Optimization). *A better initial feasible labeling can improve practical performance:*

$$l(u) = \max_{v \in V} w(u,v) \text{ for all } u \in U \text{ and } l(v) = 0 \text{ for all } v \in V \quad (63)$$

for the maximization version.

Definition 91 (Edmonds' Matching Algorithm). Let $G = (V, E)$ be an undirected graph. Edmonds' matching algorithm, also known as the Blossom algorithm, is a polynomial-time algorithm that finds a maximum matching $M \subseteq E$ such that:

1. No two edges in M share a common vertex (the definition of a matching).
2. M has the maximum possible cardinality $|M|$ among all matchings in G .
3. The algorithm works for any undirected graph, not only bipartite graphs.

The algorithm operates by iteratively finding augmenting paths in G relative to the current matching M . An augmenting path is an alternating path (a path where edges alternate between being in M and not in M) that starts and ends with unmatched vertices. When an augmenting path P is found, the matching is updated by taking the symmetric difference $M \oplus P$, which increases the size of the matching by one edge.

The key innovation of Edmonds' algorithm is the handling of *blossoms* – odd-length cycles in the graph that require special treatment. When a blossom is detected, it is temporarily contracted into a single vertex, the matching problem is solved in the resulting contracted graph, and then the blossom is expanded to obtain a solution for the original graph.

The time complexity of the original algorithm is $O(|V|^4)$, where $|V|$ is the number of vertices in the graph. More efficient implementations achieve $O(|V|^3)$ and even $O(|V| \cdot |E|)$ complexity, and the algorithm of Micali and Vazirani runs in $O(|E| \sqrt{|V|})$ time.

Lemma 74 (Blossom Lemma). Let $G = (V, E)$ be a graph with matching M . Suppose $B \subset V$ forms a blossom with respect to M , i.e. B is the vertex set of a cycle of length $2k + 1$ containing k edges of M , and suppose that the base of B (the vertex of the cycle not matched to another vertex of the cycle) is either free or joined to a free vertex by an M -alternating path of even length that meets B only in the base (a *stem*); every blossom contracted by FINDAUGMENTINGPATH has this property. If G' is the graph obtained by contracting B to a single vertex b , with matching M' derived from M , then G contains an augmenting path with respect to M if and only if G' contains an augmenting path with respect to M' .

Algorithm 23 Find Augmenting Path

```
1: procedure FINDAUGMENTINGPATH( $G = (V, E), M$ )
2:   Let  $F$  be the forest with one single-vertex tree  $\{r\}$  for each free vertex  $r$ ; label each root even
3:   while there is an unexamined edge  $(u, v)$  such that  $u$  is an even vertex of  $F$  do
4:     Mark  $(u, v)$  as examined
5:     if  $v$  is not in  $F$  then ▷ then  $v$  is matched, since every free vertex is a root of  $F$ 
6:       Let  $w$  be the vertex matched to  $v$  in  $M$ 
7:       Add  $v, w$ , and edges  $(u, v), (v, w)$  to  $F$ ; label  $v$  odd and  $w$  even
8:     else if  $v$  is even and lies in a different tree from  $u$  then
9:       return the augmenting path from the root of  $u$ 's tree to  $u$ , then to  $v$ , then to the root of  $v$ 's
      tree
10:    else if  $v$  is even and lies in the same tree as  $u$  then
11:      Contract the blossom formed by  $(u, v)$  and the tree paths from  $u$  and  $v$  to their lowest common
      ancestor into a single even vertex
12:      Update  $F, G$  and  $M$  accordingly ▷ a path found later is lifted by expanding the blossom
13:    else ▷  $v$  is odd
14:      Ignore the edge  $(u, v)$ 
15:    end if
16:  end while
17:  return No augmenting path exists
18: end procedure
```

Theorem 81 (Tutte-Berge Formula). For any graph $G = (V, E)$, the size of a maximum matching in G is given by:

$$\nu(G) = \min_{S \subseteq V} \frac{1}{2}(|V| - \text{odd}(G - S) + |S|)$$

where $\text{odd}(G - S)$ denotes the number of connected components in $G - S$ having an odd number of vertices, and the minimum is taken over all subsets S of V .

Lemma 75 (Gallai-Edmonds Decomposition). For any graph $G = (V, E)$, the vertex set V can be partitioned into three sets D, A , and C as follows:

- $D = \{v \in V : v \text{ is not covered by at least one maximum matching}\}$
- $A = \{v \in V \setminus D : v \text{ is adjacent to at least one vertex in } D\}$
- $C = V \setminus (D \cup A)$

Moreover, this decomposition has the following properties:

1. Each connected component of the subgraph induced by D has an odd number of vertices.
2. Every maximum matching in G contains a perfect matching on the subgraph induced by C .
3. Every maximum matching in G matches each vertex in A with a vertex in D , with each component of $G[D]$ matched to at most one vertex in A .
4. The size of a maximum matching in G is $\frac{1}{2}(|V| - \text{odd}(G[D]) + |A|)$, where $\text{odd}(G[D])$ is the number of odd components in $G[D]$.

Theorem 82 (Dulmage-Mendelsohn Decomposition). Let $G = (X \cup Y, E)$ be a bipartite graph with bipartition (X, Y) . Let X_1 (resp. Y_3) be the set of vertices of X (resp. Y) that are left unmatched by at least one maximum matching of G . Put $Y_1 = N(X_1)$, $X_3 = N(Y_3)$, $X_2 = X \setminus (X_1 \cup X_3)$ and $Y_2 = Y \setminus (Y_1 \cup Y_3)$. Then:

1. X_1, X_2, X_3 is a partition of X and Y_1, Y_2, Y_3 is a partition of Y (in particular, there are no edges between X_1 and Y_3).

2. There are no edges between X_1 and $Y_2 \cup Y_3$, and no edges between Y_3 and $X_1 \cup X_2$.
3. Every maximum matching of G matches each vertex of Y_1 to a vertex of X_1 , matches each vertex of X_3 to a vertex of Y_3 , and matches X_2 perfectly onto Y_2 .
4. Consequently $|X_2| = |Y_2|$ and $\nu(G) = |Y_1| + |X_2| + |X_3|$.

Theorem 83 (Petersen's Theorem). Every 3-regular graph with no bridges has a perfect matching.

Remark 93. *Tutte's theorem is a generalization of Petersen's theorem, which states that every 3-regular graph with no bridges has a perfect matching.*

Lemma 76 (Blossoms Intersection). Let B_1 and B_2 be two distinct blossoms that are contracted during the same search for an augmenting path in Edmonds' algorithm, viewed as sets of vertices of the original graph G . If $B_1 \cap B_2 \neq \emptyset$, then either:

1. $B_1 \subset B_2$, i.e., one blossom is entirely contained within the other, or
2. $B_2 \subset B_1$.

That is, these blossoms form a laminar family. (For two arbitrary blossoms with respect to M this fails: two 5-cycles through the same four vertices can both be blossoms.)

Algorithm 24 Contract Blossom

```

1: procedure CONTRACTBLOSSOM( $G = (V, E)$ ,  $F$ ,  $u$ ,  $v$ )
2:   Find the lowest common ancestor  $w$  of  $u$  and  $v$  in  $F$ 
3:   Let  $B$  be the vertices in the cycle formed by the paths from  $w$  to  $u$  and  $w$  to  $v$ , plus the edge  $(u, v)$ 
4:   Create a new vertex  $b$  representing the blossom  $B$ 
5:   Replace all edges incident to vertices in  $B$  with edges to  $b$ 
6:   Update the matching  $M$  to reflect the contraction
7:   Update the forest  $F$  to replace the blossom with  $b$ 
8:   return the contracted graph and updated forest
9: end procedure

```

Algorithm 25 Expand Blossom

```

1: procedure EXPANDBLOSSOM( $G$ ,  $M$ ,  $B$ )
2:   Let  $b$  be the vertex representing blossom  $B$ 
3:   if  $b$  is not matched in  $M$  then
4:     Expand  $b$  back to the original vertices in  $B$ 
5:     Keep the matching edges within  $B$  as they were before contraction
6:   else
7:     Let  $(b, v)$  be the edge in  $M$  incident to  $b$ 
8:     Find the vertex  $u \in B$  such that  $(u, v)$  was an edge in the original graph
9:     Expand  $b$  back to the original vertices in  $B$ 
10:    Include  $(u, v)$  in  $M$ 
11:    Match the remaining vertices in  $B$  optimally along the blossom
12:   end if
13:   return the expanded graph and updated matching
14: end procedure

```

Remark 94. *The time complexity of Edmonds' original algorithm is $O(|V|^4)$ where $|V|$ is the number of vertices in the graph; the straightforward implementation described above runs in $O(|V|^2|E|)$ time. This can be improved to $O(|V|^3)$ or even $O(|V||E|)$ with more sophisticated implementations, and to $O(|E|\sqrt{|V|})$ by the algorithm of Micali and Vazirani.*

Definition 92 (Stable Matching). Let A and B be two disjoint sets of equal size, where each element in set A has a strict preference ordering over the elements in set B , and each element in set B has a strict preference ordering over the elements in set A . A matching M is a one-to-one correspondence between the elements of A and B .

A matching M is said to be *stable* if and only if there does not exist any pair $(a, b) \in A \times B$ such that:

1. $(a, b) \notin M$, i.e., a and b are not matched to each other in M , and
2. a prefers b to its current partner in M , and
3. b prefers a to its current partner in M .

Such a pair (a, b) is called a *blocking pair* or an *unstable pair*. A stable matching is thus a matching with no blocking pairs.

Theorem 84 (Gale-Shapley). For any instance of the stable matching problem with equal-sized sets, a stable matching always exists and can be found in $O(n^2)$ time, where n is the size of each set.

Definition 93 (Hospital-Resident Problem). The Hospital-Resident problem (also known as the College Admissions problem) is a generalization of the stable matching problem where:

- Set B (hospitals) may be matched to multiple elements from set A (residents).
- Each hospital $h \in B$ has a capacity c_h indicating the maximum number of residents it can accommodate.
- A matching M is stable if there is no resident-hospital pair (r, h) such that:
 1. r is unmatched or prefers h to their current assignment, and
 2. h is under capacity or prefers r to at least one of its currently matched residents.

Definition 94 (Stable Roommates Problem). The Stable Roommates problem is a variation of the stable matching problem where:

- There is only one set of individuals (rather than two disjoint sets).
- Each person has a preference ranking over all other individuals.
- A matching pairs individuals as roommates.
- A matching is stable if there are no two individuals who prefer each other to their assigned roommates.

Unlike the original stable matching problem, a stable solution is not guaranteed to exist in the Stable Roommates problem.

Definition 95 (Gale-Shapley Algorithm). The Gale-Shapley Algorithm, also known as the Deferred Acceptance Algorithm, is a procedure to find a stable matching between two equally sized sets, where each element has a strict preference ordering over elements in the other set.

Let $A = \{a_1, a_2, \dots, a_n\}$ and $B = \{b_1, b_2, \dots, b_n\}$ be the two sets, with each element having a preference list ranking all elements of the other set.

Theorem 85 (Properties of Gale-Shapley Algorithm). The Gale-Shapley algorithm has the following properties:

1. It always terminates in at most n^2 iterations, where $n = |A| = |B|$.
2. It always produces a stable matching.
3. The resulting matching is A -optimal, meaning that each $a \in A$ receives the best partner possible in any stable matching.
4. The resulting matching is B -pessimal, meaning that each $b \in B$ receives the worst partner possible in any stable matching.

Algorithm 26 Gale-Shapley Algorithm

```
1: procedure GALESHAPLEY( $A, B$ , preferences)
2:   Initialize all  $a \in A$  and  $b \in B$  as unmatched
3:   Initialize a list nextProposal[ $a$ ] for each  $a \in A$ , initially pointing to their most preferred  $b$ 
4:   while there exists an unmatched  $a \in A$  who has not proposed to all  $b \in B$  do
5:     Select any such unmatched  $a$ 
6:     Let  $b = \text{nextProposal}[a]$  be the highest-ranked  $b$  to whom  $a$  has not yet proposed
7:     Advance nextProposal[ $a$ ] to the next  $b$  in  $a$ 's preference list
8:     if  $b$  is unmatched then
9:       Match  $a$  and  $b$ 
10:    else if  $b$  prefers  $a$  to their current match  $a'$  then
11:      Unmatch  $b$  and  $a'$ 
12:      Match  $a$  and  $b$ 
13:    end if
14:  end while
15:  return the current matching
16: end procedure
```

By reversing the roles of sets A and B , we obtain the B -optimal (and A -pessimal) stable matching.

Theorem 86 (Optimality of Gale-Shapley). In the matching produced by the Gale-Shapley algorithm with elements from set A proposing:

1. Every $a \in A$ receives the best partner that they can receive in any stable matching.
2. Every $b \in B$ receives the worst partner that they can receive in any stable matching.

Theorem 87 (Rural Hospitals Theorem). In the hospital-resident variation of the stable matching problem:

1. The same set of hospitals and residents are matched in all stable matchings.
2. Any hospital that does not fill its quota in one stable matching will have exactly the same residents in all stable matchings.
3. The same number of residents is matched in every stable matching.

Theorem 88 (König-Egerváry). Let $G = (X \cup Y, E)$ be a bipartite graph with bipartition (X, Y) . Then the maximum size of a matching in G equals the minimum size of a vertex cover in G .

In other words:

$$\nu(G) = \tau(G)$$

where $\nu(G)$ denotes the matching number (the size of a maximum matching) and $\tau(G)$ denotes the vertex cover number (the size of a minimum vertex cover).

Corollary 35 (Dual Form of König-Egerváry). In a bipartite graph $G = (X \cup Y, E)$, the maximum size of a matching equals the number of vertices minus the maximum size of an independent set:

$$\nu(G) = |V| - \alpha(G)$$

where $\alpha(G)$ is the independence number of G (the size of a maximum independent set).

Corollary 36 (König's Line Coloring Theorem). For any bipartite graph G , the edge chromatic number equals the maximum degree:

$$\chi'(G) = \Delta(G)$$

where $\chi'(G)$ is the minimum number of colors needed to color the edges of G such that no two adjacent edges receive the same color, and $\Delta(G)$ is the maximum degree of any vertex in G .

Remark 95. *The König-Egerváry theorem is a special case of Edmonds' matching polytope theorem for bipartite graphs, and it fails to hold for general non-bipartite graphs.*

Remark 96. *The theorem provides a min-max relation that is fundamental in combinatorial optimization and establishes a duality between matching and vertex cover problems in bipartite graphs. It is also equivalent to the integrality of the matching polytope for bipartite graphs.*

Theorem 89 (Relationship Between König-Egerváry Theorem and König-Ore Formula). Let $G = (A \cup B, E)$ be a bipartite graph. The following relationship exists between the König-Egerváry theorem and the König-Ore formula:

$$\tau(G) = \nu(G) \quad (\text{König-Egerváry}) \quad (64)$$

$$\nu(G) = |A| - \max_{S \subseteq A} \{|S| - |N(S)|\} \quad (\text{From König-Ore}) \quad (65)$$

where:

- $\tau(G)$ is the minimum size of a vertex cover
- $\nu(G)$ is the maximum size of a matching
- $\nu(G, A) = \max_{S \subseteq A} \{|S| - |N(S)|\}$ is the deficiency of G with respect to A (from König-Ore)

Thus, the maximum matching size can be expressed as:

$$\nu(G) = |A| - \nu(G, A)$$

This shows that the deficiency $\nu(G, A)$ from the König-Ore formula directly determines how far a maximum matching is from being perfect (covering all vertices in A).

Corollary 37. The relationships can be further connected through Hall's Marriage Theorem:

1. A bipartite graph has a matching covering all vertices in A if and only if $\nu(G, A) = 0$
2. This is equivalent to the condition that for all $S \subseteq A$, $|N(S)| \geq |S|$ (Hall's condition)
3. When $|A| = |B|$, this means the graph has a perfect matching if and only if $\nu(G) = |A| = |B|$
4. By König-Egerváry, this means there is a perfect matching if and only if the minimum vertex cover has size $|A| = |B|$

Theorem 90 (Complementary Perspective). The dual perspectives of König-Egerváry and König-Ore can be unified:

1. König-Egerváry establishes the global equality between maximum matching and minimum vertex cover
2. König-Ore provides a local characterization of deficiency through the "worst" substructures
3. Together, they show that:

$$\tau(G) = |A| - \max_{S \subseteq A} \{|S| - |N(S)|\}$$

Remark 97. *Both theorems are manifestations of LP duality:*

- *König-Egerváry reflects that the matching and vertex cover polytopes have integer extreme points in bipartite graphs*
- *König-Ore formula shows that the dual solution can be constructed from subsets with the maximum deficiency*
- *Edmonds' algorithm for non-bipartite graphs addresses precisely the cases where these relationships become more complex due to odd cycles (blossoms)*

7.3 Matchings and Factors Exercises

7.3.1 Easy Questions

1. Let G be a bipartite graph with parts A and B . Show that if G has a matching covering A , then $|A| \leq |B|$. Give an example showing that $|A| \leq |B|$ alone does not guarantee such a matching.
2. Find a maximum matching and a minimum vertex cover of the path P_7 and of the cycle C_7 . Verify that $\nu(P_7) = \tau(P_7) = 3$ but $\nu(C_7) = 3 < 4 = \tau(C_7)$, and explain why this does not contradict the König-Egerváry theorem.
3. Show that K_{2n} has exactly $(2n-1)(2n-3)\cdots 3\cdot 1$ perfect matchings and that $K_{n,n}$ has exactly $n!$ perfect matchings.
4. Prove that a tree has at most one perfect matching.
5. Let M be a matching in a graph G and let P be an M -augmenting path. Prove that $M \oplus E(P)$ is a matching with $|M| + 1$ edges.
6. Verify Hall's condition for the sets $S_1 = \{1, 2\}$, $S_2 = \{2, 3\}$, $S_3 = \{1, 3\}$, $S_4 = \{3, 4\}$, and find an SDR.
7. Explain why a graph with an odd number of vertices has no perfect matching, and show that this is the case $S = \emptyset$ of Tutte's condition.

7.3.2 Medium Questions

8. Prove that every k -regular bipartite graph with $k \geq 1$ has a perfect matching, and deduce that its edge set can be partitioned into k perfect matchings.
9. Prove the König-Ore formula $\nu(G) = |A| - \max_{S \subseteq A} \{|S| - |N(S)|\}$ for a bipartite graph G with parts A and B . (Hint: add $d = \max_{S \subseteq A} \{|S| - |N(S)|\}$ new vertices to B , each adjacent to every vertex of A , and apply Hall's theorem.)
10. Use the Hungarian algorithm to find a minimum-cost assignment for the cost matrix

$$\begin{pmatrix} 4 & 1 & 3 \\ 2 & 0 & 5 \\ 3 & 2 & 2 \end{pmatrix},$$

where entry (i, j) is the cost of assigning worker i to job j . (Answer: worker 1 to job 2, worker 2 to job 1, worker 3 to job 3, with total cost 5.)

11. Show that the Petersen graph has exactly 6 perfect matchings.
12. A matching M is *maximal* if no edge of G can be added to it. Prove that every maximal matching M satisfies $|M| \geq \nu(G)/2$, and show that equality can occur in the path P_4 .
13. Let M be a matching in a graph G such that G has no M -augmenting path of length at most $2k - 1$, where $k \geq 1$. Prove that $|M| \geq \frac{k}{k+1} \nu(G)$.

7.3.3 Hard Questions

14. Prove Petersen's theorem from Tutte's theorem: every 3-regular graph with no bridges has a perfect matching.
15. Construct a 3-regular graph with no perfect matching and verify that it violates Tutte's condition. (Hint: take K_4 , subdivide one of its edges by a new vertex x , make three copies of this 5-vertex graph, and join the three copies of x to one new vertex.)
16. Prove the Tutte-Berge formula from Tutte's theorem. (Hint: let $d = \max_{S \subseteq V} \{\text{odd}(G - S) - |S|\}$ and join a complete graph K_d to every vertex of G .)

17. Prove Petersen's 2-factor theorem: every $2k$ -regular graph with $k \geq 1$ has a 2-factor, i.e. a spanning 2-regular subgraph. (Hint: orient each component along an Eulerian circuit, form a bipartite graph with a copy v^+ and a copy v^- of each vertex v and an edge u^+v^- for each arc uv , and apply Question 8.)
18. Prove Berge's theorem: a matching M in a graph G is maximum if and only if G has no M -augmenting path. (Hint: consider the components of $M \oplus M^*$ for a maximum matching M^* .)

7.3.4 Diagram Questions

19. Draw a bipartite graph with parts $A = \{a_1, a_2, a_3, a_4\}$ and $B = \{b_1, b_2, b_3, b_4\}$ that has no perfect matching, and mark a set $X \subseteq A$ with $|N(X)| < |X|$.
20. Draw the 16-vertex 3-regular graph of Question 15, mark its three bridges, and mark a set S of vertices such that $G - S$ has more than $|S|$ odd components.

8 Vertex and Edge-Coloring

8.1 Basic Definitions

Definition 96 (Proper k -edge-coloring). A *proper k -edge-coloring* of a graph G is an assignment of k colors to the edges of G such that no two adjacent edges (i.e., edges that share a common vertex) receive the same color. Formally, it is a function $c : E(G) \rightarrow \{1, 2, \dots, k\}$ such that if edges e_1 and e_2 share a vertex, then $c(e_1) \neq c(e_2)$.

Definition 97 (Edge-chromatic number). The *edge-chromatic number* of a graph G , denoted by $\chi'(G)$, is the minimum number of colors needed for a proper edge-coloring of G . That is, $\chi'(G) = \min\{k : G \text{ has a proper } k\text{-edge-coloring}\}$.

Definition 98 (k -edge-colorable). A graph G is said to be *k -edge-colorable* if there exists a proper edge-coloring of G using at most k colors. Equivalently, G is k -edge-colorable if $\chi'(G) \leq k$.

Definition 99 (k -edge-chromatic). A graph G is said to be *k -edge-chromatic* if its edge-chromatic number is exactly k , i.e., $\chi'(G) = k$. This means that G requires exactly k colors for a proper edge-coloring, no more and no less.

Definition 100 (Proper k -coloring). A *proper k -coloring* of a graph G is an assignment of k colors to the vertices of G such that no two adjacent vertices (i.e., vertices connected by an edge) receive the same color. Formally, it is a function $c : V(G) \rightarrow \{1, 2, \dots, k\}$ such that if vertices u and v are adjacent, then $c(u) \neq c(v)$.

Definition 101 (Chromatic number). The *chromatic number* of a graph G , denoted by $\chi(G)$, is the minimum number of colors needed for a proper vertex-coloring of G . That is, $\chi(G) = \min\{k : G \text{ has a proper } k\text{-coloring}\}$.

Definition 102 (k -colorable). A graph G is said to be *k -colorable* if there exists a proper vertex-coloring of G using at most k colors. Equivalently, G is k -colorable if $\chi(G) \leq k$.

Definition 103 (k -chromatic). A graph G is said to be *k -chromatic* if its chromatic number is exactly k , i.e., $\chi(G) = k$. This means that G requires exactly k colors for a proper vertex-coloring, no more and no less.

8.2 König's Theorem on Edge-Coloring of Bipartite Graphs

8.2.1 Background and Motivation

In graph theory, the problem of edge-coloring concerns assigning colors to the edges of a graph such that no two adjacent edges (those sharing a common vertex) receive the same color. A key question is determining the minimum number of colors required for such a proper edge-coloring of a given graph.

For any graph G , a trivial lower bound on the edge-chromatic number $\chi'(G)$ is the maximum degree $\Delta(G)$ of the graph. This is because all edges incident to a vertex of maximum degree must receive distinct colors. In general graphs, determining the exact value of $\chi'(G)$ is NP-hard, but for bipartite graphs, König's Theorem gives us a precise answer.

8.2.2 Formal Statement

Theorem 91 (König's Theorem on Edge-Coloring). If G is a bipartite graph, then the edge-chromatic number $\chi'(G)$ equals the maximum degree $\Delta(G)$ of the graph:

$$\chi'(G) = \Delta(G) \tag{66}$$

Proof via Regular Extension. The key insight in this proof is to extend the bipartite graph to a regular bipartite graph, which allows us to apply properties of matchings.

We first establish that $\chi'(G) \geq \Delta(G)$ for any graph G , since edges incident to a vertex of maximum degree must have different colors. To prove that $\chi'(G) \leq \Delta(G)$ for a bipartite graph $G = (A \cup B, E)$, we

construct a $\Delta(G)$ -regular bipartite graph H that contains G as a subgraph. To construct H , take two copies of G , denoted $G_1 = (A_1 \cup B_1, E_1)$ and $G_2 = (A_2 \cup B_2, E_2)$. For each vertex $v \in G$ with degree $d_G(v) < \Delta(G)$, we add $\Delta(G) - d_G(v)$ new edges between the corresponding vertices $v_1 \in G_1$ and $v_2 \in G_2$. The resulting graph H is $\Delta(G)$ -regular, and it is bipartite with partite sets $A_1 \cup B_2$ and $B_1 \cup A_2$: the edges of G_1 join A_1 to B_1 , the edges of G_2 join A_2 to B_2 , and each new edge $v_1 v_2$ joins A_1 to A_2 (if $v \in A$) or B_1 to B_2 (if $v \in B$). If $\Delta(G) - d_G(v) \geq 2$ for some v , then H has parallel edges, so in general H is a bipartite multigraph. By Hall's Theorem, which holds for bipartite multigraphs as well, a k -regular bipartite (multi)graph with $k \geq 1$ can be decomposed into k perfect matchings. Therefore, H can be decomposed into $\Delta(G)$ perfect matchings $M_1, M_2, \dots, M_{\Delta(G)}$. Each matching M_i defines a color class. The subgraph of H corresponding to G inherits this coloring, which gives us a proper $\Delta(G)$ -edge-coloring of G . Therefore, $\chi'(G) \leq \Delta(G)$. Combining this with the lower bound, we conclude that $\chi'(G) = \Delta(G)$. $\square$

Proof via Induction. This proof uses induction on the number of edges and a color-interchange argument. Base case: If $|E(G)| = 0$, then $\chi'(G) = 0 = \Delta(G)$, and the theorem holds. Induction hypothesis: Assume the theorem holds for all bipartite graphs with fewer than m edges, where $m \geq 1$. Induction step: Let G be a bipartite graph with exactly m edges and maximum degree $\Delta(G)$. Select any edge $e = \{x, y\} \in E(G)$. By the induction hypothesis, the graph $G' = G - e$ has a proper edge-coloring using $\Delta(G')$ colors. If $\Delta(G') < \Delta(G)$, then G' can be colored with at most $\Delta(G) - 1$ colors. In this case, we can assign a new color $\Delta(G)$ to edge e , obtaining a proper $\Delta(G)$ -edge-coloring of G . If $\Delta(G') = \Delta(G)$, then G' has a proper $\Delta(G)$ -edge-coloring. If there exists a color c that doesn't appear at either x or y , we can assign color c to edge e to obtain a proper $\Delta(G)$ -edge-coloring of G . Otherwise, no color is missing at both x and y . Since x and y each have at most $\Delta(G) - 1$ incident edges in G' , some color is missing at x and some color is missing at y ; after renaming the colors we may assume that color $\Delta(G)$ is missing at x and color 1 is missing at y . Then color 1 appears at x and color $\Delta(G)$ appears at y . Consider the subgraph H of G' containing only edges colored 1 or $\Delta(G)$. Each component of H is either a path or an even cycle (since G is bipartite). The component containing x must be a path P (not a cycle) that starts at x , since x has no incident edge of color $\Delta(G)$ in G' . This path P cannot end at y , because if it did, P would have odd length (since G is bipartite), and y would be incident with an edge of color 1, which contradicts our assumption. So P ends at a vertex $z \neq y$. Now we swap colors 1 and $\Delta(G)$ along the path P . This is still a proper edge-coloring of G' , but now color 1 is missing at both x and y . Finally, we assign color 1 to edge $e = \{x, y\}$, resulting in a proper $\Delta(G)$ -edge-coloring of G . Therefore, $\chi'(G) \leq \Delta(G)$. Since we already know that $\chi'(G) \geq \Delta(G)$ for any graph, we conclude that $\chi'(G) = \Delta(G)$. $\square$

8.2.3 Implications and Significance

König's Theorem has several important implications:

1. **Computational Tractability:** Unlike general graphs where determining the edge-chromatic number is NP-hard, for bipartite graphs, we can determine the edge-chromatic number simply by finding the maximum degree of the graph, which can be done in linear time.
2. **Relation to Matchings:** König's Theorem is closely related to matchings in bipartite graphs. In fact, a proper edge-coloring of any graph decomposes its edge set into matchings (its color classes); when G is k -regular and bipartite and k colors are used, every color class is a perfect matching. Thus König's Theorem says that the edges of a bipartite graph can be partitioned into $\Delta(G)$ matchings.
3. **Sharpening of Vizing's Theorem for Bipartite Graphs:** Vizing's Theorem states that for any simple graph G , $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$. König's Theorem does not follow from Vizing's Theorem; it sharpens it for bipartite graphs by showing that they always achieve the lower bound (and it holds for bipartite multigraphs as well).
4. **Application to Scheduling:** In scheduling problems where conflicts are represented by a bipartite graph, König's Theorem guarantees that the minimum number of time slots needed equals the maximum number of conflicts any single entity has.

8.2.4 Equivalent Formulations

König's Theorem can be equivalently formulated in several ways:

Theorem 92 (Equivalent Formulation - Decomposition into Matchings). A bipartite graph G with maximum degree $\Delta(G)$ can be decomposed into exactly $\Delta(G)$ matchings.

Theorem 93 (Equivalent Formulation - Regular Supergraph). Every bipartite graph G can be embedded in a $\Delta(G)$ -regular bipartite graph H such that $\chi'(H) = \Delta(G)$.

8.2.5 Historical Context

König's Theorem was first proven by the Hungarian mathematician Dénes König in 1916, making it one of the earliest results in graph theory. It laid the foundation for many subsequent developments in edge-coloring theory and bipartite graph theory.

The importance of this theorem extends beyond theoretical graph theory into various practical applications, including scheduling, resource allocation, and network design where bipartite conflict structures are common.

Lemma 77 (Color Alternating Path). Let G be a graph with a proper edge coloring. For any two colors α and β , each component of the subgraph induced by edges colored α or β is either a path or an even cycle where colors α and β alternate.

Proof. Let H be the subgraph of G induced by edges colored α or β . Since the coloring is proper, each vertex in H has degree at most 2 (at most one incident edge of each color). Therefore, each component of H is either a path or a cycle.

In any such component, the colors must alternate, since no two adjacent edges can have the same color. In a cycle, this alternation implies that the cycle must have even length, since we need to return to the starting color after traversing the cycle. $\square$

8.3 Vizing's Theorem

8.3.1 Formal Statement

Theorem 94 (Vizing's Theorem). For any simple graph G , the edge-chromatic number $\chi'(G)$ satisfies:

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1 \quad (67)$$

where $\Delta(G)$ is the maximum degree of G .

This theorem establishes tight bounds on the edge-chromatic number of any simple graph, classifying all graphs into two categories:

- **Class 1 graphs:** Graphs where $\chi'(G) = \Delta(G)$
- **Class 2 graphs:** Graphs where $\chi'(G) = \Delta(G) + 1$

The lower bound $\chi'(G) \geq \Delta(G)$ is immediate since all edges incident to a vertex of maximum degree must receive different colors. The upper bound $\chi'(G) \leq \Delta(G) + 1$ is the non-trivial part of the theorem.

Proof. Since Δ different colors are needed at a vertex of degree Δ in G , we have $\chi'(G) \geq \Delta$.

We now prove by induction on $|E(G)|$ that G is $(\Delta + 1)$ -edge-colorable, which gives $\chi'(G) \leq \Delta + 1$.

Base Case: If $|E(G)| = 0$, then the theorem is trivially true.

Induction Step: Suppose $|E(G)| > 0$, and let $\{x, y_1\} \in E(G)$ be any edge of G . By induction hypothesis, $G_1 = G - \{x, y_1\}$ is $(\Delta + 1)$ -edge-colorable.

If there is a color, say color c_1 , missing at both y_1 and x , then we can assign edge $\{x, y_1\}$ the color c_1 and we're done.

So we may assume that every color missing at x appears on an edge incident to y_1 . Let c be a color missing at x – we know c appears on y_1 , otherwise $\{x, y_1\}$ could be colored with color c .

In general, we construct a maximal sequence $y_1, y_2, \dots, y_k$ of distinct neighbors of x such that:

- c_i is missing at y_i for all $i \leq k$
- $\{x, y_{i+1}\}$ has color c_i for all $i < k$
- Color c_k is missing at y_k and does not appear on any edge $\{x, y\}$ for $y \notin \{y_1, y_2, \dots, y_k\}$

We now consider two cases:

Case 1. For all $i < k$, $c_k \neq c_i$.

In this case, a proper edge-coloring of G is found by recoloring $\{x, y_j\}$ with color c_j for all $j \leq k$. Note that the coloring is proper since color c_j is missing at y_j for all $j \leq k$. This is essentially a "fan" recoloring around vertex x .

Case 2. For some $i < k$, $c_k = c_i$.

In this case, we recolor all edges $\{x, y_j\}$ for $j \leq i$ with color c_j – so far we still have a proper coloring since color c_j is missing at y_j for $j \leq i$.

Then $\{x, y_{i+1}\}$ is the new uncolored edge, since the edge $\{x, y_1\}$ has now received color c_1 . Let H denote the subgraph of G consisting of edges of color c and edges of color c_k . Then the components of H are paths and cycles, since H has maximum degree at most two.

Also x has degree exactly one in H (color c is missing at x , while $c_k = c_i$ appears on $\{x, y_i\}$), and y_{i+1} and y_k have degree at most one in H , since c_k is missing at both of them: at y_k by the choice of c_k , and at y_{i+1} because its only edge of color $c_i = c_k$ was $\{x, y_{i+1}\}$, which is now uncolored. Moreover $y_{i+1} \neq y_k$: the edge $\{x, y_k\}$ has color c_{k-1} while c_k is missing at y_k , so $c_k \neq c_{k-1}$ and hence $i < k - 1$. A path has only two ends, so x , y_{i+1} , and y_k cannot all lie in the same component of H . Therefore either x and y_{i+1} are in different components of H , or x and y_k are in different components of H . We consider these cases separately:

Subcase 2.1: If x and y_{i+1} are in different components of H , then we interchange colors c and c_i in the component of H containing y_{i+1} . In this new coloring, color c is missing at x and missing at y_{i+1} , so we can assign the edge $\{x, y_{i+1}\}$ the color c .

Subcase 2.2: If x and y_k are in different components of H , then we recolor the edge $\{x, y_j\}$ for $i < j < k$ with color c_j , so that $\{x, y_k\}$ is the new uncolored edge. Then H is unchanged (we never recolored edges of color c or c_i) so we may interchange the colors c and c_k in the component of H containing y_k . In doing so, c becomes a missing color at x and y_k , so the uncolored edge $\{x, y_k\}$ can be colored with color c .

This completes the proof. $\square$

8.3.2 Key Lemmas Supporting Vizing's Theorem

Lemma 78 (Missing Colors). Let G be a simple graph, let uv be an edge of G , and let ϕ be a proper $(\Delta(G) + 1)$ -edge-coloring of $G - uv$. Then at least one color is missing at every vertex (that is, it appears on no edge incident to that vertex), and at least two colors are missing at each of u and v .

Proof. The edges incident to a vertex w receive distinct colors, and w has at most $\Delta(G)$ incident edges in $G - uv$, so at most $\Delta(G)$ of the $\Delta(G) + 1$ colors appear at w . The vertices u and v have at most $\Delta(G) - 1$ incident edges in $G - uv$, so at least $(\Delta(G) + 1) - (\Delta(G) - 1) = 2$ colors are missing at each of them. $\square$

Remark 98 (Vizing's Adjacency Lemma). *The name Vizing's Adjacency Lemma refers to a deeper result of Vizing (1965) about critical graphs. A simple graph G with maximum degree Δ is Δ -critical if $\chi'(G) = \Delta + 1$ and $\chi'(H) < \chi'(G)$ for every proper subgraph H of G . Vizing's Adjacency Lemma states that if G is Δ -critical and xy is an edge of G , then x is adjacent to at least $\Delta - d(y) + 1$ vertices of degree Δ other than y . Its proof refines the fan-recoloring argument used above.*

Lemma 79 (Color Alternating Path Lemma). Let ϕ be a proper edge-coloring of a graph G , and let α and β be two colors. Each component of the subgraph of G induced by edges colored α or β is either a path or an even cycle where the colors α and β alternate.

Proof. Consider the subgraph H of G induced by edges colored α or β . At each vertex, there can be at most one edge of color α and at most one edge of color β . Thus, each vertex in H has degree at most 2. A connected graph with maximum degree 2 is either a path or a cycle.

In any such component, the colors must alternate between α and β , since no two adjacent edges can have the same color in a proper coloring. In a cycle, this alternation implies that the cycle must have even length, since we need to return to the starting color after traversing the cycle. $\square$

8.3.3 Implications and Applications

Vizing's Theorem has profound implications for edge coloring theory and practical applications:

1. **Classification of Graphs:** The theorem partitions all simple graphs into exactly two classes: Class 1 graphs where $\chi'(G) = \Delta(G)$ and Class 2 graphs where $\chi'(G) = \Delta(G) + 1$.
2. **Computational Complexity:** While Vizing's Theorem gives tight bounds, determining whether a graph is Class 1 or Class 2 is NP-complete in general.
3. **Relationship to Bipartite Graphs:** König's Theorem states that all bipartite graphs are Class 1, i.e., $\chi'(G) = \Delta(G)$ for bipartite graphs.
4. **Scheduling Applications:** In scheduling problems, Vizing's Theorem guarantees that any conflict graph can be edge-colored with at most one more color than the maximum degree, providing efficient resource allocation.
5. **Network Design:** For communication networks, the theorem provides bounds on the minimum number of channels needed to avoid interference.

8.3.4 Historical Context

Vizing's Theorem was first proven by Vadim G. Vizing in 1964. The proof presented here is a refinement of Vizing's original proof, emphasizing the constructive aspects of the proof which naturally lead to polynomial-time algorithms for finding a $(\Delta(G) + 1)$ -edge-coloring of any simple graph.

The theorem represents one of the most elegant and important results in edge coloring theory, and despite its age, finding efficient algorithms for determining whether a graph is Class 1 or Class 2 remains an active area of research.

8.3.5 Examples of Vizing's and König's Theorem

A key result related to Vizing's Theorem is König's Line Coloring Theorem, which states that any bipartite graph is Class 1. Here's an example of a bipartite graph with maximum degree 3:

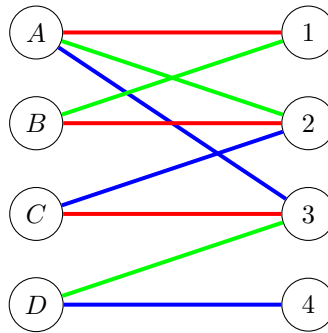


Figure 125: This bipartite graph has a maximum degree $\Delta(G) = 3$ (vertex A has 3 edges). As König's theorem predicts, we can color all edges using exactly 3 colors (red, blue, and green). This demonstrates that the graph is Class 1.

The complete graph K_3 (a triangle) has maximum degree $\Delta(K_3) = 2$. According to Vizing's theorem, it requires either 2 or 3 colors. Let's see:

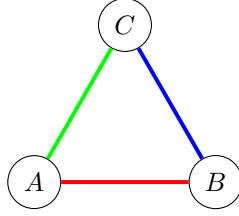


Figure 126: Here, we need 3 colors for the 3 edges, showing that K_3 is actually Class 2 since $\chi'(K_3) = 3 = \Delta(K_3) + 1$.

The complete graph K_4 has maximum degree $\Delta(K_4) = 3$. Let's try to color it:

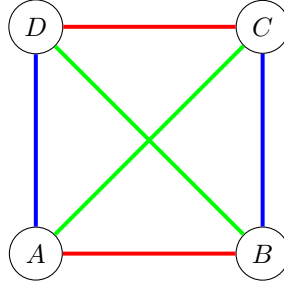


Figure 127: The three perfect matchings $\{AB, CD\}$, $\{BC, DA\}$, and $\{AC, BD\}$ form a proper 3-edge-coloring of K_4 . Hence $\chi'(K_4) = 3 = \Delta(K_4)$, and K_4 is a Class 1 graph, as is K_n for every even n .

The cycle graph C_5 has maximum degree $\Delta(C_5) = 2$. Let's color its edges:

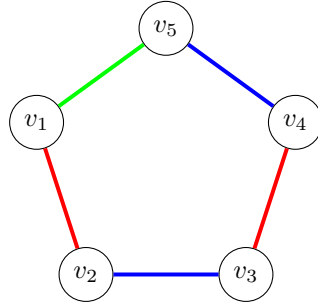


Figure 128: Two colors do not suffice for C_5 : they would have to alternate around the cycle, which is impossible for a cycle of odd length. Giving one edge a third color yields a proper edge-coloring, so $\chi'(C_5) = 3 = \Delta(C_5) + 1$ and C_5 is a Class 2 graph, as is every odd cycle.

The Petersen graph is a famous example of a Class 2 graph. It has maximum degree $\Delta = 3$:

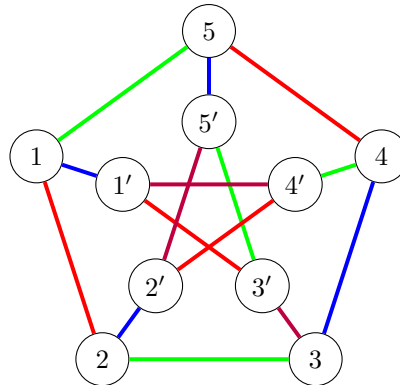


Figure 129: The Petersen graph requires 4 colors, making it a Class 2 graph since $\chi'(Petersen) = 4 = \Delta + 1$.

Here's a cubic graph (3-regular) that is Class 1. Every vertex has degree 3, and we can color it with exactly 3 colors:

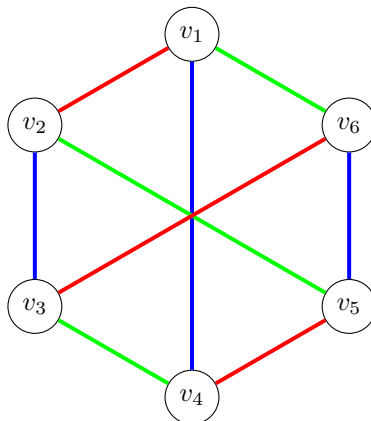


Figure 130: This 3-regular graph can be colored with exactly 3 colors, so it's a Class 1 graph.

Here's another example of a Class 2 graph with maximum degree 3:

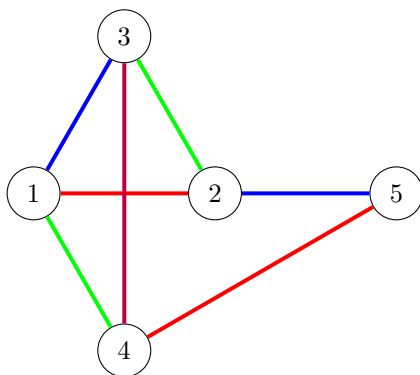


Figure 131: This graph is obtained from K_4 by replacing the edge between 2 and 4 with the path 2, 5, 4. It has maximum degree $\Delta = 3$ but requires 4 colors for its edges: each color class is a matching, and a matching in a graph with 5 vertices has at most 2 edges, so 3 colors could cover at most 6 of the 7 edges. Hence $\chi' = 4 = \Delta + 1$, and the graph is Class 2.

Determining whether a graph is Class 1 or Class 2 is generally NP-complete. However, some specific cases are known:

- Bipartite graphs are always Class 1 (König's theorem)
- Complete graphs K_n are Class 1 if n is even, and Class 2 if $n \geq 3$ is odd
- Cycles C_n are Class 1 if n is even, and Class 2 if n is odd
- A graph with n vertices and more than $\Delta \lfloor n/2 \rfloor$ edges (an *overfull* graph) is Class 2, because each color class is a matching and so has at most $\lfloor n/2 \rfloor$ edges; in particular, every regular graph with an odd number of vertices and at least one edge is Class 2. A large maximum degree alone does not force Class 2: K_4 , K_6 , and the wheel W_5 have $\Delta > \frac{n}{2}$ but are Class 1
- The Petersen graph is Class 2. In contrast, every other generalized Petersen graph $GP(n, k)$ with $1 \leq k < n/2$ (an outer cycle $u_1 u_2 \cdots u_n$, inner edges $v_i v_{i+k}$ with indices modulo n , and spokes $u_i v_i$; the Petersen graph is $GP(5, 2)$) is Class 1 (Castagna and Prins, 1972)

The complete graph K_5 has maximum degree $\Delta(K_5) = 4$. According to Vizing's theorem, it requires at most 5 colors:

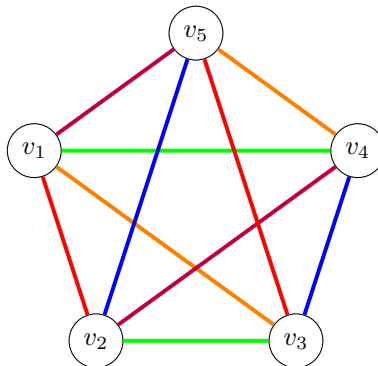


Figure 132: K_5 requires 5 colors, demonstrating it's a Class 2 graph since $\chi'(K_5) = 5 = \Delta(K_5) + 1$.

Before Vizing's theorem (1964), Shannon proved in 1949 that for any loopless multigraph G (in particular, for any simple graph), the edge chromatic number is bounded by:

$$\chi'(G) \leq \left\lfloor \frac{3\Delta(G)}{2} \right\rfloor \quad (68)$$

For simple graphs, Vizing later improved this bound to:

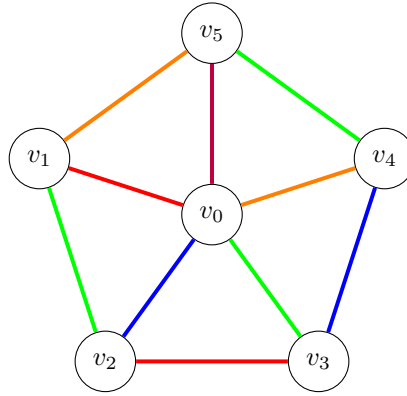
$$\chi'(G) \leq \Delta(G) + 1 \quad (69)$$

For multigraphs, Vizing's theorem gives $\chi'(G) \leq \Delta(G) + \mu(G)$ instead, where $\mu(G)$ is the largest number of parallel edges joining two vertices, and this is not always better than Shannon's bound.

For a simple graph with maximum degree $\Delta = 4$:

- Shannon's bound gives $\chi'(G) \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = \lfloor 6 \rfloor = 6$
- Vizing's bound gives $\chi'(G) \leq 4 + 1 = 5$

For comparison, here is a graph that attains the lower bound $\chi'(G) = \Delta(G)$: the wheel W_5 has $\Delta = 5$ and its edges can be colored with exactly 5 colors. (A graph with $\Delta = 4$ that attains Vizing's bound, K_5 , is discussed right after it.)



W_5 (Wheel graph): $\Delta = 5$, $\chi'(W_5) = 5$

Figure 133: This is a wheel graph W_5 with maximum degree $\Delta = 5$ at the center vertex. It requires exactly 5 colors for its edge coloring, showing a case where $\chi'(G) = \Delta(G)$ (Class 1).

For K_5 :

- Maximum degree $\Delta(K_5) = 4$
- Shannon's bound gives $\chi'(K_5) \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = 6$
- Vizing's bound gives $\chi'(K_5) \leq 4 + 1 = 5$
- Actual edge chromatic number $\chi'(K_5) = 5$

K_5 requires exactly 5 colors, showing that Vizing's bound is tight (this is a Class 2 graph).

To better understand Shannon's bound, here is a multigraph with maximum degree $\Delta = 6$ that needs all $\lfloor \frac{3\Delta}{2} \rfloor = 9$ colors. No simple graph can do this: by Vizing's theorem, a simple graph with $\Delta = 6$ needs at most 7 colors.

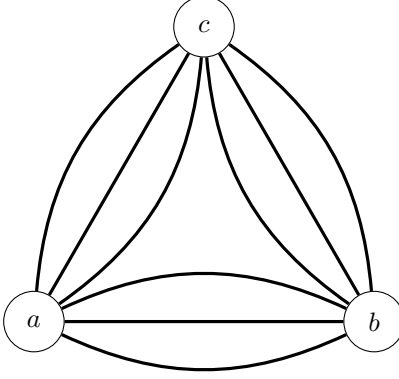


Figure 134: The Shannon multigraph with three parallel edges between each pair of vertices. Every vertex has degree $\Delta = 6$, and any two of the 9 edges share an endpoint, so all 9 edges need distinct colors: $\chi' = 9 = \lfloor \frac{3\Delta}{2} \rfloor$.

Here's a simple example showing a graph where the difference between Shannon's and Vizing's bounds is clear:

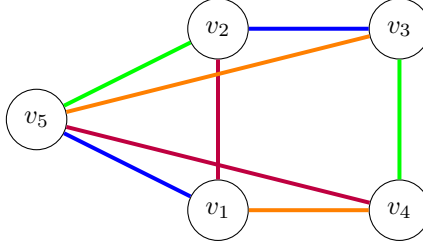


Figure 135: Graph with $\Delta = 4$ (at vertex v_5). Edge chromatic number $\chi' = 4$.

This graph (the wheel W_4) has:

- Maximum degree $\Delta = 4$ at vertex v_5
- Shannon's bound: $\chi' \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = 6$
- Vizing's bound: $\chi' \leq 4 + 1 = 5$
- Actual edge chromatic number: $\chi' = 4$, attained by the coloring shown (so this graph is Class 1)

This example demonstrates that, for simple graphs, Vizing's bound provides a tighter and more accurate upper limit than Shannon's earlier bound.

8.4 Shannon's Edge Coloring Theorem

In 1949, Claude Shannon established a fundamental result in graph theory regarding edge coloring. This theorem predates Vizing's more refined result from 1964.

Definition 104 (Edge Coloring). An edge coloring of a graph $G = (V, E)$ is an assignment of colors to the edges such that no two adjacent edges (i.e., edges sharing a common vertex) have the same color. The minimum number of colors needed for an edge coloring of G is called the edge chromatic number, denoted by $\chi'(G)$.

Theorem 95 (Shannon's Edge Coloring Theorem (1949)). For any loopless multigraph G (in particular, for any simple graph), the edge chromatic number $\chi'(G)$ is bounded by:

$$\chi'(G) \leq \left\lfloor \frac{3\Delta(G)}{2} \right\rfloor \quad (70)$$

where $\Delta(G)$ is the maximum degree of G (parallel edges are counted in the degree).

8.4.1 Explanation of Shannon's Bound

Shannon's bound tells us that for any graph with maximum degree Δ , we can color all of its edges using at most $\lfloor \frac{3\Delta}{2} \rfloor$ colors.

For simple graphs, Vizing's later theorem tightened the bound to $\Delta(G)+1$, so for simple graphs Shannon's bound is looser, especially as Δ increases. (For multigraphs, Vizing's bound is $\Delta(G) + \mu(G)$, where $\mu(G)$ is the largest number of parallel edges joining two vertices, and neither bound implies the other: for two vertices joined by m parallel edges, $\Delta + \mu = 2m$ exceeds Shannon's bound $\lfloor 3m/2 \rfloor$.)

Maximum Degree Δ	Shannon's Bound $\lfloor \frac{3\Delta}{2} \rfloor$	Vizing's Bound $\Delta + 1$ (simple)	Difference
2	3	3	0
3	4	4	0
4	6	5	1
5	7	6	1
6	9	7	2
7	10	8	2
8	12	9	3
10	15	11	4

As we can see, the gap between Shannon's bound and Vizing's bound grows as the maximum degree increases.

8.4.2 Proof of Shannon's Edge Coloring Theorem

The proof of Shannon's edge coloring theorem requires a methodical approach that builds upon several foundational results in graph theory. Unlike the previous sections, the proof necessitates the introduction of auxiliary lemmas and theorems that establish the necessary framework for understanding Shannon's elegant bound. This structured approach allows us to navigate the technical aspects of the proof while maintaining clarity throughout the argument.

Preliminaries

Let $G = (V, E)$ be a loopless multigraph with maximum degree $\Delta = \Delta(G)$. We aim to show that G can be edge-colored using at most $\lfloor 3\Delta/2 \rfloor$ colors.

Definition 105 (Line Graph). The line graph $L(G)$ of a graph G is constructed as follows:

- Each vertex in $L(G)$ corresponds to an edge in G .
- Two vertices in $L(G)$ are adjacent if and only if their corresponding edges in G are adjacent (share a common endpoint).

Definition 106 (Vertex Coloring). A vertex coloring of a graph $G = (V, E)$ is an assignment of colors to the vertices such that no two adjacent vertices receive the same color. The minimum number of colors needed for a vertex coloring of G is called the vertex chromatic number, denoted by $\chi(G)$.

Lemma 80. Edge coloring G is equivalent to vertex coloring $L(G)$.

Proof. A proper edge coloring of G assigns different colors to adjacent edges. This is equivalent to assigning different colors to adjacent vertices in $L(G)$, which is a proper vertex coloring of $L(G)$. $\square$

Key Insight: Analyzing the Maximum Degree of $L(G)$

Lemma 81. If G has maximum degree Δ , then the maximum degree of $L(G)$ is at most $2\Delta - 2$.

Proof. Consider an edge $e = \{u, v\}$ in G . This corresponds to a vertex in $L(G)$. How many adjacent vertices can this vertex have in $L(G)$?

The edge e is adjacent to:

- At most $\Delta - 1$ other edges incident to u (since u has degree at most Δ , and we're excluding e itself)
- At most $\Delta - 1$ other edges incident to v (since v has degree at most Δ , and we're excluding e itself)

Therefore, the total number of edges adjacent to e in G is at most $(\Delta - 1) + (\Delta - 1) = 2\Delta - 2$. This means the vertex in $L(G)$ corresponding to e has degree at most $2\Delta - 2$.

Since this applies to any edge e in G , the maximum degree of $L(G)$ is at most $2\Delta - 2$. $\square$

Constructive Coloring Approach

Recall that the color classes of a proper edge-coloring are matchings, and the edges at a single vertex must lie in pairwise different color classes.

Lemma 82. Let v be a vertex of maximum degree Δ in G . The edges incident to v can be partitioned into Δ matchings $M_1, M_2, \dots, M_\Delta$.

Proof. Let $e_1, e_2, \dots, e_\Delta$ be the edges incident to v . We can define:

- $M_1 = \{e_1\}$
- $M_2 = \{e_2\}$
- $\dots$
- $M_\Delta = \{e_\Delta\}$

Each M_i is trivially a matching since it contains just one edge. $\square$

Let $G = (V, E)$ be a loopless multigraph with maximum degree $\Delta = \Delta(G)$. We aim to show that G can be edge-colored using at most $\lceil 3\Delta/2 \rceil$ colors.

Definition 107 (Clique). A clique in a graph is a subset of vertices such that every two distinct vertices in the subset are adjacent (connected by an edge). In other words, a clique forms a complete subgraph where all possible edges between the vertices exist.

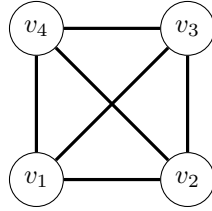


Figure 136: A 4-vertex clique K_4 where every pair of vertices is connected by an edge

Lemma 83. For each vertex v of G , the edges incident to v form a clique Q_v in $L(G)$ with $|Q_v| = d_G(v) \leq \Delta$. Every vertex of $L(G)$ lies in exactly two of the cliques Q_v (one for each endpoint of the corresponding edge), and every edge of $L(G)$ lies in at least one of them.

Proof. Any two edges sharing the vertex v are adjacent in G , so their corresponding vertices in $L(G)$ are adjacent; hence Q_v is a clique, and it has $d_G(v) \leq \Delta$ vertices. An edge e of G with endpoints u and w lies in Q_u and Q_w and in no other Q_v . Finally, if e and f are adjacent vertices of $L(G)$, then the edges e and f share an endpoint v in G , and both lie in Q_v . $\square$

Note that the cliques Q_v overlap, so in general they do not partition $V(L(G))$. For example, $L(C_5)$ is again a 5-cycle, whose cliques have at most two vertices, so it cannot be partitioned into $\Delta(C_5) = 2$ cliques.

Brooks' Theorem and Its Application

Brooks' Theorem is a fundamental result in graph coloring that refines the trivial upper bound on the vertex chromatic number.

Theorem 96 (Brooks' Theorem, 1941). Let G be a connected simple graph with maximum degree Δ . Then:

$$\chi(G) \leq \Delta \tag{71}$$

unless G is a complete graph $K_{\Delta+1}$ or an odd cycle C_{2k+1} (where $k \geq 1$), in which case $\chi(G) = \Delta + 1$.

Definition 108 (Greedy Coloring Algorithm). The greedy coloring algorithm is a sequential vertex coloring procedure that works as follows:

1. Arrange the vertices in some order $v_1, v_2, \dots, v_n$.
2. For i from 1 to n :
 - (a) Assign to v_i the smallest color (typically represented by a positive integer) that is not already assigned to any of its adjacent vertices that have already been colored (i.e., neighbors among $v_1, v_2, \dots, v_{i-1}$).

The performance of the greedy algorithm strongly depends on the chosen vertex ordering.

Complete Proof of Brooks' Theorem. We follow the proof of Lovász (1975). The idea is to run the greedy coloring algorithm with the colors $1, 2, \dots, \Delta$ in an order in which every vertex except the last one still has an uncolored neighbor at the moment it is colored. Such a vertex has at most $\Delta - 1$ colored neighbors, so one of the Δ colors is free for it; only the last vertex needs a separate argument.

Case 1: If G is a complete graph $K_{\Delta+1}$ with $\Delta + 1$ vertices, then each vertex is adjacent to all other vertices. A proper coloring requires each vertex to have a distinct color, so $\chi(G) = \Delta + 1$. This establishes the first exception in the theorem.

Case 2: If $\Delta \leq 2$, then G is a path or a cycle. If G is an odd cycle, then $\chi(G) = 3 = \Delta + 1$; this is the second exception, and it cannot occur when $\Delta \geq 3$. If G is an even cycle or a path with at least three vertices, then coloring the vertices alternately with two colors shows that $\chi(G) = 2 = \Delta$. (A path with at most two vertices is K_1 or K_2 , which is covered by Case 1.)

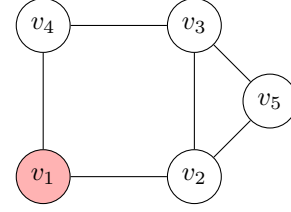
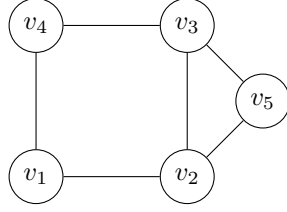
From now on we assume that $\Delta \geq 3$ and that G is not a complete graph, and we show that $\chi(G) \leq \Delta$.

Case 3: G is not regular. Choose a vertex v_0 with $d(v_0) \leq \Delta - 1$, and list the vertices as $v_0, v_1, \dots, v_{n-1}$ in order of nondecreasing distance from v_0 (for example, in the order in which breadth-first search from v_0 reaches them). Apply the greedy coloring algorithm in the *reverse* order $v_{n-1}, v_{n-2}, \dots, v_1, v_0$. When a vertex v_i with $i \geq 1$ is colored, the neighbor of v_i that follows it on a shortest path from v_i to v_0 is closer to v_0 , so it appears earlier in the list and is still uncolored. Hence at most $\Delta - 1$ neighbors of v_i are colored, and one of the Δ colors is free for v_i . Finally, v_0 has at most $\Delta - 1$ neighbors altogether, so a color is free for v_0 as well.

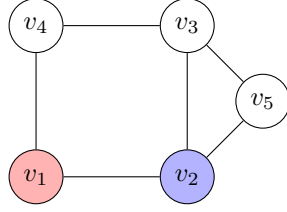
Case 4: G has a cut vertex x . Let $H_1, \dots, H_r$ ($r \geq 2$) be the components of $G - x$, and let G_j be the subgraph of G induced by $V(H_j) \cup \{x\}$. Each G_j is connected with maximum degree at most Δ , and x has fewer than Δ neighbors in G_j , because x also has a neighbor in every other component of $G - x$. The argument of Case 3, applied to G_j with $v_0 = x$, colors G_j properly with colors from $\{1, \dots, \Delta\}$. After permuting the colors in each G_j so that x receives color 1 every time, these colorings combine into a proper Δ -coloring of G .

Case 5: G is Δ -regular and 2-connected. We first find vertices u, v, w with $uv, vw \in E(G)$ and $uw \notin E(G)$ such that $G - \{u, w\}$ is connected.

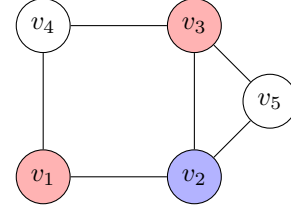
- If $G - \{a, b\}$ is connected for every two vertices a and b , let u and w be two vertices at distance 2 (they exist because G is connected but not complete), and let v be a common neighbor of u and w .
- Otherwise, choose vertices v and x such that $G - \{v, x\}$ is disconnected. Since G is 2-connected, $G - v$ is connected, and x is a cut vertex of $G - v$. The block-cutpoint graph of $G - v$ is a tree with at least two block vertices, so it has at least two leaves. Every cut vertex lies in at least two blocks, so these



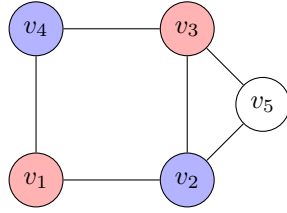
Step 1: Assign color 1 to v_1



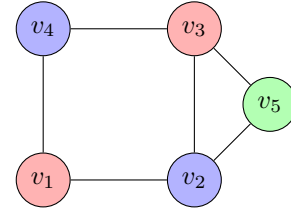
Step 2: Assign color 2 to v_2



Step 3: Assign color 1 to v_3



Step 4: Assign color 2 to v_4



Step 5: Assign color 3 to v_5

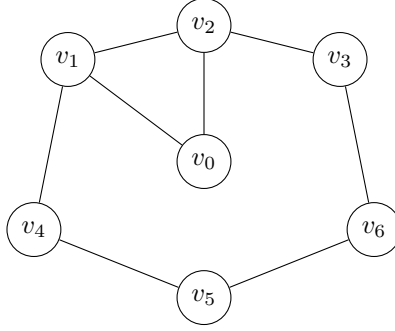
Figure 137: Step-by-step execution of the greedy coloring algorithm on a simple graph with the vertex ordering v_1, v_2, v_3, v_4, v_5 . The final coloring uses 3 colors.

leaves are blocks B_1 and B_2 of $G - v$, and each of them contains exactly one cut vertex of $G - v$, say $z_1 \in B_1$ and $z_2 \in B_2$. If v had no neighbor in $B_1 - z_1$, then z_1 would be a cut vertex of G , which is impossible; so v has a neighbor $u \in B_1 - z_1$, and similarly a neighbor $w \in B_2 - z_2$. Each of u and w lies in only one block of $G - v$, and these blocks are different, so u and w are not adjacent. All neighbors of u in $G - v$ lie in $B_1 - u$, which is connected (a block is 2-connected or a single edge), so $G - \{v, u\}$ is connected; similarly, all neighbors of w in $G - \{v, u\}$ lie in $B_2 - w$, which is connected and does not contain u , so $G - \{v, u, w\}$ is connected. Since $d(v) = \Delta \geq 3$, the vertex v has a neighbor other than u and w , and hence $G - \{u, w\}$ is connected.

Now give both u and w color 1. List the vertices of the connected graph $G - \{u, w\}$ in order of nondecreasing distance from v in $G - \{u, w\}$, starting with v , and color them greedily in the reverse order, so that v is colored last. As in Case 3, every vertex other than v still has an uncolored neighbor when it is colored, so at most $\Delta - 1$ colors appear on its colored neighbors. Finally, v has Δ neighbors, but two of them, u and w , have the same color, so at most $\Delta - 1$ colors appear on the neighbors of v , and a color is free for v .

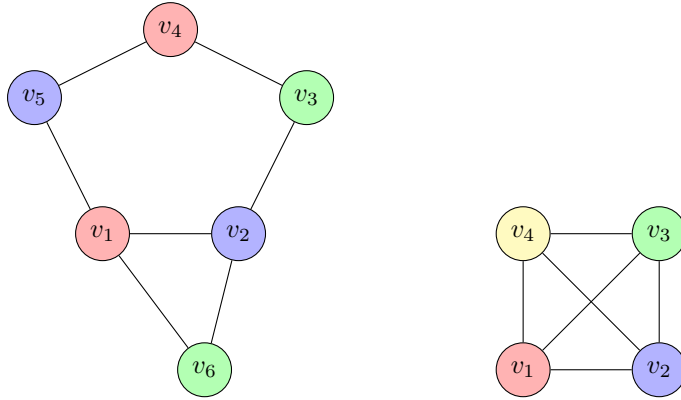
In Cases 3, 4, and 5 the graph G has a proper coloring with at most Δ colors, which completes the proof. $\square$

We now apply the greedy coloring algorithm to the graph of Figure 138 in the *reverse* order, starting with v_6 and ending with v_0 :



Vertex ordering: $v_0, v_1, v_2, v_3, v_4, v_5, v_6$

Figure 138: Case 3 of the proof for a graph with $\Delta = 3$. The vertex v_0 has degree $2 < \Delta$, and the vertices are listed in order of nondecreasing distance from v_0 (distance 1: v_1, v_2 ; distance 2: v_3, v_4 ; distance 3: v_5, v_6). The greedy algorithm colors them in the reverse order $v_6, v_5, \dots, v_0$.



A graph with $\Delta = 3$ colored with 3 colors

K_4 - requires 4 colors

Figure 139: Comparison of a graph with maximum degree 3 (left) that satisfies Brooks' Theorem and can be colored with $\Delta = 3$ colors, and K_4 (right) which requires $\Delta + 1 = 4$ colors.

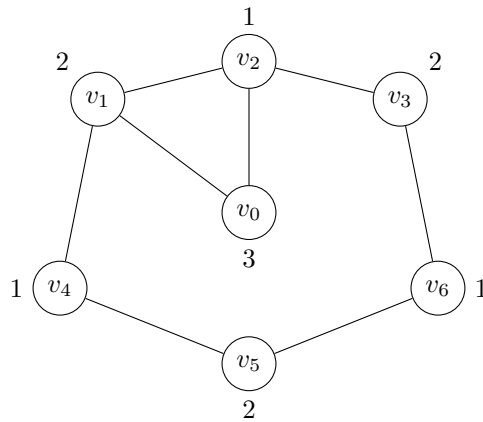


Figure 140: Final coloring using $\Delta = 3$ colors (the number next to each vertex is its color)

The coloring process works as follows:

1. Color v_6 with color 1 - none of its neighbors is colored yet
2. Color v_5 with color 2 - since it is adjacent to v_6 , which has color 1
3. Color v_4 with color 1 - its only colored neighbor, v_5 , has color 2
4. Color v_3 with color 2 - its only colored neighbor, v_6 , has color 1
5. Color v_2 with color 1 - its only colored neighbor, v_3 , has color 2
6. Color v_1 with color 2 - its colored neighbors v_2 and v_4 both have color 1
7. Color v_0 with color 3 - it is adjacent to v_1 (color 2) and v_2 (color 1); since $d(v_0) = 2 < \Delta$, a color is still free

This demonstrates that the graph can be colored with $\Delta = 3$ colors, as Brooks' Theorem guarantees.

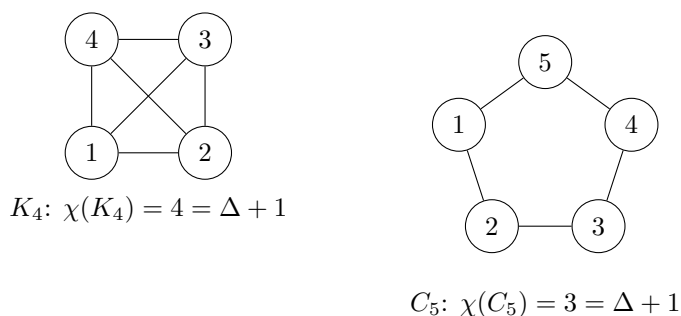


Figure 141: Special cases for Brooks' Theorem: Complete graph K_4 and odd cycle C_5 both require $\Delta + 1$ colors.

Example 35 (Application of Brooks' Theorem). Consider a connected graph G with maximum degree $\Delta = 3$ that is not the complete graph K_4 (it cannot be an odd cycle, since odd cycles have $\Delta = 2$). By Brooks' Theorem, $\chi(G) \leq 3$, meaning we can color the vertices using at most 3 colors.

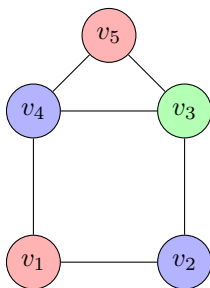


Figure 142: G with $\Delta = 3$, $\chi(G) = 3$

Each vertex in this example has degree at most 3, and the graph is connected and is neither K_4 nor an odd cycle. As predicted by Brooks' Theorem, it can be colored with 3 colors; it needs exactly 3 colors because it contains the triangle $v_3v_4v_5$.

Remark 99 (What Brooks' Theorem Gives for Edge-Coloring). Since $\chi'(G) = \chi(L(G))$ and $\Delta(L(G)) \leq 2\Delta - 2$, a greedy coloring of $L(G)$ gives $\chi'(G) \leq 2\Delta - 1$, and Brooks' Theorem applied to $L(G)$ improves this to $\chi'(G) \leq \Delta(L(G)) \leq 2\Delta - 2$ whenever $L(G)$ is connected, has maximum degree at least 3, and is neither a complete graph nor an odd cycle. For $\Delta \geq 5$ this is weaker than Shannon's bound $\lfloor 3\Delta/2 \rfloor$, so Brooks' Theorem alone does not give Shannon's bound. The proof below uses recoloring along two-colored paths instead.

Completing the Proof

Together with the greedy coloring algorithm, the lemmas above give only $\chi'(G) = \chi(L(G)) \leq \Delta(L(G)) + 1 \leq 2\Delta - 1$. To reach $\lfloor 3\Delta/2 \rfloor$ we recolor edges along two-colored paths, as in the proofs of König's and Vizing's theorems. For a proper coloring of some of the edges and a vertex z , a color is *missing* at z if no colored edge incident to z has that color, and $M(z)$ denotes the set of colors missing at z .

Proof of Shannon's Theorem. If $\Delta = 0$, then G has no edges and there is nothing to prove. Let $\Delta \geq 1$ and $k = \lfloor 3\Delta/2 \rfloor$. We prove by induction on the number of edges that every loopless multigraph with maximum degree at most Δ has a proper edge-coloring with colors from $\{1, 2, \dots, k\}$. This is clear if there are no edges. Otherwise, let G be such a multigraph, let e be an edge of G with endpoints x and y , and let ϕ be a proper edge-coloring of $G - e$ with colors from $\{1, \dots, k\}$, which exists by the induction hypothesis. In $G - e$ the vertices x and y have degree at most $\Delta - 1$ and every vertex has degree at most Δ ; since $k - \Delta = \lfloor \Delta/2 \rfloor$, this gives

$$|M(x)| \geq \lfloor \frac{\Delta}{2} \rfloor + 1, \quad |M(y)| \geq \lfloor \frac{\Delta}{2} \rfloor + 1, \quad |M(z)| \geq \lfloor \frac{\Delta}{2} \rfloor \text{ for every vertex } z.$$

We show that ϕ can be modified so that e can be colored as well.

Step 1. If some color lies in $M(x) \cap M(y)$, we give e that color and are done. So assume from now on that $M(x) \cap M(y) = \emptyset$.

Step 2. Let $\alpha \in M(x)$ and $\beta \in M(y)$. Then $\beta \notin M(x)$, so x has an incident edge of color β but none of color α . By the Color Alternating Path Lemma, the component of the subgraph formed by the edges of colors α and β that contains x is a path P with x as one end. If P does not end at y , then y is not on P at all (since β is missing at y , the vertex y could only be an end of P); interchange α and β on P . Now β is missing at both x and y , and we color e with β . So we may assume that for all $\alpha \in M(x)$ and $\beta \in M(y)$, the path of colors α and β starting at x ends at y .

Step 3. Fix $\alpha \in M(x)$ and $\beta \in M(y)$. Since $\alpha \notin M(y)$, some edge f incident to y has color α ; let z be its other endpoint. Then $z \neq x$ because α is missing at x , and $z \neq y$ because G has no loops. If some color γ lies in $M(y) \cap M(z)$, recolor f with γ ; then α is missing at both x and y , and we color e with α . So we may assume that $M(y) \cap M(z) = \emptyset$.

Step 4. By Steps 1 and 3, both $M(x)$ and $M(z)$ are disjoint from $M(y)$. Since

$$|M(x)| + |M(y)| + |M(z)| \geq 3 \lfloor \frac{\Delta}{2} \rfloor + 2 > \lfloor \frac{3\Delta}{2} \rfloor = k,$$

the three sets cannot be pairwise disjoint subsets of $\{1, \dots, k\}$. As $M(y)$ is disjoint from the other two, there is a color $\gamma \in M(x) \cap M(z)$.

Step 5. By Step 2 (applied to $\gamma \in M(x)$ and $\beta \in M(y)$), the path P of colors γ and β starting at x ends at y . The vertex z is not on P : since γ is missing at z , the vertex z could only be an end of P , and the ends of P are x and y . Interchange γ and β on P . Now γ is missing at y (the edge of P at y had color γ , since β is missing at y , and it now has color β), and γ is still missing at z . Moreover, $\alpha \notin \{\beta, \gamma\}$ because α appears at y and at z , so the edge f still has color α , and α is still missing at x . As in Step 3, recolor f with γ and color e with α .

In every case the edge-coloring of $G - e$ can be modified into a proper edge-coloring of G with colors from $\{1, \dots, k\}$. Hence $\chi'(G) \leq \lfloor 3\Delta/2 \rfloor$. $\square$

Tightness of Shannon's Bound

The Shannon multigraph shown above has:

- k parallel edges between each pair of vertices (here $k = 2$)
- Maximum degree $\Delta = 2k = 4$
- Edge chromatic number $3k = 6$, since all $3k$ edges are pairwise adjacent

This demonstrates that Shannon's bound is tight for certain families of multigraphs. The edge chromatic number is exactly $3k = \lfloor \frac{3 \cdot (2k)}{2} \rfloor = \lfloor \frac{3\Delta}{2} \rfloor$.

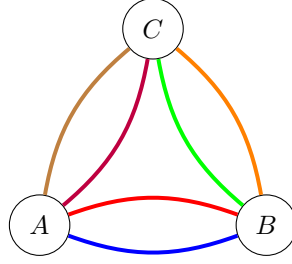


Figure 143: Shannon multigraph with $k = 2$ parallel edges between each pair of vertices (maximum degree $\Delta = 4$)

Refined Proof Techniques

Shannon's original proof involves a more detailed analysis of the constraints between different matchings. The refinement uses the concept of a "color exchange graph" and applies graph theoretic techniques to show that $\lfloor \frac{3\Delta}{2} \rfloor$ colors are sufficient.

The key insights include:

- Analyzing how colors can be interchanged along alternating paths in the graph
- Strategic recoloring of edges to free up colors
- Careful analysis of the constraints between different color classes

Implications and Applications

Shannon's edge coloring theorem has several important implications:

1. It provides an upper bound on the number of time slots needed for scheduling operations where conflict constraints can be modeled as a graph.
2. It establishes a relationship between the maximum degree and the edge chromatic number, showing that edge chromatic number grows at most proportionally to the maximum degree.
3. It applies to multigraphs where Vizing's tighter bound does not apply.

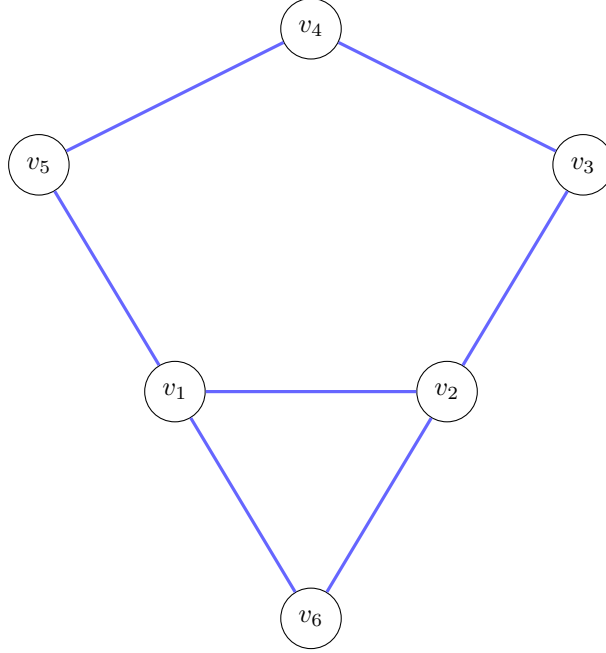
Shannon originally developed this result in the context of analyzing switching networks in communication systems, demonstrating the practical motivations behind this theoretical advance.

8.5 Degenerate Graphs and Their Coloring Properties

Definition 109 (d -Degenerate Graph). Let $d \geq 0$ be an integer. A graph G is called **d -degenerate** if every subgraph of G (with at least one vertex) has a vertex of degree at most d . The smallest integer d such that G is d -degenerate is called the **degeneracy** of G . Note that a d -degenerate graph is also d' -degenerate for every $d' \geq d$.

Theorem 97 (Equivalence of Degenerate Definitions). The following statements are equivalent for a graph G :

1. G is d -degenerate.
2. Every induced subgraph of G has a vertex of degree at most d .
3. There exists an ordering $v_1, v_2, \dots, v_n$ of the vertices of G such that each vertex v_i has at most d neighbors among vertices $v_1, v_2, \dots, v_{i-1}$.



A 2-degenerate graph with a valid vertex ordering

Figure 144: Example of a 2-degenerate graph

Proof. (1 $\Rightarrow$ 2): Every induced subgraph is a subgraph, so this follows directly from the definition of d -degeneracy.

(2 $\Rightarrow$ 3): We first remove the vertices one at a time:

- Start with the original graph G .
- Find a vertex u_1 with degree at most d (which exists by assumption).
- Remove u_1 from G to get G' .
- Find a vertex u_2 with degree at most d in G' (which exists since G' is an induced subgraph of G).
- Continue this process until all vertices have been removed.

When u_j was selected, it had degree at most d in the remaining graph, so each u_j has at most d neighbors among $u_{j+1}, \dots, u_n$. Now list the vertices in the reverse order of removal: $v_i = u_{n+1-i}$ for $i = 1, \dots, n$. The neighbors of v_i among $v_1, \dots, v_{i-1}$ are exactly its neighbors that were removed after it, so each v_i has at most d neighbors among $v_1, v_2, \dots, v_{i-1}$.

(3 $\Rightarrow$ 1): Let $v_1, v_2, \dots, v_n$ be an ordering where each v_i has at most d neighbors among $v_1, \dots, v_{i-1}$. Let H be any subgraph of G with at least one vertex, and let v_i be the vertex of H that appears last in this ordering. Every neighbor of v_i in H comes before v_i in the ordering, so v_i has at most d neighbors in H . Hence every subgraph of G has a vertex of degree at most d , that is, G is d -degenerate. $\square$

Theorem 98 (Characterization of d -Degeneracy). The degeneracy d of a graph G is equal to:

$$d = \max_{H \subseteq G} \delta(H) \quad (72)$$

where the maximum is taken over all induced subgraphs H of G and $\delta(H)$ denotes the minimum degree of a vertex in H .

Proof. Let $d^* = \max_{H \subseteq G} \delta(H)$. We need to show that $d = d^*$.

First, we prove that $d \geq d^*$. Consider an induced subgraph H of G such that $\delta(H) = d^*$. Since every vertex in H has degree at least d^* , the graph G cannot be $(d^* - 1)$ -degenerate by definition. Therefore, $d \geq d^*$.

Next, we prove that $d \leq d^*$. Since $\delta(H) \leq d^*$ for all induced subgraphs H of G , every induced subgraph has a vertex of degree at most d^* . This means G is d^* -degenerate, so $d \leq d^*$.

Combining these inequalities, we have $d = d^*$. □

Theorem 99 (Bounds on Degeneracy). For any graph G :

1. The degeneracy d is at most the maximum degree $\Delta(G)$.
2. The degeneracy d is at least the minimum degree $\delta(G)$.
3. For a k -regular graph, the degeneracy d equals k .

Theorem 100 (d -Degeneracy and Coloring). If a graph G has degeneracy d , then:

$$\chi(G) \leq d + 1 \tag{73}$$

where $\chi(G)$ is the chromatic number of G .

Proof. Let G be a graph with degeneracy d . By the Equivalence of Degenerate Definitions theorem, there exists an ordering $v_1, v_2, \dots, v_n$ of the vertices such that each vertex v_i has at most d neighbors among vertices $v_1, v_2, \dots, v_{i-1}$.

We can color G using the greedy algorithm in this order. When we color vertex v_i , at most d of its neighbors have already been colored. Therefore, at most d colors are forbidden for v_i , and we can assign to v_i a color from the set $\{1, 2, \dots, d + 1\}$ that is different from the colors of its already-colored neighbors.

This process produces a proper coloring of G using at most $d + 1$ colors. Therefore, $\chi(G) \leq d + 1$. □

Example 36 (Degeneracy of Specific Graph Classes). • Trees and forests are 1-degenerate: Every subgraph of a forest is a forest, and every forest with at least one vertex has an isolated vertex or a leaf (vertex of degree 1).

- Outerplanar graphs are 2-degenerate: Every outerplanar graph with $n \geq 3$ vertices has at least one vertex of degree at most 2.
- Planar graphs are 5-degenerate: By Euler's formula and the handshaking lemma, every planar graph with $n \geq 3$ vertices has at least one vertex of degree at most 5.
- A graph with arboricity a is $(2a - 1)$ -degenerate: Arboricity is the minimum number of forests needed to cover all edges. If a graph has arboricity a , then $|E(H)| \leq a(|V(H)| - 1)$ for every induced subgraph H , implying H has a vertex of degree at most $2a - 1$.

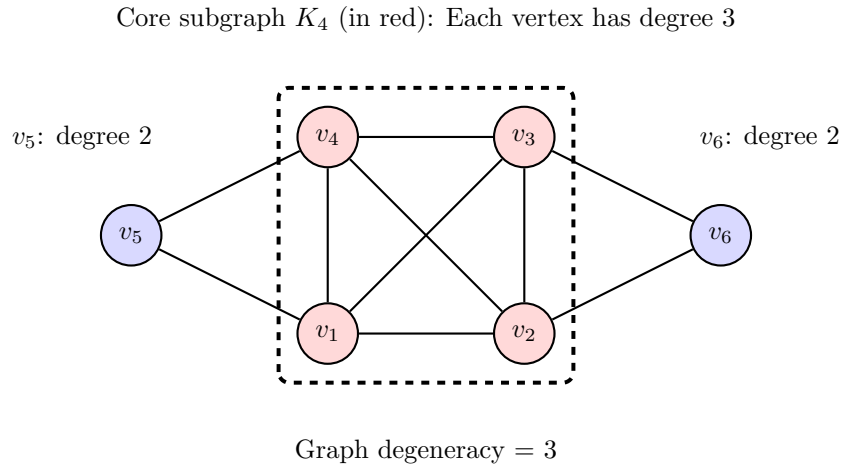


Figure 145: A 3-degenerate graph containing K_4 as an induced subgraph (highlighted in red). The degeneracy is determined by this subgraph where each vertex has degree 3.

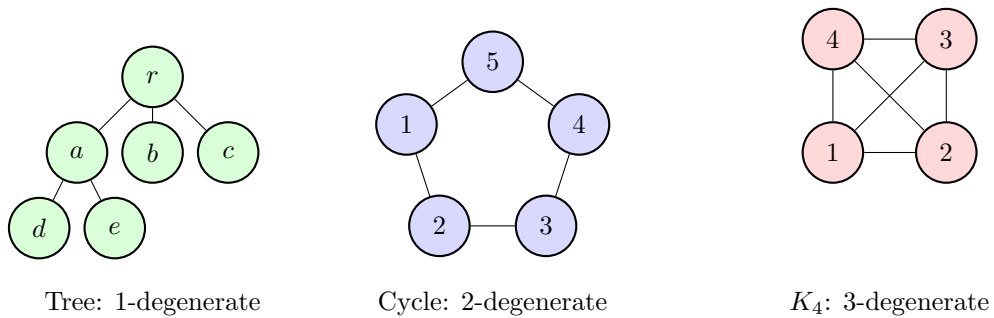


Figure 146: Examples of graphs with different degeneracy values: a tree (1-degenerate), a cycle (2-degenerate), and K_4 (3-degenerate).

Algorithm 27 Computing Graph Degeneracy

Require: Graph $G = (V, E)$ **Ensure:** Degeneracy d and degeneracy ordering L

```
1:  $L \leftarrow \emptyset$  ▷ Initialize empty ordering
2:  $d \leftarrow 0$  ▷ Initialize degeneracy
3:  $G' \leftarrow G$  ▷ Working copy of the graph
4: while  $G'$  is not empty do
5:   Find vertex  $v \in G'$  with minimum degree
6:    $d \leftarrow \max(d, \deg_{G'}(v))$  ▷ Update degeneracy
7:   Add  $v$  to the beginning of ordering  $L$ 
8:   Remove  $v$  from  $G'$ 
9: end while
10: return degeneracy  $d$  and ordering  $L$ 
```

Lemma 84 (Tightness of Degeneracy Bound). For any $d \geq 1$, there exists a d -degenerate graph G with $\chi(G) = d + 1$.

Proof. The complete graph K_{d+1} on $d + 1$ vertices is d -degenerate and requires exactly $d + 1$ colors. $\square$

Corollary 38 (Degeneracy vs. Maximum Degree). For any graph G with degeneracy d and maximum degree Δ :

$$\chi(G) \leq d + 1 \leq \Delta + 1 \quad (74)$$

Lemma 85 (Degeneracy and Clique Number). For any graph G with degeneracy d and clique number $\omega(G)$:

$$\omega(G) \leq d + 1 \quad (75)$$

Proof. Let K be a clique of size $\omega(G)$ in G . In the subgraph K , every vertex has degree $\omega(G) - 1$, so $\delta(K) = \omega(G) - 1$. Since $d = \max_{H \subseteq G} \delta(H)$, we have $\omega(G) - 1 \leq d$, which gives $\omega(G) \leq d + 1$. $\square$

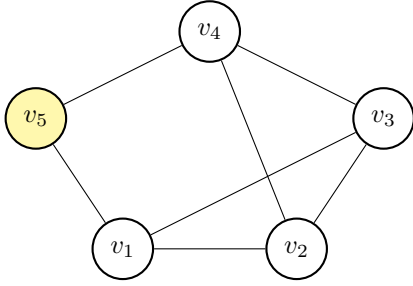
Algorithm 28 Matula-Beck Algorithm for Computing Degeneracy

Require: Graph $G = (V, E)$ **Ensure:** Degeneracy value d and degeneracy ordering L

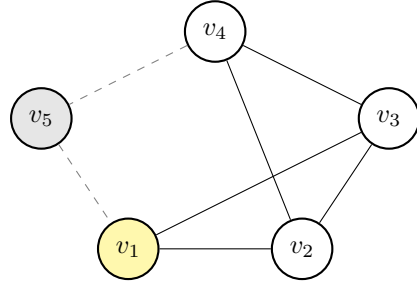
```
1: Initialize empty list  $L \leftarrow \emptyset$ 
2:  $d \leftarrow 0$  ▷ Initialize degeneracy value
3:  $G' \leftarrow G$  ▷ Working copy of the graph
4: while  $G'$  is not empty do
5:   Find vertex  $v \in V(G')$  with minimum degree
6:    $d \leftarrow \max(d, \deg_{G'}(v))$  ▷ Update degeneracy
7:   Remove  $v$  from  $G'$ 
8:   Add  $v$  to the beginning of  $L$ 
9: end while
10: return degeneracy  $d$  and ordering  $L$ 
```

Lemma 86 (Computational Complexity of Degeneracy). The Matula-Beck algorithm computes the degeneracy of a graph and a corresponding degeneracy ordering in $O(n + m)$ time, where n is the number of vertices and m is the number of edges.

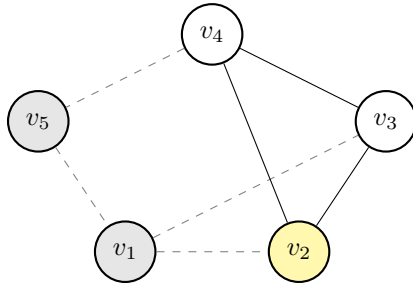
Proof Sketch. Using a bucket sort structure to maintain vertices by their current degree, we can efficiently find and remove the minimum degree vertex in each iteration. The total time complexity is $O(n + m)$ because each vertex is processed once (taking $O(n)$ total) and each edge is examined at most once when updating degrees (taking $O(m)$ total). $\square$



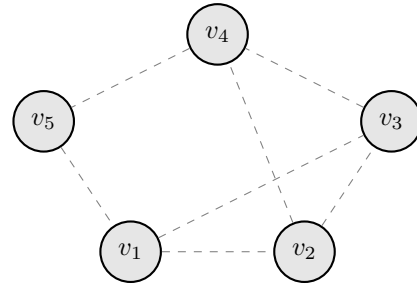
Step 1: Identify v_5 with minimum degree



Step 2: Remove v_5 , find next minimum



Step 3: Remove v_1 , find next minimum



Final ordering: $L = [v_4, v_3, v_2, v_1, v_5]$

Figure 147: Computation of degeneracy ordering: Vertices are removed in order of minimum degree, with the current minimum degree vertex highlighted in yellow. The vertices are removed in the order v_5, v_1, v_2, v_3, v_4 , and the final ordering L lists them in reverse order of removal.

Algorithm 29 Efficient Implementation of Matula-Beck Algorithm

```
1: procedure COMPUTEDEGENERACY( $G = (V, E)$ )
2:    $d \leftarrow 0$  ▷ Initialize degeneracy value
3:    $L \leftarrow \emptyset$  ▷ Initialize empty ordering list
4:   Compute  $\deg(v)$  for each vertex  $v \in V$ 
5:    $\Delta \leftarrow \max_{v \in V} \deg(v)$  ▷ Maximum degree in  $G$ 
6:   Create buckets  $B_0, B_1, \dots, B_\Delta$ 
7:   for all  $v \in V$  do
8:     Add  $v$  to bucket  $B_{\deg(v)}$ 
9:   end for
10:  while  $\exists i : B_i \neq \emptyset$  do
11:    Find minimum  $i$  such that  $B_i \neq \emptyset$ 
12:    Remove a vertex  $v$  from  $B_i$ 
13:     $d \leftarrow \max(d, i)$  ▷ Update degeneracy
14:    Prepend  $v$  to ordering  $L$ 
15:    for all neighbors  $u$  of  $v$  that remain in the graph do
16:      Move  $u$  from  $B_{\deg(u)}$  to  $B_{\deg(u)-1}$ 
17:       $\deg(u) \leftarrow \deg(u) - 1$  ▷ Update degree
18:    end for
19:  end while
20:  return  $(d, L)$  ▷ Return degeneracy and ordering
21: end procedure
```

8.6 Degeneracy Relationships for Specific Graph Classes

Theorem 101 (Degeneracy of Basic Graph Classes). The following bounds on degeneracy hold:

1. Trees and forests are 1-degenerate.
2. K_n (complete graph on n vertices) is $(n - 1)$ -degenerate.
3. Cycles C_n are 2-degenerate.
4. Bipartite graphs can have arbitrarily large degeneracy: for $1 \leq m \leq n$, the complete bipartite graph $K_{m,n}$ has degeneracy exactly m .
5. A k -regular graph is k -degenerate.

Lemma 87 (Trees are 1-Degenerate). Every tree T is 1-degenerate.

Proof. Every subgraph of a tree is a forest. A forest with at least one vertex either has an isolated vertex (of degree 0), or has a component with at least two vertices; this component is a tree and therefore has a leaf (a vertex of degree 1). Hence every subgraph of a tree has a vertex of degree at most 1, making the tree 1-degenerate. $\square$

Lemma 88 (Cycles are 2-Degenerate). Every cycle C_n is 2-degenerate.

Proof. In a cycle, every vertex has degree exactly 2. When we remove any vertex from the cycle, we get a path, in which every vertex has degree at most 2, and there is at least one vertex of degree 1 (the endpoints of the path). Therefore, every cycle is 2-degenerate. $\square$

Lemma 89 (Complete Graphs). The complete graph K_n is $(n - 1)$ -degenerate.

Proof. In K_n , every vertex has degree $n - 1$. When we remove a vertex, we get K_{n-1} , where every vertex has degree $n - 2$. Continuing this process, we see that the degeneracy is exactly $n - 1$. $\square$

Lemma 90 (Bipartite Graphs). A connected bipartite graph that is not a tree has degeneracy at least 2. However, the degeneracy of a bipartite graph can be arbitrarily large: for $1 \leq m \leq n$, the complete bipartite graph $K_{m,n}$ has degeneracy exactly m .

Proof. A connected graph that is not a tree contains a cycle, and a cycle is a subgraph in which every vertex has degree 2. Hence such a graph is not 1-degenerate, and its degeneracy is at least 2.

Now let A and B be the parts of $K_{m,n}$, with $|A| = m \leq n = |B|$. Every vertex of $K_{m,n}$ has degree $n \geq m$ or m , so $\delta(K_{m,n}) = m$ and the degeneracy is at least m . Conversely, let H be any subgraph of $K_{m,n}$ with at least one vertex. If H contains a vertex of B , that vertex has at most $|A| = m$ neighbors in H ; otherwise all vertices of H lie in A , and H has no edges. In either case H has a vertex of degree at most m , so $K_{m,n}$ is m -degenerate. For example, $K_{3,3}$ is a connected bipartite graph with degeneracy 3. $\square$

Lemma 91 (k -Regular Graphs). A k -regular graph is k -degenerate; in fact, its degeneracy is exactly k .

Proof. Every subgraph H of a k -regular graph G has maximum degree at most k , so H has a vertex of degree at most k ; hence G is k -degenerate. On the other hand, G itself is a subgraph in which every vertex has degree k , so G is not $(k - 1)$ -degenerate. Therefore the degeneracy of G is exactly k . $\square$

8.7 Extensions and Applications

Theorem 102 (Generalized Degeneracy). For any non-negative integer function f on the vertices of a graph G , there exists a vertex ordering such that each vertex v has at most $d + f(v)$ neighbors preceding it in the ordering, where d is the degeneracy of G .

Theorem 103 (Clustered Coloring of Bounded Degeneracy Graphs). Let $d \geq 1$. Every d -degenerate graph can be properly colored with $d + 1$ colors, so that every color class induces a subgraph whose components have size 1. Allowing larger monochromatic components does not reduce this number: for every positive integer t there is a d -degenerate graph that cannot be colored with d colors so that each color class induces a subgraph with components of size at most t .

Lemma 92 (p -Centered Coloring). A p -centered coloring is a vertex coloring such that for any connected subgraph H , either H receives more than p colors or there is a color that appears exactly once in H . The number of colors needed in a p -centered coloring cannot be bounded in terms of p and the degeneracy alone: for every $k \geq 2$ there is a 2-degenerate graph with no 2-centered coloring using k colors.

Proof. A 2-centered coloring is proper, since an edge whose ends have the same color is a connected subgraph that uses one color twice. It is also acyclic: in a proper coloring, a cycle that uses only two colors alternates between them, so it is a connected subgraph with at most two colors in which no color appears exactly once. By the lemma on acyclic coloring below, for every $k \geq 2$ there is a 2-degenerate graph with no acyclic coloring using k colors, and hence no 2-centered coloring using k colors. $\square$

Definition 110 (Sparse Graph). A graph $G = (V, E)$ is called **sparse** if $|E| = O(|V|)$, that is, the number of edges is asymptotically linear in the number of vertices. More precisely, a family of graphs is considered sparse if there exists a constant c such that for all graphs G in the family with n vertices and m edges, we have $m \leq cn$.

Equivalently, a sparse graph has an average degree that is bounded by a constant independent of the graph size. That is, the average degree $\frac{2|E|}{|V|} = O(1)$.

Examples of sparse graphs include:

- Trees, which have exactly $n - 1$ edges for n vertices
- Planar graphs, which have at most $3n - 6$ edges for $n \geq 3$ vertices
- Graphs with bounded degeneracy d , which have at most dn edges
- Graphs with bounded arboricity a , which have at most an edges

Algorithm 30 Coloring Algorithm for Sparse Graphs

Require: Graph $G = (V, E)$ with degeneracy d

Ensure: A proper coloring of G using at most $d + 1$ colors

```
1: Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
2: Initialize color palette  $C = \{1, 2, \dots, d + 1\}$ 
3: Initialize empty color assignment  $\text{color}(v) \leftarrow \text{null}$  for all  $v \in V$ 
4: for  $i = 1$  to  $n$  do
5:    $N_{\text{before}}(v_i) \leftarrow \{v_j \in N(v_i) : j < i\}$  ▷ Neighbors earlier in ordering
6:    $C_{\text{used}} \leftarrow \{\text{color}(v) : v \in N_{\text{before}}(v_i)\}$  ▷ Colors used by neighbors
7:    $C_{\text{avail}} \leftarrow C \setminus C_{\text{used}}$  ▷ Available colors
8:    $\text{color}(v_i) \leftarrow \min(C_{\text{avail}})$  ▷ Assign smallest available color
9: end for
10: return color assignment
```

For some special classes, other methods can use fewer colors than the bound $d + 1$ of Algorithm 30: a bipartite graph can be colored with 2 colors using a bipartition, which improves on the degeneracy bound when $d \geq 2$, and a planar graph can be colored with 4 colors by the Four Color Theorem, which improves on the degeneracy bound when $d \geq 4$.

Remark 100 (Application to Graph Mining). *Computing all maximal cliques in a graph with degeneracy d can be done in $O(3^{d/3} \cdot n \cdot d)$ time, making it tractable for graphs with small degeneracy; this is nearly optimal, since an n -vertex graph with degeneracy d can have $(n - d)3^{d/3}$ maximal cliques when d is a multiple of 3 and $n \geq d + 3$.*

Lemma 93 (Acyclic Coloring and Degeneracy). An acyclic coloring is a proper coloring such that any two color classes induce a forest. The number of colors needed in an acyclic coloring is not bounded by any function of the degeneracy: for every $k \geq 2$ there is a 2-degenerate graph with no acyclic coloring using k colors.

Proof. Start with K_{k+1} and replace each edge uv by k new vertices, each adjacent to both u and v (the edge uv itself is deleted); that is, each edge is replaced by a copy of $K_{2,k}$. Call the result G' . The original vertices are pairwise non-adjacent in G' and every new vertex has degree 2, so every subgraph of G' either has no edges or contains a new vertex, which has degree at most 2 in it. Hence G' is 2-degenerate.

Suppose G' had an acyclic coloring with k colors. Two of the $k + 1$ original vertices, say u and v , would receive the same color a . Their k common neighbors must receive colors different from a , and no two of them can share a color b , since otherwise u , v and these two common neighbors would form a cycle of length 4 using only the colors a and b . Thus the k common neighbors would need k distinct colors other than a , which is impossible with only k colors. Hence every acyclic coloring of G' uses at least $k + 1$ colors. $\square$

In contrast, some sparse classes do have bounded acyclic chromatic number; for instance, every planar graph has an acyclic coloring with at most 5 colors (Borodin, 1979).

Remark 101 (Relationship to Brooks' Theorem). *Brooks' Theorem states that if G is a connected graph that is neither a complete graph nor an odd cycle, then $\chi(G) \leq \Delta(G)$. Since $d \leq \Delta$, the degeneracy bound $\chi(G) \leq d + 1$ may be tighter in some cases.*

8.8 Algorithms for d -Degenerate Graphs

Theorem 104 (Linear-time Recognition of k -degenerate Graphs). For a fixed value of k , determining whether a graph G is k -degenerate can be done in linear time $O(|V| + |E|)$.

Lemma 94 (Clique Enumeration in d -Degenerate Graphs). For a fixed integer $h \geq 2$, the number of copies of the complete graph K_h in a d -degenerate graph G with n vertices is at most $n \binom{d}{h-1} = O(n \cdot d^{h-1})$, and all such copies can be enumerated in time $O(n \cdot d^{h-1})$.

Algorithm 31 Recognition of k -Degenerate Graphs

```
1: procedure IskDEGENERATE( $G = (V, E), k$ )
2:   Create a copy  $G' \leftarrow G$ 
3:   while  $V(G') \neq \emptyset$  do
4:     Find vertex  $v \in V(G')$  with minimum degree
5:     if  $\deg_{G'}(v) > k$  then
6:       return false ▷ Found a subgraph with minimum degree  $> k$ 
7:     end if
8:     Remove  $v$  from  $G'$ 
9:   end while
10:  return true ▷ Every subgraph had a vertex with degree  $\leq k$ 
11: end procedure
```

Sketch. Order the vertices so that each vertex has at most d neighbors that come later in the order (the reverse of the ordering produced by Algorithm 27). Every copy of K_h is counted at its earliest vertex v , and its other $h - 1$ vertices form a subset of the at most d later neighbors of v ; hence each vertex is the earliest vertex of at most $\binom{d}{h-1}$ copies. For the running time, Chiba and Nishizeki (1985) list all copies of K_h in $O(h a^{h-2} m)$ time, where $a \leq d$ is the arboricity of G and $m \leq dn$; this is $O(n \cdot d^{h-1})$ for fixed h . $\square$

The restriction to complete graphs is essential: for other patterns the number of copies need not be linear in n . For example, the star $K_{1,n-1}$ is 1-degenerate but contains $\binom{n-1}{2}$ paths with three vertices, and $K_{2,n-2}$ is 2-degenerate but contains $\binom{n-2}{2}$ cycles of length four.

Algorithm 32 Enumeration of Maximal Cliques in d -Degenerate Graphs

```
1: procedure ENUMERATEMAXIMALCLIQUES( $G = (V, E)$ )
2:   Compute a degeneracy ordering  $L$  of  $G$  (Algorithm 27), and let  $v_1, v_2, \dots, v_n$  be the vertices in the
   order in which they were removed, i.e., the reverse of  $L$ , so that each  $v_i$  has at most  $d$  neighbors  $v_j$  with
    $j > i$ 
3:   for  $i = n$  downto 1 do
4:      $N^+(v_i) \leftarrow \{v_j \in N(v_i) : j > i\}$  ▷ Forward neighbors in ordering
5:      $G_i \leftarrow G[N^+(v_i)]$  ▷ Subgraph induced by forward neighbors
6:      $\mathcal{C}_i \leftarrow \text{AllMaximalCliques}(G_i)$  ▷  $\mathcal{C}_i = \{\emptyset\}$  if  $G_i$  is empty
7:     for all cliques  $C \in \mathcal{C}_i$  do
8:       if no  $v_j$  with  $j < i$  is adjacent to  $v_i$  and to every vertex of  $C$  then
9:         output  $C \cup \{v_i\}$  ▷ A maximal clique of  $G$ 
10:      end if
11:    end for
12:  end for
13: end procedure
```

Theorem 105 (Bron-Kerbosch Algorithm for d -Degenerate Graphs). The Bron-Kerbosch algorithm with degeneracy ordering runs in $O(d \cdot n \cdot 3^{d/3})$ time on a d -degenerate graph with n vertices.

Algorithm 33 Bron-Kerbosch Algorithm for d -Degenerate Graphs

```
1: procedure BRONKERBOSCHDEGENERACY( $G = (V, E)$ )
2:   Compute a degeneracy ordering  $L$  of  $G$  (Algorithm 27), and let  $v_1, v_2, \dots, v_n$  be the vertices in the
   order in which they were removed, i.e., the reverse of  $L$ , so that each  $v_i$  has at most  $d$  neighbors  $v_j$  with
    $j > i$ 
3:    $\mathcal{C} \leftarrow \emptyset$  ▷ Initialize set of all maximal cliques
4:   for  $i = n$  downto 1 do
5:      $P \leftarrow N(v_i) \cap \{v_{i+1}, \dots, v_n\}$  ▷ Neighbors after  $v_i$  in ordering
6:      $X \leftarrow N(v_i) \cap \{v_1, \dots, v_{i-1}\}$  ▷ Neighbors before  $v_i$  in ordering
7:     BRONKERBOSCHPIVOT( $\{v_i\}, P, X, \mathcal{C}$ )
8:      $X \leftarrow X \cup \{v_i\}$  ▷ Add  $v_i$  to excluded vertices
9:   end for
10:  return  $\mathcal{C}$  ▷ Return all maximal cliques
11: end procedure

12: procedure BRONKERBOSCHPIVOT( $R, P, X, \mathcal{C}$ )
13:  if  $P = \emptyset$  and  $X = \emptyset$  then
14:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{R\}$  ▷  $R$  is a maximal clique
15:    return
16:  end if
17:  Choose a pivot vertex  $u \in P \cup X$  that maximizes  $|P \cap N(u)|$ 
18:  for all  $v \in P \setminus N(u)$  do
19:    BRONKERBOSCHPIVOT( $R \cup \{v\}, P \cap N(v), X \cap N(v), \mathcal{C}$ )
20:     $P \leftarrow P \setminus \{v\}$ 
21:     $X \leftarrow X \cup \{v\}$ 
22:  end for
23: end procedure
```

Sketch. The algorithm processes the vertices in the order $v_n, v_{n-1}, \dots, v_1$. For each vertex v_i , we initiate a Bron-Kerbosch search with pivot from $\{v_i\}$ as the initial clique R , considering only neighbors that appear later in the ordering as potential additions (P).

Since $v_1, v_2, \dots, v_n$ is the order in which the vertices were removed, each vertex has at most d neighbors that come later in the ordering. This limits the size of the potential vertex set P to at most d vertices.

The Bron-Kerbosch algorithm with pivot has a worst-case time complexity of $O(3^{n/3})$ on a graph with n vertices. When restricted to the subgraph of forward neighbors (of which there are at most d), this becomes $O(3^{d/3})$.

Since we perform this search for each of the n vertices, and each recursive call may take $O(d)$ time to compute set intersections, the overall time complexity is $O(d \cdot n \cdot 3^{d/3})$. $\square$

8.9 Introduction to Scheduling Problems

This bonus section explores an advanced application of degenerate graphs, introducing scheduling problems and their relationship to the core concepts we've covered. While not essential for understanding the fundamental principles, this material provides valuable insights into how these concepts extend to practical optimization challenges. If you are taking Math 154, skip to page 216.

Scheduling problems and graph theory share a deep and mutually beneficial relationship. Many scheduling problems can be naturally formulated as graph problems, allowing graph-theoretic techniques to be applied to their analysis and solution. Conversely, certain graph problems arise naturally from scheduling contexts, leading to algorithms with practical applications.

This section explores these connections, with a particular focus on how the concept of graph degeneracy relates to scheduling problems. We will see that d -degenerate graphs provide an important structural characterization that can be exploited in various scheduling contexts.

Definition 111 (Precedence Graph). Given a scheduling problem with precedence constraints, the corresponding precedence graph $G = (V, E)$ has:

- A vertex $v_i \in V$ for each job J_i
- A directed edge $(v_i, v_j) \in E$ if job J_i must be completed before job J_j can start

Example 37. The single-machine scheduling problem with precedence constraints $(1|prec|\sum C_j)$ is NP-hard for general precedence graphs (Lenstra and Rinnooy Kan, 1978). If the precedence constraints are series-parallel, however, an optimal schedule can be found in polynomial time, even for the weighted objective $\sum w_j C_j$ (Lawler, 1978). Series-parallel precedence constraints are not the only polynomially solvable case: $1|prec|\sum w_j C_j$ can also be solved in polynomial time when the precedence order has dimension 2 (Ambühl and Mastrolilli, 2009), a strictly larger class.

Conflict Graphs for Resource Conflicts

Resource conflicts in scheduling can be modeled using undirected conflict graphs:

Definition 112 (Conflict Graph). Given a scheduling problem with resource constraints, the corresponding conflict graph $G = (V, E)$ has:

- A vertex $v_i \in V$ for each job J_i
- An edge $(v_i, v_j) \in E$ if jobs J_i and J_j cannot be processed simultaneously due to resource constraints

Theorem 106. Suppose that every job has unit processing time and that any number of mutually compatible jobs may be processed in the same time slot. Then the problem of scheduling jobs with incompatibility constraints to minimize makespan is equivalent to coloring the corresponding conflict graph G , where each color represents a time slot; in particular, the minimum makespan equals $\chi(G)$.

Graph Algorithms in Scheduling

Several graph algorithms play crucial roles in solving scheduling problems:

Topological Sorting

Theorem 107. For scheduling problems with precedence constraints, any valid schedule must respect a topological ordering of the precedence graph.

Critical Path Method

The Critical Path Method (CPM) uses the structure of the precedence graph to determine the minimum project completion time:

Definition 113 (Critical Path). In a project network (a weighted directed acyclic graph where vertices are tasks and edge weights are durations), the critical path is the longest path from the start vertex to the end vertex, representing the minimum completion time of the project.

Theorem 108. The earliest completion time of a project with precedence constraints is equal to the length of the critical path in the corresponding project network.

8.9.1 Degenerate Graphs in Scheduling Problems

Degenerate graphs provide a powerful structural characterization that has significant implications for scheduling problems. This section explores the concept of graph degeneracy and its applications in scheduling.

Algorithm 34 Topological Sort for Precedence-Constrained Scheduling

Require: Precedence graph $G = (V, E)$

Ensure: A valid ordering of jobs respecting precedence constraints

```
1:  $L \leftarrow$  empty list
2:  $S \leftarrow$  set of all vertices with no incoming edges
3: while  $S$  is not empty do
4:   Remove a vertex  $v$  from  $S$ 
5:   Add  $v$  to the end of  $L$ 
6:   for all edges  $(v, u)$  emanating from  $v$  do
7:     Remove edge  $(v, u)$  from the graph
8:     if  $u$  has no other incoming edges then
9:       Add  $u$  to  $S$ 
10:    end if
11:  end for
12: end while
13: if graph has remaining edges then
14:   return Error (cycle detected)
15: else
16:   return  $L$ 
17: end if
```

Definition and Properties of d -degenerate Graphs

- Trees and forests are 1-degenerate
- Planar graphs are 5-degenerate
- Outerplanar graphs are 2-degenerate
- Graphs with maximum degree Δ are at most Δ -degenerate

Algorithm 35 Computing Graph Degeneracy

```
1: procedure COMPUTEDEGENERACY( $G = (V, E)$ )
2:    $d \leftarrow 0$  ▷ Initialize degeneracy value
3:    $L \leftarrow \emptyset$  ▷ Initialize empty ordering list
4:   Compute  $\deg(v)$  for each vertex  $v \in V$ 
5:   while  $V$  is not empty do
6:     Find vertex  $v \in V$  with minimum current degree  $\deg(v)$ 
7:      $d \leftarrow \max(d, \deg(v))$  ▷ Update degeneracy
8:     for all neighbors  $u$  of  $v$  that remain in  $G$  do
9:        $\deg(u) \leftarrow \deg(u) - 1$  ▷ Degrees refer to the remaining graph
10:    end for
11:    Remove  $v$  from  $G$  and add it to the beginning of  $L$ 
12:  end while
13:  return  $(d, L)$  ▷ Return degeneracy and ordering
14: end procedure
```

Applications of Degeneracy in Scheduling

The concept of degeneracy has several important applications in scheduling problems:

Resource-Constrained Scheduling

Theorem 109. For a scheduling problem where jobs correspond to vertices in a conflict graph G , if G is d -degenerate, then the jobs can be scheduled using at most $d + 1$ resources or time slots.

This follows directly from the fact that d -degenerate graphs are $(d + 1)$ -colorable, and a proper coloring corresponds to a valid schedule where jobs of the same color can be processed simultaneously.

Algorithm 36 Scheduling using Degeneracy Ordering

Require: Conflict graph $G = (V, E)$ representing incompatible jobs

Ensure: A schedule using at most $d + 1$ time slots, where d is the degeneracy of G

```

1: Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
2: Initialize color assignment  $\text{color}(v) \leftarrow \text{null}$  for all  $v \in V$ 
3: for  $i = 1$  to  $n$  do
4:    $C_{\text{used}} \leftarrow \{\text{color}(v_j) : v_j \in N(v_i) \text{ and } j < i\}$   $\triangleright$  Colors of neighbors that precede  $v_i$ 
5:    $\text{color}(v_i) \leftarrow \min\{c \geq 1 : c \notin C_{\text{used}}\}$   $\triangleright$  First available color
6: end for
7: return color assignment

```

Lemma 95. The coloring produced by Algorithm 36 uses at most $d + 1$ colors, where d is the degeneracy of the graph. This follows from the fact that each vertex has at most d neighbors preceding it in the degeneracy ordering, thus at most d colors are unavailable.

Sparse Dependency Networks

In many practical scheduling problems, the dependency graph (representing precedence constraints) is sparse, often corresponding to a low-degeneracy graph.

Sparse dependency networks often arise in real-world scheduling problems due to:

- Physical constraints limiting the number of interdependencies
- Modular design in projects, where tasks are loosely coupled
- Hierarchical organization of tasks, resulting in tree-like structures

If the precedence graph, viewed as an undirected graph, is d -degenerate, then:

- The jobs can be partitioned into at most $d + 1$ groups such that no two jobs in the same group are joined by a precedence constraint (a proper $(d + 1)$ -coloring)
- Critical path analysis takes $O(n + m)$ time for n jobs and m precedence constraints, and a d -degenerate graph has $m \leq dn$ edges

However, degeneracy does not bound the number of jobs on which a single job depends: if one job depends on all $n - 1$ other jobs and there are no other precedence constraints, the precedence graph is a star, which is 1-degenerate. Nor does it bound the number of consecutive phases needed to respect the precedences: a chain of n jobs, each depending on the previous one, is 1-degenerate but requires n phases.

Wireless Transmission Scheduling

One particularly important application of degeneracy in scheduling is in wireless networks.

Theorem 110. In a wireless network where nodes correspond to transmitters and edges represent interference between transmitters, if the interference graph is d -degenerate, then the transmitters can be scheduled in at most $d + 1$ time slots without interference.

Example 38. Consider a wireless sensor network deployed in a planar environment. If the interference graph of such a network is planar, then it is 5-degenerate. This means that, regardless of network size, transmissions can be scheduled in at most 6 time slots without interference. Graphs that are only “nearly planar” need not be 5-degenerate, however: for example, optimal 1-planar graphs (graphs that can be drawn so that each edge crosses at most one other edge, and that have the maximum possible number $4n - 8$ of edges) have minimum degree 6.

Distributed Task Allocation

Degeneracy is also useful in distributed computing environments. In a distributed system where tasks are assigned to processors and the graph of conflicts between tasks (pairs of tasks that cannot run at the same time) is d -degenerate, the tasks can be split into at most $d + 1$ groups of pairwise non-conflicting tasks (a proper $(d + 1)$ -coloring), and the tasks in each group can be run simultaneously. Finding such a grouping by a distributed algorithm is not free, however: the number of communication rounds cannot be bounded by a function of d alone, since any deterministic distributed algorithm that 3-colors an n -cycle (a 2-degenerate graph) needs at least $\frac{1}{2} \log^* n - 1$ rounds (Linial, 1992), where $\log^* n$ is the number of times the logarithm must be applied to n to obtain a number at most 1.

Algorithm 37 Distributed Task Scheduling for d -degenerate Dependency Graphs

Require: Dependency graph $G = (V, E)$ with degeneracy d

Ensure: A valid task schedule respecting dependencies

- 1: Compute a degeneracy ordering $L = [v_1, v_2, \dots, v_n]$ of G
 - 2: Partition the tasks into layers $L_1, L_2, \dots, L_m$ such that every task in L_i depends only on tasks in $L_1 \cup \dots \cup L_{i-1}$ (for example, L_i consists of the tasks whose longest chain of prerequisite tasks has exactly $i - 1$ tasks); the tasks in one layer can then be executed in parallel
 - 3: Assign tasks in layer L_i to processors only after all tasks in layers $L_1, \dots, L_{i-1}$ have completed
 - 4: **return** Layer-based schedule
-

Advantages of Degeneracy-Based Scheduling Algorithms

Degeneracy-based approaches offer several advantages for scheduling problems:

1. **Efficiency:** Degeneracy ordering can be computed in linear time $O(|V| + |E|)$
2. **Adaptability:** The approach works for any d -degenerate graph, regardless of other structural properties
3. **Simplicity:** Greedy algorithms based on degeneracy are easy to implement
4. **Performance guarantees:** Provides worst-case bounds on resource usage
5. **Incremental updates:** The degeneracy ordering can be efficiently updated as jobs are added or removed

Case Study: Project Scheduling with Limited Resources

Example 39 (Software Development Project). Consider a software development project with tasks represented as vertices and dependencies as edges. The dependency graph is typically sparse and often d -degenerate for small d due to modular design practices. Using the degeneracy ordering:

- The tasks can be split into at most $d + 1$ groups such that no two tasks in the same group are joined by a dependency (greedy coloring along the degeneracy ordering)
- The bound $d + 1$ is a guarantee, not an optimum: finding the minimum number of such groups is NP-hard in general (deciding 3-colorability is NP-complete even for 4-regular planar graphs, which are 4-degenerate)

- Critical paths can be identified in $O(n + m)$ time for n tasks and m dependencies, and $m \leq dn$ for a d -degenerate graph

Other Graph-Theoretic Approaches to Scheduling

Beyond degeneracy, other graph-theoretic concepts are valuable in scheduling.

Interval Graphs in Scheduling

Interval graphs provide a powerful representation for certain scheduling problems:

Definition 114 (Interval Graph). A graph $G = (V, E)$ is an interval graph if its vertices can be represented as intervals on the real line, with an edge between two vertices if and only if their corresponding intervals overlap.

Theorem 111. The problem of scheduling jobs with fixed start and end times on a minimum number of identical machines is equivalent to finding the chromatic number of the corresponding interval graph.

Corollary 39. Since interval graphs can be colored in polynomial time, the minimum number of machines needed for scheduling jobs with fixed start and end times can be determined efficiently.

Flow Shop and Job Shop Problems as Graph Problems

Flow shop and job shop scheduling problems have natural graph-theoretic interpretations:

Theorem 112 (Johnson's Algorithm for $F_2||C_{max}$). For a two-machine flow shop, the optimal schedule can be found using Johnson's algorithm:

1. Consider the jobs one at a time in nondecreasing order of $\min(p_{j1}, p_{j2})$ (this is the order in which the jobs are placed into the sequence, not the order in which they are processed).
2. If $p_{j1} \leq p_{j2}$, put job J_j in the earliest position of the sequence that is still free.
3. If $p_{j1} > p_{j2}$, put job J_j in the latest position of the sequence that is still free.

Both machines then process the jobs in the resulting sequence. Equivalently, the jobs with $p_{j1} \leq p_{j2}$ come first, in nondecreasing order of p_{j1} , followed by the remaining jobs in nonincreasing order of p_{j2} .

Definition 115 (Disjunctive Graph). For job shop scheduling problems, the disjunctive graph model represents:

- Vertices as operations
- Conjunctive (directed) edges as precedence constraints within jobs
- Disjunctive (undirected) edges as pairs of operations that must use the same machine

A feasible schedule corresponds to an orientation of the disjunctive edges that results in an acyclic graph.

Theorem 113. The makespan of a job shop schedule is equal to the length of the longest path in the corresponding disjunctive graph after orienting the disjunctive edges.

Computational Complexity and Approximation Algorithms

Many scheduling problems are NP-hard, meaning that finding optimal solutions is computationally intractable for large instances. Approximation algorithms and heuristics are often used in practice.

Theorem 114. The following scheduling problems are NP-hard:

- $P_2||C_{max}$ (two identical parallel machines, minimize makespan)

- $1|r_j|\sum C_j$ (single machine with release dates, minimize total completion time)
- $1||\sum w_j U_j$ (single machine, minimize weighted number of tardy jobs)
- $F_3||C_{max}$ (three-machine flow shop, minimize makespan)
- $J_2||C_{max}$ (two-machine job shop, minimize makespan)

Online Scheduling

Problem 4 (Online Scheduling). Schedule jobs that arrive over time without knowing future arrivals, to optimize an objective function.

Theorem 115. For online scheduling of jobs on $m \geq 2$ identical machines to minimize makespan, where the jobs are presented one at a time and each job must be assigned irrevocably to a machine as soon as it is presented, the competitive ratio of any deterministic algorithm is at least 1.5.

Algorithm 38 Online List Scheduling

- 1: When job J_j arrives, assign it to the machine with the earliest completion time
-

Theorem 116. Online List Scheduling has a competitive ratio of $2 - \frac{1}{m}$ for minimizing makespan on m identical parallel machines.

Applications of Scheduling Problems

Scheduling problems arise in numerous real-world applications:

- **Computing:** CPU scheduling, task scheduling in distributed systems
- **Manufacturing:** Job scheduling in production lines, resource allocation
- **Transportation:** Vehicle routing, flight scheduling, crew scheduling
- **Healthcare:** Operating room scheduling, appointment scheduling
- **Project Management:** Resource allocation, task scheduling
- **Education:** Course scheduling, exam timetabling
- **Service Industries:** Staff scheduling, appointment systems

Conclusion

Scheduling problems form a rich and varied family of optimization problems with significant practical applications. While many scheduling problems are computationally challenging (NP-hard), various techniques such as approximation algorithms, heuristics, and preprocessing methods can be used to find good solutions in practice.

The field of scheduling theory continues to evolve, with researchers developing new algorithms and studying the complexity of various scheduling problems. The interaction between scheduling theory and practice leads to both theoretical advances and improved practical solutions to real-world scheduling problems.

8.9.2 Auxiliary Algorithms for Scheduling Problems

Definition 116 (Scheduling Problem $1||\sum U_j$). Consider the single-machine scheduling problem denoted as $1||\sum U_j$, where:

- 1: Single machine environment
- $||$: No special constraints
- $\sum U_j$: Objective function to minimize the number of late jobs

For each job j , we have:

- Processing time $p_j > 0$
- Due date $d_j \geq 0$
- Completion time C_j (determined by the schedule)
- Unit penalty $U_j = \begin{cases} 1 & \text{if } C_j > d_j \\ 0 & \text{otherwise} \end{cases}$

The goal is to find a schedule that minimizes $\sum_{j=1}^n U_j$, i.e., the total number of late jobs.

Lemma 96 (Moore's Algorithm). Moore's algorithm provides an optimal solution to the $1||\sum U_j$ problem with time complexity $O(n \log n)$.

Algorithm 39 Moore's Algorithm

```

1: procedure MOORE( $J$ )
2:   Sort jobs in non-decreasing order of due dates (EDD rule)
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $d_{j_1} \leq d_{j_2} \leq \dots \leq d_{j_n}$ 
4:    $S \leftarrow \emptyset$  ▷ Initialize empty schedule
5:    $t \leftarrow 0$  ▷ Initialize current time
6:   for  $j = 1$  to  $n$  do
7:      $S \leftarrow S \cup \{j\}$  ▷ Add job  $j$  to schedule
8:      $t \leftarrow t + p_j$  ▷ Update current time
9:     if  $t > d_j$  then ▷ If job  $j$  would be late
10:      find  $j' \in S$  with maximum  $p_{j'}$  ▷ Find longest job in  $S$ 
11:       $S \leftarrow S \setminus \{j'\}$  ▷ Remove  $j'$  from schedule
12:       $t \leftarrow t - p_{j'}$  ▷ Update current time
13:     end if
14:   end for
15:   return  $S$ 
16: end procedure

```

Remark 102. The time complexity consists of:

- Sorting jobs by due dates: $O(n \log n)$
- For each job, potentially finding the maximum processing time job: $O(n)$ per iteration
- Overall complexity: $O(n^2)$

This can be improved to $O(n \log n)$ by using a max-heap to efficiently find and remove the job with maximum processing time.

Theorem 117 (Optimality of Moore's Algorithm). Moore's algorithm produces an optimal solution to the $1||\sum U_j$ problem.

Proof. Index the jobs in EDD order, $d_1 \leq d_2 \leq \dots \leq d_n$, and call a set A of jobs *on-time feasible* if every job of A meets its due date when A is sequenced in EDD order. After each iteration of the loop, the set S maintained by Moore's algorithm is on-time feasible: if job j is late and the longest job j' is removed, then the other jobs of S finish no later than before, and j (if $j' \neq j$) now finishes at time $t - p_{j'} \leq t - p_j$, the value of t before j was added, which was at most d_j .

Step 1: EDD order suffices. If the EDD sequence of a set A has a first late job k , then $\sum_{i \in A, i \leq k} p_i > d_k$, and in *any* ordering of A the last of the jobs $i \in A$ with $i \leq k$, say ℓ , completes at time at least $\sum_{i \in A, i \leq k} p_i > d_k \geq d_\ell$, so it is late. Hence we may assume that an optimal schedule processes an on-time feasible set A of maximum size in EDD order, followed by all other jobs.

Step 2: the first removal is safe. If the algorithm never removes a job, every job is on time and there is nothing to prove. Otherwise let k be the first job at which $t > d_k$ occurs, so that jobs $1, \dots, k-1$ are on time in the EDD sequence while $p_1 + \dots + p_k > d_k$, and let $j' \leq k$ be the job removed at that moment, so $p_{j'} = \max_{i \leq k} p_i$. We claim that some optimal schedule has j' late. Take A as in Step 1. Since $\{1, \dots, k\}$ is not on-time feasible, A misses some job $r \leq k$. If $j' \notin A$ we are done. Otherwise let $A' = (A \setminus \{j'\}) \cup \{r\}$, so $|A'| = |A|$. In the EDD sequence of A' , each job $i \in A'$ with $i \leq k-1$ completes by time $p_1 + \dots + p_i \leq d_i$; if $k \in A'$, it completes by time $\sum_{i \leq k, i \neq j'} p_i \leq p_1 + \dots + p_{k-1} \leq d_{k-1} \leq d_k$ (because $p_{j'} \geq p_k$); and each job $i \in A'$ with $i > k$ completes $p_{j'} - p_r \geq 0$ time units earlier than in the EDD sequence of A . Thus A' is on-time feasible, has maximum size, and does not contain j' .

Step 3: induction on n . Let $\text{opt}(J)$ denote the minimum number of late jobs. Since S is on-time feasible, it suffices to show that the algorithm removes exactly $\text{opt}(J)$ jobs. Deleting j' from the optimal schedule of Step 2 leaves a schedule of $J \setminus \{j'\}$ with at most $\text{opt}(J) - 1$ late jobs, while appending j' to any schedule of $J \setminus \{j'\}$ adds at most one late job; hence $\text{opt}(J) = \text{opt}(J \setminus \{j'\}) + 1$. On input $J \setminus \{j'\}$ the algorithm removes none of the jobs $1, \dots, k$ (by the estimates of Step 2), so once these jobs have been considered it has the same set S and time t as the run on J , and from then on the two runs coincide. By the induction hypothesis the run on $J \setminus \{j'\}$ removes $\text{opt}(J \setminus \{j'\})$ jobs, so the run on J removes $\text{opt}(J \setminus \{j'\}) + 1 = \text{opt}(J)$ jobs. $\square$

Lemma 97 (Heap-Based Implementation). A more efficient implementation of Moore's algorithm uses a max-heap data structure:

Algorithm 40 Moore's Algorithm with Heap

```

1: procedure MOOREWITHHEAP( $J$ )
2:   Sort jobs in non-decreasing order of due dates (EDD rule)
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $d_{j_1} \leq d_{j_2} \leq \dots \leq d_{j_n}$ 
4:    $S \leftarrow \emptyset$  ▷ Initialize empty schedule
5:    $H \leftarrow$  empty max-heap ▷ For tracking processing times
6:    $t \leftarrow 0$  ▷ Initialize current time
7:   for  $j = 1$  to  $n$  do
8:      $S \leftarrow S \cup \{j\}$  ▷ Add job  $j$  to schedule
9:      $H.\text{INSERT}(p_j)$  ▷ Add processing time to heap
10:     $t \leftarrow t + p_j$  ▷ Update current time
11:    if  $t > d_j$  then ▷ If job  $j$  would be late
12:       $p_{\max} \leftarrow H.\text{EXTRACTMAX}()$  ▷ Extract max processing time
13:       $S \leftarrow S \setminus \{\text{job with processing time } p_{\max}\}$ 
14:       $t \leftarrow t - p_{\max}$  ▷ Update current time
15:    end if
16:  end for
17:  return  $S$ 
18: end procedure

```

Remark 103. With this implementation, the time complexity is $O(n \log n)$, as each heap operation takes $O(\log n)$ time, and we perform at most $O(n)$ such operations.

Definition 117 (Scheduling Problem $1|prec|\sum w_j T_j$). In this single-machine problem, each job j has a processing time $p_j > 0$, a due date $d_j \geq 0$ and a weight $w_j \geq 0$; its completion time C_j is determined by the schedule, and its tardiness is $T_j = \max(0, C_j - d_j)$.

Precedence constraints are given as a directed acyclic graph where an edge (i, j) means job i must be completed before job j can start.

The goal is to find a schedule that minimizes $\sum_{j=1}^n w_j T_j$, i.e., the weighted sum of tardiness across all jobs while respecting precedence constraints.

Lemma 98 (Lawler's Dynamic Programming Algorithm). Lawler's algorithm provides an optimal solution to the $1|prec|\sum w_j T_j$ problem using dynamic programming based on subsets of jobs.

Algorithm 41 Lawler's Dynamic Programming Algorithm

```

1: procedure LAWLERDP( $J, prec$ )
2:   Initialize  $OPT[\emptyset] \leftarrow 0$                                 ▷ Base case: empty set has zero cost
3:   Initialize  $schedule[\emptyset] \leftarrow \emptyset$                 ▷ Empty schedule for empty set
4:   for  $k = 1$  to  $n$  do                                       ▷ Iterate over all subset sizes
5:     for each subset  $S \subseteq J$  with  $|S| = k$  do
6:        $OPT[S] \leftarrow \infty$                                 ▷ Initialize optimal cost for subset  $S$ 
7:       for each job  $j \in S$  that has no successors in  $S$  according to  $prec$  do
8:          $S' \leftarrow S \setminus \{j\}$                         ▷ Subset without job  $j$ 
9:         Calculate completion time  $C_j$  when  $j$  is scheduled after  $S'$ 
10:        Calculate tardiness  $T_j = \max(0, C_j - d_j)$ 
11:        Calculate cost  $cost \leftarrow OPT[S'] + w_j \cdot T_j$ 
12:        if  $cost < OPT[S]$  then
13:           $OPT[S] \leftarrow cost$                                 ▷ Update optimal cost
14:           $schedule[S] \leftarrow schedule[S'] + j$             ▷ Update optimal schedule
15:        end if
16:      end for
17:    end for
18:  end for
19:  return  $schedule[J]$                                        ▷ Return optimal schedule for all jobs
20: end procedure

```

Remark 104. The time complexity consists of:

- Number of subsets considered: $O(2^n)$
- Work per subset: $O(n)$ to find feasible last jobs and compute costs
- Overall complexity: $O(n \cdot 2^n)$

Despite the exponential worst-case complexity, this algorithm is practical for instances with up to a few dozen jobs, especially when problem-specific optimizations are applied.

Theorem 118 (Optimality of Lawler's Algorithm). Lawler's dynamic programming algorithm produces an optimal solution to the $1|prec|\sum w_j T_j$ problem.

Proof (sketch). The proof follows from the principle of optimality for dynamic programming. For any subset S of jobs, if we know the optimal schedule for every proper subset, then we can determine the optimal schedule for S by trying each feasible last job and selecting the one that minimizes the total cost.

A job j is a feasible last job for subset S if it has no successors in S according to the precedence constraints. When j is scheduled last in S , its completion time depends only on the schedule of jobs in $S \setminus \{j\}$, and its contribution to the objective function is $w_j \cdot \max(0, C_j - d_j)$.

By systematically building solutions from smaller to larger subsets and maintaining optimality at each step, the algorithm guarantees an optimal solution for the complete set of jobs. $\square$

Lemma 99 (Implementation Optimizations). Several optimizations can reduce the practical complexity of Lawler's algorithm:

- **Dominance rules:** Eliminate subsets that cannot lead to optimal solutions based on job characteristics.
- **State-space reduction:** Exploit precedence constraints to reduce the number of subsets considered.
- **Bounding:** Use lower bounds to prune suboptimal partial solutions early.
- **Memoization:** Only compute and store values for subsets that are actually reached during the recursive computation.

Remark 105. *With these optimizations, many practical instances with up to 20-30 jobs can be solved efficiently, despite the exponential worst-case complexity. The algorithm is particularly effective when the precedence constraints significantly restrict the set of feasible schedules.*

Remark 106 (Special Cases). *Neither restriction alone makes the problem easy. Without precedence constraints, $1||\sum w_j T_j$ is still strongly NP-hard (Lenstra, Rinnooy Kan and Brucker 1977). With equal weights, $1|prec|\sum T_j$ is still NP-hard, since it contains $1|prec|\sum C_j$ (take all $d_j = 0$), which Lenstra and Rinnooy Kan (1978) proved NP-hard. Only when both restrictions hold, i.e. for $1||\sum T_j$, is a pseudo-polynomial algorithm available (Lawler 1977); even this problem is NP-hard in the ordinary sense (Du and Leung 1990). Subset dynamic programming of the above type therefore remains a standard exact method for the general $1|prec|\sum w_j T_j$ problem.*

Definition 118 (Scheduling Problem $P||C_{\max}$). Consider the parallel identical machines scheduling problem denoted as $P||C_{\max}$, where:

- P : Parallel identical machines environment
- $||$: No special constraints
- $C_{\max}$: Objective function to minimize the makespan (completion time of the last job)

We have m identical machines and n jobs, where:

- Each job j has processing time $p_j > 0$
- Each job must be processed without interruption on one of the machines
- All machines are identical and can process any job
- The makespan $C_{\max} = \max_{i=1}^m C_i$ is the maximum completion time across all machines

The goal is to find a schedule that minimizes the makespan $C_{\max}$.

Lemma 100 (List Scheduling Algorithm). The List Scheduling algorithm is a simple greedy approach for the $P||C_{\max}$ problem:

Remark 107. *The time complexity is $O(n \log m)$ if a min-heap is used to keep track of machine loads.*

Theorem 119 (Approximation Ratio of List Scheduling). List Scheduling is a 2-approximation algorithm for the $P||C_{\max}$ problem.

Proof. Let OPT be the makespan of an optimal schedule. We know that:

$$OPT \geq \frac{\sum_{j=1}^n p_j}{m} \quad (\text{average load}) \quad (76)$$

$$OPT \geq \max_{j=1}^n p_j \quad (\text{maximum job processing time}) \quad (77)$$

Algorithm 42 List Scheduling

```
1: procedure LISTSCHEDULING( $J, m$ )
2:   Initialize  $load[i] \leftarrow 0$  for all machines  $i = 1, 2, \dots, m$  ▷ Current load on each machine
3:   for each job  $j \in J$  do ▷ In arbitrary order
4:     Find machine  $i$  with minimum current load  $load[i]$ 
5:     Assign job  $j$  to machine  $i$ 
6:      $load[i] \leftarrow load[i] + p_j$  ▷ Update machine load
7:   end for
8:   return assignment of jobs to machines
9: end procedure
```

Let j^* be the last job to complete in the List Scheduling solution, and let i^* be the machine it was assigned to. When j^* was assigned, machine i^* had the lowest load among all machines. Let C_{j^*} be the completion time of job j^* , which equals the makespan of the List Scheduling solution.

We have:

$$C_{j^*} = load_{i^*} \text{ (before } j^* \text{ was assigned)} + p_{j^*} \quad (78)$$

$$\leq \frac{\sum_{j \neq j^*} p_j}{m} + p_{j^*} \quad (\text{machine } i^* \text{ had minimum load}) \quad (79)$$

$$= \frac{\sum_{j=1}^n p_j - p_{j^*}}{m} + p_{j^*} \quad (80)$$

$$= \frac{\sum_{j=1}^n p_j}{m} + p_{j^*} \left(1 - \frac{1}{m}\right) \quad (81)$$

$$\leq OPT + OPT \left(1 - \frac{1}{m}\right) \quad (82)$$

$$< 2 \cdot OPT \quad (83)$$

Therefore, the makespan of the List Scheduling solution is less than twice the optimal makespan. $\square$

Lemma 101 (Longest Processing Time First (LPT) Algorithm). The LPT algorithm is an improvement over List Scheduling for the $P||C_{\max}$ problem:

Algorithm 43 Longest Processing Time First (LPT)

```
1: procedure LPT( $J, m$ )
2:   Sort jobs in non-increasing order of processing times
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $p_{j_1} \geq p_{j_2} \geq \dots \geq p_{j_n}$ 
4:   Initialize  $load[i] \leftarrow 0$  for all machines  $i = 1, 2, \dots, m$ 
5:   for each job  $j \in J$  do ▷ In sorted order (longest first)
6:     Find machine  $i$  with minimum current load  $load[i]$ 
7:     Assign job  $j$  to machine  $i$ 
8:      $load[i] \leftarrow load[i] + p_j$ 
9:   end for
10:  return assignment of jobs to machines
11: end procedure
```

Remark 108. The time complexity is $O(n \log n)$ for sorting plus $O(n \log m)$ for job assignments, giving a total of $O(n \log n)$ since typically $n \geq m$.

Theorem 120 (Approximation Ratio of LPT). The LPT algorithm is a $(\frac{4}{3} - \frac{1}{3m})$ -approximation algorithm for the $P||C_{\max}$ problem.

Proof (sketch). The proof is more complex than for List Scheduling, involving a careful analysis of the structure of an LPT schedule. The key insight is that when LPT performs poorly, there must be a specific structure to the schedule that can be bounded.

Through case analysis, one can show that for any instance:

$$C_{\max}^{LPT} \leq \left(\frac{4}{3} - \frac{1}{3m} \right) \cdot OPT \quad (84)$$

This bound is tight, meaning there exist instances where LPT produces a makespan exactly $(\frac{4}{3} - \frac{1}{3m})$ times the optimal. $\square$

Lemma 102 (PTAS for $P_m||C_{\max}$). For a fixed number of machines m , there exists a Polynomial Time Approximation Scheme (PTAS) for the $P_m||C_{\max}$ problem:

Algorithm 44 PTAS for $P_m||C_{\max}$

- 1: **procedure** PTAS(J, m, ϵ)
 - 2: Let T be a guessed lower bound on OPT (binary search on possible values)
 - 3: Partition jobs into "large" ($p_j > \epsilon T$) and "small" ($p_j \leq \epsilon T$)
 - 4: Round processing times of large jobs to multiples of $\frac{\epsilon^2 T}{n}$
 - 5: Enumerate all possible assignments of large jobs to machines (patterns)
 - 6: For each feasible pattern (load $\leq (1 + \epsilon)T$ on each machine):
 - 7: Assign small jobs greedily in any order to machines with smallest load
 - 8: Return the best schedule found
 - 9: **end procedure**
-

Remark 109. For fixed m and ϵ , the algorithm runs in polynomial time $O(n^{O(m/\epsilon)})$.

Theorem 121 (Approximation Ratio of PTAS). For any $\epsilon > 0$, the PTAS algorithm produces a schedule with makespan at most $(1 + \epsilon) \cdot OPT$ for the $P_m||C_{\max}$ problem with a fixed number of machines m .

Proof (sketch). The proof consists of several steps:

1. There are at most $O(1/\epsilon)$ large jobs per machine in an optimal solution
2. Rounding large jobs introduces at most $\epsilon \cdot OPT$ error
3. The number of different patterns to consider is polynomial for fixed m and ϵ
4. Greedy assignment of small jobs adds at most $\epsilon \cdot OPT$ to the makespan

Combining these observations, we get a $(1 + \epsilon)$ -approximation algorithm that runs in polynomial time for fixed m and ϵ . $\square$

Definition 119 (Scheduling Problem $R||C_{\max}$). Consider the unrelated parallel machines scheduling problem denoted as $R||C_{\max}$, where:

- R : Unrelated parallel machines environment
- $||$: No special constraints
- $C_{\max}$: Objective function to minimize the makespan

We have m unrelated machines and n jobs, where:

- Each job j has processing time $p_{ij} > 0$ on machine i
- Processing times may vary across machines (unrelated machines)
- Each job must be processed without interruption on one of the machines

- The makespan $C_{\max} = \max_{i=1}^m C_i$ is the maximum completion time across all machines

The goal is to find a schedule that minimizes the makespan $C_{\max}$.

Lemma 103 (Shmoys-Tardos Algorithm). The Shmoys-Tardos algorithm for the $R||C_{\max}$ problem uses linear programming relaxation and rounding:

Algorithm 45 Shmoys-Tardos Algorithm

- 1: Guess a value T for the optimal makespan (binary search)
 - 2: Formulate the linear programming relaxation $LP(T)$ of the assignment problem:
 - 3: Variables $x_{ij} \geq 0$: the fraction of job j assigned to machine i
 - 4: Constraints: $\sum_{i=1}^m x_{ij} = 1$ for all jobs j
 - 5: Constraints: $\sum_{j=1}^n p_{ij} \cdot x_{ij} \leq T$ for all machines i
 - 6: Constraints: $x_{ij} = 0$ if $p_{ij} > T$
 - 7: Let T^* be the smallest T found for which $LP(T)$ is feasible, and let x_{ij}^* be a feasible solution of $LP(T^*)$
 - 8: Convert the fractional solution to an integral assignment using bipartite matching:
 - 9: For each machine i , create $k_i = \lceil \sum_j x_{ij}^* \rceil$ slots $(i, 1), \dots, (i, k_i)$
 - 10: Sort the jobs with $x_{ij}^* > 0$ in non-increasing order of p_{ij} and pour their fractions x_{ij}^* , in this order, into the slots $(i, 1), (i, 2), \dots$, filling each slot up to total 1 before using the next one (a fraction may be split between two consecutive slots)
 - 11: Construct the bipartite graph with jobs on one side and slots on the other, with an edge between job j and slot (i, s) if part of x_{ij}^* was poured into (i, s)
 - 12: Find a matching in this graph that covers every job
 - 13: Assign each job to the machine owning its matched slot
-

Remark 110. The algorithm runs in polynomial time, with the LP solving being the most expensive step.

Theorem 122 (Approximation Ratio of Shmoys-Tardos). The Shmoys-Tardos algorithm is a 2-approximation algorithm for the $R||C_{\max}$ problem. The first 2-approximation algorithm for $R||C_{\max}$, which rounds a vertex of $LP(T)$, is due to Lenstra, Shmoys and Tardos (1990); the slot-matching rounding above is due to Shmoys and Tardos (1993).

Proof (sketch). The proof follows these key steps:

1. If $T \geq OPT$, then $LP(T)$ is feasible: an optimal schedule gives a 0–1 solution of $LP(T)$, since every machine load in it, and hence every p_{ij} of a job j assigned to machine i , is at most $OPT \leq T$. Hence $T^* \leq OPT$. (The converse fails: $LP(T)$ may be feasible for some $T < OPT$; for three unit jobs on two identical machines, $LP(3/2)$ is feasible while $OPT = 2$.)
2. The poured fractions form a fractional matching that covers every job exactly once and every slot at most once. Since bipartite matching polytopes are integral, a matching covering every job exists, so each job is assigned to exactly one machine i , and only to a machine with $x_{ij}^* > 0$, hence with $p_{ij} \leq T^*$.
3. Machine i receives at most one job per slot. The job matched to slot $(i, 1)$ has $p_{ij} \leq T^*$. For $s \geq 2$, slot $(i, s - 1)$ is full, and every job poured into slot (i, s) is at most as long as every job poured into slot $(i, s - 1)$; so the job matched to (i, s) has processing time at most the weighted average $\sum_j p_{ij} \cdot (\text{fraction of } j \text{ in slot } (i, s - 1))$. Summing over $s \geq 2$ gives at most $\sum_j p_{ij} x_{ij}^* \leq T^*$.

Therefore the load of every machine is at most $T^* + T^* = 2T^* \leq 2 \cdot OPT$. $\square$

Definition 120 (Mixed-Integer Programming for Scheduling). Mixed-Integer Programming (MIP) is a general-purpose optimization approach that can be applied to a wide range of scheduling problems. For scheduling problems, we typically formulate:

- **Decision variables:**

- Binary assignment variables $x_{ij} \in \{0, 1\}$ indicating if job j is assigned to machine i

- Binary sequencing variables $y_{jk} \in \{0, 1\}$ indicating if job j precedes job k
- Continuous time variables $C_j \geq 0$ representing completion time of job j

- **Objective function:** Minimize relevant metric (e.g., makespan, total weighted completion time)
- **Constraints:** Machine capacity, precedence relations, disjunctive constraints for resource conflicts

Lemma 104 (Time-Indexed MIP Formulation). A common approach for single-machine scheduling problems is the time-indexed formulation:

Let $x_{jt} \in \{0, 1\}$ indicate whether job j starts at time t .

$$\min \sum_{j=1}^n \sum_{t=0}^{T-p_j} w_j(t + p_j)x_{jt} \quad (85)$$

$$\text{s.t.} \quad \sum_{t=0}^{T-p_j} x_{jt} = 1 \quad \forall j \in \{1, 2, \dots, n\} \quad (86)$$

$$\sum_{j=1}^n \sum_{s=\max(0, t-p_j+1)}^{\min(t, T-p_j)} x_{js} \leq 1 \quad \forall t \in \{0, 1, \dots, T-1\} \quad (87)$$

$$x_{jt} \in \{0, 1\} \quad \forall j \in \{1, 2, \dots, n\}, t \in \{0, 1, \dots, T-p_j\} \quad (88)$$

where T is an upper bound on the makespan, p_j is the processing time of job j , and w_j is the weight of job j .

Remark 111. The first constraint ensures each job is scheduled exactly once. The second constraint ensures that at most one job is processed at any time t . The objective minimizes the weighted sum of completion times.

Theorem 123 (Strengthening MIP Formulations). MIP formulations for scheduling problems can be significantly strengthened through the addition of valid inequalities and symmetry-breaking constraints, particularly:

- **Cover inequalities:** If a subset S of jobs cannot all be processed within a given deadline d , then $\sum_{j \in S} \sum_{t=0}^{d-p_j} x_{jt} \leq |S| - 1$
- **Strengthened precedence constraints:** If job i must precede job j , then $C_i + p_j \leq C_j + M(1 - y_{ij})$ can be replaced with $C_i + p_j \leq C_j$ when $y_{ij} = 1$ must hold
- **Symmetry-breaking constraints:** For identical jobs, impose an arbitrary ordering, e.g., $C_1 \leq C_2 \leq \dots \leq C_k$ for identical jobs $1, 2, \dots, k$

Proof (sketch). Cover inequalities and strengthened precedence constraints are valid inequalities: every feasible schedule satisfies them, so they cut off only fractional points of the linear programming relaxation, which can reduce the integrality gap and prune more of the branch-and-bound tree. Symmetry-breaking constraints are not valid inequalities in this sense, since they cut off feasible schedules; however, relabelling identical jobs turns every feasible schedule into one that satisfies them and has the same objective value, so at least one optimal solution is kept, and the search no longer explores symmetric copies of the same schedule. $\square$

Remark 112 (Practical Performance of MIP). MIP solvers like Gurobi and CPLEX (commercial) and SCIP (open source) can handle scheduling problems with up to a few hundred jobs, particularly when:

- The formulation is tight (small integrality gap)
- Problem-specific cuts are implemented

- Warm-start solutions from heuristics are provided
- Appropriate branching priorities are set

The practical limit depends on the specific problem structure and time constraints, but problems with 50-200 jobs are typically manageable with modern solvers.

Definition 121 (Column Generation for Scheduling). Column Generation is a decomposition technique for solving large-scale linear programs by generating columns (variables) on-demand. For scheduling problems, columns typically represent:

- Complete schedules for a single machine
- Feasible routing paths for a vehicle
- Workshift assignments for an employee

The approach decomposes the problem into:

- **Master Problem:** Select which columns (schedules) to use
- **Pricing Subproblem:** Generate new promising columns (schedules)

Lemma 105 (Column Generation Algorithm). The Column Generation approach for scheduling problems follows this general pattern:

Algorithm 46 Column Generation for Scheduling

```

1: procedure COLUMNGENERATION(Jobs, Machines, Constraints)
2:   Initialize restricted master problem (RMP) with a small set of feasible columns
3:   repeat
4:     Solve the RMP to obtain dual variables  $\pi$ 
5:     for each machine or resource do
6:       Solve pricing subproblem using  $\pi$  to find column with negative reduced cost
7:       if column with negative reduced cost found then
8:         Add column to RMP
9:       end if
10:    end for
11:  until no column with negative reduced cost is found
12:  return optimal solution to the linear relaxation
13: end procedure

```

Remark 113. The pricing subproblem often takes the form of a resource-constrained shortest path problem or a knapsack problem. These problems are NP-hard in general, but they can often be solved effectively in practice using dynamic programming (for the knapsack problem, in pseudo-polynomial time).

Theorem 124 (Branch-and-Price Framework). Branch-and-Price combines column generation with branch-and-bound to solve integer programs:

Proof of optimality. The optimality of Branch-and-Price follows from:

- Column generation guarantees the optimal solution to the LP relaxation at each node
- Branch-and-bound with proper branching rules eventually explores all feasible integer solutions
- Pruning based on bounds maintains optimality while improving efficiency

□

Remark 114 (Applications in Scheduling). Branch-and-Price is particularly effective for:

Algorithm 47 Branch-and-Price for Scheduling

```
1: procedure BRANCHANDPRICE(Jobs, Machines, Constraints)
2:   Initialize branch-and-bound tree with root node
3:   while tree has unexplored nodes do
4:     Select unexplored node
5:     Apply column generation to solve LP relaxation at node
6:     if LP solution is integer then
7:       Update incumbent solution if better
8:     else if LP bound  $\geq$  incumbent then
9:       Identify branching variable (e.g., job assignment or sequencing decision)
10:      Create child nodes based on branching decision
11:    end if
12:  end while
13:  return optimal integer solution
14: end procedure
```

- *Crew scheduling and rostering problems*
- *Vehicle routing with time windows*
- *Parallel machine scheduling with sequence-dependent setup times*
- *Resource-constrained project scheduling*

The technique enables solving scheduling problems with complex constraints by encapsulating these constraints in the pricing subproblem, while the master problem handles the global resource allocation.

Definition 122 (Benders Decomposition for Scheduling). Benders Decomposition partitions variables in a MIP into:

- **Complicating variables:** Often binary assignment decisions (which job goes to which machine)
- **Continuous variables:** Typically timing decisions (when each job starts/completes)

The problem is decomposed into:

- **Master Problem:** Makes decisions on complicating variables
- **Subproblem:** Given fixed complicating variables, optimizes continuous variables or generates feasibility cuts

Lemma 106 (Benders Decomposition Algorithm). The Benders Decomposition approach for scheduling problems follows this iterative pattern:

Remark 115. *For scheduling problems, Benders cuts often have a natural interpretation:*

- *Feasibility cuts:* "These job assignments cannot be scheduled feasibly"
- *Optimality cuts:* "Any assignment similar to this one has completion time at least z "

Theorem 125 (Convergence of Benders Decomposition). Under standard assumptions, Benders Decomposition converges to an optimal solution in a finite number of iterations.

Proof (sketch). Assume that the complicating variables take finitely many values (for example, they are binary) and that every cut is valid (it removes no feasible solution of the original problem) and tight at the master solution x^* that generated it: a feasibility cut excludes x^* , and an optimality cut forces the master problem's estimate of the subproblem cost at x^* to be at least z_{sub} .

- By validity, the optimal value of the restricted master problem is a lower bound on the optimal value, and every feasible subproblem solution gives an upper bound

Algorithm 48 Benders Decomposition for Scheduling

```
1: procedure BENDERSDECOMPOSITION(Jobs, Machines, Constraints)
2:   Initialize master problem with complicating variables and no cuts
3:   repeat
4:     Solve restricted master problem to obtain assignment  $x^*$ 
5:     Fix assignment variables to  $x^*$  and solve subproblem
6:     if subproblem is infeasible then
7:       Generate feasibility cut:  $\sum_{(i,j):x_{ij}^*=1}(1-x_{ij}) + \sum_{(i,j):x_{ij}^*=0}x_{ij} \geq 1$ 
8:       Add feasibility cut to master problem
9:     else if subproblem is feasible with objective value  $z_{sub}$  then
10:      Generate optimality cut using dual solution of subproblem
11:      Add optimality cut to master problem
12:     end if
13:   until no more cuts needed (optimal solution found)
14:   return optimal solution
15: end procedure
```

- An optimality cut need not eliminate the assignment x^* itself; it only eliminates x^* together with too small an estimate of its subproblem cost
- If the master problem returns an assignment x^* that it has returned before, the cut generated at the earlier visit makes its estimate at x^* equal to the true subproblem cost; then the lower bound equals an upper bound already found, and the algorithm stops with an optimal solution
- Hence every iteration before termination produces a new assignment; since there are finitely many assignments, the algorithm terminates after finitely many iterations

□

Remark 116 (Modern Benders Implementations). *Modern implementations of Benders Decomposition improve performance through:*

- **Cut management:** *Only adding cuts that significantly improve bounds*
- **In-tree implementation:** *Generating cuts at nodes of the branch-and-bound tree*
- **Stabilization techniques:** *Preventing oscillation in the master problem solutions*
- **Initialization:** *Starting with strong valid inequalities and heuristic solutions*

These enhancements make Benders Decomposition particularly effective for large-scale scheduling problems with complex timing constraints, where the subproblem structure (after fixing assignments) is relatively simple.

Definition 123 (Constraint Programming for Scheduling). Constraint Programming (CP) is a declarative paradigm particularly well-suited for scheduling problems. In CP, a scheduling problem is modeled as:

- **Decision variables:**
 - start_j : Start time of job j
 - end_j : End time of job j
 - machine_j : Machine assigned to job j
- **Domains:** Finite sets of possible values for each variable
- **Constraints:** Relations that must be satisfied by variable assignments
- **Objective function:** Measure to minimize or maximize (e.g., makespan)

CP solvers use:

- **Constraint propagation:** Reducing variable domains based on constraints
- **Search:** Systematically exploring combinations of variable assignments
- **Backtracking:** Undoing assignments that lead to constraint violations

Lemma 107 (Global Constraints for Scheduling). CP for scheduling relies on specialized global constraints that model common scheduling requirements:

1. **cumulative(starts, durations, resources, capacity):** Ensures resource usage never exceeds capacity
 - $starts_j$: Start time of job j
 - $durations_j$: Processing time of job j
 - $resources_j$: Resource requirement of job j
 - $capacity$: Maximum available resource
2. **noOverlap(starts, durations):** Ensures jobs do not overlap in time (for single machine scheduling)
3. **precedence(start _{i} , duration _{i} , start _{j}):** Ensures job i completes before job j starts
4. **alternative(start, duration, machines, machineStarts):** Models optional assignment of jobs to machines
5. **disjunctive(starts, durations):** Special case of cumulative with unit resource requirements and unit capacity

Algorithm 49 CP Model for Job Shop Scheduling

```

1: procedure CPJOBSSHOP(Jobs, Machines, ProcessingTimes, Operations)
2:   Define variables:
3:      $start_{j,o}$  for start time of operation  $o$  of job  $j$ 
4:      $end_{j,o} = start_{j,o} + ProcessingTimes_{j,o,m}$  where  $m$  is the machine for operation  $o$ 
5:   Add constraints:
6:     // Precedence constraints between operations of the same job
7:     for each job  $j$  and consecutive operations  $o_1, o_2$  do
8:        $precedence(start_{j,o_1}, ProcessingTimes_{j,o_1,m}, start_{j,o_2})$ 
9:     end for
10:    // No overlap constraints for operations on the same machine
11:    for each machine  $m$  do
12:       $operations_m \leftarrow$  operations assigned to machine  $m$ 
13:       $starts_m \leftarrow$  start times of operations in  $operations_m$ 
14:       $durations_m \leftarrow$  processing times of operations in  $operations_m$ 
15:       $noOverlap(starts_m, durations_m)$ 
16:    end for
17:   Define objective:
18:      $makespan \leftarrow \max_{j,o} end_{j,o}$ 
19:      $minimize(makespan)$ 
20:   Solve using CP search
21:   return schedule
22: end procedure

```

Remark 117. The power of CP for scheduling comes from specialized filtering algorithms (propagators) for each global constraint. For example:

- The **cumulative** constraint uses edge-finding, time-tabling, and energetic reasoning
- The **disjunctive** constraint uses specialized algorithms to detect mandatory precedences

These propagators efficiently detect and remove inconsistent values from variable domains, drastically reducing the search space.

Theorem 126 (Propagation Strength of Global Constraints). Global constraints in CP scheduling models provide stronger inference than equivalent decompositions into elementary constraints.

Proof (sketch). Consider a disjunctive constraint over n operations. Its direct propagation can:

- Detect necessary precedences through edge-finding and not-first/not-last algorithms
- Update earliest start times and latest completion times of operations
- Establish order relationships even before variable domains are reduced to singletons

In contrast, a decomposition into $O(n^2)$ binary constraints ($start_i + duration_i \leq start_j$ OR $start_j + duration_j \leq start_i$) would achieve weaker propagation, typically waiting until one variable becomes fixed before propagating to others.

The cumulative constraint similarly achieves stronger inference through specialized energy-based reasoning that cannot be matched by decomposition into simpler constraints. $\square$

Remark 118 (Performance Characteristics of CP for Scheduling). *CP solvers like Google OR-Tools, IBM ILOG CP Optimizer, and Gecode excel at scheduling problems with:*

- Complex temporal constraints and resource requirements
- Highly combinatorial decision spaces
- Small to medium-sized instances (typically up to 100-200 operations)
- Problems where finding feasible solutions is challenging

CP often outperforms MIP on highly constrained scheduling problems, but may struggle with:

- Large-scale problems (thousands of operations)
- Problems where the bound on the objective is weak
- Problems requiring reasoning about sums or averages

Lemma 108 (Hybrid CP-MIP Approach). Hybrid approaches combine the strengths of CP and MIP by:

Algorithm 50 Hybrid CP-MIP for Scheduling

```

1: procedure HYBRIDCPMIP(Jobs, Machines, Constraints)
2:   // Phase 1: Use CP to find good feasible solutions quickly
3:   CPModel  $\leftarrow$  Formulate problem using CP with global constraints
4:   Set time limit for CP search
5:   feasibleSchedules  $\leftarrow$  Run CP search with diversification
6:   // Phase 2: Use MIP to improve solutions or prove optimality
7:   MIPModel  $\leftarrow$  Formulate problem as MIP
8:   Warm-start MIP with best solutions from CP phase
9:   Add constraints derived from CP search (e.g., bounds, valid inequalities)
10:  Run MIP solver to improve solution or prove optimality
11:  return best solution found
12: end procedure

```

Remark 119. *Several strategies exist for CP-MIP hybridization:*

- **Sequential:** Use CP to find feasible solutions, then MIP to prove optimality
- **Parallel:** Run CP and MIP solvers concurrently, sharing bounds and solutions
- **Integrated:** Use MIP for assignment decisions and CP for scheduling decisions
- **Logic-based Benders:** Use MIP master problem and CP subproblems

The appropriate strategy depends on the problem characteristics and the relative strengths of CP and MIP for specific constraint types.

Theorem 127 (Complementarity of CP and MIP). CP and MIP exhibit complementary strengths for scheduling problems, making hybrid approaches particularly effective.

Proof (informal). The complementarity arises from fundamental differences in how each technique handles constraints:

- **CP excels at:** Logical constraints, disjunctions, global constraints with specialized propagators
- **MIP excels at:** Linear constraints, strong relaxations providing bounds, large-scale problems

For a scheduling problem with both assignment and sequencing decisions:

1. CP can quickly explore the combinatorial space of feasible solutions
2. MIP can provide strong lower bounds and gradually improve solution quality
3. CP struggles with objective-based reasoning that MIP handles effectively
4. MIP struggles with logical constraints that CP handles naturally

Empirically, hybrid approaches have been shown to outperform pure CP or pure MIP on a variety of benchmark scheduling problems, particularly those combining assignment, sequencing, and resource constraints. $\square$

Remark 120 (Example Implementation). A CP model for job shop scheduling can be implemented using the following pseudocode:

This pseudocode captures the essence of scheduling with global constraints for precedence relations and no-overlap requirements. It demonstrates how CP models use specialized constraints like noOverlap to efficiently handle disjunctive resources.

Definition 124 (Real-Time Scheduling). Real-time scheduling addresses problems where:

- Jobs have release times r_j and hard deadlines d_j
- Missing a deadline results in system failure or significant penalty
- Scheduler must make decisions as jobs arrive (online setting)
- Preemption (interrupting and resuming jobs) is often allowed

Key metrics include:

- **Schedulability:** Whether all jobs can meet their deadlines
- **Processor utilization:** $U = \sum_{j=1}^n \frac{C_j}{T_j}$, where C_j is computation time and T_j is period
- **Response time:** Time between release and completion

Lemma 109 (Earliest Deadline First (EDF) Algorithm). EDF is a dynamic-priority scheduling algorithm that always executes the ready job with the earliest absolute deadline:

Remark 121. For periodic task systems where job j arrives every T_j time units and requires C_j processing time:

Algorithm 51 CP Implementation for Job Shop Scheduling

```
1: procedure CPJOBSHOPIMPLEMENTATION(Jobs, Machines, ProcessingTimes)
2:   Create CP model
3:    $horizon \leftarrow \sum$  all processing times
4:   Initialize empty maps taskStarts, taskEnds
5:   Initialize empty list allTasks
6:   for each job in Jobs do
7:     for each task in job do
8:       machine  $\leftarrow$  machine required by task
9:       duration  $\leftarrow$  ProcessingTimes[job, task]
10:      Create variable start  $\in [0, horizon]$ 
11:      Create variable end  $\in [0, horizon]$ 
12:      Add constraint: end = start + duration
13:      taskStarts[job, task]  $\leftarrow$  start
14:      taskEnds[job, task]  $\leftarrow$  end
15:      Add (job, task, machine, start, end) to allTasks
16:    end for
17:  end for ▷ Add precedence constraints within jobs

18:  for each job in Jobs do
19:    for task = 0 to length(job) - 2 do
20:      Add constraint: taskStarts[job, task + 1]  $\geq$  taskEnds[job, task]
21:    end for
22:  end for ▷ Add no-overlap constraints for machines

23:  Initialize map machineTasks grouping tasks by machine
24:  for each (job, task, machine, start, end) in allTasks do
25:    Add (start, end) to machineTasks[machine]
26:  end for
27:  for each machine in Machines do
28:    Add noOverlap constraint on all machineTasks[machine]
29:  end for ▷ Define and minimize makespan

30:  Create variable makespan  $\in [0, horizon]$ 
31:  Add constraint: makespan = max of all task end times
32:  Set objective: Minimize makespan
33:  Run CP solver
34:  return resulting schedule
35: end procedure
```

Algorithm 52 Earliest Deadline First (EDF)

```
1: procedure EDF(JobQueue)
2:   Initialize empty schedule
3:   time  $\leftarrow$  0
4:   while jobs remain to be scheduled do
5:     Update JobQueue with newly released jobs at time
6:     Remove completed jobs from JobQueue
7:     if JobQueue is not empty then
8:       Find job  $j^*$  with earliest deadline in JobQueue
9:       Execute job  $j^*$  for one time unit or until next event
10:      (Next event = job release, job completion, or preemption)
11:     else
12:       Advance time to next job release
13:     end if
14:     Update time accordingly
15:   end while
16:   return schedule
17: end procedure
```

- EDF can be implemented with $O(\log n)$ time complexity per scheduling decision using a priority queue
- For aperiodic task systems, EDF can be combined with acceptance tests to determine if a new job can be admitted without causing deadline misses

Theorem 128 (Optimality of EDF). For preemptive scheduling on a single processor with feasible job sets, EDF is optimal in terms of schedulability. Specifically:

- For periodic task systems: EDF can schedule any task set with utilization $U \leq 1$
- For aperiodic task systems: If there exists any algorithm that can schedule a given set of jobs without missing deadlines, then EDF can also schedule the same set without missing deadlines

Proof (sketch). The proof uses a contradiction technique and schedule transformation:

1. Assume a feasible schedule S exists but EDF fails
2. Consider the first time t where EDF makes a "wrong" decision compared to S
3. We can swap jobs in S to prioritize the earlier deadline job (as EDF would)
4. This transformation does not make S infeasible
5. By repeated transformations, S can be converted to an EDF schedule
6. Thus, if any feasible schedule exists, EDF produces a feasible schedule

For the utilization bound, consider periodic tasks. When $U \leq 1$:

- Total processing demand never exceeds available processor time
- EDF's optimal scheduling ensures no idle time is introduced if jobs are ready
- Therefore all deadlines are met

When $U > 1$, deadline misses are inevitable in the long run as demand exceeds capacity. $\square$

Lemma 110 (Rate Monotonic Scheduling (RMS)). RMS is a static-priority scheduling algorithm that assigns priorities based on job periods:

Algorithm 53 Rate Monotonic Scheduling (RMS)

```
1: procedure RMS(Tasks)
2:   // Preprocessing: Assign fixed priorities based on periods
3:   for each task  $\tau_i$  in Tasks do
4:      $priority_i \leftarrow 1/T_i$  ▷ Shorter period = higher priority
5:   end for
6:   Sort Tasks by decreasing priority
7:   // Online scheduling
8:   Initialize empty schedule
9:    $time \leftarrow 0$ 
10:  while scheduling continues do
11:    Update ready queue with newly released jobs at  $time$ 
12:    Remove completed jobs from ready queue
13:    if ready queue is not empty then
14:      Find highest priority job  $j^*$  in ready queue
15:      Execute job  $j^*$  for one time unit or until next event
16:    else
17:      Advance  $time$  to next job release
18:    end if
19:    Update  $time$  accordingly
20:  end while
21:  return schedule
22: end procedure
```

Remark 122. *RMS has several practical advantages:*

- *Static priorities simplify implementation in real-time operating systems*
- *No runtime overhead for priority recalculation*
- *Predictable behavior under overload (higher frequency tasks are prioritized)*
- *Works well with fixed-priority hardware schedulers*

Theorem 129 (Optimality and Utilization Bound of RMS). For fixed-priority scheduling of periodic tasks on a single processor:

- RMS is optimal among all fixed-priority scheduling algorithms
- A sufficient schedulability test for RMS is:

$$U \leq n(2^{1/n} - 1) \tag{89}$$

where n is the number of tasks

- This bound approaches $\ln(2) \approx 0.693$ as n approaches infinity

Proof (sketch). For optimality among fixed-priority schedulers:

1. Assume some task set τ is schedulable by a fixed-priority algorithm A but not by RMS
2. Consider the highest-priority task where A and RMS differ
3. We can show that this task's response time under RMS is no worse than under A
4. This contradicts the assumption that RMS fails while A succeeds

For the utilization bound:

1. Consider the critical instant scenario (all tasks released simultaneously)
2. For each task, calculate worst-case response time using recurrence relation
3. Through analysis of this equation, derive the utilization bound
4. The bound $n(2^{1/n} - 1)$ represents a threshold below which schedulability is guaranteed

The bound of 0.693 for large n means that RMS can guarantee schedulability for periodic task sets using up to approximately 69.3% of processor capacity, regardless of the number of tasks. $\square$

Definition 125 (Online Scheduling). Online scheduling addresses problems where:

- Jobs arrive over time, with characteristics unknown until arrival
- Scheduling decisions must be made without knowledge of future jobs
- Once a decision is made, it typically cannot be revoked (irrevocability)
- Performance is evaluated using competitive analysis against an omniscient optimal offline algorithm

The competitive ratio ρ of an online algorithm A is:

$$\rho = \sup_{\sigma} \frac{A(\sigma)}{OPT(\sigma)} \quad (90)$$

where σ is any job sequence, $A(\sigma)$ is the objective value obtained by algorithm A , and $OPT(\sigma)$ is the optimal offline solution.

Lemma 111 (Greedy Algorithm for Online $1|r_j|C_{\max}$). In the online version of $1|r_j|C_{\max}$ (minimizing makespan when jobs arrive over time), a job becomes known only at its release date r_j . The greedy algorithm never keeps the processor idle while a released job is waiting:

Algorithm 54 Greedy Online Scheduling

```

1: procedure GREEDYONLINE
2:   Initialize empty schedule
3:    $time \leftarrow 0$ 
4:   while not all jobs completed do
5:     if new job  $j$  arrives at  $time$  then
6:       Add  $j$  to the set of available jobs
7:     end if
8:     if processor is idle AND available jobs exist then
9:       Schedule an available job (for instance, the longest one)
10:    end if
11:    Update  $time$  to next event (job arrival or completion)
12:  end while
13:  return schedule
14: end procedure

```

Remark 123. *On a single machine, deliberately keeping the processor idle to wait for longer jobs never helps: by the next theorem, the greedy schedule is already optimal.*

Theorem 130 (Competitive Ratio of the Greedy Algorithm). The greedy algorithm is 1-competitive for the online scheduling problem $1|r_j|C_{\max}$; that is, it always produces an optimal schedule, although it does not know the future jobs.

Proof. Let C be the makespan of the greedy schedule, and let t_0 be the start of its last busy period: the processor is busy throughout $[t_0, C)$ and is idle just before t_0 (or $t_0 = 0$). The greedy algorithm idles only when no released job is waiting, so every job with $r_j < t_0$ has been completed before t_0 . Hence the jobs processed during $[t_0, C)$ are exactly the jobs with $r_j \geq t_0$, and

$$C = t_0 + \sum_{j: r_j \geq t_0} p_j.$$

In any schedule none of these jobs can start before t_0 , so every schedule has makespan at least $t_0 + \sum_{j: r_j \geq t_0} p_j = C$. Therefore $C = OPT$. $\square$

Remark 124. *Waiting can pay off when there are several machines. For the online version of $P|r_j|C_{\max}$ on identical parallel machines, Chen and Vestjens (1997) showed that the online LPT rule (whenever a machine becomes idle, start the longest available job) is $\frac{3}{2}$ -competitive and that this bound is tight, and that no deterministic online algorithm is better than 1.3473-competitive. For two machines, Noga and Seiden (2001) gave an algorithm that deliberately delays some jobs and is $\frac{5-\sqrt{5}}{2} \approx 1.382$ -competitive, the best possible ratio for deterministic algorithms on two machines.*

Remark 125 (Extensions and Limitations). *Online scheduling algorithms have several practical implications:*

- *Competitive ratios (such as the $\frac{3}{2}$ of online LPT) are worst-case guarantees; average-case performance is typically better*
- *For multiple machines, more complex algorithms are required, with competitive ratios depending on the number of machines*
- *When jobs have deadlines, competitive analysis often considers throughput (number of completed jobs) rather than makespan*
- *Randomized algorithms can achieve better expected competitive ratios against oblivious adversaries*
- *For periodic real-time tasks, EDF and RMS provide deterministic guarantees rather than competitive ratios*

The gap between online and offline scheduling highlights the value of predictive models in practical scheduling systems.

Definition 126 (Algorithm Selection Framework). Choosing an appropriate scheduling algorithm requires analyzing several dimensions:

- **Problem structure:** Machine environment, constraints, objective function
- **Instance size:** Number of jobs and machines
- **Solution quality:** Optimality vs. approximation requirements
- **Computational resources:** Available time and solvers
- **Implementation complexity:** Required expertise

Lemma 112 (Problem Structure-Based Selection). The structural characteristics of a scheduling problem dictate algorithm choice:

- **Small n , highly constrained:** CP, DP, Branch-and-Bound
- **Identical machines, $C_{\max}$:** LPT (simple), PTAS (for small m)
- **Unrelated machines:** Shmoys-Tardos LP rounding
- **Precedence constraints:** List scheduling with topological ordering

- **Single machine, tardiness:** Lawler's algorithm (small), Local search (larger)
- **Job shop:** CP with global constraints, Shifting bottleneck
- **Online/real-time:** EDF (utilization ≤ 1), RMS (fixed priorities)

Remark 126. *When problems combine multiple characteristics, decomposition methods can separate decisions, applying different algorithms to each sub-problem.*

Theorem 131 (Quality-Speed Trade-off Hierarchy). Scheduling algorithms form a hierarchy based on solution quality vs. speed:

- **Exact methods** (DP, B&B, MIP)
 - Guarantee: Optimal
 - Speed: Slowest
 - Use when: Optimality is required and instances are small
- **Approximation schemes** (PTAS/FPTAS)
 - Guarantee: $(1 + \epsilon)$ -optimal
 - Speed: Slow
 - Use when: Near-optimality with guarantees is needed
- **Constant-factor approximations**
 - Guarantee: ρ -approximation
 - Speed: Medium
 - Use when: Bounded quality is sufficient
- **List scheduling heuristics**
 - Guarantee: Bounded worst-case for some problems
 - Speed: Fast
 - Use when: Quick solutions are needed
- **Meta-heuristics**
 - Guarantee: No theoretical guarantees
 - Speed: Medium
 - Use when: High-quality solutions for large instances
- **Dispatching rules**
 - Guarantee: None
 - Speed: Fastest
 - Use when: Real-time decisions are required

Proof. This hierarchy follows from complexity theory and empirical evidence. Exact methods guarantee optimality but, on NP-hard problems, cannot run in polynomial time unless $P = NP$. Approximation schemes provide worst-case bounds at the cost of higher runtime. Heuristics sacrifice guarantees for speed. $\square$

Lemma 113 (Instance Size-Based Selection). The size of the scheduling instance influences algorithm selection:

Remark 127. *These thresholds are approximate and depend on available computational resources, required solution time, implementation efficiency, and problem-specific structure.*

Algorithm 55 Algorithm Selection Based on Instance Size

```
1: procedure SELECTALGORITHM(Problem, n, m)           ▷ n = number of jobs, m = number of machines
2:   if  $n \leq 50$  AND highly constrained then
3:     return Exact methods (CP, DP, MIP)
4:   else if  $n \leq 200$  AND m is small AND identical machines then
5:     return PTAS or approximation algorithm
6:   else if  $n \leq 1000$  AND moderately constrained then
7:     return Hybrid CP-MIP or meta-heuristic
8:   else if  $n > 1000$  OR real-time requirements then
9:     return List scheduling or dispatching rules
10:  else
11:    return Decomposition with problem-specific heuristics
12:  end if
13: end procedure
```

Theorem 132 (Toolkit-Based Selection Strategy). Available implementation resources influence algorithm selection:

- **With MIP solver** (e.g., Gurobi, CPLEX)
 - Formulate as MIP with problem-specific cuts
 - Consider decomposition (Benders, column generation) for large instances
 - Best for: Problems with linear constraints and objectives
- **With CP solver** (e.g., OR-Tools, CP Optimizer)
 - Model with global constraints
 - Consider hybrid CP-MIP for optimality proof
 - Best for: Highly constrained problems with logical relationships
- **With programming expertise**
 - Implement metaheuristics (GA, SA)
 - Customize neighborhood structures for local search
 - Best for: Complex problems requiring domain-specific knowledge
- **With limited resources**
 - Simple list scheduling with priority rules
 - Randomized greedy algorithms with restarts
 - Best for: When quick solutions are needed with minimal development

Proof. Based on computational studies across scheduling domains: commercial solvers often outperform custom implementations for small/medium instances; hybrid approaches leveraging multiple technologies typically outperform single-method approaches; and problem-specific metaheuristics can achieve near-optimal solutions when properly designed. $\square$

Remark 128 (Decision Framework for Practitioners). *A practical decision framework for scheduling algorithm selection:*

1. First, characterize your problem:

- *Machine environment (single, parallel, unrelated)*
- *Objective function (makespan, tardiness)*

- Constraints (precedence, release dates)
- Instance size (jobs, machines)
- Time sensitivity (batch vs. real-time)

2. **Then, evaluate solution requirements:**

- Is optimality required or is approximation sufficient?
- Are theoretical guarantees important?
- What is the acceptable computation time?
- Will the solution need explanation?

3. **Finally, assess available resources:**

- Available solvers
- Programming expertise
- Computational resources
- Development time constraints

Based on these assessments, the appropriate algorithm selection typically becomes clear. When in doubt, prototype multiple approaches on representative instances.

Lemma 114 (Example Selection Scenarios). To illustrate the selection process, consider these common scheduling scenarios:

1. **Production scheduling (50 jobs, 5 identical machines):**

- Makespan objective: LPT is quick and effective, or PTAS for near-optimality
- Tardiness objective: MIP for optimality, or local search and metaheuristics for larger instances

2. **Employee shift scheduling with complex constraints:**

- CP excels due to global constraints for expressing rules
- Column generation works when shifts can be pre-enumerated
- Hybrid CP-MIP leverages strengths of both approaches

3. **Online appointment scheduling:**

- Competitive online algorithms with bounded worst-case performance
- Stochastic optimization if service time distributions are known
- Simple heuristics with simulation validation

4. **Real-time embedded system tasks:**

- EDF for preemptive scheduling with full utilization
- RMS for static priorities with robustness under overload
- Specialized algorithms for mixed-criticality systems

Remark 129 (Empirical Performance Considerations). When selecting algorithms, consider these empirical observations:

- Simple heuristics like LPT often achieve within 5-10% of optimal for makespan minimization
- Local search with problem-specific neighborhoods can find solutions within 1-2% of optimal
- Time-indexed MIP formulations typically outperform sequence-based formulations
- Genetic algorithms with specialized crossover work well for job shop scheduling
- For real-world applications, robustness to uncertainty often matters more than worst-case guarantees
- Algorithm portfolios with different strengths can provide both quality and efficiency

Scheduling Problem

Suppose we have students $s_1, s_2, \dots, s_m$, each has to write a set $E_1, E_2, \dots, E_m$ of exams.

What is the minimum number of time slots to schedule the exams so that no student has to write two exams at the same time?

Example

Suppose we have students

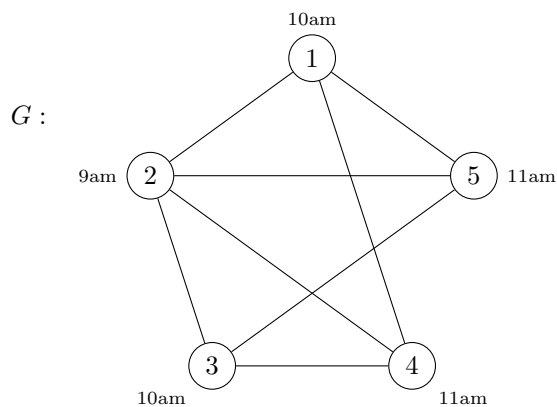
$$s_1, s_2, s_3, s_4$$

and

s_1 has to write exams 1 and 4
 s_2 has to write exams 1, 2 and 5
 s_3 has 2, 3, 4
 s_4 has 3, 5

Form a graph G where $V(G)$ is the set of exams and $\{i, j\}$ is an edge if some student has to write both exams i and j .

In the example,



This is the
 schedule graph.
 $\chi(G) = 3$: the triangle
 1, 2, 4 needs 3 colors,
 and 3 time slots suffice.

Figure 148: The minimum number of time slots equals the chromatic number of the schedule graph.

$$3 \text{ time slots } \begin{cases} \text{Exam 2 at 9am} \\ \text{Exams 1 and 3 at 10am} \\ \text{Exams 4 and 5 at 11am} \end{cases}$$

8.10 Vertex and Edge-Coloring Exercises

8.10.1 Easy Questions

1. Define the chromatic number $\chi(G)$ of a graph G . What is the chromatic number of a complete graph K_n ?
2. State König's theorem for bipartite graphs. What does it say about the relationship between vertex cover and maximum matching?
3. What is the edge chromatic number (chromatic index) $\chi'(G)$ of a graph G ? How does it relate to the maximum degree $\Delta(G)$?
4. Find the chromatic number of the cycle C_5 . Explain your reasoning.
5. Prove that any bipartite graph has chromatic number at most 2.
6. What is the chromatic number of the complete bipartite graph $K_{m,n}$ where $m, n \geq 1$?
7. State Vizing's theorem. What are the two possible values for the chromatic index of any simple graph?
8. Find the chromatic index of the complete graph K_4 . Show a proper edge coloring.
9. Explain why the chromatic number of any graph G satisfies $\chi(G) \geq \omega(G)$, where $\omega(G)$ is the clique number.
10. What is a degenerate graph? Give an example of a 2-degenerate graph.

8.10.2 Medium Questions

11. Prove that if G is a graph with maximum degree Δ , then $\chi(G) \leq \Delta + 1$.
12. Show that the chromatic number of the Petersen graph is 3 by providing an explicit 3-coloring.
13. Prove König's theorem: In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.
14. Let G be a graph with chromatic number k . Prove that G contains a subgraph with minimum degree at least $k - 1$.
15. Find the chromatic index of the complete bipartite graph $K_{3,4}$. Is this graph Class 1 or Class 2 according to Vizing's classification?
16. Prove that if G is a bipartite graph with maximum degree Δ , then $\chi'(G) = \Delta$.
17. Show that the chromatic number of the wheel graph W_n (a cycle C_n with an additional central vertex connected to all vertices of the cycle) depends on whether n is odd or even.
18. Prove that any planar graph has chromatic number at most 4 (Four Color Theorem - you may state this without proof, but explain its significance).
19. Let G be a k -degenerate graph. Prove that $\chi(G) \leq k + 1$ using the degeneracy ordering.
20. Describe the greedy coloring algorithm and analyze its performance. Under what conditions does it achieve the optimal coloring?

8.10.3 Hard Questions

21. Prove or disprove: If G is a graph where every subgraph has a vertex of degree at most k , then $\chi(G) \leq k + 1$.
22. Let G be a graph with n vertices and chromatic number $\chi(G) = k$. Prove that G has at least $\binom{k}{2}$ edges.
23. Prove Vizing's theorem: For any simple graph G with maximum degree Δ , either $\chi'(G) = \Delta$ or $\chi'(G) = \Delta + 1$.
24. Show that determining whether a graph is 3-colorable is NP-complete, while determining whether a graph is 2-colorable is polynomial-time solvable.
25. Let G be a graph with minimum degree $\delta \geq 2$ and girth g (length of shortest cycle). Prove that G has at least $1 + \delta + \delta(\delta - 1) + \dots + \delta(\delta - 1)^{\lfloor (g-1)/2 \rfloor - 1}$ vertices. (For even g this can be improved to $2(1 + (\delta - 1) + \dots + (\delta - 1)^{g/2 - 1})$; compare Theorem 158.)
26. Prove Shannon's theorem: For any loopless multigraph G , $\chi'(G) \leq \lfloor \frac{3\Delta}{2} \rfloor$ where Δ is the maximum degree.
27. Let G be a graph such that every subgraph of G has degeneracy at most k . Prove that $\chi(G) \leq k + 1$ and that this bound is tight.
28. Design a polynomial-time algorithm to find the chromatic index of a bipartite graph. Analyze its time complexity.
29. Prove that if G is a graph with n vertices, $\chi(G) = k$ and $\alpha(G) = \ell$ (independence number), then $n \leq k \cdot \ell$.
30. Investigate the relationship between the chromatic number and the fractional chromatic number. Show that they can differ by an arbitrarily large factor.

8.10.4 Diagram Questions

31. Draw the complete graph K_5 and provide an optimal vertex coloring. Explain why this is the minimum number of colors needed.
32. Illustrate König's theorem by drawing a bipartite graph, finding a maximum matching, and showing the corresponding minimum vertex cover.
33. Draw the Petersen graph and show that it requires exactly 3 colors. Mark the color classes clearly.
34. Create a graph that is Class 2 according to Vizing's classification (i.e., $\chi'(G) = \Delta(G) + 1$). Show why $\Delta(G)$ colors are insufficient.
35. Draw the wheel graph W_5 and W_6 . Show optimal colorings for both and explain why their chromatic numbers differ.
36. Illustrate the greedy coloring algorithm on a specific graph where it does NOT produce an optimal coloring. Show both the greedy result and the optimal coloring.
37. Draw a 3-degenerate graph with 8 vertices. Show the degeneracy ordering and use it to find a 4-coloring.
38. Create a bipartite graph and show its edge coloring using exactly Δ colors, illustrating the result that bipartite graphs are always Class 1.
39. Draw two different graphs, both with maximum degree 3, where one is Class 1 and the other is Class 2. Show the edge colorings for both.
40. Illustrate Shannon's bound by drawing a multigraph (for example, a Shannon multigraph) whose chromatic index is close to $\frac{3\Delta}{2}$. Show the edge coloring that achieves this bound.

9 Planar Graphs

9.1 Definition and Basic Properties

Definition 127. A graph is *planar* if it can be drawn in the plane without crossing edges.

Example 40. The complete graph K_4 is planar. Although it may initially appear to have crossing edges when drawn as a square with diagonals, it can be redrawn without crossings.

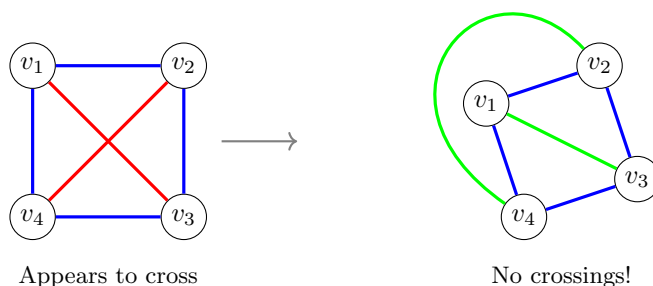


Figure 149: K_4 drawn in non-planar and planar representations

K_4 is planar.

9.2 Faces of Planar Graphs

Definition 128. If G is a planar graph drawn in the plane without crossings, then the *faces* of G are the maximal connected regions of the plane that are disjoint from the drawing of G .

Definition 129. The *degree* of a face f , denoted $\deg(f)$, is the length of the shortest closed walk containing all points on the boundary of f . (If the boundary of f is disconnected, which can happen only when G is disconnected, then $\deg(f)$ is the sum of these lengths over the connected pieces of the boundary.)

Example 41. Consider a triangle abc together with a vertex d inside it joined to b and c :

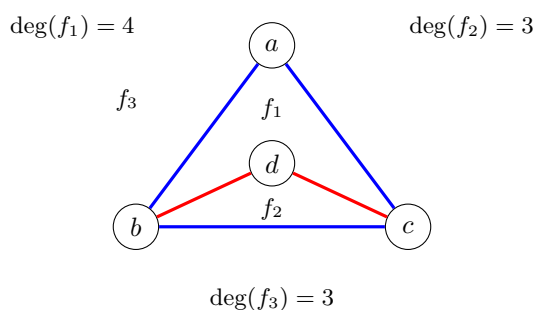


Figure 150: Triangle with an additional path bdc , showing three faces

The face f_1 is bounded by the closed walk $abdc a$, so $\deg(f_1) = 4$; the face f_2 is bounded by the triangle $bdc b$, so $\deg(f_2) = 3$; and the outer face f_3 is bounded by the triangle $abc a$, so $\deg(f_3) = 3$. Each edge is counted exactly twice: bd and dc once for f_1 and once for f_2 ; ab and ac once for f_1 and once for the outer face f_3 ; and bc once for f_2 and once for f_3 .

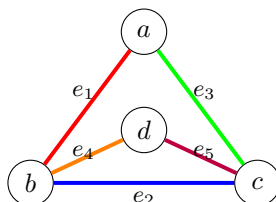
9.3 Fundamental Theorems

Lemma 115 (Handshaking Lemma for Faces). For any planar graph G ,

$$\sum_{f \in F(G)} \deg(f) = 2|E(G)|$$

where $F(G)$ is the set of faces of G .

Example 42. Verification with the triangle example above:



$$\sum \deg(f) = 4 + 3 + 3 = 10 = 2 \times 5 = 2|E|$$

Figure 151: Verification of Handshaking Lemma for Faces

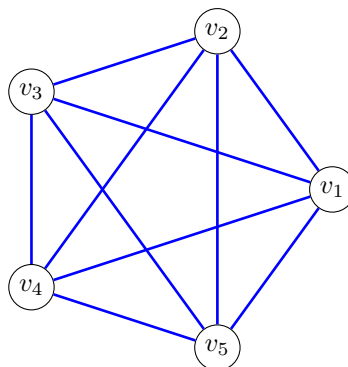
Theorem 133 (Euler's Formula). If G is a connected planar graph, then

$$|V(G)| - |E(G)| + |F(G)| = 2$$

Example 43. Verification with our triangle example: $|V| = 4$, $|E| = 5$, $|F| = 3$, so $4 - 5 + 3 = 2$ (correct)

9.4 Applications: Proving Non-Planarity

Example 44. K_5 is not planar.



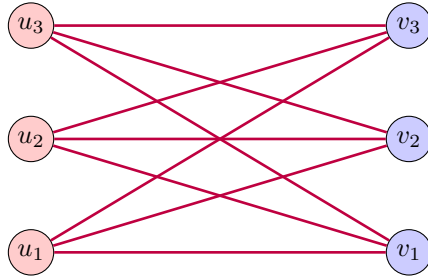
K_5 cannot be drawn without crossings
 $|V| = 5$, $|E| = 10$, but if planar: $|E| \leq 3(|V| - 2) = 9$

Figure 152: K_5 showing inevitable edge crossings

Example 45. $K_{3,3}$ is not planar.

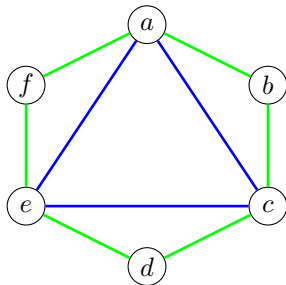
Example 46. Planar vs Non-planar comparison:

Remark 130. The results that K_5 and $K_{3,3}$ are not planar will be on the exam.

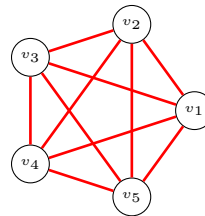


$K_{3,3}$ cannot be drawn without crossings
 $|V| = 6, |E| = 9$, but if planar: $|E| \leq 2(|V| - 2) = 8$

Figure 153: $K_{3,3}$ complete bipartite graph showing crossings



Planar: $|V| = 6, |E| = 9$
 Satisfies: $9 \leq 3(6 - 2) = 12$ (valid)



Non-planar: K_5
 Violates: $10 > 3(5 - 2) = 9$ (invalid)

Figure 154: Comparison of planar and non-planar graphs

9.5 Introduction and Formal Definitions of Art Gallery Problems

Definition 130 (Simple Polygon). A **simple polygon** P is the closed, bounded region of $\mathbb{R}^2$ enclosed by a Jordan curve ∂P (the *boundary* of P) consisting of a finite sequence of line segments $e_1, e_2, \dots, e_n$ called edges, where:

- Each edge e_i connects vertices v_i and v_{i+1} (with $v_{n+1} = v_1$)
- No two non-consecutive edges intersect
- P consists of the curve ∂P together with the bounded region it encloses; in particular $\partial P \subseteq P$

Definition 131 (Visibility). Two points $p, q \in P$ are **visible** to each other if the line segment $\overline{pq}$ lies entirely within P . We write $p \sim q$ if p and q are visible to each other.

Definition 132 (Guard Set). A set $G \subseteq P$ is a **guard set** for polygon P if for every point $p \in P$, there exists a guard $g \in G$ such that $p \sim g$.

Definition 133 (The Art Gallery Problem). Given a simple polygon P with n vertices, determine:

1. $g(P)$: The minimum number of guards needed to guard P
2. $g(n)$: The maximum of $g(P)$ over all simple polygons with n vertices

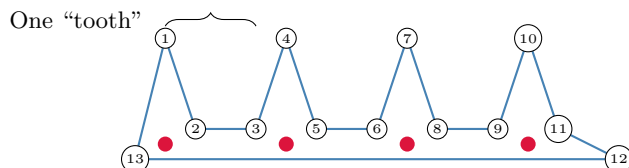
Theorem 134 (Chvátal's Art Gallery Theorem, 1975). For every $n \geq 3$,

$$g(n) = \left\lfloor \frac{n}{3} \right\rfloor$$

9.6 Construction and Visualization

The comb polygon demonstrates that the bound $\lfloor n/3 \rfloor$ is tight. It resembles the triangular teeth of Toblerone chocolate.

Comb Polygon with $n = 13$ vertices (requires 4 guards)



Definition 134 (Comb Polygon). A **comb polygon** C_k with $n = 3k + 1$ vertices is constructed as:

- k triangular “teeth” protruding upward
- A horizontal base connecting all teeth
- Each tooth requires its own guard due to visibility constraints

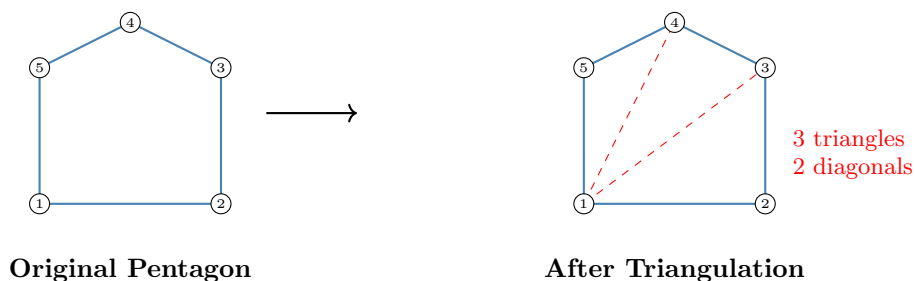
Lemma 116 (Comb Lower Bound). The comb polygon C_k with $n = 3k + 1$ vertices requires exactly $k = \lfloor n/3 \rfloor$ guards.

Proof. Let $t_1, \dots, t_k$ be the peaks of the teeth. A segment from t_i to a point below the i -th tooth must pass through the opening at the base of that tooth, so every point of C_k that sees t_i lies in the i -th tooth or in the part of the base between the two lines through t_i and the two base vertices of the i -th tooth. Because the teeth are narrow and separated from each other, these regions are pairwise disjoint (in the figure they meet the bottom edge in the disjoint intervals $[0, \frac{7}{6}]$, $[\frac{11}{6}, \frac{19}{6}]$, $[\frac{23}{6}, \frac{31}{6}]$, $[\frac{35}{6}, \frac{43}{6}]$). Hence no point sees two peaks, and every guard set has at least k guards. Conversely, $k = \lfloor n/3 \rfloor$ guards suffice by Chvátal's theorem; in the figure, a guard on the base directly below each peak sees its whole tooth and the entire (convex) base. Therefore C_k needs exactly k guards. $\square$

9.7 Triangulation

Definition 135 (Polygon Triangulation). A **triangulation** of a simple polygon P is a maximal set of non-crossing diagonals that partition P into triangles.

Theorem 135 (Triangulation Existence). Every simple polygon with $n \geq 3$ vertices can be triangulated into exactly $n - 2$ triangles using $n - 3$ diagonals.



Algorithm 56 Ear Clipping Triangulation

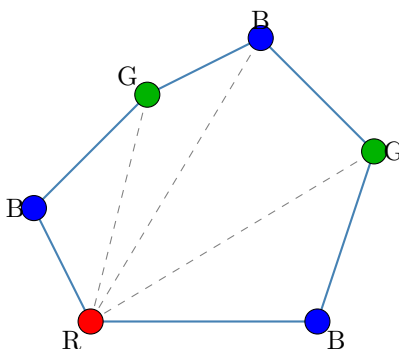
Require: Simple polygon P with vertices $v_1, \dots, v_n$

Ensure: Triangulation of P

- 1: **while** $n > 3$ **do**
 - 2: Find an ear vertex v_i (convex vertex with no vertices inside triangle $v_{i-1}v_iv_{i+1}$)
 - 3: Add diagonal $v_{i-1}v_{i+1}$
 - 4: Remove vertex v_i
 - 5: $n \leftarrow n - 1$
 - 6: **end while**
 - 7: **Complexity:** $O(n^2)$ time, $O(n)$ space
-

9.8 Three-Coloring

Theorem 136 (3-Colorability of Triangulated Polygons). Every triangulated simple polygon has a proper 3-coloring of its vertices such that no triangle has vertices of the same color.



3-Coloring Example

Each triangle has exactly one vertex of each color

Proof of 3-Colorability. By induction on the number of triangles:

- **Base case:** A single triangle can be 3-colored

- **Inductive step:** Remove an ear, 3-color the remaining polygon, then color the removed vertex with the third color not used by its two neighbors

□

9.9 The Complete Guard Placement Algorithm

Algorithm 57 Art Gallery Guard Placement

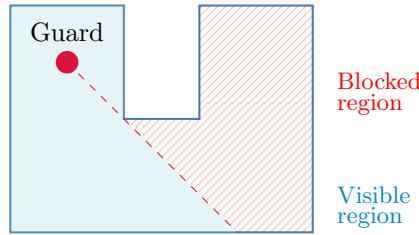
Require: Simple polygon P with n vertices

Ensure: Guard set S with $|S| \leq \lfloor n/3 \rfloor$

- 1: Triangulate P into $n - 2$ triangles
 - 2: 3-color the vertices: $V = R \cup B \cup G$ (Red, Blue, Green)
 - 3: $S \leftarrow$ a smallest of the classes R, B, G
 - 4: **return** Guard set S
-

Theorem 137 (Correctness of Algorithm). The algorithm produces a valid guard set with at most $\lfloor n/3 \rfloor$ guards.

Proof. Since we have a 3-coloring with color classes R, B, G , we have $|R| + |B| + |G| = n$. By the pigeonhole principle, $\min\{|R|, |B|, |G|\} \leq n/3$. The chosen color class guards all triangles since each triangle contains exactly one vertex of each color. □

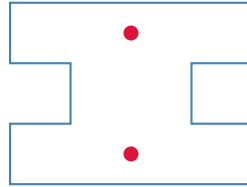


Visibility Region from a Guard

9.10 Orthogonal Polygons

Theorem 138 (Kahn-Klawe-Kleitman, 1983). For orthogonal polygons (edges parallel to coordinate axes) with n vertices:

$$g_{\text{orth}}(n) = \left\lfloor \frac{n}{4} \right\rfloor$$



Orthogonal Polygon (12 vertices, 2 guards)

9.11 Mobile Guards

Theorem 139 (O'Rourke, 1983). For mobile guards that can patrol along an edge or a diagonal of P (or, more generally, along any line segment contained in P), and $n \geq 4$:

$$g_{\text{mobile}}(n) = \left\lfloor \frac{n}{4} \right\rfloor$$

Remark 131. For guards that may patrol only along edges of P , $\lfloor n/4 \rfloor$ guards do not always suffice: there is a polygon with $n = 7$ vertices that needs 2 edge guards, while $\lfloor 7/4 \rfloor = 1$. Toussaint conjectured that $\lfloor n/4 \rfloor$ edge guards suffice except for a few small polygons. Shermer proved that $\lfloor 3n/10 \rfloor$ edge guards always suffice, except for $n = 3, 6, 13$, where one more may be needed.

9.12 Monotone Polygons

Definition 136 (Monotone Polygon). A polygon P is **y -monotone** if every horizontal line meets P in a connected set (a segment, a single point, or the empty set).

Theorem 140 (Aggarwal, 1984). For y -monotone polygons with $n \geq 3$ vertices:

$$g_{\text{monotone}}(n) = \left\lfloor \frac{n}{3} \right\rfloor$$

Thus monotonicity does not help in the worst case: the upper bound is Chvátal's theorem, and the comb polygon, turned on its side so that its teeth point horizontally, is y -monotone. (In terms of the number r of reflex vertices, $\lfloor r/2 \rfloor + 1$ guards are always sufficient and sometimes necessary for monotone polygons.)

Definition 137 (Art Gallery Decision Problem). INPUT: Simple polygon P and integer k
QUESTION: Is $g(P) \leq k$?

Theorem 141 (Lee and Lin, 1986). The Art Gallery Decision Problem is NP-hard, even for simple polygons. (Lee and Lin proved this for vertex guards, for which the problem is NP-complete; the extension to point guards is due to Aggarwal.)

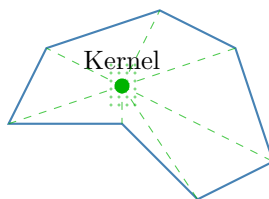
Remark 132. For point guards, as in our definition of $g(P)$, the decision problem is not known to lie in NP: Abrahamsen, Adamaszek, and Miltzow (2018) proved that it is $\exists\mathbb{R}$ -complete, so it lies in NP only if $\text{NP} = \exists\mathbb{R}$.

Theorem 142 (Approximation Algorithm, Ghosh 1987). For the vertex-guard version of the Art Gallery Problem (guards restricted to vertices of P), there exists an $O(\log n)$ -approximation algorithm using a greedy set cover approach.

Definition 138 (Star-Shaped Polygon). A polygon P is **star-shaped** if there exists a point $p \in P$ such that p can see all of P .

Definition 139 (Kernel). The **kernel** of polygon P is:

$$\text{Kernel}(P) = \{p \in P : \forall q \in P, p \sim q\}$$



Star-Shaped Polygon with Kernel

Theorem 143. A polygon is star-shaped if and only if its kernel is non-empty. Star-shaped polygons require only 1 guard.

9.13 Fortress Problem

Guard the **exterior** of a polygon. With guards placed at vertices of the polygon (O'Rourke and Wood),

$$g_{\text{fortress}}(n) = \left\lceil \frac{n}{2} \right\rceil;$$

if guards may be placed at arbitrary points outside the polygon or on its boundary, then $\lceil n/3 \rceil$ guards are always sufficient and sometimes necessary (Aggarwal and O'Rourke).

9.14 Prison Yard Problem

Guards placed at vertices must simultaneously see the interior and the exterior of the polygon. Füredi and Kleitman (1994) proved

$$g_{\text{prison}}(n) = \left\lceil \frac{n}{2} \right\rceil$$

(a convex n -gon needs this many, and $\lfloor n/2 \rfloor$ vertex guards suffice for every non-convex polygon).

Theorem 144 (Negative Result for 3D, Seidel). There exist polyhedra with n vertices that require $\Omega(n^{3/2})$ guards; moreover, for these polyhedra even a guard at every vertex does not see the whole interior. Hence no bound that is a constant fraction of n suffices in 3D.

This demonstrates that the art gallery problem becomes much harder in higher dimensions.

Polygon Type	Guard Bound	Reference
Simple polygon	$\lfloor n/3 \rfloor$	Chvátal (1975)
Orthogonal polygon	$\lfloor n/4 \rfloor$	Kahn-Klawe-Kleitman (1983)
Mobile (diagonal) guards	$\lfloor n/4 \rfloor$	O'Rourke (1983)
y -monotone polygon	$\lfloor n/3 \rfloor$	Aggarwal (1984)
Fortress problem (vertex guards)	$\lfloor n/2 \rfloor$	O'Rourke–Wood (1983)
Star-shaped polygon	1	

Problem 5. Prove that a convex polygon with n vertices requires only 1 guard.

Problem 6. Show that the visibility graph of a convex polygon is the complete graph K_n .

Problem 7. Construct a polygon with 15 vertices that requires exactly 5 guards.

Problem 8. Prove that every quadrilateral requires at most 1 guard.

9.15 Open Problems

1. Better approximation algorithms: Ghosh's $O(\log n)$ ratio for vertex guards has been improved to $O(\log \log \text{OPT})$ (King and Kirkpatrick, 2011); for point guards an $O(\log \text{OPT})$ ratio is known under general-position assumptions (Bonnet and Miltzow, 2017); Ghosh conjectured that a constant-factor approximation exists for vertex guards
2. Exact algorithms for special polygon classes
3. Dynamic art gallery problems with moving targets
4. Efficient algorithms for 3D polyhedron guarding

9.16 Planar Graphs and Art Gallery Problems

9.16.1 Easy Questions

1. Define a planar graph. What is the difference between a planar graph and a plane graph?
2. State Euler's formula for connected planar graphs. What does each variable represent?
3. What is the degree of a face in a planar graph? How is it calculated?
4. Prove the handshaking lemma for faces: For any planar graph G , $\sum_{f \in F(G)} \deg(f) = 2|E(G)|$.
5. Find the number of faces in a connected planar graph with 8 vertices and 12 edges using Euler's formula.
6. Show that the complete graph K_4 is planar by drawing it without edge crossings.
7. Define the chromatic number $\chi(G)$ of a graph G . What is $\chi(K_4)$?
8. What is a guard set in the context of the art gallery problem? Define visibility between two points.
9. State Chvátal's Art Gallery Theorem. What does it tell us about the number of guards needed?
10. What is a star-shaped polygon? How many guards does it require and why?

9.16.2 Medium Questions

11. Prove that K_5 is not planar using Euler's formula and the fact that every face of a connected simple planar graph with at least three vertices has degree at least 3.
12. Show that the complete bipartite graph $K_{3,3}$ is not planar by counting faces and applying appropriate bounds.
13. For a simple planar graph with $n \geq 3$ vertices, prove that $|E| \leq 3|V| - 6$.
14. Construct a comb polygon with 13 vertices and show that it requires exactly 4 guards, demonstrating that the bound $\lfloor n/3 \rfloor$ is tight.
15. Prove that every simple polygon with n vertices can be triangulated into exactly $n - 2$ triangles using $n - 3$ diagonals.
16. Show that every triangulated simple polygon has a proper 3-coloring of its vertices such that no triangle has vertices of the same color.
17. Describe the algorithm for solving the art gallery problem using triangulation and 3-coloring. Prove its correctness.
18. For triangle-free simple planar graphs with $|V| \geq 3$, prove the stronger bound $|E| \leq 2|V| - 4$.
19. What is the chromatic number of a planar graph? State the Four Color Theorem and explain its significance for art gallery problems.
20. Define the kernel of a polygon. Prove that a polygon is star-shaped if and only if its kernel is non-empty.

9.16.3 Hard Questions

21. Prove Kuratowski's theorem (statement only): A graph is planar if and only if it contains no subdivision of K_5 or $K_{3,3}$ as a subgraph.
22. Let G be a planar graph with n vertices where every face has degree at least k . Derive a bound on the number of edges in terms of n and k .
23. Prove that the art gallery decision problem (determining if k guards suffice for a given polygon) is NP-hard. (Hint: begin with vertex guards, following Lee and Lin.)
24. For orthogonal polygons (edges parallel to coordinate axes), state and prove the bound $g_{\text{orth}}(n) = \lfloor n/4 \rfloor$.
25. Show that for y -monotone polygons with n vertices, $g_{\text{monotone}}(n) = \lfloor n/3 \rfloor$. (Hint: the comb polygon can be made monotone.)
26. Prove that in 3D, there exist polyhedra with n vertices that require $\Omega(n^{3/2})$ guards, showing that no constant fraction suffices.
27. Design and analyze a greedy approximation algorithm for the art gallery problem. What approximation ratio can you achieve?
28. For the fortress problem (guarding the exterior of a polygon with guards at its vertices), prove that $g_{\text{fortress}}(n) = \lceil n/2 \rceil$.
29. Investigate mobile guards that can patrol along edges or diagonals of the polygon. Prove that $g_{\text{mobile}}(n) = \lfloor n/4 \rfloor$ for $n \geq 4$.
30. Analyze the complexity of computing the exact chromatic number of planar graphs and its relationship to guard placement problems.

9.16.4 Construction and Diagram Questions

31. Draw two different planar embeddings of K_4 . Label all vertices, edges, and faces, and verify Euler's formula for each.
32. Illustrate the comb polygon construction. Draw a comb with 16 vertices, place the optimal number of guards, and show why fewer guards are insufficient.
33. Create a planar graph with 7 vertices and exactly 15 edges. Verify that this achieves the maximum possible number of edges, $3 \cdot 7 - 6 = 15$, for a simple planar graph with 7 vertices.
34. Draw the wheel graph W_6 (cycle C_6 with central vertex). Determine if it's planar and find its chromatic number.
35. Construct a simple polygon and demonstrate the triangulation process step by step. Show the 3-coloring and resulting guard placement.
36. Illustrate why K_5 cannot be drawn without crossings by attempting different embeddings and showing where crossings are unavoidable.
37. Draw an orthogonal polygon with 12 vertices that requires exactly 3 guards. Show the guard placement and prove optimality.
38. Create a y -monotone polygon with 10 vertices. Find a minimum guard set and verify that it uses at most $\lfloor 10/3 \rfloor = 3$ guards, as guaranteed for monotone polygons.
39. Design a star-shaped polygon and clearly indicate its kernel region. Show that one guard placed in the kernel can see the entire polygon.
40. Illustrate the fortress problem by drawing a polygon and showing how exterior guards must be placed to see the entire boundary from outside.

9.16.5 Proof-Based Questions

41. Prove that if G is a connected planar graph with $n \geq 3$ vertices and m edges, then $m \leq 3n - 6$.
42. Show that every planar graph contains a vertex of degree at most 5. Use this to prove that every planar graph is 6-colorable.
43. Prove that the complete graph K_n is planar if and only if $n \leq 4$.
44. Demonstrate that the complete bipartite graph $K_{m,n}$ is planar if and only if $\min(m, n) \leq 2$.
45. For a simple polygon P , prove that any triangulation results in the same number of triangles and diagonals, regardless of the triangulation method chosen.
46. Show that the greedy algorithm for 3-coloring a triangulated polygon always produces a valid coloring and that one color class has size at most $\lfloor n/3 \rfloor$.
47. Prove that every quadrilateral (4-sided polygon) can be guarded by exactly one guard, regardless of its shape.
48. Show that for convex polygons with n vertices, exactly one guard suffices and can be placed anywhere inside the polygon.
49. Prove that the visibility graph of any set of points in a convex polygon is the complete graph.
50. Show that if a set S of vertices guards a polygon P , then S is a dominating set of the visibility graph of P . Give examples showing that the converse fails and that the minimum number of guards can differ from the minimum size of a dominating set of the visibility graph.

9.16.6 Advanced Applications

51. Design an algorithm to determine if a given graph contains a K_5 or $K_{3,3}$ subdivision. Analyze its time complexity.
52. Investigate the relationship between the thickness of a graph (minimum number of planar graphs needed to cover all edges) and its planarity.
53. Explore the connection between graph coloring and art gallery problems. How does the chromatic number of the visibility graph relate to guard placement?
54. Study the dual graph of a planar graph. How do the properties of the original graph relate to its dual?
55. Analyze the art gallery problem for polygons with holes. How does the presence of holes affect the guard requirement bounds?
56. Examine rectilinear art gallery problems where guards and visibility are restricted to horizontal and vertical lines.
57. Investigate approximation algorithms for non-planar graphs. What bounds can be achieved for graphs embedded on surfaces of higher genus?
58. Study dynamic art gallery problems where the polygon can change over time. Design algorithms to maintain guard sets efficiently.
59. Explore probabilistic methods in graph coloring and their applications to randomized guard placement strategies.
60. Analyze the parameterized complexity of art gallery problems. What parameters make the problem tractable?

10 Network Flows and the Max-Flow Min-Cut Theorem

This chapter develops, from first principles, every object and technique needed to state and prove the Max-Flow Min-Cut Theorem of Ford and Fulkerson. Up to this point the book has treated (mostly) undirected graphs; flows live most naturally on directed graphs with non-negative edge capacities, so we begin by introducing the setting carefully.

10.1 Directed Graphs and Capacities

Definition 140 (Directed graph). A **directed graph** (or *digraph*) is a pair $D = (V, E)$ where V is a finite set of *vertices* and $E \subseteq V \times V$ is a set of ordered pairs called *arcs* (or directed edges). If $e = (u, v) \in E$, we call u the *tail* and v the *head* of e , and we write $u \rightarrow v$.

Definition 141 (In- and out-neighborhoods). For $v \in V$,

$$N^+(v) = \{w \in V : (v, w) \in E\}, \quad N^-(v) = \{u \in V : (u, v) \in E\}.$$

The *out-degree* and *in-degree* of v are $d^+(v) = |N^+(v)|$ and $d^-(v) = |N^-(v)|$.

Definition 142 (Flow network). A **flow network** is a tuple $N = (V, E, s, t, c)$ where (V, E) is a finite directed graph, $s, t \in V$ are two distinguished vertices called the *source* and *sink* (with $s \neq t$), and $c : E \rightarrow \mathbb{R}_{\geq 0}$ is a *capacity* function. We extend c to all pairs by setting $c(u, v) = 0$ whenever $(u, v) \notin E$.

Remark 133. *It is convenient to assume that N contains no pair of antiparallel arcs (i.e. both (u, v) and (v, u)). Any network can be converted into one with this property by subdividing one of the two antiparallel arcs with a new intermediate vertex whose incoming and outgoing arcs keep the original capacity. All theorems of this chapter transfer unchanged.*

10.2 Flows and the Value of a Flow

Definition 143 (Flow). A **flow** in N is a function $f : V \times V \rightarrow \mathbb{R}$ satisfying the following two properties:

1. *Capacity constraint:* $0 \leq f(u, v) \leq c(u, v)$ for all (u, v) .
2. *Conservation:* for every vertex $v \in V \setminus \{s, t\}$,

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w).$$

That is, total flow into v equals total flow out of v .

Remark 134. *Some authors work instead with the net flow $g(u, v) = f(u, v) - f(v, u)$, which is skew-symmetric, $g(u, v) = -g(v, u)$, and satisfies only $g(u, v) \leq c(u, v)$ (it may be negative). The two formulations are equivalent, but they cannot be mixed: a function with $f(u, v) \geq 0$ and $f(u, v) = -f(v, u)$ for all u, v is identically zero. Throughout this chapter flows are non-negative, as in the definition above.*

Definition 144 (Value of a flow). The **value** of a flow f is the net flow leaving the source,

$$|f| = \sum_{w \in V} f(s, w) - \sum_{u \in V} f(u, s).$$

Remark 135. *By conservation, $|f|$ equals the net flow into the sink:*

$$|f| = \sum_{u \in V} f(u, t) - \sum_{w \in V} f(t, w).$$

We prove this rigorously in Lemma 117.

The **maximum-flow problem** asks: given a network N , find a flow f of largest possible value $|f|$.

10.3 Cuts and the Capacity of a Cut

Definition 145 (*s-t cut*). An *s-t cut* of N is a partition (S, T) of V with $s \in S$ and $t \in T$. The **capacity** of the cut is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v),$$

i.e. the total capacity of arcs that go from the S -side to the T -side. (Arcs going from T to S are *not* counted.)

Definition 146 (Flow across a cut). For a flow f and an *s-t cut* (S, T) , the **net flow** across the cut is

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u).$$

The **minimum-cut problem** asks: find an *s-t cut* (S, T) of smallest possible capacity $c(S, T)$.

10.4 Flow Value Equals Net Flow Across Any Cut

Lemma 117 (Flow across a cut). For any flow f and any *s-t cut* (S, T) ,

$$|f| = f(S, T).$$

Proof. Start from the definition of $|f|$ and add 0's using conservation. For every $v \in S \setminus \{s\}$ we have $\sum_w f(v, w) - \sum_u f(u, v) = 0$, so

$$|f| = \sum_w f(s, w) - \sum_u f(u, s) = \sum_{v \in S} \left(\sum_{w \in V} f(v, w) - \sum_{u \in V} f(u, v) \right).$$

Expand the inner sums by splitting $V = S \cup T$:

$$= \sum_{v \in S} \sum_{w \in S} f(v, w) + \sum_{v \in S} \sum_{w \in T} f(v, w) - \sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in T} f(u, v).$$

The first and third double sums are equal (rename $(v, w) \leftrightarrow (u, v)$ inside $S \times S$) and cancel. What remains is exactly $f(S, T)$. $\square$

Corollary 40 (Weak duality). For any flow f and any *s-t cut* (S, T) ,

$$|f| \leq c(S, T).$$

In particular, $\max |f| \leq \min c(S, T)$.

Proof. By Lemma 117, $|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$. The first sum is at most $\sum_{u \in S, v \in T} c(u, v) = c(S, T)$ by the capacity constraint, and the second sum is non-negative, so $|f| \leq c(S, T)$. $\square$

Corollary 40 is already one half of the Max-Flow Min-Cut Theorem: *any* flow value is at most *any* cut capacity. The harder half is to show equality is attained. For that we need the key construction of residual graphs.

10.5 Residual Graphs and Augmenting Paths

The idea: if we have a current flow f and want to push more flow along an arc (u, v) , we can either (a) use *spare* capacity $c(u, v) - f(u, v) > 0$, or (b) *cancel* flow that was previously sent from v to u . Both options are encoded in a new graph.

Definition 147 (Residual capacity). Given a flow f in N , the **residual capacity** is

$$c_f(u, v) = (c(u, v) - f(u, v)) + f(v, u),$$

where we treat c and f as 0 on non-existent arcs. The **residual graph** is

$$N_f = (V, E_f), \quad E_f = \{(u, v) : c_f(u, v) > 0\}.$$

So the residual graph may contain arcs not present in E (these are *backward* arcs, corresponding to canceling existing flow) as well as arcs from E with leftover capacity.

Definition 148 (Augmenting path). An **augmenting path** (for the flow f) is a directed s - t path in the residual graph N_f . Its **bottleneck** $\Delta > 0$ is the minimum residual capacity of its arcs:

$$\Delta = \min_{(u,v) \in P} c_f(u,v).$$

Lemma 118 (Pushing along an augmenting path). Let P be an augmenting path in N_f with bottleneck Δ . Define a new function f' by

$$f'(u,v) = \begin{cases} f(u,v) + \Delta & \text{if } (u,v) \in P \text{ and } (u,v) \in E, \\ f(u,v) - \Delta & \text{if } (v,u) \in P \text{ and } (u,v) \in E, \\ f(u,v) & \text{otherwise.} \end{cases}$$

Then f' is a flow in N with $|f'| = |f| + \Delta$.

Proof. Capacity. If we pushed forward on $(u,v) \in E$, then $f'(u,v) = f(u,v) + \Delta \leq f(u,v) + (c(u,v) - f(u,v)) = c(u,v)$ because $\Delta \leq c_f(u,v) \leq c(u,v) - f(u,v)$ is ensured by the definition of c_f when the path uses the “forward” part. If we pushed backward, $f'(u,v) = f(u,v) - \Delta \geq 0$ for the same reason. In either case capacity is preserved.

Conservation. At every internal vertex v on P , exactly one arc of P enters v and exactly one leaves. In each of the four combinations (forward/forward, forward/backward, etc.) the net change at v is 0: the contribution added on one side is matched by an identical contribution on the other side. For vertices not on P , nothing changes.

Value. At the source s , exactly one arc of P is incident, and it is directed away from s in N_f . If that arc is a forward arc $(s,w) \in E$, then $|f'| = |f| + \Delta$. If it is a backward arc, i.e. $(w,s) \in E$ with flow being canceled, then $\sum_u f'(u,s) = \sum_u f(u,s) - \Delta$, which again yields $|f'| = |f| + \Delta$. $\square$

10.6 The Max-Flow Min-Cut Theorem

Theorem 145 (Ford–Fulkerson, 1956). Let $N = (V, E, s, t, c)$ be a flow network with a flow f . The following are equivalent:

1. f is a maximum flow in N .
2. The residual graph N_f contains no augmenting path (i.e. no directed $s \rightarrow t$ path).
3. There exists an s - t cut (S, T) with $|f| = c(S, T)$.

In particular, the value of a maximum flow equals the capacity of a minimum s - t cut:

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

Proof. (1) $\Rightarrow$ (2). If N_f contained an augmenting path with bottleneck $\Delta > 0$, Lemma 118 produces a flow of value $|f| + \Delta > |f|$, contradicting maximality.

(2) $\Rightarrow$ (3). Suppose no augmenting path exists. Let

$$S = \{v \in V : \text{there is a directed path from } s \text{ to } v \text{ in } N_f\}, \quad T = V \setminus S.$$

Then $s \in S$ by reflexivity; $t \in T$ because no s - t path exists in N_f . So (S, T) is a valid s - t cut. For every $u \in S$ and $v \in T$, the arc (u, v) is *not* in E_f (otherwise v would be reachable from s), so $c_f(u, v) = 0$, which means

$$(c(u, v) - f(u, v)) + f(v, u) = 0.$$

Since both summands are non-negative, both must be zero: $f(u, v) = c(u, v)$ and $f(v, u) = 0$. Summing over $u \in S, v \in T$ and using Lemma 117,

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u) = \sum_{u \in S, v \in T} c(u, v) - 0 = c(S, T).$$

(3) $\Rightarrow$ (1). By weak duality (Corollary 40), $|f| \leq c(S, T)$ for every cut, so any flow whose value already equals *some* cut capacity is already as large as possible.

Finally, a maximum flow exists: the set of flows is a non-empty (it contains the zero flow), closed and bounded subset of $\mathbb{R}^{V \times V}$, being cut out by finitely many linear equations and non-strict inequalities with $0 \leq f(u, v) \leq c(u, v)$, and $f \mapsto |f|$ is continuous, so it attains its maximum. Applying (1) $\Rightarrow$ (3) to a maximum flow and using weak duality gives $\max_f |f| = \min_{(S, T)} c(S, T)$. $\square$

10.7 The Ford–Fulkerson Algorithm

Theorem 145 suggests an algorithm: starting from the zero flow, repeatedly find an augmenting path and push flow along it, until no augmenting path remains. When the algorithm terminates, the set S of vertices reachable from s in the final residual graph gives a minimum cut.

Algorithm 58 Ford–Fulkerson

```

1:  $f(u, v) \leftarrow 0$  for every arc  $(u, v)$ .
2: while there exists a directed  $s$ - $t$  path  $P$  in  $N_f$  do
3:    $\Delta \leftarrow \min_{(u, v) \in P} c_f(u, v)$ 
4:   for all  $(u, v) \in P$  do
5:     if  $(u, v) \in E$  then  $f(u, v) \leftarrow f(u, v) + \Delta$ 
6:     else  $f(v, u) \leftarrow f(v, u) - \Delta$ 
7:     end if
8:   end for
9: end while
10: return  $f$ 

```

Theorem 146 (Integrality). If all capacities are integers, then at every iteration the flow values remain integers, and the algorithm terminates in at most $|f^*|$ iterations, where f^* is a maximum flow. Consequently, some maximum flow is integer-valued.

Proof. By induction: initially the flow is $0 \in \mathbb{Z}$. If f is integer-valued then every residual capacity is an integer, so the bottleneck Δ is a positive integer, so the new flow f' is integer-valued. Each iteration increases $|f|$ by at least 1, and $|f|$ is bounded above by the capacity of any cut, hence finite. $\square$

Remark 136 (Termination with real capacities). *For irrational capacities the naive algorithm may fail to terminate (Ford and Fulkerson gave an explicit example). The Edmonds–Karp refinement—always choose a shortest augmenting path (fewest arcs) via BFS—terminates in $O(|V| \cdot |E|^2)$ time regardless of capacities; Dinic’s algorithm brings this down to $O(|V|^2 |E|)$. We do not prove these bounds here, but they rely on the same residual graph machinery introduced above.*

10.8 Applications

The Max-Flow Min-Cut Theorem is a workhorse. We derive two classical consequences.

10.8.1 Menger’s Theorem

Theorem 147 (Menger, edge version). Let G be a finite directed graph and s, t two distinct vertices. The maximum number of pairwise edge-disjoint directed s - t paths equals the minimum number of arcs whose removal disconnects t from s .

Sketch. Turn G into a flow network by giving every arc capacity 1. By Theorem 146 there is an integer maximum flow f^* ; since capacities are 1, flow on each arc is either 0 or 1. The arcs carrying flow decompose into $|f^*|$ edge-disjoint s - t paths (remove them greedily, using conservation at every internal vertex). Conversely, a minimum cut is a set of arcs of total capacity $|f^*|$ —i.e. a set of $|f^*|$ arcs whose removal puts t on a different side of the cut than s , disconnecting them. Equality is Theorem 145. $\square$

A vertex version of Menger's theorem follows by the standard *vertex splitting* reduction: replace each internal vertex v by two copies $v^{\text{in}}, v^{\text{out}}$ joined by an arc of capacity 1, and replicate incoming/outgoing arcs accordingly.

10.8.2 König's Theorem Revisited

Theorem 148 (König). In any bipartite graph $G = (A \cup B, E)$, the size of a maximum matching equals the size of a minimum vertex cover.

Proof. Build a network: add source s with arcs $s \rightarrow a$ (capacity 1) for each $a \in A$, add sink t with arcs $b \rightarrow t$ (capacity 1) for each $b \in B$, and for every edge $\{a, b\} \in E$ add an arc $a \rightarrow b$ of capacity $|A| + 1$ (any integer larger than $|A|$ works; it plays the role of an “infinite” capacity while keeping all capacities integers). By Theorem 146 there is an integer max flow f^* , and the arcs $a \rightarrow b$ with $f^*(a, b) = 1$ form a matching of size $|f^*|$ (at most one unit of flow enters each a and at most one unit leaves each b , so each arc $a \rightarrow b$ carries 0 or 1, and no two arcs carrying flow share an endpoint).

Let (S, T) be a minimum cut. The cut $(\{s\}, A \cup B \cup \{t\})$ has capacity $|A|$, so $c(S, T) \leq |A|$; since each arc $a \rightarrow b$ has capacity $|A| + 1$, no such arc may cross from S to T , so whenever $a \in A \cap S$ and $\{a, b\} \in E$ we have $b \in B \cap S$. Therefore

$$C = (A \cap T) \cup (B \cap S)$$

is a vertex cover. Its size is $|A \cap T| + |B \cap S| = c(S, T) = |f^*|$, matching the size of the maximum matching. Weak duality (a cover has size at least the size of any matching because each matching edge needs a cover vertex) gives the other inequality. $\square$

10.9 Worked Example

Example 47. Consider the network with vertices $\{s, a, b, t\}$ and arcs

$$s \rightarrow a \ (3), \quad s \rightarrow b \ (2), \quad a \rightarrow b \ (1), \quad a \rightarrow t \ (2), \quad b \rightarrow t \ (3).$$

The diagram below shows the network, the final flow, and the minimum cut.

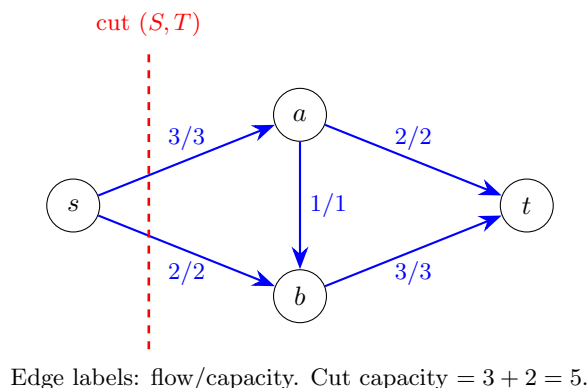


Figure 155: A flow network of value 5 with the saturating min cut $S = \{s\}$.

One augmenting path is $s \rightarrow a \rightarrow t$ with bottleneck 2; push 2 units. Next, $s \rightarrow b \rightarrow t$ has bottleneck 2; push 2 more. Residual graph now has $s \rightarrow a$ (1), $a \rightarrow b$ (1), $b \rightarrow t$ (1), yielding the path $s \rightarrow a \rightarrow b \rightarrow t$ with bottleneck 1; push another unit. No augmenting path remains: the set of vertices reachable from s is $S = \{s\}$, giving cut $(\{s\}, \{a, b, t\})$ of capacity $3 + 2 = 5$. Max flow value is $2 + 2 + 1 = 5 = c(S, T)$, confirming Theorem 145.

10.10 Exercises

10.10.1 Easy Questions

Exercise 10.1. Prove the “other half” of Lemma 117: for any flow f , the net flow into the sink t equals $|f|$.

Exercise 10.2. Derive the vertex form of Menger’s theorem from the edge form using the vertex-splitting construction described above.

Exercise 10.3. On the network of Figure 155, list all augmenting paths the algorithm could explore in some order, and verify that the final flow value is independent of the order.

10.10.2 Medium Questions

Exercise 10.4. Let N be a flow network with integer capacities bounded by C and m arcs. Show that the Ford–Fulkerson algorithm (using any augmenting paths) runs in $O(mC|V|)$ time. Give an example where this bound is tight up to constants.

Exercise 10.5. Use the max-flow min-cut theorem to prove Hall’s marriage theorem: a bipartite graph $G = (A \cup B, E)$ has a matching saturating A iff $|N(S)| \geq |S|$ for every $S \subseteq A$.

Exercise 10.6. Given an undirected graph G with edge capacities, define the *global minimum cut* as the partition $(S, V \setminus S)$ with $\emptyset \neq S \neq V$ minimizing the total capacity of edges crossing the cut. Show how to compute it with $|V| - 1$ calls to a max-flow subroutine.

10.10.3 Hard Questions

Exercise 10.7. Construct an example with *irrational* capacities on which a particular choice of augmenting paths causes Ford–Fulkerson to run forever while the flow value converges to a number strictly smaller than the true maximum. (This is the classical Ford–Fulkerson counterexample; it motivates the Edmonds–Karp rule.)

Exercise 10.8. Prove the Edmonds–Karp bound: if augmenting paths are always chosen by BFS (shortest first), Ford–Fulkerson terminates after at most $O(|V| \cdot |E|)$ augmentations and runs in $O(|V| \cdot |E|^2)$ time, regardless of capacities.

10.10.4 Diagram Questions

Exercise 10.9. Draw a flow network on $\{s, x, y, z, t\}$ with five arcs of capacity at most 4 such that the max-flow value equals 3 and the min-cut consists of exactly two arcs. Mark the saturating cut.

Exercise 10.10. For the diamond network with vertices $\{s, u, v, t\}$ and arcs $s \rightarrow u(10)$, $s \rightarrow v(10)$, $u \rightarrow v(1)$, $u \rightarrow t(10)$, $v \rightarrow t(10)$, draw the residual graph after pushing 10 units along $s \rightarrow u \rightarrow t$ and identify the next augmenting path.

11 Advanced Topics in Graph Theory

The remainder of the book is a tour of topics commonly taught in first-year graduate courses on graph theory and combinatorics. Each subsection is self-contained modulo the definitions established in earlier chapters, but together they sketch the modern toolkit: *extremal* arguments (forbidden-subgraph problems), *Ramsey-type* unavoidability, the *probabilistic method*, *spectral* (linear-algebraic) techniques, *random graph* models, Szemerédi’s *regularity lemma*, and the *graph minor* programme of Robertson and Seymour.

11.1 Extremal Graph Theory

Extremal graph theory studies the maximum (or minimum) possible value of some graph parameter (typically edges) among graphs satisfying a given constraint (typically excluding a fixed subgraph H). The central object is the Turán number $\text{ex}(n, H)$. This subsection develops the non-bipartite theory completely (Turán, Erdős–Simonovits stability, Erdős–Stone) and then treats the much subtler bipartite theory, including the Kővári–Sós–Turán theorem, the Bondy–Simonovits even-cycle bound, the Zarankiewicz problem, supersaturation, and the Moore bound for graphs of high girth.

11.1.1 The Turán Number and the Turán Graph

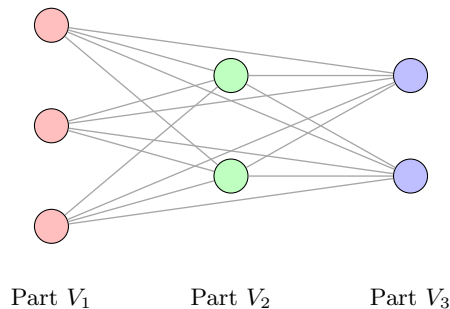
Definition 149 (Extremal number). For a graph H and integer $n \geq 1$, the **extremal number** (or *Turán number*) is

$$\text{ex}(n, H) = \max\{|E(G)| : |V(G)| = n, G \text{ contains no copy of } H\}.$$

A graph attaining this maximum is called *H-extremal*. The density version is $\pi(H) = \lim_{n \rightarrow \infty} \text{ex}(n, H) / \binom{n}{2}$, called the **Turán density** of H (the limit exists by an averaging argument of Katona, Nemetz, and Simonovits).

Definition 150 (Turán graph). The **Turán graph** $T(n, r)$ is the complete r -partite graph on n vertices whose part sizes differ by at most one (a *balanced* r -partition). Its edge count is

$$t(n, r) = \binom{n}{2} - \sum_{i=1}^r \binom{|V_i|}{2} = \left(1 - \frac{1}{r}\right) \frac{n^2}{2} + O(1).$$



$T(7, 3)$: balanced complete 3-partite, K_4 -free, $|E| = 16$

Figure 156: The Turán graph $T(7, 3)$ achieves $\text{ex}(7, K_4)$.

11.1.2 Mantel’s Theorem: Three Proofs

Theorem 149 (Mantel, 1907). $\text{ex}(n, K_3) = \lfloor n^2/4 \rfloor$, attained uniquely (up to isomorphism) by the balanced complete bipartite graph $K_{\lfloor n/2 \rfloor, \lfloor n/2 \rfloor}$.

Proof 1: local neighborhoods and Cauchy–Schwarz. Let G be triangle-free with n vertices and m edges. For any edge $uv \in E(G)$, the sets $N(u)$ and $N(v)$ must be disjoint, as a common neighbor w would produce triangle uvw . Therefore $d(u) + d(v) \leq n$. Summing over all edges,

$$\sum_{uv \in E} (d(u) + d(v)) = \sum_{v \in V} d(v)^2 \leq n \cdot m.$$

By the Cauchy–Schwarz inequality $\sum_v d(v)^2 \geq (\sum_v d(v))^2/n = (2m)^2/n$. Combining, $4m^2/n \leq nm$, hence $m \leq n^2/4$. $\square$

Proof 2: induction on n . The inequality $m \leq \lfloor n^2/4 \rfloor$ is trivial for $n \leq 2$. Suppose it holds for $n - 2$ and let G be triangle-free on n vertices. If G has no edges we are done; otherwise pick any edge uv . Write $G' = G \setminus \{u, v\}$, which has at most $\lfloor (n - 2)^2/4 \rfloor$ edges by induction. Triangle-freeness and the disjointness $N(u) \cap N(v) = \emptyset$ show that u and v together contribute $d(u) + d(v) - 1 \leq n - 1$ edges, so

$$m \leq \left\lfloor \frac{(n-2)^2}{4} \right\rfloor + (n - 1) = \left\lfloor \frac{n^2}{4} \right\rfloor.$$

$\square$

Proof 3: weight-shifting / LP relaxation. Assign each vertex v weight $x_v \geq 0$ with $\sum x_v = 1$. Define $f(\mathbf{x}) = \sum_{uv \in E} x_u x_v$. Triangle-freeness makes f at most $1/4$: indeed, let $\mathbf{x}$ maximize f with the fewest positive coordinates. If two *non-adjacent* vertices u, v both have positive weight, put $s_u = \sum_{w \in N(u)} x_w$ and $s_v = \sum_{w \in N(v)} x_w$; since $uv \notin E$, neither sum involves x_u or x_v , so $f = x_u s_u + x_v s_v + (\text{terms free of } x_u, x_v)$. If $s_u \geq s_v$, shifting all weight of v onto u does not decrease f (symmetrically if $s_v > s_u$) and shrinks the support, a contradiction. Hence the support of $\mathbf{x}$ is a clique, so (as G is triangle-free) it has at most two vertices, giving $f \leq \max_{t \in [0,1]} t(1 - t) = 1/4$. Now set $x_v = 1/n$ to obtain $m/n^2 \leq 1/4$. $\square$

Remark 137. *Proof 3 is the weighted-density technique central to flag-algebra arguments; it generalizes immediately to any forbidden subgraph. Proof 1 generalizes to a fractional “number-of-edges in triangle” count used in Goodman’s theorem (below).*

Theorem 150 (Goodman, 1959). Every graph G on n vertices with m edges contains at least

$$\frac{m}{3}(4m/n - n)$$

triangles. In particular a triangle-free graph has $m \leq n^2/4$.

Proof. For an edge xy , both $N(x)$ and $N(y)$ lie in V , so $|N(x) \cap N(y)| \geq d(x) + d(y) - n$, and each common neighbor z gives a triangle xyz . Every triangle contains exactly three edges, so, writing T for the number of triangles,

$$3T = \sum_{xy \in E} |N(x) \cap N(y)| \geq \sum_{xy \in E} (d(x) + d(y)) - mn = \sum_v d(v)^2 - mn \geq \frac{4m^2}{n} - mn,$$

the last step by Cauchy–Schwarz exactly as in Proof 1 of Mantel’s theorem. Dividing by 3 gives $T \geq \frac{m}{3}(4m/n - n)$. $\square$

11.1.3 Turán’s Theorem

Theorem 151 (Turán, 1941). For $r \geq 2$ and $n \geq r$, if G is a graph on n vertices with no K_{r+1} subgraph, then $|E(G)| \leq t(n, r)$, with equality only when $G \cong T(n, r)$.

Proof 1: Zykov symmetrization. Let G be K_{r+1} -free with the maximum number of edges among all K_{r+1} -free graphs on n vertices. We show that “not adjacent” is an equivalence relation on $V(G)$.

Symmetry and *reflexivity* are automatic. For *transitivity*, suppose $u \not\sim v$ and $v \not\sim w$ but $u \sim w$. If $d(v) < d(u)$, delete v and replace it by a clone v' of u with $N(v') = N(u)$. The new graph is still K_{r+1} -free (a clique contains at most one of the non-adjacent twins u, v' , and v' may be replaced by u), and its edge

count is $|E| - d(v) + d(u) > |E|$, contradicting maximality; the case $d(v) < d(w)$ is symmetric. Otherwise $d(v) \geq d(u)$ and $d(v) \geq d(w)$: delete u and w and add two new clones of v . This removes $d(u) + d(w) - 1$ edges (the edge uw is counted twice) and adds $2d(v)$, a net gain of at least 1; the result is again K_{r+1} -free, since v and its clones are pairwise non-adjacent with the same neighborhood. Either way maximality is contradicted, so non-adjacency is transitive.

Thus in the extremal case $V(G)$ splits into classes of mutually non-adjacent vertices, with all edges between distinct classes present. That is, G is complete multipartite with some number of parts $s \leq r$. Among complete s -partite graphs with s parts of total size n , the edge count $\binom{n}{2} - \sum \binom{|V_i|}{2}$ is maximized (by convexity of $x \mapsto \binom{x}{2}$) when the parts are as equal as possible. Among $s \in \{1, \dots, r\}$, the choice $s = r$ dominates, giving $T(n, r)$. $\square$

Proof 2: Motzkin–Straus (probabilistic / LP). Define the *Lagrangian*

$$\lambda(G) = \max \left\{ \sum_{uv \in E} x_u x_v : x \geq 0, \sum_v x_v = 1 \right\}.$$

For any graph G on n vertices, setting $x_v = 1/n$ gives $\lambda(G) \geq m/n^2$. Motzkin and Straus proved $\lambda(G) = \frac{1}{2}(1 - 1/\omega(G))$, where $\omega(G)$ is the clique number. If $\omega(G) \leq r$, then $m/n^2 \leq \frac{1}{2}(1 - 1/r)$, giving Turán's bound in the weaker form $|E(G)| \leq (1 - 1/r)n^2/2$, which equals $t(n, r)$ when $r \mid n$ (the exact bound and the uniqueness need a finer argument such as Proof 1). The key to Motzkin–Straus is to observe that *some* optimal $\mathbf{x}$ is supported on a clique of G (not every one is: on C_4 the uniform weighting is optimal). $\square$

Proof 3: induction via a max-degree vertex. Let v be a vertex of maximum degree Δ in a K_{r+1} -free graph G . Write $N = N(v)$, $M = V \setminus (N \cup \{v\})$. The induced subgraph $G[N]$ is K_r -free (a K_r in $G[N]$ plus v forms K_{r+1}), so by induction $|E(G[N])| \leq t(\Delta, r - 1)$. Edges not inside N total at most $\Delta \cdot (n - \Delta)$ (the max-degree bound). Combining and optimizing over Δ yields the result. $\square$

Corollary 41 (Turán density of a clique). $\pi(K_{r+1}) = 1 - 1/r$.

Corollary 42 (Chromatic consequence). Every graph G on n vertices with $|E(G)| > t(n, r)$ edges contains a K_{r+1} ; in particular, any G with $|E(G)| > (1 - 1/r)n^2/2$ has chromatic number at least $r + 1$.

11.1.4 Stability: The Erdős–Simonovits Theorem

A hallmark of modern extremal graph theory: if a graph is K_{r+1} -free with *nearly* extremal edge count, it must itself look almost like $T(n, r)$.

Theorem 152 (Erdős–Simonovits stability, 1968). For every $r \geq 2$ and $\varepsilon > 0$ there exist $\delta > 0$ and n_0 such that any K_{r+1} -free graph G on $n \geq n_0$ vertices with $|E(G)| \geq t(n, r) - \delta n^2$ edges can be transformed into $T(n, r)$ by adding or deleting at most εn^2 edges.

Proof sketch. If G were far from $T(n, r)$ in edit distance, the Zykov symmetrization of Proof 1 of Theorem 151 would be strictly improving, contradicting extremality up to δn^2 edges. A full proof uses the Szemerédi regularity lemma (see the subsection on regularity) to replace “close to $T(n, r)$ ” by a regularity-partition statement and then extracts a Turán-type improvement. $\square$

Remark 138. *Stability statements are the workhorse of “solve by perturbation” arguments: prove the result for the extremal configuration, then extend by smallness of perturbations.*

11.1.5 The Erdős–Stone–Simonovits Theorem

Theorem 153 (Erdős–Stone, 1946; Erdős–Simonovits, 1966). For any graph H with chromatic number $\chi(H) \geq 2$,

$$\text{ex}(n, H) = \left(1 - \frac{1}{\chi(H)-1} + o(1)\right) \frac{n^2}{2} \quad \text{as } n \rightarrow \infty.$$

Equivalently $\pi(H) = 1 - 1/(\chi(H) - 1)$.

Proof sketch via Erdős–Stone. The Erdős–Stone theorem asserts that for every $r \geq 2$, every $t \geq 1$, and $\varepsilon > 0$, if n is large enough then every graph on n vertices with $|E(G)| \geq (1 - 1/r + \varepsilon)n^2/2$ edges contains the complete $(r+1)$ -partite graph $K_{r+1}(t)$ with each part of size t . Given H of chromatic number $r+1$, choose t large enough that $H \subseteq K_{r+1}(t)$. Any graph with more than $(1 - 1/r + \varepsilon)n^2/2$ edges therefore contains $K_{r+1}(t) \supseteq H$. Combining with Turán’s lower bound $t(n, r) \leq \text{ex}(n, H)$ (since $T(n, r)$ is $(r+1)$ -chromatic-free, thus H -free) gives the theorem. $\square$

Two lessons from this theorem.

Remark 139 (The bipartite cliff). *For bipartite H we have $\chi(H) = 2$, so Erdős–Stone merely gives $\text{ex}(n, H) = o(n^2)$ —the leading order vanishes and we must determine the exact exponent of n . This is the central difficulty in the degenerate (bipartite) theory.*

Remark 140 (Supersaturation). *Erdős and Simonovits also proved that exceeding $\text{ex}(n, H)$ by εn^2 edges yields not merely one but $\Theta(n^{|V(H)|})$ copies of H : the dependence is supersaturated. This is a quantitative strengthening routinely exploited.*

11.1.6 The Bipartite Case: Kővári–Sós–Turán

Theorem 154 (Kővári–Sós–Turán, 1954). For integers $2 \leq s \leq t$,

$$\text{ex}(n, K_{s,t}) \leq \frac{1}{2}(t-1)^{1/s}(n^{2-1/s}) + \frac{s-1}{2}n.$$

Proof. Double-count the number of “cherries” $S = \{A \subseteq V(G) : |A| = s, A \subseteq N(v) \text{ for some } v\}$. For each $v \in V(G)$ the number of s -subsets of $N(v)$ is $\binom{d(v)}{s}$, so

$$\sum_{v \in V} \binom{d(v)}{s} = \#\{(A, v) : |A| = s, A \subseteq N(v)\}.$$

If G contains no $K_{s,t}$, then each s -set A has at most $t-1$ common neighbors v , so the right-hand side is at most $(t-1)\binom{n}{s}$. By convexity of $\binom{x}{s}$,

$$n \binom{2m/n}{s} \leq \sum_v \binom{d(v)}{s} \leq (t-1) \binom{n}{s}.$$

Solving for m , and using $\binom{x}{s} \geq (x-s+1)^s/s!$, gives the claimed bound. $\square$

Corollary 43. $\text{ex}(n, C_4) = \text{ex}(n, K_{2,2}) \leq \frac{1}{2}(1 + \sqrt{4n-3}) \cdot n/2 = O(n^{3/2})$.

Theorem 155 (Reiman; Erdős–Rényi–Sós, Brown). The upper bound $\text{ex}(n, C_4) \leq \frac{n}{4}(1 + \sqrt{4n-3})$ is attained to leading order: the Erdős–Rényi *polarity graph* of the projective plane $\text{PG}(2, q)$, whose $n = q^2 + q + 1$ vertices are the points of $\text{PG}(2, q)$, with distinct points x, y adjacent iff $x_0y_0 + x_1y_1 + x_2y_2 = 0$, is C_4 -free and has $\frac{1}{2}q(q+1)^2 = (1 + o(1))n^{3/2}/2$ edges. (The point–line incidence graph of $\text{PG}(2, q)$ is also C_4 -free, but it has $2n$ vertices and only $(q+1)n$ edges.)

Theorem 156 (Bondy–Simonovits, 1974). For every integer $k \geq 2$,

$$\text{ex}(n, C_{2k}) \leq 100k \cdot n^{1+1/k}.$$

Whether the exponent $1 + 1/k$ is tight for $k = 4$ and for $k \geq 6$ is a long-standing open problem; matching constructions are only known for $k \in \{2, 3, 5\}$ (projective plane, generalized quadrangle, generalized hexagon).

11.1.7 The Zarankiewicz Problem

Definition 151 (Zarankiewicz function). $z(m, n; s, t)$ is the maximum number of 1’s in an $m \times n$ 0/1-matrix with no all-ones $s \times t$ submatrix. Equivalently, it is the maximum edge count of a bipartite graph with parts of sizes m and n containing no $K_{s,t}$ whose s -vertex side lies in the part of size m .

Theorem 157 (Kővári–Sós–Turán, bipartite form). $z(m, n; s, t) \leq (t-1)^{1/s}(m-s+1)n^{1-1/s} + (s-1)n$.

11.1.8 Graphs of High Girth: The Moore Bound

The interaction between extremal counts and girth is the complementary theme: how few vertices can a graph with high minimum degree and high girth have?

Definition 152 (Girth). The **girth** $g(G)$ is the length of the shortest cycle in G (or ∞ if G is a forest).

Theorem 158 (Moore bound). Let G be a graph with minimum degree δ and girth at least $2k + 1$. Then

$$|V(G)| \geq 1 + \delta + \delta(\delta - 1) + \cdots + \delta(\delta - 1)^{k-1}.$$

For even girth $2k$,

$$|V(G)| \geq 2(1 + (\delta - 1) + \cdots + (\delta - 1)^{k-1}).$$

Proof for odd girth. Pick any vertex v and perform BFS for k layers. Girth $\geq 2k + 1$ ensures all vertices in layers $0, 1, \dots, k$ are distinct: a repeat in layer $\leq k$ would close a cycle of length $\leq 2k$. Each vertex in layer $i < k$ contributes $\geq \delta - 1$ neighbors to layer $i + 1$ (except v which contributes δ to layer 1), giving the stated count. $\square$

Definition 153 (Cage). A (δ, g) -**cage** is a δ -regular graph of girth g with the smallest possible number of vertices.

Cages attaining the Moore bound with equality are called *Moore graphs*. Famous examples include the Petersen graph $((3, 5)$ -cage, 10 vertices), the Hoffman–Singleton graph $((7, 5)$ -cage, 50 vertices), and the Heawood graph $((3, 6)$ -cage, 14 vertices).

Theorem 159 (Hoffman–Singleton, 1960). A Moore graph of girth 5 and degree δ exists only if $\delta \in \{2, 3, 7, 57\}$.

The $\delta = 57$ case remains open after more than sixty years—one of the most famous unsolved problems in algebraic graph theory.

11.1.9 Supersaturation

Theorem 160 (Erdős–Simonovits supersaturation). For every graph H and $\varepsilon > 0$ there are $c > 0$ and n_0 such that any graph G on $n \geq n_0$ vertices with $|E(G)| \geq (\pi(H) + \varepsilon) \binom{n}{2}$ edges contains at least $c \cdot n^{|V(H)|}$ copies of H .

Proof sketch. Apply the regularity lemma to G and pass to the cluster graph. The regularity partition preserves the excess density, and the counting lemma turns it into an embedding count of H . $\square$

11.1.10 Exercises

Easy Questions.

Exercise 11.1. Prove Mantel’s theorem by induction on n : remove an edge uv from a triangle-free graph G , bound $|E(G - \{u, v\})|$ inductively, and account for the removed degrees.

Exercise 11.2. Show that the Turán graph $T(n, r)$ has chromatic number exactly r when $n \geq r$. Deduce from Turán’s theorem the *chromatic* density bound: any graph on n vertices with chromatic number at most r has at most $t(n, r)$ edges.

Exercise 11.3. Compute the number of triangles in $T(n, r)$. For which r does $T(n, r)$ maximize the number of triangles among all K_{r+1} -free graphs on n vertices?

Medium Questions.

Exercise 11.4. Prove that every K_4 -free graph on n vertices contains an independent set of size $\geq \lfloor n^{1/3} \rfloor$. (Hint: the neighborhood of any vertex spans a triangle-free graph, and a triangle-free graph on m vertices has an independent set of size $\geq \lfloor \sqrt{m} \rfloor$; if every degree is below $n^{2/3}$, use Turán's theorem on the complement.) Show that $\sqrt{n}$ would be false in general: the Paley graph on 17 vertices is K_4 -free with independence number $3 < \sqrt{17}$.

Exercise 11.5. Prove Reiman's bound $\text{ex}(n, K_{2,2}) \leq \frac{n}{4}(1 + \sqrt{4n-3})$ directly, using the fact that any two vertices have at most one common neighbor in a $K_{2,2}$ -free graph.

Exercise 11.6. Show that the incidence graph of a projective plane of order q has $2(q^2 + q + 1)$ vertices, is $(q + 1)$ -regular, and has girth 6. Verify that it achieves the Moore bound with equality.

Hard Questions.

Exercise 11.7 (Kruskal–Katona warm-up). Let G be a graph with m edges, and write $m = \binom{x}{2}$ with real $x \geq 2$. Show that the number of triangles in G is at most $\binom{x}{3} \leq \frac{\sqrt{2}}{3} m^{3/2}$, with equality in the first inequality for $G = K_x$ when x is an integer.

Exercise 11.8. Prove that every graph on n vertices and at least $\lfloor n^2/4 \rfloor + 1$ edges contains at least $\lfloor n/2 \rfloor$ triangles. (This is Rademacher's theorem, a supersaturation result extending Mantel.)

Exercise 11.9. Let G be a triangle-free graph with $|V(G)| = n$ and $\delta(G) > 2n/5$. Prove that G is bipartite (Andrásfai–Erdős–Sós). Construct a triangle-free, non-bipartite graph with $\delta(G) = 2n/5$, showing that the hypothesis cannot be weakened to $\delta(G) \geq 2n/5$. (Hint: blow up C_5 .)

Exercise 11.10 (Erdős–Gallai). Show that every graph on n vertices with more than $\frac{1}{2}(k-1)n$ edges contains a path of length k .

Diagram Questions.

Exercise 11.11. Draw $T(8, 3)$ and $T(8, 4)$. For each, count the edges directly and verify the formula $t(n, r) = \binom{n}{2} - \sum_i \binom{|V_i|}{2}$.

Exercise 11.12. Draw a 3-regular graph on 10 vertices with girth 5 that achieves the odd Moore bound $1 + 3 + 3 \cdot 2 = 10$ with equality. The Petersen graph is the canonical example—draw it in the standard pentagon-pentagram form and label the girth witness. (By Theorem 158, a 4-regular graph of girth 5 needs at least $1 + 4 + 4 \cdot 3 = 17$ vertices; the smallest one, the Robertson graph, has 19.)

11.2 Ramsey Theory

Ramsey theory sits opposite extremal theory. Instead of asking how dense a graph can be before it must contain H , we ask how large a complete graph must be before *every* red/blue coloring of its edges contains a monochromatic copy of H . The unifying meta-theorem, sometimes phrased as “complete disorder is impossible”, asserts that any sufficiently large structure contains a highly organized substructure.

11.2.1 The Classical Ramsey Numbers

Definition 154 (Ramsey number). $R(s, t)$ is the least integer N such that every red/blue coloring of the edges of K_N contains a red K_s or a blue K_t . When $s = t$ we write $R(s) := R(s, s)$ for the *diagonal* Ramsey number.

Definition 155 (Multicolor Ramsey number). $R(s_1, \dots, s_k)$ is the least N such that in any k -coloring of the edges of K_N , some color class i contains K_{s_i} .

Theorem 161 (Ramsey, 1930). For all $s, t \geq 2$, $R(s, t)$ is finite. Moreover

$$R(s, t) \leq R(s-1, t) + R(s, t-1),$$

and consequently $R(s, t) \leq \binom{s+t-2}{s-1}$.

Proof. Induct on $s + t$, with base cases $R(2, t) = t$ and $R(s, 2) = s$ being immediate. For the inductive step, pick any vertex v in K_N with $N = R(s-1, t) + R(s, t-1)$. Among its $N-1$ incident edges, either $\geq R(s-1, t)$ are red or $\geq R(s, t-1)$ are blue (pigeonhole). In the first case, apply induction to the induced red neighborhood to obtain either a red K_{s-1} (which together with v gives a red K_s) or a blue K_t ; the second case is symmetric.

For the closed-form bound, one verifies $\binom{(s-1)+t-2}{s-2} + \binom{s+(t-1)-2}{s-1} = \binom{s+t-2}{s-1}$ and concludes by induction using Pascal's identity. $\square$

Corollary 44 (Diagonal upper bound). $R(k) \leq \binom{2k-2}{k-1} \sim \frac{4^{k-1}}{\sqrt{\pi k}}$.

Theorem 162 (Greenwood–Gleason refinement, 1955). If $R(s-1, t)$ and $R(s, t-1)$ are both even, then the recursive bound is strict: $R(s, t) \leq R(s-1, t) + R(s, t-1) - 1$. For example, $R(3, 4) \leq R(2, 4) + R(3, 3) - 1 = 9$.

Remark 141 (Small cases). *The following exact values are known, largely by exhaustive computer search:*

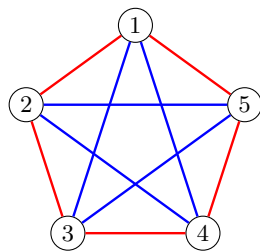
$$R(3, 3) = 6, \quad R(3, 4) = 9, \quad R(3, 5) = 14, \quad R(3, 6) = 18, \quad R(3, 7) = 23,$$

$$R(3, 8) = 28, \quad R(3, 9) = 36, \quad R(4, 4) = 18, \quad R(4, 5) = 25.$$

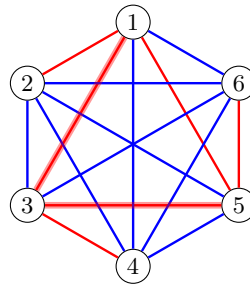
The value of $R(5, 5)$ is unknown, with $43 \leq R(5) \leq 46$ as of 2025. Erdős's famous quip: if aliens demanded the exact value of $R(5, 5)$, we should devote every computer on Earth to it; if they demanded $R(6, 6)$, we should attack them first.

Theorem 163 ($R(3, 3) = 6$). In any red/blue coloring of the edges of K_6 there is a monochromatic triangle; K_5 admits a coloring with no monochromatic triangle (the pentagon plus pentagram coloring).

Proof. Pick any vertex v of K_6 : among its 5 incident edges, by pigeonhole at least three, say vu_1, vu_2, vu_3 , have the same color, say red. If any $u_i u_j$ is red we have a red triangle; if all three are blue we have a blue triangle $u_1 u_2 u_3$. For K_5 , color the 5-cycle edges red and the “chord” edges of the pentagram blue; the only triangles of K_5 use both red and blue edges. $\square$



K_5 : no monochromatic K_3



K_6 : a monochromatic $\triangle$ is unavoidable

Figure 157: Witnessing $R(3, 3) = 6$. Left: the unique (up to isomorphism) good K_5 colouring. Right: every 2-colouring of K_6 forces a monochromatic triangle.

11.2.2 Probabilistic Lower Bounds

Theorem 164 (Erdős, 1947). For all $k \geq 3$, $R(k, k) > 2^{k/2}$.

Proof. Color each edge of K_N red or blue independently and uniformly. For any fixed k -subset S of vertices, the probability that $K_N[S]$ is monochromatic is $2 \cdot 2^{-\binom{k}{2}}$. Let X be the total number of monochromatic K_k 's. Then

$$\mathbb{E}[X] = \binom{N}{k} \cdot 2 \cdot 2^{-\binom{k}{2}} \leq \frac{N^k}{k!} \cdot 2^{1-k(k-1)/2}.$$

For $N = \lfloor 2^{k/2} \rfloor$ and $k \geq 3$, one checks $\mathbb{E}[X] < 1$, so there is a coloring realizing $X = 0$. Any such coloring witnesses $R(k, k) > N$. $\square$

Theorem 165 (Spencer, via Lovász Local Lemma). $R(k, k) > (1 + o(1)) \frac{\sqrt{2}}{e} \cdot k \cdot 2^{k/2}$.

Proof sketch. Apply the symmetric Lovász Local Lemma to the events “the k -set S is monochromatic”. Two such events are dependent only if S, S' share at least two vertices; for each S there are at most $\binom{k}{2} \binom{N-2}{k-2}$ dependent events. Using $e \cdot p \cdot (d+1) \leq 1$ and optimizing N yields the factor-of- k improvement over the first-moment bound. $\square$

Theorem 166 (Off-diagonal: Ajtai–Komlós–Szemerédi / Kim). $\frac{c \cdot k^2}{\log k} \leq R(3, k) \leq \frac{C \cdot k^2}{\log k}$.

The matching upper and lower bounds on $R(3, k)$ (Ajtai–Komlós–Szemerédi 1980 and Shearer 1983 for the upper bound; Kim 1995 for the lower bound) are one of the crowning achievements of Ramsey theory; the lower bound uses a subtle *semirandom* / random greedy construction.

11.2.3 Ramsey Numbers for Graphs

Definition 156. For graphs H_1, H_2 , $R(H_1, H_2)$ is the least N such that every red/blue edge-coloring of K_N contains a red copy of H_1 or a blue copy of H_2 .

Theorem 167 (Chvátal, 1977). For every tree T on t vertices and every $s \geq 2$,

$$R(T, K_s) = (s-1)(t-1) + 1.$$

Proof. Upper bound. Let $N = (s-1)(t-1) + 1$ and color the edges of K_N red/blue with no blue K_s . Let G be the red graph. Every s vertices span a red edge, so $\alpha(G) \leq s-1$ and hence $\chi(G) \geq N/(s-1) > t-1$, i.e. $\chi(G) \geq t$. Since greedy coloring along a degeneracy order gives $\chi(G) \leq 1 + \max_{H \subseteq G} \delta(H)$, some subgraph $H \subseteq G$ has minimum degree at least $t-1$. Such an H contains every tree T on t vertices: embed T one vertex at a time, each new vertex being a leaf attached to an already embedded vertex x ; at most $t-2$ neighbors of x are already used, so a free neighbor exists. Hence G contains a red copy of T .

Lower bound. Partition $(s-1)(t-1)$ vertices into $s-1$ blocks of $t-1$. Color within-block edges red and cross-block edges blue; the red graph is a disjoint union of K_{t-1} 's (no tree on t vertices fits), and the blue graph is $(s-1)$ -partite (no K_s). $\square$

Theorem 168 (Gerencsér–Gyárfás, 1967). For the path P_t on t vertices, $R(P_t, P_s) = t + \lfloor s/2 \rfloor - 1$ for $t \geq s \geq 2$.

Theorem 169 (Bondy–Erdős; Rosta; Faudree–Schelp). For cycles of equal length, $R(C_n, C_n) = 2n-1$ if n is odd ($n \geq 5$), and $R(C_n, C_n) = 3n/2 - 1$ if n is even ($n \geq 6$).

Theorem 170 (Chvátal–Rödl–Szemerédi–Trotter). For every graph H of maximum degree Δ , $R(H, H) \leq c_\Delta \cdot |V(H)|$.

This linear-in- $|V(H)|$ bound for bounded-degree graphs was a landmark 1983 result proved via the regularity lemma.

11.2.4 Hypergraph Ramsey Numbers

Definition 157. $R_k(s, t)$ is the least N such that every red/blue coloring of the k -subsets of $\{1, \dots, N\}$ yields a red s -set (all k -subsets red) or a blue t -set.

Theorem 171 (Erdős–Hajnal stepping-up lemma). Let $k \geq 3$ and $s > k$. If $R_k(s, s) > m$, then

$$R_{k+1}(2s + k - 4, 2s + k - 4) > 2^m.$$

Corollary 45. $R_3(k, k) \geq 2^{ck^2}$ (by a random coloring) and, applying the stepping-up lemma repeatedly, $R_k(s, s) \geq \text{tower}_{k-1}(cs^2)$ for $k \geq 3$, where $\text{tower}_1(x) = x$ and $\text{tower}_{i+1}(x) = 2^{\text{tower}_i(x)}$.

Remark 142. For 3-uniform hypergraphs the known bounds are $2^{ck^2} \leq R_3(k, k) \leq 2^{2^{c'k}}$ (Erdős–Hajnal–Rado); the stepping-up lemma does not apply to the step from graphs to 3-uniform hypergraphs. Erdős, Hajnal, and Rado conjectured that the doubly-exponential upper bound is tight up to the constant in the exponent; this is one of the most fundamental open problems in Ramsey theory.

11.2.5 Schur’s Theorem and van der Waerden

Theorem 172 (Schur, 1916). For every $k \geq 1$ there exists N such that in every k -coloring of $\{1, 2, \dots, N\}$ there is a monochromatic triple a, b, c with $a + b = c$.

Proof. Set $N = R(3, 3, \dots, 3)$ (k colors). Given a k -coloring of $\{1, \dots, N\}$, build an edge-coloring of K_N by coloring edge ij (with $i < j$) by the color of $j - i$. A monochromatic triangle $i < j < \ell$ gives monochromatic numbers $j - i, \ell - j$, and $\ell - i$ with $(j - i) + (\ell - j) = \ell - i$. $\square$

Theorem 173 (van der Waerden, 1927). For every $k \geq 2, \ell \geq 2$ there exists $W(k, \ell)$ such that in every k -coloring of $\{1, 2, \dots, W(k, \ell)\}$ there is a monochromatic arithmetic progression of length ℓ .

Remark 143. Shelah gave primitive-recursive bounds on $W(k, \ell)$; Gowers’s work on Szemerédi’s theorem on arithmetic progressions in dense sets gave

$$W(k, \ell) \leq 2^{2^{k2^{2^{\ell+9}}}},$$

a tower of five exponentials. The best known bounds are still astronomical.

11.2.6 Erdős–Szekeres: Happy Ending

Theorem 174 (Erdős–Szekeres, 1935). Every sequence of more than $(r - 1)(s - 1)$ distinct real numbers contains an increasing subsequence of length r or a decreasing subsequence of length s .

Proof 1: labeling. Let $(a_i)_{i=1}^n$ be the sequence with $n > (r - 1)(s - 1)$. For each i , let x_i be the length of the longest increasing subsequence ending at i , and y_i the longest decreasing. If $i < j$ and $a_i < a_j$ then $x_i < x_j$; if $a_i > a_j$ then $y_i < y_j$. So $i \mapsto (x_i, y_i)$ is injective; if there were no increasing subsequence of length r and no decreasing one of length s , it would map into $\{1, \dots, r - 1\} \times \{1, \dots, s - 1\}$, giving $n \leq (r - 1)(s - 1)$, a contradiction. $\square$

Proof 2: chains and antichains. Write $i \prec j$ if $i < j$ and $a_i < a_j$; this is a partial order on $\{1, \dots, n\}$ whose chains are increasing subsequences and whose antichains are decreasing subsequences. Suppose there is no chain of size r . The indices i with a given value of x_i (the length of the longest chain ending at i , as in Proof 1) form an antichain, since $i \prec j$ forces $x_i < x_j$; so the $n > (r - 1)(s - 1)$ indices split into at most $r - 1$ antichains (Mirsky’s theorem), one of which has at least s elements. (Coloring ij red if $a_i < a_j$ and blue otherwise and quoting Theorem 161 would only give the weaker threshold $n \geq R(r, s)$; for $r = s = 3$ this is 6, not 5.) $\square$

Theorem 175 (Erdős–Szekeres on points in general position). For every $n \geq 3$ there exists $\text{ES}(n)$ such that any $\text{ES}(n)$ points in the plane in general position contain n in convex position.

The asymptotic $\text{ES}(n) = 2^{n(1+o(1))}$ was established by Suk (preprint 2016, published 2017), nearly settling the 1960 conjecture $\text{ES}(n) = 2^{n-2} + 1$ of Erdős and Szekeres and improving their 1935 upper bound $\binom{2n-4}{n-2} + 1$.

11.2.7 Canonical Ramsey and Induced Ramsey

Theorem 176 (Erdős–Rado canonical Ramsey, 1950). For every k there is N such that in any coloring (with arbitrarily many colors) of the edges of K_N , there is a k -subset S such that the coloring of $K_N[S]$ falls into one of four types: monochromatic, rainbow (all distinct), *min-colored* (color determined by $\min\{i, j\}$), or *max-colored*.

Theorem 177 (Induced Ramsey; Deuber, Erdős–Hajnal–Pósa). For every graph H there is a graph G such that in every red/blue edge-coloring of G , there is an induced copy of H in G whose edges are monochromatic.

11.2.8 Exercises

Easy Questions.

Exercise 11.13. Prove that $R(3, 3) = 6$ by exhibiting a two-coloring of K_5 with no monochromatic triangle (Figure 157, left) and arguing $R(3, 3) \leq 6$ directly via pigeonhole on a vertex.

Exercise 11.14. Compute $R(3, 4)$ exactly. (Upper bound: $R(3, 4) \leq R(2, 4) + R(3, 3) = 4 + 6 = 10$; then sharpen to 9.)

Exercise 11.15. Verify the values $R(2, t) = t$ and $R(s, 2) = s$ that anchor the inductive proof of Theorem 161.

Medium Questions.

Exercise 11.16. Suppose $R(s-1, t)$ and $R(s, t-1)$ are both even, and let $N = R(s-1, t) + R(s, t-1) - 1$. Show that in a red/blue coloring of K_N with no red K_s and no blue K_t every vertex would have red degree exactly $R(s-1, t) - 1$, an odd number, contradicting the handshake lemma since N is odd. Conclude that $R(s, t) < R(s-1, t) + R(s, t-1)$ when both addends are even.

Exercise 11.17. Prove Schur’s theorem for three colors by showing $R(3, 3, 3) \leq 17$. (Due to Greenwood and Gleason, 1955, who also gave the 3-coloring of K_{16} without monochromatic triangles that shows $R(3, 3, 3) = 17$; Kalbfleisch and Stanton later showed there are exactly two such colorings.)

Exercise 11.18. Prove the infinite Ramsey theorem—every 2-coloring of the edges of K_∞ has an infinite monochromatic clique—and deduce the finite version (Theorem 161) from it by a compactness argument (König’s lemma).

Exercise 11.19. Let $\chi(G)$ denote chromatic number. Show that $R(K_s, G) \geq (s-1)(\chi(G)-1)+1$ for every graph G , and that $R(K_s, G) \geq (s-1)(|V(G)|-1)+1$ when G is connected (Chvátal–Harary).

Hard Questions.

Exercise 11.20. Use the probabilistic method to show $R(s, t) > (1 - o(1)) t^{(s-1)/2}$ for fixed s and $t \rightarrow \infty$.

Exercise 11.21. Prove Chvátal’s theorem $R(T_n, K_s) = (s-1)(n-1)+1$ for trees, filling in the degeneracy details.

Exercise 11.22. Use the Lovász Local Lemma to show $R(3, k) \geq c \cdot k^2 / \log^2 k$. (A weaker form of Kim’s bound, but significantly sharper than the first-moment lower bound.)

Exercise 11.23. Prove van der Waerden’s theorem $W(2, 3) = 9$: any 2-coloring of $\{1, \dots, 9\}$ has a monochromatic 3-AP, and some 2-coloring of $\{1, \dots, 8\}$ avoids one.

Diagram Questions.

Exercise 11.24. Reproduce the Greenwood–Gleason 3-coloring of K_{16} with no monochromatic triangle. Hint: the nonzero cubes form a subgroup C of index 3 in the multiplicative group $\mathbb{F}_{16}^*$; color the edge xy by the coset of C containing $x + y (= x - y)$. Each color class is a copy of the Clebsch graph.

Exercise 11.25. Draw the blow-up coloring witnessing the Chvátal lower bound $R(T, K_s) > (s-1)(t-1)$: $s-1$ red cliques of size $t-1$ joined blue across blocks. Verify directly that no red T_t and no blue K_s appears.

11.3 The Probabilistic Method

The *probabilistic method*, pioneered by Erdős in the 1940s and 50s, is now the single most pervasive technique in combinatorics. The philosophy: to prove an object with property P exists, sample a random object from an appropriate distribution and show $\Pr[\text{sample has } P] > 0$. In practice this unfolds as four refinable tools: *first moment* (expectation and Markov), *alteration* (modify the random object to fix defects), *second moment* (Chebyshev for sharp thresholds), and *Lovász Local Lemma* (escape dependencies). We then discuss *concentration* via martingales (Azuma), entropy, and *dependent random choice*.

11.3.1 First Moment: Linearity of Expectation

Lemma 119 (Markov’s inequality). For any non-negative random variable X and $t > 0$, $\Pr[X \geq t] \leq \mathbb{E}[X]/t$.

Lemma 120 (First moment principle). If X is a non-negative integer-valued random variable with $\mathbb{E}[X] < 1$ then $\Pr[X = 0] > 0$.

Theorem 178 (Large independent set). Every graph G with n vertices and average degree $d \geq 1$ has an independent set of size at least $n/(d+1)$.

Proof. Pick a uniformly random permutation π of $V(G)$, and let $S = \{v : \pi(v) < \pi(w) \text{ for every } w \in N(v)\}$. Then S is always an independent set: if $uv \in E$ and both $u, v \in S$, both would precede the other in π , absurd.

Each v joins S iff v is the first among the $d(v) + 1$ vertices in $\{v\} \cup N(v)$, so $\Pr[v \in S] = 1/(d(v) + 1)$. By linearity of expectation and Jensen’s inequality,

$$\mathbb{E}[|S|] = \sum_{v \in V} \frac{1}{d(v) + 1} \geq \frac{n}{\frac{1}{n} \sum_v d(v) + 1} = \frac{n}{d + 1}.$$

So some realization has $|S| \geq n/(d + 1)$. □

Corollary 46 (Turán-type lower bound for independence). $\alpha(G) \geq \sum_v 1/(d(v) + 1)$.

Theorem 179 (Hamiltonian paths in tournaments). The expected number of Hamiltonian paths in a tournament on n vertices (edges oriented uniformly at random) is $n!/2^{n-1}$. In particular some tournament on n vertices has at least $n!/2^{n-1}$ Hamiltonian paths (Szele, 1943). (That *every* tournament has a Hamiltonian path is Rédei’s theorem, which needs a different argument: consider a longest directed path.)

Proof. For a fixed ordering $v_1, \dots, v_n$ of vertices, the probability that all $n - 1$ consecutive arcs point forward is $2^{-(n-1)}$. Sum over all $n!$ orderings: expectation equals $n!/2^{n-1}$. □

Theorem 180 (Dominating set; Alon). Every graph G with minimum degree $\delta \geq 1$ has a dominating set of size at most $n(1 + \ln(\delta + 1))/(\delta + 1)$.

Proof. Let $p = \ln(\delta + 1)/(\delta + 1)$. Include each vertex in S independently with probability p . Let $T = \{v \notin S : N(v) \cap S = \emptyset\}$. Then $S \cup T$ is dominating, and

$$\mathbb{E}[|S| + |T|] \leq np + \sum_v (1 - p)^{d(v)+1} \leq np + n(1 - p)^{\delta+1} \leq np + ne^{-p(\delta+1)}.$$

Plugging in p yields the bound. □

11.3.2 The Alteration (Deletion) Method

When the random object has a few more “bad” features than desired, simply delete them.

Theorem 181 (Erdős lower bound for Ramsey, via deletion). $R(k, k) > (1 + o(1)) \frac{k}{e} \cdot 2^{k/2}$.

Proof. Color edges of K_n uniformly. Expected number of monochromatic k -cliques $X = 2 \binom{n}{k} 2^{-\binom{k}{2}}$. Delete one vertex from each; the remainder witnesses $R(k, k) > n - X$. Optimize over n to obtain the bound, improving Theorem 164 by a factor $(1 + o(1))k/e$. □

Theorem 182 (High girth, high chromatic number; Erdős, 1959). For every $k, g \geq 3$ there is a graph G with girth $g(G) \geq g$ and chromatic number $\chi(G) \geq k$.

Proof. Fix $\theta \in (0, 1/g)$ and let $p = n^{\theta-1}$. Sample $G \sim G(n, p)$. The expected number of cycles of length less than g is

$$\sum_{\ell=3}^{g-1} \frac{n!}{(n-\ell)! \cdot 2\ell} p^\ell \leq \sum_{\ell=3}^{g-1} (np)^\ell = o(n),$$

so by Markov with probability $\geq 1/2$ the number of short cycles is $\leq n/2$.

Meanwhile, the probability that G has an independent set of size $\lceil n/k \rceil$ is at most

$$\binom{n}{n/k} (1-p)^{\binom{n/k}{2}} \leq 2^n e^{-pn^2/(3k^2)} \rightarrow 0.$$

So with positive probability both events hold: G has fewer than $n/2$ short cycles and $\alpha(G) < n/k$. Delete one vertex from each short cycle to obtain G' with $|V(G')| \geq n/2$, girth $\geq g$, and $\alpha(G') < n/k \leq 2|V(G')|/k$. Thus $\chi(G') \geq |V(G')|/\alpha(G') > k/2$; rescale. $\square$

Remark 144. No constructive proof of Theorem 182 with comparable parameters was known until the expander-graph constructions of Lubotzky–Phillips–Sarnak in the 1980s.

11.3.3 Second Moment and Paley–Zygmund

Theorem 183 (Chebyshev). For any random variable X and $t > 0$, $\Pr[|X - \mathbb{E}[X]| \geq t] \leq \text{Var}(X)/t^2$.

Lemma 121 (Paley–Zygmund). For non-negative X with $\mathbb{E}[X] > 0$,

$$\Pr[X > 0] \geq \frac{(\mathbb{E}[X])^2}{\mathbb{E}[X^2]} = \frac{1}{1 + \text{Var}(X)/(\mathbb{E}[X])^2}.$$

Theorem 184 (Triangle threshold in $G(n, p)$). As $n \rightarrow \infty$, $G(n, p)$ contains a triangle with probability tending to 0 if $pn \rightarrow 0$, and to 1 if $pn \rightarrow \infty$.

Proof. Let X be the number of triangles. Then $\mathbb{E}[X] = \binom{n}{3} p^3$, so Markov gives $\Pr[X \geq 1] \rightarrow 0$ when $np \rightarrow 0$. For the other direction, $\text{Var}(X) = O(n^3 p^3 + n^4 p^5)$ (the first term from $\sum_T \text{Var}(\mathbf{1}_T)$, the second from pairs of triangles sharing an edge) and $(\mathbb{E}X)^2 = \Theta(n^6 p^6)$, so $\text{Var}(X)/(\mathbb{E}X)^2 = O((np)^{-3} + (n^2 p)^{-1}) \rightarrow 0$ and the Paley–Zygmund ratio tends to 1 when $np \rightarrow \infty$. $\square$

11.3.4 The Lovász Local Lemma

First-moment and alteration arguments fail when dependencies are dense; LLL escapes them.

Definition 158 (Dependency graph). Given events $A_1, \dots, A_m$, a **dependency graph** D has vertex set $[m]$ with A_i mutually independent of $\{A_j : j \notin N_D(i) \cup \{i\}\}$.

Theorem 185 (General LLL; Erdős–Lovász). If there exist $x_1, \dots, x_m \in (0, 1)$ with $\Pr[A_i] \leq x_i \prod_{j \in N_D(i)} (1 - x_j)$, then

$$\Pr\left[\bigcap_i \overline{A_i}\right] \geq \prod_i (1 - x_i) > 0.$$

Theorem 186 (Symmetric LLL). If $\Pr[A_i] \leq p$ for all i , each A_i is mutually independent of all but at most d other events, and $ep(d+1) \leq 1$, then $\Pr[\bigcap_i \overline{A_i}] > 0$.

Proof sketch. Set $x_i = 1/(d+1)$ in Theorem 185 and verify $p \leq 1/(e(d+1))$ using $(1 - 1/(d+1))^d \geq 1/e$. $\square$

Theorem 187 (LLL lower bound for $R(k, k)$). $R(k, k) > (1 + o(1)) \frac{\sqrt{2}}{e} k \cdot 2^{k/2}$.

Proof. Color edges of K_n uniformly. For each k -set S let A_S be “ $K_n[S]$ monochromatic”; $p = 2^{1-\binom{k}{2}}$. Events A_S and $A_{S'}$ are independent unless $|S \cap S'| \geq 2$, so the dependency degree is $d \leq \binom{k}{2} \binom{n-2}{k-2}$. Apply the symmetric LLL and optimize n . $\square$

Theorem 188 (Moser–Tardos, 2010). Under the symmetric LLL hypothesis with events defined by independent variables, the following algorithm terminates in expected polynomial time with an assignment avoiding every A_i :

Sample all variables; while some A_i holds, pick one and resample its variables.

Remark 145. Moser–Tardos turns LLL from a pure existence result into a constructive algorithm. The proof uses entropy compression: a log of resampling events is encoded as a prefix of a fair coin sequence, so too-long executions contradict the entropy of the random tape.

11.3.5 Concentration: Azuma and Talagrand

Definition 159 (Martingale, bounded differences). A sequence $(X_0, \dots, X_n)$ is a **martingale** with respect to $(\mathcal{F}_i)$ if $\mathbb{E}[X_{i+1} \mid \mathcal{F}_i] = X_i$. It has **bounded differences** c_i if $|X_i - X_{i-1}| \leq c_i$ a.s.

Theorem 189 (Azuma–Hoeffding). For a martingale with $|X_i - X_{i-1}| \leq c_i$,

$$\Pr[|X_n - X_0| \geq t] \leq 2 \exp\left(-\frac{t^2}{2 \sum_i c_i^2}\right).$$

Corollary 47 (Lipschitz concentration in $G(n, p)$). Any f on $G(n, p)$ with $|f(G) - f(G')| \leq 1$ when G, G' differ on one edge satisfies

$$\Pr[|f - \mathbb{E}f| \geq t] \leq 2 \exp(-t^2/(2 \binom{n}{2})).$$

Theorem 190 (Talagrand’s inequality, informal). Let $f : \{0, 1\}^n \rightarrow \mathbb{R}$ be 1-Lipschitz and c -certifiable, meaning $f(x) \geq s$ is certified by a subset $I \subseteq [n]$ of size $\leq cs$. Let $X = f(\xi)$ for independent coordinates ξ with median M . Then

$$\Pr[X \leq M - t] \leq 2 \exp(-t^2/(4cM)), \quad \Pr[X \geq M + t] \leq 2 \exp(-t^2/(4c(M + t))).$$

Talagrand’s inequality is the preferred concentration tool for triangle counts and $\chi(G(n, p))$ because the certificate condition is often much sharper than the Lipschitz constant alone would suggest.

11.3.6 Dependent Random Choice

Lemma 122 (Dependent random choice). Let G be a graph on n vertices with average degree d . For integers $a, r, m, t \geq 1$, if $d^t/n^{t-1} - \binom{n}{r}(m/n)^t \geq a$, then G has a subset U with $|U| \geq a$ in which every r -tuple has at least m common neighbors.

Proof sketch. Let T be a sequence of t vertices chosen uniformly at random *with repetition* and $U = \bigcap_{v \in T} N(v)$. Then $\mathbb{E}[|U|] = \sum_v (d(v)/n)^t \geq d^t/n^{t-1}$ by convexity. Expected number of bad r -sets inside U is at most $\binom{n}{r}(m/n)^t$. Delete and apply linearity. $\square$

Corollary 48 (Bipartite Turán via DRC). Every graph on n vertices with at least $2n^{2-1/t}$ edges contains $K_{t,t}$. (Theorem 154 gives the same conclusion already with $n^{2-1/t}$ edges.)

Dependent random choice powers modern proofs of Ramsey-type statements for sparse graphs, the Balog–Szemerédi–Gowers theorem, and refined bipartite Turán bounds.

11.3.7 The Entropy Method

Definition 160 (Shannon entropy). For a discrete random variable X , $H(X) = -\sum_x \Pr[X = x] \log \Pr[X = x]$. For jointly distributed (X, Y) , the conditional entropy is $H(X | Y) = \mathbb{E}_y H(X | Y = y)$.

Theorem 191 (Shearer’s lemma). If $\mathcal{F}$ is a collection of subsets of $[n]$ with each element covered at least k times, then for any joint distribution on $(X_1, \dots, X_n)$,

$$k \cdot H(X_1, \dots, X_n) \leq \sum_{F \in \mathcal{F}} H(X_F).$$

Corollary 49 (Kruskal–Katona-type triangle bound). A graph with m edges has at most $\frac{1}{6}(2m)^{3/2}$ triangles.

11.3.8 Exercises

Easy Questions.

Exercise 11.26. Using linearity of expectation, show that every graph on m edges has a bipartite subgraph with at least $m/2$ edges, and a k -partite subgraph with at least $(1 - 1/k)m$ edges.

Exercise 11.27. Show that every tournament on $n \geq 2$ vertices has a dominating set of size at most $\lceil \log_2 n \rceil$. (Hint: by averaging, some vertex beats at least $\lceil (n-1)/2 \rceil$ others; take it, discard it and the vertices it beats, and repeat.)

Exercise 11.28. Compute the variance of the triangle count X in $G(n, p)$ for $p = 1/2$ and verify $\text{Var}(X) = O(n^4)$ while $\mathbb{E}[X] = \Theta(n^3)$.

Medium Questions.

Exercise 11.29. Use the vertex-exposure martingale and Azuma’s inequality to show that $\chi(G(n, 1/2))$ is concentrated within $O(\sqrt{n \log n})$ of its median.

Exercise 11.30. Prove that some tournament on n vertices contains at least $n!/2^{n-1}$ Hamiltonian paths (Szele), and show that the transitive tournament contains exactly one, so “some” cannot be replaced by “every”. (Alon showed, using Brégman’s permanent inequality, that no tournament has more than $O(n^{3/2} n!/2^n)$ Hamiltonian paths.)

Exercise 11.31. Identify the regime $p = p(n)$ where the triangle count X in $G(n, p)$ satisfies $X/\mathbb{E}[X] \rightarrow 1$ in probability via Chebyshev’s inequality (Theorem 183).

Hard Questions.

Exercise 11.32. Use LLL to show that every graph G of maximum degree Δ has a *proper* edge-colouring with at most $C \cdot \Delta$ colours in which the union of any two colour classes is *acyclic* (no bichromatic cycle). (This is the acyclic edge-colouring theorem of Alon, McDiarmid, and Reed; $C = 64$ suffices.)

Exercise 11.33. State and prove the *Lopsided* LLL: the dependency condition may be relaxed to a bound on $\Pr[A_i | \bigcap_{j \in J} \overline{A_j}]$ for J disjoint from the dependency neighborhood of i .

Exercise 11.34. Use dependent random choice (Lemma 122) to prove: every n -vertex graph with $\geq 2n^{2-1/t}$ edges contains $K_{t,t}$.

Exercise 11.35. Use Shearer’s entropy lemma to bound the number of labeled copies of C_4 in a graph with m edges.

Exercise 11.36. Prove the second-moment concentration for subgraph counts: if H has v vertices and e edges, $m^*(H) = \max_{H' \subseteq H} |E(H')|/|V(H')|$, and $np^{m^*(H)} \rightarrow \infty$, then the number X_H of copies of H in $G(n, p)$ satisfies $X_H/\mathbb{E}[X_H] \rightarrow 1$ in probability. (For balanced H , i.e. $m^*(H) = e/v$, the condition is $np^{e/v} \rightarrow \infty$; show by an example such as K_4 plus a pendant edge that $np^{e/v} \rightarrow \infty$ alone does not suffice.)

Diagram Questions.

Exercise 11.37. Draw the dependency graph D for the events “the 4-set S is monochromatic” as S ranges over $\binom{[6]}{4}$ in the random 2-coloring of $E(K_6)$. Estimate the dependency degree d and check whether the symmetric LLL applies.

Exercise 11.38. For a triangle-free graph G found by the deletion method on $G(n, c/n)$ with $c = 5$, sketch (schematically) the typical local structure: degree distribution, isolated vertices, short cycles, and the surviving sparse independent-set witness for high chromatic number.

11.4 Spectral Graph Theory

Spectral graph theory studies the eigenvalues and eigenvectors of matrices associated with a graph—primarily the adjacency matrix A and the Laplacian L . The central thesis is that these spectra encode deep combinatorial information: connectivity, expansion, bipartiteness, walk-counting, mixing times of random walks, and the asymptotic complexity of many combinatorial algorithms.

11.4.1 Matrices of a Graph

Definition 161 (Adjacency and Laplacian matrices). Let $G = (V, E)$ be a graph on $V = \{1, \dots, n\}$.

- The **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ has $A_{ij} = 1$ iff $ij \in E$.
- The **degree matrix** D is diagonal with $D_{ii} = d(i)$.
- The **combinatorial Laplacian** is $L = D - A$.
- The **normalized Laplacian** is $\mathcal{L} = I - D^{-1/2}AD^{-1/2}$ (where degrees are positive).
- The **signless Laplacian** is $Q = D + A$.

All these matrices are symmetric (when G is undirected), hence have real spectra and an orthonormal eigenbasis.

Definition 162 (Spectrum notation). For the adjacency matrix we write $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. For the Laplacian, $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n$.

Theorem 192 (Spectral theorem). Any real symmetric $n \times n$ matrix M admits an orthonormal eigenbasis $v_1, \dots, v_n$ with real eigenvalues $\tau_1, \dots, \tau_n$; equivalently $M = \sum_i \tau_i v_i v_i^\top$.

11.4.2 Walk Counting and Basic Identities

Proposition 193 (Walk counting). For any $k \geq 0$, $(A^k)_{ij}$ equals the number of walks of length k from i to j .

Corollary 50 (Traces). $\text{tr}(A^2) = 2|E|$, $\text{tr}(A^3) = 6 \cdot (\# \text{triangles in } G)$.

Proposition 194 (Laplacian quadratic form). For any $x \in \mathbb{R}^n$,

$$x^\top Lx = \sum_{ij \in E} (x_i - x_j)^2 \geq 0.$$

Hence L is positive semidefinite, all $\mu_i \geq 0$, and $\mathbf{1}$ is always a 0-eigenvector.

Theorem 195 (Multiplicity of 0). The multiplicity of the eigenvalue 0 of L equals the number of connected components of G .

Proof. If $x^\top Lx = 0$ then x is constant on every connected component by Proposition 194. Conversely, the indicator vectors of the components form a 0-eigenspace basis. $\square$

Proposition 196 (Regular case). If G is d -regular, then $L = dI - A$, so $\mu_i = d - \lambda_i$. In particular $\lambda_1 = d$ and $\lambda_n \geq -d$.

Theorem 197 (Bipartiteness and $\lambda_n = -d$). A connected d -regular graph G is bipartite if and only if $\lambda_n = -d$. In general, the multiplicity of $-d$ equals the number of bipartite connected components.

11.4.3 The Min-Max Principle and Rayleigh Quotient

Definition 163 (Rayleigh quotient). For a symmetric matrix M and $x \neq 0$, the **Rayleigh quotient** is $R_M(x) = x^\top Mx / x^\top x$.

Theorem 198 (Courant–Fischer). Let M be symmetric with eigenvalues $\tau_1 \geq \dots \geq \tau_n$. Then

$$\tau_k = \max_{\dim U=k} \min_{x \in U, x \neq 0} R_M(x) = \min_{\dim U=n-k+1} \max_{x \in U, x \neq 0} R_M(x).$$

Corollary 51 (Interlacing). Let M be a symmetric $n \times n$ matrix and M' any principal submatrix of order m . Then the eigenvalues $\tau'_1 \geq \dots \geq \tau'_m$ of M' interlace those of M : $\tau_{i+n-m}(M) \leq \tau'_i(M') \leq \tau_i(M)$.

Corollary 52 (Cauchy interlacing for graphs). If H is an induced subgraph of G , the adjacency eigenvalues of H interlace those of G .

11.4.4 Perron–Frobenius

Theorem 199 (Perron–Frobenius). Let $M \geq 0$ be a non-negative irreducible matrix. Then:

1. The spectral radius $\rho(M)$ is a simple eigenvalue.
2. There exists a strictly positive eigenvector $v > 0$ for $\rho(M)$.
3. $\rho(M) \geq |\tau|$ for every eigenvalue τ ; if h is the period of M 's underlying digraph, the eigenvalues with $|\tau| = \rho(M)$ are exactly $\rho(M) e^{2\pi i k/h}$, $k = 0, 1, \dots, h-1$.

Corollary 53. For a connected graph G , $\lambda_1(A) = \rho(A)$ is simple and its eigenvector has strictly positive entries.

Corollary 54. $\lambda_1(A(G)) \geq \bar{d} = 2|E|/n$ (average degree), with equality iff G is regular.

Proof. Take $x = \mathbf{1}$. Then $R_A(\mathbf{1}) = \bar{d} \leq \lambda_1$ by Courant–Fischer; equality forces $\mathbf{1}$ to be a λ_1 -eigenvector, i.e. G regular. Conversely, if G is d -regular then $\lambda_1 = d = \bar{d}$. $\square$

11.4.5 The Matrix-Tree Theorem

Theorem 200 (Kirchhoff, 1847). Let G be a connected graph. The number of spanning trees of G equals any cofactor of L : delete any row i and column i , and take the determinant of the resulting $(n-1) \times (n-1)$ matrix. Equivalently,

$$\tau(G) = \frac{1}{n} \mu_2 \mu_3 \cdots \mu_n.$$

Proof sketch via Cauchy–Binet. Orient the edges of G arbitrarily and define the *signed incidence matrix* $B \in \{-1, 0, 1\}^{n \times m}$ with $B_{ve} = \pm 1$ depending on whether v is the tail or head of e . Then $L = BB^\top$. The reduced incidence matrix $\tilde{B}$ (obtained by deleting row i) has $\det(\tilde{B}\tilde{B}^\top)$ equal by Cauchy–Binet to the sum over $(n-1)$ -subsets S of edges of $(\det \tilde{B}_S)^2$. But $\det \tilde{B}_S = \pm 1$ if S is a spanning tree, 0 otherwise; so the cofactor is the spanning tree count. $\square$

Corollary 55 (Cayley's formula). The complete graph K_n has n^{n-2} spanning trees.

Proof. $L(K_n) = nI - J$ (where J is all-ones). Its eigenvalues are 0 (once) and n ($n-1$ times), so $\tau(K_n) = \frac{1}{n} \cdot n^{n-1} = n^{n-2}$. $\square$

11.4.6 Algebraic Connectivity and Cheeger's Inequality

Definition 164 (Algebraic connectivity and Cheeger constant). The **algebraic connectivity** of G is μ_2 . The **edge expansion** (or *Cheeger constant*) is

$$h(G) = \min_{\emptyset \neq S \subsetneq V, |S| \leq n/2} \frac{|E(S, V \setminus S)|}{|S|}.$$

Theorem 201 (Discrete Cheeger inequality). For a d -regular graph G ,

$$\frac{\mu_2}{2} \leq h(G) \leq \sqrt{2d\mu_2}.$$

Proof of the easy direction. Let S be the minimizer and let $\mathbf{1}_S$ be its indicator. The vector $x = \mathbf{1}_S - \frac{|S|}{n}\mathbf{1}$ is orthogonal to $\mathbf{1}$, so by Courant–Fischer

$$\mu_2 \leq \frac{x^\top Lx}{x^\top x} = \frac{|E(S, V \setminus S)|}{|S|(1 - |S|/n)} \leq \frac{2|E(S, V \setminus S)|}{|S|} = 2h(G).$$

□

Proof of the hard direction, sketch. Let v_2 be the Fiedler eigenvector of L . Sort the vertices by v_2 -coordinate and consider the $n - 1$ threshold cuts $S_k = \{v_{(1)}, \dots, v_{(k)}\}$. Show that the minimum Cheeger ratio over all S_k is at most $\sqrt{2d\mu_2}$. This is the spectral analogue of the Federer–Fleming isoperimetric trick. □

11.4.7 Expander Graphs

Definition 165 (Expander). An infinite family $\{G_n\}$ of d -regular graphs is a **one-sided spectral expander** if $\lambda_2(G_n) \leq \lambda$ for a fixed $\lambda < d$, and a **two-sided spectral expander** if $\max\{\lambda_2(G_n), -\lambda_n(G_n)\} \leq \lambda$ for a fixed $\lambda < d$ (each G_n is then a λ -expander). It is a **combinatorial expander** if $h(G_n) \geq c > 0$. By Cheeger (Theorem 201), being a combinatorial expander is equivalent to being a one-sided spectral expander, up to quantitative parameters; two-sided expansion is strictly stronger (a bipartite graph has $\lambda_n = -d$ but may have large h).

Theorem 202 (Expander Mixing Lemma). Let G be d -regular on n vertices with $|\lambda| = \max(|\lambda_2|, |\lambda_n|)$. For all $S, T \subseteq V$ (where $|E(S, T)|$ counts the pairs $(s, t) \in S \times T$ with $st \in E$, so that $|E(S, T)| = \mathbf{1}_S^\top A \mathbf{1}_T$),

$$\left| |E(S, T)| - \frac{d|S||T|}{n} \right| \leq |\lambda| \sqrt{|S||T|}.$$

Proof. Write $\mathbf{1}_S = \alpha \mathbf{1} + x$ and $\mathbf{1}_T = \beta \mathbf{1} + y$ with $\alpha = |S|/n, \beta = |T|/n$ and $x, y \perp \mathbf{1}$. Then

$$\mathbf{1}_S^\top A \mathbf{1}_T = |E(S, T)| = \alpha\beta \cdot nd + x^\top A y.$$

The second term is bounded by $|\lambda| \|x\| \|y\| \leq |\lambda| \sqrt{|S||T|}$. □

Theorem 203 (Alon–Boppana, 1986). For every d -regular graph on n vertices,

$$\lambda_2(G) \geq 2\sqrt{d-1} - o_n(1).$$

Definition 166 (Ramanujan graph). A d -regular graph G is **Ramanujan** if $\max(\lambda_2, -\lambda_n) \leq 2\sqrt{d-1}$. A bipartite d -regular graph has $\lambda_n = -d$, so for $d \geq 3$ it is never Ramanujan in this sense; it is called a **bipartite Ramanujan graph** if $|\lambda_i| \leq 2\sqrt{d-1}$ for $2 \leq i \leq n-1$.

Theorem 204 (Lubotzky–Phillips–Sarnak; Margulis, 1988). For every prime $p \equiv 1 \pmod{4}$, there is an explicit infinite family of $(p+1)$ -regular Ramanujan graphs.

Theorem 205 (Marcus–Spielman–Srivastava, 2015). For every $d \geq 3$ there is an infinite family of d -regular bipartite Ramanujan graphs.

Marcus–Spielman–Srivastava introduced the method of *interlacing families* of polynomials to prove this: they showed that every d -regular graph has a 2-lift (a double cover) whose new eigenvalues are all at most $2\sqrt{d-1}$, a weak form of a conjecture of Bilu and Linial, and applied such lifts repeatedly starting from $K_{d,d}$. Shortly afterwards they used the same method to solve the Kadison–Singer problem. In a sequel they showed that d -regular bipartite Ramanujan multigraphs exist on $2n$ vertices for every $n \geq 1$ and every $d \geq 3$.

11.4.8 Random Walks and Mixing

Definition 167 (Random walk matrix). The **transition matrix** of the simple random walk on G is $P = D^{-1}A$. For a d -regular graph, $P = A/d$ and its spectrum is $\{\lambda_i/d\}$.

Theorem 206 (Mixing time bound). Let G be a connected non-bipartite d -regular graph and $\lambda_* = \max(|\lambda_2|, |\lambda_n|) < d$. The simple random walk has total variation mixing time

$$t_{\text{mix}}(\varepsilon) \leq \frac{\log(n/\varepsilon)}{1 - \lambda_*/d}.$$

Proof sketch. The stationary distribution is uniform. The second-largest absolute eigenvalue λ_*/d of P governs the convergence: by the spectral decomposition P^t converges to $\mathbf{1}\mathbf{1}^\top/n$ at rate $(\lambda_*/d)^t \leq e^{-t(1-\lambda_*/d)}$. The $\log(n/\varepsilon)$ factor absorbs the $L^2 \rightarrow L^1$ conversion. (The one-sided gap $1 - \lambda_2/d$ does not suffice: if λ_n is close to $-d$ the walk nearly alternates between two sides. For the lazy walk $(I + P)/2$, whose eigenvalues $(1 + \lambda_i/d)/2$ are non-negative, one may use $1 - \lambda_2/d$ at the cost of a factor 2.) $\square$

11.4.9 Hoffman's Bound and Chromatic Number

Theorem 207 (Hoffman's bound). For any graph G with at least one edge,

$$\chi(G) \geq 1 - \frac{\lambda_1(A)}{\lambda_n(A)}.$$

Proof sketch. Suppose G admits a proper k -coloring with color classes $C_1, \dots, C_k$. Let $y = (y_1, \dots, y_k)$ optimize the quadratic form $y^\top B y / y^\top y$ for B a $k \times k$ auxiliary matrix built from block-averaged eigenvectors. An interlacing argument applied to A restricted to a color class (an independent set) gives the bound. $\square$

Theorem 208 (Independence number bound). For a regular graph with at least one edge,

$$\alpha(G) \leq \frac{-\lambda_n}{\lambda_1 - \lambda_n} \cdot n.$$

11.4.10 Strongly Regular Graphs

Definition 168 (Strongly regular). A graph G on n vertices, neither complete nor edgeless, is **strongly regular** with parameters (n, k, λ, μ) if it is k -regular, every pair of adjacent vertices has λ common neighbors, and every pair of non-adjacent vertices has μ common neighbors.

Theorem 209 (Spectrum of SRG). A connected strongly regular graph with parameters (n, k, λ, μ) has exactly three adjacency eigenvalues: k with multiplicity 1, and

$$r, s = \frac{1}{2} \left((\lambda - \mu) \pm \sqrt{(\lambda - \mu)^2 + 4(k - \mu)} \right),$$

with multiplicities determined by k, r, s, n .

Strongly regular graphs are a central playground for algebraic graph theory, connecting to finite geometries (projective planes, generalized quadrangles), coding theory, and the classification of finite simple groups.

Example 48 (Petersen graph spectrum). The Petersen graph P is the strongly regular graph with parameters $(10, 3, 0, 1)$: 3-regular, no triangles, and any two non-adjacent vertices share exactly one common neighbor. By Theorem 209 its adjacency spectrum is $\{3^{(1)}, 1^{(5)}, (-2)^{(4)}\}$ (eigenvalues with multiplicities). The Hoffman bound gives $\chi(P) \geq 1 - 3/(-2) = 5/2$, so $\chi(P) \geq 3$; in fact $\chi(P) = 3$.

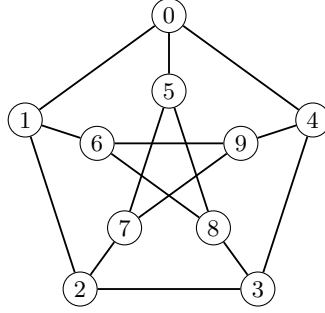


Figure 158: The Petersen graph; spectrum $\{3, 1, 1, 1, 1, -2, -2, -2, -2\}$.

11.4.11 Exercises

Easy Questions.

Exercise 11.39. Compute the adjacency spectrum of K_n , $K_{n,n}$, C_n , and the Petersen graph.

Exercise 11.40. Use the Matrix-Tree theorem to count spanning trees of $K_{m,n}$. Answer: $m^{n-1}n^{m-1}$.

Exercise 11.41. Prove that a graph is bipartite iff its adjacency spectrum is symmetric about 0.

Medium Questions.

Exercise 11.42. For the cycle C_n , compute the Laplacian eigenvalues explicitly as $2 - 2\cos(2\pi k/n)$ for $k = 0, 1, \dots, n-1$. Identify μ_2 and comment on how it scales with n .

Exercise 11.43. Prove that the hypercube $Q_n = K_2^{\square n}$ has Laplacian spectrum $\{2k : k = 0, 1, \dots, n\}$ with multiplicity $\binom{n}{k}$.

Exercise 11.44. Prove Hoffman's theorem: $\chi(G) \geq 1 - \lambda_1/\lambda_n$, by applying interlacing to any independent-set partition.

Exercise 11.45. Show that the proof of the expander mixing lemma actually gives $||E(S, T) - d|S||T|/n| \leq |\lambda| \sqrt{|S|(1 - |S|/n)|T|(1 - |T|/n)}$, and apply it with $T = V \setminus S$ to prove the lower bound $h(G) \geq (d - |\lambda|)/2$. Explain why the expander mixing lemma cannot give Cheeger's upper bound $h(G) \leq \sqrt{2d\mu_2}$.

Hard Questions.

Exercise 11.46. Prove Theorem 209 on the three-eigenvalue spectrum of a strongly regular graph.

Exercise 11.47. Show that if G is a d -regular λ -expander, then the diameter of G satisfies $\text{diam}(G) \leq \lceil \log_{d/\lambda} n \rceil$.

Exercise 11.48. Prove the Alon-Boppana bound $\lambda_2 \geq 2\sqrt{d-1} - o(1)$ by a trace / walk-counting argument on the d -regular infinite tree.

Diagram Questions.

Exercise 11.49. Draw the Fiedler eigenvector v_2 of the path P_6 as a colored sequence (positive on one end, negative on the other) and use the threshold-cut sweep on v_2 to recover the bisection $\{1, 2, 3\}$ vs $\{4, 5, 6\}$. Verify directly that this is the Cheeger optimal cut.

Exercise 11.50. Draw the Petersen graph (Figure 158) with a proper 3-coloring of its vertices, and verify that no smaller proper coloring exists by combining the Hoffman bound with a parity argument on the outer pentagon.

11.5 Random Graphs

Random graph theory, initiated by Erdős and Rényi in their landmark 1959–61 papers, studies the typical behavior of graphs sampled from a distribution. The discovery of *sharp thresholds* and *phase transitions* was both mathematically startling and enormously influential: the same phenomena appear in percolation, spin glasses, random matrices, and the complexity of random constraint satisfaction.

11.5.1 Random Graph Models

Definition 169 (Erdős–Rényi model). $G(n, p)$ is the probability space of graphs on n labeled vertices in which each of the $\binom{n}{2}$ possible edges appears independently with probability $p = p(n)$.

Definition 170 (Uniform model). $G(n, M)$ is the uniform distribution over graphs on n labeled vertices having exactly M edges. The models $G(n, p)$ and $G(n, M)$ with $M \approx p\binom{n}{2}$ are “essentially equivalent” for most purposes via conditioning.

Definition 171 (Random regular model). $G_{n,d}$ is the uniform distribution over d -regular graphs on n labeled vertices. Sampling is non-trivial but well-approximated by the *configuration model*.

Definition 172 (Threshold function). A function $t(n)$ is a **threshold** for a monotone graph property P if

$$\Pr[G(n, p) \models P] \rightarrow \begin{cases} 0 & \text{if } p/t(n) \rightarrow 0, \\ 1 & \text{if } p/t(n) \rightarrow \infty. \end{cases}$$

Theorem 210 (Bollobás–Thomason, 1987). Every non-trivial monotone graph property has a threshold function (unique up to constants).

Proof sketch. Let $q(n, p)$ be the probability that $G(n, p)$ has P . Monotonicity of P and a coupling argument show q is non-decreasing in p (it is a polynomial increasing from 0 to 1). Define $t(n)$ by $q(n, t(n)) = 1/2$. The union of k independent copies of $G(n, p)$ is distributed as $G(n, 1 - (1 - p)^k)$, which is contained in $G(n, kp)$ under a coupling. With $p = t(n)$ this gives $q(n, kt(n)) \geq 1 - 2^{-k} \geq 1 - \delta$ once $k \geq \log_2(1/\delta)$. With $p = t(n)/k$ it gives $1/2 = q(n, t(n)) \geq 1 - (1 - q(n, t(n)/k))^k$, so $q(n, t(n)/k) \leq 1 - 2^{-1/k} \leq (\ln 2)/k \leq \delta$ once $k \geq 1/\delta$. $\square$

11.5.2 Subgraph Thresholds

Definition 173 (Density and balanced graph). The **density** of H is $m(H) = |E(H)|/|V(H)|$. The maximum density of a subgraph of H is $m^*(H) = \max_{H' \subseteq H} m(H')$. A graph H is **balanced** if $m(H) = m^*(H)$ and **strictly balanced** if $m(H') < m(H)$ for all proper subgraphs H' .

Theorem 211 (Bollobás, 1981). For every fixed graph H with at least one edge, the property “ H is a subgraph of G ” has threshold $t(n) = n^{-1/m^*(H)}$.

Proof sketch. Let H' be the densest subgraph of H . The expected number of copies of H' in $G(n, p)$ is $\Theta(n^{|V(H')|} p^{|E(H')|})$. This tends to 0 when $p \ll n^{-1/m(H')}$ and to ∞ otherwise. First-moment gives the 0-statement; second-moment (a moderately involved variance computation) gives the 1-statement. $\square$

Example 49.

- The threshold for containing K_3 is $p = 1/n$.
- The threshold for containing K_4 is $p = n^{-2/3}$.
- The threshold for containing any tree on k vertices is $p = n^{-k/(k-1)}$.
- The threshold for containing a Hamilton cycle is $p = \log n/n$ —the same as connectivity.

11.5.3 Degree Distribution

Theorem 212 (Poissonization of low degrees). For $p = c/n$ with $c > 0$ fixed, the degree sequence of $G(n, p)$ converges in distribution to i.i.d. $\text{Poisson}(c)$: for any fixed k , the number of vertices of degree k is $(1 + o(1))ne^{-c}c^k/k!$ with high probability.

Corollary 56 (Maximum degree). For $p = c/n$ with $c > 0$ fixed, $\Delta(G(n, p)) = (1 + o(1))\log n / \log \log n$ whp.

11.5.4 Threshold for Connectivity

Theorem 213 (Erdős–Rényi, 1959). Let $p = (\log n + c_n)/n$. Then

$$\Pr[G(n, p) \text{ is connected}] \rightarrow \begin{cases} 0 & \text{if } c_n \rightarrow -\infty, \\ e^{-e^{-c}} & \text{if } c_n \rightarrow c \in \mathbb{R}, \\ 1 & \text{if } c_n \rightarrow \infty. \end{cases}$$

Proof sketch. Isolated vertices. Let X_0 count isolated vertices. $\mathbb{E}[X_0] = n(1-p)^{n-1} = e^{\log n - pn + O(\log^2 n/n)} = e^{-c_n + o(1)}$. More generally, for each fixed r the factorial moment

$$\mathbb{E}[X_0(X_0 - 1) \cdots (X_0 - r + 1)] = n(n-1) \cdots (n-r+1) (1-p)^{r(n-r)+\binom{r}{2}}$$

tends to e^{-rc} , so by the method of moments X_0 is asymptotically $\text{Poisson}(e^{-c})$; in particular $\Pr[X_0 = 0] \rightarrow e^{-e^{-c}}$.

Connected iff no isolated. Show that whp at $p \sim \log n/n$ the only obstruction to connectivity is isolated vertices: a component of size $2 \leq k \leq n/2$ must contain a spanning tree and send no edges to the other $n-k$ vertices, and by a union bound over k the probability of such a component tends to 0. $\square$

Corollary 57 (Komlós–Szemerédi; Bollobás). The thresholds for connectivity and Hamilton cycle coincide at $p = \log n/n$.

11.5.5 The Giant Component Phase Transition

Theorem 214 (Erdős–Rényi giant component). Set $p = c/n$. Let $C_{\max}(G(n, p))$ denote the largest component. As $n \rightarrow \infty$, with high probability:

- **Subcritical** ($c < 1$): $|C_{\max}| = \Theta(\log n)$, and all components have size $O(\log n)$.
- **Critical** ($c = 1$): $|C_{\max}| = \Theta(n^{2/3})$.
- **Supercritical** ($c > 1$): There is a unique giant component of size $(1 + o(1))\rho_c \cdot n$, where ρ_c is the unique positive root of $1 - \rho = e^{-c\rho}$, and all other components have size $O(\log n)$.

Proof sketch via branching processes. Explore the component of a fixed vertex v by breadth-first search. At each step, when exploring a vertex, its number of new neighbors is approximately $\text{Binomial}(n-k, p) \approx \text{Poisson}(c)$ for $k = o(n)$. This BFS approximates a Galton–Watson branching process with offspring $\text{Poisson}(c)$. When $c < 1$ the branching process dies a.s. in finite time (with geometric tail), giving $O(\log n)$ component size. When $c > 1$, with probability ρ_c the process survives, yielding a macroscopic component.

The concentration/uniqueness of the giant is proved by second-moment methods: showing two “branches” of size $\gtrsim n$ must eventually intersect. $\square$

Remark 146 (Critical window). *Bollobás and Luczak identified the critical window $p = (1 + \lambda n^{-1/3})/n$, inside which $|C_{\max}|$ is of order $n^{2/3}$. Aldous (1997) proved that $|C_{\max}|/n^{2/3}$ (indeed the whole sequence of rescaled component sizes) converges in distribution, describing the limit through excursions of a Brownian motion with parabolic drift; the continuum (metric-space) limit of the components themselves was obtained by Addario-Berry, Broutin and Goldschmidt (2012).*

11.5.6 Chromatic Number of $G(n, p)$

Theorem 215 (Bollobás, 1988). For $p = 1/2$ fixed, $\chi(G(n, 1/2)) = (1 + o(1)) \frac{n}{2 \log_2 n}$.

Proof sketch. The lower bound uses $\chi \geq n/\alpha$ together with $\alpha(G(n, 1/2)) \sim 2 \log_2 n$ whp. The matching upper bound requires a clever greedy algorithm (“Bollobás’s martingale”) that iteratively removes independent sets of size near the maximum and applies Azuma’s inequality for concentration. $\square$

Theorem 216 (Shamir–Spencer, 1987; Łuczak, 1991). Let $\alpha > 5/6$ be fixed and $p = n^{-\alpha}$. Then there exist integer-valued sequences a_n such that $\chi(G(n, p)) \in \{a_n, a_n + 1\}$ whp.

Alon and Krivelevich (1997) extended this two-point concentration to every fixed $\alpha > 1/2$. For dense random graphs nothing of the kind holds: Heckel (2021) showed that $\chi(G(n, 1/2))$ is, for infinitely many n , not concentrated on fewer than $n^{1/4-\varepsilon}$ consecutive values.

11.5.7 Hamiltonicity

Theorem 217 (Pósa, 1976; Komlós–Szemerédi, 1983). The threshold for $G(n, p)$ to contain a Hamilton cycle is $p = (\log n + \log \log n + c_n)/n$. Specifically, if $c_n \rightarrow \infty$, $G(n, p)$ is Hamiltonian whp; if $c_n \rightarrow -\infty$, it is not.

Remark 147. *The proof introduces Pósa’s rotation–extension technique, which iteratively rotates a non-extendable path to find new endpoints, combined with a “no small bad sets” lemma via expansion.*

11.5.8 Random Regular Graphs

Theorem 218 (Wormald, 1999). For any fixed $d \geq 3$, the random d -regular graph $G_{n,d}$ is Hamiltonian whp, has chromatic number 3 whp for $d = 3$ (by Brooks’ theorem, since $G_{n,3}$ is whp connected and contains an odd cycle), with $\chi(G_{n,d}) \sim d/(2 \log d)$ as d grows, and has spectral gap bounded away from zero whp.

Theorem 219 (Friedman, 2008). For every $\varepsilon > 0$ and any fixed $d \geq 3$, the second-largest absolute eigenvalue of $G_{n,d}$ satisfies

$$|\lambda|(G_{n,d}) \leq 2\sqrt{d-1} + \varepsilon \quad \text{whp.}$$

In particular, random regular graphs are “asymptotically Ramanujan”.

11.5.9 Preferential Attachment and Scale-Free Models

Definition 174 (Barabási–Albert model). Start with a small seed graph. At each time step, add a new vertex v_t with m edges; each new edge attaches to an existing vertex u with probability proportional to $d(u)$.

Theorem 220 (Bollobás–Riordan–Spencer–Tusnády, 2001). In the BA model with $m \geq 1$, the fraction of vertices with degree exactly k converges in probability to $\alpha_{m,k} = 2m(m+1)/(k(k+1)(k+2))$ for each fixed $k \geq m$. Since $\alpha_{m,k} \sim 2m(m+1)k^{-3}$ as $k \rightarrow \infty$, this is a power-law (*scale-free*) degree distribution with exponent 3.

11.5.10 Small Subgraph Central Limit Theorem

Theorem 221 (Ruciński, 1988). Let H be a fixed graph with at least one edge and let $X_H = X_H(n, p)$ count copies of H in $G(n, p)$. If $p n^{1/m^*(H)} \rightarrow \infty$ (with $m^*(H)$ the maximum subgraph density) and $n^2(1-p) \rightarrow \infty$, then (and only then)

$$\frac{X_H - \mathbb{E}[X_H]}{\sqrt{\text{Var}(X_H)}} \xrightarrow{d} \mathcal{N}(0, 1).$$

11.5.11 Exercises

Easy Questions.

Exercise 11.51. Prove that the threshold for $G(n, p)$ to contain a triangle is $p = 1/n$: show $\mathbb{E}[\#K_3] = \binom{n}{3}p^3$ and apply Markov's inequality (first moment) for the 0-statement, and use Chebyshev's inequality (second moment) for the 1-statement.

Exercise 11.52. Compute the threshold for the appearance of C_4 in $G(n, p)$. Answer: $p = n^{-1}$ (using $m^*(C_4) = 1$).

Exercise 11.53. Prove that the random 3-regular graph is connected whp.

Medium Questions.

Exercise 11.54. Prove that $\alpha(G(n, 1/2)) = (2 + o(1)) \log_2 n$ whp. (First moment for the upper bound; second moment / alteration for the matching lower bound.)

Exercise 11.55. Prove Theorem 210 (Bollobás–Thomason) assuming monotonicity, using the fact that the union of k independent copies of $G(n, p)$ is distributed as $G(n, 1 - (1 - p)^k)$.

Exercise 11.56. Prove that for $p = 1/n$ (critical), the expected number of *tree* components of size k in $G(n, p)$ is asymptotically $n \cdot k^{k-2}e^{-k}/k!$. (Use Cayley's formula.)

Hard Questions.

Exercise 11.57. Prove that if $pn^{2/3} \rightarrow \infty$ and $n^2(1 - p) \rightarrow \infty$, the number of triangles in $G(n, p)$ is asymptotically normal. (Variance computation plus Stein's method or the method of moments. By Ruciński's theorem, $np \rightarrow \infty$ and $n^2(1 - p) \rightarrow \infty$ already suffice.)

Exercise 11.58. Show that for $p = c/n$ with $c > 1$, the distance between two uniformly random vertices of the giant component of $G(n, p)$ is $(1 + o(1)) \log n / \log c$ whp. (The diameter of the giant is larger: Fernholz–Ramachandran and Riordan–Wormald showed that it is $(1 + o(1))(\log n / \log c + 2 \log n / \log(1/c_*))$ whp, where $c_* < 1$ satisfies $c_* e^{-c_*} = c e^{-c}$.)

Exercise 11.59. Let $p = c/n$ with $c > 1$. Show that the fraction of vertices in components of size $\leq k$ converges to an explicit value involving the Borel distribution.

Exercise 11.60. Use Azuma's inequality on the vertex-exposure martingale to show $\chi(G(n, p))$ is concentrated within $O(\sqrt{n \log n})$ of its median, for any p .

Diagram Questions.

Exercise 11.61. Sketch the qualitative shape of the survival probability ρ_c (the unique positive root of $1 - \rho = e^{-c\rho}$) as a function of c on $[0, 3]$. Mark the critical point $c = 1$ and the horizontal asymptote $\rho_\infty = 1$. Annotate the picture with the three regimes (subcritical / critical / supercritical) of Theorem 214.

Exercise 11.62. Draw a sample $G(8, 1/2)$ and identify by inspection: the largest component, all isolated vertices, the maximum and minimum degree. Compare with the expectations $\mathbb{E}[X_0] = 8 \cdot (1/2)^7 = 1/16$ and $\mathbb{E}[d(v)] = 7/2$ (so $\mathbb{E}[\Delta] \geq 7/2$; in fact $\mathbb{E}[\Delta] \approx 5.2$).

11.6 Szemerédi's Regularity Lemma

Szemerédi's regularity lemma, first proved as a technical tool toward his 1975 theorem on arithmetic progressions in dense sets, has become one of the most important results in extremal combinatorics. Intuitively it says: *every large dense graph is “close” to a bounded-complexity weighted object*. Combined with the *counting lemma* (embedding counts match random predictions to within error ε) and the *graph removal lemma*, it provides a general-purpose machine for translating structural statements about H -free-ness into approximate statements about the ambient graph.

11.6.1 ε -Regular Pairs

Definition 175 (Density). For disjoint non-empty $X, Y \subseteq V(G)$, the **edge density** is $d(X, Y) = e(X, Y)/(|X||Y|)$, where $e(X, Y)$ is the number of edges with one endpoint in each.

Definition 176 (ε -regular pair). A pair (X, Y) of disjoint non-empty vertex sets is **ε -regular** if for every $X' \subseteq X$ with $|X'| \geq \varepsilon|X|$ and every $Y' \subseteq Y$ with $|Y'| \geq \varepsilon|Y|$,

$$|d(X', Y') - d(X, Y)| \leq \varepsilon.$$

Remark 148 (Pseudorandomness). A regular pair (X, Y) is a “pseudorandom” bipartite graph: its edges are spread so uniformly that no large sub-rectangle has density deviating significantly from the average. In particular, random bipartite graphs are regular (with high probability), but so are many deterministic constructions such as Paley graphs.

Proposition 222 (Sliceability). Let $\alpha \geq \varepsilon$. If (X, Y) is ε -regular with density d , $X' \subseteq X$ with $|X'| \geq \alpha|X|$, $Y' \subseteq Y$ with $|Y'| \geq \alpha|Y|$, then (X', Y') is $\max\{2\varepsilon, \varepsilon/\alpha\}$ -regular with density in $[d - \varepsilon, d + \varepsilon]$. (For $\alpha = \varepsilon$ this gives nothing, and indeed (X', Y') need not be 2ε -regular then.)

Lemma 123 (Typical degree in a regular pair). If (X, Y) is ε -regular with density d , then at most $\varepsilon|X|$ vertices $x \in X$ have $|N(x) \cap Y| < (d - \varepsilon)|Y|$, and at most $\varepsilon|X|$ have $|N(x) \cap Y| > (d + \varepsilon)|Y|$. Hence all but at most $2\varepsilon|X|$ vertices $x \in X$ have $|N(x) \cap Y| \in [(d - \varepsilon)|Y|, (d + \varepsilon)|Y|]$.

Proof. Suppose more than $\varepsilon|X|$ vertices have $|N(x) \cap Y| < (d - \varepsilon)|Y|$. Let X' be that exceptional set. Then $d(X', Y) < d - \varepsilon$, but $|X'| \geq \varepsilon|X|$, $|Y| \geq \varepsilon|Y|$, contradicting ε -regularity. The bound for vertices with more than $(d + \varepsilon)|Y|$ neighbors is symmetric. $\square$

11.6.2 The Regularity Lemma

Definition 177 (Equipartition). An **equipartition** of $V(G)$ into k parts is a partition $V = V_1 \cup V_2 \cup \dots \cup V_k$ where $||V_i| - |V_j|| \leq 1$ for all i, j .

Definition 178 (ε -regular partition). A partition $V = V_0 \cup V_1 \cup \dots \cup V_k$ is **ε -regular** if $|V_0| \leq \varepsilon n$, the parts $V_1, \dots, V_k$ are equipartitioned, and at most εk^2 pairs (V_i, V_j) fail to be ε -regular.

Theorem 223 (Szemerédi Regularity Lemma, 1975). For every $\varepsilon > 0$ and $m_0 \geq 1$ there exists $M = M(\varepsilon, m_0)$ such that every graph G with at least m_0 vertices admits an ε -regular partition into k parts with $m_0 \leq k \leq M$.

V_0	V_1	V_2	V_3	V_4
	0.35	0.62	??	0.41
	0.62	0.18	0.55	0.30
	??	0.55	0.27	0.48
	0.41	0.30	0.48	0.39

Density matrix $d(V_i, V_j)$ for the partition.

Numbered cell: ε -regular pair. “??” cell: irregular pair (counted toward $\leq \varepsilon k^2$ budget).

V_0 is the small “garbage” set, $|V_0| \leq \varepsilon n$.

Figure 159: An ε -regular partition of $V(G)$ into $V_0 \cup V_1 \cup \dots \cup V_4$.

Proof via mean-square density / energy increment. Define the *energy* (or *index*) of a partition $\mathcal{P} = \{V_1, \dots, V_k\}$:

$$q(\mathcal{P}) = \sum_{i,j} \frac{|V_i||V_j|}{n^2} \cdot d(V_i, V_j)^2.$$

Note $0 \leq q(\mathcal{P}) \leq 1$ since densities lie in $[0, 1]$. Key lemma (*energy increment*): if a partition has more than εk^2 irregular pairs, one can refine it to a partition $\mathcal{P}'$ (with each V_i split into $\leq 2^k$ smaller sets) such that $q(\mathcal{P}') \geq q(\mathcal{P}) + \varepsilon^5$.

Starting from any initial partition, iterate. Since energy increases by ε^5 each step and is bounded by 1, the process terminates in $\leq \varepsilon^{-5}$ steps, yielding the claimed bound on k . The partition size bound is a tower of 2's of height $O(\varepsilon^{-5})$. $\square$

Remark 149 (Gowers's lower bound). *The tower-type dependence $M = M(\varepsilon)$ is essentially unavoidable: Gowers (1997) constructed graphs for which every ε -regular partition has $k \geq \text{tower}(1/\varepsilon^{1/16})$.*

11.6.3 The Counting Lemma

Theorem 224 (Counting Lemma). Let H be a graph with vertex set $\{1, \dots, h\}$ and m edges. Suppose $V_1, \dots, V_h$ are disjoint vertex sets in G such that for every $ij \in E(H)$, the pair (V_i, V_j) is ε -regular with density d_{ij} . Then the number $N(H; V_1, \dots, V_h)$ of labeled copies of H in G with vertex i in V_i satisfies

$$\left| N(H; V_1, \dots, V_h) - \left(\prod_{ij \in E(H)} d_{ij} \right) \prod_{i=1}^h |V_i| \right| \leq m\varepsilon \prod_{i=1}^h |V_i|.$$

Moreover, if $H = K_3$ and $d_{12}, d_{13}, d_{23} \geq 2\varepsilon$, then

$$N(K_3; V_1, V_2, V_3) \geq (1 - 2\varepsilon)(d_{12} - \varepsilon)(d_{13} - \varepsilon)(d_{23} - \varepsilon) |V_1||V_2||V_3|.$$

Proof sketch. For the first bound induct on m (the case $m = 0$ is trivial). Delete an edge ab from H to obtain H' , and fix images of all vertices other than a, b ; let $A \subseteq V_a$ and $B \subseteq V_b$ be the admissible images of a and b . If $|A| \geq \varepsilon|V_a|$ and $|B| \geq \varepsilon|V_b|$, then ε -regularity gives $|e(A, B) - d_{ab}|A||B|| \leq \varepsilon|A||B|$; otherwise both $e(A, B)$ and $d_{ab}|A||B|$ lie in $[0, |A||B|]$ and $|A||B| < \varepsilon|V_a||V_b|$. Summing over the fixed images gives $|N(H; V_1, \dots, V_h) - d_{ab}N(H'; V_1, \dots, V_h)| \leq \varepsilon \prod_i |V_i|$, and induction (with $d_{ab} \leq 1$) finishes. For the triangle bound, by Lemma 123 (applied to (V_1, V_2) and to (V_1, V_3)) all but at most $2\varepsilon|V_1|$ vertices $x \in V_1$ have at least $(d_{12} - \varepsilon)|V_2| \geq \varepsilon|V_2|$ neighbors in V_2 and at least $(d_{13} - \varepsilon)|V_3| \geq \varepsilon|V_3|$ neighbors in V_3 ; by ε -regularity of (V_2, V_3) the number of edges between these two neighborhoods is at least $(d_{23} - \varepsilon)$ times the product of their sizes, and each such edge closes a triangle with x . $\square$

11.6.4 The Removal Lemma

Theorem 225 (Triangle Removal Lemma; Ruzsa-Szemerédi, 1976). For every $\eta > 0$ there exists $\delta > 0$ such that if a graph G on n vertices contains at most δn^3 triangles, then G can be made triangle-free by removing at most ηn^2 edges.

Proof. We may assume $\eta \leq 1$. Let $\varepsilon = \eta/5$, $m_0 = \lceil 5/\eta \rceil$ and $M = M(\varepsilon, m_0)$ as in Theorem 223. The estimates below hold for all $n \geq n_1$, where n_1 depends only on η ; if $n < n_1$ and $\delta < n_1^{-3}$, then G has no triangles at all and there is nothing to prove. For $n \geq n_1$ take an ε -regular partition $V_0 \cup V_1 \cup \dots \cup V_k$ with $m_0 \leq k \leq M$. Remove: (a) all edges incident to V_0 and all edges inside any single part V_i , (b) all edges between pairs that are not ε -regular, (c) all edges across regular pairs of density $< 2\varepsilon$. Since $|V_0| \leq \varepsilon n$ and $|V_i| \leq n/k + 1$, these are at most $\varepsilon n^2 + n^2/(2k)$, εn^2 and εn^2 edges respectively, up to an error $O(Mn)$; as $n^2/(2k) \leq \eta n^2/10$, the total is at most $0.7\eta n^2 + O(Mn) \leq \eta n^2$. Now claim the resulting graph is triangle-free. Otherwise a triangle $v_1 v_2 v_3$ with $v_i \in V_{i'}$ survives; the parts $V_{1'}, V_{2'}, V_{3'}$ are distinct and different from V_0 , and they form a triangle of ε -regular pairs of density $\geq 2\varepsilon$. By the triangle case of Theorem 224 there are at least $(1 - 2\varepsilon)\varepsilon^3 |V_{1'}||V_{2'}||V_{3'}| \geq c \cdot n^3$ triangles in G , where $c = \varepsilon^3/(16M^3)$ (using $|V_{i'}| \geq n/(2M)$). Choose $\delta < \min\{c, n_1^{-3}\}$. $\square$

Theorem 226 (General Graph Removal Lemma; Erdős–Frankl–Rödl). For every graph H and every $\eta > 0$ there exists $\delta > 0$ such that if G on n vertices has fewer than $\delta n^{|V(H)|}$ copies of H , then at most ηn^2 edges need be removed to make G H -free.

11.6.5 From Triangle Removal to Roth

Theorem 227 (Roth, 1953). For every $\alpha > 0$ there exists N_0 such that every subset of $\{1, 2, \dots, N\}$ with $N \geq N_0$ and at least αN elements contains a non-trivial 3-term arithmetic progression.

Proof via triangle removal; Ruzsa–Szemerédi. Let $A \subseteq [N]$ have density α . Build the *Ruzsa 3-AP graph* G_A with vertex set $X \cup Y \cup Z$ where each copy is $[3N]$. Edges: $(x, y) \in X \times Y$ iff $y - x \in A$; $(y, z) \in Y \times Z$ iff $z - y \in A$; $(x, z) \in X \times Z$ iff $(z - x)/2 \in A$. A triangle (x, y, z) yields the elements $y - x$, $(z - x)/2$, $z - y$ of A , which form a 3-AP (the middle one is the average of the other two); this 3-AP is trivial exactly when $x + z = 2y$.

G_A has $9N$ vertices and $\Theta(\alpha N^2)$ edges. The trivial triangles $(x, x + a, x + 2a)$ with $x \in [N]$, $a \in A$ number at least αN^2 and are pairwise *edge-disjoint* (any edge of such a triangle determines x and a). Hence any edge-removal making G_A triangle-free requires at least $\alpha N^2 > (\alpha/100)(9N)^2$ edges, so by Theorem 225 with $\eta = \alpha/100$ the graph G_A has more than $\delta(9N)^3 = \Omega(N^3)$ triangles. At most $3N|A| = O(N^2)$ of them are trivial, so for large N some triangle is non-trivial—yielding a non-trivial 3-AP in A . $\square$

11.6.6 The Blow-up Lemma

Theorem 228 (Kömlos–Sárközy–Szemerédi blow-up lemma, 1997). For every graph R on vertex set $\{1, \dots, r\}$ and all $\delta > 0$, $\Delta \geq 1$ there is $\varepsilon = \varepsilon(\delta, \Delta, r) > 0$ such that the following holds. Let $V_1, \dots, V_r$ be disjoint sets, let R^* be the graph on $V_1 \cup \dots \cup V_r$ in which V_i and V_j are completely joined whenever $ij \in E(R)$, and let $G \subseteq R^*$ be a spanning subgraph in which every pair (V_i, V_j) , $ij \in E(R)$, is (ε, δ) -*super-regular*: $e(X, Y) > \delta|X||Y|$ whenever $X \subseteq V_i$, $Y \subseteq V_j$, $|X| > \varepsilon|V_i|$, $|Y| > \varepsilon|V_j|$, and every vertex of V_i has more than $\delta|V_j|$ neighbors in V_j and vice versa. Then every graph H with $\Delta(H) \leq \Delta$ that is a subgraph of R^* is also a subgraph of G .

The blow-up lemma is the counterpart of the counting lemma when one wants to embed a *spanning* or nearly-spanning subgraph rather than count all copies of a fixed H . It is a key ingredient in the proofs (for large n) of the Pósa–Seymour conjecture on powers of Hamilton cycles, the Alon–Yuster and Kömlos tiling conjectures, and the Bollobás–Kömlos bandwidth conjecture.

11.6.7 Graphons and the Limit Theory

Definition 179 (Graphon). A **graphon** is a symmetric measurable function $W : [0, 1]^2 \rightarrow [0, 1]$. Every graph G defines a graphon W_G (the indicator of its adjacency matrix rescaled), and the space of graphons carries a natural *cut-distance* metric.

Theorem 229 (Lovász–Szegedy; Borgs–Chayes et al., 2006). The space of graphons modulo measure-preserving relabelings is compact under cut-distance, and any sequence of graphs with convergent subgraph densities converges to some graphon limit.

Graphons are the “real-valued” limit objects encoding what large graphs look like; they underlie the modern theory of graph property testing and extremal graph theory.

11.6.8 Property Testing

Theorem 230 (Alon–Shapira: testability of hereditary properties). Every hereditary graph property $\mathcal{P}$ is *testable* with one-sided error: there is an algorithm which, given oracle access to a graph G and a parameter ε , queries a number of vertex pairs of G bounded by a function $q_{\mathcal{P}}(\varepsilon)$ of ε alone (independent of $|V(G)|$) and (a) accepts if $G \in \mathcal{P}$, (b) rejects with probability $\geq 2/3$ if G is ε -far from $\mathcal{P}$.

The proof relies directly on the regularity lemma and the graph-removal lemma: a large graph ε -far from $\mathcal{P}$ must contain “many” forbidden subgraphs, detectable by sampling. The resulting bounds on $q_{\mathcal{P}}(\varepsilon)$ are

enormous, and in general $q_{\mathcal{P}}$ cannot be polynomial in $1/\varepsilon$: already for triangle-freeness the query complexity of one-sided testers is super-polynomial (via Behrend’s construction), and Alon showed that H -freeness can be tested with one-sided error using $\text{poly}(1/\varepsilon)$ queries if and only if H is bipartite.

11.6.9 Exercises

Easy Questions.

Exercise 11.63. Prove the *symmetry* of regularity: if (X, Y) is ε -regular with density d , then so is (Y, X) .

Exercise 11.64. Show that a random bipartite graph with edge probability p is ε -regular with high probability for any fixed $\varepsilon > 0$.

Exercise 11.65. Using the counting lemma, give a short proof of the Erdős–Stone theorem.

Medium Questions.

Exercise 11.66. Prove that the triangle removal lemma implies the corner theorem (Ajtai–Szemerédi): every subset of $[N]^2$ of positive density contains, for N large, a *corner* $\{(x, y), (x + d, y), (x, y + d)\}$ with $d > 0$.

Exercise 11.67. Use the triangle removal lemma together with Mantel’s theorem $\text{ex}(n, K_3) \leq n^2/4$ to prove the supersaturation statement: every graph with more than $(1/4 + \varepsilon)n^2$ edges contains $\Omega(n^3)$ triangles.

Exercise 11.68. Show that the graph removal lemma is equivalent to the following statement: for any fixed H and $\varepsilon > 0$, every graph with fewer than $\delta n^{|V(H)|}$ copies of H contains an H -free spanning subgraph with at least $e(G) - \varepsilon n^2$ edges.

Exercise 11.69. Using the regularity lemma, give a proof of the Erdős–Simonovits stability theorem.

Hard Questions.

Exercise 11.70. Using Behrend’s construction of a subset of $[N]$ of size $N/\exp(O(\sqrt{\log N}))$ with no 3-AP, together with the Ruzsa 3-AP graph, show that $\delta(\eta)$ in the triangle removal lemma must satisfy $\delta \leq \eta^{c \log(1/\eta)}$ for some absolute constant $c > 0$ and all small η ; in particular δ cannot be taken polynomial in η .

Exercise 11.71. Prove the hypergraph version: for 3-uniform hypergraphs, every H with few copies can be made H -free by deleting few edges. (Use the hypergraph regularity lemma of Gowers / Rödl–Schacht.)

Exercise 11.72. Show that cut-distance $d_{\square}(W_1, W_2)$ is a metric on the space of graphons modulo measure-preserving relabelings, and that it is compatible with subgraph densities in the sense of Lovász–Szegedy.

Diagram Questions.

Exercise 11.73. For the graph G obtained as the disjoint union of $K_{4,4}$ and an independent set of 4 vertices, construct an ε -regular partition with $\varepsilon = 0.1$ into $k = 3$ parts (the two sides of the biclique and the independent set). Tabulate the 3×3 density matrix as in Figure 159 and verify that all pairs are 0.1-regular.

Exercise 11.74. Draw the Ruzsa 3-AP graph G_A for $A = \{1, 4\} \subset [5]$. Identify all triangles of G_A and verify that the trivial triangles correspond to the elements 1 and 4 of A themselves, and that A contains no 3-AP.

11.7 Graph Minors and Treewidth

The graph minor programme of Robertson and Seymour (1983–2011) is arguably the deepest body of work in structural graph theory. The two central pillars are the **Graph Minor Theorem** (well-quasi-ordering of finite graphs under the minor relation) and the **Graph Minor Structure Theorem** (every graph excluding a fixed minor is built, in a precise sense, from pieces that nearly embed in surfaces of bounded genus). Alongside these, **treewidth** has emerged as the single most important parameter for measuring a graph’s “distance to being a tree” and the crux of parameterized algorithm design.

11.7.1 Minors and Topological Minors

Definition 180 (Minor). A graph H is a **minor** of G , written $H \preceq G$, if H can be obtained from a subgraph of G by contracting edges. Equivalently, there exist disjoint connected vertex sets $\{V_u : u \in V(H)\} \subseteq V(G)$ (the *branch sets*) such that whenever $uv \in E(H)$, some edge of G goes between V_u and V_v . Each branch set V_u induces a connected subgraph of G .

Definition 181 (Topological minor). H is a **topological minor** of G if some subdivision of H is a subgraph of G . Every topological minor is a minor; the converse fails (e.g. K_5 is a minor but not a topological minor of the Petersen graph).

Proposition 231 (Subdivisions and maximum degree). If H is a topological minor of G , then H is a minor of G : in a subdivision of H inside G , take as branch set of u the branch vertex of u together with the interior vertices of the subdivided edges assigned to u (each subdivided edge being assigned to one of its ends); each branch set is then a subdivided star. If $\Delta(H) \leq 3$, the converse also holds, so topological-minor and minor coincide.

Theorem 232 (Kuratowski, 1930). A graph G is planar iff it contains no subdivision of K_5 or $K_{3,3}$.

Theorem 233 (Wagner, 1937). A graph G is planar iff neither K_5 nor $K_{3,3}$ is a minor of G .

11.7.2 Tree Decompositions and Treewidth

Definition 182 (Tree decomposition). A **tree decomposition** of G is a pair $(T, \mathcal{B})$ where T is a tree and $\mathcal{B} = (B_t)_{t \in V(T)}$ is a family of **bags** $B_t \subseteq V(G)$ such that:

1. $\bigcup_{t \in V(T)} B_t = V(G)$;
2. for each edge $uv \in E(G)$ there is t with $\{u, v\} \subseteq B_t$;
3. for each $v \in V(G)$, the set $\{t \in V(T) : v \in B_t\}$ induces a non-empty subtree of T (the **interpolation property**).

The **width** of $(T, \mathcal{B})$ is $\max_t |B_t| - 1$; the **treewidth** of G is $\text{tw}(G) = \min\{\text{width of a tree decomposition of } G\}$.

Example 50 (Small treewidth).

- $\text{tw}(T) = 1$ for any tree T with ≥ 2 vertices.
- $\text{tw}(C_n) = 2$ for $n \geq 3$.
- $\text{tw}(K_n) = n - 1$.
- $\text{tw}(\text{series-parallel graph}) \leq 2$.
- $\text{tw}(\text{planar graph on } n \text{ vertices}) = O(\sqrt{n})$.
- $\text{tw}(n \times n \text{ grid}) = n$.

Proposition 234 (Subadditivity under separators). If S is a separator of G (i.e. $G \setminus S$ is disconnected, say with components inducing subgraphs $G_1, \dots, G_k$), then

$$\text{tw}(G) \leq \max_i \text{tw}(G_i \cup S) + |S|.$$

More precisely, tree decompositions glue along S .

Theorem 235 (Pathwidth and intervals). A graph G has pathwidth at most k iff G is a subgraph of an *interval graph* with clique number $\leq k + 1$.

Theorem 236 (Treewidth and chordal graphs). $\text{tw}(G) \leq k$ iff G is a subgraph of a chordal graph with clique number $\leq k + 1$.

Proof sketch. ($\Leftarrow$) Chordal graphs have tree decompositions whose bags are maximal cliques.

($\Rightarrow$) Given a tree decomposition, add edges between any two vertices sharing a bag; the resulting graph is chordal (clique trees exist). $\square$

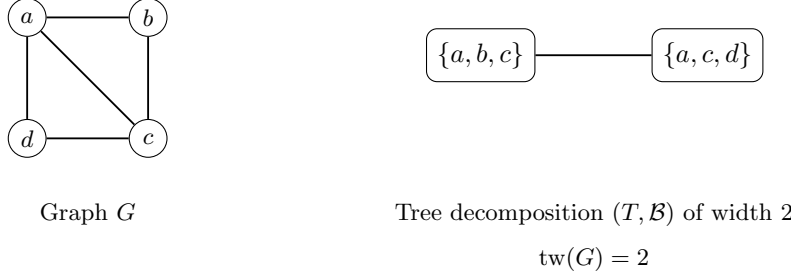


Figure 160: A graph G with chord ac and a width-2 tree decomposition. The vertex a (and likewise c) appears in both bags; this is allowed by the interpolation property because the bags containing a form a (sub)tree.

11.7.3 Duality: Brambles and Cops–Robbers

Definition 183 (Bramble). A **bramble** in G is a family $\mathcal{B}$ of connected subgraphs such that any two elements either share a vertex or are connected by an edge. A **hitting set** for $\mathcal{B}$ is a vertex set meeting every element. The **bramble number** $\text{bn}(G)$ is the maximum size of the minimum hitting set over all brambles.

Theorem 237 (Seymour–Thomas, 1993). $\text{tw}(G) + 1 = \text{bn}(G)$.

This min–max duality is the graph-theoretic analogue of LP duality and is the most convenient tool for proving *lower bounds* on treewidth.

Definition 184 (Cops and robber game). In the k -cop game on G , k cops and a robber are placed on vertices. Each round the cops, currently on a set X , announce a new position Y with $|Y| \leq k$ and fly there by helicopter; while they are in the air, the robber (who sees Y) may run along any path of G that avoids the cops staying in place, i.e. the vertices of $X \cap Y$. Cops win if a cop ever lands on the robber. The **cop number** is the least k for which cops win.

Theorem 238 (Seymour–Thomas). In the *visible-robber* variant of the cops game, k cops win on G iff $\text{tw}(G) \leq k - 1$.

11.7.4 Courcelle’s Theorem and Fixed-Parameter Tractability

Definition 185 (MSO_2 logic). MSO_2 graph logic allows variables for vertices, edges, sets of vertices, and sets of edges, together with the membership relations and a binary predicate $\text{inc}(v, e)$ for “ v is incident to e ”. Most natural graph properties— k -colorability, Hamiltonicity, existence of a given minor, etc.—are expressible in MSO_2 .

Theorem 239 (Courcelle, 1990). Every graph property expressible in MSO_2 can be decided in time $f(k, \phi) \cdot n$ on graphs of treewidth at most k , where ϕ is the MSO_2 formula and f is a computable (but typically tower-type) function.

Corollary 58. 3-coloring, Hamilton cycle, vertex cover, independent set, and minor containment are all solvable in linear time on graphs of bounded treewidth.

Remark 150 (Practical tractability). *Bodlaender’s algorithm (1996) computes a tree decomposition of width $\leq k$ in time $2^{O(k^3)}n$; many MSO_2 problems then admit explicit DP algorithms with single-exponential dependence on k .*

11.7.5 Excluded Minors and Sparsity

Theorem 240 (Mader; Thomason; Kostochka). Every graph with average degree at least $ck\sqrt{\log k}$ contains K_k as a minor, where c is an absolute constant. Moreover the bound is tight up to the constant.

Conjecture 1 (Hadwiger’s conjecture). Every graph G of chromatic number k contains K_k as a minor.

Hadwiger’s conjecture is an enormous strengthening of the Four Color Theorem. It is easy for $k \leq 3$, was proved for $k = 4$ by Hadwiger, is equivalent to the Four Color Theorem for $k = 5$ (Wagner), was proved for $k = 6$ by Robertson–Seymour–Thomas (1993), and is open for $k \geq 7$. Towards a linear bound, Norin–Postle–Song showed that graphs with no K_k minor are $O(k(\log k)^\beta)$ -colorable for every $\beta > 1/4$; Postle improved this to $O(k(\log \log k)^6)$, and Delcourt–Postle to $O(k \log \log k)$.

11.7.6 The Graph Minor Structure Theorem

Theorem 241 (Robertson–Seymour Grid Minor Theorem, 1986). For every k there exists $f(k)$ such that every graph of treewidth at least $f(k)$ contains the $k \times k$ grid as a minor. Quantitatively, Chuzhoy and Tan showed $f(k) = O(k^9 \text{polylog}(k))$.

Theorem 242 (Graph Minor Structure Theorem; Robertson–Seymour, 2003). For every graph H , every graph G with no H -minor admits a tree decomposition into “almost-embeddable” pieces: each bag is obtained from a graph drawable on a surface of bounded genus by adding a bounded number of *vortices* (local perturbations) and *apex* vertices.

The Structure Theorem is the core technical tool underlying both the proof of the Graph Minor Theorem and the design of FPT algorithms for minor-closed problems (e.g. disjoint paths).

11.7.7 The Graph Minor Theorem

Definition 186 (Well-quasi-order). A relation $\preceq$ on a set X is a **well-quasi-order** if it is reflexive, transitive, and every infinite sequence $x_1, x_2, x_3, \dots$ contains indices $i < j$ with $x_i \preceq x_j$. Equivalently, there is no infinite antichain and no infinite strictly decreasing sequence.

Theorem 243 (Higman’s lemma). If $(X, \preceq)$ is a wqo, then so is $(X^*, \preceq^*)$ on finite sequences under the subsequence embedding.

Theorem 244 (Kruskal’s tree theorem, 1960). Finite rooted trees form a wqo under the *homeomorphic embedding* relation.

Theorem 245 (Robertson–Seymour Graph Minor Theorem, 1983–2004). Finite graphs form a well-quasi-order under the minor relation: in any infinite sequence $G_1, G_2, \dots$ there exist $i < j$ with $G_i \preceq G_j$.

Corollary 59 (Finite obstruction sets). Every minor-closed graph property $\mathcal{P}$ is characterized by a *finite* list of forbidden minors: there exist $H_1, \dots, H_k \notin \mathcal{P}$ such that $G \in \mathcal{P}$ iff no H_i is a minor of G .

Proof. Let $\text{Obs}(\mathcal{P})$ be the set of minor-minimal graphs not in $\mathcal{P}$. Any two distinct elements are incomparable (else the smaller one would contradict minimality), so Obs is an antichain. By Theorem 245 antichains are finite. $\square$

Corollary 60 (Algorithmic). For every fixed minor H , there is an algorithm deciding in time $O(n^3)$ whether H is a minor of a given graph G . Combined with Corollary 59, every minor-closed property admits a cubic-time recognition algorithm—even when the obstruction set is unknown!

Remark 151. *The Graph Minor Theorem spans more than 500 pages across twenty papers. The proof relies on the Structure Theorem plus an intricate induction over partial orderings of embedded structures. The cubic running time for minor testing is attributed to Robertson–Seymour for fixed H and has since been refined.*

11.7.8 The Disjoint Paths Problem

Theorem 246 (Robertson–Seymour disjoint paths). For every fixed k , the **k -disjoint paths problem**—given G and pairs $(s_1, t_1), \dots, (s_k, t_k)$, determine whether there exist vertex-disjoint paths P_i from s_i to t_i —is solvable in time $O(n^3)$.

This is a major consequence of the Structure Theorem: while the problem is NP-hard when k is part of the input, any fixed number of terminal pairs can be handled in polynomial time.

11.7.9 Exercises

Easy Questions.

Exercise 11.75. Show that every graph with treewidth at most k has at most kn edges. Show that the converse fails badly: planar graphs have at most $3n - 6$ edges, yet the $m \times m$ grid (planar, with $n = m^2$ vertices and fewer than $2n$ edges) has treewidth $m = \sqrt{n}$.

Exercise 11.76. Compute the treewidth of the Petersen graph.

Exercise 11.77. Verify directly that the cycle C_5 has treewidth 2 by exhibiting a width-2 tree decomposition.

Medium Questions.

Exercise 11.78. Prove that $\text{tw}(G) \leq k$ iff G has a chordal supergraph with clique number $\leq k + 1$ (Theorem 236).

Exercise 11.79. Show that $K_{3,3}$ and K_5 are the only two minor-minimal non-planar graphs, completing Wagner's theorem (Theorem 233).

Exercise 11.80. Prove Kruskal's tree theorem from Higman's lemma.

Exercise 11.81. Show that the class of *series-parallel* graphs (graphs with no K_4 -minor) coincides with graphs of treewidth at most 2.

Exercise 11.82. Using Courcelle's theorem, show that 3-coloring is solvable in linear time on graphs of bounded treewidth. Construct an explicit DP algorithm with running time $O(3^k \cdot k \cdot n)$.

Hard Questions.

Exercise 11.83. Prove that the $n \times n$ grid graph has treewidth exactly n . (The upper bound is easy via a sweeping decomposition; the lower bound uses Theorem 237.)

Exercise 11.84. Prove Hadwiger's conjecture for $k = 4$: every graph of chromatic number 4 contains K_4 as a minor. (Use an inductive decomposition.)

Exercise 11.85. Prove that every graph with no K_t -minor has chromatic number $O(t\sqrt{\log t})$. (Use the Kostochka–Thomason density bound.)

Exercise 11.86. Show that well-quasi-ordering of finite graphs under *topological minor* fails: construct an infinite antichain of graphs under that relation.

Exercise 11.87. Prove the grid-minor theorem (Theorem 241) with a weaker, exponential bound such as $f(k) = 2^{O(k^5)}$ (Robertson–Seymour–Thomas), using an induction on treewidth and the separator / tangle machinery.

Diagram Questions.

Exercise 11.88. For the 3×3 grid, construct an explicit width-3 tree decomposition (the upper bound is achieved by a path of 6 bags of size 4: list the vertices row by row as $v_1, \dots, v_9$ and take $B_t = \{v_t, v_{t+1}, v_{t+2}, v_{t+3}\}$ for $t = 1, \dots, 6$). Draw the decomposition analogously to Figure 160.

Exercise 11.89. Draw K_5 and $K_{3,3}$ side by side. For each, exhibit a K_5 -minor (resp. $K_{3,3}$ -minor) inside the Petersen graph by choosing five (resp. six) connected branch sets and verifying that the required adjacencies hold.

11.8 Comprehensive Chapter Exercises

Easy Questions.

Exercise 11.90. Prove Mantel's theorem by induction on n : remove an edge uv from a triangle-free graph G , bound $|E(G - \{u, v\})|$ inductively, and account for the removed degrees.

Exercise 11.91. Show that the chromatic number of the Turán graph $T(n, r)$ is exactly r when $n \geq r$. Deduce from Turán's theorem that every graph with more than $(1 - 1/r)n^2/2$ edges has chromatic number $\geq r + 1$.

Exercise 11.92. Prove that $R(3, 3) = 6$ by exhibiting a two-coloring of K_5 with no monochromatic triangle and arguing $R(3, 3) \leq 6$ directly.

Medium Questions.

Exercise 11.93. Show that every tournament on n vertices contains a Hamiltonian path (induct on n , inserting the new vertex into a Hamiltonian path of the rest). Then use linearity of expectation to show that some tournament on n vertices has at least $n!/2^{n-1}$ Hamiltonian paths.

Exercise 11.94. For the cycle C_n , compute the Laplacian eigenvalues explicitly as $2 - 2\cos(2\pi k/n)$ for $k = 0, 1, \dots, n-1$. Identify μ_2 and comment on how it scales with n .

Exercise 11.95. Prove that the threshold for $G(n, p)$ to contain a triangle is $p = 1/n$: show $\mathbb{E}[\#K_3] = \binom{n}{3}p^3$ and apply Markov's inequality (first moment) for the 0-statement, and use Chebyshev's inequality (second moment) for the 1-statement.

Exercise 11.96. Prove Kirchhoff's Matrix-Tree theorem for the complete graph K_n and deduce Cayley's formula: K_n has n^{n-2} spanning trees.

Hard Questions.

Exercise 11.97. Show that every graph with treewidth at most k has at most kn edges. Show that the converse fails badly: planar graphs have at most $3n - 6$ edges, yet the $m \times m$ grid (planar, with $n = m^2$ vertices and fewer than $2n$ edges) has treewidth $m = \sqrt{n}$.

Exercise 11.98. Prove that the $n \times n$ grid graph has treewidth exactly n . (The upper bound is easy via a sweeping decomposition; the lower bound requires a connectivity/cops-and-robbers argument.)

Exercise 11.99. Using the triangle removal lemma, prove the following: for every $\eta > 0$ there exists n_0 such that any subset of $\{1, \dots, n\}$ with $n \geq n_0$ and density at least η contains a non-trivial 3-term arithmetic progression. (Construct the Ruzsa 3-AP graph.)

12 Algebraic Graph Theory: Auxiliary Topics

This chapter is a graduate-level tour of the corner of algebraic combinatorics that interacts most strongly with graph theory. Each section pairs an algebraic construction (a polynomial, a Hopf algebra, a Chow ring) with a concrete graph-theoretic payoff (real-rootedness of the matchings polynomial, the chromatic symmetric function, log-concavity of the chromatic polynomial, rapid mixing of matroid Markov chains). The chapter is largely independent of Chapter 11 but freely cites spectral results from §11.4.

Roadmap and graph-theoretic motivation

- §12.1 **Matchings polynomial / Heilmann–Lieb.** A graph polynomial whose roots are always real. Generalises the adjacency-spectrum theorem from forests to all graphs; implies the absence of phase transitions in the monomer–dimer model.
- §12.2 **Equitable partitions / interlacing.** Symmetry groups of a graph induce equitable partitions; their quotient matrices give explicit eigenvalues. Yields the Hoffman / Haemers ratio bounds for $\alpha(G)$ and $\chi(G)$.
- §12.3 **Distance-regular graphs / association schemes.** Algebraic generalisation of strongly regular graphs. Underlies Delsarte’s LP bound for codes (Hamming scheme) and Erdős–Ko–Rado-type results (Johnson scheme).
- §12.4 **Cayley graphs.** Eigenvalues of $\text{Cay}(\Gamma, S)$ from characters of Γ ; explicit Ramanujan graph constructions; the spectral source for nearly all deterministic expanders.
- §12.5 **Tutte polynomial / matroids.** Universal deletion–contraction graph invariant; specialises to chromatic, flow, reliability, Ising, Jones polynomials. The matroid framework abstracts cycle/bond duality.
- §12.6 **Heaps of pieces.** Walk generating functions on graphs as continued fractions; heap-theoretic view of the matchings polynomial; bridge to orthogonal polynomial theory.
- §12.7 **Ihara zeta function.** Graph Riemann zeta whose “Riemann hypothesis” equals the Ramanujan property for connected non-bipartite regular graphs; Hashimoto non-backtracking matrix is the foundation of modern spectral community detection.
- §12.8 **Combinatorial Hopf algebras.** Schmitt’s graph Hopf algebra recovers Stanley’s chromatic symmetric function via the Aguiar–Bergeron–Sottile universal property; Stanley–Stembridge conjecture and Hikita’s recent proof.
- §12.9 **Hodge theory of matroids.** Adiprasito–Huh–Katz prove the chromatic polynomial coefficients are log-concave by importing Kähler-package machinery into combinatorics.
- §12.10 **Permanent and #P.** Counting perfect matchings is #P-complete; the Jerrum–Sinclair–Vigoda FPRAS for the permanent uses a Markov chain on perfect and near-perfect matchings; holographic algorithms reduce certain counting problems to counting perfect matchings in planar graphs, i.e. to Pfaffians (FKT).
- §12.11 **Weisfeiler–Leman and graph isomorphism.** Color refinement, the k -WL hierarchy, the Cai–Fürer–Immerman counterexample, Babai’s quasipolynomial algorithm, and the connection to graph neural network expressivity.
- §12.12 **Cluster algebras from quivers.** Quivers as directed graphs; mutation and cluster variables; cluster algebras of finite type and the Cartan–Killing classification; triangulations of surfaces.
- §12.13 **Chip-firing, sandpiles, and Riemann–Roch for graphs.** The sandpile group (the degree-zero Picard group) of a graph; Baker–Norine Riemann–Roch theorem; explicit graph-theoretic analogues of algebraic-geometric intersection theory.

- §12.14 **Random walks via representation theory.** Diaconis–Shahshahani upper-bound lemma; cutoff phenomenon; explicit mixing-time bounds for the riffle shuffle, the hypercube, and Cayley graphs of the symmetric group.
- §12.15 **Electrical networks and effective resistance.** Resistor networks dual to random walks; the spanning-tree-counting / commute-time / hitting-time identities of Chandra–Raghavan–Ruzzo–Smolensky; the Koebe–Andreev–Thurston circle packing theorem.
- §12.16 **Crystal graphs and the plactic monoid.** Kashiwara crystals as edge-labelled graphs whose connected components encode irreducible $\mathfrak{gl}_n$ -representations; Knuth equivalence and the plactic structure on words.

References (canonical). Godsil–Royle, *Algebraic Graph Theory* (2001); Godsil, *Algebraic Combinatorics* (1993); Stanley, *Algebraic Combinatorics: Walks, Trees, Tableaux* (UTM, 2013); Garsia–Egecioglu, *Lectures in Algebraic Combinatorics* (2020); Brouwer–Haemers, *Spectra of Graphs* (2012); Brouwer–Cohen–Neumaier, *Distance-Regular Graphs* (1989); Bannai–Ito, *Algebraic Combinatorics I* (1984); Oxley, *Matroid Theory* (2nd ed., 2011); Diaconis, *Group Representations in Probability and Statistics* (1988); Lyons–Peres, *Probability on Trees and Networks* (2016); Bump, *Lie Groups* (2nd ed., 2013) for crystal bases. Selected research papers and theses are cited inline throughout.

12.1 The Matchings Polynomial and the Heilmann–Lieb Theorem

The adjacency-matrix characteristic polynomial $\phi(G, x) = \det(xI - A(G))$ encodes the spectrum of G , but it is highly non-trivial to relate to the combinatorial structure of G . The *matchings polynomial* plays a parallel role for matchings and turns out to coincide with $\phi(G, x)$ when G is a forest, while always remaining a polynomial with real roots only.

12.1.1 Definition and Basic Recurrences

Definition 187 (Matchings polynomial). Let G be a graph on n vertices and let $p(G, k)$ denote the number of k -matchings (sets of k pairwise non-adjacent edges). The **matchings polynomial** of G is

$$\mu(G, x) = \sum_{k \geq 0} (-1)^k p(G, k) x^{n-2k}.$$

By convention, $p(G, 0) = 1$ and $\mu(\emptyset, x) = 1$.

Example 51 (Small graphs).

- $\mu(K_2, x) = x^2 - 1$.
- $\mu(P_3, x) = x^3 - 2x$.
- $\mu(C_3, x) = x^3 - 3x$ (three edges, no 2-matching).
- $\mu(C_4, x) = x^4 - 4x^2 + 2$ (two 2-matchings: opposite-edge pairs).

Theorem 247 (Edge and vertex recurrences). For any edge $e = uv$ and any vertex u of G ,

$$\begin{aligned} \mu(G, x) &= \mu(G \setminus e, x) - \mu(G \setminus \{u, v\}, x), \\ \mu(G, x) &= x \mu(G \setminus u, x) - \sum_{v \sim u} \mu(G \setminus \{u, v\}, x). \end{aligned}$$

Proof. For the first identity, split the k -matchings M of G according to whether $e \in M$. Those with $e \notin M$ are exactly the k -matchings of $G \setminus e$, which has the same n vertices, so they contribute $\mu(G \setminus e, x)$. Those with $e \in M$ correspond bijectively, via $M \mapsto M \setminus \{e\}$, to the $(k-1)$ -matchings of $G \setminus \{u, v\}$, which has $n-2$ vertices; since $(-1)^k x^{n-2k} = -(-1)^{k-1} x^{(n-2)-(2(k-1))}$, they contribute $-\mu(G \setminus \{u, v\}, x)$. For the vertex recurrence, apply the edge recurrence successively to the edges $uv_1, \dots, uv_d$ at u : deleting $uv_1, \dots, uv_{j-1}$ does not change $G \setminus \{u, v_j\}$, and once all these edges are deleted u is an isolated vertex, so the remaining graph has matchings polynomial $x \mu(G \setminus u, x)$. $\square$

Proposition 248 (Matching polynomial of a forest). If T is a forest, then $\mu(T, x) = \phi(T, x) = \det(xI - A(T))$.

Proof. The Leibniz formula expresses $\det(xI - A)$ as a sum over permutations σ , which decompose into cycles. In a forest the only cycles admissible to a non-zero contribution are fixed points (giving an x) and 2-cycles (giving -1 via an edge). The non-fixed-point support of σ thus selects a matching M , and the contribution is $(-1)^{|M|} x^{n-2|M|}$. Summing matches Definition 187. $\square$

12.1.2 The Path-Tree Construction

The fundamental insight of Godsil (1981) is that for any graph G with a distinguished vertex u , one can build a tree $T(G, u)$ whose characteristic polynomial controls the matchings polynomial of G .

Definition 188 (Path-tree). Let G be a graph and $u \in V(G)$. The **path-tree** $T(G, u)$ is the rooted tree whose vertices are the *paths* $u = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k$ in G starting at u (the v_i are pairwise distinct; the root is the trivial path u), and whose edges connect each path of length k to its extensions by one further vertex, i.e. to the paths of length $k+1$ of which it is the maximal proper subpath. Thus $T(G, u)$ is finite, has depth at most $|V(G)| - 1$, and is isomorphic to G when G is a tree. (Allowing all non-backtracking walks instead would give the universal cover of G , which is infinite as soon as the component of u contains a cycle.)

Theorem 249 (Godsil's path-tree identity). For any vertex u of G ,

$$\frac{\mu(G \setminus u, x)}{\mu(G, x)} = \frac{\mu(T(G, u) \setminus u, x)}{\mu(T(G, u), x)}.$$

In words, the ratio is identical to the corresponding ratio for the characteristic polynomials of the path-tree. Moreover, if G is connected, then $\mu(G, x)$ divides $\mu(T(G, u), x)$.

Corollary 61 (Reduction to a tree). If G is connected, every root of $\mu(G, x)$ is a root of $\phi(T(G, u), x)$ for any choice of u . Since $T(G, u)$ is a tree, $\phi(T(G, u), x)$ has only real roots; therefore so does $\mu(G, x)$. For disconnected G , apply this to each component and use $\mu(G_1 \cup G_2, x) = \mu(G_1, x) \mu(G_2, x)$. (Connectivity is needed in the first sentence: for $G = K_1 \cup K_2$ with u the isolated vertex, $T(G, u) = K_1$ and $\phi(T(G, u), x) = x$, while $\mu(G, x) = x(x^2 - 1)$.)

12.1.3 The Heilmann–Lieb Theorem

Theorem 250 (Heilmann–Lieb, 1972). Let G be a graph with maximum degree $\Delta \geq 2$. Then all roots of $\mu(G, x)$ are real, and they lie in the open interval $(-2\sqrt{\Delta-1}, 2\sqrt{\Delta-1})$. (If $\Delta \leq 1$, then G is a matching plus isolated vertices and every root of $\mu(G, x)$ lies in $\{-1, 0, 1\}$.)

Proof sketch. Reality follows from Corollary 61. For the bound it suffices, by the same corollary, to take a component G' of G and a vertex u of G' and to show that every eigenvalue of the tree $T = T(G', u)$ has absolute value less than $2\sqrt{\Delta-1}$; this is clear if T is a single vertex, so assume it is not. The root of T has $\deg(u) \leq \Delta$ children, and every other vertex of T (a path ending at w) has one parent and at most $\deg(w) - 1 \leq \Delta - 1$ children. Let $h(p)$ be the depth of p in T and put $D = \text{diag}((\Delta - 1)^{h(p)/2})$. In the non-negative matrix $M = DA(T)D^{-1}$, the row of the root has at most Δ entries, each equal to $1/\sqrt{\Delta-1}$; any other row has one entry $\sqrt{\Delta-1}$ (its parent) and at most $\Delta-1$ entries $1/\sqrt{\Delta-1}$ (its children). Since $\Delta/\sqrt{\Delta-1} \leq 2\sqrt{\Delta-1}$ for $\Delta \geq 2$, every row sum r_p of M satisfies $r_p \leq 2\sqrt{\Delta-1}$, and $r_p = \sqrt{\Delta-1}$ when p is a leaf other than the root. Let ρ be the largest eigenvalue of $A(T)$ and $v > 0$ a Perron eigenvector (Theorem 199). Then $w = D^{-1}v > 0$ satisfies $w^\top M = \rho w^\top$, so $\rho \sum_p w_p = \sum_p w_p r_p < 2\sqrt{\Delta-1} \sum_p w_p$. Hence every eigenvalue θ of $A(T)$ satisfies $|\theta| \leq \rho < 2\sqrt{\Delta-1}$. (The constant $2\sqrt{\Delta-1}$ is the spectral radius of the infinite Δ -regular tree; the Alon–Boppana theorem, a *lower* bound on λ_2 of Δ -regular graphs, plays no role here.) $\square$

Remark 152 (Statistical-physics interpretation). *The matchings polynomial is, up to substitution, the partition function of the monomer–dimer model on G : if $Z_G(\lambda) = \sum_M \lambda^{|M|}$, where the sum runs over all matchings, then $\mu(G, x) = x^n Z_G(-x^{-2})$. Positivity of $Z_G(\lambda)$ for real $\lambda \geq 0$ is trivial (all coefficients are*

positive). The content of Theorem 250 is that every zero of Z_G is real and negative, and (for $\Delta \geq 2$) lies in $(-\infty, -1/(4(\Delta - 1)))$, uniformly over all graphs of maximum degree at most Δ . Since a finite system never has a phase transition ($\log Z_G$ is always real-analytic on $(0, \infty)$), what matters is the thermodynamic limit; Heilmann and Lieb used this location of the zeros, valid for graphs of every size, to show that monomer–dimer systems have no phase transition at any positive dimer activity.

12.1.4 Special Cases and Classical Polynomials

Theorem 251 (Matching polynomials of named graphs).

- $\mu(K_n, x)$ is (up to scaling) the Hermite polynomial $H_n(x)$.
- $\mu(P_n, x)$ is the Chebyshev polynomial of the second kind $U_n(x/2)$.
- $\mu(K_{m,n}, x) = (-1)^n n! x^{m-n} L_n^{(m-n)}(x^2)$ for $m \geq n$, where $L_n^{(\alpha)}$ is a generalized Laguerre polynomial.

Example 52 (Matching polynomial of K_4). K_4 has six edges and three perfect matchings. Direct expansion: $p(K_4, 0) = 1$, $p(K_4, 1) = 6$, $p(K_4, 2) = 3$. Hence

$$\mu(K_4, x) = x^4 - 6x^2 + 3.$$

The roots are $\pm\sqrt{3 \pm \sqrt{6}}$, all real and inside the Heilmann–Lieb interval $(-2\sqrt{2}, 2\sqrt{2}) \approx (-2.83, 2.83)$.

12.1.5 Exercises

Easy Questions.

Exercise 12.1. Compute $\mu(C_5, x)$ and $\mu(C_6, x)$ from the recurrence $\mu(C_n, x) = \mu(P_n, x) - \mu(P_{n-2}, x)$, which follows from Theorem 247 applied to a single edge of C_n .

Exercise 12.2. Verify Proposition 248 for the path P_4 by computing both $\mu(P_4, x)$ from the matching definition and $\det(xI - A(P_4))$ from the adjacency matrix.

Medium Questions.

Exercise 12.3. Prove $\mu(G, x) = \mu(G \setminus e, x) - \mu(G \setminus \{u, v\}, x)$ in detail by induction on $|E(G)|$, taking care of the edge-case $|M| = 0$.

Exercise 12.4. Show that $\mu(G \cup H, x) = \mu(G, x) \cdot \mu(H, x)$ for the disjoint union of G and H , and use this to compute $\mu(mK_2, x)$. Then show that there is no such shortcut for Cartesian products: $C_4 \cup K_1$ and the tree obtained from $K_{1,3}$ by subdividing one edge both have matchings polynomial $x^5 - 4x^3 + 2x$, but their Cartesian products with K_2 have different matchings polynomials. (So $\mu(Q_5, x)$, where $Q_5 = Q_4 \square K_2$ is two copies of Q_4 joined by a perfect matching, cannot be computed from $\mu(Q_4, x)$ and $\mu(K_2, x)$ by a general rule; apply the edge recurrence of Theorem 247 to the 16 edges joining the two copies instead.)

Hard Questions.

Exercise 12.5. Prove the path-tree identity (Theorem 249) by applying the vertex recurrence simultaneously to G and to $T(G, u)$ at the root, then inducting on the depth of expansion.

Exercise 12.6. Use the Heilmann–Lieb theorem and the identity $\mu(G, x) = x^n Z_G(-x^{-2})$ to show that every zero of the partition function $Z_G(\lambda) = \sum_M \lambda^{|M|}$ is real and negative and, when $\Delta \geq 2$, lies in $(-\infty, -1/(4(\Delta - 1)))$. Conclude that $\log Z_G$ extends holomorphically to the slit plane $\mathbb{C} \setminus (-\infty, -1/(4(\Delta - 1))]$, a zero-free region for Z_G that does not depend on the number of vertices of G .

Diagram Questions.

Exercise 12.7. Draw the path-tree $T(C_4, v)$ for any vertex v of the 4-cycle C_4 . Verify that $T(C_4, v)$ is the path P_7 with v in the middle (the infinite 2-regular tree truncated at depth 3), compute its characteristic polynomial, and check that it is divisible by $\mu(C_4, x)$.

Exercise 12.8. Mark the roots of $\mu(K_4, x)$ on the real line and shade the Heilmann–Lieb interval $(-2\sqrt{2}, 2\sqrt{2})$ to confirm strict containment. Compare with the actual adjacency spectrum $\{3, -1, -1, -1\}$ of K_4 (which lies on $\mathbb{Z}$, not on the matching polynomial roots).

12.2 Equitable Partitions and Quotient Interlacing

Equitable partitions provide a systematic way to extract eigenvalues from a graph using its symmetries. Every automorphism orbit gives an equitable partition, but many graphs admit equitable partitions with no automorphism behind them.

12.2.1 Definition and Basic Theory

Definition 189 (Equitable partition). A partition $\pi = (V_1, V_2, \dots, V_k)$ of $V(G)$ is **equitable** if there exist non-negative integers b_{ij} ($1 \leq i, j \leq k$) such that every vertex of V_i has exactly b_{ij} neighbors in V_j . The $k \times k$ matrix $B = (b_{ij})$ is the **quotient matrix** of the partition.

Example 53 (Trivial partitions).

- The discrete partition into singletons is always equitable; its quotient matrix is the adjacency matrix $A(G)$.
- For a k -regular graph, the trivial 1-cell partition $\pi = (V)$ is equitable with $B = (k)$.
- For a bipartite graph $G = (A \cup B, E)$ where every vertex of A has degree d_A and every vertex of B has degree d_B , the partition (A, B) is equitable with $B = \begin{pmatrix} 0 & d_A \\ d_B & 0 \end{pmatrix}$.

Theorem 252 (Spectrum of the quotient). If π is an equitable partition of G with quotient matrix B , then every eigenvalue of B is also an eigenvalue of $A(G)$. Specifically, if $By = \lambda y$ then the vector $x \in \mathbb{R}^V$ defined by $x_v = y_i$ whenever $v \in V_i$ satisfies $Ax = \lambda x$.

Proof. Let $S \in \{0, 1\}^{V \times k}$ be the characteristic matrix of π (column i is the indicator of V_i). Equitability means $AS = SB$. Hence if $By = \lambda y$, then $A(Sy) = S(By) = \lambda(Sy)$. The lifted vector Sy is non-zero because the columns of S are linearly independent. $\square$

12.2.2 Cauchy Interlacing for Quotients

Theorem 253 (Cauchy interlacing for quotients). Let A be a real symmetric $n \times n$ matrix with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$, and let π be any partition (not necessarily equitable) with characteristic matrix S . Define the *normalized quotient* $\hat{B} = (S^\top S)^{-1} S^\top A S$, a $k \times k$ matrix with eigenvalues $\beta_1 \geq \dots \geq \beta_k$. Then

$$\lambda_i \geq \beta_i \geq \lambda_{n-k+i} \quad (1 \leq i \leq k).$$

Proof. Let $D = S^\top S = \text{diag}(|V_1|, \dots, |V_k|)$ and $N = SD^{-1/2}$, so that $N^\top N = I_k$. The matrix $C = N^\top A N = D^{-1/2} S^\top A S D^{-1/2}$ is symmetric, and $\hat{B} = D^{-1} S^\top A S = D^{-1/2} C D^{1/2}$ is similar to it, so $\beta_1 \geq \dots \geq \beta_k$ are real and are the eigenvalues of C . (The unnormalized matrix $S^\top A S$ does *not* interlace in general: for the one-cell partition of a d -regular graph it is (nd) , while $\hat{B} = (d)$.) Fix $i \leq k$ and let $W \subseteq \mathbb{R}^k$ be spanned by orthonormal eigenvectors of C for $\beta_1, \dots, \beta_i$. Because N has orthonormal columns, NW is i -dimensional and $R_A(Ny) = R_C(y) \geq \beta_i$ for every non-zero $y \in W$, so Theorem 198 gives $\lambda_i \geq \beta_i$. Applying this to $-A$ (whose normalized quotient is $-\hat{B}$) gives $\beta_i \geq \lambda_{n-k+i}$. $\square$

Corollary 62 (Hoffman ratio bound; interlacing proof: Haemers, 1995). Let G be a k -regular graph on n vertices, $k \geq 1$, with smallest adjacency eigenvalue λ_n . Then

$$\alpha(G) \leq \frac{-\lambda_n}{k - \lambda_n} \cdot n.$$

Proof. Let $S \subseteq V(G)$ be a maximum independent set, $s = |S|$, and consider the partition $(S, V \setminus S)$; since $k \geq 1$ we have $s < n$. Every vertex of S has all k neighbours in $V \setminus S$, so $e(S, V \setminus S) = ks$ and the normalized quotient is

$$\hat{B} = \begin{pmatrix} 0 & k \\ \frac{ks}{n-s} & k - \frac{ks}{n-s} \end{pmatrix}.$$

Its row sums equal k , so k is an eigenvalue of $\hat{B}$; since $\text{tr } \hat{B} = k - \frac{ks}{n-s}$, the other eigenvalue is $-\frac{ks}{n-s} \leq 0 < k$. Thus $\beta_2 = -\frac{ks}{n-s}$, and Theorem 253 (with two cells) gives $\beta_2 \geq \lambda_n$, i.e. $ks \leq -\lambda_n(n-s)$, so $s(k - \lambda_n) \leq -\lambda_n n$; as $k - \lambda_n > 0$, this is the claimed bound. $\square$

12.2.3 Walk-Regularity and Distance Partitions

Definition 190 (Walk-regular). A graph G is **walk-regular** if for every $k \geq 0$, the diagonal $(A^k)_{vv}$ is the same for all v . Equivalently, the number of closed walks of length k at v does not depend on v .

Theorem 254 (Vertex-transitive implies walk-regular). Every vertex-transitive graph is walk-regular.

Definition 191 (Distance partition). For a vertex v in a connected graph G , the **distance partition from v** is $\pi_v = (G_0(v), G_1(v), G_2(v), \dots, G_d(v))$, where $G_i(v)$ is the set of vertices at distance exactly i from v .

Theorem 255 (Distance partition of a distance-regular graph). A connected graph G is distance-regular (Section 12.3 below) iff for every vertex v , the distance partition π_v is equitable with the same quotient matrix.

This sets up Section 12.3, where the quotient matrix is the canonical tridiagonal intersection matrix.

12.2.4 Exercises

Easy Questions.

Exercise 12.9. Show that the orbit partition of any subgroup of $\text{Aut}(G)$ is equitable.

Exercise 12.10. Find an equitable partition of the Petersen graph into two parts (outer pentagon and inner pentagram). Write its 2×2 quotient matrix and verify both eigenvalues appear in the Petersen spectrum $\{3, 1^{(5)}, -2^{(4)}\}$.

Medium Questions.

Exercise 12.11. Prove Theorem 252 from the matrix identity $AS = SB$. Show further that the k eigenvalues of B are exactly the eigenvalues of A whose eigenvectors are constant on each cell of π .

Exercise 12.12. Apply the Hoffman ratio bound (Corollary 62) to the Petersen graph and to the Kneser graph $K(5, 2)$ (which is the Petersen graph). Compare with the exact independence number $\alpha(P) = 4$.

Hard Questions.

Exercise 12.13. Prove that a connected graph is distance-regular iff every distance partition π_v ($v \in V$) is equitable with the *same* quotient matrix (Theorem 255).

Exercise 12.14. Let G be a graph with an equitable partition π . Show that the spectrum of G decomposes as $\sigma(B) \cup \sigma(A_{\pi^\perp})$, where $A_{\pi^\perp}$ is A restricted to vectors that sum to zero on each cell. Conclude that all “new” eigenvalues of G relative to B have eigenvectors orthogonal to the constant-on-cells subspace.

Diagram Questions.

Exercise 12.15. Draw the cycle C_6 and find an equitable partition into three cells of size 2 each (alternating cells). Write the 3×3 quotient matrix and verify that its eigenvalues $\{2, -1, -1\}$ appear among the cycle spectrum $\{2, 1, -1, -2, -1, 1\}$.

Exercise 12.16. Sketch the distance partition of the cube Q_3 from a corner vertex. Verify that it is equitable into four cells of sizes 1, 3, 3, 1, and write its tridiagonal quotient matrix.

12.3 Distance-Regular Graphs and Association Schemes

Distance-regular graphs (DRGs) sit at the algebraic core of extremal combinatorics: they include all connected strongly regular graphs (Section 11.4), the Hamming and Johnson graphs (which underlie coding theory and the Erdős–Ko–Rado problem), and the foundational examples of finite geometry. Their unified algebraic theory is the language of *association schemes*.

12.3.1 Distance-Regular Graphs

Definition 192 (Distance-regular). A connected graph G of diameter d is **distance-regular** if there exist integers $b_0, b_1, \dots, b_{d-1}$ and $c_1, c_2, \dots, c_d$ such that, for every pair u, v at distance i in G ,

$$c_i = |N(v) \cap G_{i-1}(u)|, \quad b_i = |N(v) \cap G_{i+1}(u)|,$$

i.e. the neighbours of v counted by their distance from u depend only on i . The sequence

$$\iota(G) = \{b_0, b_1, \dots, b_{d-1}; c_1, c_2, \dots, c_d\}$$

is the **intersection array**. The valency is $b_0 = k$.

Example 54 (Familiar distance-regular graphs).

- K_n has intersection array $\{n-1; 1\}$ (diameter 1).
- C_n ($n \geq 5$) has intersection array $\{2, 1, 1, \dots, 1; 1, 1, \dots, 1, c_d\}$, where the last $c_d \in \{1, 2\}$ depending on parity.
- The Petersen graph has $\iota = \{3, 2; 1, 1\}$ (diameter 2).
- The Hamming graph $H(n, q)$ (vertices: words in $[q]^n$; edges: words differing in one coordinate) is distance-regular with $b_i = (n-i)(q-1)$ and $c_i = i$.
- The Johnson graph $J(n, k)$ (vertices: k -subsets of $[n]$; edges: subsets sharing $k-1$ elements) is distance-regular with $b_i = (k-i)(n-k-i)$ and $c_i = i^2$.

Theorem 256 (Tridiagonal quotient). Let G be distance-regular with intersection array $\{b_0, \dots, b_{d-1}; c_1, \dots, c_d\}$. Set $a_i = k - b_i - c_i$ for $0 \leq i \leq d$ (where $b_d = c_0 = 0$). The quotient matrix of the distance partition π_v from any vertex v is

$$L_1 = \begin{pmatrix} a_0 & b_0 & 0 & \cdots & 0 \\ c_1 & a_1 & b_1 & \cdots & 0 \\ 0 & c_2 & a_2 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & c_d & a_d \end{pmatrix}.$$

By Theorem 252, every eigenvalue of L_1 is an eigenvalue of $A(G)$; in fact G has *exactly* $d+1$ distinct adjacency eigenvalues, the eigenvalues of L_1 .

12.3.2 Association Schemes

Definition 193 (Association scheme). A (symmetric) **association scheme** with d classes on a finite set X is a partition $\{R_0, R_1, \dots, R_d\}$ of $X \times X$ such that, with $A_i \in \{0, 1\}^{X \times X}$ the indicator matrix of R_i :

1. $R_0 = \{(x, x) : x \in X\}$, so $A_0 = I$.
2. Each R_i is symmetric: $A_i^\top = A_i$.
3. $\sum_{i=0}^d A_i = J$ (the all-ones matrix).
4. There exist non-negative integers p_{ij}^k with $A_i A_j = \sum_{k=0}^d p_{ij}^k A_k$.

The numbers p_{ij}^k are the **intersection numbers** of the scheme.

Theorem 257 (Bose–Mesner algebra). The complex span of $A_0, A_1, \dots, A_d$ is a commutative algebra $\mathfrak{A}$ under matrix multiplication, called the **Bose–Mesner algebra**. Since the A_i are symmetric and commute pairwise, $\mathfrak{A}$ has a basis of mutually orthogonal idempotents $E_0, E_1, \dots, E_d$ (the *primitive idempotents*). Both $\{A_i\}$ and $\{E_j\}$ are bases of $\mathfrak{A}$, related by the **first** and **second eigenmatrices** P, Q :

$$A_i = \sum_j P_{ji} E_j, \quad E_j = \frac{1}{|X|} \sum_i Q_{ij} A_i.$$

Theorem 258 (Distance-regular = P -polynomial). A distance-regular graph G defines an association scheme on $X = V(G)$ with $R_i = \{(u, v) : d(u, v) = i\}$. The Bose–Mesner algebra is generated as a polynomial algebra by A_1 (the adjacency matrix), so distance-regular schemes are exactly the *P -polynomial* schemes.

12.3.3 The Hamming and Johnson Schemes

Example 55 (Hamming scheme $H(n, q)$). $X = [q]^n$; $(x, y) \in R_i$ iff the Hamming distance is i . The graph $H(n, q)$ above is the corresponding distance-regular graph. Its Bose–Mesner algebra is spanned by the $n + 1$ distance matrices $A_0, \dots, A_n$ of $H(n, q)$ (for $n = 2$, $H(2, q)$ is the $q \times q$ grid, a strongly regular graph with Latin square parameters); the P -matrix entries are values of *Krawtchouk polynomials*.

Example 56 (Johnson scheme $J(n, k)$). $X = \binom{[n]}{k}$; $(A, B) \in R_i$ iff $|A \cap B| = k - i$. The P -matrix entries are values of *Eberlein polynomials* (also called dual Hahn polynomials), and the LP bounds for codes/cliques in $J(n, k)$ specialize to the Erdős–Ko–Rado theorem.

Theorem 259 (Delsarte LP bound). Let Γ be an association scheme on X with second eigenmatrix Q , and let $D \subseteq \{1, \dots, d\}$. Call $C \subseteq X$ a D -code if any two distinct elements of C are in some R_i with $i \in D$ (for $D = \{1\}$, a *clique* of the graph R_1). Then $|C|$ is at most the optimum of the linear program

$$\text{maximize } \sum_{i=0}^d x_i \quad \text{subject to} \quad \begin{cases} x_0 = 1, & x \geq 0, & Q^\top x \geq 0, \\ x_i = 0 & \text{for } i \notin D \cup \{0\}, \end{cases}$$

because the inner distribution $x_i = |(C \times C) \cap R_i|/|C|$ of C is feasible and has $\sum_i x_i = |C|$: if $\mathbf{1}_C$ is the indicator vector of C , then $(Q^\top x)_j = \frac{|X|}{|C|} \mathbf{1}_C^\top E_j \mathbf{1}_C \geq 0$ because E_j is positive semidefinite. (Without the constraints $x_i = 0$ the optimum is always $|X|$.)

The Delsarte LP gives the best-known general bounds on optimal codes (in $H(n, q)$), set systems with bounded intersection (in $J(n, k)$), and packings in spherical caps (continuous analogue).

12.3.4 Exercises

Easy Questions.

Exercise 12.17. Verify that the cycle C_6 is distance-regular and compute its intersection array. Compare with the array of the Petersen graph.

Exercise 12.18. Show that every connected strongly regular graph with parameters (n, k, λ, μ) that is not complete is distance-regular of diameter 2 with intersection array $\{k, k - 1 - \lambda; 1, \mu\}$. (Both hypotheses are needed with the definition of §11.4: $2K_3$ is strongly regular with parameters $(6, 2, 1, 0)$ but disconnected, and K_n has diameter 1.)

Medium Questions.

Exercise 12.19. Compute the intersection array of the Hamming graph $H(3, 2) = Q_3$. Verify directly using Definition 192.

Exercise 12.20. Verify that for the Johnson scheme $J(5, 2)$, the graph R_2 is the Petersen graph (vertices: 2-subsets of $[5]$; edges: disjoint 2-subsets), while R_1 is its complement, the triangular graph $T(5) = J(5, 2)$.

Hard Questions.

Exercise 12.21. Prove Theorem 257: the matrices $A_0, \dots, A_d$ of an association scheme commute pairwise and span a commutative algebra. Use the symmetry $A_i^\top = A_i$ and the multiplication rule $A_i A_j = \sum_k p_{ij}^k A_k$.

Exercise 12.22. Use the Delsarte LP bound on the Johnson scheme $J(n, k)$ to recover the Erdős–Ko–Rado theorem: a family of pairwise-intersecting k -subsets of $[n]$ ($n \geq 2k$) has size at most $\binom{n-1}{k-1}$.

Diagram Questions.

Exercise 12.23. Draw the Hamming graph $H(2, 3)$ (vertices: $\{0, 1, 2\}^2$; edges: words differing in one coordinate). Identify it as the rook graph 3×3 and compute its spectrum.

Exercise 12.24. Sketch the partition $\{R_0, R_1, R_2\}$ of the vertex pairs of C_5 by distance. Verify the intersection numbers p_{ij}^k from the multiplication table $A_i A_j = \sum_k p_{ij}^k A_k$.

12.4 Cayley Graphs and Vertex-Transitive Graphs

Cayley graphs encode group structure as graph structure. Their eigenvalues are computable from the irreducible characters of the group, making them the cleanest available source of explicit d -regular graphs with prescribed spectral properties (notably, expanders).

12.4.1 Definitions and First Examples

Definition 194 (Cayley graph). Let Γ be a finite group and $S \subseteq \Gamma \setminus \{e\}$ a symmetric subset (closed under inversion: $s \in S \Rightarrow s^{-1} \in S$). The **Cayley graph** $\text{Cay}(\Gamma, S)$ has vertex set Γ and edge set $\{\{g, gs\} : g \in \Gamma, s \in S\}$. It is $|S|$ -regular and connected iff S generates Γ .

Example 57 (Familiar Cayley graphs).

- $C_n = \text{Cay}(\mathbb{Z}_n, \{\pm 1\})$.
- Hypercube $Q_n = \text{Cay}(\mathbb{Z}_2^n, \{e_1, \dots, e_n\})$.
- Complete graph $K_n = \text{Cay}(\Gamma, \Gamma \setminus \{e\})$ for any group Γ of order n .
- The *circulant* graph $C_n(S)$ for $S \subseteq \{1, 2, \dots, \lfloor n/2 \rfloor\}$ is $\text{Cay}(\mathbb{Z}_n, S \cup (-S))$.

Theorem 260 (Cayley implies vertex-transitive). Every Cayley graph is vertex-transitive: left translation by any $h \in \Gamma$ is an automorphism. The converse is false (e.g. the Petersen graph is vertex-transitive but not Cayley for any group).

12.4.2 Spectrum from Characters

Theorem 261 (Character formula for abelian Cayley spectra). Let Γ be a *finite abelian* group and $X = \text{Cay}(\Gamma, S)$. The eigenvalues of $A(X)$ are exactly

$$\lambda_\chi = \sum_{s \in S} \chi(s), \quad \chi \in \widehat{\Gamma},$$

where $\widehat{\Gamma}$ is the group of characters of Γ . The character χ itself, viewed as a vector $(\chi(g))_{g \in \Gamma}$, is the eigenvector.

Proof. Since Γ is abelian, characters form an orthonormal basis of $\mathbb{C}^\Gamma$. Direct computation:

$$(A\chi)(g) = \sum_{s \in S} \chi(gs) = \chi(g) \sum_{s \in S} \chi(s) = \lambda_\chi \chi(g).$$

□

Example 58 (Hypercube spectrum). For $\Gamma = \mathbb{Z}_2^n$ the characters are $\chi_T(x) = (-1)^{|T \cap x|}$ for $T \subseteq [n]$. The spectrum of Q_n is therefore

$$\lambda_T = \sum_{i=1}^n \chi_T(e_i) = n - 2|T|, \quad T \subseteq [n],$$

giving eigenvalues $n - 2k$ ($0 \leq k \leq n$) with multiplicity $\binom{n}{k}$.

Example 59 (Cycle spectrum). For $\Gamma = \mathbb{Z}_n$ the characters are $\chi_j(k) = \omega^{jk}$ with $\omega = e^{2\pi i/n}$. The spectrum of $C_n = \text{Cay}(\mathbb{Z}_n, \{\pm 1\})$ is $\lambda_j = \omega^j + \omega^{-j} = 2 \cos(2\pi j/n)$.

12.4.3 Non-Abelian Cayley Graphs

Theorem 262 (Diaconis–Shahshahani trace formula). Let Γ be a finite (possibly non-abelian) group and $S \subseteq \Gamma$ symmetric. Then the spectrum of $\text{Cay}(\Gamma, S)$ consists of the eigenvalues of the matrices $\rho(\hat{S}) = \sum_{s \in S} \rho(s)$ as ρ ranges over the irreducible (unitary) representations of Γ , with multiplicities $\dim(\rho)$.

Example 60 (Random walks on S_n). The random transposition walk on S_n is a random walk on $\text{Cay}(S_n, S)$ with S the set of all transpositions that stays put with probability $1/n$: it chooses two cards independently and uniformly and swaps them, so $\mu(\text{id}) = 1/n$ and $\mu(\tau) = 2/n^2$ for each transposition τ . (The simple random walk on $\text{Cay}(S_n, S)$ alternates between even and odd permutations, so it never converges to the uniform distribution.) Diaconis and Shahshahani used the character formula to prove that the mixing time is $\frac{1}{2}n \log n$, the canonical example of a sharp *cutoff phenomenon* for Markov chains.

12.4.4 Vertex-Transitive Graphs and Cayley-ness

Theorem 263 (Sabidussi, 1958). A vertex-transitive graph X is a Cayley graph iff $\text{Aut}(X)$ contains a subgroup acting regularly (transitively and freely) on $V(X)$.

Example 61 (Petersen is not Cayley). $|\text{Aut}(\text{Petersen})| = 120 = 5!$, with full automorphism group S_5 . There is no subgroup of S_5 of order 10 acting regularly on the 10 vertices, so Petersen fails Sabidussi's criterion.

12.4.5 Exercises

Easy Questions.

Exercise 12.25. Identify each of the following as a Cayley graph if possible: K_4 , $K_{3,3}$, C_5 , Q_3 , the Petersen graph.

Exercise 12.26. Compute the spectrum of the circulant graph $C_8(\{1, 3\})$ using Theorem 261.

Medium Questions.

Exercise 12.27. Show that $\text{Cay}(\mathbb{Z}_n, S)$ is connected iff $\gcd(S) = 1$, where $\gcd(S)$ is the GCD of the elements of $S \cup (-S) \cup \{n\}$.

Exercise 12.28. Verify that the Cayley graph $\text{Cay}(D_8, \{r, r^{-1}, s\})$ (dihedral group with rotation r of order 4 and reflection s) is isomorphic to the cube Q_3 . Use Sabidussi's theorem to conclude that Q_3 has at least two non-equivalent presentations as a Cayley graph.

Hard Questions.

Exercise 12.29. Let Γ be a finite abelian group of order N and S a symmetric generating set with $|S| = d$. Prove that for every integer $m \geq 2$ with $m^d < N$, the second-largest eigenvalue λ_2 of $\text{Cay}(\Gamma, S)$ satisfies $\lambda_2 \geq d \cos(2\pi/m)$. (Hint: write $\chi(s) = e^{2\pi i \theta_\chi(s)}$ and apply the pigeonhole principle to the N points $(\theta_\chi(s))_{s \in S}$, $\chi \in \hat{\Gamma}$, of the torus $(\mathbb{R}/\mathbb{Z})^d$ cut into m^d boxes of side $1/m$.) Conclude that abelian Cayley graphs cannot be expanders if the generating set has bounded size.

Exercise 12.30. Use the character formula for non-abelian Cayley graphs to compute the spectrum of the Cayley graph of S_3 with generating set $\{(12), (13), (23)\}$. Identify the resulting graph.

Diagram Questions.

Exercise 12.31. Draw $\text{Cay}(\mathbb{Z}_2^3, \{e_1, e_2, e_3\}) = Q_3$ and label the eigenvectors χ_T for each $T \subseteq \{1, 2, 3\}$. Color each vertex by the sign $\chi_T(g) = \pm 1$ for $T = \{1, 2\}$ and verify that this colouring is balanced and orthogonal to the all-ones vector.

Exercise 12.32. Draw the Cayley graph of $\mathbb{Z}_5$ with generating set $\{\pm 1\}$ (the pentagon C_5). Mark the eigenvectors $\chi_j(k) = e^{2\pi i j k / 5}$ for $j = 1, 2$ and verify that the corresponding eigenvalues are $2 \cos(2\pi/5)$ and $2 \cos(4\pi/5)$.

12.5 The Tutte Polynomial and Graphic Matroids

The Tutte polynomial $T_G(x, y)$ is the universal multiplicative graph invariant on deletion–contraction. It specializes to the chromatic polynomial, the flow polynomial, the all-terminal reliability polynomial, the Jones polynomial of an alternating link, and the Potts model partition function. Its natural setting is the *matroid* of G .

12.5.1 Deletion, Contraction, and the Tutte Polynomial

Definition 195 (Deletion and contraction). For a graph G and edge $e \in E(G)$:

- $G \setminus e$ is the graph obtained by removing e .
- G/e is the graph obtained by contracting e : delete e and identify its two endpoints. All other edges are kept, so G/e may have multiple edges, and edges parallel to e become loops; these must *not* be discarded (if they were, Theorem 264 would give $T_{K_3} = x^2 + x + 1$ instead of $x^2 + x + y$).

Definition 196 (Tutte polynomial). For a graph G with vertex set V , edge set E , and rank $r(G) = |V| - c(G)$ (where $c(G)$ is the number of connected components), the **Tutte polynomial** is

$$T_G(x, y) = \sum_{A \subseteq E} (x - 1)^{r(E) - r(A)} (y - 1)^{|A| - r(A)},$$

where $r(A) = |V| - c(V, A)$ is the rank of the spanning subgraph (V, A) .

Theorem 264 (Deletion–contraction recursion). For an edge $e \in E(G)$:

- If e is neither a loop nor a bridge: $T_G(x, y) = T_{G \setminus e}(x, y) + T_{G/e}(x, y)$.
- If e is a bridge: $T_G(x, y) = x \cdot T_{G \setminus e}(x, y)$.
- If e is a loop: $T_G(x, y) = y \cdot T_{G \setminus e}(x, y)$.
- If G has no edges: $T_G(x, y) = 1$.

These four rules determine T_G uniquely.

Example 62 (Tutte polynomial of small graphs).

- $T_{K_2}(x, y) = x$.
- $T_{K_3}(x, y) = x^2 + x + y$.
- $T_{K_4}(x, y) = x^3 + 3x^2 + 2x + 4xy + 2y + 3y^2 + y^3$.

12.5.2 Specializations: Chromatic, Flow, Reliability

Theorem 265 (Whitney/Tutte specialization). For any graph G on n vertices,

$$\chi_G(k) = (-1)^{r(E)} k^{c(G)} T_G(1 - k, 0),$$

where $\chi_G(k)$ is the chromatic polynomial counting proper k -colorings.

Theorem 266 (Flow polynomial). The number of nowhere-zero $\mathbb{Z}_k$ -flows on G (with arbitrary edge orientation) is

$$F_G(k) = (-1)^{|E|-r(E)} T_G(0, 1 - k).$$

Theorem 267 (Reliability polynomial). If G is connected and each edge of G fails independently with probability $1 - p$, the probability that the surviving subgraph remains connected (the **all-terminal reliability**) is

$$R_G(p) = p^{r(E)} (1 - p)^{|E|-r(E)} T_G(1, 1/(1 - p)).$$

12.5.3 Whitney's Broken-Circuit Theorem

Definition 197 (Broken circuit). Fix a total order on $E(G)$. A **broken circuit** is a set $C \setminus \{e\}$ where C is a cycle of G and e is the largest edge of C .

Theorem 268 (Whitney's broken-circuit theorem). For any total order on $E(G)$,

$$\chi_G(k) = \sum_{B \subseteq E, B \text{ contains no broken circuit}} (-1)^{|B|} k^{n-|B|}.$$

In particular, $\chi_G(k)$ has alternating-sign integer coefficients.

Theorem 269 (Huh, 2012). The coefficients of the chromatic polynomial form a *log-concave* sequence: if $\chi_G(k) = \sum_{i=0}^n (-1)^i a_i k^{n-i}$, then $a_i^2 \geq a_{i-1} a_{i+1}$ for $1 \leq i \leq n - 1$.

Huh's proof is algebro-geometric (via Milnor-number-type invariants of projective hypersurfaces attached to G). Huh–Katz (2012) extended it to the characteristic polynomial of every matroid representable over some field, using intersection theory on toric varieties; the Rota–Heron–Welsh conjecture in full generality (arbitrary matroids) was proved only later by Adiprasito–Huh–Katz (2018), via Hodge theory for the Chow ring of a matroid (§12.9).

12.5.4 Graphic Matroids and Cycle/Bond Duality

Definition 198 (Matroid). A **matroid** on a finite set E is a pair $(E, \mathcal{I})$ where $\mathcal{I} \subseteq 2^E$ (the **independent sets**) satisfies:

- $\emptyset \in \mathcal{I}$.
- If $I \in \mathcal{I}$ and $J \subseteq I$, then $J \in \mathcal{I}$.
- (*Augmentation*) If $I, J \in \mathcal{I}$ with $|I| < |J|$, then there exists $e \in J \setminus I$ with $I \cup \{e\} \in \mathcal{I}$.

The maximal independent sets are the **bases**; the rank function is $r(A) = \max\{|I| : I \subseteq A, I \in \mathcal{I}\}$.

Example 63 (Graphic matroid). For a graph G , let $E = E(G)$ and $\mathcal{I} = \{A \subseteq E : (V, A) \text{ is acyclic}\}$. This is the **graphic matroid** $M(G)$. Bases of $M(G)$ are the maximal spanning forests of G (a spanning tree of each component); the rank is $|V| - c(G)$.

Theorem 270 (Cycle/bond duality on planar graphs). For a planar graph G with planar dual G^* , $M(G^*) = M(G)^*$, where $M(G)^*$ is the matroid dual of $M(G)$. Equivalently, cycles in G correspond to bonds (minimal cuts) in G^* and vice versa.

Theorem 271 (Tutte polynomial as matroid invariant). The Tutte polynomial extends to all matroids via the same formula: $T_M(x, y) = \sum_{A \subseteq E} (x - 1)^{r(E)-r(A)} (y - 1)^{|A|-r(A)}$. For graphic matroids $M(G)$, this recovers $T_G(x, y)$. Matroid duality interchanges x and y : $T_{M^*}(x, y) = T_M(y, x)$.

12.5.5 Connections: Stanley's Chromatic Symmetric Function

Definition 199 (Chromatic symmetric function; Stanley, 1995). For a graph G on vertex set V , define the **chromatic symmetric function**

$$X_G(x_1, x_2, \dots) = \sum_{\kappa: V \rightarrow \mathbb{Z}_{\geq 1}} \prod_{\text{proper } v \in V} x_{\kappa(v)}.$$

Theorem 272 (Stanley). X_G is a symmetric function in $x_1, x_2, \dots$, and $X_G(1, 1, \dots, 1, 0, 0, \dots) = \chi_G(k)$ when there are k ones. Thus X_G refines the chromatic polynomial.

Conjecture 2 (Stanley's tree conjecture, 1995, OPEN). If T_1 and T_2 are non-isomorphic trees on the same number of vertices, then $X_{T_1} \neq X_{T_2}$.

This conjecture has been verified by computer search for trees on up to ~ 30 vertices but remains open. It would imply, in particular, that the chromatic symmetric function distinguishes any pair of non-isomorphic trees—a much finer invariant than the chromatic polynomial $\chi_T(k) = k(k-1)^{n-1}$, which depends only on the number of vertices.

12.5.6 Exercises

Easy Questions.

Exercise 12.33. Compute $T_{C_4}(x, y)$ from Definition 196 by enumerating all $2^4 = 16$ subsets of edges. Verify your answer against the deletion-contraction recursion (Theorem 264).

Exercise 12.34. Use Theorem 265 to derive the chromatic polynomial $\chi_{K_n}(k)$ from $T_{K_n}(x, y)$. Compare with the direct formula $\chi_{K_n}(k) = k(k-1)(k-2) \cdots (k-n+1)$.

Medium Questions.

Exercise 12.35. Prove the deletion-contraction recursion (Theorem 264) from Definition 196 by partitioning $A \subseteq E$ into those containing e and those not.

Exercise 12.36. Show that $T_G(2, 1)$ = number of spanning forests of G , $T_G(1, 1)$ = number of spanning trees (for connected G), and $T_G(2, 0)$ = number of acyclic orientations of G (Stanley).

Hard Questions.

Exercise 12.37. Prove Whitney's broken-circuit theorem (Theorem 268) by inclusion-exclusion on the cycles of G , ordered by their largest edge.

Exercise 12.38. Show that if G is chordal, then every root of $\chi_G(k)$ is a non-negative integer. (Hint: chromatic polynomials of chordal graphs factor into linear pieces by a perfect elimination ordering.) Show that the converse fails: if H is obtained from K_6 minus an edge ab by adding a new vertex adjacent to exactly a and b , then H has an induced 4-cycle, yet $\chi_H(k) = k(k-1)(k-2)(k-3)^3(k-4)$.

Diagram Questions.

Exercise 12.39. For the planar graph $G = K_4$ minus an edge, draw both G and its planar dual G^* . Verify directly that $T_{G^*}(x, y) = T_G(y, x)$ by computing both polynomials and comparing.

Exercise 12.40. Draw the broken-circuit complex of the triangle K_3 with edges ordered $e_1 < e_2 < e_3$. Identify the unique broken circuit $\{e_1, e_2\}$, and verify $\chi_{K_3}(k) = k^3 - 3k^2 + 2k$ from the broken-circuit sum of Theorem 268.

12.6 Heaps of Pieces and Walk Generating Functions

This graduate-level section develops Viennot’s theory of *heaps of pieces* and uses it to derive closed-form generating functions for walks, matchings, and orthogonal polynomial moments on graphs. The central theorems—Cartier–Foata’s master inversion formula and Flajolet’s continued-fraction expansion—convert combinatorial counts into rational and continued-fraction expressions, providing the algebraic skeleton behind the matchings polynomial of §12.1 and the Ihara zeta function of §12.7.

12.6.1 Commutation Monoids and Heaps

Definition 200 (Commutation graph and partially commutative monoid). Let A be a finite alphabet (the *pieces*). A **commutation graph** on A is an undirected graph $C = (A, E_C)$. The **partially commutative (Cartier–Foata) monoid** $M(C)$ is the quotient of the free monoid A^* by the relations $ab \sim ba$ for every $\{a, b\} \in E_C$.

Definition 201 (Heap of pieces). A **heap** on C is an element of $M(C)$, i.e. an equivalence class of words in A^* . Pictorially: drop the letters of a word one by one; each piece falls until it lands on a previously dropped piece with which it does *not* commute (or on the floor). Thus pieces a, b with $\{a, b\} \in E_C$ never block each other, while non-commuting pieces (including two copies of the same piece) are stacked in the order in which they were dropped. A piece **is at the bottom** of a heap H if it has no piece below it. A heap is **trivial** if all its pieces are at the bottom (i.e. pairwise commuting).

Example 64 (Heaps for monomer-dimer). Let G be a graph and let $A = E(G)$ with commutation graph $C =$ the *non-incidence graph* of $E(G)$ (edges a, b commute iff they share no vertex). Then trivial heaps on A are exactly **matchings** of G . General heaps correspond to sequences of edges placed with the constraint that overlapping edges must respect a temporal order.

12.6.2 The Cartier–Foata Master Theorem

Theorem 273 (Cartier–Foata Master Theorem). Let C be a commutation graph on A with weights $w : A \rightarrow R$ (any commutative ring). Define

$$\mathcal{H}(z) = \sum_{H \text{ heap}} w(H) z^{|H|}, \quad \mathcal{T}(z) = \sum_{T \text{ trivial heap}} (-1)^{|T|} w(T) z^{|T|}.$$

Then in the formal power series ring $R[[z]]$,

$$\mathcal{H}(z) \cdot \mathcal{T}(z) = 1, \quad \text{equivalently} \quad \mathcal{H}(z) = \frac{1}{\mathcal{T}(z)}.$$

Corollary 63 (Viennot’s matching-polynomial formula). For a graph G with edge weights $w : E(G) \rightarrow R$, define the **weighted matchings polynomial** $\mu_w(G, z) = \sum_M (-1)^{|M|} \prod_{e \in M} w(e) z^{|M|}$, the sum over all matchings $M \subseteq E(G)$. Then the generating function of all heaps of edges (i.e. all words in the edges modulo commutation of vertex-disjoint edges) is exactly $1/\mu_w(G, z)$.

Remark 153. *Corollary 63 explains the appearance of the matchings polynomial $\mu(G, x)$ as a reciprocal: heaps of edges (words in the edges modulo commutation of vertex-disjoint edges) are dual to matchings in a precise generating-function sense, and this perspective immediately generalises to q -analogues used in mathematical physics. The inversion formula by itself does not yield the real-rootedness in the Heilmann–Lieb theorem (250): Theorem 273 holds for every commutation graph, including ones whose trivial-heap polynomial has non-real zeros (e.g. taking as pieces the vertices of the claw $K_{1,3}$, commuting iff non-adjacent, gives $\mathcal{T}(z) = 1 - 4z + 3z^2 - z^3$). Real-rootedness needs an extra argument such as the path-tree.*

12.6.3 Walks on Graphs as Heaps of Cycles

Definition 202 (Cycle heap). For a graph G , the **cycle commutation graph** $C^{\text{cyc}}(G)$ has vertex set $\mathcal{C}(G) =$ the set of *oriented* cycles of G , i.e. the directed cycles of the digraph obtained by replacing each edge

uv by the two arcs $u \rightarrow v$ and $v \rightarrow u$: for every edge uv the 2-cycle $u \rightarrow v \rightarrow u$, and for every simple cycle of length ≥ 3 its two orientations. The **length** $\ell(\gamma)$ of a cycle is its number of arcs. Two cycles commute (are connected in C^{cyc}) iff they are vertex-disjoint.

Theorem 274 (Walks and heaps of cycles; Cartier–Foata, Viennot). Let $Q(z) = \sum_T (-1)^{\#\text{cycles}(T)} z^{\ell(T)}$, the sum over trivial heaps T of $C^{\text{cyc}}(G)$ (families of pairwise vertex-disjoint oriented cycles; $\ell(T)$ is the total length). Then $Q(z) = \det(I - zA)$, the generating function of all heaps of cycles (weighted by $z^{\text{total length}}$) is $1/Q(z)$ by Theorem 273, and for $k \geq 1$ the number of closed walks of length k in G is

$$W_k(G) = \text{tr}(A^k) = [z^k] \frac{P(z)}{Q(z)}, \quad \text{where} \quad P(z) = -zQ'(z) = \sum_T (-1)^{\#\text{cycles}(T)-1} \ell(T) z^{\ell(T)}.$$

This formula expresses the trace $\text{tr}(A^k)$ as a sum over families of vertex-disjoint cycles—a graph-theoretic version of the Lindström–Gessel–Viennot lemma.

12.6.4 Flajolet’s Continued-Fraction Theorem

Theorem 275 (Flajolet, 1980). Let G be the path graph $\{0, 1, 2, \dots\}$ with a self-loop of weight b_n at vertex n , in which each step from n down to $n - 1$ has weight λ_n and each step from $n - 1$ up to n has weight 1 (any up/down weights with product λ_n give the same result); the weight of a walk is the product of the weights of its steps. The generating function of weighted closed walks at 0 is the Jacobi continued fraction

$$\sum_{k \geq 0} m_k z^k = \frac{1}{1 - b_0 z - \frac{\lambda_1 z^2}{1 - b_1 z - \frac{\lambda_2 z^2}{1 - b_2 z - \ddots}}},$$

where m_k is the total weight of all closed walks at 0 of length k . Equivalently, (m_k) is the moment sequence of the orthogonal polynomials with three-term recurrence $P_{n+1}(x) = (x - b_n)P_n(x) - \lambda_n P_{n-1}(x)$.

Corollary 64 (Closed walks $\leftrightarrow$ orthogonal polynomial moments). Closed walks on weighted half-line graphs are in bijection with **Motzkin paths**, whose moments coincide with classical orthogonal-polynomial moments (Hermite, Laguerre, Meixner, etc.). This is the algebraic root of why the matching polynomial of K_n is Hermite, of P_n is Chebyshev, of $K_{m,n}$ is Laguerre.

12.6.5 Exercises

Easy Questions.

Exercise 12.41. List all heaps of pieces on the path commutation graph $C = P_3$ with pieces $\{a, b, c\}$ (so a commutes with b , and b with c) and weights $w(a) = w(b) = w(c) = 1$, of total size ≤ 3 . Verify the trivial-heap generating function $\mathcal{T}(z) = 1 - 3z + 2z^2$ and the master theorem $\mathcal{H}(z) = 1/(1 - 3z + 2z^2)$.

Exercise 12.42. For the weighted matchings polynomial of K_3 with unit weights (Corollary 63), verify that $1/\mu_w(K_3, z) = 1/(1 - 3z) = \sum_{k \geq 0} 3^k z^k$ in formal power series, and interpret the coefficients as counts of heaps of edges in K_3 (no two edges of K_3 commute).

Medium Questions.

Exercise 12.43. Use Theorem 273 to derive the Lindström–Gessel–Viennot lemma in the special case of vertex-disjoint paths in a planar acyclic digraph.

Exercise 12.44. Apply Flajolet’s theorem (275) with $b_n = 0$ and $\lambda_n = n$ to recover the moment generating function of the Hermite polynomials, $\sum_k (2k - 1)!! z^{2k}$.

Hard Questions.

Exercise 12.45. Prove Theorem 274: expand $\det(I - zA)$ over permutations (the non-trivial cycles of a permutation form a family of vertex-disjoint oriented cycles) to obtain $Q(z) = \det(I - zA)$, apply Theorem 273 on the cycle commutation graph $C^{\text{cyc}}(G)$, and use $\sum_{k \geq 1} \text{tr}(A^k)z^k = -z \frac{d}{dz} \log \det(I - zA)$.

Exercise 12.46. Generalise Flajolet's theorem to closed walks on a weighted bipartite graph: derive the Stieltjes continued fraction $\sum m_k z^k = 1/(1 - c_1 z/(1 - c_2 z/(1 - \dots)))$ and identify the corresponding orthogonal polynomials.

Diagram Questions.

Exercise 12.47. Draw a sample heap of 4 pieces on the commutation graph $C =$ disjoint union of K_2 and an isolated vertex (i.e. pieces a, b, c with $a \sim b$ and c isolated). Sketch the heap as a “stack” picture and identify which pieces are at the bottom.

Exercise 12.48. Sketch the cycle commutation graph $C^{\text{cyc}}(K_4)$. Identify the oriented cycles of K_4 (six 2-cycles, one for each edge; eight oriented triangles; six oriented 4-cycles) and the disjointness relations between them (the only vertex-disjoint pairs are the three pairs of 2-cycles on opposite edges), so that $Q(z) = 1 - 6z^2 - 8z^3 - 3z^4 = \det(I - zA(K_4))$. Verify Theorem 274 gives $W_4(K_4) =$ number of closed walks of length 4 in K_4 ($= \text{tr}(A(K_4)^4) = 3^4 + (-1)^4 + (-1)^4 + (-1)^4 = 84$).

12.7 The Ihara Zeta Function of a Graph

The Ihara zeta function is the graph-theoretic analogue of the Riemann zeta function. Originally introduced by Ihara (1966) in the context of p -adic groups and rediscovered combinatorially by Sunada, Hashimoto, Bass, and Stark–Terras, it packages *prime non-backtracking closed walks* into a generating function whose poles encode spectral properties of the graph. For connected non-bipartite regular graphs, the Ramanujan property of §11.4 turns out to be exactly the “Riemann hypothesis” for the Ihara zeta.

12.7.1 Non-Backtracking Walks and Primes

Definition 203 (Non-backtracking walk; prime walk). A walk $w = v_0 e_1 v_1 e_2 v_2 \cdots e_k v_k$ in G is **non-backtracking** if $e_{i+1} \neq \bar{e}_i$ for every $1 \leq i \leq k-1$, where $\bar{e}$ denotes the reversed direction of edge e . A non-backtracking closed walk ($v_k = v_0$) is **tailless** if also $e_1 \neq \bar{e}_k$, i.e. every cyclic rotation of it is again non-backtracking. A tailless non-backtracking closed walk is **prime** (or *primitive*) if it is not a proper power of a shorter closed walk, and we identify walks differing only by cyclic rotation. Let $\mathcal{P}(G)$ denote the set of equivalence classes of prime closed walks.

Definition 204 (Ihara zeta function). For a finite connected graph G with no degree-1 vertex, the **Ihara zeta function** is

$$Z_G(u) = \prod_{[C] \in \mathcal{P}(G)} (1 - u^{|C|})^{-1}.$$

Example 65 (Cycle graph). For C_n ($n \geq 3$), there are exactly two prime closed walks (the clockwise and counterclockwise traversals), each of length n . Hence $Z_{C_n}(u) = (1 - u^n)^{-2}$.

12.7.2 The Determinantal Formula

Theorem 276 (Ihara–Bass formula). Let G be a finite connected graph with n vertices, m edges, and adjacency matrix A . Set $r = m - n + 1$ (the cycle rank), and let D be the degree matrix. Then

$$Z_G(u)^{-1} = (1 - u^2)^{r-1} \cdot \det(I - uA + u^2(D - I)).$$

For a d -regular graph, the formula simplifies to

$$Z_G(u)^{-1} = (1 - u^2)^{r-1} \cdot \det(I - uA + u^2(d - 1)I).$$

Proof sketch via the Hashimoto matrix. Define the **non-backtracking matrix** $B \in \{0, 1\}^{2m \times 2m}$ indexed by directed edges: $B_{(u,v),(v,w)} = 1$ iff $w \neq u$. The number of tailless non-backtracking closed walks of length k is exactly $\text{tr}(B^k)$, so by the standard zeta-function identity $Z_G(u) = 1/\det(I - uB)$. Bass's three-term identity then relates $\det(I - uB)$ to the smaller $n \times n$ determinant of Theorem 276. $\square$

12.7.3 Riemann Hypothesis for Graphs

Definition 205 (Graph Riemann hypothesis). Let G be a connected d -regular graph ($d \geq 2$). The poles of $Z_G(u)$ at $u = \pm 1$ and $u = \pm 1/(d-1)$ are called *trivial*. G satisfies the **Riemann Hypothesis** (GRH) if all non-trivial poles of $Z_G(u)$ lie on the circle $|u| = 1/\sqrt{d-1}$.

Theorem 277 (Ramanujan $\Leftrightarrow$ GRH). A connected d -regular graph G satisfies the graph Riemann Hypothesis if and only if every eigenvalue λ of A with $|\lambda| \neq d$ satisfies $|\lambda| \leq 2\sqrt{d-1}$. In particular, a connected non-bipartite d -regular graph G is Ramanujan (Definition 166 – see §11.4) if and only if it satisfies the graph Riemann Hypothesis. Equivalently:

$$G \text{ Ramanujan} \iff \text{spec}(A) \subseteq \{d\} \cup [-2\sqrt{d-1}, 2\sqrt{d-1}].$$

Proof sketch. By Theorem 276, $Z_G(u)^{-1} = (1 - u^2)^{r-1} \prod_{\lambda} (1 - \lambda u + (d-1)u^2)$, the product over the eigenvalues λ of A . The factor $(1 - u^2)^{r-1}$ gives only the trivial poles $u = \pm 1$, and $\lambda = \pm d$ gives the trivial poles $\pm 1, \pm 1/(d-1)$. For any other λ the two poles are the roots of $(d-1)u^2 - \lambda u + 1 = 0$, whose product is $1/(d-1)$: they are complex conjugates with $|u| = 1/\sqrt{d-1}$ exactly when $\lambda^2 \leq 4(d-1)$, and otherwise real with different moduli. $\square$

Remark 154 (Non-backtracking spectrum and community detection). *Krzakala et al. (2013) showed that the spectrum of the Hashimoto matrix B (rather than the adjacency matrix A) detects communities in sparse random graphs all the way down to the Kesten–Stigum threshold, which is below the threshold detectable by A alone. The non-backtracking matrix has thus become the central tool in modern spectral methods for community detection in stochastic block models.*

12.7.4 Functional Equation and Special Values

Theorem 278 (Functional equation). Let G be a connected d -regular graph with n vertices and cycle rank r , and put $q = d-1$. Then

$$\xi_G(u) := (1+u)^{r-1}(1-u)^{r-1+n}(1-qu)^n Z_G(u) \quad \text{satisfies} \quad \xi_G\left(\frac{1}{qu}\right) = \xi_G(u).$$

For n even this can also be written (Terras) as $\Lambda_G(1/(qu)) = \Lambda_G(u)$ with $\Lambda_G(u) = (1 - u^2)^{r-1+n/2}(1 - q^2 u^2)^{n/2} Z_G(u)$. With $u = q^{-s}$ the substitution $u \mapsto 1/(qu)$ is $s \mapsto 1-s$, the analogue of the symmetry $\zeta(s) \leftrightarrow \zeta(1-s)$ for Riemann zeta. (Proof: substitute $u \mapsto 1/(qu)$ in Theorem 276; each factor $1 - \lambda u + qu^2$ is multiplied by $(qu^2)^{-1}$. Note that it is the *ratio* $Z_G(1/(qu))/Z_G(u)$, not the product, that is an elementary factor.)

Theorem 279 (The pole at $u = 1$; Hashimoto, Northshield). Let G be a connected graph with cycle rank $r = m - n + 1 \geq 1$. Then $Z_G(1)^{-1} = 0$; more precisely, $Z_G(u)^{-1}$ vanishes at $u = 1$ to order at least r , and

$$\lim_{u \rightarrow 1} \frac{Z_G(u)^{-1}}{(1-u)^r} = -2^r(r-1)\tau(G), \quad \text{equivalently} \quad \left. \frac{d^r}{du^r} Z_G(u)^{-1} \right|_{u=1} = r!(-1)^{r+1} 2^r(r-1)\tau(G),$$

where $\tau(G)$ is the number of spanning trees. In particular, for $r \geq 2$ the zeta function has a pole of order exactly r at $u = 1$. By Theorem 276 this amounts to

$$\left. \frac{d}{du} \det(I - uA + u^2(D - I)) \right|_{u=1} = 2(m-n)\tau(G).$$

This is a graph-zeta analogue of the class number formula.

12.7.5 Exercises

Easy Questions.

Exercise 12.49. Compute $Z_{K_4}(u)$ explicitly using the Ihara–Bass formula. (K_4 is 3-regular with adjacency spectrum $\{3, -1, -1, -1\}$ and $r = 6 - 4 + 1 = 3$.)

Exercise 12.50. Verify Theorem 277 for the Petersen graph: its spectrum $\{3, 1^{(5)}, -2^{(4)}\}$ lies in $\{3\} \cup [-2\sqrt{2}, 2\sqrt{2}]$, so it is Ramanujan, and hence all non-trivial poles of $Z_P(u)$ lie on $|u| = 1/\sqrt{2}$.

Medium Questions.

Exercise 12.51. Prove the cycle formula $Z_{C_n}(u) = (1 - u^n)^{-2}$ directly from Definition 204 by enumerating prime closed walks.

Exercise 12.52. Verify Bass’s three-term identity for the Hashimoto matrix: $\det(I - uB) = (1 - u^2)^{r-1} \det(I - uA + u^2(D - I))$, by explicitly computing both sides for K_3 .

Hard Questions.

Exercise 12.53. Use the non-backtracking matrix B and the Perron–Frobenius theorem to prove that for a connected d -regular non-bipartite graph, the spectral radius of B is exactly $d - 1$, and the corresponding eigenvector is the all-ones vector.

Exercise 12.54. Show that the Ihara zeta function of an infinite d -regular tree has no poles, in agreement with the universal-cover spectrum $[-2\sqrt{d-1}, 2\sqrt{d-1}]$.

Diagram Questions.

Exercise 12.55. For the graph K_4 , list all prime closed walks of length ≤ 6 (up to cyclic rotation and inversion), grouped by length. Compute the first few coefficients of $\log Z_{K_4}(u) = \sum_k N_k u^k / k$ where N_k counts tailless non-backtracking closed walks of length k .

Exercise 12.56. Sketch the Hashimoto matrix B for the triangle K_3 , showing the 6×6 structure indexed by the 6 directed edges. Verify by hand that $\det(I - uB) = (1 - u^3)^2 = (1 - u)^2(1 + u + u^2)^2$, in agreement with $Z_{C_3}(u) = (1 - u^3)^{-2}$.

12.8 Combinatorial Hopf Algebras of Graphs

A *combinatorial Hopf algebra* packages a family of combinatorial objects together with operations of *merging* (product) and *splitting* (coproduct) compatible with one another. The graph Hopf algebra of Schmitt (1994) puts the chromatic polynomial and Stanley’s chromatic symmetric function (1995) into a clean algebraic frame—and the Aguiar–Bergeron–Sottile theorem (2006) shows that this frame is *universal*: every combinatorial Hopf algebra (a graded connected Hopf algebra equipped with a character) has a unique character-preserving morphism to the quasisymmetric functions, and the chromatic symmetric function is the image of a graph under this morphism for the chromatic character.

12.8.1 Hopf Algebras: A Five-Line Reminder

Definition 206 (Hopf algebra). A **Hopf algebra** over a field k is a k -vector space H equipped with linear maps:

- product $\mu : H \otimes H \rightarrow H$ (associative),
- unit $\eta : k \rightarrow H$,
- coproduct $\Delta : H \rightarrow H \otimes H$ (coassociative),
- counit $\varepsilon : H \rightarrow k$,

- antipode $S : H \rightarrow H$,

satisfying the standard compatibility axioms (the antipode satisfies $\mu(S \otimes \text{id})\Delta = \mu(\text{id} \otimes S)\Delta = \eta\varepsilon$). A Hopf algebra is **graded** if $H = \bigoplus_{n \geq 0} H_n$ with all structure maps respecting the grading, and is **connected** if $H_0 = k \cdot 1$.

Example 66 (Symmetric functions). The ring Λ of symmetric functions is a graded connected Hopf algebra: the coproduct is $\Delta(p_n) = p_n \otimes 1 + 1 \otimes p_n$ on the power-sum basis (i.e. the p_n are *primitive*). It is self-dual: $\Lambda \cong \Lambda^*$ as Hopf algebras, with the Hall inner product realising the duality.

12.8.2 The Graph Hopf Algebra of Schmitt

Definition 207 (Schmitt’s graph Hopf algebra). Let $\mathcal{G}_n$ be the set of (isomorphism classes of) finite simple graphs with vertex set $[n]$, and let $\mathcal{G} = \bigoplus_{n \geq 0} k \mathcal{G}_n$. Define:

- Product: $G \cdot H = G \sqcup H$ (disjoint union, relabelled).
- Coproduct: $\Delta(G) = \sum_{S \subseteq V(G)} G[S] \otimes G[V \setminus S]$, the sum over all bipartitions of $V(G)$ into induced subgraphs.
- Unit: the empty graph; counit: $\varepsilon(G) = [G = \emptyset]$.

Then $(\mathcal{G}, \mu, \Delta, \eta, \varepsilon)$ is a graded connected commutative cocommutative Hopf algebra.

Theorem 280 (Antipode formula; Humpert–Martin, 2012). The antipode of $\mathcal{G}$ admits a cancellation-free formula:

$$S(G) = \sum_{F: \text{flat}} (-1)^{c(F)} a(G/F) G_F,$$

where the sum is over flats F of the cycle matroid of G , $G_F = (V(G), F)$ is the spanning subgraph with edge set F , $c(F)$ is the number of connected components of G_F (so $c(F) = |V(G)| - \text{rk}(F)$), and $a(G/F)$ is the number of acyclic orientations of the contraction G/F .

12.8.3 Characters and the Aguiar–Bergeron–Sottile Theorem

Definition 208 (Character). A **character** on a graded connected Hopf algebra H is a multiplicative linear functional $\zeta : H \rightarrow k$ with $\zeta(1) = 1$ and $\zeta(xy) = \zeta(x)\zeta(y)$.

Theorem 281 (Aguiar–Bergeron–Sottile, 2006). The Hopf algebra QSym of quasisymmetric functions, equipped with the character $\zeta_Q : F_\alpha \mapsto [\alpha = (n)]$ (the “forget compositions” character), is the **terminal object** in the category of combinatorial Hopf algebras: for every (H, ζ_H) , there is a unique morphism of graded Hopf algebras $\Psi : H \rightarrow \text{QSym}$ making

$$\zeta_Q \circ \Psi = \zeta_H.$$

12.8.4 Application: Stanley’s Chromatic Symmetric Function

Definition 209 (Chromatic character on $\mathcal{G}$). Define $\zeta_{\text{chr}} : \mathcal{G} \rightarrow \mathbb{Q}$ by $\zeta_{\text{chr}}(G) =$ number of proper 1-colorings of G (which is 1 if G has no edges and 0 otherwise). Then ζ_{chr} is a character on $\mathcal{G}$.

Theorem 282 (Stanley, 1995; ABS reformulation). The unique graded Hopf algebra morphism $\Psi : \mathcal{G} \rightarrow \text{QSym}$ satisfying $\zeta_Q \circ \Psi = \zeta_{\text{chr}}$ sends G to the **chromatic symmetric function**:

$$\Psi(G) = X_G(x_1, x_2, \dots) = \sum_{\kappa \text{ proper}} \prod_{v \in V(G)} x_{\kappa(v)}.$$

In particular X_G is symmetric (not merely quasisymmetric).

Remark 155 (Positivity of X_G). The celebrated **Stanley–Stembridge conjecture** (1993) asserts that if G is the incomparability graph of a $(3+1)$ -free poset, then X_G is a non-negative integer combination of the elementary symmetric functions e_λ ; it was proved by Hikita (2024). The weaker statement that such X_G is a non-negative combination of Schur functions s_λ was proved by Gasharov (1996). The conjecture, recorded by Stanley (1998), that X_G is Schur positive for every claw-free graph G is false: in 2026 Matherne–Morales and, independently, Prajapati found claw-free counterexamples on 12 vertices. For instance, if G is the line graph of the graph obtained from a 4-cycle by attaching triangles at two opposite vertices and pendant edges at the other two, then $[s_{(3,3,3,3)}]X_G = -64$.

12.8.5 Other Combinatorial Hopf Algebras

Example 67 (Posets). The Hopf algebra of finite posets, with disjoint union as product and $\Delta(P) = \sum_{I \text{ order ideal}} I \otimes (P \setminus I)$, maps under ABS to the P -partition generating function (Stanley, Gessel).

Example 68 (Matroids). Schmitt’s matroid Hopf algebra has $\Delta(M) = \sum_{A \subseteq E(M)} M|_A \otimes M/A$ and maps to QSym via the rank generating function. The image recovers the Tutte polynomial as a quasisymmetric specialisation.

Example 69 (The Connes–Kreimer Hopf algebra of trees). Used in renormalisation theory of quantum field theory. Coproduct extracts admissible cuts of a rooted tree; antipode encodes the “BPHZ” renormalisation procedure.

12.8.6 Exercises

Easy Questions.

Exercise 12.57. Verify that the coproduct $\Delta(G) = \sum_{S \subseteq V} G[S] \otimes G[V \setminus S]$ is coassociative on the path P_3 .

Exercise 12.58. Compute $\Delta(K_3)$ in $\mathcal{G}$ explicitly. The result should be a sum of $2^3 = 8$ terms, including the trivial pieces $K_3 \otimes \emptyset$ and $\emptyset \otimes K_3$.

Medium Questions.

Exercise 12.59. Verify Theorem 282 for $G = K_3$: compute X_{K_3} directly from the definition, then show that the unique character-preserving map $\mathcal{G} \rightarrow \text{QSym}$ produces the same symmetric function.

Exercise 12.60. Show that the chromatic polynomial $\chi_G(k)$ is the principal specialisation of X_G : $\chi_G(k) = X_G(1, 1, \dots, 1, 0, 0, \dots)$ with k ones.

Hard Questions.

Exercise 12.61. Prove the Humpert–Martin antipode formula for $\mathcal{G}$ stated in this section. (Hint: start from Schmitt’s formula $S(G) = \sum_{\pi} (-1)^{|\pi|} |\pi|! G_{\pi}$, the sum over set partitions π of $V(G)$, where G_{π} is the disjoint union of the induced subgraphs on the blocks; group the terms according to the flat $F = E(G_{\pi})$, and use Stanley’s theorem $\chi_H(-1) = (-1)^{|V(H)|} a(H)$ with $H = G/F$.)

Exercise 12.62. Use the Aguiar–Bergeron–Sottile universal property (Theorem 281) to derive the Stanley–Reisner h -vector formula for the order complex of a poset.

Diagram Questions.

Exercise 12.63. Draw all $2^4 = 16$ pairs of induced subgraphs of C_4 = the 4-cycle that appear in $\Delta(C_4)$. Group them by the size of the left factor and verify the Δ sum.

Exercise 12.64. Sketch the Hopf algebra structure on rooted trees of size ≤ 3 via the Connes–Kreimer coproduct $\Delta(T) = \sum_c T_c^{\text{trunk}} \otimes T_c^{\text{branches}}$ over admissible cuts c . Compute Δ on the path P_3 and the star S_3 .

12.9 Hodge Theory of Matroids and Log-Concavity

The chromatic polynomial of a graph—and more generally the characteristic polynomial of a matroid—has integer coefficients with *alternating signs* (Whitney, 1932). Read (1968) conjectured that these coefficients in absolute value are unimodal and Hoggar (1974) that they form a *log-concave* sequence; Rota, Heron and Welsh extended the conjecture to characteristic polynomials of all matroids, and Welsh and Mason conjectured log-concavity for the f -vector of any matroid. Both conjectures are now theorems, proved using combinatorial Hodge theory by Adiprasito–Huh–Katz (2018), bringing algebraic-geometric machinery to bear on a purely combinatorial question. This section is the most advanced in the chapter and sketches the key ideas at a graduate level.

12.9.1 Log-Concavity Conjectures

Definition 210 (Log-concave sequence). A finite sequence $a_0, a_1, \dots, a_n$ of non-negative reals is **log-concave** if $a_i^2 \geq a_{i-1}a_{i+1}$ for $1 \leq i \leq n-1$. It is **ultra log-concave** if $a_i/\binom{n}{i}$ is log-concave.

Theorem 283 (Heron–Rota–Welsh / Adiprasito–Huh–Katz, 2018). Let M be a matroid of rank r on ground set E , and let $\chi_M(t) = \sum_{i=0}^r (-1)^i w_i t^{r-i}$ be its characteristic polynomial, where $w_i \geq 0$ are the **Whitney numbers of the first kind**. Then $(w_0, w_1, \dots, w_r)$ is a log-concave sequence with no internal zeros.

Corollary 65 (Read–Hoggar conjecture; Huh, 2012). For any graph G on n vertices, the coefficients of the chromatic polynomial $\chi_G(k) = \sum_{i=0}^{n-1} (-1)^i a_i k^{n-i}$ satisfy $a_i^2 \geq a_{i-1}a_{i+1}$.

Theorem 284 (Mason’s conjecture; Anari–Liu–Oveis Gharan–Vinzant; Brändén–Huh, 2018). For any matroid M on a ground set E with $|E| = n$, the f -vector $(f_0, f_1, \dots, f_r)$ counting independent sets by size is ultra-log-concave in the sense that $f_i/\binom{n}{i}$ is log-concave (here $n = |E|$, not the rank r). The weaker statement that (f_i) itself is log-concave was proved by Adiprasito–Huh–Katz.

12.9.2 The Chow Ring of a Matroid

Definition 211 (Bergman fan and Chow ring; Feichtner–Yuzvinsky, 2004). Let M be a loopless matroid on $E = [n]$ with lattice of flats L_M and rank function r . The **Chow ring** $A^*(M)$ is the $\mathbb{Z}$ -graded commutative ring

$$A^*(M) = \mathbb{Z}[x_F : F \in L_M, F \neq \hat{0}, F \neq \hat{1}] / (I_1 + I_2),$$

where:

- I_1 is generated by $x_F x_{F'}$ for incomparable flats F, F' (the **Stanley–Reisner relations**).
- I_2 is generated by $\sum_{F \ni i} x_F - \sum_{F \ni j} x_F$ for $i, j \in E$ (the **linear relations**).

The grading places $\deg(x_F) = 1$. The ring is concentrated in degrees $0, 1, \dots, r-1$, and $A^{r-1}(M)_{\mathbb{R}}$ is one-dimensional: there is a unique linear isomorphism $\deg : A^{r-1}(M)_{\mathbb{R}} \rightarrow \mathbb{R}$ with $\deg(x_{F_1} x_{F_2} \cdots x_{F_{r-1}}) = 1$ for every flag $F_1 \subsetneq F_2 \subsetneq \cdots \subsetneq F_{r-1}$ of nonempty proper flats. The Whitney numbers are *not* the dimensions $\dim A^k(M)$, which are palindromic by Poincaré duality (e.g. for $U_{3,4}$ they are 1, 7, 1, while $(w_0, \dots, w_3) = (1, 4, 6, 3)$). Instead they come from intersection numbers. Fix $i \in E$ and let $\alpha = \sum_{F \ni i} x_F$ and $\beta = \sum_{F \not\ni i} x_F$ (sums over nonempty proper flats; neither class depends on i). Then the reduced characteristic polynomial $\chi_M(t)/(t-1) = \sum_{k=0}^{r-1} (-1)^k \mu^k t^{r-1-k}$ satisfies $\mu^k = \deg(\alpha^{r-1-k} \beta^k)$ (Adiprasito–Huh–Katz), and $w_k = \mu^k + \mu^{k-1}$.

Theorem 285 (Adiprasito–Huh–Katz Hodge package). For every loopless matroid M of rank r , the Chow ring $A^*(M)_{\mathbb{R}}$ satisfies the analogue of the Kähler package from algebraic geometry:

1. **Poincaré duality**: there is a degree map $\deg : A^{r-1}(M) \rightarrow \mathbb{R}$ such that the induced pairing $A^k \otimes A^{r-1-k} \rightarrow \mathbb{R}$ is non-degenerate.
2. **Hard Lefschetz**: for any *strictly convex* element $\ell \in A^1(M)$, multiplication by ℓ^{r-1-2k} is an isomorphism $A^k \rightarrow A^{r-1-k}$.
3. **Hodge–Riemann relations**: the bilinear form $(\eta_1, \eta_2) \mapsto (-1)^k \deg(\eta_1 \cdot \ell^{r-1-2k} \cdot \eta_2)$ is positive definite on the primitive part $\ker(\ell^{r-2k} \cdot - : A^k \rightarrow A^{r-k})$.

12.9.3 From Hodge–Riemann to Log-Concavity

Theorem 286 (How HR implies log-concavity). Let M be a loopless matroid of rank $r \geq 3$ and let $\ell_1, \ell_2 \in A^1(M)_{\mathbb{R}}$, where ℓ_2 is nef (a limit of strictly convex elements). Then

$$\deg(\ell_1 \ell_2^{r-2})^2 \geq \deg(\ell_1^2 \ell_2^{r-3}) \cdot \deg(\ell_2^{r-1}).$$

The classes α, β of Definition 211 are nef, and for $\ell_1 = \alpha, \ell_2 = \beta$ this Cauchy–Schwarz-type inequality reads $(\mu^{r-2})^2 \geq \mu^{r-3} \mu^{r-1}$. Applying it to the truncations of M gives $(\mu^k)^2 \geq \mu^{k-1} \mu^{k+1}$ for $1 \leq k \leq r-2$, and Theorem 283 follows because $\chi_M(t)$ is $(t-1)$ times the reduced characteristic polynomial and the convolution of two log-concave sequences is log-concave.

Proof sketch of Theorem 286. First let ℓ_2 be strictly convex, and put $Q(\eta_1, \eta_2) = \deg(\eta_1 \eta_2 \ell_2^{r-3})$ on A^1 . By Hard Lefschetz, $A^1 = \mathbb{R}\ell_2 \oplus P^1$ with $P^1 = \ker(\ell_2^{r-2} \cdot - : A^1 \rightarrow A^{r-1})$, and $Q(\ell_2, P^1) = 0$; by the Hodge–Riemann relations in degrees 0 and 1, $Q(\ell_2, \ell_2) > 0$ and Q is negative definite on P^1 . So Q has signature $(1, \dim A^1 - 1)$. If ℓ_1, ℓ_2 are linearly independent, the restriction of Q to $\text{span}(\ell_1, \ell_2)$ contains the positive vector ℓ_2 but is not positive definite; hence the Gram determinant $Q(\ell_1, \ell_1)Q(\ell_2, \ell_2) - Q(\ell_1, \ell_2)^2$ is ≤ 0 (it is 0 if they are dependent), which is the inequality. For nef ℓ_2 , apply this to $\ell_2 + t\ell$ with ℓ strictly convex and let $t \rightarrow 0$. (A positivity hypothesis is essential: for nonzero Q -orthogonal $\eta_1, \eta_2 \in P^1$ one has $Q(\eta_1, \eta_1)Q(\eta_2, \eta_2) > 0 = Q(\eta_1, \eta_2)^2$.) $\square$

Remark 156 (The Hodge–Riemann elephant in the room). *For realizable matroids (those arising from a hyperplane arrangement over $\mathbb{C}$), the Chow ring $A^*(M)$ coincides with the cohomology ring of a projective variety (the wonderful compactification of the arrangement complement, due to De Concini–Procesi). For such varieties Hodge–Riemann is a classical theorem of Kähler geometry. The miracle of Adiprasito–Huh–Katz is that the Kähler package holds combinatorially, even when the matroid is non-realizable (e.g. the Vámos matroid, the non-Pappus matroid). The proof uses an intricate inductive flip argument on the lattice of flats.*

12.9.4 Spectral Independence: A Modern Cousin

Theorem 287 (Anari–Liu–Oveis Gharan–Vinzant, 2019). Let M be a matroid of rank r on n elements and let μ be the uniform distribution on bases of M . The down-up walk on bases of M has spectral gap at least $1/r$, and started from any basis it is within total variation distance ε of μ after at most $r \log(n^r/\varepsilon) \leq r^2 \log(n/\varepsilon)$ steps.

Remark 157. *Theorem 287 resolved a conjecture of Mihail–Vazirani (1989) that the bases-exchange graph of every matroid has edge expansion at least 1. The proof combines the (strong) log-concavity of the basis generating polynomial of M —which Anari–Oveis Gharan–Vinzant derived from the Hodge–Riemann relations of Adiprasito–Huh–Katz—with the local-to-global spectral bounds of Kaufman–Oppenheim for high-dimensional expanders. Anari–Liu–Oveis Gharan (2020) developed this viewpoint into the technique of **spectral independence**, which has since become foundational in the modern probabilistic method (e.g. for sampling proper colorings, independent sets, and Ising / Potts models).*

12.9.5 Exercises

Easy Questions.

Exercise 12.65. Verify the Read–Hoggar log-concavity conjecture by hand for the chromatic polynomial of C_4 : $\chi_{C_4}(k) = (k-1)^4 + (k-1) = k^4 - 4k^3 + 6k^2 - 4k + (k-1) + 1$. Identify the absolute coefficients and check $a_i^2 \geq a_{i-1}a_{i+1}$.

Exercise 12.66. For the uniform matroid $U_{3,5}$ of rank 3 on 5 elements, compute the f -vector $(f_0, f_1, f_2, f_3) = (1, 5, 10, 10)$ and verify ultra-log-concavity: $f_i/\binom{5}{i}$ should be log-concave.

Medium Questions.

Exercise 12.67. Prove that log-concavity is preserved under *Hadamard products*: if (a_i) and (b_i) are non-negative log-concave, so is $(a_i b_i)$.

Exercise 12.68. Use Theorem 286 (Hodge–Riemann implies log-concavity) to re-derive the log-concavity of the coefficients of the chromatic polynomial of a cycle C_n via the Chow ring of the cycle matroid $U_{n-1,n}$.

Hard Questions.

Exercise 12.69. Construct the Chow ring $A^*(U_{2,3})$ of the uniform matroid of rank 2 on 3 elements explicitly, including its Stanley–Reisner relations. Verify Poincaré duality and Hard Lefschetz directly.

Exercise 12.70. Show that the matroid Hodge–Riemann relations of Theorem 285 reduce, in the case of a realisable matroid, to the Hodge–Riemann relations on the cohomology of the De Concini–Procesi wonderful compactification.

Diagram Questions.

Exercise 12.71. Draw the lattice of flats of the cycle matroid of K_4 (which is the partition lattice Π_4 , with 15 elements). Identify the Whitney numbers of the second kind W_i = number of flats of rank i , and verify that $(W_0, W_1, W_2, W_3) = (1, 6, 7, 1)$ is log-concave. Compare with the Whitney numbers of the first kind $(w_0, w_1, w_2, w_3) = (1, 6, 11, 6)$ read off from $\chi_{M(K_4)}(t) = (t-1)(t-2)(t-3)$.

Exercise 12.72. Sketch the geometric picture: for the rank-2 uniform matroid $U_{2,4}$, draw the Bergman fan in $\mathbb{R}^3$ as a tropical linear space, and identify the cones corresponding to the flats of $U_{2,4}$.

12.10 The Permanent, Matching Counting, and #P-Hardness

Graph-theoretic motivation. The number of perfect matchings of a bipartite graph equals the *permanent* of its biadjacency matrix—an algebraic invariant that looks like the determinant but lacks its sign cancellations. This single observation places the seemingly innocuous problem “count the perfect matchings of G ” at the head of an entire complexity-theoretic hierarchy: it is the canonical #P-complete problem (Valiant, 1979), in stark contrast to “decide whether G has a perfect matching” (which is in P by the Hopcroft–Karp algorithm of §7).

12.10.1 The Permanent and Bipartite Matchings

Definition 212 (Permanent). For an $n \times n$ matrix $A = (a_{ij})$ over a commutative ring, the **permanent** is

$$\text{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i,\sigma(i)}.$$

This is the determinant formula with the sign $\text{sgn}(\sigma)$ removed.

Theorem 288 (Permanent counts perfect matchings). Let $G = (A \cup B, E)$ be a bipartite graph with $|A| = |B| = n$ and biadjacency matrix $M \in \{0, 1\}^{n \times n}$ (with $M_{ij} = 1$ iff $a_i b_j \in E$). Then

$$\text{perm}(M) = \#\{\text{perfect matchings of } G\}.$$

Proof. A non-zero contribution from $\sigma \in S_n$ requires $M_{i,\sigma(i)} = 1$ for all i , i.e. $\{a_i b_{\sigma(i)}\}$ is a perfect matching. Conversely, every perfect matching is encoded by exactly one permutation. Each contributes 1. $\square$

12.10.2 Ryser’s Inclusion–Exclusion Formula

Theorem 289 (Ryser, 1963). For an $n \times n$ matrix A ,

$$\text{perm}(A) = (-1)^n \sum_{S \subseteq [n]} (-1)^{|S|} \prod_{i=1}^n \left(\sum_{j \in S} a_{ij} \right).$$

Evaluated with the subsets S in Gray-code order, this formula computes the permanent in $O(n \cdot 2^n)$ arithmetic operations—the fastest known algorithm for general matrices.

Proof sketch. By inclusion–exclusion on the columns. Subtract off contributions from σ avoiding column j , then add back those avoiding two columns, and so on. $\square$

Remark 158. *Despite its exponential complexity, Ryser’s formula is the basis of many practical permanent codes; the Balasubramanian–Bax–Franklin–Glynn formula*

$$\text{perm}(A) = \frac{1}{2^{n-1}} \sum_{\delta} \left(\prod_{k=1}^n \delta_k \right) \prod_{j=1}^n \sum_{i=1}^n \delta_i a_{ij},$$

summed over $\delta \in \{\pm 1\}^n$ with $\delta_1 = 1$, has the same $O(n \cdot 2^n)$ cost with half as many terms.

12.10.3 Valiant’s Theorem: #P-Completeness

Theorem 290 (Valiant, 1979; Dagum–Luby, 1992). Computing the permanent of a 0/1 matrix is **#P-complete** (Valiant), even when restricted to matrices arising from 3-regular bipartite graphs (Dagum–Luby).

Remark 159. *#P is the counting analogue of NP: a function f is in #P if there is a polynomial-time non-deterministic machine M such that $f(x)$ = number of accepting paths of M on x . Valiant’s theorem says that the seemingly innocent problem “count perfect matchings” is **strictly harder than** all of NP unless the polynomial hierarchy collapses. In particular, $\text{P}^{\#P}$ contains the entire polynomial hierarchy (Toda’s theorem, 1989).*

12.10.4 The Permanent–Determinant Gap

Theorem 291 (Pólya’s question; Szegő, 1913). For $n \geq 3$ there is no fixed pattern of sign changes of the entries of $n \times n$ matrices A such that the resulting matrix D satisfies $\det(D) = \text{perm}(A)$ for all A . Indeed, for $A = J_n$ (the all-ones matrix) every such D is a ± 1 matrix, so $|\det D| \leq n^{n/2} < n! = \text{perm}(J_n)$ by Hadamard’s inequality.

Theorem 292 (Kasteleyn, 1961–1967). For a planar graph G , the number of perfect matchings can be computed in polynomial time via the FKT *Pfaffian* algorithm: choose a special ± 1 orientation (*Kasteleyn orientation*) of the edges so that the Pfaffian of the resulting skew-symmetric matrix equals the matching count.

Remark 160. *Kasteleyn’s algorithm is the historical seed of holographic algorithms (Valiant, 2008): a meta-technique that maps certain #P-hard counting problems to determinants/Pfaffians via tensor contractions on planar graphs.*

12.10.5 Approximate Counting: JSV’s FPRAS

Theorem 293 (Jerrum–Sinclair–Vigoda, 2004). There is a fully polynomial randomized approximation scheme (FPRAS) for the permanent of any non-negative matrix: an algorithm that, given $A \in \mathbb{R}_{\geq 0}^{n \times n}$ and $\varepsilon, \delta > 0$, computes $\hat{P}$ with $\Pr[|\hat{P} - \text{perm}(A)| > \varepsilon \text{perm}(A)] \leq \delta$ in time $\text{poly}(n, 1/\varepsilon, \log(1/\delta))$.

Proof sketch. Construct a Markov chain on the union of the set of perfect matchings and *near-perfect* matchings (matchings with $n - 1$ edges, which leave exactly one vertex uncovered on each side). Bound the spectral gap using a sophisticated canonical-paths analysis. Use the chain to sample matchings nearly uniformly, then estimate the permanent by sampling and reweighting along a chain of auxiliary matrices. $\square$

12.10.6 Graph-Theoretic Synthesis

The permanent is the bridge between three areas:

- **Combinatorics:** counts matchings, 0/1 tilings, Latin squares, and lattice tilings.
- **Linear algebra:** a multilinear form in the rows that, unlike the alternating determinant, is *symmetric*: it is invariant under all row and column permutations (and under transposition).
- **Complexity:** the canonical #P-complete problem, whose tractability boundary (planar / dense / non-negative) traces a frontier of computational complexity.

Modern research includes the Barvinok polynomial interpolation method, which gives a deterministic algorithm, running in quasi-polynomial time $n^{O(\log n - \log \varepsilon)}$, that *approximates* the permanent within a factor $e^{\pm \varepsilon}$ for real matrices whose entries all lie between a fixed $\delta > 0$ and 1 (Barvinok, 2017).

12.10.7 Exercises

Easy Questions.

Exercise 12.73. Compute $\text{perm}(A)$ and $\det(A)$ for $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ and verify that $\text{perm}(A) = 1$ counts the unique perfect matching of P_4 (viewed as a bipartite graph $\{a_1, a_2\} \cup \{b_1, b_2\}$).

Exercise 12.74. Use Theorem 288 to count perfect matchings of $K_{3,3}$: its biadjacency matrix is the all-ones 3×3 matrix J_3 , so $\text{perm}(J_3) = 3! = 6$.

Medium Questions.

Exercise 12.75. Apply Ryser's formula (Theorem 289) to compute $\text{perm}(J_4)$ (the all-ones 4×4 matrix), verifying the count $4! = 24$ for perfect matchings of $K_{4,4}$.

Exercise 12.76. For a planar bipartite graph G , sketch the construction of a Kasteleyn orientation (Theorem 292). Apply it to the 2×2 grid graph and verify that the Pfaffian count matches the direct enumeration of perfect matchings.

Hard Questions.

Exercise 12.77. Show that computing $\text{perm}(A) \bmod 2$ for a 0/1 matrix is in P (it equals $\det(A) \bmod 2$), whereas Valiant (1979) showed that computing $\text{perm}(A)$ modulo any fixed integer that is not a power of 2 (for instance modulo 3) is UP-hard. This sharp dichotomy illustrates the depth of Theorem 290.

Exercise 12.78. Use the Bregman–Minc inequality $\text{perm}(A) \leq \prod_i (r_i!)^{1/r_i}$ for a 0/1 matrix A with row sums r_i , to bound the number of perfect matchings of a d -regular bipartite graph by $(d!)^{n/d}$.

Diagram Questions.

Exercise 12.79. Draw the bipartite graph corresponding to the biadjacency matrix $\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$ and enumerate its perfect matchings to verify the permanent is 2.

Exercise 12.80. For the 3×3 grid bipartite graph (vertices = cells, edges = horizontal/vertical adjacency), draw a Kasteleyn orientation and verify the Pfaffian count $\# \text{perfect matchings} = 0$ (impossible because of the odd-vertex parity).

12.11 Weisfeiler–Leman, Color Refinement, and Graph Isomorphism

Graph-theoretic motivation. Two graphs G and H are *isomorphic* if there is a bijection of vertices preserving adjacency. Deciding graph isomorphism (GI) is the classical problem of testing structural equality of graphs; since Babai’s work (announced in 2015; an error in the analysis, found by Helfgott, was repaired by Babai in January 2017) it has been known to be solvable in quasipoly(n) = $n^{O((\log n)^c)}$ time, placing it in a rare complexity-theoretic position between P and NP-hard. The algebraic core of every modern GI algorithm is the *Weisfeiler–Leman* (WL) hierarchy: an iterative colour-refinement procedure that captures graph invariants of increasing strength.

12.11.1 Color Refinement (1-WL)

Definition 213 (Color refinement / 1-WL). Initialize each vertex of G with the colour $c^0(v) = \text{degree of } v$. At iteration $t \geq 1$, set

$$c^t(v) = \text{hash}(c^{t-1}(v), \{\{c^{t-1}(u) : u \in N(v)\}\}),$$

where $\{\{\cdot\}\}$ is a multiset and hash is any injective encoding of (vertex-colour, neighbour-multiset) pairs. Iterate until the partition of $V(G)$ by colours stabilises (in at most $|V(G)|$ rounds).

Theorem 294 (1-WL test). Two graphs G, H are **1-WL distinguishable** if the colour multisets of G and H differ at some iteration. If G and H are 1-WL indistinguishable, they have the same degree sequence and many finer invariants—but they may still be non-isomorphic.

Example 70 (1-WL fails to distinguish CFI graphs). The *Cai–Fürer–Immerman* (1992) construction produces, for every k , a pair of non-isomorphic graphs on $O(k)$ vertices (which can even be taken 3-regular) that are indistinguishable by k -WL. In particular, for any fixed k , k -WL is strictly weaker than graph isomorphism testing.

12.11.2 Higher-Dimensional Weisfeiler–Leman

Definition 214 (k -dimensional WL). For $k \geq 2$, k -WL operates on k -tuples of vertices. Initialize $c^0(\vec{v}) =$ isomorphism type of the induced subgraph on $\{v_1, \dots, v_k\}$ (with labelled vertices). At iteration t ,

$$c^t(\vec{v}) = \text{hash}(c^{t-1}(\vec{v}), (\{\{c^{t-1}(\vec{v}[i \rightarrow w]) : w \in V\}\}_{i=1}^k)),$$

where $\vec{v}[i \rightarrow w]$ replaces the i -th coordinate of $\vec{v}$ with w . Iterate until stabilisation.

Theorem 295 (Cai–Fürer–Immerman, 1992). For every k there exists a graph pair (G_k, H_k) such that $G_k \not\cong H_k$ but k -WL fails to distinguish them. Thus no fixed level of the k -WL hierarchy decides graph isomorphism.

Remark 161. Two indexing conventions are in use. Definition 214, with a separate multiset for each coordinate, is the k -WL of the machine-learning literature (Grohe calls it the oblivious k -WL). It has exactly the distinguishing power of the $(k-1)$ -dimensional WL algorithm in the convention of Cai–Fürer–Immerman and Grohe (the “folklore” $(k-1)$ -WL of the machine-learning literature), and hence of the k -variable counting logic C^k . In particular, with Definition 214, 2-WL distinguishes exactly the same pairs of graphs as 1-WL (neither distinguishes C_6 from two disjoint triangles), so the hierarchy is not strictly increasing at the first step; from $k = 2$ on, each level is strictly stronger than the previous one.

12.11.3 Babai’s Quasi-Polynomial Algorithm

Theorem 296 (Babai, 2015–2017). Graph isomorphism is solvable in time $n^{O((\log n)^c)}$ for some absolute constant c .

Remark 162. Babai’s algorithm uses a deep blend of group theory and the Weisfeiler–Leman hierarchy. Its central divide-and-conquer tool is the group-theoretic “local certificates” routine. This is combined with a combinatorial “split-or-Johnson” procedure, which either finds a canonical splitting (reducing to smaller instances) or finds a canonically embedded large Johnson graph; in a well-defined sense, Johnson graphs are the only obstructions to effective canonical partitioning.

12.11.4 WL and Graph Neural Networks

Theorem 297 (Xu–Hu–Leskovec–Jegelka; Morris et al., 2019). The expressive power of standard message-passing graph neural networks (GNNs) for distinguishing graphs is at most that of 1-WL: if 1-WL cannot distinguish G from H , then no GNN with the same number of layers can output different graph-level representations on G and H .

Remark 163. *Theorem 297 sparked an explosion of higher-order GNN architectures (e.g. k -GNN and PPGN) explicitly designed to overcome the 1-WL bottleneck. The resulting GNN–WL correspondence is now standard background in graph machine learning.*

12.11.5 Exercises

Easy Questions.

Exercise 12.81. Apply 1-WL to the cycle C_4 and to the disjoint union of two C_3 's. Confirm that 1-WL produces different colour multisets, distinguishing the graphs.

Exercise 12.82. Show that 1-WL cannot distinguish the cycle C_6 from the disjoint union of two C_3 's. (Both are 2-regular with the same number of vertices.)

Medium Questions.

Exercise 12.83. Construct a pair of 3-regular non-isomorphic graphs on $n = 8$ vertices that are 1-WL indistinguishable. (Hint: use a small CFI-style construction.)

Exercise 12.84. Prove that k -WL is at least as strong as $(k - 1)$ -WL: any pair distinguished by $(k - 1)$ -WL is also distinguished by k -WL.

Hard Questions.

Exercise 12.85. State the precise CFI gadget construction (Theorem 295) and verify that k -WL on the resulting graphs does not distinguish them.

Exercise 12.86. Prove that 1-WL is equivalent to vertex colour-refinement *in the sense of equitable partitions*: the stable colouring of 1-WL on G corresponds to the coarsest equitable partition of $V(G)$. Connect this to the spectrum of the quotient matrix from §12.2.

Diagram Questions.

Exercise 12.87. Draw the trace of 1-WL on the Petersen graph: at each iteration, label each vertex with its current colour. Observe that the algorithm stabilises in ≤ 2 iterations with a single colour class, as it must for every vertex-transitive graph. Explain why this does *not* confirm vertex-transitivity: the 3-regular Frucht graph also receives a single colour, yet its only automorphism is the identity.

Exercise 12.88. For a small CFI gadget on 4 vertices, draw both graphs and label the vertices with their 1-WL colours after stabilisation. Verify that 1-WL fails to distinguish them.

12.12 Cluster Algebras from Quivers

Graph-theoretic motivation. A *quiver* is just a directed graph (with multiple edges allowed but no self-loops or oriented 2-cycles). Out of any such graph, Fomin and Zelevinsky (2002) conjured an algebraic structure—a *cluster algebra*—generated by recursive applications of a graph operation called *mutation*. Cluster algebras have since appeared in dozens of mathematical fields: total positivity in Lie groups, Teichmüller theory of surfaces, Calabi–Yau categories, scattering amplitudes in physics, and the discrete dynamics of Y - and T -systems. This subsection treats only the combinatorial heart: mutation as a graph operation, and the Fomin–Zelevinsky finite-type classification.

12.12.1 Quivers, Skew-Symmetric Matrices, and Mutation

Definition 215 (Quiver). A **quiver** Q is a directed multigraph with vertex set $Q_0 = [n]$ and (multi)set of arrows Q_1 . We assume Q has no loops and no oriented 2-cycles. Encode Q by its **exchange matrix** $B = B(Q) \in \mathbb{Z}^{n \times n}$ with $B_{ij} = (\text{number of arrows } i \rightarrow j) - (\text{number of arrows } j \rightarrow i)$. B is skew-symmetric.

Definition 216 (Mutation at vertex k). The **mutation** μ_k of an exchange matrix B produces a new matrix B' with entries

$$B'_{ij} = \begin{cases} -B_{ij} & \text{if } i = k \text{ or } j = k, \\ B_{ij} + \frac{1}{2}(|B_{ik}|B_{kj} + B_{ik}|B_{kj}|) & \text{otherwise.} \end{cases}$$

Equivalently, in terms of the quiver: reverse all arrows incident to vertex k , and for every directed path $i \rightarrow k \rightarrow j$ add a new arrow $i \rightarrow j$ (cancelling against any existing $j \rightarrow i$ arrows). $\mu_k^2 = \text{id}$.

Example 71 (Mutation on a triangle quiver). Consider the quiver on $\{1, 2, 3\}$ with arrows $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$ (an oriented C_3). After mutating at vertex 1: arrows $1 \rightarrow 2$ and $3 \rightarrow 1$ are reversed; the path $3 \rightarrow 1 \rightarrow 2$ contributes a new arrow $3 \rightarrow 2$, which cancels the existing $2 \rightarrow 3$. Result: the quiver with arrows $2 \rightarrow 1, 1 \rightarrow 3$, and nothing between 2 and 3 (i.e., A_3 orientation).

12.12.2 Cluster Algebras

Definition 217 (Cluster algebra of a quiver). Fix variables $x_1, \dots, x_n$ (an *initial cluster*) and a quiver Q on $[n]$. The cluster algebra $\mathcal{A}(Q)$ is the subring (the $\mathbb{Z}$ -subalgebra) of the field $\mathbb{Q}(x_1, \dots, x_n)$ generated by all *cluster variables* obtained from $(x_1, \dots, x_n; Q)$ by iterated mutation, where mutation at k replaces x_k by

$$x'_k = \frac{1}{x_k} \left(\prod_{i:i \rightarrow k} x_i^{[B_{ik}]_+} + \prod_{j:k \rightarrow j} x_j^{[-B_{jk}]_+} \right).$$

Theorem 298 (Laurent phenomenon; Fomin–Zelevinsky, 2002). Every cluster variable obtained by iterated mutation is a Laurent polynomial in the initial variables $x_1, \dots, x_n$ with integer coefficients.

Theorem 299 (Positivity; Lee–Schiffler, 2015; Gross–Hacking–Keel–Kontsevich, 2018). The Laurent polynomials of Theorem 298 have **non-negative** integer coefficients.

12.12.3 Finite-Type Classification

Theorem 300 (Fomin–Zelevinsky, 2003). A cluster algebra $\mathcal{A}(Q)$ has only finitely many cluster variables if and only if the quiver Q is mutation-equivalent to an orientation of a simply-laced Dynkin diagram of type A_n, D_n, E_6, E_7, E_8 .

Remark 164. *The total number of cluster variables in a finite-type cluster algebra depends only on the Dynkin type: $|A_n| \rightarrow \binom{n+2}{2} - 1, |D_n| \rightarrow n^2, |E_6| \rightarrow 42, |E_7| \rightarrow 70, |E_8| \rightarrow 128$. This is the same count as the number of positive roots plus rank, anticipating the deep connection between cluster algebras and Lie theory.*

12.12.4 Triangulations of Surfaces

Theorem 301 (Fomin–Shapiro–Thurston, 2008). The cluster algebra of a triangulated marked surface is encoded by the *adjacency quiver* of the triangulation: vertices = arcs; arrows = consecutive pairs of arcs in a triangle. Flips of the triangulation correspond exactly to mutations of the quiver.

Example 72 (Hexagon triangulations). For the hexagon (the disk with 6 marked boundary points), the cluster algebra is of type A_3 . Its 14 triangulations correspond to the 14 clusters; flips between them realise mutation.

12.12.5 Exercises

Easy Questions.

Exercise 12.89. Verify $\mu_k^2 = \text{id}$ on the exchange matrix $B = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ at vertex $k = 1$.

Exercise 12.90. Compute all clusters of the rank-2 cluster algebra of type A_2 (quiver: $1 \rightarrow 2$). The five clusters are $\{x_1, x_2\}, \{x_1, (1 + x_1)/x_2\}, \dots$

Medium Questions.

Exercise 12.91. Verify the Laurent phenomenon for the cluster algebra of type A_2 : all 5 cluster variables are Laurent polynomials in x_1, x_2 with non-negative integer coefficients.

Exercise 12.92. Prove that mutation μ_k on a quiver with no oriented 2-cycles again produces a quiver with no oriented 2-cycles, given that the formula resolves cancellations correctly.

Hard Questions.

Exercise 12.93. Prove that any two adjacent triangulations of a polygon (differing by one diagonal flip) correspond to clusters related by a single mutation, completing the bijection of Theorem 301 for the disk.

Exercise 12.94. Show that the cluster algebra of an orientation of A_n has finitely many clusters (indeed, exactly $\binom{2(n+1)}{n+1}/(n+2)$ Catalan-many, one for each triangulation of an $(n+3)$ -gon), confirming Theorem 300 for type A . (Finiteness of the set of quivers reachable by mutation would not suffice: the Kronecker quiver $1 \rightrightarrows 2$ is fixed by mutation up to relabeling but has infinitely many cluster variables.)

Diagram Questions.

Exercise 12.95. Draw the cluster exchange graph (vertices = clusters) of type A_2 : 5 vertices, each connected to its two neighbours by an edge labeled with the mutation index. Verify it is the pentagon.

Exercise 12.96. Sketch a triangulation of the hexagon with 3 diagonals and the corresponding adjacency quiver. Apply a flip to a chosen diagonal and verify the resulting quiver is the mutation of the original at the corresponding vertex.

12.13 Chip-Firing, Sandpile Groups, and Riemann–Roch for Graphs

Graph-theoretic motivation. On any finite graph, a deceptively simple game called *chip firing* (or the *abelian sandpile model*) gives rise to a discrete dynamical system whose orbit space is a finite abelian group—the *sandpile group* (also called the degree-zero *Picard group*, *critical group*, or *Jacobian*). Baker and Norine (2007) discovered that this group satisfies a graph-theoretic analogue of the *Riemann–Roch theorem* for divisors on a Riemann surface. Chip firing thus realises a small but rigorous version of intersection theory in pure combinatorics, and bridges graph theory with algebraic geometry, tropical geometry, and the theory of metric graphs.

12.13.1 Divisors and Chip Firing

Definition 218 (Divisor). Let G be a connected graph with vertex set V . A **divisor** on G is an element of the free abelian group $\text{Div}(G) = \mathbb{Z}^V$. Write a divisor as $D = \sum_v D(v) \cdot v$. The **degree** of D is $\deg(D) = \sum_v D(v)$.

Definition 219 (Chip-firing move). A **firing** at vertex v replaces $D(v)$ by $D(v) - \deg_G(v)$ and increases $D(u)$ by the number of edges between u and v for each $u \neq v$. In matrix language: $D \rightarrow D - L_v$, where L_v is the row of the graph Laplacian $L = \text{diag}(\deg) - A$ at vertex v . Fired vertices preserve the total chip count $\deg(D)$.

Definition 220 (Linear equivalence). Two divisors D_1, D_2 are **linearly equivalent** ($D_1 \sim D_2$) if $D_1 - D_2$ lies in the *integer* row span $\{x^\top L : x \in \mathbb{Z}^V\}$ of L (over $\mathbb{Q}$ the row span would contain every divisor of degree 0). The quotient $\text{Pic}(G) = \text{Div}(G)/\sim$ is the **Picard group** of G .

12.13.2 The Sandpile Group

Theorem 302 (Sandpile group structure). For a connected graph G on n vertices, $\text{Pic}(G) \cong \mathbb{Z} \oplus K(G)$, where $K(G)$, the **sandpile group**, is a finite abelian group with $|K(G)| = \tau(G)$, the number of spanning trees.

Example 73 (Sandpile group of small graphs).

- $K(K_n) \cong (\mathbb{Z}/n\mathbb{Z})^{n-2}$ (so $|K(K_n)| = n^{n-2}$, recovering Cayley's formula).
- $K(C_n) \cong \mathbb{Z}/n\mathbb{Z}$.
- $K(K_{m,n}) \cong \mathbb{Z}/mn\mathbb{Z} \oplus (\mathbb{Z}/m\mathbb{Z})^{n-2} \oplus (\mathbb{Z}/n\mathbb{Z})^{m-2}$ for $m, n \geq 2$ (Lorenzini).

12.13.3 The Baker–Norine Riemann–Roch Theorem

Definition 221 (Effective divisor; rank). A divisor D is **effective** ($D \geq 0$) if $D(v) \geq 0$ for all v . The **rank** of D is $r(D) = -1$ if no effective divisor is linearly equivalent to D , and otherwise

$$r(D) = \max\{k \geq 0 : D - E \sim E' \geq 0 \text{ for every effective } E \text{ of degree } k\}.$$

Definition 222 (Canonical divisor). The **canonical divisor** of G is $K_G = \sum_v (\deg_G(v) - 2) \cdot v$. Its degree is $2|E| - 2|V| = 2g - 2$, where $g = |E| - |V| + 1$ is the cycle rank (**genus**) of G .

Theorem 303 (Baker–Norine, 2007). For any connected graph G and any divisor D ,

$$r(D) - r(K_G - D) = \deg(D) - g + 1.$$

Remark 165. *Theorem 303 is a literal Riemann–Roch theorem: it has the same form as Riemann–Roch on a smooth projective curve, with the cycle rank g playing the role of geometric genus. The proof is purely combinatorial (no algebraic geometry required), but the deep parallels with algebraic curves were the spark for **tropical geometry** of graphs (Mikhalkin, Baker–Jensen, Caporaso–Viviani).*

12.13.4 Graph-Theoretic Connections

- **Spanning trees:** $|K(G)| = \tau(G)$ recovers Kirchhoff (§11.4) by the Matrix-Tree theorem applied to the Laplacian.
- **Counting recurrent configurations:** the sandpile group elements are in bijection with *recurrent* chip-firing configurations, whose number is $T_G(1, 1) = \tau(G)$; counted by their number of chips, they have generating function equal, up to a power of y , to the Tutte polynomial specialisation $T_G(1, y)$ (Merino López, 1997).
- **Random spanning trees:** Wilson's algorithm (*loop-erased random walk*) generates a uniform spanning tree in expected time proportional to the *mean hitting time* of the simple random walk on G , hence $O(|V||E|)$.
- **Clifford's theorem for graphs:** Riemann–Roch implies the *Clifford bound* $r(D) \leq \deg(D)/2$ for every effective divisor D such that $K_G - D$ is also linearly equivalent to an effective divisor (Baker–Norine), the analogue of the Clifford theorem on algebraic curves.

12.13.5 Exercises

Easy Questions.

Exercise 12.97. Compute the sandpile group of C_4 . (Answer: $\mathbb{Z}/4\mathbb{Z}$, matching $\tau(C_4) = 4$.)

Exercise 12.98. For K_4 , compute the canonical divisor K_{K_4} and verify $\deg(K_{K_4}) = 2g - 2$ where $g = 6 - 4 + 1 = 3$.

Medium Questions.

Exercise 12.99. Verify Theorem 302 for K_3 : compute L explicitly, identify the sandpile group as $\mathbb{Z}/3\mathbb{Z}$, and check that $|K(K_3)| = 3 = \tau(K_3)$.

Exercise 12.100. Apply the Baker–Norine theorem to a divisor of degree g on K_4 and verify $r(D) - r(K_G - D) = \deg(D) - g + 1 = 1$ holds for at least one such D .

Hard Questions.

Exercise 12.101. Prove the matrix-tree count $|K(G)| = \tau(G)$ from Theorem 302: the Smith normal form of the reduced Laplacian gives both the abelian-group structure and the determinant $\det(L_{\text{reduced}}) = \tau(G)$.

Exercise 12.102. Sketch a proof of the Baker–Norine Riemann–Roch theorem (303) via Dhar’s burning algorithm and the q -reduced divisor canonical form.

Diagram Questions.

Exercise 12.103. Draw a chip-firing trajectory on K_4 starting from $D_0 = 5v_1 + 0v_2 + 0v_3 + 0v_4$ until the configuration stabilises. Identify the resulting divisor as a representative of an element of $K(K_4)$.

Exercise 12.104. For the complete bipartite graph $K_{2,3}$, draw all 12 spanning trees and verify $\tau = 12 = |K(K_{2,3})|$ by computing the sandpile group explicitly via Smith normal form.

12.14 Random Walks via Representation Theory

Graph-theoretic motivation. Theorem 261 computed eigenvalues of abelian Cayley graphs from characters; the same idea, generalised to non-abelian groups, gives the sharpest known mixing-time bounds for random walks on highly symmetric graphs. The *Diaconis–Shahshahani upper bound lemma* packages this into a single inequality controlling total-variation distance to stationarity in terms of representation-theoretic data. The technique is the modern engine for bounding mixing times of card-shuffling, lattice walks, and the Markov chains used in the FPRAS algorithms of §12.10.

12.14.1 The Upper Bound Lemma

Definition 223 (Convolution Markov chain). Let Γ be a finite group and $\mu : \Gamma \rightarrow \mathbb{R}$ a probability measure. The **convolution random walk** on Γ driven by μ has transition kernel $P(g, h) = \mu(g^{-1}h)$. After k steps, the law is μ^{*k} (the k -fold convolution power).

Theorem 304 (Diaconis–Shahshahani upper bound lemma, 1981). Let Γ be a finite group with normalised Haar measure U and μ a symmetric probability measure on Γ supported on a generating set. For every $k \geq 0$,

$$4\|\mu^{*k} - U\|_{\text{TV}}^2 \leq \sum_{\rho \neq 1} \dim(\rho) \|\widehat{\mu}(\rho)^k\|_F^2,$$

where the sum is over non-trivial irreducible unitary representations ρ of Γ , $\widehat{\mu}(\rho) = \sum_g \mu(g)\rho(g)$ is the Fourier transform of μ at ρ , and $\|\cdot\|_F$ is the Frobenius norm.

12.14.2 Worked Examples

Example 74 (Random transposition walk on S_n ; Diaconis–Shahshahani, 1981). The random transposition walk on S_n is driven by the measure μ with $\mu(\text{id}) = 1/n$ and $\mu(\tau) = 2/n^2$ for each transposition τ (choose two cards independently and uniformly and swap them). (The uniform measure on transpositions alone would give a periodic walk: μ^{*k} would be supported on permutations of sign $(-1)^k$, at distance at least $1/2$ from U for every k .) For every fixed $\varepsilon \in (0, 1)$, as $n \rightarrow \infty$,

$$\|\mu^{*k} - U\|_{\text{TV}} \longrightarrow \begin{cases} 1 & \text{if } k \leq (1 - \varepsilon)\frac{1}{2}n \log n, \\ 0 & \text{if } k \geq (1 + \varepsilon)\frac{1}{2}n \log n, \end{cases}$$

exhibiting a sharp **cutoff** at time $\frac{1}{2}n \log n$. The bound uses Frobenius’s formula for character values on transpositions in terms of Young-diagram contents.

Example 75 (Riffle shuffle; Bayer–Diaconis, 1992). The Gilbert–Shannon–Reeds riffle shuffle of n cards exhibits cutoff at time $\frac{3}{2} \log_2 n$ (for $n = 52$ this is ≈ 8.5). Cutoff is an asymptotic statement; for $n = 52$ Bayer and Diaconis computed the exact total-variation distances, about 0.92, 0.61, 0.33, 0.17 after 5, 6, 7, 8 shuffles, and this sharp drop is the basis of the famous “seven shuffles” rule of thumb.

Example 76 (Hypercube random walk). The lazy random walk on Q_n (driven by the measure putting mass $1/2$ on the identity, i.e. a self-loop, and mass $1/(2n)$ on each of $e_1, \dots, e_n$) has eigenvalues $\lambda_k = (1 + (1 - 2k/n))/2$. The mixing time is $t_{\text{mix}} = \frac{1}{2}n \log n + cn$, with cutoff at this time.

12.14.3 Cutoff Phenomenon

Definition 224 (Cutoff). A family of Markov chains $\{P_n\}$ with uniform stationary distributions U_n exhibits **cutoff** at times t_n with window $w_n = o(t_n)$ if, writing $d_n(t) = \max_x d_{\text{TV}}(P_n^t(x, \cdot), U_n)$, for every $\varepsilon > 0$ there is $C = C(\varepsilon)$ such that

$$\liminf_{n \rightarrow \infty} d_n(t_n - Cw_n) \geq 1 - \varepsilon \quad \text{and} \quad \limsup_{n \rightarrow \infty} d_n(t_n + Cw_n) \leq \varepsilon.$$

Theorem 305 (Cutoff on random regular graphs; Lubetzky–Sly, 2010). Fix $d \geq 3$. With probability tending to 1 as $n \rightarrow \infty$, the simple random walk on a uniformly random d -regular graph on n vertices exhibits cutoff at time $\frac{d}{d-2} \log_{d-1} n$, with window of order $\sqrt{\log n}$.

Remark 166. *Lubetzky and Peres (2016) proved cutoff at the same time for the simple random walk on every d -regular Ramanujan graph. Whether every sequence of bounded-degree vertex-transitive expanders exhibits cutoff is a well-known open problem (a conjecture of Peres).*

12.14.4 Exercises

Easy Questions.

Exercise 12.105. For the lazy random walk on C_n , verify the eigenvalues $\lambda_k = (1 + \cos(2\pi k/n))/2$ and bound the mixing time by $O(n^2)$.

Exercise 12.106. Apply Theorem 304 to the lazy random walk on $\mathbb{Z}_2^n$ (driven by the measure with mass $1/2$ at 0 and mass $1/(2n)$ on each standard basis vector; the uniform measure on the basis vectors alone gives a periodic walk). Use the character formula to recover the cutoff time $\frac{1}{2}n \log n$.

Medium Questions.

Exercise 12.107. For the random transposition walk on S_n , compute the first-order upper bound on TV-distance using only the trivial representation, the sign representation, and the standard $(n-1)$ -dimensional representation.

Exercise 12.108. Prove the cutoff phenomenon for the lazy random walk on the hypercube Q_n at time $\frac{1}{2}n \log n$, using the Diaconis–Shahshahani lemma and the explicit eigenvalues.

Hard Questions.

Exercise 12.109. State and prove the Diaconis–Shahshahani upper bound lemma (Theorem 304) using the Plancherel formula on Γ .

Exercise 12.110. Fix a prime $\ell \equiv 1 \pmod{4}$. Construct the Lubotzky–Phillips–Sarnak Ramanujan graphs: explicit $(\ell+1)$ -regular Cayley graphs on $\text{PSL}(2, p)$ or $\text{PGL}(2, p)$ for primes $p \equiv 1 \pmod{4}$, $p \neq \ell$ (so the degree is at least 6), and prove they have spectral gap $\Omega(1)$ uniformly in p .

Diagram Questions.

Exercise 12.111. Sketch the TV-distance to stationarity as a function of step number k for the lazy random walk on Q_5 . Highlight the cutoff window around $k = \frac{1}{2} \cdot 5 \cdot \log 5 \approx 4$ steps.

Exercise 12.112. Draw the Cayley graph of S_3 generated by $\{(12), (23)\}$ (the hexagon), and trace the first three steps of the random walk from the identity. Use the explicit non-abelian character formula to compute $\mu^{*1}, \mu^{*2}, \mu^{*3}$.

12.15 Electrical Networks and Effective Resistance

Graph-theoretic motivation. Treat each edge of a graph as a unit resistor; the resulting electrical network is dual to the simple random walk. Many graph-theoretic invariants—spanning tree counts, hitting times, commute times, the Laplacian pseudoinverse—admit clean expressions in terms of *effective resistance*. This is one of the most useful and least technical bridges between linear algebra and graph theory; the standard references are Doyle–Snell’s classic *Random Walks and Electric Networks* (1984) and the modern treatment in Lyons–Peres, *Probability on Trees and Networks* (2016).

12.15.1 Effective Resistance

Definition 225 (Effective resistance). Let G be a connected graph with each edge assigned conductance 1. The **effective resistance** between vertices $u, v \in V(G)$, denoted $R_{\text{eff}}(u, v)$, is the voltage drop $\phi_u - \phi_v$ induced by injecting 1 unit of current at u and extracting it at v , where the voltage function ϕ is determined (up to an additive constant) by Kirchhoff’s laws, $L\phi = e_u - e_v$. Variationally (Dirichlet’s principle), $1/R_{\text{eff}}(u, v)$ is the minimum of the quadratic form $\sum_{e=ij \in E} (\phi_i - \phi_j)^2$ over all ϕ with $\phi_u = 1$ and $\phi_v = 0$. Equivalently, $R_{\text{eff}}(u, v) = (e_u - e_v)^\top L^+(e_u - e_v)$ where L^+ is the Moore–Penrose pseudoinverse of the Laplacian.

Theorem 306 (Foster’s network theorem, 1949). For a connected graph G on n vertices,

$$\sum_{e=ij \in E(G)} R_{\text{eff}}(i, j) = n - 1.$$

Theorem 307 (Kirchhoff’s two-laws and the Laplacian). The voltage function ϕ for unit current from u to v is the unique solution (up to additive constant) to $L\phi = e_u - e_v$. The current flowing along edge ij in the direction $i \rightarrow j$ is $\phi_i - \phi_j$. Energy dissipation $\sum_{ij} (\phi_i - \phi_j)^2 = R_{\text{eff}}(u, v)$.

12.15.2 Spanning Trees and Resistance

Theorem 308 (Kirchhoff’s effective resistance / spanning tree formula). For an edge $e = ij$ in G ,

$$R_{\text{eff}}(i, j) = \frac{\tau(G/e)}{\tau(G)},$$

where G/e is the contraction of e and τ counts spanning trees. Equivalently, the probability that edge e appears in a uniformly random spanning tree is $R_{\text{eff}}(i, j)$.

12.15.3 Hitting and Commute Times

Definition 226 (Hitting time, commute time). For the simple random walk on G , $H(u, v)$ = expected number of steps for the walk started at u to first reach v . The **commute time** is $C(u, v) = H(u, v) + H(v, u)$.

Theorem 309 (Chandra–Raghavan–Ruzzo–Smolensky–Tiware, 1996).

$$C(u, v) = 2|E| \cdot R_{\text{eff}}(u, v).$$

Remark 167. *Theorem 309 translates between random-walk and electrical quantities exactly. Effective resistances are also the basis of the spectral sparsification algorithm of Spielman–Srivastava (2008), which samples each edge with probability proportional to its effective resistance (spectral sparsifiers were introduced by Spielman–Teng, 2004): a graph G can be approximated in nearly-linear time by a sparse graph H such that $L_G \approx L_H$ in the Löwner sense, and consequently $R_{\text{eff}}^G \approx R_{\text{eff}}^H$ everywhere.*

12.15.4 Circle Packings

Theorem 310 (Koebe–Andreev–Thurston circle packing theorem). Every finite simple planar graph admits a *circle packing*: each vertex corresponds to a closed disk in the plane, the disks have pairwise disjoint interiors, and two disks are externally tangent iff the vertices are adjacent. If the graph is maximal planar, the packing is unique up to Möbius transformations and reflections.

Remark 168. For 3-connected planar graphs uniqueness holds for primal–dual circle packings, in which the graph and its planar dual are packed simultaneously with orthogonal crossings (Brightwell–Scheinerman, 1993). Circle packings give a geometric route to bounds on spectral gaps: Spielman and Teng used them to prove that a planar graph on n vertices with maximum degree Δ has $\lambda_2(L) \leq 8\Delta/n$.

12.15.5 Exercises

Easy Questions.

Exercise 12.113. Compute R_{eff} between adjacent vertices of C_4 . Verify Foster’s theorem: $4 \cdot R_{\text{eff}} = 3$.

Exercise 12.114. For the path P_n , derive $R_{\text{eff}}(1, n) = n - 1$ and verify Foster’s theorem.

Medium Questions.

Exercise 12.115. For the Petersen graph, compute R_{eff} between adjacent vertices and between non-adjacent vertices using its spectrum $\{3, 1^{(5)}, -2^{(4)}\}$ and the Laplacian-pseudoinverse formula.

Exercise 12.116. Verify Theorem 308 for K_4 : compute $\tau(K_4) = 16$ and $\tau(K_4/e) = ?$ for an edge e , and check the formula $R_{\text{eff}}(e) = \tau(K_4/e)/\tau(K_4)$.

Hard Questions.

Exercise 12.117. Prove Foster’s network theorem (306) by computing $\text{tr}(LL^+) = n - 1$ via the trace identity.

Exercise 12.118. Prove the Chandra–Raghavan formula $C(u, v) = 2|E|R_{\text{eff}}(u, v)$ by relating both quantities to harmonic functions on $G \setminus \{v\}$ and the Laplacian pseudoinverse.

Diagram Questions.

Exercise 12.119. Draw the circle packing of the cube graph Q_3 on the sphere (stereographically projected to the plane). Verify there are 8 circles, with adjacencies matching the edges.

Exercise 12.120. Sketch a current flow of 1 ampere from one vertex of K_4 to another vertex. Label each edge with the current $\phi_i - \phi_j$. Verify Kirchhoff’s current law at each internal vertex.

12.16 Crystal Graphs and the Plactic Monoid

Graph-theoretic motivation. A *crystal* is an edge-labelled directed graph (introduced by Kashiwara, 1990) whose connected components encode the irreducible representations of a Lie algebra. For type A ($\mathfrak{gl}_n$), crystal graphs are explicitly the graphs on *semistandard Young tableaux*, with edges given by Kashiwara operators $\tilde{e}_i, \tilde{f}_i$. The combinatorial structure of these graphs encodes the Schur expansion of products of Schur functions (the Littlewood–Richardson rule) and recovers the Robinson–Schensted correspondence as a graph homomorphism. This subsection is the most algebraic in the chapter; we develop only enough crystal theory to define the plactic monoid and connect it back to graph theory.

12.16.1 Crystals of Type A_{n-1}

Definition 227 (Crystal). A **type- A_{n-1} crystal** is a finite set B equipped with maps $\tilde{e}_i, \tilde{f}_i : B \rightarrow B \cup \{0\}$ for $i = 1, \dots, n-1$, and weight function $\text{wt} : B \rightarrow \mathbb{Z}^n$, satisfying:

1. $\tilde{f}_i(b) = b'$ iff $\tilde{e}_i(b') = b$ (the operators are partial inverses).
2. $\text{wt}(\tilde{f}_i b) = \text{wt}(b) - \alpha_i$ where $\alpha_i = e_i - e_{i+1}$ is the simple root.
3. Standard string-length axioms.

The **character** of B is $\text{ch}(B) = \sum_{b \in B} x^{\text{wt}(b)}$.

Theorem 311 (Crystal of a polynomial representation). For each partition λ with at most n parts, there is a unique connected crystal $B(\lambda)$ on the semistandard Young tableaux of shape λ over the alphabet $[n]$, with character

$$\text{ch}(B(\lambda)) = s_\lambda(x_1, \dots, x_n),$$

the Schur polynomial.

12.16.2 The Plactic Monoid

Definition 228 (Knuth equivalence). On the free monoid $[n]^*$ (words over $\{1, \dots, n\}$), define the **Knuth equivalence** $\sim_K$ as the congruence generated by

- $acb \sim_K cab$ if $a \leq b < c$,
- $bac \sim_K bca$ if $a < b \leq c$.

Definition 229 (Plactic monoid). The **plactic monoid** $\mathcal{P}_n$ is $[n]^* / \sim_K$, with multiplication inherited from concatenation.

Theorem 312 (Knuth, 1970; Lascoux–Schützenberger, 1981). Knuth equivalence classes are in bijection with semistandard Young tableaux: each word w has a unique tableau $P(w)$ such that $w \sim_K \text{row}(P(w))$ (the row reading word). Hence $\mathcal{P}_n \cong \bigsqcup_\lambda \text{SSYT}(\lambda, n)$ as sets, and the multiplication corresponds to the *Schensted insertion* algorithm.

12.16.3 Robinson–Schensted as a Graph Homomorphism

Theorem 313 (RSK and crystal graphs). The vertices of $B(1)^{\otimes N}$ are the words of length N over $[n]$. The Robinson–Schensted–Knuth map $w \mapsto (P(w), Q(w))$ is an isomorphism of crystals

$$B(1)^{\otimes N} \cong \bigsqcup_\lambda B(\lambda) \times \text{SYT}(\lambda),$$

where λ runs over partitions of N with at most n parts, $\text{SYT}(\lambda)$ is the set of standard Young tableaux of shape λ , the crystal operators act on the insertion tableau $P(w)$ (the Knuth-class representative), and the recording tableau $Q(w)$ is constant on connected components. Thus the connected component containing w consists of all words with recording tableau $Q(w)$, and it is isomorphic to $B(\lambda)$, where λ is the common shape of $P(w)$ and $Q(w)$. Permutations $\pi \in S_n$ are the words with $N = n$ distinct letters.

12.16.4 Application: Littlewood–Richardson Rule

Theorem 314 (Littlewood–Richardson via crystals). The structure constants $c_{\mu\nu}^\lambda$ in $s_\mu \cdot s_\nu = \sum_\lambda c_{\mu\nu}^\lambda s_\lambda$ equal the number of **Yamanouchi tableaux** of shape λ/μ and content ν , equivalently, the number of crystal isomorphisms $B(\lambda) \hookrightarrow B(\mu) \otimes B(\nu)$.

12.16.5 Connection to Graph Theory

The crystal graph of $B(\lambda)$ is an edge-labelled directed graph on $s_\lambda(1, \dots, 1) = |\text{SSYT}(\lambda, n)|$ vertices. As a graph:

- Connectivity = irreducibility of the underlying representation.
- As an edge-labelled directed graph, $B(\lambda)$ has only the trivial automorphism (an automorphism must fix the unique highest-weight vertex and commute with every $\tilde{f}_i$). The Weyl group S_n of GL_n acts on the vertex set (the simple reflection s_i reverses every i -string), but not by graph automorphisms.
- Counting the highest-weight vertices (those with no incoming edge) of weight ν in $B(\lambda) \otimes B(\mu)$ recovers the Littlewood–Richardson coefficient $c_{\lambda\mu}^\nu$.
- Specialisation to the standard alphabet recovers the *RSK correspondence* bijection between permutations and pairs of standard tableaux—a foundational result that connects the symmetric group S_n to combinatorial graph invariants like the longest increasing subsequence.

12.16.6 Exercises

Easy Questions.

Exercise 12.121. Compute the crystal graph $B(1, 1)$ over the alphabet $\{1, 2, 3\}$: the SSYT of shape $(1, 1)$ are columns $\begin{pmatrix} i \\ j \end{pmatrix}$ with $i < j$, giving $\begin{pmatrix} 3 \\ 2 \end{pmatrix} = 3$ vertices. Draw the crystal with $\tilde{f}_1, \tilde{f}_2$ edges.

Exercise 12.122. Apply Schensted insertion to the word $w = 31415$ and confirm that the reading word of the resulting tableau is the unique tableau word in the Knuth equivalence class of w (the class itself has several elements).

Medium Questions.

Exercise 12.123. Verify Theorem 311 for $\lambda = (2, 1)$ over $\{1, 2\}$: the crystal has 2 tableaux (first row 1 1 or 1 2, second row 2), and its character $\text{ch}(B(2, 1)) = x_1^2 x_2 + x_1 x_2^2$ (degree-3 Schur with two parts).

Exercise 12.124. Implement the Knuth-equivalence relations on $\{1, 2, 3\}^4$ and count the equivalence classes. Verify the count against the number $\sum_\lambda |\text{SSYT}(\lambda, 3)|$ of semistandard tableaux with 4 boxes and entries in $\{1, 2, 3\}$ (sum over partitions λ of 4), and check that $\sum_\lambda |\text{SSYT}(\lambda, 3)| f^\lambda = 3^4$.

Hard Questions.

Exercise 12.125. Prove the Littlewood–Richardson rule using crystal techniques: the connected components of $B(\mu) \otimes B(\nu)$ correspond to Yamanouchi tableaux of shape λ/μ and content ν .

Exercise 12.126. Show that the longest increasing subsequence in a permutation π equals the length of the first row of $P(\pi)$ (Schensted’s theorem), using the crystal-graph interpretation of the RSK correspondence.

Diagram Questions.

Exercise 12.127. Draw the crystal graph $B(1, 1, 1)$ over $\{1, 2, 3\}$: the unique SSYT of shape $(1, 1, 1)$ is the column 1, 2, 3, so the crystal has a single vertex. Verify $\text{ch}(B) = x_1 x_2 x_3$.

Exercise 12.128. Sketch the crystal $B(2)$ over $\{1, 2, 3\}$ (single-row SSYT of length 2). It has $\begin{pmatrix} 4 \\ 2 \end{pmatrix} = 6$ vertices; draw the $\tilde{f}_1, \tilde{f}_2$ edges and verify the character $h_2(x_1, x_2, x_3) = \sum_{i \leq j} x_i x_j$.

12.17 Further Reading and Pointers

Matchings polynomial and rootedness. Godsil’s *Algebraic Combinatorics* (1993), Chapters 1, 4, 6, develops the path-tree machinery in full and extends to the broader theory of *tree-like* walks and characteristic polynomials of covers.

Equitable partitions and walk-regularity. Godsil–Royle, *Algebraic Graph Theory* (2001), Chapters 9–10, are the canonical reference. Brouwer–Haemers, *Spectra of Graphs* (2012), Chapter 3, contains modern interlacing and Hoffman/Haemers bounds.

Distance-regular graphs and association schemes. Brouwer–Cohen–Neumaier, *Distance-Regular Graphs* (1989), is the comprehensive reference; Bannai–Ito, *Algebraic Combinatorics I* (1984), develops the Bose–Mesner algebra. Delsarte’s 1973 thesis on the LP method remains foundational.

Cayley graphs and expanders. Hoory–Linial–Wigderson, “Expander Graphs and Their Applications” (*Bull. AMS*, 2006), surveys both deterministic and probabilistic constructions. Lubotzky, *Discrete Groups, Expanding Graphs, and Invariant Measures* (1994), connects to representation theory and number theory.

Tutte polynomial and matroids. Bollobás, *Modern Graph Theory* (1998), Chapter 10, gives a graph-theoretic treatment. Oxley, *Matroid Theory* (2nd ed., 2011), is the matroid-side reference. Welsh’s *Complexity: Knots, Colourings and Counting* (1993) discusses the Jones polynomial connection.

Chromatic symmetric function. Stanley’s original paper, “A symmetric function generalization of the chromatic polynomial of a graph” (*Adv. Math.*, 1995), is short and self-contained. Recent partial progress on the tree conjecture is summarised in Aliste-Prieto–Martin–Wagner–Zamora, “Chromatic symmetric functions and polynomial invariants of trees” (*Bull. Lond. Math. Soc.*, 2024), and references therein.

Heaps of pieces and walk generating functions. Viennot’s *Heaps of Pieces, I: Basic Definitions and Combinatorial Lemmas* (1986) introduced the theory and is the canonical reference. Garsia–Egecioglu, *Lectures in Algebraic Combinatorics* (2020), Chapter 4, gives a modern self-contained exposition with applications to the matchings polynomial and orthogonal polynomials. Flajolet’s “Combinatorial aspects of continued fractions” (*Discrete Math.*, 1980) is the source of the continued-fraction theorem (Theorem 275). For the Lindström–Gessel–Viennot lemma in this language, see Stanley, *Enumerative Combinatorics, Vol. I* (2nd ed., 2012), §2.7.

Ihara zeta function. Stark–Terras, *Zeta functions of finite graphs and coverings* (parts I–III, *Adv. Math.* 1996/2000/2007) develop the theory thoroughly. Terras’s *Zeta Functions of Graphs: A Stroll through the Garden* (2010) is the canonical book-length reference. For the non-backtracking matrix in random graph theory and community detection, see Krzakala–Moore–Mossel–Neeman–Sly–Zdeborová–Zhang, “Spectral redemption in clustering sparse networks” (*PNAS*, 2013). Bass’s three-term identity is in his 1992 paper “The Ihara–Selberg zeta function of a tree lattice”.

Combinatorial Hopf algebras. Schmitt’s foundational paper “Incidence Hopf algebras” (*J. Pure Appl. Algebra*, 1994) introduces the graph and matroid Hopf algebras. Aguiar–Bergeron–Sottile, “Combinatorial Hopf algebras and generalized Dehn–Sommerville relations” (*Compos. Math.*, 2006), is the universal-property theorem; Aguiar–Mahajan, *Monoidal Functors, Species, and Hopf Algebras* (2010), is a comprehensive modern treatment. For the Stanley–Stembridge conjecture and Hikita’s recent proof, see Hikita’s 2024 arXiv preprint and the survey by Shreshth–Wachs. Connes–Kreimer, “Hopf algebras, renormalization and non-commutative geometry” (*Comm. Math. Phys.*, 1998), connects Hopf algebras to QFT.

Hodge theory of matroids. Adiprasito–Huh–Katz, “Hodge theory for combinatorial geometries” (*Annals of Math.*, 2018), is the main paper; the Bourbaki seminar of Chambert-Loir (Exp. 1144, *Astérisque* 414, 2019) and the survey of Huh, “Combinatorial applications of the Hodge–Riemann relations” (ICM, 2018), are excellent introductions. For spectral independence, see Anari–Liu–Oveis Gharan, “Spectral independence in high-dimensional expanders and applications to the hardcore model” (*FOCS*, 2020); for modern matroid sampling, see Anari–Liu–Oveis Gharan–Vinzant, “Log-concave polynomials II: high-dimensional walks and an FPRAS for counting bases of a matroid” (*STOC*, 2019). The Feichtner–Yuzvinsky paper “Chow rings of toric varieties defined by atomic lattices” (*Invent. Math.*, 2004) introduces the Chow ring construction.

Permanent and #P. Valiant’s original paper “The complexity of computing the permanent” (*Theoret. Comput. Sci.*, 1979) defines #P and proves Theorem 290. Minc’s *Permanents* (1978) is the classical compendium of permanent identities including Ryser’s formula and the Bregman–Minc inequality. For Kasteleyn’s planar Pfaffian algorithm, see Kasteleyn, “Graph theory and crystal physics” (in F. Harary, ed., *Graph Theory and Theoretical Physics*, 1967). Jerrum–Sinclair–Vigoda, “A polynomial-time approximation algorithm for the permanent of a matrix with non-negative entries” (*J. ACM*, 2004), is the FPRAS paper. Barvinok’s *Combinatorics and Complexity of Partition Functions* (2017) covers polynomial interpolation techniques.

Weisfeiler–Leman and graph isomorphism. Cai–Fürer–Immerman, “An optimal lower bound on the number of variables for graph identification” (*Combinatorica*, 1992), gives the foundational hierarchy lower bound. Babai’s quasipolynomial paper, “Graph isomorphism in quasipolynomial time” (arXiv:1512.03547, 2015–16), remains the state of the art. For the GNN–WL connection, see Xu–Hu–Leskovec–Jegelka, “How powerful are graph neural networks?” (*ICLR*, 2019), and Morris et al., “Weisfeiler and Leman go neural: higher-order graph neural networks” (*AAAI*, 2019). Grohe’s *Descriptive Complexity, Canonisation, and Definable Graph Structure Theory* (2017) is the canonical reference.

Cluster algebras. Fomin–Zelevinsky, “Cluster algebras I: Foundations” (*J. Amer. Math. Soc.*, 2002), launched the field; the finite-type classification is Part II (*Invent. Math.*, 2003), and the four-paper series concludes with “Cluster algebras IV: Coefficients” (*Compos. Math.*, 2007). Marsh’s *Lecture Notes on Cluster Algebras* (2014) is a friendly introduction. Fomin–Williams–Zelevinsky’s ongoing book *Introduction to Cluster Algebras* (chapters posted on arXiv since 2016) is the developing canonical reference. Triangulations / surfaces: Fomin–Shapiro–Thurston, “Cluster algebras and triangulated surfaces I: Cluster complexes” (*Acta Math.*, 2008). For positivity, Lee–Schiffler, “Positivity for cluster algebras” (*Annals of Math.*, 2015), and Gross–Hacking–Keel–Kontsevich, “Canonical bases for cluster algebras” (*J. Amer. Math. Soc.*, 2018).

Chip-firing and Riemann–Roch for graphs. Baker–Norine, “Riemann–Roch and Abel–Jacobi theory on a finite graph” (*Adv. Math.*, 2007), is the foundational paper. Klivans’s *The Mathematics of Chip-Firing* (2018) is the book-length treatment. Corry–Perkinson, *Divisors and Sandpiles: An Introduction to Chip-Firing* (2018), gives a graduate-level exposition with many exercises. For tropical extensions, Baker–Jensen, “Degeneration of linear series from the tropical point of view and applications” (in *Nonarchimedean and Tropical Geometry*, Simons Symposia, Springer, 2016).

Random walks via representation theory. Diaconis, *Group Representations in Probability and Statistics* (IMS Lecture Notes, 1988) is the canonical reference. The random-transposition paper is Diaconis–Shahshahani, “Generating a random permutation with random transpositions” (*Z. Wahrsch. Verw. Gebiete*, 1981). Bayer–Diaconis, “Trailing the dovetail shuffle to its lair” (*Ann. Appl. Probab.*, 1992), proves the riffle-shuffle cutoff. Lubetzky–Sly, “Cutoff phenomena for random walks on random regular graphs” (*Duke Math. J.*, 2010), proves cutoff for random regular graphs. Saloff-Coste, *Lectures on finite Markov chains* (*École d’Été de Probabilités de Saint-Flour XXVI*, 1996) is also highly recommended.

Electrical networks. Doyle–Snell, *Random Walks and Electric Networks* (1984, freely available online) is a beautifully written elementary introduction. Lyons–Peres, *Probability on Trees and Networks* (Cambridge, 2016), is the modern graduate reference. Spielman–Teng, “Spectral sparsification of graphs” (*SIAM J. Comput.*, 2011), introduces the spectral-sparsifier framework that motivates nearly-linear-time Laplacian solvers (Cohen et al., STOC 2014; Kyng–Sachdeva, FOCS 2016).

Crystal graphs and the plactic monoid. Kashiwara’s original paper, “Crystalizing the q -analogue of universal enveloping algebras” (*Comm. Math. Phys.*, 1990), established the theory. Bump–Schilling, *Crystal Bases: Representations and Combinatorics* (World Sci., 2017), is the canonical book. Hong–Kang, *Introduction to Quantum Groups and Crystal Bases* (AMS, 2002) covers the representation-theoretic background. For the plactic monoid: Lascoux–Schützenberger, “Le monoïde plaxique” (*Quad. Ricerca Sci.*, 1981); Fulton, *Young Tableaux* (LMS Student Texts, 1997).

Graph isomorphism and complexity (broader). Köbler–Schöning–Torán, *The Graph Isomorphism Problem: Its Structural Complexity* (Birkhäuser, 1993), is a comprehensive treatment. Babai’s “Group, graphs, algorithms: the graph isomorphism problem” (ICM, 2018) summarises the post-2015 algorithmic landscape.

General references. Stanley’s *Enumerative Combinatorics* (Vols. I and II, 2nd eds.) covers much of the underlying combinatorial machinery. Lovász’s *Combinatorial Problems and Exercises* (2nd ed., 1993) is a classical problem book. The arXiv `math.CO` archive contains recent research; for graduate-level surveys see the *Séminaire Lotharingien de Combinatoire* archive.